Osmocom Operator

Docs

Bastien Baranoff

2026-10-28

Table des matières

1 Synthèse **5**

2 Resultats des tests **5**

2.1 /opt/GSM/osmo-operator/README.md . . . 5
2.1.1 osmo-operator . . . 6
2.2 /opt/GSM/osmo-operator/.github/release-notes.md . . . 10
2.3 /opt/GSM/osmo-operator/configs/srsran/user_db.csv . . . 12
2.4 /opt/GSM/osmo-operator/data/adresses.txt . . . 12
2.5 /opt/GSM/osmo-operator/data/air-mesh.txt . . . 12
2.6 /opt/GSM/osmo-operator/data/c139.txt . . . 13
2.7 /opt/GSM/osmo-operator/data/noms.txt . . . 13
2.8 /opt/GSM/osmo-operator/data/op.txt . . . 14
2.9 /opt/GSM/osmo-operator/data/pays.txt . . . 15
2.10 /opt/GSM/osmo-operator/doc/README-architecture.md . . . 15
2.10.1 osmo-operator — architecture . . . 15
2.11 /opt/GSM/osmo-operator/doc/README-arm.md . . . 17
2.11.1 osmo-operator — arm64 / Raspberry Pi 4 . . . 17
2.12 /opt/GSM/osmo-operator/doc/README-docker.md . . . 19
2.12.1 osmo-operator — Docker . . . 19
2.13 /opt/GSM/osmo-operator/doc/README-docs.md . . . 21
2.13.1 osmo-operator — documentation map . . . 21
2.14 /opt/GSM/osmo-operator/doc/README-dsp.md . . . 25
2.14.1 osmo-operator — the DSP chain (`--dsp`) . . . 25
2.15 /opt/GSM/osmo-operator/doc/README-grgsm.md . . . 27
2.15.1 osmo-operator — the gr-gsm layer 1 . . . 27
2.16 /opt/GSM/osmo-operator/doc/README-iso.md . . . 29
2.16.1 osmo-operator — the live ISO . . . 29
2.17 /opt/GSM/osmo-operator/doc/README-lte.md . . . 31
2.17.1 osmo-operator — 4G, CSFB and the postmarketOS phone . . . 31
2.18 /opt/GSM/osmo-operator/doc/README-native.md . . . 32
2.18.1 osmo-operator — native install (no Docker) . . . 32
2.19 /opt/GSM/osmo-operator/doc/README-ss7.md . . . 34
2.19.1 osmo-operator — multi-operator, SS7 and WAN . . . 34
2.20 /opt/GSM/osmo-operator/doc/README-tests.md . . . 36
2.20.1 osmo-operator — tests and reference results . . . 36
2.21 /opt/GSM/osmo-operator/doc/README-usage.md . . . 39
2.21.1 osmo-operator — using the bench . . . 39
2.22 /opt/GSM/osmo-operator/doc/c54x_exe-depuis-ubuntu-vierge.md . . . 41
2.22.1 c54x_exe depuis une Ubuntu 24.04 vierge . . . 41
2.23 /opt/GSM/osmo-operator/doc/c54x_exe-from-scratch-ubuntu.en.md . . . 44
2.23.1 c54x_exe from a clean Ubuntu 24.04 . . . 44
2.24 /opt/GSM/osmo-operator/doc/install-from-scratch.en-notes.md . . . 47
2.24.1 English notes per Dockerfile stage (read by tools/dockerfile-to-install-md.sh –lang en) . . . 48

2.25 /opt/GSM/osmo-operator/doc/install-from-scratch.en.md . . . 49
2.25.1 Installing from scratch (generated from the Dockerfile) . . . 49
2.25.2 Appendix: c54x_exe-from-scratch-ubuntu.en.md . . . 63
2.26 /opt/GSM/osmo-operator/doc/install-from-scratch.md . . . 66
2.26.1 Installation depuis zero (generee depuis le Dockerfile) . . . 66
2.26.2 Annexe : c54x_exe-depuis-ubuntu-vierge.md . . . 91
2.27 /opt/GSM/osmo-operator/doc/magnetite.md . . . 94
2.27.1 FreeCalypso Magnetite in qosmo (the original TI firmware, Motorola C139) . . . 94
2.28 /opt/GSM/osmo-operator/environment/README.md . . . 97
2.28.1 `environment/` — bench configuration, by domain . . . 97
2.29 /opt/GSM/osmo-operator/navigation/QUICKSTART.md . . . 99
2.29.1 SS7 console — quick start . . . 99
2.30 /opt/GSM/osmo-operator/navigation/STATUS.md . . . 102
2.30.1 Known issues . . . 102
2.31 /opt/GSM/osmo-operator/pont/README.md . . . 103
2.31.1 `pont/` — the TRX bridge between osmo-bts and the emulated Calypso . . . 103
2.32 /opt/GSM/osmo-operator/services/README.md . . . 108
2.32.1 `services/` — systemd units . . . 108
2.33 /opt/GSM/osmo-operator/wiki/Build.md . . . 110
2.33.1 Build — Docker image, ISO, `.deb` packages, native install . . . 110
2.34 /opt/GSM/osmo-operator/wiki/Environnement.md . . . 113
2.34.1 Environment — who wins, and what reaches QEMU . . . 113
2.35 /opt/GSM/osmo-operator/wiki/Home.md . . . 116
2.35.1 osmo-operator — Quick start & reference . . . 117
2.36 /opt/GSM/osmo-operator/wiki/ISO.md . . . 121
2.36.1 The osmo-operator ISO: contents, build, installation, capabilities . . . 121
2.37 /opt/GSM/osmo-operator/wiki/Installer-c54x_exe.md . . . 136
2.37.1 Installing c54x_exe (the Calypso DSP on the TI mask ROM) and the DSP bridge . . . 136
2.38 /opt/GSM/osmo-operator/wiki/Installer-grgsm_exe.md . . . 140
2.38.1 Installing grgsm_exe, and the gr-gsm setup . . . 140
2.39 /opt/GSM/osmo-operator/wiki/Installer-qosmo.md . . . 142
2.39.1 Installing qosmo (QEMU Calypso), and the osmocom-bb firmware . . . 142
2.40 /opt/GSM/osmo-operator/wiki/Lancer-a-la-main.md . . . 146
2.40.1 Launching the emulated phone by hand, process by process . . . 146
2.41 /opt/GSM/osmo-operator/wiki/Payload-ISM.md . . . 151
2.41.1 SMS payload over ISM radio, the rest over IP (`--payload-ism`) . . . 151
2.42 /opt/GSM/osmo-operator/wiki/README-notebook.md . . . 153
2.42.1 The install notebook . . . 153
2.43 /opt/GSM/osmo-operator/wiki/Reference.md . . . 156
2.43.1 osmo-operator — reference . . . 156
2.44 /opt/GSM/osmo-operator/wiki/Resultats.md . . . 170
2.44.1 Bench checks — `checks/check_all.sh` . . . 170
2.45 /opt/GSM/osmo-operator/wiki/SIM.md . . . 172
2.45.1 SIM: a real card in a PC/SC reader, or softSIM . . . 172
2.46 /opt/GSM/osmo-operator/wiki/Start-native.md . . . 174
2.46.1 `start-native.sh` — step by step . . . 175
2.47 /opt/GSM/osmo-operator/wiki/Telephone-emule.md . . . 179
2.47.1 Installing the emulated phone on your own Osmocom stack . . . 179
2.48 /opt/GSM/osmo-operator/wiki/WAN.md . . . 182
2.48.1 WAN — linking two nodes . . . 182

3 Fichiers **185**
3.1 /opt/GSM/osmo-operator/.gitignore . . . 185
3.2 /opt/GSM/osmo-operator/compose.yaml . . . 186
3.3 /opt/GSM/osmo-operator/globals.conf . . . 189
3.4 /opt/GSM/osmo-operator/globals.conf.example . . . 196
3.5 /opt/GSM/osmo-operator/.deb-cache/.keep . . . 201
3.6 /opt/GSM/osmo-operator/.github/actions/docker-disk/action.yml . . . 202
3.7 /opt/GSM/osmo-operator/.github/workflows/build-arm.yml . . . 203
3.8 /opt/GSM/osmo-operator/.github/workflows/build-iso.yml . . . 208
3.9 /opt/GSM/osmo-operator/.github/workflows/docker-arm.yml . . . 215

3.10 /opt/GSM/osmo-operator/.github/workflows/docker.yml . . . 218
3.11 /opt/GSM/osmo-operator/configs/asound.conf . . . 224
3.12 /opt/GSM/osmo-operator/configs/extensions.conf . . . 225
3.13 /opt/GSM/osmo-operator/configs/linphonerc . . . 231
3.14 /opt/GSM/osmo-operator/configs/manager-osmo.conf . . . 233
3.15 /opt/GSM/osmo-operator/configs/mobile.cfg . . . 234
3.16 /opt/GSM/osmo-operator/configs/modules.conf . . . 236
3.17 /opt/GSM/osmo-operator/configs/osmo-bsc.cfg . . . 238
3.18 /opt/GSM/osmo-operator/configs/osmo-bts-trx-bts1.cfg . . . 244
3.19 /opt/GSM/osmo-operator/configs/osmo-bts-trx.cfg . . . 245
3.20 /opt/GSM/osmo-operator/configs/osmo-bts-virtual.cfg . . . 246
3.21 /opt/GSM/osmo-operator/configs/osmo-bts.cfg . . . 248
3.22 /opt/GSM/osmo-operator/configs/osmo-ggsn.cfg . . . 249
3.23 /opt/GSM/osmo-operator/configs/osmo-hlr.cfg . . . 251
3.24 /opt/GSM/osmo-operator/configs/osmo-mgw.cfg . . . 253
3.25 /opt/GSM/osmo-operator/configs/osmo-msc.cfg . . . 254
3.26 /opt/GSM/osmo-operator/configs/osmo-pcu.cfg . . . 258
3.27 /opt/GSM/osmo-operator/configs/osmo-sgsn.cfg . . . 258
3.28 /opt/GSM/osmo-operator/configs/osmo-sip-connector.cfg . . . 259
3.29 /opt/GSM/osmo-operator/configs/osmo-stp.cfg . . . 259
3.30 /opt/GSM/osmo-operator/configs/osmo-trx.cfg . . . 261
3.31 /opt/GSM/osmo-operator/configs/osmo-upf.cfg . . . 261
3.32 /opt/GSM/osmo-operator/configs/pjsip.conf . . . 262
3.33 /opt/GSM/osmo-operator/configs/rtp.conf . . . 266
3.34 /opt/GSM/osmo-operator/configs/sms-routing.conf . . . 267
3.35 /opt/GSM/osmo-operator/configs/conky/osmo-conky.conf . . . 267
3.36 /opt/GSM/osmo-operator/configs/conky/osmo-conky.lua . . . 269
3.37 /opt/GSM/osmo-operator/configs/crypthome/crypthome-session.desktop . . . 270
3.38 /opt/GSM/osmo-operator/configs/crypthome/crypthome.default . . . 270
3.39 /opt/GSM/osmo-operator/configs/crypthome/unlock-home.service . . . 272
3.40 /opt/GSM/osmo-operator/configs/gprs/modem-gprs.cfg . . . 272
3.41 /opt/GSM/osmo-operator/configs/gprs/osmo-bsc-bts2-egprs.frag . . . 273
3.42 /opt/GSM/osmo-operator/configs/gprs/osmo-bts-virtual-gprs.cfg . . . 274
3.43 /opt/GSM/osmo-operator/configs/gprs/osmo-pcu-gprs.cfg . . . 275
3.44 /opt/GSM/osmo-operator/configs/open5gs/hss.yaml . . . 276
3.45 /opt/GSM/osmo-operator/configs/open5gs/mme.yaml . . . 276
3.46 /opt/GSM/osmo-operator/configs/open5gs/nrf.yaml . . . 278
3.47 /opt/GSM/osmo-operator/configs/open5gs/pcrf.yaml . . . 279
3.48 /opt/GSM/osmo-operator/configs/open5gs/sgwc.yaml . . . 282
3.49 /opt/GSM/osmo-operator/configs/open5gs/sgwu.yaml . . . 283
3.50 /opt/GSM/osmo-operator/configs/open5gs/smf.yaml . . . 284
3.51 /opt/GSM/osmo-operator/configs/open5gs/subscribers.json . . . 290
3.52 /opt/GSM/osmo-operator/configs/open5gs/upf.yaml . . . 292
3.53 /opt/GSM/osmo-operator/configs/open5gs/dump/open5gs/accounts.metadata.json . . . 293
3.54 /opt/GSM/osmo-operator/configs/open5gs/dump/open5gs/prelude.json . . . 293
3.55 /opt/GSM/osmo-operator/configs/open5gs/dump/open5gs/sessions.metadata.json . . . 293
3.56 /opt/GSM/osmo-operator/configs/open5gs/dump/open5gs/subscribers.metadata.json . . . 294
3.57 /opt/GSM/osmo-operator/configs/open5gs/freeDiameter/hss.conf . . . 294
3.58 /opt/GSM/osmo-operator/configs/open5gs/freeDiameter/mme.conf . . . 294
3.59 /opt/GSM/osmo-operator/configs/open5gs/freeDiameter/pcrf.conf . . . 295
3.60 /opt/GSM/osmo-operator/configs/open5gs/freeDiameter/smf.conf . . . 295
3.61 /opt/GSM/osmo-operator/configs/plymouth/osmo-bts/osmo-bts.plymouth . . . 296
3.62 /opt/GSM/osmo-operator/configs/plymouth/osmo-bts/osmo-bts.script . . . 296
3.63 /opt/GSM/osmo-operator/configs/plymouth/vortex-ubuntu/SOURCE . . . 299
3.64 /opt/GSM/osmo-operator/configs/plymouth/vortex-ubuntu/logo.svg . . . 299
3.65 /opt/GSM/osmo-operator/configs/plymouth/vortex-ubuntu/vortex-ubuntu.grub . . . 300
3.66 /opt/GSM/osmo-operator/configs/plymouth/vortex-ubuntu/vortex-ubuntu.plymouth . . . 300
3.67 /opt/GSM/osmo-operator/configs/plymouth/vortex-ubuntu/vortex-ubuntu.script . . . 301
3.68 /opt/GSM/osmo-operator/configs/pmos/pmbootstrap_v3.cfg . . . 304
3.69 /opt/GSM/osmo-operator/configs/srsran/enb.conf . . . 304
3.70 /opt/GSM/osmo-operator/configs/srsran/rb.conf . . . 312

3.71 /opt/GSM/osmo-operator/configs/srsran/rr.conf . . . 315
3.72 /opt/GSM/osmo-operator/configs/srsran/sib.conf . . . 317
3.73 /opt/GSM/osmo-operator/configs/srsran/ue.conf . . . 321
3.74 /opt/GSM/osmo-operator/data/c123-zones.json . . . 329
3.75 /opt/GSM/osmo-operator/data/c139-zones.json . . . 331
3.76 /opt/GSM/osmo-operator/data/claude.svg . . . 331
3.77 /opt/GSM/osmo-operator/data/deka.svg . . . 332
3.78 /opt/GSM/osmo-operator/data/osmo-dino.svg . . . 332
3.79 /opt/GSM/osmo-operator/data/osmo-info.svg . . . 333
3.80 /opt/GSM/osmo-operator/data/osmo-launch.svg . . . 333
3.81 /opt/GSM/osmo-operator/data/osmo-lte.svg . . . 334
3.82 /opt/GSM/osmo-operator/data/osmo-multi.svg . . . 334
3.83 /opt/GSM/osmo-operator/data/osmo-paint.svg . . . 335
3.84 /opt/GSM/osmo-operator/data/osmo-tutorial.svg . . . 335
3.85 /opt/GSM/osmo-operator/data/desktop/claude.desktop . . . 336
3.86 /opt/GSM/osmo-operator/data/desktop/osmo-addition.desktop . . . 336
3.87 /opt/GSM/osmo-operator/data/desktop/osmo-dino.desktop . . . 337
3.88 /opt/GSM/osmo-operator/data/desktop/osmo-drivers.desktop . . . 337
3.89 /opt/GSM/osmo-operator/data/desktop/osmo-info.desktop . . . 337
3.90 /opt/GSM/osmo-operator/data/desktop/osmo-install.desktop . . . 338
3.91 /opt/GSM/osmo-operator/data/desktop/osmo-launch.desktop . . . 338
3.92 /opt/GSM/osmo-operator/data/desktop/osmo-lte.desktop . . . 339
3.93 /opt/GSM/osmo-operator/data/desktop/osmo-multi.desktop . . . 340
3.94 /opt/GSM/osmo-operator/data/desktop/osmo-paint.desktop . . . 341
3.95 /opt/GSM/osmo-operator/data/desktop/osmo-tutorial.desktop . . . 341
3.96 /opt/GSM/osmo-operator/data/desktop/osmo-update.desktop . . . 341
3.97 /opt/GSM/osmo-operator/environment/load.env . . . 342
3.98 /opt/GSM/osmo-operator/environment/modes.env . . . 342
3.99 /opt/GSM/osmo-operator/environment/paths.env . . . 343
3.100/opt/GSM/osmo-operator/installer/calamares/settings.conf . . . 345
3.101/opt/GSM/osmo-operator/installer/calamares/branding/osmo/branding.desc . . . 347
3.102/opt/GSM/osmo-operator/installer/calamares/branding/osmo/show.qml . . . 347
3.103/opt/GSM/osmo-operator/installer/calamares/jobs/osmosupplements/module.desc . . . 350
3.104/opt/GSM/osmo-operator/installer/calamares/modules/bootloader.conf . . . 350
3.105/opt/GSM/osmo-operator/installer/calamares/modules/contextualprocess-mirror.conf . . . 350
3.106/opt/GSM/osmo-operator/installer/calamares/modules/contextualprocess-nvidia.conf . . . 351
3.107/opt/GSM/osmo-operator/installer/calamares/modules/displaymanager.conf . . . 351
3.108/opt/GSM/osmo-operator/installer/calamares/modules/finished.conf . . . 352
3.109/opt/GSM/osmo-operator/installer/calamares/modules/fstab.conf . . . 352
3.110/opt/GSM/osmo-operator/installer/calamares/modules/grubcfg.conf . . . 352
3.111/opt/GSM/osmo-operator/installer/calamares/modules/keyboard.conf . . . 353
3.112/opt/GSM/osmo-operator/installer/calamares/modules/locale.conf . . . 353
3.113/opt/GSM/osmo-operator/installer/calamares/modules/mount.conf . . . 353
3.114/opt/GSM/osmo-operator/installer/calamares/modules/packagechooser-image.conf . . . 354
3.115/opt/GSM/osmo-operator/installer/calamares/modules/packagechooser-mirror.conf . . . 354
3.116/opt/GSM/osmo-operator/installer/calamares/modules/packagechooser-nvidia.conf . . . 355
3.117/opt/GSM/osmo-operator/installer/calamares/modules/packagechooser-supplements.conf . . . 355
3.118/opt/GSM/osmo-operator/installer/calamares/modules/partition.conf . . . 356
3.119/opt/GSM/osmo-operator/installer/calamares/modules/shellprocess-grub.conf . . . 357
3.120/opt/GSM/osmo-operator/installer/calamares/modules/shellprocess-osmo.conf . . . 357
3.121/opt/GSM/osmo-operator/installer/calamares/modules/unpackfs.conf . . . 359
3.122/opt/GSM/osmo-operator/installer/calamares/modules/users.conf . . . 360
3.123/opt/GSM/osmo-operator/installer/calamares/modules/welcome.conf . . . 361
3.124/opt/GSM/osmo-operator/instances/instance.env.example . . . 361
3.125/opt/GSM/osmo-operator/packaging/.gitignore . . . 364
3.126/opt/GSM/osmo-operator/services/80-osmo-operator.preset . . . 364
3.127/opt/GSM/osmo-operator/services/open5gs-webui.service . . . 365
3.128/opt/GSM/osmo-operator/services/osmo-banc.service . . . 366
3.129/opt/GSM/osmo-operator/services/osmo-bts-trx.service . . . 367
3.130/opt/GSM/osmo-operator/services/osmo-dsp-rom-max.service . . . 368
3.131/opt/GSM/osmo-operator/services/osmo-egprs-dsp.service . . . 368

3.132/opt/GSM/osmo-operator/services/osmo-egprs-web-install.service . . . 369
3.133/opt/GSM/osmo-operator/services/osmo-egprs-web.service . . . 369
3.134/opt/GSM/osmo-operator/services/osmo-lte.service . . . 371
3.135/opt/GSM/osmo-operator/services/osmo-multi.service . . . 371
3.136/opt/GSM/osmo-operator/services/osmo-reglages.service . . . 373
3.137/opt/GSM/osmo-operator/services/osmo-supplements.service . . . 373
3.138/opt/GSM/osmo-operator/tools/vty-connect.exp . . . 373
3.139/opt/GSM/osmo-operator/tools/softsim/comp128.rb . . . 374
3.140/opt/GSM/osmo-operator/tools/softsim/ruby_compat.rb . . . 376
3.141/opt/GSM/osmo-operator/wiki/Untitled.ipynb . . . 376
3.142/opt/GSM/osmo-operator/wiki/installer-osmo-jusquau-dsp.ipynb . . . 377
3.143/opt/GSM/osmo-operator/wiki/lancer-les-modes.ipynb . . . 404
3.144/opt/GSM/osmo-operator/wiki/lancer-pas-a-pas.ipynb . . . 416
3.145/opt/GSM/osmo-operator/wiki/.ipynb_checkpoints/lancer-les-modes-checkpoint.ipynb . . . 440
3.146/root/osmo-operator-20261028-171615-docs_files/mediabag/badge.svg . . . 440
3.147/root/osmo-operator-20261028-171615-docs_files/mediabag/badge.svg-branch=main-event=workflow_dispatch441

1 Synthèse

run	élément	valeur
(pas de verdict.txt)		

dossier de run	contenu
Verdict et couverture	échelle des barreaux et tableau de couverture couche 1
Résultats des tests	captures VTY, diagnostics, rapports pytest, grafcets (.txt .md .mmd)
Fichiers	sources, scripts, configs et docs du banc (tout fichier texte)
Logs	journaux (.log), en dernier

i Méthode

Dossiers : /opt/GSM/osmo-operator, /root/osmo-operator-20261028-171615-docs_files. Extensions : tous les fichiers texte (hors .git, caches, binaires, sauvegardes, LICENSE et sorties précédentes). Entrée 1138 Ko, sortie 1086 Ko.
Compactage sans perte : fichiers identiques cités une fois ; lignes consécutives identiques « ×N » ; lignes ne différant que par des nombres : gabarit avec ⟨k⟩ puis la liste ordonnée des valeurs (k-uplets), chaque ligne se reconstruit en remettant les valeurs dans l'ordre. Seuls les codes couleur, la négociation telnet des captures VTY et les caractères de commande sont retirés ; les lignes de plus de 1000 caractères sont coupées en morceaux terminés par ↩. Les .md sont inclus tels quels (titres rétrogradés) et les .mmd rendus en diagrammes Mermaid, sans compactage.

2 Resultats des tests

2.1 /opt/GSM/osmo-operator/README.md

14376 octets, 177 lignes

2.1.1 osmo-operator

A complete **GSM + LTE operator on one machine**: an Osmocom core network (BTS, BSC, MSC, HLR, MGW, SMSC, Asterisk) and **emulated Calypso phones**. The ARM runs under QEMU (qosmo) and the TI C54x DSP runs outside it, executing the real mask ROM (c54x_exe). The phone is either the OsmocomBB `layer1` + `mobile`, or a Motorola C139 running the **original TI firmware** (FreeCalypso Magnetite). It camps, authenticates, ciphers, sends SMS and holds voice calls. No SDR and no physical phone: the radio is emulated end to end, down to the baseband.



Figure 1: The bench in action: CSFB call, I/Q spectra, srsUE, Wireshark, Linphone

2.1.1.1 Start here

I want to…	Read
Boot a ready-made live image (fastest, nothing to build)	iso · latest release
Pick a mode and run it	Modes below · the **?** of the dock (`osmo-info`) · lancer-les-modes.ipynb
Install on Ubuntu 24.04 without Docker	native · installer-osmo-jusquau-dsp.ipynb
Run in Docker, or build the images	docker
Run on a Raspberry Pi 4 / arm64	arm
Study the real C54x DSP on the mask ROM	dsp
Run the original TI firmware (C139, Magnetite)	magnetite
Use the gr-gsm layer 1	grgsm
Add 4G, CSFB and the postmarketOS phone	lte
Several operators, SS7 interconnect, a WAN between machines	ss7 · WAN
Drive the bench day to day: tmux, VTY, ports, Wireshark	usage
Check that the bench is healthy, reproduce a fault	tests
See how the pieces fit	architecture · all documents

2.1.1.2 Modes

Everything goes through `./start.sh` (`start-native.sh` is a symlink to it). `--regen` rebuilds `/etc/osmocom` from the templates, `--no-attach` stays out of tmux, `./start.sh --docker` hands over to `launch/start-docker.sh`, `./start.sh --help` lists every option.

Command	What runs	State
`./start.sh --bts-classique --ms`	**Default** (red phone icon). `osmo-bts-trx` ↔ `pont/pont_dsp.py`; the phone runs osmocom-bb `layer1` + `mobile` on the DSP ROM	LU, SMS, calls
`./start.sh --bts-classique --ms-magnetite`	Same network, the phone is a C139 running the TI firmware; `--gapk` for voice on the host	LU, SMS; calls in A5/0 (`ENCRYPTION="a5 0" ./start.sh --regen …`)
`./start.sh --bts --ms --ts 2`	The Calypso of QEMU **is the BTS radio** (`trx` firmware + `transceiver`); a second Calypso camps on it over the air; one Calypso per timeslot	camps, LU; no call (see Known issues)
`./start.sh --bts --ms --ts 2/1`	TS0 + TS1 on one Calypso TRX (default of `--bts`)	calls
`./start.sh --grgsm`	gr-gsm layer 1 instead of the DSP	end to end
`banc-rssi`	The osmocom-bb `rssi` app on a 4th Calypso, fed by the bridge; screen `c123-3`	DL levels, sync, BCCH

Command	What runs	State
`./start.sh --stop`	Stop the whole stack	

Phone screens and keypads (`scripts/osmo-banc-aliases.sh`): `c123` = instance 1 (phone of `--bts-classique`, or TRX of `--bts`), `c123-1` = instance 2 (phone over the air), `c123-2` = instance 3 (second TRX), `c123-3` = instance 4 (rssi), `c123-mobile` = the first live instance that is not a TRX, `c123-tous` = all of them. The underlying command is `tools/osmo-c123-gtk.py --instance N`. Nothing telecom starts at boot (`services/80-osmo-operator.preset`): starting a network is your gesture.

Quickest path from the ISO (password `osmo`):

```
cat osmo-operator-desktop.iso.part-* > osmo-operator-desktop.iso
sha256sum -c --ignore-missing SHA256SUMS
qemu-system-x86_64 -cdrom osmo-operator-desktop.iso -m 8G -enable-kvm -cpu host -smp 4
```

From a clone (native, or inside the container):

```
./start.sh --regen --bts-classique --ms   # first run: generate the configs, start, attach to tmux
./start.sh --stop
```

2.1.1.3 Notebooks

Notebook	What it does
installer-osmo-jusquau-dsp.ipynb (nbviewer)	Fresh Ubuntu 24.04 to a call on the DSP, one cell per step with checks; optional Magnetite build
lancer-les-modes.ipynb (nbviewer)	Each mode above: start, check, stop; the phone screens inline, keys, `banc-rssi`
lancer-pas-a-pas.ipynb (nbviewer)	The bench process by process: HLR, MGW, MSC, BSC, BTS, QEMU, osmocon, mobile, bridge

How to run them (disposable container, VM, unattended `nbconvert`): README-notebook.

2.1.1.4 Supplements

What is not in the ISO installs with `sudo ./addition.sh` (a checklist window, or flags):

Flag	Installs
`--multi`	Docker, the operator image, the SS7 multi-operator topology
`--opencl` · `--claude` · `--extras` · `--sim`	GPU OpenCL and A5 tools · Claude Code · games and media · PC/SC SIM tools
`--gnuarm`	the historic OsmocomBB toolchain, arm-elf-gcc 4.5.2 (`/root/gnuarm`, added to `PATH` by `/etc/profile.d/97-osmo-gnuarm.sh`)
`--magnetite`	Wine + TI TMS470, FreeCalypso Magnetite for the C139, the bench patches, the flash image
`--trx`	osmocom-bb-trx: the Docker image and the `trx` firmware (TX) built with gnuarm
`--calypso-fw`	the bench's Calypso firmware (`layer1`, `rssi`) rebuilt with gnuarm (prebuilt in firmware)
`--fernvale`	MediaTek MT6260: fernly, fernvale-nuttx, osmocom-bb-mtk-layer1
`--simcom`	the SIMCom documentation and SDKs
`--firmwares`	all the firmware supplements above
`--all`	Docker, image, multi, OpenCL, Claude, extras, SIM
`--status`	show what is installed, install nothing

2.1.1.5 Repository layout

Path	Purpose
`start.sh`, `start-native.sh` (symlink)	the launcher: loads the environment, picks the mode, generates `mobile_*.cfg`, then runs `/opt/GSM/qosmo/run.sh`; `--docker` hands over to `launch/start-docker.sh`
`startup_scripts/`	the launcher's steps, sourced in order by `start.sh` (options, node identity, environment, DSP bench menu, mobile plan, config regeneration, start)
`launch/`	the other launchers: `start-docker.sh` (containers, N operators), `start-interstp.sh` (the SS7 hub node), `start-multi.sh` (native + Docker operators), `start-nitb.sh`, `start-oqc.sh`, `osmo-launch.sh`
`environment/`	`load.env`, `paths.env`, `modes.env`: variable precedence and path resolution (README)
`globals.conf`, `globals.conf.example`, `generate_configs.sh`	the network settings (ARFCN, PLMN, …) and the only script that substitutes them into the templates
`configs/`	configuration templates for every daemon (`osmo-*.cfg`, Asterisk, open5gs, `gprs/`, `conky/`), wallpapers and screenshots
`lib/`	`gabarits.sh` (template substitution), `audio.sh` (PulseAudio + GAPK chain)
`pont/`	the radio bridges between the emulated Calypso and `osmo-bts-trx`: `pont.py` (gr-gsm), `pont_dsp.py` (DSP), `air.py`/`airmesh.py` (over-the-air between Calypsos), `dsp/` (README)
`opt-gsm/`	gr-gsm side helpers: `si_bridge.py`, `grgsm_fft_live.py`, `record_drain.py`
`scripts/`	run-time services and diagnostics: SMS relay (`sms-interop-relay.py`), audio (`gapk-start.sh`, `audio-diag.sh`), `call-diag.sh`, `banc-repro.sh`, `egprs-dsp.sh`, `magnetite-c139.sh`, the bench aliases, the Docker entrypoint and in-container `run.sh`
`checks/`	read-only health checks of a running stack (`check_all.sh`, `ss7_check.sh`, `diag-*.sh`, `annuaire.sh`)
`network/`	WAN table, mesh, firewall, VirtualBox interconnect, ISM payload, IP plan, loopbacks
`navigation/`, `ss7-console.py`	the SS7 console (MAP/TCAP/SCCP/M3UA, TUI, diagnostics); `ss7-console.py` at the root is a shortcut (QUICKSTART)
`helpers/`, `set_stp_ip.sh`	inter-STP config generator, host preparation, SS7 addressing of a VM
`tools/`	user tools: phone screens (`osmo-c123-gtk.py`), FFT viewers, `osmo-info`, `osmo-launcher.py`, settings page (`osmo-reglages.py`), LTE/pmOS/SIM installers, `osmo-patches-apply.sh`, `osmo-firmwares-install.sh`, documentation generators (`full_qmd.sh`, `full_text.sh`, `dockerfile-to-install-md.sh`)
`patches/`	patches applied on top of the upstream trees under `/opt/GSM` by `tools/osmo-patches-apply.sh` (idempotent; used by `install_modules/40-patches.sh`, `iso_modules/70-scripts.sh`, `update.sh` and `addition.sh --calypso-fw`)
`install.sh`, `install_modules/`	native installation by steps (`prereqs`, `deps`, `user`, `sources`, `patches`, `build`, `binaires`, `configs`, `bureau`, `verify`)
`addition.sh`, `tools/osmo-*-install.sh`	the supplements above
`update.sh`	ISO session start: login animation and patch catch-up on an installed machine

Path	Purpose
`Dockerfile`, `Dockerfile.lite`, `Dockerfile.run`, `Dockerfile.stp`, `build.sh`, `compose.sh`, `compose.yaml`	the Docker images and Compose topology (README-docker)
`packaging/`	the `.deb` cache (`osmo-deb.sh`), standalone packages (`build-debs.sh`), apt mirror helpers
`build-iso.sh`, `iso_modules/`, `installer/`	the live ISO, the SD image and the Calamares installer (README-iso, wiki/ISO)
`armbian/`	the same arm64 image through the official armbian/build framework (README-arm)
`services/`	shipped systemd units and the boot preset (README)
`data/`	desktop entries, icons, phone photos and click zones (`c123*`, `c139*`), node/operator name tables
`instances/`	per-Calypso-instance settings (`instance-N.env`, written by the settings page)
`web/`, `fft-web/`	the settings web page and the two-spectrum FFT web server (port 8081)
`doc/`	topic READMEs, `magnetite.md`, step-by-step install guides
`wiki/`	the wiki pages and the notebooks
`tout-en-un.py`	gathers the text files of a tree into one shortened Markdown report

2.1.1.6 Wiki

Page	Contents
Home	quick start, launching, reference card
Start-native	the launcher step by step, option manual
Reference	architecture, layer-1 chains, build graph, multi-PLMN SS7, diagnostics
Build · ISO	`build.sh`, `build-iso.sh`, `.deb`, `install.sh`; what the ISO contains
Environnement	which variables win, what reaches QEMU
Resultats	reference `check_all.sh` runs and how to read the deviations
Telephone-emule	plug **only** the emulated phone into your own Osmocom stack
Lancer-a-la-main	the phone started by hand, with proof at each step
Installer-qosmo · Installer-c54x_exe · Installer-grgsm_exe	building the QEMU fork and the two external layer 1s
SIM	a sysmoISIM in an Omnikey reader, or a softSIM, over SAP
WAN · Payload-ISM	linking nodes; SMS text over an ISM radio link

2.1.1.7 Other documents

Document	Subject
`pont/README.md`	the radio bridge: A5 ciphering, Kc and SACCH
`doc/magnetite.md`	the TI firmware in qosmo: build, flash image, running, debugging
`environment/README.md`	environment variables and path resolution
`services/README.md`	the systemd units shipped, and the boot policy
`navigation/QUICKSTART.md`	the SS7 console
Components and their READMEs	every Osmocom module and third-party component, with the local patches

2.1.1.8 Repositories and sites

qosmo · c54x_exe · grgsm_exe	the QEMU fork and the two external layer 1s
firmware · osmocom-bb-trx	prebuilt Calypso firmware; osmocom-bb jolly/testing with TX
osmo-egprs-web	the dashboard (http://127.0.0.1:8080)
bbaranoff.github.io · pl4y.store · github.com/bbaranoff	the author's resources and projects
Upstream	Osmocom · FreeCalypso · Open5GS · srsRAN

2.1.1.9 Known issues

- `--bts --ms --ts 2` (one Calypso per timeslot): the second TRX phone's frame number is not aligned, so the TCH is unusable and calls fail (ASSIGNMENT FAILURE). Use `--ts 2/1` for calls.
- Magnetite calls only work with ciphering off (`ENCRYPTION="a5 0"`).
- 4G, CSFB and the postmarketOS phone are amd64 only.

---

osmo-operator is a teaching and research GSM bench: a full core network and emulated Calypso phones on one machine, down to the DSP mask ROM, with multi-operator SS7 interconnect when you want it.

2.2 /opt/GSM/osmo-operator/.github/release-notes.md

6068 octets, 101 lignes

2.2.0.1 osmo-operator-desktop.iso · v0.1-9

Live ISO password: **`osmo`** · postmarketOS phone password: **`147147`**

A complete GSM + LTE operator on one live image, with a GNOME desktop. No SDR and no physical phone: the radio is emulated end to end, down to the baseband. The Calypso's ARM runs under QEMU (qosmo), its TI C54x DSP runs the real mask ROM (c54x_exe).

2.2.0.1.1 Download

Download every `.part-NN` file, not just the first one (GitHub caps release files at 2 GiB). Then:

```
cat osmo-operator-desktop.iso.part-* > osmo-operator-desktop.iso
sha256sum -c --ignore-missing SHA256SUMS
```


Figure 2: osmo-operator: Open Source, No RF

2.2.0.1.2 New in 0.1-9

Phones and radio - Classic BTS is the default: `./start.sh --bts-classique --ms`. `osmo-bts-trx` talks to `pont/pont_dsp.py`; the emulated phone runs osmocom-bb `layer1` + `mobile` on the DSP mask ROM. The red phone icon starts exactly this. - **The original TI firmware**: `--ms-magnetite` boots a Motorola C139 running FreeCalypso Magnetite (TI L2/L3 and UI, colour screen and keypad in `c123`). It camps, registers, sends and receives SMS; calls work in A5/0, voice on the host with `--gapk`. Build: `sudo ./addition.sh --magnetite` (doc/magnetite.md). - **The Calypso as the BTS radio**: `--bts` runs the `trx` firmware in QEMU; `--ms` adds a second Calypso that camps on it over the air; `--ts 1 | 2 | 2/1` chooses the timeslots. - **`banc-rssi`**: the osmocom-bb `rssi` app on a fourth Calypso, fed by the pont in classic BTS mode, with a realistic air: the cell only on its ARFCN, -60 dBm downlink, noise elsewhere, AGC, BCCH decoded. - **Voice**: `--vocoder-rom` (the FR codec of the DSP ROM, default) or `--gapk` (host vocoder, keeps real time). - C123 and C139 screens and keypads in QEMU: `c123 --instance N`.

Robustness - BTS#0 runs under a watchdog: an `osmo-bts-trx` exit on "PC clock skew too high" restarts it after 3 s. - The pont caps the pause it asks of the BTS (`PONT_CLCK_PAUSE_PLAFOND`), the root cause of those exits. - **Nothing telecom starts at boot** (systemd preset `80-osmo-operator.preset`): the bench, the Osmocom core, the BTS, Asterisk, oFono, Open5GS and srsRAN wait for you. Web consoles, sound and network addressing still start.

Desktop and docs - The **?** at the top of the dock opens the help page (modes, modules, notebooks, repositories, references), served by the dashboard so that the snap Firefox can open it. - A new Plymouth boot screen: flat 2D, an antenna and its circles. - Settings page for the whole stack: http://127.0.0.1:8090 (`globals.conf`, Osmocom configs, Calypso instances). - Notebooks: install, run each mode, process by process. - The launcher is now `start.sh` (`start-native.sh` stays as a link).

Supplements (`sudo ./addition.sh`, not in the ISO) - `--gnuarm` (arm-elf-gcc 4.5.2), `--magnetite` (Wine + TI TMS470), `--trx` (osmocom-bb-trx image + TX firmware), `--calypso-fw` (the bench firmware rebuilt with gnuarm), `--fernvale` (MediaTek MT6260: fernly, NuttX, MTK layer 1), `--simcom` (SIMCom docs and SDKs), `--firmwares` (all of them).

Known limits - Magnetite: calls need A5/0 (with A5/1 the FACCH is decoded too early). - `--bts --ts 2`: camps and registers, no call yet (the second TRX is not frame-aligned); `--ts 2/1` calls. - `banc-rssi`: the uplink per-timeslot view does not show the phone yet. - The ROM vocoder costs real time on slow hosts: use `--gapk`. Do not compile while a bench runs.

2.2.0.1.3 What's inside

- **2G (Osmocom)**: osmo-stp, osmo-bsc, osmo-msc, osmo-hlr, osmo-sgsn, osmo-ggsn, osmo-pcu, osmo-mgw; two cells in LAC 1 (cell 6001 BSIC 7, cell 6012 BSIC 8); GPRS/EDGE; voice through osmo-sip-connector and Asterisk; SMS over GSUP and SMPP; deterministic RAND for reproducible runs; live GSMTAP for Wireshark.
- **The DSP**: a TMS320C54x emulated outside QEMU on the dumped mask ROM, sharing the API RAM with the ARM and stepping one TDMA frame at a time.
- **4G (Open5GS + srsRAN)** over ZMQ, PLMN 001/01, null NAS ciphering for analysis; **CSFB** through SGs between the MME and osmo-msc.
- **postmarketOS phone**: data on 4G, voice and SMS on 2G under one identity, oFono on a virtual AT modem.

2.2.0.1.4 Running the image

x86_64, **8 GB of RAM and 4 cores** minimum; in a VM, enable nested virtualization.

```
qemu-system-x86_64 -cdrom osmo-operator-desktop.iso -m 8G -enable-kvm \
  -cpu host -smp 4 -nic user,hostfwd=tcp::8080-:8080
```

VirtualBox or VMware: Linux / Ubuntu 64-bit, 8 GB, 4 CPUs, nested VT-x/AMD-V ticked, the ISO in the optical drive. USB stick: `sudo dd if=osmo-operator-desktop.iso of=/dev/sdX bs=4M status=progress conv=fsync` (Rufus: DD mode).

2.2.0.1.5 Getting started

Dock	Starts
?	this help: every mode, module, notebook and repository
red phone	the 2G bench, `./start.sh --bts-classique --ms`
signal bars	4G: Open5GS, srsENB, srsUE and SGs
phone	the postmarketOS VM (start it last)

Other modes from a terminal: `cd /opt/GSM/osmo-operator && sudo ./start.sh --help`. Health check: `checks/diag-stp-operator.sh`. Documentation: README · wiki · bbaranoff.github.io · pl4y.store.

2.3 /opt/GSM/osmo-operator/configs/srsran/user_db.csv

2000 octets, 21 lignes → 21 lignes

```
#
# .csv to store UE's information in HSS
# Kept in the following format: "Name,Auth,IMSI,Key,OP_Type,OP/OPc,AMF,SQN,QCI,IP_alloc"
#
# Name:     Human readable name to help distinguish UE's. Ignored by the HSS
# Auth:     Authentication algorithm used by the UE. Valid algorithms are XOR
#           (xor) and MILENAGE (mil)
# IMSI:     UE's IMSI value
# Key:      UE's key, where other keys are derived from. Stored in hexadecimal
# OP_Type:  Operator's code type, either OP or OPc
# OP/OPc:   Operator Code/Cyphered Operator Code, stored in hexadecimal
# AMF:      Authentication management field, stored in hexadecimal
# SQN:      UE's Sequence number for freshness of the authentication
# QCI:      QoS Class Identifier for the UE's default bearer.
# IP_alloc: IP allocation stratagy for the SPGW.
#           With 'dynamic' the SPGW will automatically allocate IPs
#           With a valid IPv4 (e.g. '172.16.0.2') the UE will have a statically assigned IP.
#
# Note: Lines starting by '#' are ignored and will be overwritten
ue2,mil,001010001000001,00112233445566778899aabbccdd0101,opc,63bfa50ee6523365ff14c1f45f88737d,8000,576,9,dynamic
]
↪  ue1,xor,001010123456789,00112233445566778899aabbccddeeff,opc,63bfa50ee6523365ff14c1f45f88737d,9001,000000001234,7,dynamic
```

2.4 /opt/GSM/osmo-operator/data/adresses.txt

899 octets, 21 lignes → 21 lignes

```
# adresses.txt - fake subscriber addresses, one "street, town" per line.
# launch/start-docker.sh prefixes a random house number. Blank lines and lines
# starting with # are ignored. None of these places exist.
rue des Ondes Courtes, Fontaine-sur-Onde
impasse du Burst, Val-de-Trame
allee Erlang, Bourg-la-Cellule
boulevard de la Trame, Saint-Handover
quai des Timeslots, Neuville-les-Relais
place du Handover, Clairmont-du-Spectre
chemin des Bursts Perdus, Rocheporte
avenue du Codec, Aubain-sur-Bande
sentier du Roaming, Montcodec
cour des Antennes, Fontaine-sur-Onde
passage du Silence Radio, Val-de-Trame
rue du Ki Perdu, Bourg-la-Cellule
venelle des Slots Vides, Saint-Handover
rond-point du Paging, Neuville-les-Relais
descente de la Bande Basse, Clairmont-du-Spectre
montee du Rejet d'Authentification, Rocheporte
square des Abonnes Fantomes, Aubain-sur-Bande
esplanade du Timeout, Montcodec
```

2.5 /opt/GSM/osmo-operator/data/air-mesh.txt

1581 octets, 33 lignes → 33 lignes

```
# air-mesh.txt - who hears whom on the burst mesh (read by pont/airmesh.py).
#
# One edge per line:
#     <node a> <node b> <attenuation dB> [uni]
```

```
#
# Without the trailing word "uni" the edge applies in BOTH directions. With it,
# it only applies from a to b: useful for a node that transmits strongly but
# receives poorly, or to force a one-way handover.
#
# A PAIR ABSENT FROM THIS FILE CANNOT HEAR EACH OTHER. This is what makes a
# mesh rather than a bus: without the graph every mobile would hear every BTS
# of the bench at full power and there would be no cell, no reselection,
# nothing to demonstrate.
#
# Orders of magnitude: the RSSI announced by the BTS is around -60 dBm and the
# airmesh floor is -110 dBm by default, so:
#     10-25 dB    two neighbouring cells, both readable - reselection
#     30-45 dB    distant cell, readable but worse than the neighbour
#     ~50 dB      at the limit: a mobile hesitates, then drops
#     > 50 dB, or no line at all: out of range
#
# Constraint: this file is NOT reloaded at runtime; airmesh loads it at startup.
# Changing it requires restarting airmesh, not the whole stack.

# -- Current bench: three nodes ---------------------------------------------
# 1 and 2 are two containers on the same machine, hence two adjacent cells.
# 3 is the VM: further away, readable from 2 only.
1 2 20
2 3 35

# 1 and 3 do not hear each other: the line does not exist, deliberately. A
# mobile on node 1 that wants to reach node 3 must go through the cell of
# node 2 - the only way to get a real two-hop path on a three-machine bench.
```

2.6 /opt/GSM/osmo-operator/data/c139.txt

265 octets, 3 lignes → 3 lignes

```
data/c139.jpg: photo "Motorola C139-1170.jpg" by Raimond Spekking (own work), Wikimedia Commons,
licence CC BY-SA 4.0 (https://commons.wikimedia.org/wiki/File:Motorola_C139-1170.jpg), cropped to the
phone and resized to 720x1561 for osmo-c123-gtk.py --modele c139.
```

2.7 /opt/GSM/osmo-operator/data/noms.txt

1098 octets, 54 lignes → 54 lignes

```
# noms.txt - fake subscriber names for the test bench, one "Firstname Lastname"
# per line. launch/start-docker.sh picks one at random per subscriber so that an
# IMSI has a face in the logs. Blank lines and lines starting with # are ignored.
# None of these people exist.
Aline Boulanger
Bastien Carpentier
Chloe Delaunay
Damien Estevez
Elodie Fontaine
Farid Grandjean
Gwenael Hautier
Hind Ibanez
Ismael Jouvenel
Jocelyne Kerbrat
Karim Lachance
Lucie Marceau
Mathis Nourrisson
Nadia Ozanne
Octave Pelletier
Prune Quesnel
```

```
Quentin Rabourdin
Rosalie Sauvage
Samir Thibault
Tiphaine Urbain
Ulysse Vasseur
Valentine Wattiez
Wassim Ximenes
Yasmine Yvenou
Zoe Zelmati
Anatole Bourdaloue
Berenice Crampon
Casimir Ducrocq
Delphine Frochot
Eusebe Gringoire
Fanny Hurluberlu
Gaspard Ippolite
Henriette Jacquemin
Isidore Klapisch
Josephine Lartigue
Kevin Mouchard
Ludivine Nectoux
Marcel Oudinot
Nathalie Pignouf
Odilon Quenneville
Pierrette Ravachol
Raoul Soubeyran
Solange Tourbillon
Thibaut Uzanne
Ursule Vaillancourt
Victorien Wachtel
Wilhelmine Xanthos
Xavier Yvrard
Yolande Zuberbuhler
Zacharie Amblard
```

2.8 /opt/GSM/osmo-operator/data/op.txt

593 octets, 25 lignes → 25 lignes

```
# op.txt - fake operator names, one per line. launch/start-docker.sh picks one at
# random per operator at startup. Blank lines and lines starting with # are ignored.
# Constraint: NO SPACES in a name. The osmo-msc VTY command "short name" takes a
# single word; a name with a space is rejected and osmo-msc starts without a
# network name.
FreeRadis
OrangeMecanique
SFRagile
BouyguesTrous
CelluleGrise
OndesDeChoc
RoamingEmpire
AntenneRelais
SansFilDeFer
BurstEnStock
TrameCassee
ReseauQuiTombe
BarreEtDemie
CoupeCircuit
HandoverPerdu
KiPerdu
PasDeSignal
DebordeTelecom
EchoLointain
GigaFlop
```

2.9 /opt/GSM/osmo-operator/data/pays.txt

702 octets, 24 lignes → 24 lignes

```
# pays.txt - fake country names for the test bench, one per line. An MCC
# designates a country and on this bench a country is a node: node 1 gets the
# first line, node 2 the second, and so on. launch/start-docker.sh shuffles the
# list at each launch and keeps that order for the whole run. Blank lines and
# lines starting with # are ignored. None of these countries exist, on purpose:
# a "Syldavie" in a log means the bench, not a real network.
Syldavie
Bordurie
Poldevie
Molvanie
Ruritanie
Freedonie
Elbonie
Latverie
Zubrowka
Grand-Fenwick
Costa Gravas
Bas-Bourbier
Haute-Ferraille
Republique du Bruit Blanc
Duche de Sans-Signal
Ile de la Trame Perdue
Confins de Bassebande
Principaute de Roamie
```

2.10 /opt/GSM/osmo-operator/doc/README-architecture.md

5147 octets, 141 lignes

2.10.1 osmo-operator — architecture

How the pieces fit: one PLMN on one machine, the emulated handset, and the flows of a location update, an SMS and a call.

← README · Details: wiki/Reference, README-grgsm, README-dsp, README-ss7, README-lte

2.10.1.1 The whole bench

```
flowchart TB
    subgraph HANDSET["emulated handset"]
        MOB["osmocom-bb mobile<br/>layers 2/3"]
        OSC["osmocon"]
        QEMU["qosmo: qemu-system-arm -M calypso<br/>layer1.highram.elf"]
        L1["layer 1<br/>gr-gsm (in QEMU) or C54x mask ROM (c54x_exe)"]
        MOB -->|L1CTL| OSC --> QEMU <--> L1
    end
    PONT["bridge<br/>pont.py / pont_dsp.py"]
    L1 <--> PONT
    subgraph RAN["radio access"]
        BTS0["osmo-bts-trx<br/>BTS 0, Cell 6001"]
        BTS1["BTS 1 (faketrx)<br/>Cell 6012"]
        BSC["osmo-bsc"]
        PCU["osmo-pcu"]
    end
    PONT <-->|"TRXD 5700-5702"| BTS0
    BTS0 -->|Abis/IP| BSC
    BTS1 -->|Abis/IP| BSC
    BTS0 --- PCU
```

```
    subgraph CORE["core network"]
        STP["osmo-stp :2905"]
        MSC["osmo-msc"]
        HLR["osmo-hlr"]
        MGW["osmo-mgw"]
        SGSN["osmo-sgsn"] --> GGSN["osmo-ggsn"]
        SIPC["osmo-sip-connector"] --> AST["Asterisk"]
        SMSC["proto-smsc-daemon"]
    end
    BSC -->|SCCP| STP
    MSC -->|SCCP| STP
    MSC -->|GSUP| HLR
    MSC -->|MGCP| MGW
    MSC -->|MNCC| SIPC
    HLR -->|GSUP| SMSC
    PCU -->|"Gb (NS/BSSGP)"| SGSN
    STP <-.->|"M3UA :2908, multi-operator"| INTER["inter-STP"]
    MSC <-.->|"SGs, CSFB"| MME["open5gs MME (4G)"]
```

Everything in one operator talks over `127.0.0.1`. The local STP listens on `127.0.0.1:2905` straight away, without waiting for the Docker interface.

With `--bts`, the Calypso of QEMU runs the `trx` firmware and the host `transceiver` application instead of `mobile` and the bridge: it is the radio of BTS 0, and a second Calypso camps on it over the air (`pont/air.py`).

2.10.1.2 Location update and authentication

```
sequenceDiagram
    participant MS as mobile
    participant BTS
    participant BSC
    participant MSC
    participant HLR
    MS->>BTS: RACH
    BTS->>MS: IMMEDIATE ASSIGNMENT (SDCCH)
    MS->>MSC: LOCATION UPDATING REQUEST
    MSC->>HLR: GSUP send auth info
    HLR-->>MSC: RAND, SRES, Kc (deterministic RAND on the bench)
    MSC->>MS: AUTHENTICATION REQUEST
    MS-->>MSC: AUTHENTICATION RESPONSE
    MSC->>BSC: CIPHER MODE COMMAND (A5/1, Kc)
    BSC->>MS: CIPHERING MODE COMMAND
    MS-->>BSC: CIPHERING MODE COMPLETE
    MSC->>HLR: GSUP update location
    MSC->>MS: LOCATION UPDATING ACCEPT (TMSI)
```

The A3A8 patches (`patches/*-force-rand-toy.patch`) make RAND deterministic, so a run is reproducible from one boot to the next.

2.10.1.3 SMS

```
sequenceDiagram
    participant A as MS (sender)
    participant MSC as MSC (sms-over-gsup)
    participant HLR
    participant SMSC as proto-smsc-daemon
    participant B as MS (recipient)
    A->>MSC: SMS MO
    MSC->>HLR: GSUP MO-forwardSM
    HLR->>SMSC: GSUP MO-forwardSM
    SMSC->>HLR: GSUP MT-forwardSM
    HLR->>MSC: GSUP MT-forwardSM
    MSC->>B: SMS MT
```

SMPP is on `:2775` with a test ESME. Between operators, `scripts/sms-interop-relay.py` forwards over TCP 7890 (README-ss7).

2.10.1.4 Voice

```
flowchart LR
    MS1["MS"] --> BTS1["BTS"] --> BSC1["BSC"] --> MSC
    MSC -->|MNCC| SIPC["osmo-sip-connector"] -->|"SIP :5060"| AST["Asterisk<br/>FR / HR"]
    AST --> SIPC2["osmo-sip-connector"] -->|MNCC| MSC2["MSC"] --> BSC2["BSC"] --> BTS2["BTS"] --> MS2["MS"]
    MSC <-->|MGCP| MGW["osmo-mgw (RTP)"]
```

`600` is the Asterisk echo test: what the mobile sends comes back to it (README-tests). On the DSP chain the FR codec runs either in the ROM (`--vocoder-rom`, default) or on the host with gapk (`--gapk`).

2.10.1.5 GPRS / EDGE

BSSGP/NS from the PCU to osmo-sgsn, then PDP contexts through osmo-ggsn (GTPv1-C/U). On the DSP chain the GPRS modem is brought up by `scripts/egprs-dsp.sh` (README-dsp).

2.10.1.6 Where the code lives

path	role
`start.sh` / `start-native.sh`, `startup_scripts/`	the launcher (native, container, ISO)
`launch/start-docker.sh`, `compose.sh`	Docker, multi-operator
`launch/start-interstp.sh`, `set_stp_ip.sh`	the SS7 hub and node addressing
`network/`	WAN table, mesh, firewall, VirtualBox, ISM
`lib/gabarits.sh`, `configs/`, `generate_configs.sh`, `globals.conf`	config templates and the network settings
`pont/`	the radio bridges (`pont.py`, `pont_dsp.py`, `air.py`, `airmesh.py`)
`scripts/`	run-time services and diagnostics (SMS relay, audio, call diag, EGPRS, Magnetite)
`checks/`	read-only health checks
`iso_modules/`, `build-iso.sh`	the ISO and SD image
`packaging/`	`.deb` cache and standalone packages
`/opt/GSM/qosmo`	QEMU fork, `run.sh`, `run_modules/`, `tmux_modules/`
`/opt/GSM/c54x_exe`, `/opt/GSM/grgsm_exe`	external layer 1s

The full repository layout is in the README.

2.11 /opt/GSM/osmo-operator/doc/README-arm.md

4720 octets, 96 lignes

2.11.1 osmo-operator — arm64 / Raspberry Pi 4

The same stack, built for arm64 and shipped as an **SD card image** for the Raspberry Pi 4 (Armbian 24.04 base). It is not a live ISO: the root filesystem is a persistent ext4.

← README · Details: wiki/ISO, wiki/Build

2.11.1.1 What you get, and what you don't

	arm64 image
Format	`osmo-operator-<…>-rpi4.img`, MBR: FAT32 `RPICFG` (firmware, kernel, DTBs) + ext4 `armbi_root`
Base	Armbian 24.04, bcm2711 kernel and rpi4b BSP (apt.armbian.com)
2G bench	yes: Osmocom core, Calypso QEMU, gr-gsm and DSP layer 1s, compiled natively for aarch64
`--lite`	yes
Desktop (`--desktop`), installer, `--all`	no: refused by `build-iso.sh` for arm64
4G / CSFB / postmarketOS	no: srsRAN and Open5GS are not built for aarch64 (`iso_modules/88-lte-pmos.sh` exits early)

2.11.1.2 Flash and boot

```
sudo dd if=osmo-operator-<…>-rpi4.img of=/dev/sdX bs=4M status=progress conv=fsync
```

At first boot the root partition grows to fill the card (`osmo-grow-root`). Then use the bench as in README-native: `./start-native.sh`.

2.11.1.3 Build it

On an x86 host everything is emulated (qemu-user-static, `docker buildx --platform linux/arm64`, `debootstrap --foreign`): expect about ten hours for the first build. The `.deb` cache makes later builds short.

```
sudo ./build.sh --arch=arm64          # the arm64 Docker image only (tagged :arm64)
sudo ./build-iso.sh --arm --lite      # the SD image
```

On an arm64 host, `build.sh` and `iso_modules/40-rootfs.sh` see that target and host match, and take the native path: no qemu-user, no buildx `--platform`.

Specific to the arm64 build:

- `iso_modules/82-arm-natif.sh` compiles `toast`, the `qosmo` launcher, `c54x_exe` / `grgsm_exe` and the bench `.deb` **inside the arm64 chroot**, so that no x86 binary ends up in the rootfs. It also installs the Pi firmware partition contents, with a hook that replays on every kernel update.
- `iso_modules/92-rpi-image.sh` assembles the image without mounting anything (`mke2fs -d`, `mcopy`, `sfdisk`, `dd`).

Host packages: `./build.sh --arm` installs everything the arm64 chain needs on the host (`qemu-user-static`, `binfmt-support`, `docker-buildx`, `debootstrap`, `mtools`, `dosfstools`, `e2fsprogs`, `kpartx`, …).

2.11.1.4 CI

- `.github/workflows/docker-arm.yml` builds the arm64 base image **natively** on an `ubuntu-24.04-arm` runner (the emulated x86 path does not fit in GitHub's 6-hour job limit) and publishes it to GHCR as `osmocom-nitb:base-arm64-<fingerprint>`.
- `.github/workflows/build-arm.yml` pulls that image and produces the SD image. It runs on tags `rpi-*` / `arm-*` or by hand.

2.11.1.5 Build it with the official Armbian framework (armbian/build)

`armbian/build-armbian.sh` builds the same image through github.com/armbian/build instead of the hand-made assembly of `build-iso.sh --arm`: Armbian produces the kernel, the board support, the firmware partition and the `.img`; the stack is dropped in through the framework's own hook, `userpatches/customize-image.sh`, with the contents of the project's arm64 Docker image bind-mounted as `/tmp/overlay`.

```
sudo ./build.sh --arch=arm64                        # host packages (qemu-user, binfmt, buildx...) + the arm64
↪  stack image, once
sudo armbian/build-armbian.sh                       # clone armbian/build, overlay, compile.sh
sudo armbian/build-armbian.sh --prepare             # everything but compile.sh (inspect userpatches/)
sudo armbian/build-armbian.sh --board=orangepi5 --branch=edge
```

option	effect
`--board=`, `--branch=`, `--release=`	Armbian target (defaults `rpi4b`, `current`, `noble`)
`--armbian-dir=`	where armbian/build is cloned (default `/opt/GSM/armbian-build`)
`--image=`	stack image to overlay; default `osmocom-nitb:arm64` if present, else `ghcr.io/<repo>/osmocom-nitb:base-arm64-<fingerprint>`
`--desktop`, `--banc`, `--multi`	what the overlay enables
`--prepare`	everything but `compile.sh`
`--no-stack`	a bare Armbian image
`--update`, `--install-deps`, `--dry-run`, `--build-image`	refresh the clone, host dependencies, show the plan, rebuild the stack image first

The image lands in `<armbian-dir>/output/images/`. `customize-image.sh` installs the runtime packages, `dpkg -i` the build `.debs`, copies `/usr/local`, `/opt/GSM`, `/etc/osmocom` and the units, compiles natively what an x86 host cannot (toast, the qosmo launcher, `c54x_exe`/`grgsm_exe`, the bench `.debs`), sets the accounts (root/osmo, osmocom/osmo) and skips Armbian's first-login wizard.

2.12 /opt/GSM/osmo-operator/doc/README-docker.md

4532 octets, 121 lignes

2.12.1 osmo-operator — Docker

Run the bench in containers, or build the images yourself. Docker is also how several operators run side by side on one host (README-ss7).

← README · Details: wiki/Build, wiki/Environnement, wiki/Reference §3

2.12.1.1 Run the published image

```
sudo docker pull ghcr.io/bbaranoff/osmo-operator/osmocom-nitb:latest
git clone https://github.com/bbaranoff/osmo-operator
cd osmo-operator
sudo ./launch/start-docker.sh          # or: sudo ./start.sh --docker
```

Then enter the container and bring the stack up:

```
sudo docker exec -ti osmo-operator-1 bash

cd /opt/GSM/osmo-operator
./start-native.sh --regen     # regenerate configs from the templates (first run)
./start-native.sh --stop      # make sure nothing is left over
./start-native.sh             # start the bench
```

`start-native.sh` attaches to tmux at the end: you are looking at a live GSM network with a mobile camped on it. Re-attach later with:

```
docker exec -ti osmo-operator-1 tmux -S /tmp/osmocom_tmux attach -t osmocom
```

Everything `start-native.sh` does is the same as in native mode: README-native.

2.12.1.2 Build the images

```
sudo ./build.sh               # build osmocom-nitb (apt-fast, compose v2, .deb cache)
sudo ./build.sh --no-cache    # recompile everything and rewrite the cache
sudo ./build.sh --lite        # without the workshops (Dockerfile.lite)
sudo ./build.sh --stp         # osmocom-stp: the inter-STP alone (Dockerfile.stp)
sudo ./build.sh --build-run   # the osmocom-run update layer (Dockerfile.run)
sudo ./build.sh --arch=arm64  # cross-build (also --arm), see README-arm
```

2.12.1.2.1 The build is a graph, not a chain

`Dockerfile` is split into BuildKit stages, so the long compilations that do not depend on each other run **at the same time**:

```
                base   (apt, apt-fast, osmo-deb, configs)
                  |
       +----------+--------+---------+
    osmo-core    qemu      lte       node        <- IN PARALLEL
        |          |
   +----+-----+    |
  bb  grgsm-venv  l1
   +----+-----+---+
             |
        osmocom-nitb   (assembly)
             |
           debs        (FROM scratch: the .deb cache, for CI)
```

`bb`, `l1` and `grgsm-venv` are deliberately **not** parallel to `osmo-core`: they link against libosmocore. `--target osmocom-nitb` is **mandatory** (`debs` is the last stage, so without it Docker builds an empty image). `build.sh` and `compose.yaml` already pass it.

2.12.1.2.2 The `.deb` cache

Everything the image compiles comes out as a `.deb` through `packaging/osmo-deb.sh` and stays **on the host**, in `/var/cache/osmo-debs` (`OSMO_DEB_CACHE` to move it). The second build installs with `dpkg -i` and compiles nothing. Each package carries both the installed files and the `/opt/GSM` source tree, so a cached rebuild still gives you a workshop you can read, patch and `make` in.

```
./compose.sh debs                  # what is in the cache
./packaging/build-debs.sh          # standalone .deb -> packaging/dist/
sudo dpkg -i packaging/dist/*.deb  # refuses to install over a git clone at the same path
```

2.12.1.3 Docker Compose

```
sudo ./compose.sh build [--no-cache] [--lite] [--stp]
sudo ./compose.sh up [--ms 2] [--phy faketrx] [--no-run]   # hub + osmo-operator-1, HLR populated
sudo ./compose.sh down | ps | logs | shell | debs
```

PHY_MODE	stack	use

2.12.1.4 PHY modes (mobile side, in a container)

Selected through PHY_MODE inside the container before calling scripts/run.sh:

PHY_MODE	stack	use
faketrx (default)	fake_trx → trxcon → mobile	multi-MS, fast, no DSP
virtphy	osmo-bts-virtual ↔ virtphy ↔ mobile	multi-MS over UDP multicast
qemu	osmo-bts-trx ↔ bridge ↔ Calypso QEMU ↔ mobile	emulated baseband, 1 MS

PHY_MODE=qemu forces N_MS=1: the bridge uses fixed UDP ports, so one emulated Calypso per container. For several virtual handsets, run several containers.

2.12.1.5 Several operators

sudo ./launch/start-docker.sh asks for a bridge mode and N operators (1 to 9), each in its own container, joined by an inter-STP container. See README-ss7.

2.12.1.6 CI images

.github/workflows/docker.yml publishes the amd64 base to GHCR (osmocom-nitb:base-<fingerprint>), and docker-arm.yml the arm64 one (base-arm64-<fingerprint>). An unchanged base (same fingerprint) is not rebuilt.

2.13 /opt/GSM/osmo-operator/doc/README-docs.md

12134 octets, 173 lignes

2.13.1 osmo-operator — documentation map

Every document in the project, and where to start depending on what you want.

← README

```
flowchart TD
    R["README.md"] --> ISO["README-iso"]
    R --> DOC["README-docker"]
    R --> NAT["README-native"]
    R --> ARM["README-arm"]
    R --> ARCH["README-architecture"]
    ARCH --> GR["README-grgsm"]
    ARCH --> DSP["README-dsp"]
    ARCH --> LTE["README-lte"]
    ARCH --> SS7["README-ss7"]
    R --> USE["README-usage"]
    R --> TST["README-tests"]
    R --> MAG["magnetite"]
    ISO --> W_ISO["wiki/ISO"]
    DOC --> W_BUILD["wiki/Build"]
    NAT --> W_SD["wiki/Start-native"]
    NAT --> W_ENV["wiki/Environnement"]
    GR --> W_GR["wiki/Installer-grgsm_exe<br/>wiki/Installer-qosmo"]
    DSP --> W_DSP["wiki/Installer-c54x_exe<br/>notebook"]
    SS7 --> W_WAN["wiki/WAN<br/>wiki/Payload-ISM"]
    TST --> W_RES["wiki/Resultats"]
    TST --> T_REPO["bbaranoff/tests"]
    GR --> PONT["pont/README"]
```

2.13.1.1 Topic READMEs (this repository)

document	subject
README	entry point, modes, supplements, repository layout, index
README-iso	the live ISO: download, run, launchers, contents
README-docker	Docker images, build graph, `.deb` cache, Compose, PHY modes
README-native	`install.sh`, `start-native.sh`, profiles, `qosmo` launcher
README-arm	arm64 / Raspberry Pi 4 SD image
README-architecture	how the pieces fit, LU / SMS / call flows, code map
README-grgsm	the gr-gsm layer 1 chain, radio diagnostics
README-dsp	the C54x mask-ROM DSP chain, GPRS on the DSP bench
magnetite	the original TI firmware (FreeCalypso Magnetite, C139) in qosmo
README-lte	4G, CSFB, postmarketOS phone
README-ss7	multi-operator, inter-STP, point codes, WAN
README-usage	tmux, VTY, numbers, ports, Wireshark
README-tests	`banc-max.sh` ladder, run reports, pytest, `checks/`

2.13.1.2 Step-by-step install guides (`doc/`)

document	subject
install-from-scratch · English	the Dockerfile replayed step by step on a bare Ubuntu 24.04 (generated by `tools/dockerfile-to-install-md.sh`; English notes in install-from-scratch.en-notes)
c54x_exe-depuis-ubuntu-vierge · English	the DSP emulator alone, from a clean machine to the running TI mask ROM; the arm-elf toolchain

2.13.1.3 Wiki

page	subject
Home	quick start and reference card
Reference	the full reference: architecture, layer-1 chains, build graph, multi-PLMN SS7, diagnostics
Start-native	`start-native.sh` step by step, option manual
Build	`build.sh`, `build-iso.sh`, `.deb`, `install.sh`
ISO	ISO contents, build, install
Environnement	variable precedence, what reaches QEMU
Resultats	reference `check_all.sh` runs
Telephone-emule	the emulated phone on your own Osmocom stack
Lancer-a-la-main · lancer-pas-a-pas.ipynb	the bench process by process (page, and runnable notebook)
lancer-les-modes.ipynb	each launch mode: start, check, stop, phone screens
Installer-qosmo	build the QEMU fork
Installer-grgsm_exe	build the gr-gsm layer 1
Installer-c54x_exe	build the DSP emulator
README-notebook · installer-osmo-jusquau-dsp.ipynb	the install notebook and how to run it
SIM	real SIM in an Omnikey reader, or softSIM, over SAP
WAN	linking two nodes, LAN or Internet
Payload-ISM	SMS text over an ISM radio link

2.13.1.4 Components and their READMEs

Every piece of software the bench is built from, with a link to its own README. "Patches" are the files of `patches/` applied on top of upstream by `tools/osmo-patches-apply.sh`.

```
flowchart LR
    subgraph MINE["bbaranoff modules"]
        QO["qosmO"]
        C54["c54x_exe"]
        GRX["grgsm_exE"]
    end
    subgraph OSMO["Osmocom"]
        BB["osmocom-bb"]
        RAN["osmo-bts / osmo-trx / osmo-bsc / osmo-pcu"]
        CN["osmo-msc / osmo-hlr / osmo-mgw / osmo-stp / osmo-sgsn / osmo-ggsn"]
        LIB["libosmocore, libosmo-*"]
    end
    C54 -->|compiles sources of| QO
    GRX -->|compiles sources of| QO
    QO -->|runs the firmware of| BB
    BB --> RAN --> CN
    LIB -.-> BB
    LIB -.-> RAN
    LIB -.-> CN
```

2.13.1.4.1 bbaranoff modules

component	role	source tree
qosmO	QEMU fork `-M calypso` (ARM7TDMI + platform), `run.sh`, `run_modules/`, the `qosmo` launcher; layer 1 chosen at `configure` (`--enable-l1-grgsm` / `--enable-l1-dsp`)	`/opt/GSM/qosmo`
c54x_exe	TMS320C54x on the TI mask ROM, outside QEMU; `pont_dsp.py`; the `--dsp` chain (README-dsp)	`/opt/GSM/c54x_exe`
grgsm_exE	gr-gsm layer 1 outside QEMU (README-grgsm)	`/opt/GSM/grgsm_exe`
qosmo-overlay	the files qosmO adds to / changes in upstream QEMU	`/opt/GSM/qosmo-overlay`
tests	end-to-end test suite, `banc-max.sh`, run reports (README-tests)	`/opt/GSM/tests`
osmo-egprs-web	the web dashboard on `:8080`	`/opt/GSM/osmo-egprs-web`
firmware	prebuilt osmocom-bb firmware (`layer1.highram.elf`, `rssi.highram.elf`, the Magnetite flash image)	`/opt/GSM/firmware`
osmocom-bb-trx	osmocom-bb jolly/testing with TX: the `trx` firmware of `--bts`	(supplement `--trx`)

2.13.1.4.2 Osmocom

Upstream on gitea.osmocom.org; sources in `/opt/GSM/<name>`.

component	role	patches
libosmocore	core library: VTY, logging, GSM coding, GSUP	

component	role	patches
libosmo-abis	Abis/IP (BTS ↔ BSC)	
libosmo-netif	network interface helpers (SCTP, IPA)	
libosmo-sigtran	SCCP / M3UA, and **osmo-stp** (local STP and inter-STP)	
libosmo-gprs	GPRS protocol stack (RLC/MAC, LLC, SNDCP)	
libosmo-dsp	DSP helpers	
libsmpp34	SMPP 3.4	
libgtpnl	kernel GTP-U through netlink	
osmo-bts	BTS (`osmo-bts-trx`, `osmo-bts-virtual`)	`osmo-bts-force-rand-toy`
osmo-trx	transceiver	`osmo-trx-ipc-ts-frame-align`, `osmo-trx-rach-per-ra-table`
osmo-bsc	BSC	
osmo-msc	MSC / VLR, SMS-over-GSUP, SGs (CSFB)	`osmo-msc-force-rand-toy`
osmo-hlr	HLR, GSUP	`osmo-hlr-force-rand-toy`
osmo-mgw	media gateway (MGCP, RTP)	
osmo-sip-connector	MNCC ↔ SIP (to Asterisk)	
osmo-pcu	PCU (GPRS/EDGE)	
osmo-sgsn	SGSN	
osmo-ggsn	GGSN (GTPv1-C/U); no top-level README.md upstream	
gapk	GSM audio codecs (FR/HR/EFR), audio bridge to the mobile	`gapk-pq-alsa-output-probe`
osmocom-bb	phone side: `mobile`, `modem`, `trxcon`, `osmocon`, `layer1` / `rssi` firmware, `fake_trx` (no README in the repo: project wiki)	`osmocom-bb-*`: `clck-gen-frame-tolerance`, `fbsb-timeout-scan-to`, `firmware-gprs` (PDCH in `layer1`), `firmware-map-ld`, `force-rand-toy`, `osmocon-kc-filter`, `plmn-a6-reinsert`, `sim-plmn-from-bcd`, `transceiver-qemu-bench`
gsup-smsc-proto	`proto-smsc-daemon` / `proto-smsc-sendmt`, the SMSC	
softsim	softSIM served over SAP (wiki/SIM)	

2.13.1.4.3 Third-party

component	role	patches
gr-gsm	GSM receiver for GNU Radio (layer 1 of the `--grgsm` chain)	`grgsm-optional-qtgui-import`, `grgsm-receiver-publish-bsic-fn`
FreeCalypso Magnetite	the original TI firmware for the C139 (magnetite)	`fc-magnetite-banc`, `fc-magnetite-alr-arfcn-forced`, `magnetite-parole-ul-au-go`, `magnetite-sms-msginfo-doublefree`
Asterisk	PBX: voice, echo test 600, SIP trunks between nodes	
open5gs	4G core (README-lte)	
srsRAN 4G	srsENB / srsUE over ZMQ	
sysmo-usim-tool	programming the sysmoISIM	
pmbootstrap	postmarketOS phone image	`pmbootstrap-osmo-bench-qemu`, `pmaports-linux-postmarketos-stable-ppp`

2.13.1.5 Other documents

document	subject
pont/README.md	the radio bridge: A5, Kc and SACCH
environment/README.md	environment variables and path resolution
services/README.md	shipped systemd units
navigation/QUICKSTART.md, navigation/STATUS.md	the SS7 console
bbaranoff/tests	end-to-end test suite and run reports
qosmO	the QEMU fork: one base, two layer 1s
c54x_exe	the C54x DSP on the TI mask ROM
grgsm_exE	the gr-gsm layer 1 outside QEMU

2.14 /opt/GSM/osmo-operator/doc/README-dsp.md

6570 octets, 132 lignes

2.14.1 osmo-operator — the DSP chain (`--dsp`)

With `--dsp` (the default), the handset's own DSP takes over. A TMS320C54x core is emulated outside QEMU (c54x_exe) and runs the mask ROM dumped from a real Calypso (PROM0-3, DROM, PDROM).

← README · Details: wiki/Installer-c54x_exe, wiki/Start-native §8, install notebook, c54x_exe README

```
./start-native.sh                       # real C54x mask ROM (default)
./start-native.sh --dsp                 # the same, explicitly
MODE=dsp PONT=1 /opt/GSM/c54x_exe/run.sh  # the mobile chain alone, no network
BANC_DSP=none ./start-native.sh --dsp     # the stack only, no DSP bench
```

2.14.1.1 How it fits together

`--dsp` does **not** select a different QEMU. The stack comes up exactly as in gr-gsm mode (core, BTS, side-car, tmux), and only the four Calypso modules (`qemu`, `pty`, `osmocon`, `l2`) are handed over to `c54x_exe/run.sh`:

```
flowchart LR
    mobile["mobile<br/>layers 2/3"] -->|L1CTL| osmocon
    osmocon -->|romload, serial pty| ARM
    subgraph QEMU["qemu-system-arm -M calypso"]
        ARM["ARM7<br/>osmocom-bb layer1<br/>(gr-gsm layer 1 cut out)"]
    end
    ARM <-->|"API RAM<br/>/dev/shm/calypso_api_ram"| DSP
    ARM <-->|"lockstep, 1 TDMA frame<br/>/tmp/calypso_dsp.sock"| DSP
    subgraph C54["c54x_exe --arm"]
        DSP["TMS320C54x<br/>TI mask ROM"]
    end
    PONT["pont_dsp.py"] -->|"DL bursts, UDP 6702 (BSP)"| DSP
    DSP -->|"UL via /dev/shm side-bands"| PONT
    PONT <-->|"TRXD 5700-5702"| BTS["osmo-bts-trx"]
    BTS --> CORE["BSC / MSC / HLR"]
```

The five steps of `c54x_exe/run.sh`:

```
1. c54x_exe --arm     the DSP: /tmp/calypso_dsp.sock + shared API RAM, DL bursts on UDP 6702
2. qemu-system-arm    the ARM + osmocom-bb layer1, gr-gsm layer 1 cut out
3. osmocon            romload on the serial pty, L1CTL relay
4. mobile             osmocom-bb layers 2/3
5. pont_dsp.py        TRXD 5700-5702 facing osmo-bts-trx
```

`--dsp` starts **no bridge of its own**: `c54x_exe/run.sh` runs one on 5700-5702 (step 5). A second one on the same ports would die on *Address already in use*. `./start-native.sh --stop` tears down the DSP bench too, without repeating `--dsp`.

Instances: `INSTANCE=N` runs a second complete chain next to the first (`/dev/shm/calypsoN`, `/tmp/osmocom_l2_N`, `/tmp/c54x-pontN`); the bench aliases `banc-mobile` (instance 2, over the air) and `banc-rssi` (instance 4) use it. Per-instance settings live in `instances/instance-N.env`.

Firmware: `/opt/GSM/firmware/board/compal_e88/layer1.highram.elf` by default (`FIRMWARE_ELF=` for another binary, `FIRMWARE_FLASH=` for a flash image such as Magnetite, see magnetite). The highram firmware is loaded through `loader.compalram.bin`; with GPRS it no longer fits the 64 KiB the Compal boot loader accepts, so no compalram variant of `layer1` is built.

2.14.1.2 What the ROM does

- Handled by the ROM: FCCH detection, SCH/BSIC, BCCH SI1/2/4, SACCH SI5/6, RACH, SDCCH both ways, A5 on the modelled coprocessor, TCH/F downlink (Viterbi), FACCH and SACCH uplink, uplink TCH speech, and the ROM's FR codec through the voice serial port (VSP, `--vocoder-rom`, default; `--gapk` moves the vocoder to the host).
- End to end: camping, location update, SMS MO/MT, USSD, MO/MT calls with A5/1, echo test 600.
- Real time: the DSP has to process one 4.62 ms TDMA frame within the frame; the bridge clock follows the DSP (`PONT_HORLOGE_MAX`, see README-grgsm).

The ladder in README-tests checks every step of this list on a running bench.

2.14.1.3 GPRS / EGPRS on the DSP bench

`scripts/egprs-dsp.sh` (and `services/osmo-egprs-dsp.service`) brings up the EGPRS workflow next to the DSP mobile:

```
scripts/egprs-dsp.sh [start|stop|status|check] [--l1 auto|virtphy|dsp] [--no-banc] [--regen-gprs] [--all]
```

step	what it does
preflight	binaries, the 7 DSP ROM files, `c54x_exe`, the osmocom-bb `modem`; checks whether the `layer1` firmware handles `L1CTL_GPRS_*` (PDCH)
configs	`/etc/osmocom`: the virtual EGPRS BTS 2 in `osmo-bsc.cfg` (`configs/gprs/osmo-bsc-bts2-egprs.frag`, re-inserted after each `--regen`), `osmo-bts-virtual-gprs.cfg`, `osmo-pcu-gprs.cfg`, `modem-gprs.cfg`
banc	`osmo-banc.service` (`start-native.sh --dsp`)
abonne	the modem's IMSI in the HLR (idempotent)
cellule	`osmo-bts-virtual` (BTS 2, Um multicast on `lo`) + `osmo-pcu` as transient systemd units
modem	GPRS attach (GMM) + PDP context (SM), tun in the `msgprs` netns; verified at the SGSN and by a ping from the netns

Layer 1 of the modem (`--l1`, or `EGPRS_L1`): `auto` picks `dsp` when the firmware source handles `L1CTL_GPRS_*` and `virtphy` otherwise. With `dsp` the `modem` replaces `mobile` on the DSP bench's L1CTL socket (`/tmp/osmocom_l2`), the GPRS firmware `layer1.highram.gprs.elf` and `UL_RAW=1` are written to `/etc/default/osmo-banc` (restart `osmo-banc` to load them), and the bench must be running in DSP mode (`c54x_exe --arm` present). With `virtphy` the modem runs on a virtual layer 1 in the same PLMN/LAC/RA while the DSP mobile camps on BTS 0. The Calypso ROM modulates GMSK only (CS-1..4): 8-PSK (MCS-5..9) stays on the virtphy modem.

Logs: `journalctl -u osmo-egprs-dsp`, `-u osmo-modem-gprs`, `-u osmo-pcu-gprs`, `-u osmo-bts-virtual-gprs`, `-u osmo-virtphy-gprs`.

2.14.1.4 Building

`c54x_exe` compiles `qosmo`'s sources (it does not copy them) and needs the ROM files in `rom/calypso_dsp.*.bin`. Step by step: wiki/Installer-c54x_exe. The notebook replays the whole Dockerfile natively, from apt to a call on the DSP. The GPRS `layer1` firmware is rebuilt with `sudo ./addition.sh --calypso-fw` (gnuarm toolchain required).

Test it: `cd /opt/GSM/tests && ./banc-max.sh --dsp --restart`.

2.14.1.5 Known issues

- TCH/F downlink: `B_BFI` is raised on really bad frames and on FACCH steals; the audio is handled by the codec's error concealment.
- Frequency hopping is not handled.
- The emulated DSP uses most of the 4.62 ms frame budget on the host; a slow host loses real time (the bridge then slows the clock, see `PONT_HORLOGE_MAX`).
- PDCH on the C54x depends on the GPRS `layer1` firmware being loaded by the bench; `egprs-dsp.sh --l1 virtphy` is the path that does not depend on it.

2.15 /opt/GSM/osmo-operator/doc/README-grgsm.md

4705 octets, 110 lignes

2.15.1 osmo-operator — the gr-gsm layer 1

`--grgsm` runs the handset with gr-gsm as layer 1. This chain works end to end: camping, location update, authentication, A5/1, SMS and voice calls. To study the real DSP instead, see README-dsp.

← README · Details: wiki/Installer-grgsm_exe, wiki/Installer-qosmo, wiki/Lancer-a-la-main, `pont/README.md`

```
./start-native.sh --grgsm      # DSP_MODE=0 in the environment is equivalent
```

2.15.1.1 The two layer-1 chains

Both chains run the same core network, the same BTS and the same `mobile`. They differ in **who demodulates the air interface**.

	`--grgsm`	**`--dsp`** (default)
Layer 1	gr-gsm inside QEMU + `grgsm_exe` bridge	real TI mask-ROM C54x, **outside** QEMU
QEMU's job	ARM7 + gr-gsm layer 1	ARM7 only (`CALYPSO_DSP_EXTERN=1`)
Bridge	`pont/pont.py`	`pont/pont_dsp.py`
Mobile VTY	4247	4347

2.15.1.2 One QEMU tree, two layer 1s

There is a **single QEMU tree**. The layer 1 is a `configure` option, not a separate repository.

```
flowchart TB
    Q["qosmO — QEMU fork, -M calypso<br/>ARM7TDMI + platform, run.sh, run_modules/"]
    Q -->|--enable-l1-grgsm| G["layer 1: gr-gsm"]
    Q -->|--enable-l1-dsp| D["layer 1: c54x (external)"]
    GE["grgsm_exE<br/>gr-gsm layer 1 outside QEMU"] -.compiles qosmo sources.-> Q
    CE["c54x_exe<br/>TMS320C54x on the TI mask ROM"] -.compiles qosmo sources.-> Q
```

tree	what it is
`/opt/GSM/qosmo`	the QEMU fork
`/opt/GSM/grgsm_exe`	the gr-gsm layer 1 as a small standalone binary outside QEMU
`/opt/GSM/c54x_exe`	the DSP, see README-dsp

`qemu-system-arm -M help` names the layer 1 actually linked in:

configure	`-M help` shows
(nothing)	`couche 1 : aucune`
`--enable-l1-grgsm`	`couche 1 : grgsm`
`--enable-l1-dsp`	`couche 1 : c54x`

2.15.1.3 The chain

```
flowchart LR
    mobile["mobile (layer23)"] -->|L1CTL| osmocon
    osmocon -->|romload, serial pty| QEMU
    subgraph QEMU["qosmo (-M calypso)"]
        ARM["ARM7<br/>layer1.highram.elf"]
        L1G["gr-gsm layer 1"]
        ARM <--> L1G
    end
    QEMU <-->|"GSMTAP 4730 / SCH 4731"| PONT["pont/pont.py"]
    PONT <-->|"TRXD 5700-5702"| BTS["osmo-bts-trx"]
    BTS -->|Abis/IP| BSC
    BSC --> CORE["MSC / HLR / MGW / Asterisk"]
```

To start it process by process, with proof at each step: wiki/Lancer-a-la-main. To plug only the emulated phone into your own Osmocom stack: wiki/Telephone-emule.

2.15.1.4 Diagnosing the radio side

Read `pont/README.md` before chasing a radio fault through a CRC counter. The causes documented there (overwritten Kc, the SACCH L1 header, C0 filling, an inert `CALYPSO_CANNED`) *all* show up as "the bridge has CRC errors".

```
./start-native.sh --check-paths     # verify the declared dependencies
./start-native.sh --verbose         # show the modules' output
```

log	contents
`/var/log/osmocom/qemu.log`	ARM/DSP emulation, BSP, IRQ
`/var/log/osmocom/bridge.log`	QEMU ticks, IND CLOCK, DL/UL bursts
`/var/log/osmocom/run.sh.log`	orchestration

symptom	likely cause	check
`bridge: timeout`	QEMU never started the TPU/DSP	`grep TINT0 /var/log/osmocom/qemu.log`
`FBSB result=255` (no cell)	layer 1 does not detect the FB	`grep "IMR change" /var/log/osmocom/qemu.log | wc -l`
`osmo-bts-trx: PC clock skew too high`	the bridge stopped sending `IND CLOCK`	restart the bridge
`/tmp/osmocom_l2` never created	the firmware does not boot	`head -50 /var/log/osmocom/qemu.log`

symptom	likely cause	check
Address already in use on 5700	two bridges: a `--dsp` run left one behind	`./start-native.sh --stop`

QEMU monitor: `socat - unix-connect:/tmp/qemu-calypso-mon.sock`. `--assembly-logs` gives a trace of the emulated ARM (`-d in_asm`: each block once, when translated). `ASSEMBLY_LOGS_FLAGS=in_asm,exec,nochain` also logs every execution: the ARM becomes ~150 times slower. The bridge clock follows the DSP down to `PONT_HORLOGE_MAX` times real time (16 with `ASSEMBLY_LOGS=1`, 4 otherwise; the trace goes to `/var/tmp/c54x-pont`), but the wall-clock timers of mobile and of the core network still run, so use that only to chase the firmware.

Test it: `cd /opt/GSM/tests && ./banc-max.sh --grgsm --restart` (README-tests).

2.16 /opt/GSM/osmo-operator/doc/README-iso.md

5889 octets, 124 lignes

2.16.1 osmo-operator — the live ISO

The fastest path: a GNOME live image with the whole bench (2G, DSP, 4G, CSFB, postmarketOS phone), Wireshark, Linphone and the I/Q spectra. Nothing to compile or configure.



Figure 3: The ISO desktop: Telecom Operator, no RF, full stack

← README · Details: wiki/ISO, wiki/Home, wiki/Build

Live ISO password	`osmo`
postmarketOS phone password	`147147`
Minimum host	x86_64, 8 GB RAM, 4 cores, nested virtualization in a VM
Image identity	`IMAGE_VERSION="OSMO_EGPRS_V2"` · `OSMO_ROLE=operator` · `OSMO_LITE=0` (`/etc/os-release`, `/etc/osmo-role`)

2.16.1.1 Download

Latest release, or:

- Google Drive (single file)
- GitHub v0.1-9: part-00 · part-01 · part-02 · part-03 · SHA256SUMS

  [!IMPORTANT] Download **every** `.part-NN` file, not just the first one. GitHub caps release files at 2 GiB, so the image ships in pieces.

```
cat osmo-operator-desktop.iso.part-* > osmo-operator-desktop.iso
sha256sum -c --ignore-missing SHA256SUMS
```

2.16.1.2 Run the image

The live root is a tmpfs (~6 GB in `toram` mode). Logs and captures are bounded and purged at every boot. The bench runs two QEMUs (Calypso and pmOS): in a VM, **enable nested virtualization**, or the phone VM falls back to software emulation and becomes very slow.

QEMU

```
qemu-system-x86_64 -cdrom osmo-operator-desktop.iso -m 8G -enable-kvm \
  -cpu host -smp 4 -nic user,hostfwd=tcp::8080-:8080
```

VirtualBox

1. New VM: Linux / Ubuntu 64-bit, 8 GB, 4 CPUs, 3D off.
2. System → Processor: tick *Nested VT-x/AMD-V* (if greyed out: `VBoxManage modifyvm "<name>" --nested-hw-virt on`).
3. ISO on the optical drive. No disk is needed.
4. NAT port forward: host 8080 → guest 8080.

VMware: new VM from the ISO, Linux / Ubuntu 64-bit, 8 GB, 4 cores, tick *Virtualize Intel VT-x/EPT or AMD-V/RVI*, skip Easy Install.

USB stick (hybrid ISO, boots UEFI or BIOS):

```
sudo dd if=osmo-operator-desktop.iso of=/dev/sdX bs=4M status=progress conv=fsync
```

On Windows, use Rufus in *DD Image* mode. On real hardware KVM is available directly, which is the most comfortable setup.

2.16.1.3 Getting started

Three launchers are pinned in the dock. 2G and 4G are independent and can start in any order. **Launch the smartphone last**, once both networks are up.

	Launcher	What it starts
	2G (red handset)	The GSM core and the emulated radio: osmo-stp / bsc / msc / hlr / sgsn / ggsn / pcu, the Calypso under QEMU on the DSP ROM, the two BTS and the two osmocom-bb mobiles, Asterisk, the SMSC.
	4G (signal bars)	open5gs, srsENB and srsUE over ZMQ, and the SGs interface to the MSC (CSFB). See README-lte.
☎	**Smartphone**	The postmarketOS VM: oFono on the virtual modem, data on 4G, voice and SMS on 2G.

The **?** of the dock (`osmo-info`) lists the modes. From a terminal, the same 2G bench:

```
cd /opt/GSM/osmo-operator && sudo ./start-native.sh           # DSP (default)
cd /opt/GSM/osmo-operator && sudo ./start-native.sh --grgsm   # gr-gsm layer 1
```

`--menu` asks interactively (profile, layer 1 `grgsm_exe` or `c54x_exe`). Check that everything is in place with `checks/diag-stp-operator.sh`, or the full suite in README-tests. What is not in the ISO installs with `sudo ./addition.sh` (README).

2.16.1.4 What's inside

- **2G / GSM (Osmocom)**: osmo-stp (M3UA, SCTP :2905), osmo-bsc, osmo-msc, osmo-hlr, osmo-sgsn, osmo-ggsn, osmo-pcu, osmo-mgw (MGCP :2427). Two DCS1800 cells in LAC 1: BTS 0 (Cell ID 6001, BSIC 7) and BTS 1 (Cell ID 6012, BSIC 8). GPRS/EDGE, voice through Asterisk (SIP :5060), SMS-over-GSUP with SMPP :2775, A3A8 with a deterministic RAND so that runs are reproducible, live GSMTAP capture.
- **The emulated Calypso**: `qemu-system-arm -M calypso -cpu arm946` runs `layer1.highram.elf`, loaded by `osmocon` exactly as on a C123. See README-grgsm.
- **The DSP**: the Calypso's TMS320C54x on TI's real mask ROM. See README-dsp.
- **4G, CSFB and postmarketOS**: see README-lte.

2.16.1.5 Installing to disk and building the ISO

The live image installs with Calamares. The accounts are the ones from the *Users* screen plus an unlocked root, LUKS is offered in the partitioning choices, and a *Graphics drivers* page offers `nvidia-driver-610` when `lspci` sees a card.

`sudo ./build-iso.sh` does **one** Docker build (`osmocom-nitb`) and derives each rootfs from it: interstp, operator, lite (operator without the workshops), desktop (`--role=operator|interstp`, `--node=N`, `--lite`, `--desktop`, `--all`, `--arm`). Options, modules (`iso_modules/NN-*.sh`) and CI are covered in wiki/ISO and wiki/Build. The Raspberry Pi image is in README-arm.

2.17 /opt/GSM/osmo-operator/doc/README-lte.md

3667 octets, 91 lignes

2.17.1 osmo-operator — 4G, CSFB and the postmarketOS phone

On top of the 2G bench: an LTE network (open5gs + srsRAN over ZMQ, no SDR), tied to the GSM core by SGs for **CSFB**, and a postmarketOS phone VM that does data on 4G and voice/SMS on 2G under the same identity.

← README · Details: wiki/ISO, wiki/Home, wiki/Telephone-emule

Available on amd64 only. srsRAN and Open5GS are not built for arm64 (README-arm).

2.17.1.1 Overview

```
flowchart LR
    subgraph PMOS["postmarketOS VM"]
        NM["NetworkManager / pppd"]
        OF["oFono"]
    end
    PS["osmo-phonesim-banc.py<br/>27.007 AT modem"]
    NM -->|"virtio osmo.data<br/>ATD*99***1#"| PS
    OF -->|"virtio osmo.modem"| PS
    PS -->|data| UE["srsUE (netns)"]
    UE -->|ZMQ| ENB["srsENB<br/>EARFCN 3350, band 7"]
    ENB -->|S1| MME
    subgraph EPC["open5gs"]
        MME --- SGW["SGW / SMF / UPF"]
        MME --- HSS
    end
    SGW -->|NAT| NET((Internet))
    MME <-->|"SGs :29118"| MSC["osmo-msc (2G)"]
    PS -->|"voice / SMS"| MS["osmocom-bb mobile"] --> BTS["2G BTS"] --> MSC
```

2.17.1.2 4G / LTE

- open5gs core (MME, SGW-C/U, SMF, UPF, PCRF, HSS) on dedicated loopbacks, Prometheus :9090, WebUI :9999, MongoDB behind the HSS.
- PLMN 001/01, TAC 7. NAS null ciphering (EIA2/EIA1 + EEA0) on purpose: this is an analysis bench, and NAS reads in clear in Wireshark.
- srsENB (10 MHz, EARFCN 3350, band 7) and srsUE over ZMQ. Milenage test subscriber IMSI 001010001000001. The UE gets an IP in a netns and reaches the Internet through NAT.

Shipped in `/opt/LTE`, launched with `osmo-lte` and `osmo-epc` (`tools/osmo-lte.sh`, `tools/osmo-epc.sh`, unit `services/osmo-lte.service`), or the **4G** launcher in the dock. Installed by `tools/osmo-lte-install.sh`.

2.17.1.3 CSFB

- SGs between osmo-msc and the MME (:29118), mapping `TAI (001-01, TAC 7) → LAI (001-01, LAC 1)`.
- Combined EPS+IMSI attach: an incoming call is paged by the MME and the UE falls back to the BTS.
- Two subscribers (100101, 100102), one per cell, wired for an inter-cell call with fallback.

```
sequenceDiagram
    participant UE as UE (on LTE)
    participant MME
    participant MSC as osmo-msc
    participant BTS as 2G BTS
    UE->>MME: combined EPS + IMSI attach
    MME->>MSC: SGs location update
    Note over MSC: incoming call for the UE
    MSC->>MME: SGs paging
    MME->>UE: paging (CS domain)
    UE->>MME: extended service request
    UE->>BTS: fallback: camps on 2G
    UE->>MSC: paging response, call set up on 2G
```

2.17.1.4 postmarketOS: the phone

- x86_64 pmOS VM with a patched kernel (`CONFIG_PPP` enabled, missing upstream; patch `patches/pmaports-linux-postmar
  The prebuilt kernel is in `/opt/user_interface/kernel/pmos/`. Password `147147`.
- 4G data: NetworkManager → pppd (`ATD*99***1#`) → virtio-console `osmo.data` → `osmo-phonesim-banc.py` → UE netns → srsUE → srsENB → UPF → Internet.
- 2G voice and SMS: virtio-console `osmo.modem`, where `osmo-phonesim-banc.py` plays a 27.007 AT modem driven by oFono. No dongle.
- Audio: two duplex PulseAudio chains (GAPK bridge to the osmocom-bb mobile, and an echo-cancelled chain).

Tools: `tools/osmo-pmos-build.sh`, `osmo-pmos-install.sh`, `osmo-pmos-setup.sh`, `osmo-pmos-qemu.sh`, `osmo-pmos.sh` (build, install, setup, QEMU, launch) and a patched `pmbootstrap` in `/opt/user_interface/pmos` (`patches/pmbootstrap-osmo-ben
Launch the phone last, once 2G and 4G are up.

Test it: `cd /opt/GSM/tests && ./banc-max.sh --4g` (README-tests).

2.18 /opt/GSM/osmo-operator/doc/README-native.md

5798 octets, 126 lignes

2.18.1 osmo-operator — native install (no Docker)

The repository installs directly on Ubuntu 24.04 (noble, the same base as the Dockerfile). The launcher, `start-native.sh` (a symlink to `start.sh`), is the same one used inside the container and on the ISO.

← README · Details: wiki/Start-native, wiki/Build, wiki/Environnement, wiki/Lancer-a-la-main

2.18.1.1 Install

The complete native path is the notebook `wiki/installer-osmo-jusquau-dsp.ipynb`: on a fresh Ubuntu 24.04 it builds everything (apt dependencies, Osmocom core, SMSC, osmocom-bb, firmware, qosmo, c54x_exe, runtime setup) and ends with a verified call on the DSP. How to run it (disposable container, fresh 24.04, unattended): wiki/README-notebook. The same steps as plain text: doc/install-from-scratch.

`install.sh` is a step engine (`install_modules/NN-<step>.sh`): `prereqs`, `deps` (apt packages extracted from the Dockerfile), `user` (the `osmocom` account), `sources`, `patches` (`tools/osmo-patches-apply.sh`), `build`, `binaires`, `configs`, `bureau` (desktop icons and shortcuts, the same files as the ISO), `verify`. `GSM_ROOT` (default `/opt/GSM`) receives sources and binaries.

```
sudo ./install.sh                 # every step
./install.sh --list               # show the steps, do nothing
./install.sh --dry-run            # walk through with no side effects
./install.sh --check              # what is already in place
sudo ./install.sh --only deps     # one step
sudo ./install.sh --skip build    # every step but this one
sudo ./install.sh --reinstall     # replay everything, even what is already there
sudo ./install.sh --only bureau   # desktop icons and shortcuts only
```

To install the pieces one by one instead: wiki/Installer-qosmo, wiki/Installer-grgsm_exe, wiki/Installer-c54x_exe. The supplements (gnuarm toolchain, Magnetite, extra firmware, 4G…) are `sudo ./addition.sh` (README).

2.18.1.2 Start and stop

```
./start-native.sh --regen     # first run: regenerate configs from the templates
./start-native.sh             # start the bench (delegates to /opt/GSM/qosmo/run.sh)
./start-native.sh --status    # what is running
./start-native.sh --stop      # stops the stack AND the DSP bench, with or without --dsp
```

Every start stops the stack first: there is no hot restart. `--force` only restarts modules that are already running instead of skipping them. `--no-attach` starts without attaching to tmux (`CALYPSO_NO_ATTACH=1`), which is what `osmo-banc.service` uses (`OSMO_BANC_ARGS` in `/etc/default/osmo-banc`).

2.18.1.3 Choosing what runs

The layer 1 (who demodulates the air interface):

```
./start-native.sh --dsp       # real C54x mask ROM (default)        → README-dsp
./start-native.sh --grgsm     # gr-gsm layer 1                      → README-grgsm
./start-native.sh --menu      # ask interactively
```

The radio setup (see the modes table): `--bts-classique` (`osmo-bts-trx` on the bridge, default), `--bts` (the Calypso is the BTS radio) with `--ts 1|2|2/1`, `--ms` / `--ms-magnetite` (the phone), and the vocoder, `--vocoder-rom` (default) or `--gapk`.

The profile (how many base stations come up, and how):

profile	stack
`faketrx-qemu` (default)	core + BTS#0 QEMU + BTS#1 faketrx (also `hybrid`)
`faketrx`	core + `fake_trx` + `trxcon` + `mobile`
`qemu`	the Calypso QEMU pipeline alone
`noproc`	core only (also `core`)

```
./start-native.sh faketrx
./start-native.sh --profile qemu
./start-native.sh --list        # show the plan without running it
./start-native.sh --dry-run     # walk through it with no side effects
```

The layer-2 client of the Calypso: `--mobile` (default), `--ccch_scan`, `--bcch_scan`, `--cell_log`. Identity and network options (`--node`, `--op`, `--hub-ip`, `--wan`) are in README-ss7. A real SIM card in an Omnikey reader or the softSIM (`--sap`, `--sap=pcsc`, `--sap=sim`, `--no-sap`) is in wiki/SIM. `./start-native.sh --help` lists everything.

2.18.1.4 The `qosmo` launcher

`run_modules/40-qemu.sh` does not call `qemu-system-arm` directly: it calls the C launcher `qosmo`, installed next to it by `ninja install`. The launcher knows which layer 1 its QEMU was built with.

```
qosmo -k /opt/GSM/firmware/board/compal_e88/layer1.highram.elf
qosmo --help
```

Defaults: `-M calypso`, `-cpu arm946`, `-gdb tcp::1234`, `-serial pty -serial pty`, a unix monitor, L1CTL on `/tmp/osmocom_l2`, TRXDv0 on `0.0.0.0:6702`. `start-native.sh` resolves it (`--launcher <bin>` or `QOSMO_LAUNCHER`), builds it if the source is there and the binary is not, and falls back to the plain `qemu-system-arm` command line without it.

2.18.1.5 Variables

Which variable wins (command line, environment, `/etc/osmo-role`, defaults) and what actually reaches QEMU: wiki/Environnement and `environment/README.md`. The network settings (ARFCN, PLMN, …) are in `globals.conf`, applied by `generate_configs.sh`. Systemd units: `services/README.md`.

Using the running bench (tmux, VTY, Wireshark): README-usage.

2.18.1.6 Known issues

- `install.sh` does not build the Osmocom core, qosmo, c54x_exe or the firmware, so on a stock 24.04 its `verify` step ends with “demons absents”. Use the notebook for a complete native build; the notebook reuses the `deps`, `user` and `configs` steps of `install.sh`.

2.19 /opt/GSM/osmo-operator/doc/README-ss7.md

5913 octets, 141 lignes

2.19.1 osmo-operator — multi-operator, SS7 and WAN

The standalone bench is one operator. The bench also runs **N operators (1 to 9)**, each with its own core network, joined by a central **inter-STP** that routes M3UA/SCCP between them, on one host (Docker) or across machines (WAN). No configuration file is edited by hand: pjsip, dialplan, SMS routing and the inter-STP config are all generated from N.

← README · Details: wiki/WAN, wiki/Payload-ISM, wiki/Start-native §7-wan, wiki/Reference §9

2.19.1.1 On one host (Docker)

```
sudo ./launch/start-docker.sh          # choose bridge mode, enter N operators
sudo ./launch/start-docker.sh stop
```

```
flowchart LR
    subgraph OP1["osmo-operator-1"]
        BSC_A["BSC"] -->|127.0.0.1:2905| STP_A["STP"]
        MSC_A["MSC"] -->|127.0.0.1:2905| STP_A
    end
    subgraph OP2["osmo-operator-2"]
        BSC_B["BSC"] -->|127.0.0.1:2905| STP_B["STP"]
        MSC_B["MSC"] -->|127.0.0.1:2905| STP_B
    end
    INTER["osmo-inter-stp<br/>172.20.0.10:2908"]
```

```
    STP_A <-->|"M3UA, RCTX N×100+50"| INTER
    STP_B <-->|"M3UA"| INTER
    A1["Asterisk 1"] <-->|SIP trunk| A2["Asterisk 2"]
    R1["SMS relay 1"] <-->|"TCP 7890"| R2["SMS relay 2"]
```

Docker network	range	role
`gsm-inter`	`172.20.0.0/24`	interop backbone (inter-STP M3UA), inter-STP at `.10`, operator N at `.(10+N)`
`gsm-net-opN`	`172.20.N.0/24`	operator N private network

Start order matters: the inter-STP must listen on `:2908` **before** the operator STPs connect.

2.19.1.2 Point codes and routing contexts

The single-host plan is `N.23.<role>`. As soon as operators are spread over several **nodes**, the node goes into the point code, otherwise three nodes would present the same address to the hub:

plan	point code	routing context
single host	`N.23.<role>` (inter-STP `0.23.0`)	`rctx_inter = N×100+50`
WAN	`1.<node><op>.<role>`, role 1=MSC 2=STP 3=BSC (inter-STP `0.0.0`)	`node×1000 + op×100 + 50`

The inter-STP uses **AS without a routing key** (catch-all per operator) and routes on the DPC alone. Each operator STP sends unknown PCs to `as-inter` through a `0.0.0/0` catch-all route.

```
flowchart LR
    MSC_A["MSC Op1"] -->|"SCTP :2905"| STP_A["STP Op1"]
    STP_A -->|"catch-all → as-inter<br/>:2908"| INTER["Inter-STP"]
    INTER -->|"DPC of Op2 → as-op2"| STP_B["STP Op2"]
    STP_B -->|"dynamic route"| MSC_B["MSC Op2"]
```

2.19.1.3 SMS and voice between operators

- **SMS**: `sms-interop-relay.py` watches the `proto-smsc-daemon` MO log, parses the GSM 03.40 TPDU, looks up `sms-routing.conf` (longest prefix), and forwards over TCP 7890 to the target relay. That relay resolves MSISDN→IMSI on its HLR and calls `proto-smsc-sendmt`.
- **Voice**: a SIP trunk between the operators' Asterisk instances.

Numbering on the WAN works like a country code. Node N has prefix `NN` (11, 22…), and a mobile's MSISDN is `<node>00<op><ms>`. From node 1, dial `22 + 20011` to reach the first mobile of node 2. The sending node strips the prefix.

2.19.1.4 Across machines (WAN)

```
./start-native.sh --wan                                  # interactive
./start-native.sh --wan-nodes "1:IP:11 2:IP:22" --wan-id 1   # scriptable
./start-native.sh --wan mynode2.conf                     # from another node's card
./start-native.sh --gen-conf                             # write this node's card, start nothing
./start-native.sh --regen-conf …                         # rebuild the WAN table from scratch
./start-native.sh --virtualbox=3                         # mesh with VirtualBox VMs
./start-native.sh --air-mesh                             # mesh the BURSTS: a shared radio medium
./start-native.sh --payload-ism                          # SMS text over an ISM radio link
```

`--node N` picks the node number **at launch**, so one ISO serves all nine nodes. One node runs the inter-STP (`--interstp`), the others point at it with `--hub-ip`.

```
flowchart LR
    subgraph A["machine A — node 1"]
        S1["stack, STP 1.11.2"]
        HUB["inter-STP :2908"]
    end
    subgraph B["machine B — node 2"]
        S2["stack, STP 1.21.2"]
    end
    S1 -->|M3UA| HUB
    S2 -->|"M3UA / SCTP 2908"| HUB
    S1 <-->|"SIP 5060, RTP 20000-20499"| S2
    S1 <-->|"SMS TCP 7890"| S2
```

Over the Internet, put the nodes in a **WireGuard** tunnel: SCTP does not cross consumer NAT, and the table address must be both reachable and local. The full method (tunnel, firewall, start order, layer-by-layer checks, common failures) is in wiki/WAN.

2.19.1.5 Diagnosing PROHIB routes

```
sudo checks/diag-interstp.sh        # from the hub: who is attached, PROHIB routes, flapping
sudo checks/diag-stp-operator.sh    # from an operator node
sudo checks/ss7_check.sh
```

A healthy operator STP route table:

```
N.23.1/14   as-dyn-…  avail  avail  dyn   ← MSC (dynamic)
N.23.3/14   as-dyn-…  avail  avail  dyn   ← BSC (dynamic)
0.0.0/0     as-inter  avail  avail        ← catch-all to the inter-STP
```

symptom	likely cause	fix
route `0.0.0/0` PROHIB	inter-STP down, or wrong IP	check the hub IP and the backbone IP
PROHIB towards a node that was switched off	stale entry in `/etc/osmo-wan.conf`	`--regen-conf` on the node that runs the hub
no dynamic routes	MSC/BSC not reaching the local STP	check that the ASPs point at `127.0.0.1`
ASP DOWN on the inter-STP	Docker race	`docker restart` the operator

The SS7 console (`ss7-console.py`, navigation/QUICKSTART.md) sends MAP queries across the interconnect.

Test it: `cd /opt/GSM/tests && ./banc-max.sh --multi` (README-tests).

2.20 /opt/GSM/osmo-operator/doc/README-tests.md

8607 octets, 175 lignes

2.20.1 osmo-operator — tests and reference results

Two families of tests:

- **bbaranoff/tests** (cloned in `/opt/GSM/tests`) climbs the bench **element by element**, from core to voice, and says how high it got. This is the end-to-end test suite.
- **`checks/`** in this repository is a set of read-only diagnostics of a running stack (daemons, SS7, interconnect, WAN, directory).

← README · Details: wiki/Resultats, wiki/Start-native

2.20.1.1 Reference run

The reference report is `tout-en-un-20261016-164539.qmd`, a Quarto report of `banc-max.sh` in **DSP mode** (layer 1 = real TI mask ROM). Render it with `quarto render tout-en-un-20261016-164539.qmd` (self-contained HTML). A healthy bench reproduces these results: **27/27 rungs, 0 failures**, 14 signal-path steps handled by the ROM and none on the host.

#	rung	expected result
1	demarrage	full stop, then `start-native.sh --dsp`: core up
2	coeur	processes + VTY 4258 4239 4254 4242 4243
3	bts	osmo-bts-trx, VTY 4241
4	services	asterisk 5060, sip-connector, `/tmp/sendmt_socket`
5	ss7	`ss7_check` with no failure, `diag-stp-operator`
6	checks-existants	`global_check`, `call-diag`, `audio-diag` with no failure
7	couche1	`c54x_exe --arm`, `qemu-system-arm`, `osmocon`, `pont_dsp.py`
8	mobile	VTY 4247, IMSI 001010001000001
9	camp	ARFCN 514 (DCS), CGI 001-01-1-6001
10	attache	MM idle normal service, in the VLR, MSISDN 100101
11	sms-mo	SMS to 100102, RP-ACK
12	sms-mt	MSC delivered 1 → 2
13	ussd	`*#100#` → 100101, `*#101#` → IMSI
14	appel	CC ACTIVE
15	voix	1 kHz tone comes back, energy concentrated in 950–1050 Hz
16–20	data, ms2, 4g, multi, liens	skipped unless `--data`, `--ms2`, `--4g`, `--multi`
21–24	pytest infra / couche1 / mobile / reseau	passed
25	pytest-legacy	skipped unless `--legacy`
26	emulateur	ISA examples checked against the reference, no BC divergence
27	couverture	layer-1 coverage, see below

Layer-1 coverage (`couverture.txt`): every step from FCCH detection to CHANNEL RELEASE is **handled by the ROM**, including A5 on the coprocessor and uplink TCH speech; the DSP stays within the 4.62 ms frame budget. Open points are listed under Known issues.

2.20.1.2 The ladder: `banc-max.sh`

The bench stacks a dozen elements that depend on each other: no core means no BTS, no layer 1 means no mobile, and without camping there is no attach, SMS or call. When something does not work, the useful question is **how far it got**. The last rung reached names the first faulty element.

```
flowchart LR
    D[demarrage] --> C[coeur] --> B[bts] --> S[services] --> SS7[ss7]
    SS7 --> L1[couche1] --> M[mobile] --> CA[camp] --> A[attache]
    A --> MO[sms-mo] --> MT[sms-mt] --> U[ussd] --> AP[appel] --> V[voix]
    V -.->|--data| DA[data]
    V -.->|--ms2| MS2[ms2]
    V -.->|--4g| G4[4g]
    V -.->|--multi| MU[multi / liens]
    V --> PY[pytest x4] --> EM[emulateur] --> CO[couverture]
```

One module per rung, in modules/NN-name.sh, played in numeric order. It never fixes anything: every verdict is a reading made at test time, with MSC counters taken as deltas.

```
cd /opt/GSM/tests
./banc-max.sh                       # default: = --dsp --restart --continue, ROM vocoder (io-handler l1phy)
BANC_VOCODEUR=hote ./banc-max.sh    # same, with gapk on the host instead of the ROM vocoder
./banc-max.sh --campagne            # grgsm then dsp (each restarted, up to voice) + PDF
./banc-max.sh --dsp --restart       # one mode, restart the stack in it
./banc-max.sh --grgsm               # one mode, on the running stack
./banc-max.sh --full                # both layer 1s, 4G and inter-operator, up to voice
./banc-max.sh --ms2 --data          # also the MS#1 -> MS#2 and GPRS rungs
./banc-max.sh --only=camp,attache   # only these rungs
./banc-max.sh --list                # the rungs, in order
```

Results land in `/root/banc-max-<date>/` (`verdict.txt`, `couverture.txt`, VTY captures, pytest reports, grafcets). Everything runs detached (`CALYPSO_NO_ATTACH=1`), never in an interactive tmux.

2.20.1.3 Run report: `tout-en-un.py`

tout-en-un.py gathers the `.log .sh .md .mmd .py .txt` of the tests and of the last `/root/banc-max-*` into **one Quarto/Markdown file**, shortened without loss: identical files are cited once, repeated lines become `×N`, and lines that differ only by numbers become a template plus the list of values. `.mmd` files are rendered as Mermaid diagrams.

```
./tout-en-un.py                                 # /opt/GSM/tests + the last /root/banc-max-*
./tout-en-un.py /root/banc-max-<date>           # a given run
SORTIE=/tmp/run.qmd EXCLURE="banc-max-*" ./tout-en-un.py
```

`qmd-worker/` serves these reports on the web (Cloudflare Worker).

2.20.1.4 pytest suites

pytest/ reads the **live** stack (logs, VTY). The four suites are called by rungs 21–24, and each writes `test_results.md`, a grafcet and Mermaid diagrams:

file	suite
test_10_infra.py	processes, ports, VTY
test_20_couche1_grgsm.py / test_21_couche1_dsp.py	layer 1, per chain
test_30_mobile.py	the osmocom-bb mobile
test_40_reseau.py	network: LU, SMS, call
test_50_actif.py	active tests

```
cd /opt/GSM/tests/pytest && ./run_tests.sh
```

Other tools in the repository: `sms-rafale.sh` (SMS burst), `inject.py`, `log_timeline.py`, `rapport.py`, and rom-tx/ (uplink encoding by the ROM, block injection).

2.20.1.5 `checks/`: diagnostics of a running stack

All read-only. `check_all.sh` chooses what makes sense on this node, runs from general to specific, and gives one verdict per check and an exit code.

```
sudo checks/check_all.sh            # everything relevant here
sudo checks/check_all.sh --dump     # + vty-debug-dump and its summary (daemons in DEBUG, several MB)
```

script	looks at
`global_check.sh`	every daemon: STP, HLR, MSC, BSC, BTS, PCU, SGSN, Asterisk, SMS relay
`ss7_check.sh`	signalling: ASPs, ASs, routes, inter-STP
`diag-stp-operator.sh`	the interconnect from an operator node: config vs running daemon, SCTP to the hub
`diag-interstp.sh`	the interconnect from the hub: who is attached, who is missing, PROHIB routes, flapping
`wan_ss7_check.sh`	what crosses between machines (only when `/etc/osmo-wan.conf` exists)
`annuaire.sh`	the subscriber directory, checked against what the network knows
`vty-debug-dump.sh` + `operator_summary.sh`	full VTY dump, then a per-operator summary

`wiki/Resultats.md` holds the reference `check_all.sh` runs (1 native operator; 3 operators with the inter-STP) and explains the known deviations.

Targeted diagnostics in `scripts/`:

```
scripts/call-diag.sh            # call path MS -> MSC -> MNCC -> sip-connector -> Asterisk
scripts/audio-diag.sh           # /dev/snd -> ALSA -> PulseAudio -> mobile l1phy
scripts/banc-repro.sh --calls 5 # a reproducible experiment for an intermittent fault
navigation/test-console.sh      # every command of the SS7 console, with a report
```

2.20.1.6 CI

The GitHub workflows (`.github/workflows/`) build the images and the ISO. They do not run the radio bench. The ladder needs a full stack with QEMU and is run on a real machine.

2.20.1.7 Known issues

- TCH/F downlink quality (`B_BFI`) is DEGRADED in the coverage report: really bad frames and FACCH steals, masked by `layer1.highram_patched`.
- Frequency hopping is NOT HANDLED (outside this setup).
- MVKD/MVDK guard, uplink speech ring and host vocoding are NOT OBSERVED by the coverage rung.
- 3 of the emulator rung's ISA examples fail and some are not assembled.

2.21 /opt/GSM/osmo-operator/doc/README-usage.md

2853 octets, 97 lignes

2.21.1 osmo-operator — using the bench

Day-to-day handling of a running bench: tmux, VTY, numbers, ports, Wireshark.

← README · Details: wiki/Home, wiki/Start-native, README-architecture

2.21.1.1 tmux

```
tmux -S /tmp/osmocom_tmux attach -t osmocom                      # native / ISO
docker exec -ti osmo-operator-1 tmux -S /tmp/osmocom_tmux attach -t osmocom   # Docker
```

key	window

key	window
`Ctrl-b 0`	faketrx
`Ctrl-b 1`	MS1 (trxcon + mobile)
`Ctrl-b 2`	Asterisk
`Ctrl-b 3`	SMSC (proto-smsc-daemon + relay)
`Ctrl-b w`	list all windows
`Ctrl-b d`	detach

2.21.1.2 VTY

port	component
`4239`	OsmoSTP
`4241`	OsmoBTS
`4242`	OsmoBSC
`4243`	OsmoMGW
`4254`	OsmoMSC
`4258`	OsmoHLR
`4247` / `4347`	osmocom-bb mobile (gr-gsm / DSP)

```
OsmoMSC# show subscriber all
OsmoMSC# subscriber msisdn 100101 sms sender msisdn 100102 send Hello

OsmoHLR# subscriber imsi 001010001000001 show
OsmoHLR# show gsup-clients
```

Subscribers are provisioned automatically at start-up by `run_modules/21-abonnes-hlr.sh`, following the numbering plan. `--regen` re-applies it.

2.21.1.3 Numbers

number	use
`<node>00<op><ms>` (e.g. `100101`)	GSM subscribers
`600`	echo test
`*#100#` / `*#101#`	USSD: own MSISDN / own IMSI
`<prefix><number>` (e.g. `2220011`)	a subscriber on another WAN node (README-ss7)

2.21.1.4 Ports on the wire

port	carries
`2905` SCTP	M3UA, local STP
`2908` SCTP	M3UA, inter-STP
`2427` UDP	MGCP (MGW)
`5060` UDP	SIP (Asterisk)
`2775` TCP	SMPP
`7890` TCP	inter-operator SMS relay
`5700-5702` UDP	TRXD, between `osmo-bts-trx` and the bridge
`4729` UDP	GSMTAP, the Um interface
`4730` UDP	full SI set into the shunt (`si_bridge.py`)
`4731` UDP	SCH BSIC/FN out of the gr-gsm receiver
`6702` UDP	DL bursts into the DSP BSP (`--dsp`)
`8080` TCP	the web dashboard (`osmo-egprs-web`)

2.21.1.5 Wireshark

Started automatically with `sctp or udp port 4729`. Useful filters:

```
gsmtap                  # radio traffic (Um)
gsm_a.dtap              # MM / CC / SMS layer 3
gsm_map                 # MAP
m3ua / sccp             # SS7, multi-operator
```

A tcpdump wrapper is installed for GSMTAP (`udp/4729`) and M3UA captures. `tools/osmo-wireshark-root.sh` opens Wireshark with capture rights.

2.21.1.6 Sending things by hand

```
scripts/send-mt-sms.sh              # an MT SMS to a mobile
tools/at-cmd.py                     # AT commands to the emulated phone's modem
```

2.21.1.7 Dashboard

`osmo-egprs-web` on `:8080` (`systemctl restart osmo-egprs-web`). The desktop also runs a conky status panel (`tools/conky-osmo-status.sh`).

2.22 /opt/GSM/osmo-operator/doc/c54x_exe-depuis-ubuntu-vierge.md

9241 octets, 171 lignes

2.22.1 c54x_exe depuis une Ubuntu 24.04 vierge

Oct 20, 2026 · Bastien Baranoff

2.22.1.1 Ce que montre cette page

En une vingtaine de minutes, le cœur TMS320C54x émulé de `c54x_exe` charge la mask-ROM TI d'un vrai Calypso et l'exécute, sans QEMU, sans ARM, sans réseau GSM.

C'est le hello world du DSP : il prouve que le cœur exécute du vrai code TI. Il ne prouve pas que le DSP décode quoi que ce soit, puisque aucun burst n'est injecté. Le montage complet (ARM sous QEMU, BTS, pont, appel voix) est décrit en fin de page et vit dans osmo-operator.

Tout se fait en root ou avec `sudo` ; les chemins `/opt/GSM/…` sont ceux que les Makefile et les scripts attendent par défaut.

2.22.1.2 1. Prérequis (Ubuntu 24.04)

Une ligne `apt` suffit : le compilateur, `git`, `curl` pour les ROM, et `libosmocore` (qui contient `libosmocoding`, utilisée par `calypso_bsp.c`).

```
sudo apt update
sudo apt install -y build-essential git curl pkg-config libosmocore-dev
sudo mkdir -p /opt/GSM
```

Vérifier que `libosmocoding` est bien là :

```
pkg-config --modversion libosmocoding
```

Si `pkg-config` ne la trouve pas, la version de `libosmocore` du dépôt Ubuntu est trop ancienne ou incomplète ; installer alors `libosmocore-dev` depuis le dépôt nightly d'Osmocom (voir osmocom.org).

2.22.1.3 2. Les sources de qosmo

`c54x_exe` ne recopie pas le cœur C54x : il compile directement les sources de qosmo (`calypso_c54x.c` et ses voisins) avec ~120 lignes de doublures d'en-têtes QEMU. Il faut donc le dépôt sur disque, mais pas le construire.

```
cd /opt/GSM
git clone https://github.com/bbaranoff/qosmo
```

Aucun `configure`, aucun `make` ici. Si qosmo est ailleurs que `/opt/GSM/qosmo`, passer `QOSMO=/chemin` au `make` de l'étape suivante.

2.22.1.4 3. Cloner et compiler c54x_exe

La compilation prend moins d'une minute et produit un seul binaire, `c54x_exe`, à la racine du dépôt.

```
cd /opt/GSM
git clone https://github.com/bbaranoff/c54x_exe
cd c54x_exe
make QOSMO=/opt/GSM/qosmo \
     CFLAGS="-O3 -g -Wall -Werror=format -Werror=format-extra-args \
             -Wno-unused-function -Wno-unused-variable \
             -Wno-unused-but-set-variable -Wno-sign-compare"
```

Ce `CFLAGS` est celui du Dockerfile d'osmo-operator : il garde les erreurs de format bloquantes et rend muets les avertissements de variables inutilisées, nombreux dans les sources QEMU reprises telles quelles. Un `make` nu marche aussi mais sort beaucoup plus de bruit.

`make` reconstruit le binaire à chaque appel (cible `.PHONY`) ; pas besoin de `make clean` après un changement dans qosmo.

Vérifier :

```
./c54x_exe --help | head
```

2.22.1.5 4. Les ROM DSP, depuis FreeCalypso

La mask-ROM du C54x n'est pas dans le dépôt : elle vient du dump publié par le projet FreeCalypso (Mychaela Falconia), sur son FTP public. Le script `rom/fetch-rom.sh` la télécharge et la découpe en 7 sections au format attendu par `--rom-dir`.

```
cd /opt/GSM/c54x_exe
export FREECALYPSO_URL=ftp://ftp.freecalypso.org/pub/GSM/Calypso
bash rom/fetch-rom.sh --dest /opt/GSM --dest /opt/GSM/c54x_exe/rom
curl -fsS --retry 2 -o /opt/GSM/calypso_dsp.txt "$FREECALYPSO_URL/dsp-rom-3606-dump.txt"
cp /opt/GSM/calypso_dsp.txt rom/calypso_dsp.txt
```

`3606` est la version de ROM DSP du Calypso des Motorola C1xx ; c'est aussi la valeur que la L1 DSP écrit dans l'API RAM au boot, ce qui permettra de vérifier plus loin que la bonne ROM tourne.

Après le script, `/opt/GSM` doit contenir exactement ces 7 fichiers :

```
ls -l /opt/GSM/calypso_dsp.*.bin
# calypso_dsp.PROM0.bin  calypso_dsp.PROM1.bin  calypso_dsp.PROM2.bin
# calypso_dsp.PROM3.bin  calypso_dsp.DROM.bin   calypso_dsp.PDROM.bin
# calypso_dsp.Registers.bin
```

Si `curl` refuse le FTP (certaines builds le compilent sans), le dump se télécharge à la main depuis le même répertoire sur ftp.freecalypso.org et se passe à `fetch-rom.sh` en local.

2.22.1.6 5. Premier run : la ROM tourne

```
cd /opt/GSM/c54x_exe
./c54x_exe --trames 200
```

`--trames 200` fait tourner le DSP sur 200 trames TDMA (une trame = 4,615 ms de temps GSM), avec le budget d'instructions par défaut de 2300 par trame en mode autonome. Sans option, seules les erreurs du cœur passent sur stderr ; le bilan final dit combien de lignes ont été masquées et avec quel niveau `-v` les voir.

Ce qu'on doit voir, dans l'ordre :

1. Les 7 sections de ROM chargées depuis `/opt/GSM` (ou `--rom-dir`).
2. `c54x_reset()` : la ROM repart de `0xff80`, son vecteur de reset.
3. Des lignes `BRANCH-TRACE` autour de `PC=0xb41f` : c'est la boucle d'attente de la ROM TI, du vrai code que le cœur exécute.
4. Un bilan de fin avec `a_sch[3] = 0x771a`.

Pour voir une ligne par trame :

```
./c54x_exe --trames 50 --verbeux
```

Et pour les traces du cœur, `-v` à `-vvvvvv` (voir `--help`).

2.22.1.7 6. Lire le résultat

`a_sch[3] = 0x771a` est une valeur constante d'un run à l'autre : la ROM a booté, s'est parquée en IDLE et a laissé cet état dans sa fenêtre API. C'est la preuve que le cœur exécute fidèlement la ROM jusque-là.

Ce que ça ne prouve pas : le décodage. L'API RAM est vierge, aucun burst n'arrive, le DSP tourne à vide. Un décodeur dont la sortie ne dépend pas de l'entrée ne décode pas ; faire varier l'entrée demande le montage complet de la section 8.

Point de repère utile : le README de `qemu-calypso` donne `0xf8d8` pour ce même mot. La différence vient de l'entrée (QEMU fournit une API RAM initialisée par l'ARM, ici elle est vide), pas d'un bug : deux entrées différentes, deux constantes différentes.

2.22.1.8 7. Erreurs classiques

Symptôme	Cause	Remède
`make` : `qemu/osdep.h` ou `calypso_c54x.c` introuvable	qosmo n'est pas à `/opt/GSM/qosmo`	`make QOSMO=/chemin/vers/qosmo`
`make` : `osmocom/coding/...` introuvable	`libosmocore-dev` absent ou trop vieux	section 1, dépôt nightly Osmocom
`fetch-rom.sh` échoue ou `curl: (1) Protocol "ftp" not supported`	`curl` compilé sans FTP, ou pare-feu	télécharger à la main, puis `fetch-rom.sh` sur le fichier local
`c54x_exe` : ROM introuvable	les `.bin` ne sont pas dans `/opt/GSM`	`--rom-dir /chemin` ou refaire la section 4
Le bilan ne montre pas `0x771a`	mauvaise ROM, ou sections mal découpées	comparer `rom/calypso_dsp.txt` au dump 3606 ; refaire la section 4
`PC=0xb41f` n'apparaît jamais, le cœur sort en erreur	régression dans qosmo	`cd /opt/GSM/qosmo && git log -3`, remonter au commit du README

2.22.1.9 8. La suite : injecter des bursts

Pour que le DSP décode vraiment, il faut l'ARM (le firmware `layer1.highram.elf` d'osmocom-bb sous QEMU `qosmo`), une BTS et le pont qui lui livre des bursts. Deux chemins :

Le banc complet. Il pose tout, dans les mêmes chemins `/opt/GSM/…` que cette page :

```
cd /opt/GSM
git clone https://github.com/bbaranoff/osmo-operator
cd osmo-operator
sudo ./install.sh
sudo ./start-direct.sh --dsp
```

Le `--dsp` remplace la couche 1 gr-gsm par `c54x_exe` ; `--menu` pose la même question interactivement. Le wiki d'osmo-operator décrit les options, les ports et le plan de numérotation.

Exe par exe. Pour voir chaque étape produire son log : le DSP (`c54x_exe --arm -v`), puis QEMU avec `CALYPSO_DSP_EXTERN=1`, puis `osmocon`, puis `mobile`. La séquence, avec les lignes de journal attendues à chaque pas, est dans LAUNCH.md et le README de c54x_exe.

À partir de là, `a_sch` change avec l'entrée, et la question « est-ce que ça décode ? » a une réponse mesurable.

2.22.1.10 9. Le firmware Calypso : la toolchain arm-elf (gnuarm)

Le firmware d'osmocom-bb (`layer1.highram.elf`, `trx.highram.elf`…) se compile avec la toolchain historique `arm-elf` (gcc 4.5.2, binutils 2.21.1, newlib 1.19) du gnu-arm-installer, installée dans `/root/gnuarm/install`. Sur Ubuntu 24.04 son `cc1` réclame `libmpfr.so.4`, que le système ne fournit plus (il a `libmpfr.so.6`) ; un lien suffit :

```
ln -sf /usr/lib/x86_64-linux-gnu/libmpfr.so.6 /usr/local/lib/libmpfr.so.4 && ldconfig
/root/gnuarm/install/bin/arm-elf-gcc --version      # gcc 4.5.2
```

L'arbre `osmocom-bb-transceiver` (branche jolly/testing, appli `trx`) se compile aussi avec le `arm-none-eabi-gcc` du système, avec les adaptations de `patches/osmocom-bb-transceiver-qemu-bench.patch` (préfixe, édition de liens par gcc, `-fcommon`, `solve_envs.py` en python 3) :

```
cd /opt/GSM/osmocom-bb-transceiver/src/target/firmware
make BOARDS=compal_e88 APPLICATIONS=trx ENVIRONMENTS=highram CROSS_COMPILE=arm-none-eabi-
cp board/compal_e88/trx.highram.{elf,bin} /opt/GSM/firmware/board/compal_e88/
```

Tous les patchs appliqués aux arbres osmocom-bb (mobile, transceiver, burst-ind) sont dans `patches/` d'osmo-operator et appliqués par le Dockerfile et `install_modules/40-patches.sh`.

2.23 /opt/GSM/osmo-operator/doc/c54x_exe-from-scratch-ubuntu.en.md

8836 octets, 171 lignes

2.23.1 c54x_exe from a clean Ubuntu 24.04

Oct 20, 2026 · Bastien Baranoff

2.23.1.1 What this page shows

In about twenty minutes, the emulated TMS320C54x core of `c54x_exe` loads the TI mask ROM of a real Calypso and runs it, without QEMU, without the ARM, without a GSM network.

This is the DSP's hello world: it proves the core executes real TI code. It does not prove the DSP decodes anything, since no burst is injected. The full setup (ARM under QEMU, BTS, bridge, voice call) is described at the end of the page and lives in osmo-operator.

Everything runs as root or with `sudo`; the `/opt/GSM/…` paths are the ones the Makefiles and scripts expect by default.

2.23.1.2 1. Prerequisites (Ubuntu 24.04)

One `apt` line is enough: the compiler, `git`, `curl` for the ROMs, and `libosmocore` (which contains `libosmocoding`, used by `calypso_bsp.c`).

```
sudo apt update
sudo apt install -y build-essential git curl pkg-config libosmocore-dev
sudo mkdir -p /opt/GSM
```

Check that `libosmocoding` is there:

```
pkg-config --modversion libosmocoding
```

If `pkg-config` cannot find it, the `libosmocore` of the Ubuntu repository is too old or incomplete; install `libosmocore-dev` from the Osmocom nightly repository instead (see osmocom.org).

2.23.1.3 2. The qosmo sources

`c54x_exe` does not copy the C54x core: it compiles the qosmo sources directly (`calypso_c54x.c` and its neighbours) with ~120 lines of QEMU header stand-ins. The repository must be on disk, but it does not need to be built.

```
cd /opt/GSM
git clone https://github.com/bbaranoff/qosmo
```

No `configure`, no `make` here. If qosmo is somewhere other than `/opt/GSM/qosmo`, pass `QOSMO=/path` to the `make` of the next step.

2.23.1.4 3. Clone and build c54x_exe

The build takes less than a minute and produces a single binary, `c54x_exe`, at the root of the repository.

```
cd /opt/GSM
git clone https://github.com/bbaranoff/c54x_exe
cd c54x_exe
make QOSMO=/opt/GSM/qosmo \
     CFLAGS="-O3 -g -Wall -Werror=format -Werror=format-extra-args \
             -Wno-unused-function -Wno-unused-variable \
             -Wno-unused-but-set-variable -Wno-sign-compare"
```

This `CFLAGS` is the one of the osmo-operator Dockerfile: it keeps format errors fatal and silences the unused-variable warnings, numerous in the QEMU sources taken as they are. A bare `make` works too but is much noisier.

`make` rebuilds the binary on every call (`.PHONY` target); no `make clean` is needed after a change in qosmo.

Check:

```
./c54x_exe --help | head
```

2.23.1.5 4. The DSP ROMs, from FreeCalypso

The C54x mask ROM is not in the repository: it comes from the dump published by the FreeCalypso project (Mychaela Falconia) on its public FTP. The `rom/fetch-rom.sh` script downloads it and splits it into the 7 sections `--rom-dir` expects.

```
cd /opt/GSM/c54x_exe
export FREECALYPSO_URL=ftp://ftp.freecalypso.org/pub/GSM/Calypso
bash rom/fetch-rom.sh --dest /opt/GSM --dest /opt/GSM/c54x_exe/rom
curl -fsS --retry 2 -o /opt/GSM/calypso_dsp.txt "$FREECALYPSO_URL/dsp-rom-3606-dump.txt"
cp /opt/GSM/calypso_dsp.txt rom/calypso_dsp.txt
```

`3606` is the DSP ROM version of the Calypso in the Motorola C1xx; it is also the value the DSP L1 writes into the API RAM at boot, which will later confirm that the right ROM is running.

After the script, `/opt/GSM` must contain exactly these 7 files:

```
ls -l /opt/GSM/calypso_dsp.*.bin
# calypso_dsp.PROM0.bin  calypso_dsp.PROM1.bin  calypso_dsp.PROM2.bin
# calypso_dsp.PROM3.bin  calypso_dsp.DROM.bin   calypso_dsp.PDROM.bin
# calypso_dsp.Registers.bin
```

If `curl` refuses FTP (some builds come without it), download the dump by hand from the same directory on ftp.freecalypso.org and give the local file to `fetch-rom.sh`.

2.23.1.6 5. First run: the ROM runs

```
cd /opt/GSM/c54x_exe
./c54x_exe --trames 200
```

`--trames 200` runs the DSP for 200 TDMA frames (one frame = 4.615 ms of GSM time), with the default budget of 2300 instructions per frame in standalone mode. Without options only core errors go to stderr; the final summary says how many lines were hidden and which `-v` level shows them.

What you should see, in order:

1. The 7 ROM sections loaded from `/opt/GSM` (or `--rom-dir`).
2. `c54x_reset()`: the ROM restarts at `0xff80`, its reset vector.
3. `BRANCH-TRACE` lines around `PC=0xb41f`: the TI ROM's wait loop, real code executed by the core.
4. A final summary with `a_sch[3] = 0x771a`.

For one line per frame:

```
./c54x_exe --trames 50 --verbeux
```

And for core traces, `-v` to `-vvvvvv` (see `--help`).

2.23.1.7 6. Reading the result

`a_sch[3] = 0x771a` is a constant from one run to the next: the ROM booted, parked itself in IDLE and left this state in its API window. It is the proof that the core executes the ROM faithfully up to that point.

What it does not prove: decoding. The API RAM is blank, no burst arrives, the DSP runs idle. A decoder whose output does not depend on its input does not decode; varying the input requires the full setup of section 8.

Useful reference: the `qemu-calypso` README gives `0xf8d8` for the same word. The difference comes from the input (QEMU provides an API RAM initialised by the ARM, here it is empty), not from a bug: two different inputs, two different constants.

2.23.1.8 7. Common errors

Symptom	Cause	Fix
`make`: `qemu/osdep.h` or `calypso_c54x.c` not found	qosmo is not at `/opt/GSM/qosmo`	`make QOSMO=/path/to/qosmo`
`make`: `osmocom/coding/...` not found	`libosmocore-dev` missing or too old	section 1, Osmocom nightly repository
`fetch-rom.sh` fails or `curl: (1) Protocol "ftp" not supported`	`curl` built without FTP, or a firewall	download by hand, then `fetch-rom.sh` on the local file
`c54x_exe`: ROM not found	the `.bin` files are not in `/opt/GSM`	`--rom-dir /path` or redo section 4
The summary does not show `0x771a`	wrong ROM, or badly split sections	compare `rom/calypso_dsp.txt` with the 3606 dump; redo section 4
`PC=0xb41f` never appears, the core exits with an error	regression in qosmo	`cd /opt/GSM/qosmo && git log -3`, go back to the README's commit

2.23.1.9 8. Next: injecting bursts

For the DSP to really decode, you need the ARM (the osmocom-bb `layer1.highram.elf` firmware under the `qosmo` QEMU), a BTS and the bridge that delivers bursts to it. Two paths:

The complete bench. It installs everything, in the same `/opt/GSM/…` paths as this page:

```
cd /opt/GSM
git clone https://github.com/bbaranoff/osmo-operator
cd osmo-operator
sudo ./install.sh
sudo ./start-direct.sh --dsp
```

`--dsp` replaces the gr-gsm layer 1 with `c54x_exe`; `--menu` asks the same question interactively. The osmo-operator wiki describes the options, the ports and the numbering plan.

Executable by executable. To watch each step produce its log: the DSP (`c54x_exe --arm -v`), then QEMU with `CALYPSO_DSP_EXTERN=1`, then `osmocon`, then `mobile`. The sequence, with the expected log lines at each step, is in LAUNCH.md and the c54x_exe README.

From there, `a_sch` changes with the input, and the question "does it decode?" has a measurable answer.

2.23.1.10 9. The Calypso firmware: the arm-elf toolchain (gnuarm)

The osmocom-bb firmware (`layer1.highram.elf`, `trx.highram.elf`…) is built with the historical `arm-elf` toolchain (gcc 4.5.2, binutils 2.21.1, newlib 1.19) from the gnu-arm-installer, installed in `/root/gnuarm/install`. On Ubuntu 24.04 its `cc1` asks for `libmpfr.so.4`, which the system no longer ships (it has `libmpfr.so.6`); a symlink is enough:

```
ln -sf /usr/lib/x86_64-linux-gnu/libmpfr.so.6 /usr/local/lib/libmpfr.so.4 && ldconfig
/root/gnuarm/install/bin/arm-elf-gcc --version      # gcc 4.5.2
```

The `osmocom-bb-transceiver` tree (jolly/testing branch, `trx` application) also builds with the system's `arm-none-eabi-gcc`, with the adaptations of `patches/osmocom-bb-transceiver-qemu-bench.patch` (prefix, linking through gcc, `-fcommon`, `solve_envs.py` ported to Python 3):

```
cd /opt/GSM/osmocom-bb-transceiver/src/target/firmware
make BOARDS=compal_e88 APPLICATIONS=trx ENVIRONMENTS=highram CROSS_COMPILE=arm-none-eabi-
cp board/compal_e88/trx.highram.{elf,bin} /opt/GSM/firmware/board/compal_e88/
```

Every patch applied to the osmocom-bb trees (mobile, transceiver, burst-ind) lives in `patches/` of osmo-operator and is applied by the Dockerfile and `install_modules/40-patches.sh`.

2.24 /opt/GSM/osmo-operator/doc/install-from-scratch.en-notes.md

4257 octets, 75 lignes

2.24.1 English notes per Dockerfile stage (read by tools/dockerfile-to-install-md.sh –lang en)

One section per stage name (`## <stage>`); the text is inserted under the stage heading of the English install guide. The Dockerfile comments themselves are in French and are reproduced only in the French guide.

2.24.1.1 base

Ubuntu 24.04 (noble) and everything that is long to install and rarely changes: the whole apt package list (one list for the entire project, `apt-fast` to download it), Wireshark allowed for non-root capture, the `osmo-deb` helper that caches every compiled tree as a `.deb` in `/var/cache/osmo-debs`, the Asterisk and ALSA configuration files, the `osmocom` system user and the `/opt/GSM` tree that every script expects. On a bare machine the `.deb` cache is empty: every `osmo-deb install` fails and the build branch runs.

2.24.1.2 osmo-core

The Osmocom stack compiled from source, in dependency order and at pinned versions (libosmocore, libosmo-netif, libosmo-abis, libosmo-sigtran, libsmpp34, libgtpnl, osmo-hlr, osmo-mgw, osmo-ggsn, osmo-sgsn, osmo-msc, osmo-bsc, osmo-trx, osmo-bts, osmo-pcu, osmo-sip-connector, libosmo-gprs), then the project patches kept in `patches/`: osmo-trx IPC device aligned on the TDMA frame and a per-RA RACH modulation table, and the "force RAND" toy patches for osmo-bts, osmo-hlr and osmo-msc (the RAND of the authentication challenge is forced only when the `RAND` environment variable is set). The stage ends with osmo-gapk (voice codecs), the GSUP SMSC protocol library, the SMS coding utilities and libosmo-dsp.

2.24.1.3 bb

osmocom-bb (the `mobile` application, `osmocon`, layer23) with the patches from `patches/osmocom-bb-*` (clock generator frame tolerance, PLMN handling, SIM PLMN from BCD), the Calypso firmware images (`layer1.highram.elf` and friends) and the two gcc flavours the old trees need (gcc-9 selected through `update-alternatives` while building, gcc-11 restored afterwards). It also builds the `osmocom-bb-transceiver` tree (Sylvain Munaut's jolly/testing branch: the `trx` firmware application and the host `transceiver` program that turns the phone into a BTS transceiver) with `patches/osmocom-bb-transceiver-qemu-bench.patch`, and the `burst-ind` tree.

2.24.1.4 qemu

The single QEMU tree of the project, `/opt/GSM/qosmo`: a QEMU fork with the Calypso machine (ARM core, TPU/TSP, DSP API RAM, LCD and keypad emulation) built with `--enable-l1-grgsm`, the gr-gsm layer 1 shunt. The C54x DSP itself is no longer run inside QEMU: see the `l1` stage.

2.24.1.5 l1

The two layer-1 engines that run outside QEMU: `c54x_exe` (the TMS320C54x core emulated from the qosmo sources, loading the real TI mask ROM, see the appendix) and `grgsm_exe` (the gr-gsm based layer 1). The DSP ROM images come from the FreeCalypso dump and are split into the seven sections `c54x_exe` reads.

2.24.1.6 grgsm-venv

GNU Radio 3.10, gr-osmosdr and gr-gsm installed in the Python virtual environment `/root/.env`, with the project patches for gr-gsm (optional Qt GUI import, receiver publishing BSIC and frame number). The venv is activated from root's profile at the end of the final stage.

2.24.1.7 node

Node 22 and the web dashboard `osmo-egprs-web` (the operations console served as a systemd unit), plus its npm dependencies.

2.24.1.8 lte

The 4G side of the bench: srsRAN_4G built with the ZeroMQ radio (no SDR needed) and Open5GS, both under `/opt/LTE`, with their configuration files.

2.24.1.9 osmocom-nitb

The assembly stage, the image that is actually published. Nothing is compiled here: the `.deb` packages produced by the previous stages are installed (`dpkg -i`), then the project files are laid out: the `/etc/osmocom` configurations, the scripts under `/usr/local/bin`, the systemd units, the PulseAudio system daemon configuration (GSM audio), the runtime directories, the shell prompt, the desktop pieces and the `/opt/GSM` helper scripts. On a single machine the `COPY --from=<stage>` instructions are no-ops: everything is already installed where the previous stages put it. The container entrypoint and stop signal are Docker metadata only.

2.24.1.10 debs

A `scratch` stage holding only `/var/cache/osmo-debs`, used by the CI to extract the compiled packages without materialising the full image. Nothing to do on a machine.

2.25 /opt/GSM/osmo-operator/doc/install-from-scratch.en.md

46532 octets, 1144 lignes

2.25.1 Installing from scratch (generated from the Dockerfile)

> Generated by `tools/dockerfile-to-install-md.sh --lang en` on 2026-10-20 from `Dockerfile`. Do not edit by hand: fix the Dockerfile, the script or `doc/install-from-scratch.en-notes.md`, then regenerate. The French version (`doc/install-from-scratch.md`) carries the full rationale written in the Dockerfile comments; this one keeps the commands and a short English note per stage.

2.25.1.1 How to read this page

The osmo-operator Dockerfile is the complete, tested description of the installation. This page replays it step by step on a bare machine (Ubuntu 24.04, as root), without Docker:

- every `FROM ... AS <name>` of the Dockerfile is a chapter; on a single machine the stages simply follow each other and the `COPY --from=<stage>` instructions are not needed (everything is already in place);
- every `RUN` is a bash block to paste as is into a root shell, in order;
- every `ARG`/`ENV` becomes an `export` to set before the blocks that follow (in Docker an `ARG` does not survive a `FROM`; here it just stays in the shell);
- every `COPY` becomes a `cp` from a checkout of the osmo-operator repository (`$REPO`, `/opt/GSM/osmo-operator` by default).

The `.deb` package cache (`osmo-deb install <name> <version>`) is an optimisation of the Docker build: without the cache the command fails and the `if ! osmo-deb install ...; then ... fi` branch builds from source, which is exactly what a bare machine needs. `osmo-deb pack` then wraps the result (`make install`); without `/var/cache/osmo-debs` it just installs.

```
export REPO=/opt/GSM/osmo-operator      # checkout of the repository (git clone
↪  https://github.com/bbaranoff/osmo-operator)
export ROOT=/opt/GSM
mkdir -p "$ROOT"
```

2.25.1.2 Table of contents

1. base (from `ubuntu:24.04`)
2. osmo-core (from `base`)
3. bb (from `osmo-core`)
4. qemu (from `base`)
5. l1 (from `qemu`)
6. grgsm-venv (from `osmo-core`)
7. node (from `base`)
8. lte (from `base`)
9. osmocom-nitb (from `base`)
10. debs (from `scratch`)

2.25.1.3 Stage 1 : base

Ubuntu 24.04 (noble) and everything that is long to install and rarely changes: the whole apt package list (one list for the entire project, `apt-fast` to download it), Wireshark allowed for non-root capture, the `osmo-deb` helper that caches every compiled tree as a `.deb` in `/var/cache/osmo-debs`, the Asterisk and ALSA configuration files, the `osmocom` system user and the `/opt/GSM` tree that every script expects. On a bare machine the `.deb` cache is empty: every `osmo-deb install` fails and the build branch runs.

In Docker: `FROM ubuntu:24.04`. *On a machine, carry on in the same system.*

```
export DEBIAN_FRONTEND=noninteractive


export ROOT=/opt/GSM


export container=docker
export OSMO_IMAGE=1
export PKG_CONFIG_PATH=/usr/local/lib/pkgconfig
export LD_LIBRARY_PATH=/usr/local/lib
export GIT_TERMINAL_PROMPT=0


mkdir -p "$(dirname /usr/local/sbin/apt-fast-install)" && cp -r "$REPO"/packaging/apt-fast-install.sh
 ↪  /usr/local/sbin/apt-fast-install


export OSMO_UBUNTU_MIRROR=""


chmod 755 /usr/local/sbin/apt-fast-install && apt-fast-install \
    && rm -rf /var/lib/apt/lists/*


export DEBIAN_FRONTEND=noninteractive


echo 'wireshark-common wireshark-common/install-setuid boolean true' | debconf-set-selections


if [ -f /etc/apt/sources.list.d/ubuntu.sources ]; then \
        sed -i 's/^Types: deb$/Types: deb deb-src/' /etc/apt/sources.list.d/ubuntu.sources; \
    else \
        sed -nE 's|^deb (http\S+) (\S+) .*|deb-src \1 \2 main restricted universe multiverse|p' \
            /etc/apt/sources.list | sort -u > /etc/apt/sources.list.d/deb-src.list; \
    fi


apt-fast update && apt-fast install -y --no-install-recommends \
    # Outils de build
    build-essential git gcc g++ make cmake autoconf automake libtool pkg-config wget curl \
    # Dépendances Osmocom Core & Network
    libtalloc-dev libpcsclite-dev libsctp-dev libmnl-dev liburing-dev asterisk-moh* \
    libdbi-dev libdbd-sqlite3 libsqlite3-dev sqlite3 libc-ares-dev libgnutls28-dev \
    # Audio, Radio & SIP
    libortp-dev libfftw3-dev libusb-1.0-0-dev libsofia-sip-ua-dev libsofia-sip-ua-glib-dev \
    # Python & Outils système
    python3 python3-dev python3-scapy ca-certificates tmux systemd systemd-sysv \
    # Debug — gdb-multiarch pour attacher au gdb-stub QEMU (ARM Calypso)
    gdb-multiarch \
    # ALSA — requis par osmo-gapk pour l'I/O audio matériel
    # (libasound2t64 : nom noble de libasound2, transition time_t 64 bits)
    libasound2-dev libasound2t64 alsa-utils \
    # libgsm — codec GSM-FR natif (accélère gapk en mode gsmfr)
    libgsm1-dev libgsm1 \
    iptables iproute2 asterisk ffmpeg \
    # Sync build-iso : psmisc (pkill/killall, cleanup) + pulseaudio (chaîne audio gapk/parec)
```

```
    psmisc pulseaudio pulseaudio-utils binutils-arm-none-eabi \
    # Toolchains alternatives : osmocom-bb jolly/testing et fixeria/burst_ind ne
    # compilent qu'avec gcc-9 ; gcc-11 reste le compilateur par defaut du reste
    # (gcc-13 est celui de noble : il compile la pile Osmocom en tete de fichier,
    # les alternatives ci-dessous ne sont posees qu apres).
    # (Ex-bloc apt dedie, supprime : les `update-alternatives` plus bas echouaient
    #  « alternative path /usr/bin/gcc-9 doesn't exist » sans ces paquets.)
    gcc-9 g++-9 gcc-11 g++-11 \
    # log4cpp : etait installe a part avec le build-dep gnuradio (gr-gsm).
    liblog4cpp5-dev \
    # QEMU Calypso (bbaranoff/qosmO) : venv + numpy/scipy pour l'outillage
    # DSP, glib/pixman/slirp pour la cible arm-softmmu, ninja pour meson, socat
    # pour les PTY de scripts/run.sh. (Ex-bloc apt-get juste avant le build QEMU.)
    python3-venv python3-pip python3-numpy python3-scipy \
    libglib2.0-dev libpixman-1-dev libslirp-dev socat ninja-build \
    # Ex-Dockerfile.run — outillage d'exploitation du conteneur :
    #   telnet   -> repli VTY (run_modules/21-abonnes-hlr.sh) + transport dashboard
    #   whiptail -> menus de tools/vty-menu.sh et de start-docker.sh
    #   xz-utils -> `tar -xJf` du tarball Node 22 (bloc dashboard, en fin de fichier)
    #   libasound2-plugins -> greffon ALSA->PulseAudio, EXIGE par configs/asound.conf
    #        (`type pulse` sur gsm_out, gsm_in ET pcm.!default) : sans lui, toute
    #        ouverture ALSA (gapk, mobile) echoue et l'audio est mort des 2 cotes
    #   tshark/tcpdump/libcap2-bin -> capture ; libcap2-bin fournit le setcap ci-dessous
    telnet nano whiptail xz-utils libasound2-plugins \
    tcpdump tshark libcap2-bin \
    # Deps GNU Radio 3.10 (gr-gsm, venv /root/.env plus bas) : meme resolution
    # apt que le reste, pas de second RUN apt. (Le && ferme la liste pour
    # l extracteur de install_modules/10-deps.sh.)
    && apt-fast build-dep -y gnuradio \
    && rm -rf /var/lib/apt/lists/*


setcap cap_net_raw,cap_net_admin+eip "$(command -v dumpcap)"


export OSMO_DEB_REFRESH=0


mkdir -p "$(dirname /usr/local/sbin/osmo-deb)" && cp -r "$REPO"/packaging/osmo-deb.sh /usr/local/sbin/osmo-deb


mkdir -p /var/cache/osmo-debs/ && cp -r "$REPO"/.deb-cache/ /var/cache/osmo-debs/


chmod 755 /usr/local/sbin/osmo-deb && osmo-deb list || true


git config --global http.postBuffer 524288000
```

Docker metadata, no equivalent on a machine: `SHELL ["/bin/bash", "-c"]`.

```
mkdir -p /etc/asterisk/ && cp -r "$REPO"/configs/*conf /etc/asterisk/


mkdir -p "$(dirname /etc/asound.conf)" && cp -r "$REPO"/configs/asound.conf /etc/asound.conf


cd ${ROOT}


groupadd osmocom && useradd -r -g osmocom -s /sbin/nologin -d /var/lib/osmocom osmocom && \
    mkdir -p /var/lib/osmocom && chown osmocom:osmocom /var/lib/osmocom
```

2.25.1.4 Stage 2 : osmo-core

The Osmocom stack compiled from source, in dependency order and at pinned versions (libosmocore, libosmo-netif, libosmo-abis, libosmo-sigtran, libsmpp34, libgtpnl, osmo-hlr, osmo-mgw, osmo-ggsn, osmo-sgsn, osmo-msc, osmo-bsc, osmo-trx, osmo-bts, osmo-pcu, osmo-sip-connector, libosmo-gprs), then the project patches kept in `patches/`: osmo-trx IPC device aligned on the TDMA frame and a per-RA RACH modulation table, and the "force RAND" toy patches for osmo-bts, osmo-hlr and osmo-msc (the RAND of the authentication challenge is forced only when the `RAND` environment variable is set). The stage ends with osmo-gapk (voice codecs), the GSUP SMSC protocol library, the SMS coding utilities and libosmo-dsp.

In Docker: `FROM base`. On a machine, carry on in the same system.

```
export ROOT=/opt/GSM


export OSMO_DEB_REFRESH=0


for repo in \
    libosmocore:1.12.1 \
    libosmo-netif:1.7.0 \
    libosmo-abis:2.1.0 \
    libosmo-sigtran:2.2.1 \
    libsmpp34:1.14.5 \
    libgtpnl:1.3.3 \
    osmo-hlr:1.9.2 \
    osmo-mgw:1.15.0 \
    osmo-ggsn:1.14.0 \
    osmo-sgsn:1.13.1 \
    osmo-msc:1.15.0 \
    osmo-bsc:1.14.0 \
    osmo-trx:1.7.2 \
    osmo-bts:1.10.0 \
    osmo-pcu:1.5.2 \
    osmo-sip-connector:1.7.2 \
    libosmo-gprs:0.2.1; \
    do \
    name=$(echo $repo | cut -d: -f1) && \
    version=$(echo $repo | cut -d: -f2) && \
    \
    if [[ "$name" =~ ^libosmo ]]; then \
        GIT_URL="https://gitea.osmocom.org/osmocom/$name"; \
    else \
        GIT_URL="https://gitea.osmocom.org/cellular-infrastructure/$name"; \
    fi && \
    \
    # Le paquet du cache d abord : s il est la, ni clone ni compilation.
    if osmo-deb install "$name" "$version"; then continue; fi && \
    \
    cd ${ROOT} && \
    git clone "$GIT_URL" && cd "$name" && \
    git checkout "$version" && \
    \
    autoreconf -fi && \
    EXTRA_FLAGS="" && \
    if [ "$name" = "libosmo-abis" ]; then EXTRA_FLAGS="--disable-dahdi"; fi && \
    if [ "$name" = "osmo-msc" ]; then EXTRA_FLAGS="--enable-smpp"; fi && \
    if [ "$name" = "osmo-mgw" ]; then EXTRA_FLAGS="--enable-alsa"; fi && \
    if [ "$name" = "osmo-trx" ]; then EXTRA_FLAGS="--with-ipc"; fi && \
    if [ "$name" = "osmo-bts" ]; then EXTRA_FLAGS="--enable-virtual --enable-trx"; fi && \
    if [ "$name" = "osmo-ggsn" ]; then EXTRA_FLAGS="--enable-gtp-linux"; fi && \
    \
    ./configure $EXTRA_FLAGS && \
    make -j$(nproc) && \
    # make install sous DESTDIR -> .deb dans le cache -> dpkg -i dans la racine
    osmo-deb pack "$name" "$version" make install && \
```

```
    ldconfig \
    || { echo "ECHEC build osmocom: $name"; exit 1; }; \
    done


mkdir -p "$(dirname /tmp/osmo-trx-ipc-ts-frame-align.patch)" && cp -r
 ↪  "$REPO"/patches/osmo-trx-ipc-ts-frame-align.patch /tmp/osmo-trx-ipc-ts-frame-align.patch


mkdir -p "$(dirname /tmp/osmo-trx-rach-per-ra-table.patch)" && cp -r
 ↪  "$REPO"/patches/osmo-trx-rach-per-ra-table.patch /tmp/osmo-trx-rach-per-ra-table.patch


if ! osmo-deb install osmo-trx 1.7.2+ipc; then \
      { [ -d ${ROOT}/osmo-trx ] || { cd ${ROOT} \
          && git clone https://gitea.osmocom.org/cellular-infrastructure/osmo-trx \
          && git -C osmo-trx checkout 1.7.2; }; } \
      && git -C ${ROOT}/osmo-trx apply /tmp/osmo-trx-ipc-ts-frame-align.patch \
      && git -C ${ROOT}/osmo-trx apply /tmp/osmo-trx-rach-per-ra-table.patch \
      && cd ${ROOT}/osmo-trx \
      && autoreconf -fi \
      && ./configure --with-ipc \
      && make -j$(nproc) \
      && osmo-deb pack osmo-trx 1.7.2+ipc make install \
      && ldconfig; \
    fi


mkdir -p "$(dirname /tmp/osmo-bts-force-rand-toy.patch)" && cp -r "$REPO"/patches/osmo-bts-force-rand-toy.patch
 ↪  /tmp/osmo-bts-force-rand-toy.patch


if ! osmo-deb install osmo-bts 1.10.0+rand; then \
      { [ -d ${ROOT}/osmo-bts ] || { cd ${ROOT} \
          && git clone https://gitea.osmocom.org/cellular-infrastructure/osmo-bts \
          && git -C osmo-bts checkout 1.10.0; }; } \
      && git -C ${ROOT}/osmo-bts apply /tmp/osmo-bts-force-rand-toy.patch \
      && cd ${ROOT}/osmo-bts \
      && autoreconf -fi \
      && ./configure --enable-virtual --enable-trx \
      && make -j$(nproc) \
      && osmo-deb pack osmo-bts 1.10.0+rand make install \
      && ldconfig; \
    fi


mkdir -p "$(dirname /tmp/osmo-hlr-force-rand-toy.patch)" && cp -r "$REPO"/patches/osmo-hlr-force-rand-toy.patch
 ↪  /tmp/osmo-hlr-force-rand-toy.patch


if ! osmo-deb install osmo-hlr 1.9.2+rand; then \
      { [ -d ${ROOT}/osmo-hlr ] || { cd ${ROOT} \
          && git clone https://gitea.osmocom.org/cellular-infrastructure/osmo-hlr \
          && git -C osmo-hlr checkout 1.9.2; }; } \
      && git -C ${ROOT}/osmo-hlr apply /tmp/osmo-hlr-force-rand-toy.patch \
      && cd ${ROOT}/osmo-hlr \
      && autoreconf -fi \
      && ./configure \
      && make -j$(nproc) \
      && osmo-deb pack osmo-hlr 1.9.2+rand make install \
      && ldconfig; \
    fi


mkdir -p "$(dirname /tmp/osmo-msc-force-rand-toy.patch)" && cp -r "$REPO"/patches/osmo-msc-force-rand-toy.patch
 ↪  /tmp/osmo-msc-force-rand-toy.patch
```

```
if ! osmo-deb install osmo-msc 1.15.0+rand; then \
      { [ -d ${ROOT}/osmo-msc ] || { cd ${ROOT} \
          && git clone https://gitea.osmocom.org/cellular-infrastructure/osmo-msc \
          && git -C osmo-msc checkout 1.15.0; }; } \
      && git -C ${ROOT}/osmo-msc apply /tmp/osmo-msc-force-rand-toy.patch \
      && cd ${ROOT}/osmo-msc \
      && autoreconf -fi \
      && ./configure --enable-smpp \
      && make -j$(nproc) \
      && osmo-deb pack osmo-msc 1.15.0+rand make install \
      && ldconfig; \
    fi


if ! osmo-deb install osmo-gapk 0.git; then \
      cd ${ROOT} && \
      git clone https://gitea.osmocom.org/osmocom/gapk osmo-gapk && \
      cd osmo-gapk && \
      autoreconf -fi && \
      ./configure --enable-alsa && \
      make -j$(nproc) && \
      osmo-deb pack osmo-gapk 0.git make install && \
      ldconfig; \
    fi


if ! osmo-deb install gsup-smsc-proto 0.git; then \
      cd ${ROOT} && \
      git clone https://gitea.osmocom.org/themwi/gsup-smsc-proto && \
      cd gsup-smsc-proto && \
      ./configure --with-osmo=/usr/local && \
      make -j$(nproc) && \
      osmo-deb pack gsup-smsc-proto 0.git \
        sh -c 'for i in daemon sendmt; do make -C "$i" DESTDIR="$DESTDIR" install || exit 1; done' && \
      ldconfig; \
    fi


if ! osmo-deb install sms-coding-utils 0.r1; then \
      cd ${ROOT} && \
      wget -q https://www.freecalypso.org/pub/GSM/FreeCalypso/sms-coding-utils-latest.tar.bz2 && \
      tar xf sms-coding-utils-latest.tar.bz2 && \
      cd sms-coding-utils-r1 && \
      ./configure && \
      make -j$(nproc) && \
      osmo-deb pack sms-coding-utils 0.r1 \
        sh -c 'mkdir -p "$DESTDIR/usr/local/bin" && make install DESTDIR="$DESTDIR"'; \
    fi


if ! osmo-deb install libosmo-dsp 0.git; then \
      cd /opt/GSM \
      && git clone https://gitea.osmocom.org/sdr/libosmo-dsp.git \
      && cd libosmo-dsp \
      && autoreconf -fi \
      && ./configure \
      && make -j$(nproc) \
      && osmo-deb pack libosmo-dsp 0.git make install \
      && ldconfig; \
    fi
```

2.25.1.5 Stage 3 : bb

osmocom-bb (the `mobile` application, `osmocon`, layer23) with the patches from `patches/osmocom-bb-*` (clock generator frame tolerance, PLMN handling, SIM PLMN from BCD), the Calypso firmware images (`layer1.highram.elf` and friends) and the two gcc flavours the old trees need (gcc-9 selected through `update-alternatives` while building, gcc-11 restored afterwards). It also builds the `osmocom-bb-transceiver` tree (Sylvain Munaut's jolly/testing branch: the `trx` firmware application and the host `transceiver` program that turns the phone into a BTS transceiver) with `patches/osmocom-bb-transceiver-qemu-bench.patch`, and the `burst-ind` tree.

In Docker: `FROM osmo-core`. On a machine, carry on in the same system.

```
export ROOT=/opt/GSM


export OSMO_DEB_REFRESH=0


mkdir -p "$(dirname /tmp/osmocom-bb-clck-gen-frame-tolerance.patch)" && cp -r
 ↪  "$REPO"/patches/osmocom-bb-clck-gen-frame-tolerance.patch /tmp/osmocom-bb-clck-gen-frame-tolerance.patch


mkdir -p "$(dirname /tmp/osmocom-bb-plmn-a6-reinsert.patch)" && cp -r
 ↪  "$REPO"/patches/osmocom-bb-plmn-a6-reinsert.patch /tmp/osmocom-bb-plmn-a6-reinsert.patch


mkdir -p "$(dirname /tmp/osmocom-bb-sim-plmn-from-bcd.patch)" && cp -r
 ↪  "$REPO"/patches/osmocom-bb-sim-plmn-from-bcd.patch /tmp/osmocom-bb-sim-plmn-from-bcd.patch


if ! osmo-deb install osmocom-bb 0.git; then \
      cd ${ROOT} && \
      git clone https://gitea.osmocom.org/phone-side/osmocom-bb && \
      git -C ${ROOT}/osmocom-bb apply /tmp/osmocom-bb-clck-gen-frame-tolerance.patch && \
      git -C ${ROOT}/osmocom-bb apply /tmp/osmocom-bb-plmn-a6-reinsert.patch && \
      git -C ${ROOT}/osmocom-bb apply /tmp/osmocom-bb-sim-plmn-from-bcd.patch && \
      cd osmocom-bb/src && \
      # Build complet : firmware (layer1.bin/.elf pour Calypso) + outils host
      # (mobile, trxcon, virtphy, ccch_scan). Le firmware est nécessaire pour
      # le mode PHY_MODE=qemu où QEMU émule un Calypso et exécute layer1.
      make nofirmware && \
      osmo-deb snapshot osmocom-bb 0.git ${ROOT}/osmocom-bb; \
    fi


if ! osmo-deb install calypso-firmware 0.git; then \
      GIT_TERMINAL_PROMPT=0 git clone --depth 1 \
        https://github.com/bbaranoff/firmware /opt/GSM/firmware \
      && osmo-deb snapshot calypso-firmware 0.git /opt/GSM/firmware; \
    fi


update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-9 90 \
       --slave /usr/bin/g++ g++ /usr/bin/g++-9 \
    && update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-11 110 \
       --slave /usr/bin/g++ g++ /usr/bin/g++-11


update-alternatives --set gcc /usr/bin/gcc-9


mkdir -p "$(dirname /tmp/osmocom-bb-transceiver-qemu-bench.patch)" && cp -r
 ↪  "$REPO"/patches/osmocom-bb-transceiver-qemu-bench.patch /tmp/osmocom-bb-transceiver-qemu-bench.patch
```

```
if ! osmo-deb install osmocom-bb-transceiver 0.git; then \
      git clone --branch jolly/testing --depth 1 \
        https://gitea.osmocom.org/phone-side/osmocom-bb.git \
        /opt/GSM/osmocom-bb-transceiver \
      && git -C /opt/GSM/osmocom-bb-transceiver apply /tmp/osmocom-bb-transceiver-qemu-bench.patch \
      && cd /opt/GSM/osmocom-bb-transceiver/src \
      && make HOST_layer23_CONFARGS=--enable-transceiver nofirmware -j$(nproc) \
      && cp /opt/GSM/osmocom-bb-transceiver/src/host/layer23/src/transceiver/transceiver \
         /usr/local/bin/transceiver \
      && osmo-deb snapshot osmocom-bb-transceiver 0.git /usr/local/bin/transceiver \
            /opt/GSM/osmocom-bb-transceiver; \
    fi


if ! osmo-deb install osmocom-bb-burst-ind 0.git; then \
      git clone --branch fixeria/burst_ind --depth 1 \
        https://gitea.osmocom.org/phone-side/osmocom-bb.git \
        /opt/GSM/osmocom-bb-burst_ind \
      && cd /opt/GSM/osmocom-bb-burst_ind/src \
      && make nofirmware -j$(nproc) \
      && cp /opt/GSM/osmocom-bb-burst_ind/src/host/layer23/src/misc/ccch_scan \
         /usr/local/bin/ccch_scan \
      && { cp /opt/GSM/osmocom-bb-burst_ind/src/host/layer23/src/misc/bcch_scan \
         /usr/local/bin/bcch_scan 2>/dev/null || true; } \
      && { cp /opt/GSM/osmocom-bb-burst_ind/src/host/layer23/src/misc/cell_log \
         /usr/local/bin/cell_log 2>/dev/null || true; } \
      && osmo-deb snapshot osmocom-bb-burst-ind 0.git /opt/GSM/osmocom-bb-burst_ind \
           $(ls /usr/local/bin/ccch_scan /usr/local/bin/bcch_scan /usr/local/bin/cell_log 2>/dev/null); \
    fi


update-alternatives --set gcc /usr/bin/gcc-11
```

2.25.1.6 Stage 4 : qemu

The single QEMU tree of the project, `/opt/GSM/qosmo`: a QEMU fork with the Calypso machine (ARM core, TPU/TSP, DSP API RAM, LCD and keypad emulation) built with `--enable-l1-grgsm`, the gr-gsm layer 1 shunt. The C54x DSP itself is no longer run inside QEMU: see the `l1` stage.

In Docker: `FROM base`. On a machine, carry on in the same system.

```
export ROOT=/opt/GSM


export OSMO_DEB_REFRESH=0


if ! osmo-deb install qosmo 0.git; then \
      git clone https://github.com/bbaranoff/qosmO /opt/GSM/qosmo \
      && cd /opt/GSM/qosmo \
      && python3 -m venv /root/.venv-qemu \
      && . /root/.venv-qemu/bin/activate \
      && pip install --no-cache-dir tomli \
      && mkdir -p build && cd build \
      && ../configure --target-list=arm-softmmu --enable-l1-grgsm \
            --prefix=/opt/GSM/qemu-install --disable-werror --disable-docs \
      && make -j$(nproc) \
      && make install \
      && cp /opt/GSM/qemu-install/bin/qemu-system-arm /usr/local/bin/qemu-system-arm \
      && cp /opt/GSM/qemu-install/bin/qosmo /usr/local/bin/qosmo \
      && osmo-deb snapshot qosmo 0.git /opt/GSM/qosmo /opt/GSM/qemu-install \
            /root/.venv-qemu /usr/local/bin/qemu-system-arm /usr/local/bin/qosmo; \
    fi
```

2.25.1.7 Stage 5 : l1

The two layer-1 engines that run outside QEMU: `c54x_exe` (the TMS320C54x core emulated from the qosmo sources, loading the real TI mask ROM, see the appendix) and `grgsm_exe` (the gr-gsm based layer 1). The DSP ROM images come from the FreeCalypso dump and are split into the seven sections `c54x_exe` reads.

In Docker: `FROM qemu`. On a machine, carry on in the same system.

```
export ROOT=/opt/GSM
```

```
export OSMO_DEB_REFRESH=0
```

Copy between Docker stages (`--from=osmo-core /usr/local /usr/local`): on a single machine `/usr/local` is already in place.

```
ldconfig
```

```
export FREECALYPSO_URL=ftp://ftp.freecalypso.org/pub/GSM/Calypso
```

```
if ! osmo-deb install c54x-exe 0.git; then \
      git clone https://github.com/bbaranoff/c54x_exe /opt/GSM/c54x_exe \
      && cd /opt/GSM/c54x_exe \
      && make QOSMO=/opt/GSM/qosmo \
             CFLAGS="-O3 -g -Wall -Werror=format -Werror=format-extra-args -Wno-unused-function
 ↪  -Wno-unused-variable -Wno-unused-but-set-variable -Wno-sign-compare" \
      && FREECALYPSO_URL="$FREECALYPSO_URL" bash rom/fetch-rom.sh --dest /opt/GSM --dest /opt/GSM/c54x_exe/rom \
      && curl -fsS --retry 2 -o /opt/GSM/calypso_dsp.txt "$FREECALYPSO_URL/dsp-rom-3606-dump.txt" \
      && cp /opt/GSM/calypso_dsp.txt rom/calypso_dsp.txt \
      && osmo-deb snapshot c54x-exe 0.git /opt/GSM/c54x_exe \
             /opt/GSM/calypso_dsp.PROM0.bin /opt/GSM/calypso_dsp.PROM1.bin /opt/GSM/calypso_dsp.PROM2.bin \
             /opt/GSM/calypso_dsp.PROM3.bin /opt/GSM/calypso_dsp.DROM.bin /opt/GSM/calypso_dsp.PDROM.bin \
             /opt/GSM/calypso_dsp.Registers.bin /opt/GSM/calypso_dsp.txt; \
    fi
```

```
if ! osmo-deb install grgsm-exe 0.git; then \
      git clone https://github.com/bbaranoff/grgsm_exE /opt/GSM/grgsm_exe \
      && cd /opt/GSM/grgsm_exe \
      && make QOSMO=/opt/GSM/qosmo \
      && osmo-deb snapshot grgsm-exe 0.git /opt/GSM/grgsm_exe; \
    fi
```

2.25.1.8 Stage 6 : grgsm-venv

GNU Radio 3.10, gr-osmosdr and gr-gsm installed in the Python virtual environment `/root/.env`, with the project patches for gr-gsm (optional Qt GUI import, receiver publishing BSIC and frame number). The venv is activated from root's profile at the end of the final stage.

In Docker: `FROM osmo-core`. On a machine, carry on in the same system.

```
export ROOT=/opt/GSM
```

```
export OSMO_DEB_REFRESH=0
```

```
mkdir -p "$(dirname /tmp/grgsm-receiver-publish-bsic-fn.patch)" && cp -r
 ↪  "$REPO"/patches/grgsm-receiver-publish-bsic-fn.patch /tmp/grgsm-receiver-publish-bsic-fn.patch
```

```
if ! osmo-deb install grgsm-venv 0.git; then \
      curl -fsSL
       ↪  https://gist.githubusercontent.com/bbaranoff/3683811057933af0954b661821e950d1/raw/fcdb4092483ec383440b67fc002db0c1583
       ↪  | bash \
      && git -C /opt/GSM/gr-gsm apply /tmp/grgsm-receiver-publish-bsic-fn.patch \
      && cd /opt/GSM/gr-gsm/build \
      && make -j"$(nproc)" \
      && make install \
      && . ~/.env/bin/activate && pip install matplotlib \
      && osmo-deb snapshot grgsm-venv 0.git /root/.env /etc/ld.so.conf.d/gnuradio.conf; \
    fi && ldconfig
```

2.25.1.9 Stage 7 : node

Node 22 and the web dashboard `osmo-egprs-web` (the operations console served as a systemd unit), plus its npm dependencies.

In Docker: `FROM base`. *On a machine, carry on in the same system.*

```
export ROOT=/opt/GSM
```

```
export OSMO_DEB_REFRESH=0
```

```
if [ ! -d "/opt/GSM/osmo-egprs-web/" ]; then \
      mkdir -p /opt/GSM && \
      git clone -b main https://github.com/bbaranoff/osmo-egprs-web /opt/GSM/osmo-egprs-web; \
    fi
```

```
export NODE_VERSION=v22.23.2
```

```
case "$(uname -m)" in x86_64) _na=x64 ;; aarch64) _na=arm64 ;; *) echo "arch node inconnue: $(uname -m)" >&2; exit
 ↪  1 ;; esac && \
    curl -fsSL "https://nodejs.org/dist/${NODE_VERSION}/node-${NODE_VERSION}-linux-${_na}.tar.xz" \
        -o /tmp/node.tar.xz && \
    mkdir -p /opt/node && \
    tar -xJf /tmp/node.tar.xz -C /opt/node --strip-components=1 && \
    rm -f /tmp/node.tar.xz && \
    ln -sf /opt/node/bin/node /usr/local/bin/node && \
    ln -sf /opt/node/bin/npm  /usr/local/bin/npm && \
    ln -sf /opt/node/bin/npx  /usr/local/bin/npx && \
    node --version
```

```
mkdir -p "$(dirname /etc/systemd/system/osmo-egprs-web.service)" && cp -r "$REPO"/services/osmo-egprs-web.service
 ↪  /etc/systemd/system/osmo-egprs-web.service
```

```
ln -sf /etc/systemd/system/osmo-egprs-web.service \
        /etc/systemd/system/multi-user.target.wants/osmo-egprs-web.service
```

```
mkdir -p "$(dirname /opt/GSM/osmo-egprs-web/osmo-egprs-web.service)" && cp -r
 ↪  "$REPO"/services/osmo-egprs-web.service /opt/GSM/osmo-egprs-web/osmo-egprs-web.service
```

```
test -s /opt/GSM/osmo-egprs-web/install-web-service.sh
```

```
cd /opt/GSM/osmo-egprs-web && npm install --omit=dev --no-audit --no-fund || true
```

```
bash /opt/GSM/osmo-egprs-web/install-web-service.sh || true
```

2.25.1.10 Stage 8 : lte

The 4G side of the bench: srsRAN_4G built with the ZeroMQ radio (no SDR needed) and Open5GS, both under `/opt/LTE`, with their configuration files.

In Docker: `FROM base`. On a machine, carry on in the same system.

```
export ROOT=/opt/GSM


export OSMO_DEB_REFRESH=0


git clone https://github.com/bbaranoff/osmo-operator /opt/GSM/osmo-operator


mkdir -p "$(dirname /usr/local/sbin/osmo-lte-install)" && cp -r "$REPO"/tools/osmo-lte-install.sh
 ↪  /usr/local/sbin/osmo-lte-install


chmod 755 /usr/local/sbin/osmo-lte-install && \
    OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --deps && \
    { osmo-deb install libzmq 4.3.5+git || true; } && \
    { osmo-deb install srsgui 0.1+git || true; } && \
    SRS_DEB_VER="$(osmo-lte-install --srs-deb-version)" && \
    if ! osmo-deb install srsran "$SRS_DEB_VER"; then \
        OSMO_DEB=1 OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --build || { echo "ECHEC build srsRAN"; exit
         ↪  1; }; \
    fi && \
    if ! osmo-deb install open5gs 2.8.0+git; then \
        OSMO_DEB=1 OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --build || { echo "ECHEC build Open5GS"; exit
         ↪  1; }; \
    fi && \
    OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --configs --launchers && \
    ldconfig && test -x /usr/local/bin/srsenb && test -x /opt/LTE/open5gs/install/bin/open5gs-mmed \
    && test -s /opt/LTE/open5gs/install/etc/open5gs/mme.yaml && grep -q '^  sgsap:'
     ↪  /opt/LTE/open5gs/install/etc/open5gs/mme.yaml \
    && test -s /opt/LTE/open5gs/install/etc/freeDiameter/mme.conf && test -s
     ↪  /opt/LTE/open5gs/install/etc/open5gs/tls/mme.crt \
    && test -s /root/.config/srsran/enb.conf && test -x /usr/local/bin/osmo-lte && test -x /usr/local/bin/osmo-epc


mkdir -p "$(dirname /tmp/pmbootstrap-osmo-bench-qemu.patch)" && cp -r
 ↪  "$REPO"/patches/pmbootstrap-osmo-bench-qemu.patch /tmp/pmbootstrap-osmo-bench-qemu.patch


if ! osmo-deb install pmbootstrap 0.git; then \
        mkdir -p /opt/user_interface/pmos && \
        git clone https://gitlab.postmarketos.org/postmarketOS/pmbootstrap.git
         ↪  /opt/user_interface/pmos/pmbootstrap && \
        cd /opt/user_interface/pmos/pmbootstrap && \
        git apply /tmp/pmbootstrap-osmo-bench-qemu.patch && \
        grep -q osmo_bench_args pmb/commands/qemu.py && \
        osmo-deb snapshot pmbootstrap 0.git /opt/user_interface/pmos/pmbootstrap; \
    fi && test -f /opt/user_interface/pmos/pmbootstrap/pmbootstrap.py
```

2.25.1.11 Stage 9 : osmocom-nitb

The assembly stage, the image that is actually published. Nothing is compiled here: the `.deb` packages produced by the previous stages are installed (`dpkg -i`), then the project files are laid out: the `/etc/osmocom` configurations, the scripts under `/usr/local/bin`, the systemd units, the PulseAudio system daemon configuration (GSM audio), the runtime directories, the shell prompt, the desktop pieces and the `/opt/GSM` helper scripts. On a single machine the `COPY --from=<stage>` instructions are no-ops: everything is already installed where the previous stages put it. The container entrypoint and stop signal are Docker metadata only.

In Docker: `FROM base`. On a machine, carry on in the same system.

```
export ROOT=/opt/GSM
```

Copy between Docker stages (`--from=bb /var/cache/osmo-debs/ /var/cache/osmo-debs/`): on a single machine `/var/cache/osmo-debs/` is already in place.

Copy between Docker stages (`--from=l1 /var/cache/osmo-debs/ /var/cache/osmo-debs/`): on a single machine `/var/cache/osmo-debs/` is already in place.

Copy between Docker stages (`--from=grgsm-venv /var/cache/osmo-debs/ /var/cache/osmo-debs/`): on a single machine `/var/cache/osmo-debs/` is already in place.

Copy between Docker stages (`--from=lte /var/cache/osmo-debs/ /var/cache/osmo-debs/`): on a single machine `/var/cache/osmo-debs/` is already in place.

```
set -eux; \
    dpkg -i --force-overwrite /var/cache/osmo-debs/osmo-build-*.deb; \
    dpkg -i --force-overwrite \
        /var/cache/osmo-debs/osmo-build-osmo-trx_1.7.2+ipc~*.deb \
        /var/cache/osmo-debs/osmo-build-osmo-bts_1.10.0+rand~*.deb \
        /var/cache/osmo-debs/osmo-build-osmo-hlr_1.9.2+rand~*.deb \
        /var/cache/osmo-debs/osmo-build-osmo-msc_1.15.0+rand~*.deb; \
    ldconfig
```

Copy between Docker stages (`--from=node /opt/node /opt/node`): on a single machine `/opt/node` is already in place.

Copy between Docker stages (`--from=node /opt/GSM/osmo-egprs-web /opt/GSM/osmo-egprs-web`): on a single machine `/opt/GSM/osmo-egprs-web` is already in place.

Copy between Docker stages (`--from=node /etc/systemd/system/osmo-egprs-web.service /etc/systemd/system/osmo-egprs-web.service`): on a single machine `/etc/systemd/system/osmo-egprs-web.service` is already in place.

```
ln -sf /opt/node/bin/node /usr/local/bin/node \
    && ln -sf /opt/node/bin/npm  /usr/local/bin/npm \
    && ln -sf /opt/node/bin/npx  /usr/local/bin/npx \
    && ln -sf /etc/systemd/system/osmo-egprs-web.service \
        /etc/systemd/system/multi-user.target.wants/osmo-egprs-web.service \
    && node --version
```

```
git clone https://github.com/bbaranoff/osmo-operator /opt/GSM/osmo-operator
```

```
mkdir -p "$(dirname /usr/local/sbin/osmo-lte-install)" && cp -r "$REPO"/tools/osmo-lte-install.sh
 ↪  /usr/local/sbin/osmo-lte-install
```

```
chmod 755 /usr/local/sbin/osmo-lte-install
```

Copy between Docker stages (`--from=lte /opt/LTE /opt/LTE`): on a single machine `/opt/LTE` is already in place.

```
OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --deps && \
    OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --configs --launchers && \
    ldconfig && test -x /usr/local/bin/srsenb \
    && test -x /opt/LTE/open5gs/install/bin/open5gs-mmed \
    && test -s /opt/LTE/open5gs/install/etc/open5gs/mme.yaml && grep -q '^  sgsap:'
     ↪  /opt/LTE/open5gs/install/etc/open5gs/mme.yaml \
    && test -s /opt/LTE/open5gs/install/etc/freeDiameter/mme.conf \
    && test -s /opt/LTE/open5gs/install/etc/open5gs/tls/mme.crt \
    && test -s /root/.config/srsran/enb.conf && test -x /usr/local/bin/osmo-lte && test -x /usr/local/bin/osmo-epc


update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-9 90 \
       --slave /usr/bin/g++ g++ /usr/bin/g++-9 \
    && update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-11 110 \
       --slave /usr/bin/g++ g++ /usr/bin/g++-11 \
    && update-alternatives --set gcc /usr/bin/gcc-11


cd /etc/osmocom


mkdir -p /etc/osmocom/ && cp -r "$REPO"/scripts/. /etc/osmocom/


mkdir -p /etc/osmocom/ && cp -r "$REPO"/configs/*cfg /etc/osmocom/


mv /etc/osmocom/run.sh /root/run.sh


chmod +x /etc/osmocom/gapk-start.sh && \
    ln -sf /etc/osmocom/gapk-start.sh /usr/local/bin/gapk-start.sh && \
    mkdir -p /var/lib/gapk


cp -f /usr/local/bin/osmo* /usr/bin/ || true && \
    cp -f /usr/local/bin/proto-smsc-* /usr/bin/ || true && \
    cp -f /usr/local/bin/sms-* /usr/bin/ || true && \
    cp -f /usr/local/bin/gen-sms-* /usr/bin/ || true && \
    # Si tu as des fichiers .service dans configs/
    cp /etc/osmocom/configs/*.service /lib/systemd/system/ 2>/dev/null || true


which proto-smsc-daemon && which proto-smsc-sendmt && \
    echo "proto-smsc binaries OK" || \
    echo "WARNING: proto-smsc binaries not found in PATH"


sed -i 's/^CPUScheduling/#CPUScheduling/g' /lib/systemd/system/osmo-*.service && \
    sed -i 's/User=osmocom/User=root/g' /lib/systemd/system/osmo-ggsn.service && \
    sed -i 's/User=osmocom/User=root/g' /lib/systemd/system/osmo-sgsn.service && \
    chmod +x /etc/osmocom/*.sh


mkdir -p "$(dirname /etc/systemd/system/osmo-bts-trx.service)" && cp -r "$REPO"/services/osmo-bts-trx.service
 ↪  /etc/systemd/system/osmo-bts-trx.service


systemctl enable osmo-bts-trx.service && \
    passwd -d root && \
    systemctl mask getty@tty1.service serial-getty@tty1.service


mkdir -p /root/.osmocom/bb/ \
    && install -m755 ${ROOT}/osmocom-bb/src/host/trxcon/src/trxcon           /usr/local/bin/trxcon \
    && install -m755 ${ROOT}/osmocom-bb/src/host/layer23/src/mobile/mobile   /usr/local/bin/mobile \
    && install -m755 ${ROOT}/osmocom-bb/src/host/virt_phy/src/virtphy        /usr/local/bin/virtphy
```

```
echo "alias faketrx='python3 ${ROOT}/osmocom-bb/src/target/trx_toolkit/fake_trx.py'" >> ~/.bashrc


printf '%s\n' '' \
    '### calypso-prompt ###' \
    "export PS1='\[\033[1;31m\]\u\[\033[0m\]@\[\033[1;34m\]\h\[\033[0m\]:\[\033[1;32m\]\w\[\033[0m\]☎# '" \
    '### end calypso-prompt ###' >> /root/.bashrc


mkdir -p "$(dirname /root/.osmocom/bb/mobile.cfg)" && cp -r "$REPO"/configs/mobile.cfg
 ↪  /root/.osmocom/bb/mobile.cfg


chmod +x /root/run.sh


mkdir -p /var/log/osmocom /var/run/smsc \
        /etc/asterisk /var/lib/asterisk /var/log/asterisk /var/run/asterisk \
        /root/.osmocom/bb && \
    chmod 755 /etc/asterisk /var/lib/asterisk /var/log/asterisk \
        /var/run/asterisk /var/log/osmocom


set -eux; \
    usermod -aG pulse-access root 2>/dev/null || true; \
    test -f /etc/pulse/system.pa; \
    sed -i 's|^#\?load-module module-native-protocol-unix.*|load-module module-native-protocol-unix
     ↪  auth-anonymous=1 socket=/var/run/pulse/native|' /etc/pulse/system.pa; \
    grep -q 'auth-anonymous=1' /etc/pulse/system.pa \
      || echo 'load-module module-native-protocol-unix auth-anonymous=1 socket=/var/run/pulse/native' >>
       ↪  /etc/pulse/system.pa; \
    grep -q 'sink_name=gsm_audio' /etc/pulse/system.pa \
      || echo 'load-module module-null-sink sink_name=gsm_audio format=s16le rate=8000 channels=1
       ↪  sink_properties=device.description=GSM_Audio' >> /etc/pulse/system.pa; \
    grep -q 'sink_name=gsm_mic' /etc/pulse/system.pa \
      || echo 'load-module module-null-sink sink_name=gsm_mic format=s16le rate=8000 channels=1
       ↪  sink_properties=device.description=GSM_Mic' >> /etc/pulse/system.pa; \
    grep -q 'sink_name=osmo_tts_off' /etc/pulse/system.pa \
      || echo 'load-module module-null-sink sink_name=osmo_tts_off format=s16le rate=8000 channels=1
       ↪  sink_properties=device.description=TTS_off' >> /etc/pulse/system.pa; \
    sed -i 's|^load-module module-suspend-on-idle|#load-module module-suspend-on-idle|' /etc/pulse/system.pa; \
    mkdir -p /var/run/pulse && chown -R pulse:pulse /var/run/pulse


echo 'source ~/.env/bin/activate' >> ~/.bashrc


mkdir -p /opt/GSM/ && cp -r "$REPO"/opt-gsm/. /opt/GSM/
```

Docker metadata, no equivalent on a machine: `STOPSIGNAL SIGRTMIN+3`.

Docker metadata, no equivalent on a machine: `ENTRYPOINT ["/etc/osmocom/entrypoint.sh"]`.

Docker metadata, no equivalent on a machine: `CMD ["/bin/bash"]`.

2.25.1.12 Stage 10 : debs

A `scratch` stage holding only `/var/cache/osmo-debs`, used by the CI to extract the compiled packages without materialising the full image. Nothing to do on a machine.

Docker stage without content of its own (`scratch` image): nothing to do on a machine.

Copy between Docker stages (`--from=osmocom-nitb /var/cache/osmo-debs/ /`): on a single machine `/var/cache/osmo-debs/` is already in place.

2.25.2 Appendix: c54x_exe-from-scratch-ubuntu.en.md

Contents of `doc/c54x_exe-from-scratch-ubuntu.en.md`, included as is.

2.25.2.1 c54x_exe from a clean Ubuntu 24.04

Oct 20, 2026 · Bastien Baranoff

2.25.2.1.1 What this page shows

In about twenty minutes, the emulated TMS320C54x core of `c54x_exe` loads the TI mask ROM of a real Calypso and runs it, without QEMU, without the ARM, without a GSM network.

This is the DSP's hello world: it proves the core executes real TI code. It does not prove the DSP decodes anything, since no burst is injected. The full setup (ARM under QEMU, BTS, bridge, voice call) is described at the end of the page and lives in osmo-operator.

Everything runs as root or with `sudo`; the `/opt/GSM/…` paths are the ones the Makefiles and scripts expect by default.

2.25.2.1.2 1. Prerequisites (Ubuntu 24.04)

One `apt` line is enough: the compiler, `git`, `curl` for the ROMs, and `libosmocore` (which contains `libosmocoding`, used by `calypso_bsp.c`).

```
sudo apt update
sudo apt install -y build-essential git curl pkg-config libosmocore-dev
sudo mkdir -p /opt/GSM
```

Check that `libosmocoding` is there:

```
pkg-config --modversion libosmocoding
```

If `pkg-config` cannot find it, the `libosmocore` of the Ubuntu repository is too old or incomplete; install `libosmocore-dev` from the Osmocom nightly repository instead (see osmocom.org).

2.25.2.1.3 2. The qosmo sources

`c54x_exe` does not copy the C54x core: it compiles the qosmo sources directly (`calypso_c54x.c` and its neighbours) with ~120 lines of QEMU header stand-ins. The repository must be on disk, but it does not need to be built.

```
cd /opt/GSM
git clone https://github.com/bbaranoff/qosmo
```

No `configure`, no `make` here. If qosmo is somewhere other than `/opt/GSM/qosmo`, pass `QOSMO=/path` to the `make` of the next step.

2.25.2.1.4 3. Clone and build c54x_exe

The build takes less than a minute and produces a single binary, `c54x_exe`, at the root of the repository.

```
cd /opt/GSM
git clone https://github.com/bbaranoff/c54x_exe
cd c54x_exe
make QOSMO=/opt/GSM/qosmo \
     CFLAGS="-O3 -g -Wall -Werror=format -Werror=format-extra-args \
             -Wno-unused-function -Wno-unused-variable \
             -Wno-unused-but-set-variable -Wno-sign-compare"
```

This `CFLAGS` is the one of the osmo-operator Dockerfile: it keeps format errors fatal and silences the unused-variable warnings, numerous in the QEMU sources taken as they are. A bare `make` works too but is much noisier.

`make` rebuilds the binary on every call (`.PHONY` target); no `make clean` is needed after a change in qosmo.

Check:

```
./c54x_exe --help | head
```

2.25.2.1.5 4. The DSP ROMs, from FreeCalypso

The C54x mask ROM is not in the repository: it comes from the dump published by the FreeCalypso project (Mychaela Falconia) on its public FTP. The `rom/fetch-rom.sh` script downloads it and splits it into the 7 sections `--rom-dir` expects.

```
cd /opt/GSM/c54x_exe
export FREECALYPSO_URL=ftp://ftp.freecalypso.org/pub/GSM/Calypso
bash rom/fetch-rom.sh --dest /opt/GSM --dest /opt/GSM/c54x_exe/rom
curl -fsS --retry 2 -o /opt/GSM/calypso_dsp.txt "$FREECALYPSO_URL/dsp-rom-3606-dump.txt"
cp /opt/GSM/calypso_dsp.txt rom/calypso_dsp.txt
```

`3606` is the DSP ROM version of the Calypso in the Motorola C1xx; it is also the value the DSP L1 writes into the API RAM at boot, which will later confirm that the right ROM is running.

After the script, `/opt/GSM` must contain exactly these 7 files:

```
ls -l /opt/GSM/calypso_dsp.*.bin
## calypso_dsp.PROM0.bin  calypso_dsp.PROM1.bin  calypso_dsp.PROM2.bin
## calypso_dsp.PROM3.bin  calypso_dsp.DROM.bin   calypso_dsp.PDROM.bin
## calypso_dsp.Registers.bin
```

If `curl` refuses FTP (some builds come without it), download the dump by hand from the same directory on ftp.freecalypso.org and give the local file to `fetch-rom.sh`.

2.25.2.1.6 5. First run: the ROM runs

```
cd /opt/GSM/c54x_exe
./c54x_exe --trames 200
```

`--trames 200` runs the DSP for 200 TDMA frames (one frame = 4.615 ms of GSM time), with the default budget of 2300 instructions per frame in standalone mode. Without options only core errors go to stderr; the final summary says how many lines were hidden and which `-v` level shows them.

What you should see, in order:

1. The 7 ROM sections loaded from `/opt/GSM` (or `--rom-dir`).
2. `c54x_reset()`: the ROM restarts at `0xff80`, its reset vector.
3. `BRANCH-TRACE` lines around `PC=0xb41f`: the TI ROM's wait loop, real code executed by the core.
4. A final summary with `a_sch[3] = 0x771a`.

For one line per frame:

```
./c54x_exe --trames 50 --verbeux
```

And for core traces, `-v` to `-vvvvvv` (see `--help`).

2.25.2.1.7 6. Reading the result

`a_sch[3] = 0x771a` is a constant from one run to the next: the ROM booted, parked itself in IDLE and left this state in its API window. It is the proof that the core executes the ROM faithfully up to that point.

What it does not prove: decoding. The API RAM is blank, no burst arrives, the DSP runs idle. A decoder whose output does not depend on its input does not decode; varying the input requires the full setup of section 8.

Useful reference: the `qemu-calypso` README gives `0xf8d8` for the same word. The difference comes from the input (QEMU provides an API RAM initialised by the ARM, here it is empty), not from a bug: two different inputs, two different constants.

2.25.2.1.8 7. Common errors

Symptom	Cause	Fix
`make`: `qemu/osdep.h` or `calypso_c54x.c` not found	qosmo is not at `/opt/GSM/qosmo`	`make QOSMO=/path/to/qosmo`
`make`: `osmocom/coding/...` not found	`libosmocore-dev` missing or too old	section 1, Osmocom nightly repository
`fetch-rom.sh` fails or `curl: (1) Protocol "ftp" not supported`	`curl` built without FTP, or a firewall	download by hand, then `fetch-rom.sh` on the local file
`c54x_exe`: ROM not found	the `.bin` files are not in `/opt/GSM`	`--rom-dir /path` or redo section 4
The summary does not show `0x771a`	wrong ROM, or badly split sections	compare `rom/calypso_dsp.txt` with the 3606 dump; redo section 4
`PC=0xb41f` never appears, the core exits with an error	regression in qosmo	`cd /opt/GSM/qosmo && git log -3`, go back to the README's commit

2.25.2.1.9 8. Next: injecting bursts

For the DSP to really decode, you need the ARM (the osmocom-bb `layer1.highram.elf` firmware under the `qosmo` QEMU), a BTS and the bridge that delivers bursts to it. Two paths:

The complete bench. It installs everything, in the same `/opt/GSM/…` paths as this page:

```
cd /opt/GSM
git clone https://github.com/bbaranoff/osmo-operator
cd osmo-operator
sudo ./install.sh
sudo ./start-direct.sh --dsp
```

`--dsp` replaces the gr-gsm layer 1 with `c54x_exe`; `--menu` asks the same question interactively. The osmo-operator wiki describes the options, the ports and the numbering plan.

Executable by executable. To watch each step produce its log: the DSP (`c54x_exe --arm -v`), then QEMU with `CALYPSO_DSP_EXTERN=1`, then `osmocon`, then `mobile`. The sequence, with the expected log lines at each step, is in LAUNCH.md and the c54x_exe README.

From there, `a_sch` changes with the input, and the question "does it decode?" has a measurable answer.

2.25.2.1.10 9. The Calypso firmware: the arm-elf toolchain (gnuarm)

The osmocom-bb firmware (`layer1.highram.elf`, `trx.highram.elf`…) is built with the historical `arm-elf` toolchain (gcc 4.5.2, binutils 2.21.1, newlib 1.19) from the gnu-arm-installer, installed in `/root/gnuarm/install`. On Ubuntu 24.04 its `cc1` asks for `libmpfr.so.4`, which the system no longer ships (it has `libmpfr.so.6`); a symlink is enough:

```
ln -sf /usr/lib/x86_64-linux-gnu/libmpfr.so.6 /usr/local/lib/libmpfr.so.4 && ldconfig
/root/gnuarm/install/bin/arm-elf-gcc --version      # gcc 4.5.2
```

The `osmocom-bb-transceiver` tree (jolly/testing branch, `trx` application) also builds with the system's `arm-none-eabi-gcc`, with the adaptations of `patches/osmocom-bb-transceiver-qemu-bench.patch` (prefix, linking through gcc, `-fcommon`, `solve_envs.py` ported to Python 3):

```
cd /opt/GSM/osmocom-bb-transceiver/src/target/firmware
make BOARDS=compal_e88 APPLICATIONS=trx ENVIRONMENTS=highram CROSS_COMPILE=arm-none-eabi-
cp board/compal_e88/trx.highram.{elf,bin} /opt/GSM/firmware/board/compal_e88/
```

Every patch applied to the osmocom-bb trees (mobile, transceiver, burst-ind) lives in `patches/` of osmo-operator and is applied by the Dockerfile and `install_modules/40-patches.sh`.

2.26 /opt/GSM/osmo-operator/doc/install-from-scratch.md

87927 octets, 1555 lignes

2.26.1 Installation depuis zero (generee depuis le Dockerfile)

> Fichier genere par `tools/dockerfile-to-install-md.sh` le 2026-10-20 depuis `Dockerfile`. Ne pas editer a la main : corriger le Dockerfile ou le script, puis regenerer.

2.26.1.1 Comment lire cette page

Le Dockerfile d'osmo-operator est la description complete et testee de l'installation. Cette page le rejoue etape par etape sur une machine nue (Ubuntu 24.04, en root), sans Docker :

- chaque `FROM ... AS <nom>` du Dockerfile est un chapitre ; sur une seule machine les etapes se suivent simplement, les `COPY --from=<etape>` ne sont plus necessaires (tout est deja en place) ;
- chaque `RUN` est un bloc bash a coller tel quel dans un shell root, dans l'ordre ;
- chaque `ARG`/`ENV` devient un `export` a poser avant les blocs qui suivent (les `ARG` ne traversent pas un `FROM` dans Docker, ici on les garde dans le shell) ;
- chaque `COPY` devient un `cp` depuis une copie du depot osmo-operator (`$REPO`, par defaut `/opt/GSM/osmo-operator`).

Le cache de paquets `.deb` (`osmo-deb install <nom> <version>`) est une optimisation du build Docker : sans cache, la commande echoue et la branche `if ! osmo-deb install ...; then ... fi` compile depuis les sources, ce qui est exactement ce qu'on veut sur une machine nue. `osmo-deb pack` emballe ensuite le resultat (`make install`) ; sans `/var/cache/osmo-debs` il se contente d'installer.

```
export REPO=/opt/GSM/osmo-operator      # copie du depot (git clone https://github.com/bbaranoff/osmo-operator)
export ROOT=/opt/GSM
mkdir -p "$ROOT"
```

2.26.1.2 Sommaire

1. base (a partir de `ubuntu:24.04`)
2. osmo-core (a partir de `base`)
3. bb (a partir de `osmo-core`)
4. qemu (a partir de `base`)
5. l1 (a partir de `qemu`)
6. grgsm-venv (a partir de `osmo-core`)
7. node (a partir de `base`)
8. lte (a partir de `base`)
9. osmocom-nitb (a partir de `base`)
10. debs (a partir de `scratch`)

2.26.1.3 Etape 1 : base

Dockerfile — IMAGE DE BASE osmocom-nitb (COUCHE STABLE)

Tout ce qui est LONG et rarement modifie vit ici : la TOTALITE des paquets apt (une seule liste, plus aucun apt dans Dockerfile.run), la pile Osmocom compilee depuis les sources, osmocom-bb + le firmware Calypso, QEMU et son device IPC, gr-gsm/GNU Radio, le runtime Node et le dashboard web, la config PulseAudio, les units systemd, les repertoires, le prompt. ~11 Go et ~40 min de build : on ne la rebatit que quand une dependance change.

[2026-09-03] BASE ubuntu:24.04 (noble) - python 3.12, gcc 13 par defaut, bibliotheques renommees *t64 (libasound2t64, libgnutls30t64…). L ISO (build-iso.sh) part du MEME noble : le venv /root/.env et les .so de /usr/local sont copies tels quels dans le rootfs, ils doivent trouver la meme glibc et le meme python. Changer la base ici, c est la changer la-bas.

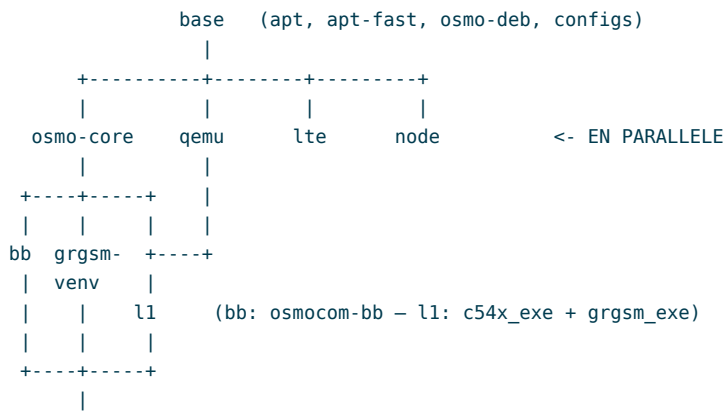
CACHE .deb - packaging/osmo-deb.sh. Chaque dossier compile ci-dessous sort en paquet .deb dans /var/cache/osmo-debs (COPY depuis .deb-cache/, que build.sh synchronise avec /var/cache/osmo-debs de l HOTE). Au rebuild, `osmo-deb install` pose le paquet et saute le clone + la compilation ; `osmo-deb pack` et `osmo-deb snapshot` fabriquent le paquet la premiere fois. build.sh –no-cache passe OSMO_DEB_REFRESH=1 : tout est recompile et le cache reecrit. LES SOURCES RESTENT DANS /opt/GSM : `osmo-deb pack` embarque l arbre courant (/opt/GSM/, .git et objets compris) dans le paquet, et les snapshots nomment l arbre. Un rebuild depuis le cache redonne donc la meme image, avec ses ateliers - pas une image ou seul /usr/local subsiste.

L'iteration quotidienne se fait dans Dockerfile.run, qui repart de cette image (`FROM osmocom-nitb`) et n'y rafraichit que les scripts, les configs, le pont et les arbres git qosmo / osmo-operator — en secondes, pas en 40 minutes.

Cette image reste AUTONOME : elle a son propre ENTRYPOINT et start-nitb.sh la lance seule. Ne pas retirer ses COPY de configs/scripts sous pretexte que Dockerfile.run les refait : le recouvrement est voulu des deux cotes.

[2026-09-25] LE BUILD EST UN GRAPHE, PLUS UNE CHAINE — stages BuildKit

Ce fichier etait une seule cible de ~75 etapes : une chaine, donc un seul chemin d execution, meme sur une machine a 32 coeurs. Or quatre des cinq compilations longues n ont AUCUNE dependance entre elles - la pile Osmocom, QEMU (qosmo), le venv GNU Radio, la 4G (srsRAN + Open5GS). Elles s additionnaient pour rien.



```
osmocom-nitb      (assemblage : dpkg -i, puis configs)
      |
    debs          (FROM scratch : le cache .deb pour la CI)
```

CE QUI REND LE DECOUPAGE POSSIBLE : packaging/osmo-deb.sh. Chaque bloc sort deja en .deb qui contient A LA FOIS les fichiers installes ET l arbre de sources de /opt/GSM (cmd_pack, « L ARBRE DE SOURCES PART AVEC »). Un stage parallele n a donc pas a exporter un systeme de fichiers : il exporte ses .deb, et l assemblage les repose avec un `dpkg -i`. C est EXACTEMENT ce que fait deja iso_modules/50-injection-image.sh pour le rootfs de l ISO — le precedent existe dans ce depot, on le reutilise.

TROIS STAGES NE SONT PAS PARALLELES A osmo-core, et c est voulu — tous les trois se lient a libosmocore d une facon ou d une autre :

```
- osmocom-bb : `make nofirmware` lie mobile/trxcon a libosmocore ;
- c54x_exe   : son Makefile fait `pkg-config --libs libosmocoding libosmocore` ;
- gr-gsm     : son CMakeLists cherche libosmocore/libosmocoding/libosmogsm.
```

Les en sortir donnerait un build qui casse, pas un build plus rapide. Ils tournent en revanche EN PARALLELE ENTRE EUX, une fois osmo-core fini.

CE QUI NE CHANGE PAS : les versions, les patchs, les ./configure, le contenu de /opt/GSM, et –target osmocom-nitb (build.sh, compose.yaml) reste OBLIGATOIRE puisque `debs` est toujours la derniere etape du fichier.

Dans Docker : `FROM ubuntu:24.04`. Sur une machine, on continue dans le meme systeme.

ROOT : ou vivent les sources dans l'image. Chemin FIXE et assume — dans un conteneur, il n'y a rien a rendre portable.

```
export DEBIAN_FRONTEND=noninteractive
```

```
export ROOT=/opt/GSM
```

OSMO_IMAGE=1 : « image de ce depot, son reseau est pose par start-docker.sh / compose » - network/osmo-ip-plan.sh ne touche alors a aucune adresse. Un conteneur quelconque (ubuntu:24.04 nu) ne l a pas et s adresse comme un natif.

```
export container=docker
export OSMO_IMAGE=1
export PKG_CONFIG_PATH=/usr/local/lib/pkgconfig
export LD_LIBRARY_PATH=/usr/local/lib
export GIT_TERMINAL_PROMPT=0
```

2.26.1.3.1 apt-fast : les téléchargements apt en parallèle

Cette image installe plusieurs centaines de paquets ; apt les récupère un par un, apt-fast les met en parallèle via aria2. Le gain porte sur le téléchargement, pas sur dpkg — l'installation reste séquentielle.

UN SEUL installeur pour tous les environnements : packaging/apt-fast-install.sh (cette image, Dockerfile.stp, le chroot de l ISO, l hote via build.sh). Il pose aussi /etc/apt/apt.conf.d/90osmo-operator, les reglages de telechargement communs. Repli integre : sans GitHub, apt-fast appelle apt-get et le build continue, plus lentement.

LE CACHE APT DU BUILD. [2026-09-04] Chaque etape apt de ce fichier monte /var/cache/apt/archives en cache BuildKit (id osmo-apt-archives, partage avec Dockerfile.stp) : les .deb telecharges une fois y restent, sur l hote, dans le cache de BuildKit - une reconstruction ne retelecharge que ce qui a change. apt-fast-install retire docker-clean et pose Keep-Downloaded-Packages, sinon apt effacerait les paquets a peine installes. Necessite BuildKit (docker compose, ou docker-buildx : build.sh les installe) - le builder historique ne connait pas –mount.

```
mkdir -p "$(dirname /usr/local/sbin/apt-fast-install)" && cp -r "$REPO"/packaging/apt-fast-install.sh
↪  /usr/local/sbin/apt-fast-install
```

Le miroir Ubuntu mesure par build.sh (packaging/apt-mirror.sh) : vide = les sources de l image de base (archive.ubuntu.com), qui peut etre inutilisable depuis l hote (3 Ko/s le 2026-09-04) sans qu apt le dise.

```
export OSMO_UBUNTU_MIRROR=""
```

```
chmod 755 /usr/local/sbin/apt-fast-install && apt-fast-install \
    && rm -rf /var/lib/apt/lists/*
```

```
export DEBIAN_FRONTEND=noninteractive
```

2.26.1.3.2 1. Dépendances système — TOUTES ici, y compris celles qu'installait

```
   Dockerfile.run. Une seule liste, un seul endroit où la faire évoluer :
   l'image d'exécution n'a plus à connaître apt du tout.
```

tshark tire wireshark-common, qui pose une question debconf (capture non-root) et bloquerait un build non interactif : on y répond d'avance.

⚠ COUPLAGE EXTERIEUR — install_modules/10-deps.sh EXTRAIT cette liste du Dockerfile (« la liste fait autorite dans le Dockerfile ») avec

```
  awk '/^RUN apt-get update && apt-get install/,/[^\\]$/'
```

Ce motif ne connait QUE `apt-get` : depuis le passage a apt-fast, il ne capture plus que le bloc d'amorcage (aria2 curl ca-certificates) et l'installation NATIVE (hors docker) repart avec une liste tronquee — inst_deps_verify echoue sur build-essential/libtalloc-dev/libsctp-dev. Le correctif est dans 10-deps.sh (accepter `apt-(get|fast)` dans l'awk ET dans le sed qui suit), pas ici : ne pas repasser cette liste en apt-get pour contourner le probleme.

```
echo 'wireshark-common wireshark-common/install-setuid boolean true' | debconf-set-selections
```

deb-src AVANT l unique `apt-fast update` : le build-dep gnuradio (gr-gsm, fin du bloc ci-dessous) lit les index Sources. Il avait son propre RUN avec un deuxieme update + une deuxieme resolution apt : fusionne ici. Noble ecrit ses sources en deb822 (ubuntu.sources, ligne “Types: deb”) ; l ancien /etc/apt/sources.list (jammy) reste gere au cas ou la base change (gnuradio est dans universe : tous les composants).

```
if [ -f /etc/apt/sources.list.d/ubuntu.sources ]; then \
        sed -i 's/^Types: deb$/Types: deb deb-src/' /etc/apt/sources.list.d/ubuntu.sources; \
    else \
        sed -nE 's|^deb (http\S+) (\S+) .*|deb-src \1 \2 main restricted universe multiverse|p' \
            /etc/apt/sources.list | sort -u > /etc/apt/sources.list.d/deb-src.list; \
    fi
```

```
apt-fast update && apt-fast install -y --no-install-recommends \
    # Outils de build
    build-essential git gcc g++ make cmake autoconf automake libtool pkg-config wget curl \
    # Dépendances Osmocom Core & Network
    libtalloc-dev libpcsclite-dev libsctp-dev libmnl-dev liburing-dev asterisk-moh* \
    libdbi-dev libdbd-sqlite3 libsqlite3-dev sqlite3 libc-ares-dev libgnutls28-dev \
    # Audio, Radio & SIP
    libortp-dev libfftw3-dev libusb-1.0-0-dev libsofia-sip-ua-dev libsofia-sip-ua-glib-dev \
    # Python & Outils système
    python3 python3-dev python3-scapy ca-certificates tmux systemd systemd-sysv \
    # Debug — gdb-multiarch pour attacher au gdb-stub QEMU (ARM Calypso)
    gdb-multiarch \
    # ALSA — requis par osmo-gapk pour l'I/O audio matériel
    # (libasound2t64 : nom noble de libasound2, transition time_t 64 bits)
    libasound2-dev libasound2t64 alsa-utils \
    # libgsm — codec GSM-FR natif (accélère gapk en mode gsmfr)
    libgsm1-dev libgsm1 \
```

```
    iptables iproute2 asterisk ffmpeg \
    # Sync build-iso : psmisc (pkill/killall, cleanup) + pulseaudio (chaîne audio gapk/parec)
    psmisc pulseaudio pulseaudio-utils binutils-arm-none-eabi \
    # Toolchains alternatives : osmocom-bb jolly/testing et fixeria/burst_ind ne
    # compilent qu'avec gcc-9 ; gcc-11 reste le compilateur par defaut du reste
    # (gcc-13 est celui de noble : il compile la pile Osmocom en tete de fichier,
    # les alternatives ci-dessous ne sont posees qu apres).
    # (Ex-bloc apt dedie, supprime : les `update-alternatives` plus bas echouaient
    #  « alternative path /usr/bin/gcc-9 doesn't exist » sans ces paquets.)
    gcc-9 g++-9 gcc-11 g++-11 \
    # log4cpp : etait installe a part avec le build-dep gnuradio (gr-gsm).
    liblog4cpp5-dev \
    # QEMU Calypso (bbaranoff/qosmO) : venv + numpy/scipy pour l'outillage
    # DSP, glib/pixman/slirp pour la cible arm-softmmu, ninja pour meson, socat
    # pour les PTY de scripts/run.sh. (Ex-bloc apt-get juste avant le build QEMU.)
    python3-venv python3-pip python3-numpy python3-scipy \
    libglib2.0-dev libpixman-1-dev libslirp-dev socat ninja-build \
    # Ex-Dockerfile.run — outillage d'exploitation du conteneur :
    #   telnet   -> repli VTY (run_modules/21-abonnes-hlr.sh) + transport dashboard
    #   whiptail -> menus de tools/vty-menu.sh et de start-docker.sh
    #   xz-utils -> `tar -xJf` du tarball Node 22 (bloc dashboard, en fin de fichier)
    #   libasound2-plugins -> greffon ALSA->PulseAudio, EXIGE par configs/asound.conf
    #        (`type pulse` sur gsm_out, gsm_in ET pcm.!default) : sans lui, toute
    #        ouverture ALSA (gapk, mobile) echoue et l'audio est mort des 2 cotes
    #   tshark/tcpdump/libcap2-bin -> capture ; libcap2-bin fournit le setcap ci-dessous
    telnet nano whiptail xz-utils libasound2-plugins \
    tcpdump tshark libcap2-bin \
    # Deps GNU Radio 3.10 (gr-gsm, venv /root/.env plus bas) : meme resolution
    # apt que le reste, pas de second RUN apt. (Le && ferme la liste pour
    # l extracteur de install_modules/10-deps.sh.)
    && apt-fast build-dep -y gnuradio \
    && rm -rf /var/lib/apt/lists/*
```

Capture non-root pour le dashboard et tcpdump. Position OBLIGATOIRE : dumpcap n'existe qu'une fois wireshark-common installe (tire par tshark, juste au-dessus).

```
setcap cap_net_raw,cap_net_admin+eip "$(command -v dumpcap)"
```

2.26.1.3.3 Le cache .deb : l outil, puis les paquets deja construits

.deb-cache/ est le miroir, dans le contexte de build, de /var/cache/osmo-debs de l hote (build.sh l y recopie avant, et ramene les nouveaux paquets apres). Vide au premier build : tout se compile et se met en cache. Plein ensuite : chaque etape ci-dessous pose son paquet et ne compile rien. OSMO_DEB_REFRESH=1 (build.sh –no-cache) : le cache est ignore et reecrit.

```
export OSMO_DEB_REFRESH=0
```

```
mkdir -p "$(dirname /usr/local/sbin/osmo-deb)" && cp -r "$REPO"/packaging/osmo-deb.sh /usr/local/sbin/osmo-deb
```

```
mkdir -p /var/cache/osmo-debs/ && cp -r "$REPO"/.deb-cache/ /var/cache/osmo-debs/
```

```
chmod 755 /usr/local/sbin/osmo-deb && osmo-deb list || true
```

2.26.1.3.4 git : plus de forçage HTTP/1.1

[2026-09-03] RETIRE. Le contournement http.version=HTTP/1.1 (contre “expected flush after ref listing”, un flux HTTP/2 coupe par un intermediaire) etait pose ici en global et repete a chaque clone. Il n a plus lieu d etre : git parle HTTP/2 normalement, et forcer HTTP/1.1 ralentissait tous les clones pour un incident de reseau local. GIT_TERMINAL_PROMPT=0 (ENV, plus haut) reste : sans terminal, un clone qui veut un login doit ECHOUER net, pas attendre. postBuffer large pour les gros clones.

```
git config --global http.postBuffer 524288000
```

Metadonnee Docker, sans equivalent sur une machine : `SHELL ["/bin/bash", "-c"]`.

```
mkdir -p /etc/asterisk/ && cp -r "$REPO"/configs/*conf /etc/asterisk/
```

asound.conf DOIT AUSSI ETRE A LA RACINE. Le COPY ci-dessus le depose dans /etc/asterisk/ - ALSA ne regarde jamais la : il lit /etc/asound.conf (ou ~/.asoundrc). Sans ce second exemplaire, `aplay -D gsm_out` repond

```
    ALSA lib pcm.c: (snd_pcm_open_noupdate) Unknown PCM gsm_out
```

et le mobile du conteneur est MUET, sans qu aucun log ne parle d audio. start-docker.sh l.469 monte bien celui de l hote (-v …:/etc/asound.conf), mais un conteneur lance autrement - docker run a la main, diagnostic - n a rien.

```
mkdir -p "$(dirname /etc/asound.conf)" && cp -r "$REPO"/configs/asound.conf /etc/asound.conf


cd ${ROOT}
```

2.26.1.3.5 2. Création de l’utilisateur osmocom

```
groupadd osmocom && useradd -r -g osmocom -s /sbin/nologin -d /var/lib/osmocom osmocom && \
    mkdir -p /var/lib/osmocom && chown osmocom:osmocom /var/lib/osmocom
```

2.26.1.4 Etape 2 : osmo-core

STAGE osmo-core — la pile Osmocom : 17 depots, 4 patchs, gapk, SMSC, libosmo-dsp

Dans Docker : `FROM base`. *Sur une machine, on continue dans le meme systeme.*

ARG ne traverse pas un FROM : les stages qui appellent osmo-deb ou ${ROOT} doivent les redeclarer, sinon ROOT est vide et OSMO_DEB_REFRESH=1 (build.sh –no-cache) ne descend pas jusqu ici.

```
export ROOT=/opt/GSM


export OSMO_DEB_REFRESH=0
```

2.26.1.4.1 3. Compilation de la pile Osmocom (Ordre respecté)

```
for repo in \
    libosmocore:1.12.1 \
    libosmo-netif:1.7.0 \
    libosmo-abis:2.1.0 \
    libosmo-sigtran:2.2.1 \
    libsmpp34:1.14.5 \
    libgtpnl:1.3.3 \
    osmo-hlr:1.9.2 \
    osmo-mgw:1.15.0 \
    osmo-ggsn:1.14.0 \
    osmo-sgsn:1.13.1 \
    osmo-msc:1.15.0 \
    osmo-bsc:1.14.0 \
    osmo-trx:1.7.2 \
    osmo-bts:1.10.0 \
    osmo-pcu:1.5.2 \
    osmo-sip-connector:1.7.2 \
    libosmo-gprs:0.2.1; \
    do \
    name=$(echo $repo | cut -d: -f1) && \
    version=$(echo $repo | cut -d: -f2) && \
    \
    if [[ "$name" =~ ^libosmo ]]; then \
        GIT_URL="https://gitea.osmocom.org/osmocom/$name"; \
    else \
        GIT_URL="https://gitea.osmocom.org/cellular-infrastructure/$name"; \
    fi && \
    \
    # Le paquet du cache d abord : s il est la, ni clone ni compilation.
    if osmo-deb install "$name" "$version"; then continue; fi && \
    \
    cd ${ROOT} && \
    git clone "$GIT_URL" && cd "$name" && \
    git checkout "$version" && \
    \
    autoreconf -fi && \
    EXTRA_FLAGS="" && \
    if [ "$name" = "libosmo-abis" ]; then EXTRA_FLAGS="--disable-dahdi"; fi && \
    if [ "$name" = "osmo-msc" ]; then EXTRA_FLAGS="--enable-smpp"; fi && \
    if [ "$name" = "osmo-mgw" ]; then EXTRA_FLAGS="--enable-alsa"; fi && \
    if [ "$name" = "osmo-trx" ]; then EXTRA_FLAGS="--with-ipc"; fi && \
    if [ "$name" = "osmo-bts" ]; then EXTRA_FLAGS="--enable-virtual --enable-trx"; fi && \
    if [ "$name" = "osmo-ggsn" ]; then EXTRA_FLAGS="--enable-gtp-linux"; fi && \
    \
    ./configure $EXTRA_FLAGS && \
    make -j$(nproc) && \
    # make install sous DESTDIR -> .deb dans le cache -> dpkg -i dans la racine
    osmo-deb pack "$name" "$version" make install && \
    ldconfig \
    || { echo "ECHEC build osmocom: $name"; exit 1; }; \
    done
```

2.26.1.4.2 Patch osmo-trx IPC : alignement ts_initial sur la trame TDMA (fix RACH/LU)

Le device IPC (calypso-ipc-device) commit des buffers RX a des timestamps multiples de 2500 (CALYPSO_SHM_BUFSIZE) ; sans arrondi, ts_initial%5000 valait 0 ou 2500 -> device-TN0 mappe sur osmo-TN4 -> la RACH (UL) ratait le slot TS0/RACH -> NOPE/-110 -> Location Update bloquee. Le patch arrondit ts_initial a la frontiere de trame (8*625=5000 @ 4 SPS). Patch maintenu dans patches/.

2.26.1.4.3 Patch osmo-trx RACH UL : table de modulation per-RA (fix LU, no-hardcode)

La vraie RA du mobile (d_rach@0x0474, plombee via /dev/shm/calypso_rach par QEMU) varie a chaque burst ; le device rejouait un RA=3 fixe -> request-reference de l'IMM ASSIGN jamais matchee -> LU en boucle. sigProcLib pre-genere la modulation Laurent EXACTE d'osmo-trx pour chaque RA 0x00..0x0f (-> /root/rach_ref_RA.cs16), le device selectionne le ref de la VRAIE RA. Ajoute aussi les logs RACH-DET (Transceiver.cpp) + le lien libosmocoding dans COMMON_LDADD (Makefile.am, requis pour gsm0503_rach_ext_encode). Touche Makefile.am -> autoreconf+configure requis.

```
mkdir -p "$(dirname /tmp/osmo-trx-ipc-ts-frame-align.patch)" && cp -r
 ↪  "$REPO"/patches/osmo-trx-ipc-ts-frame-align.patch /tmp/osmo-trx-ipc-ts-frame-align.patch


mkdir -p "$(dirname /tmp/osmo-trx-rach-per-ra-table.patch)" && cp -r
 ↪  "$REPO"/patches/osmo-trx-rach-per-ra-table.patch /tmp/osmo-trx-rach-per-ra-table.patch
```

Meme nom de paquet que dans la boucle (osmo-trx), version 1.7.2+ipc : dpkg fait une mise a jour, les fichiers sont les memes. Si la boucle est sortie du cache, l arbre source n existe pas : on le reclone avant d appliquer les patchs.

```
if ! osmo-deb install osmo-trx 1.7.2+ipc; then \
      { [ -d ${ROOT}/osmo-trx ] || { cd ${ROOT} \
          && git clone https://gitea.osmocom.org/cellular-infrastructure/osmo-trx \
          && git -C osmo-trx checkout 1.7.2; }; } \
      && git -C ${ROOT}/osmo-trx apply /tmp/osmo-trx-ipc-ts-frame-align.patch \
      && git -C ${ROOT}/osmo-trx apply /tmp/osmo-trx-rach-per-ra-table.patch \
      && cd ${ROOT}/osmo-trx \
      && autoreconf -fi \
      && ./configure --with-ipc \
      && make -j$(nproc) \
      && osmo-deb pack osmo-trx 1.7.2+ipc make install \
      && ldconfig; \
    fi
```

2.26.1.4.4 Patch osmo-bts RAND : forcer le defi d'authentification

osmo-bts relaie normalement l'AUTHENTICATION REQUEST du reseau telle quelle. Ce patch, dans rsl_rx_rll() (src/common/rsl.c), reecrit le RAND de cette trame quand la variable d'environnement RAND est posee sur le processus BTS :

```
    RAND non defini -> rien de touche (comportement normal) ;
    RAND=0          -> RAND nul sur 16 octets (defi fixe COMP128v1,
                       defi fixe connu, Ki = prefixe connu
                       + 2 octets a retrouver) ;
    RAND=<hexa>     -> jusqu'a 16 octets lus en hexa, completes de zeros.
```

« seulement quand precise, mais force quand precise ». Ne touche que rsl.c, donc pas de Makefile.am : autoreconf/configure quand meme, l'arbre reclone arrive sans ./configure. Meme nom de paquet (osmo-bts), version 1.10.0+rand : dpkg fait une mise a jour. Patch maintenu dans patches/.

```
mkdir -p "$(dirname /tmp/osmo-bts-force-rand-toy.patch)" && cp -r "$REPO"/patches/osmo-bts-force-rand-toy.patch
 ↪  /tmp/osmo-bts-force-rand-toy.patch


if ! osmo-deb install osmo-bts 1.10.0+rand; then \
      { [ -d ${ROOT}/osmo-bts ] || { cd ${ROOT} \
          && git clone https://gitea.osmocom.org/cellular-infrastructure/osmo-bts \
          && git -C osmo-bts checkout 1.10.0; }; } \
      && git -C ${ROOT}/osmo-bts apply /tmp/osmo-bts-force-rand-toy.patch \
      && cd ${ROOT}/osmo-bts \
      && autoreconf -fi \
      && ./configure --enable-virtual --enable-trx \
      && make -j$(nproc) \
      && osmo-deb pack osmo-bts 1.10.0+rand make install \
      && ldconfig; \
    fi
```

2.26.1.4.5 Patch osmo-hlr RAND : forcer le RAND du vecteur d'auth

auc.c/auc_compute_vectors() : gate minimal. Si la variable d'environnement RAND est posee sur le process HLR, le RAND tire pour CHAQUE vecteur est force a 0 (16 octets) AVANT osmo_auth_gen_vec2 : SRES et Kc sont alors calcules sur RAND=0 (le HLR a le Ki). C'est l'etage indispensable pour une LU ACCEPTEE : osmo-msc envoie ce RAND et compare au SRES du meme vecteur -> match. Le seul gate osmo-msc (cote tx) donnerait un mismatch -> LU reject. RAND absent -> reseau normal. Ne touche pas Makefile.am ; autoreconf/configure quand meme (arbre reclone sans ./configure). Meme nom de paquet (osmo-hlr), version 1.9.2+rand : dpkg met a jour. Patch maintenu dans patches/.

```
mkdir -p "$(dirname /tmp/osmo-hlr-force-rand-toy.patch)" && cp -r "$REPO"/patches/osmo-hlr-force-rand-toy.patch
↪  /tmp/osmo-hlr-force-rand-toy.patch


if ! osmo-deb install osmo-hlr 1.9.2+rand; then \
      { [ -d ${ROOT}/osmo-hlr ] || { cd ${ROOT} \
          && git clone https://gitea.osmocom.org/cellular-infrastructure/osmo-hlr \
          && git -C osmo-hlr checkout 1.9.2; }; } \
      && git -C ${ROOT}/osmo-hlr apply /tmp/osmo-hlr-force-rand-toy.patch \
      && cd ${ROOT}/osmo-hlr \
      && autoreconf -fi \
      && ./configure \
      && make -j$(nproc) \
      && osmo-deb pack osmo-hlr 1.9.2+rand make install \
      && ldconfig; \
    fi
```

2.26.1.4.6 Patch osmo-msc RAND : forcer le defi d'authentification

gsm48_tx_mm_auth_req() (src/libmsc/gsm_04_08.c) : gate minimal. Si la variable d'environnement RAND est posee sur le processus MSC, le RAND de l'AUTH REQUEST descendante est force (0 sur 16 octets dans notre cas ; changer l'initialiseur forced_rand[] pour un autre RAND). RAND absent -> comportement normal du reseau. Ne touche pas Makefile.am ; autoreconf/configure quand meme (arbre reclone sans ./configure). Meme nom de paquet (osmo-msc), version 1.15.0+rand : dpkg met a jour. Patch maintenu dans patches/. Meme gate cote BSC (openbsc gsm_04_08.c).

```
mkdir -p "$(dirname /tmp/osmo-msc-force-rand-toy.patch)" && cp -r "$REPO"/patches/osmo-msc-force-rand-toy.patch
↪  /tmp/osmo-msc-force-rand-toy.patch


if ! osmo-deb install osmo-msc 1.15.0+rand; then \
      { [ -d ${ROOT}/osmo-msc ] || { cd ${ROOT} \
          && git clone https://gitea.osmocom.org/cellular-infrastructure/osmo-msc \
          && git -C osmo-msc checkout 1.15.0; }; } \
      && git -C ${ROOT}/osmo-msc apply /tmp/osmo-msc-force-rand-toy.patch \
      && cd ${ROOT}/osmo-msc \
      && autoreconf -fi \
      && ./configure --enable-smpp \
      && make -j$(nproc) \
      && osmo-deb pack osmo-msc 1.15.0+rand make install \
      && ldconfig; \
    fi
```

2.26.1.4.7 Patch gapk : sonde sur la sortie ALSA (GAPK_ALSA_PROBE)

Diagnostic pur, inerte tant que GAPK_ALSA_PROBE != 1. Tranche une question que rien d'autre ne permet de trancher : quand l'ecouteur du MS est muet, le PCM remis a snd_pcm_writei est-il deja nul (defaut en amont, dans la chaine gapk) ou non nul (defaut en aval, ALSA/PulseAudio) ? osmo_gapk_pq_execute ne journalise rien ici, car il ne se plaint que d'un retour negatif et pq_cb_alsa_output rend « succes » meme pour une ecriture vide. Patch maintenu dans patches/ (regenere si pq_alsa.c change).

[2026-08-12] LE PATCH RESTE DANS LE DEPOT, LE BUILD NE L'APPLIQUE PLUS. Demande explicite : garder patches/gapk-pq-alsa-output-probe.patch versionne (il tranche encore la question amont/aval quand

l'ecouteur est muet) mais ne plus le poser sur le gapk construit — un `git apply` dans le build casse l'image des que pq_alsa.c bouge en amont, et la sonde n'est plus la question du jour (descendant muet resolu par CALYPSO_PULSE_LATENCY_MSEC, 240 depuis le 2026-09-09 : cf. start-native.sh, le tampon du greffon ALSA-PulseAudio). POUR LE REMETTRE, deux gestes, dans cet ordre :

```
1. decommenter la ligne COPY ci-dessous ;
2. reinserer dans le RUN, entre le `git clone` et le `cd osmo-gapk` :
     git -C ${ROOT}/osmo-gapk apply /tmp/gapk-pq-alsa-output-probe.patch && \
   (elle ne peut pas rester en commentaire : une ligne `#` au milieu d'une
    continuation `\` couperait la chaine shell du RUN).
```

Verifier d'abord que le patch s'applique toujours :

```
git -C <clone gapk> apply --check patches/gapk-pq-alsa-output-probe.patch
```

POUR L'APPLIQUER A CHAUD SANS REBUILD : le poser dans le conteneur sur un clone de gapk, puis reconstruire gapk seul. COPY patches/gapk-pq-alsa-output-probe.patch /tmp/gapk-pq-alsa-output-probe.patch

```
if ! osmo-deb install osmo-gapk 0.git; then \
      cd ${ROOT} && \
      git clone https://gitea.osmocom.org/osmocom/gapk osmo-gapk && \
      cd osmo-gapk && \
      autoreconf -fi && \
      ./configure --enable-alsa && \
      make -j$(nproc) && \
      osmo-deb pack osmo-gapk 0.git make install && \
      ldconfig; \
    fi
```

2.26.1.4.8 gsup-smsc-proto : SMSC externe connecté à OsmoHLR via GSUP

Programmes : proto-smsc-daemon (réception MO SMS + relai MT via GSUP)

```
            proto-smsc-sendmt (injection MT SMS via socket UNIX local)
```

Dépendances build : libosmocore, libosmogsm, libosmo-gsup-client [2026-09-03] Son Makefile de tete lance `make DESTDIR= install` dans daemon/ et sendmt/ - DESTDIR force a vide, le staging d osmo-deb restait vide (“rien n a ete installe dans DESTDIR”) alors que la compilation (gcc courant) etait bonne. On installe donc les deux sous-repertoires nous-memes, avec le DESTDIR qu osmo-deb pose : leurs Makefiles, eux, l honorent.

```
if ! osmo-deb install gsup-smsc-proto 0.git; then \
      cd ${ROOT} && \
      git clone https://gitea.osmocom.org/themwi/gsup-smsc-proto && \
      cd gsup-smsc-proto && \
      ./configure --with-osmo=/usr/local && \
      make -j$(nproc) && \
      osmo-deb pack gsup-smsc-proto 0.git \
        sh -c 'for i in daemon sendmt; do make -C "$i" DESTDIR="$DESTDIR" install || exit 1; done' && \
      ldconfig; \
    fi
```

2.26.1.4.9 sms-coding-utils : encodage/décodage SMS PDU (GSM 03.40)

sms-encode-text, gen-sms-deliver-pdu, sms-pdu-decode, etc. Son Makefile de tete pose `DESTDIR=` (vide) et le repasse aux sous-repertoires : une variable donnee SUR LA LIGNE DE COMMANDE de make l emporte et descend avec, d ou `make install DESTDIR=...` explicite (l ancien INSTDIR=… n existait dans aucun de ses Makefiles). bindir vaut /usr/local/bin par configure.

```
if ! osmo-deb install sms-coding-utils 0.r1; then \
      cd ${ROOT} && \
      wget -q https://www.freecalypso.org/pub/GSM/FreeCalypso/sms-coding-utils-latest.tar.bz2 && \
      tar xf sms-coding-utils-latest.tar.bz2 && \
      cd sms-coding-utils-r1 && \
      ./configure && \
      make -j$(nproc) && \
      osmo-deb pack sms-coding-utils 0.r1 \
        sh -c 'mkdir -p "$DESTDIR/usr/local/bin" && make install DESTDIR="$DESTDIR"'; \
    fi
```

2.26.1.4.10 libosmo-dsp (dépendance transceiver/burst_ind)

```
if ! osmo-deb install libosmo-dsp 0.git; then \
      cd /opt/GSM \
      && git clone https://gitea.osmocom.org/sdr/libosmo-dsp.git \
      && cd libosmo-dsp \
      && autoreconf -fi \
      && ./configure \
      && make -j$(nproc) \
      && osmo-deb pack libosmo-dsp 0.git make install \
      && ldconfig; \
    fi
```

2.26.1.5 Etape 3 : bb

STAGE bb — osmocom-bb, le firmware Calypso et les deux branches gcc-9

Dans Docker : `FROM osmo-core`. Sur une machine, on continue dans le meme systeme.

ARG ne traverse pas un FROM : les stages qui appellent osmo-deb ou ${ROOT} doivent les redeclarer, sinon ROOT est vide et OSMO_DEB_REFRESH=1 (build.sh –no-cache) ne descend pas jusqu ici.

```
export ROOT=/opt/GSM
```

```
export OSMO_DEB_REFRESH=0
```

2.26.1.5.1 Calypso build

2.26.1.5.2 Patch osmocon : filtre Kc — RETIRE le 2026-08-30

Ce patch faisait ecrire /dev/shm/calypso_kc a osmocon, en espionnant L1CTL_CRYPTO_REQ au passage sur le lien serie. Il etait la SEULE source du Kc a l'epoque ; il ne l'est plus, et il faisait desormais plus de mal que de bien.

POURQUOI ON LE RETIRE. Un espion voit passer la cle, mais pas sa fin de vie : il doit la DEVINER. Le patch la devinait sur DM_EST_REQ et DM_REL_REQ, et remettait le fichier a 32 zeros. L'intention est juste pour le RELEASE, fausse pour l'ESTABLISH – le mobile emet aussi un DM_EST_REQ pour ouvrir un lien SUPPLEMENTAIRE sur un canal DEJA chiffre (le SAPI 3 du SMS). La cle disparaissait donc en pleine session chiffree.

Pire, il y avait TROIS ecrivains sur ce fichier (osmocon, l1ctl_sock.c, le shunt DSP) sans le moindre arbitrage : le dernier qui ecrit gagne, et c'est l'effaceur. Mesure du 30/08 : fichier a 32 zeros, « A5=non » sur tout le canal dedie, premier appel mort.

CE QUI LE REMPLACE. calypso_dsp_shunt.c publie /dev/shm/calypso_kc_l1 depuis d_a5mode et a_kc du NDB – ce que le FIRMWARE a reellement charge dans le DSP (calypso/dsp.c:dsp_load_ciph_param). Ce n'est plus un espion qui devine, c'est l'etat de la couche 1, remis a zero par le firmware lui-meme quand il repart en clair. Et le fichier lui est PROPRE : plus de course a l'ecriture.

Le patch reste dans patches/ a titre documentaire, il n'est plus applique. L arbre ENTIER part dans le paquet (snapshot) : trx_toolkit, osmocon et les binaires host sont lus dans l arbre au runtime, pas seulement dans /usr/local. [2026-09-08] trx_toolkit/clck_gen.py : l horloge TDMA de fake_trx ne se remet a zero que pour un retard superieur a une trame (4,615 ms). Avant, un retard de 0,3 a 2 ms decalait la base de temps a chaque fois ; osmo-bts-trx compensait en boucle et la parole perdait des trames. Patch maintenu dans patches/, applique aussi par install_modules/40-patches.sh en natif. mobile : une SIM reinseree sans RPLMN (reconnexion SAP) restait en A6 et la recherche de PLMN finissait « unhandled » (patches/osmocom-bb-plmn-a6-reinsert.patch). PLMNsel / FPLMN lus avec osmo_plmn_to_bcd (ecriture) au lieu de from_bcd : 000-00 (patches/osmocom-bb-sim-plmn-from-bcd.patch). Les deux, aussi, par 40-patches.sh.

```
mkdir -p "$(dirname /tmp/osmocom-bb-clck-gen-frame-tolerance.patch)" && cp -r
 ↪  "$REPO"/patches/osmocom-bb-clck-gen-frame-tolerance.patch /tmp/osmocom-bb-clck-gen-frame-tolerance.patch


mkdir -p "$(dirname /tmp/osmocom-bb-plmn-a6-reinsert.patch)" && cp -r
 ↪  "$REPO"/patches/osmocom-bb-plmn-a6-reinsert.patch /tmp/osmocom-bb-plmn-a6-reinsert.patch


mkdir -p "$(dirname /tmp/osmocom-bb-sim-plmn-from-bcd.patch)" && cp -r
 ↪  "$REPO"/patches/osmocom-bb-sim-plmn-from-bcd.patch /tmp/osmocom-bb-sim-plmn-from-bcd.patch


if ! osmo-deb install osmocom-bb 0.git; then \
      cd ${ROOT} && \
      git clone https://gitea.osmocom.org/phone-side/osmocom-bb && \
      git -C ${ROOT}/osmocom-bb apply /tmp/osmocom-bb-clck-gen-frame-tolerance.patch && \
      git -C ${ROOT}/osmocom-bb apply /tmp/osmocom-bb-plmn-a6-reinsert.patch && \
      git -C ${ROOT}/osmocom-bb apply /tmp/osmocom-bb-sim-plmn-from-bcd.patch && \
      cd osmocom-bb/src && \
      # Build complet : firmware (layer1.bin/.elf pour Calypso) + outils host
      # (mobile, trxcon, virtphy, ccch_scan). Le firmware est nécessaire pour
      # le mode PHY_MODE=qemu où QEMU émule un Calypso et exécute layer1.
      make nofirmware && \
      osmo-deb snapshot osmocom-bb 0.git ${ROOT}/osmocom-bb; \
    fi
```

2.26.1.5.3 Note historique — patch fake_trx TRXD v0 (RETIRE)

Conserve verbatim : il documente une panne vecue. Il flottait dans Dockerfile.run juste au-dessus du bloc firmware, avec lequel il n'a AUCUN rapport — voisinage accidentel, pas lien de cause a effet.

2.26.1.5.4 Patch fake_trx : forcer TRXD v0

osmo-bts-trx 1.10+ négocie TRXD v1 via SETFORMAT, mais trxcon (OsmocomBB) ne supporte que v0. Le header v1 a 2 octets de plus → décalage des soft-bits → BER constant 55/456 sur chaque frame → FBSB_SEARCH échoue en boucle. Fix : forcer ver_req=0 dans la réponse SETFORMAT → le BTS reste en v0. (patch TRXD v0 retire : burst_fwd.py:66 re-encode deja par interface,

```
 trxcon recoit du v0 quoi qu'il arrive -> le forcage etait inutile)
```

2.26.1.5.5 Firmware Calypso prebuild (layer1.highram.*) — ex-Dockerfile.run

`make nofirmware` ci-dessus ne construit justement PAS le firmware : on prend les binaires prebuild de bbaranoff/firmware et on les depose la ou le mode PHY_MODE=qemu les cherche. ORDRE IMPOSE : ce bloc exige A LA FOIS ce clone ET l'arbre /opt/GSM/osmocom-bb ci-dessus — le placer plus haut ferait echouer les cp et PHY_MODE=qemu partirait sans layer1. (Le `rm -rf /opt/GSM/firmware` qui precedait le clone dans Dockerfile.run est

```
 supprime : ici le chemin n'existe pas encore, l'instruction etait morte.)
```

GIT_TERMINAL_PROMPT=0 : sans terminal (docker build), un clone qui demande un login doit echouer NET (message clair) au lieu d attendre un identifiant que personne ne tapera. –depth 1 : on ne veut que les binaires prebuild, pas l historique. (Le -c http.version=HTTP/1.1 qui etait ici est retire.)

```
if ! osmo-deb install calypso-firmware 0.git; then \
      GIT_TERMINAL_PROMPT=0 git clone --depth 1 \
        https://github.com/bbaranoff/firmware /opt/GSM/firmware \
      && osmo-deb snapshot calypso-firmware 0.git /opt/GSM/firmware; \
    fi
```

[2026-08-28] Les trois `cp` vers /opt/GSM/osmocom-bb/src/target/firmware qui suivaient sont SUPPRIMES. Ils entretenaient un deuxieme exemplaire du firmware que plus personne ne lit : depuis la normalisation, environnement/paths.env du depot qemu ne connait qu'un chemin, $GSM_ROOT/firmware, et c'est celui que QEMU (-kernel .elf) comme osmocon (romload .bin) recoivent. Verifie sur le banc 192.168.1.7 : les deux exemplaires etaient bit a bit identiques (md5 3363105f…), et seul /opt/GSM/firmware etait charge. Si vous recompilez le firmware dans osmocom-bb, deposez le resultat dans /opt/GSM/firmware : c'est desormais le seul endroit consulte.

2.26.1.5.6 GCC 9 pour osmocom-bb branches expérimentales (jolly/testing, burst_ind)

gcc-9 et gcc-11 sont installés avec le reste, plus haut : ici on ne fait que déclarer les alternatives, dont l'ordre compte pour osmocom-bb.

```
update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-9 90 \
       --slave /usr/bin/g++ g++ /usr/bin/g++-9 \
    && update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-11 110 \
       --slave /usr/bin/g++ g++ /usr/bin/g++-11


update-alternatives --set gcc /usr/bin/gcc-9
```

osmocom-bb jolly/testing → transceiver (BTS soft-SDR pour Calypso) Le binaire ET l arbre partent dans le paquet : les sources restent dans /opt/GSM au rebuild depuis le cache (voir l en-tete, CACHE .deb). [2026-10-19] patches/osmocom-bb-transceiver-qemu-bench.patch : l appli hote `transceiver` sert le Calypso EMULE (start-native.sh –osmo-trx, c54x_exe/run.sh OSMO_TRX=1) : –no-sync (pas de cellule a ecouter, mode BTS des le reset), –ts-mask (un seul timeslot), –l1ctl/–base-port, et SETTSC/SETFORMAT/ NOMTXPOWER acquiesces (osmo-bts-trx >= 1.3 les tient pour critiques). Applique aussi par install_modules/40-patches.sh en natif.

```
mkdir -p "$(dirname /tmp/osmocom-bb-transceiver-qemu-bench.patch)" && cp -r
 ↪  "$REPO"/patches/osmocom-bb-transceiver-qemu-bench.patch /tmp/osmocom-bb-transceiver-qemu-bench.patch


if ! osmo-deb install osmocom-bb-transceiver 0.git; then \
      git clone --branch jolly/testing --depth 1 \
        https://gitea.osmocom.org/phone-side/osmocom-bb.git \
        /opt/GSM/osmocom-bb-transceiver \
      && git -C /opt/GSM/osmocom-bb-transceiver apply /tmp/osmocom-bb-transceiver-qemu-bench.patch \
      && cd /opt/GSM/osmocom-bb-transceiver/src \
      && make HOST_layer23_CONFARGS=--enable-transceiver nofirmware -j$(nproc) \
      && cp /opt/GSM/osmocom-bb-transceiver/src/host/layer23/src/transceiver/transceiver \
         /usr/local/bin/transceiver \
```

```
      && osmo-deb snapshot osmocom-bb-transceiver 0.git /usr/local/bin/transceiver \
             /opt/GSM/osmocom-bb-transceiver; \
    fi
```

osmocom-bb fixeria/burst_ind → ccch_scan / bcch_scan / cell_log

```
if ! osmo-deb install osmocom-bb-burst-ind 0.git; then \
      git clone --branch fixeria/burst_ind --depth 1 \
        https://gitea.osmocom.org/phone-side/osmocom-bb.git \
        /opt/GSM/osmocom-bb-burst_ind \
      && cd /opt/GSM/osmocom-bb-burst_ind/src \
      && make nofirmware -j$(nproc) \
      && cp /opt/GSM/osmocom-bb-burst_ind/src/host/layer23/src/misc/ccch_scan \
         /usr/local/bin/ccch_scan \
      && { cp /opt/GSM/osmocom-bb-burst_ind/src/host/layer23/src/misc/bcch_scan \
         /usr/local/bin/bcch_scan 2>/dev/null || true; } \
      && { cp /opt/GSM/osmocom-bb-burst_ind/src/host/layer23/src/misc/cell_log \
         /usr/local/bin/cell_log 2>/dev/null || true; } \
      && osmo-deb snapshot osmocom-bb-burst-ind 0.git /opt/GSM/osmocom-bb-burst_ind \
           $(ls /usr/local/bin/ccch_scan /usr/local/bin/bcch_scan /usr/local/bin/cell_log 2>/dev/null); \
    fi
```

Retour au compilateur par defaut : ce stage ne sert qu a produire des .deb, mais le laisser sur gcc-9 piegerait quiconque en derive une etape.

```
update-alternatives --set gcc /usr/bin/gcc-11
```

2.26.1.6 Etape 4 : qemu

STAGE qemu — le seul arbre QEMU : /opt/GSM/qosmo (–enable-l1-grgsm)

Dans Docker : `FROM base`. Sur une machine, on continue dans le meme systeme.

ARG ne traverse pas un FROM : les stages qui appellent osmo-deb ou ${ROOT} doivent les redeclarer, sinon ROOT est vide et OSMO_DEB_REFRESH=1 (build.sh –no-cache) ne descend pas jusqu ici.

```
export ROOT=/opt/GSM
```

```
export OSMO_DEB_REFRESH=0
```

QEMU Calypso — RAN virtuel (baseband émulé)

Architecture :

```
  - QEMU émule un SoC Calypso (ARM7TDMI + DSP TMS320C54x)
  - L'ARM exécute le vrai firmware osmocom-bb layer1.highram.elf
  - Le DSP charge le ROM réel (calypso_dsp.txt) au boot
  - bridge.py relaie les bursts entre osmo-bts-trx (UDP 5700-5702)
    et la BSP du DSP (UDP 6702), avec QEMU comme maître d'horloge TDMA
  - Le mobile (layer23) se connecte directement au socket L1CTL
    publié par le firmware via la PTY série de QEMU
```

Voir scripts/run.sh PHY_MODE=qemu pour l'orchestration runtime.

(Le ‘apt-get install python3-venv python3-pip python3-numpy python3-scipy

```
 libglib2.0-dev libpixman-1-dev libslirp-dev socat ninja-build` qui etait ici
 a ete fusionne dans la liste apt-fast en tete de fichier : une seule liste,
 un seul endroit ou la faire evoluer. Aucun paquet perdu.)
```

[2026-09-25] qosmo + c54x_exe + grgsm_exe — remplacent qosmo-grgsm et qosmo-dsp

Les deux forks QEMU sont retires (cf. start-native.sh, environment/paths.env) :

```
  /opt/GSM/qosmo      UN seul arbre QEMU (bbaranoff/qosm0), --enable-l1-grgsm.
                      Il porte run.sh, run_modules, cfgs. Le meme binaire sert
                      au mode --dsp : CALYPSO_DSP_EXTERN=1 y coupe la L1 gr-gsm.
                      `ninja install` pose aussi le lanceur C `qosmo`.
  /opt/GSM/c54x_exe   le C54x HORS de QEMU (mask-ROM TI, sa ROM dans rom/).
                      Il compile les sources de /opt/GSM/qosmo : APRES qosmo.
  /opt/GSM/grgsm_exe  la couche 1 gr-gsm hors QEMU, memes sources qosmo.
```

Snapshot de l arbre qosmo ENTIER, build/ compris : QEMU_BIN pointe sur $OQC_ROOT/build/qemu-system-arm, et QEMU lit build/qemu-bundle pour se relocaliser (voir Dockerfile.lite). L ancien RUN « /opt/GSM/qemu/{build,*.py} » et calypso-ipc-device disparaissent avec qosmo-grgsm : plus rien ne les lit.

```
if ! osmo-deb install qosmo 0.git; then \
      git clone https://github.com/bbaranoff/qosm0 /opt/GSM/qosmo \
      && cd /opt/GSM/qosmo \
      && python3 -m venv /root/.venv-qemu \
      && . /root/.venv-qemu/bin/activate \
      && pip install --no-cache-dir tomli \
      && mkdir -p build && cd build \
      && ../configure --target-list=arm-softmmu --enable-l1-grgsm \
             --prefix=/opt/GSM/qemu-install --disable-werror --disable-docs \
      && make -j$(nproc) \
      && make install \
      && cp /opt/GSM/qemu-install/bin/qemu-system-arm /usr/local/bin/qemu-system-arm \
      && cp /opt/GSM/qemu-install/bin/qosmo /usr/local/bin/qosmo \
      && osmo-deb snapshot qosmo 0.git /opt/GSM/qosmo /opt/GSM/qemu-install \
             /root/.venv-qemu /usr/local/bin/qemu-system-arm /usr/local/bin/qosmo; \
    fi
```

2.26.1.7 Etape 5 : l1

STAGE l1 — les deux couches 1 hors QEMU : c54x_exe et grgsm_exe

Dans Docker : `FROM qemu`. Sur une machine, on continue dans le meme systeme.

ARG ne traverse pas un FROM : les stages qui appellent osmo-deb ou ${ROOT} doivent les redeclarer, sinon ROOT est vide et OSMO_DEB_REFRESH=1 (build.sh –no-cache) ne descend pas jusqu ici.

```
export ROOT=/opt/GSM
```

```
export OSMO_DEB_REFRESH=0
```

Elles compilent les sources de /opt/GSM/qosmo (deja la, ce stage en derive) et se lient a libosmocoding/libosmocore pour c54x_exe. On rapatrie donc /usr/local depuis osmo-core plutot que de faire deriver ce stage de lui : copier /usr/local (quelques centaines de Mo) coute bien moins que de copier l arbre de build de QEMU (~1,5 Go d objets) dans l autre sens.

Copie entre etapes Docker (`--from=osmo-core /usr/local /usr/local`) : sur une seule machine, `/usr/local` est deja en place.

```
ldconfig
```

c54x_exe : le Makefile met -march=native dans CFLAGS - dans une image qui tourne sur d autres CPU que celui du build, c est un SIGILL au demarrage. On garde ses drapeaux, sans celui-la. La ROM est aussi posee en /opt/GSM/calypso_dsp.*.bin, le –rom-dir par defaut de c54x_exe. [2026-10-10] La ROM n est plus dans le depot c54x_exe : le `cp rom/...` tombait sur « cannot stat rom/calypso_dsp.txt ». rom/fetch-rom.sh telecharge le dump FreeCalypso 3606, le convertit en .bin et verifie SHA256SUMS.3606 ; calypso_dsp.txt, c est le dump brut lui-meme, qu il ne garde pas : on le telecharge a part.

```
export FREECALYPSO_URL=ftp://ftp.freecalypso.org/pub/GSM/Calypso
```

```
if ! osmo-deb install c54x-exe 0.git; then \
      git clone https://github.com/bbaranoff/c54x_exe /opt/GSM/c54x_exe \
      && cd /opt/GSM/c54x_exe \
      && make QOSMO=/opt/GSM/qosmo \
             CFLAGS="-O3 -g -Wall -Werror=format -Werror=format-extra-args -Wno-unused-function
↪  -Wno-unused-variable -Wno-unused-but-set-variable -Wno-sign-compare" \
      && FREECALYPSO_URL="$FREECALYPSO_URL" bash rom/fetch-rom.sh --dest /opt/GSM --dest /opt/GSM/c54x_exe/rom \
      && curl -fsS --retry 2 -o /opt/GSM/calypso_dsp.txt "$FREECALYPSO_URL/dsp-rom-3606-dump.txt" \
      && cp /opt/GSM/calypso_dsp.txt rom/calypso_dsp.txt \
      && osmo-deb snapshot c54x-exe 0.git /opt/GSM/c54x_exe \
             /opt/GSM/calypso_dsp.PROM0.bin /opt/GSM/calypso_dsp.PROM1.bin /opt/GSM/calypso_dsp.PROM2.bin \
             /opt/GSM/calypso_dsp.PROM3.bin /opt/GSM/calypso_dsp.DROM.bin /opt/GSM/calypso_dsp.PDROM.bin \
             /opt/GSM/calypso_dsp.Registers.bin /opt/GSM/calypso_dsp.txt; \
    fi


if ! osmo-deb install grgsm-exe 0.git; then \
      git clone https://github.com/bbaranoff/grgsm_exE /opt/GSM/grgsm_exe \
      && cd /opt/GSM/grgsm_exe \
      && make QOSMO=/opt/GSM/qosmo \
      && osmo-deb snapshot grgsm-exe 0.git /opt/GSM/grgsm_exe; \
    fi
```

2.26.1.8 Etape 6 : grgsm-venv

STAGE grgsm-venv — GNU Radio 3.10 + gr-osmosdr + gr-gsm dans /root/.env

gr-gsm se lie a libosmocore/libosmocoding/libosmogsm (son CMakeLists les cherche par pkg-config) : ce stage DOIT donc partir d osmo-core, pas de base. Il reste parallele a `bb` et a `l1`, qui en derivent aussi.

Dans Docker : `FROM osmo-core`. Sur une machine, on continue dans le meme systeme.

ARG ne traverse pas un FROM : les stages qui appellent osmo-deb ou ${ROOT} doivent les redeclarer, sinon ROOT est vide et OSMO_DEB_REFRESH=1 (build.sh –no-cache) ne descend pas jusqu ici.

```
export ROOT=/opt/GSM


export OSMO_DEB_REFRESH=0
```

2.26.1.8.1 gr-gsm : GNU Radio 3.10 + gr-osmosdr + gr-gsm dans le venv /root/.env

(= moteur de démod du SI réel utilisé par si_bridge.py / grgsm_decode). Les deps GNU Radio (apt build-dep) sont posees par l unique bloc apt en tete de fichier, deb-src compris : plus aucun apt ici.

2.26.1.8.2 GNU Radio + gr-osmosdr + gr-gsm, en UN paquet : le venv /root/.env

On TÉLÉCHARGE et on exécute CE script (le gist, pinné au commit fcdb409). Il cree /root/.env et y installe les trois. Puis :

Patch gr-gsm : le receiver poste le BSIC/FN du SCH (decode_sch) sur le port `measurements` ET sur stdout (“SCHBSIC ”). Le shunt DSP le recoit (si_bridge.py parse le stdout de grgsm_decode -> UDP 4731 -> feed_sb) et encode le VRAI BSIC dans dispatch_sb (remplace SHUNT_CANNED_BSIC 63). Applique APRES le gist (qui clone+build gr-gsm propre), puis recompile/reinstalle dans le venv. Patch maintenu dans patches/ (regenere a chaque changement gr-gsm).

matplotlib est consomme par tools/fft_global.sh et tools/matrix.sh.

Le paquet du cache (grgsm-venv) est le venv COMPLET, patch et matplotlib compris : c est pour cela que les trois etapes tiennent dans un seul RUN. Les arbres /opt/GSM/{gnuradio,gr-osmosdr,gr-gsm} ne sont pas dans le paquet - rien ne les lit au runtime (le venv porte ses .so avec un RPATH sur /root/.env).

```
mkdir -p "$(dirname /tmp/grgsm-receiver-publish-bsic-fn.patch)" && cp -r
↪  "$REPO"/patches/grgsm-receiver-publish-bsic-fn.patch /tmp/grgsm-receiver-publish-bsic-fn.patch
```

```
if ! osmo-deb install grgsm-venv 0.git; then \
      curl -fsSL
       ↪  https://gist.githubusercontent.com/bbaranoff/3683811057933af0954b661821e950d1/raw/fcdb4092483ec383440b67fc002db0c1583
       ↪  | bash \
      && git -C /opt/GSM/gr-gsm apply /tmp/grgsm-receiver-publish-bsic-fn.patch \
      && cd /opt/GSM/gr-gsm/build \
      && make -j"$(nproc)" \
      && make install \
      && . ~/.env/bin/activate && pip install matplotlib \
      && osmo-deb snapshot grgsm-venv 0.git /root/.env /etc/ld.so.conf.d/gnuradio.conf; \
    fi && ldconfig
```

2.26.1.9 Etape 7 : node

STAGE node — Node 22 et le dashboard web osmo-egprs-web

Dans Docker : `FROM base`. Sur une machine, on continue dans le meme systeme.

ARG ne traverse pas un FROM : les stages qui appellent osmo-deb ou ${ROOT} doivent les redeclarer, sinon ROOT est vide et OSMO_DEB_REFRESH=1 (build.sh –no-cache) ne descend pas jusqu ici.

```
export ROOT=/opt/GSM
```

```
export OSMO_DEB_REFRESH=0
```

Node.js + dashboard web osmo-egprs-web (ex-Dockerfile.run, chaine complete)

Place EN FIN DE FICHIER a dessein : aucune etape du build ne depend de Node, et la fin de fichier preserve le cache des couches longues de compilation. L'ORDRE INTERNE DE CETTE CHAINE EST CONTRAINT — voir les ⚠ ci-dessous. /opt/GSM et pas /opt : c'est /opt/GSM que l'ISO recupere en bloc (docker cp "$CID:/opt/GSM"). Le dashboard vivait a cote, dans /opt, et devait donc etre reclone une seconde fois a la construction de l'image - deux sources pour le meme depot, qui n'avancaient pas ensemble.

```
if [ ! -d "/opt/GSM/osmo-egprs-web/" ]; then \
      mkdir -p /opt/GSM && \
      git clone -b main https://github.com/bbaranoff/osmo-egprs-web /opt/GSM/osmo-egprs-web; \
    fi
```

2.26.1.9.1 Runtime Node.js + service web osmo-egprs-web

Le dashboard /opt/GSM/osmo-egprs-web/server.js tourne en mode NATIF (telnet VTY local, pas de docker) et est servi sur :8080. Le DÉMARRAGE est géré par start-native.sh (`systemctl restart osmo-egprs-web`) — ici on ne fait qu'INSTALLER le runtime + les dépendances + le unit (enable au boot).

Node 22 (LTS « Jod ») — même famille majeure que l'ISO, qui installe nodesource setup_22.x (build-iso.sh). Garder les deux alignés : le dashboard est le même server.js des deux côtés.

```
export NODE_VERSION=v22.23.2
```

Le tarball suit l architecture de l image (x64 sur amd64, arm64 sur un build –platform linux/arm64 pour le Raspberry Pi) : un node x64 dans une image arm64 ne se lancerait pas, et le dashboard avec lui.

```
case "$(uname -m)" in x86_64) _na=x64 ;; aarch64) _na=arm64 ;; *) echo "arch node inconnue: $(uname -m)" >&2; exit
 ↪  1 ;; esac && \
    curl -fsSL "https://nodejs.org/dist/${NODE_VERSION}/node-${NODE_VERSION}-linux-${_na}.tar.xz" \
        -o /tmp/node.tar.xz && \
    mkdir -p /opt/node && \
    tar -xJf /tmp/node.tar.xz -C /opt/node --strip-components=1 && \
    rm -f /tmp/node.tar.xz && \
    ln -sf /opt/node/bin/node /usr/local/bin/node && \
    ln -sf /opt/node/bin/npm  /usr/local/bin/npm && \
    ln -sf /opt/node/bin/npx  /usr/local/bin/npx && \
    node --version
```

Unit systemd (fichier versionné dans le repo osmo-nitb-for-calypso, source unique). Le START reste géré par start-native.sh (`systemctl restart osmo-egprs-web`).

```
mkdir -p "$(dirname /etc/systemd/system/osmo-egprs-web.service)" && cp -r "$REPO"/services/osmo-egprs-web.service
 ↪  /etc/systemd/system/osmo-egprs-web.service


ln -sf /etc/systemd/system/osmo-egprs-web.service \
        /etc/systemd/system/multi-user.target.wants/osmo-egprs-web.service
```

2.26.1.9.2 Dashboard web : install-web-service.sh joue AU BOOT

[2026-08-12] Remplace le geste manuel `bash /opt/GSM/osmo-egprs-web/install-web-service.sh` qu'il fallait refaire dans chaque conteneur pour armer le HTTPS.

Ce qui reste au BUILD : node et le unit du service (plus haut), le `npm install` (juste en dessous). Ce qui passe au BOOT : le certificat TLS auto-signe — une cle privee generee au build serait la meme pour tous ceux qui tirent l'image. Le detail du raisonnement est dans services/osmo-egprs-web-install.service.

⚠ ORDRE : ce bloc DOIT rester APRES le clone de /opt/GSM/osmo-egprs-web (haut de cette section) — la COPY ci-dessous ecrit dans ce depot.

SOURCE UNIQUE DU UNIT. install-web-service.sh installe le unit en le copiant depuis /opt/GSM/osmo-egprs-web/osmo-egprs-web.service — la copie du depot osmo-egprs-web, qui avait DIVERGE de celle-ci (elle avait perdu `CAP_IFACE=any`, donc la capture du dashboard). On ecrase donc cette copie par services/osmo-egprs-web.service : les deux emplacements servent desormais le meme fichier, et le script ne peut plus reintroduire la regression.

```
mkdir -p "$(dirname /opt/GSM/osmo-egprs-web/osmo-egprs-web.service)" && cp -r
 ↪  "$REPO"/services/osmo-egprs-web.service /opt/GSM/osmo-egprs-web/osmo-egprs-web.service
```

Fail-fast : si l'amont retire le script, on le sait au build, pas au boot par un HTTPS muet.

```
test -s /opt/GSM/osmo-egprs-web/install-web-service.sh
```

Dependances JS (ws) — UN SEUL `npm install`, apres le clone. Non fatal (`|| true`) : node_modules est versionne dans le depot, un build hors-ligne reste valable. Mais la sortie est conservee, pas avalee.

```
cd /opt/GSM/osmo-egprs-web && npm install --omit=dev --no-audit --no-fund || true
```

2.26.1.9.3 L INSTALLATION EST JOUEE ICI, AU BUILD

[2026-08-31] On se contentait de VERIFIER que le script existe (le `test -s` ci-dessus) et on remettait tout au boot. L unite arrivait donc dans l image telle que le gabarit la portait, avec son ExecStart fige — et dans un conteneur, ou node vit en /usr/local/bin et non /usr/bin, systemd sortait en

```
    status=203/EXEC
```

en boucle, sans que rien ne parle d un chemin.

Le script sait resoudre node et poser l unite avec le bon chemin : on le joue donc au build, pour que l image contienne une unite deja juste.

`|| true` : dans un build il n y a PAS de systemd. Le script le sait et ne s arrete pas dessus (voir son en-tete), mais ses `systemctl` echouent forcement ; les avaler ici evite qu un detail d environnement fasse tomber une image de 11 Go. Ce qui compte - le unit, le certificat, le runtime - est ecrit avant ces appels.

⚠ ET IL N Y A PLUS DE REJEU AU BOOT. osmo-egprs-web-install.service a ete retire des trois chemins de deploiement, pour une raison simple : ce script TELECHARGE node quand il manque. Le laisser au demarrage faisait dependre d Internet un banc concu pour tourner isole - et un premier boot hors ligne repartait sans dashboard. Tout est desormais fige dans l image ; le demarrage ne fait plus que lancer osmo-egprs-web.service.

```
bash /opt/GSM/osmo-egprs-web/install-web-service.sh || true
```

2.26.1.10 Etape 8 : lte

STAGE lte — la 4G du banc : srsRAN_4G (ZeroMQ) et Open5GS dans /opt/LTE

Dans Docker : `FROM base`. Sur une machine, on continue dans le meme systeme.

ARG ne traverse pas un FROM : les stages qui appellent osmo-deb ou ${ROOT} doivent les redeclarer, sinon ROOT est vide et OSMO_DEB_REFRESH=1 (build.sh –no-cache) ne descend pas jusqu ici.

```
export ROOT=/opt/GSM
```

```
export OSMO_DEB_REFRESH=0
```

osmo-lte-install –configs lit $OSMO_REPO : ce stage a besoin du depot, et il ne derive pas de celui qui le clone. Le clone y est donc refait — c est quelques secondes, et l assemblage refait le sien, qui reste LA source pour le `git pull` de Dockerfile.run.

```
git clone https://github.com/bbaranoff/osmo-operator /opt/GSM/osmo-operator
```

2.26.1.10.1 LA 4G DU BANC : srsRAN_4G (ZeroMQ) et Open5GS, dans /opt/LTE

[2026-09-08] DEUX CHEMINS VERS LES MEMES PAQUETS (voir packaging/ snapshot-lte-debs.sh) : le raccourci photographie le natif de la machine de reference en osmo-build-libzmq / osmo-build-srsran / osmo-build-open5gs, et le cache .deb les rend a `osmo-deb install` ci-dessous - rien n est alors compile. Cache vide : on compile depuis les sources, par le MEME script que le natif (tools/osmo-lte-install.sh –build, OSMO_DEB=1 -> osmo-deb pack). libzmq vient d Ubuntu (libzmq3-dev, pose par `--deps` ci-dessous) : le /opt/LTE/libzmq compile a la main n a de raison d etre que sur le natif. Open5GS : prefixe /opt/LTE/open5gs/install (bin, etc/open5gs, var/log) - pas /root.

LES CONFIGS ET LES LANCEURS AUSSI (`--configs --launchers`, en fin de RUN). [2026-09-09] Ils n etaient poses que dans l ISO (88-lte-pmos.sh) et en natif (addition.sh), « pas ici : l image docker n a pas de HOME de session a servir ». Faux pour le conteneur : il tourne en root, et /root/.config/srsran EST le HOME que srsenb/srsue lisent. L image sortait donc avec les binaires 4G mais sans mme.yaml (donc sans SGs, donc sans CSFB), sans enb.conf, et sans la commande osmo-lte : une 4G qu on ne pouvait pas lancer. Et le .deb Open5GS n apporte AUCUN yaml (ninja install sous DESTDIR ne pose pas les exemples : 128 fichiers, tous dans bin/ et lib/) - les configs du depot sont la seule source. Elles viennent du clone GitHub de /opt/GSM/ osmo-operator (ligne « git clone » plus haut) : ce qui est pousse est ce que le conteneur voit. Les abonnes (configs/open5gs/dump, subscribers.json) ne sont pas poses au build : osmo-epc start les restaure dans MongoDB.

⚠ `--deps` D ABORD, TOUJOURS (pas seulement quand on compile). [2026-09-09] Le commentaire ci-dessus affirmait que libzmq3-dev etait « dans la liste apt plus haut » : il n y etait pas, ni lui ni libmbedtls-dev, libboost-program-options-dev, libconfig++-dev, meson, flex, bison, libmongoc. Le build tombait donc net des le cmake de srsRAN :

```
    Could NOT find MbedTLS (missing: MBEDTLS_LIBRARIES MBEDTLS_INCLUDE_DIRS)
```

La liste apt du haut de ce fichier est celle de la pile Osmocom (et elle est RELUE par install_modules/10-deps.sh pour le natif) : la 4G a la sienne, tenue dans tools/osmo-lte-install.sh (lte_deps), une seule fois pour les trois chemins. On l appelle, on ne la recopie pas. Inconditionnel parce que ces paquets sont aussi le RUNTIME : les .deb du cache ne portent que les binaires (osmo-deb pack = un tar), pas leurs dependances - srsenb sorti du cache reclame quand meme libmbedtls, libzmq5, libconfig++ et boost_program_options. `--deps` tire aussi mongodb-org (base d abonnes du HSS/PCRF) et n est fatal sur aucun paquet.

```
mkdir -p "$(dirname /usr/local/sbin/osmo-lte-install)" && cp -r "$REPO"/tools/osmo-lte-install.sh
 ↪  /usr/local/sbin/osmo-lte-install
```

```
chmod 755 /usr/local/sbin/osmo-lte-install && \
    OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --deps && \
    { osmo-deb install libzmq 4.3.5+git || true; } && \
    { osmo-deb install srsgui 0.1+git || true; } && \
    SRS_DEB_VER="$(osmo-lte-install --srs-deb-version)" && \
    if ! osmo-deb install srsran "$SRS_DEB_VER"; then \
        OSMO_DEB=1 OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --build || { echo "ECHEC build srsRAN"; exit
        ↪  1; }; \
    fi && \
    if ! osmo-deb install open5gs 2.8.0+git; then \
        OSMO_DEB=1 OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --build || { echo "ECHEC build Open5GS"; exit
        ↪  1; }; \
    fi && \
    OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --configs --launchers && \
    ldconfig && test -x /usr/local/bin/srsenb && test -x /opt/LTE/open5gs/install/bin/open5gs-mmed \
    && test -s /opt/LTE/open5gs/install/etc/open5gs/mme.yaml && grep -q '^  sgsap:'
    ↪  /opt/LTE/open5gs/install/etc/open5gs/mme.yaml \
    && test -s /opt/LTE/open5gs/install/etc/freeDiameter/mme.conf && test -s
    ↪  /opt/LTE/open5gs/install/etc/open5gs/tls/mme.crt \
    && test -s /root/.config/srsran/enb.conf && test -x /usr/local/bin/osmo-lte && test -x /usr/local/bin/osmo-epc
```

2.26.1.10.2 L UI SMARTPHONE : pmbootstrap PATCHE, dans /opt/user_interface/pmos

Le telephone du banc est une VM postmarketOS lancee par pmbootstrap (patche pour le modem serie PCI, le son du banc, la taille d ecran, et le port 5038 qui n est pas un adb : patches/pmbootstrap-osmo-bench-qemu.patch). Le clone et le patch sont faits ici, une fois, et sortent en osmo-build-pmbootstrap : l ISO (50-injection-image.sh) le pose tel quel, tools/osmo-pmos-install.sh le trouve en place et n a plus rien a cloner. Le NOYAU PPP, lui, ne se construit pas ici (pmbootstrap, non root, montages de boucle) : il arrive par osmo-build-pmos-kernel (packaging/build-pmos-kernel-deb.sh, ou GitHub bbaranoff/pmos_ppp_kernel), pose par 88-lte-pmos.sh.

```
mkdir -p "$(dirname /tmp/pmbootstrap-osmo-bench-qemu.patch)" && cp -r
↪  "$REPO"/patches/pmbootstrap-osmo-bench-qemu.patch /tmp/pmbootstrap-osmo-bench-qemu.patch


if ! osmo-deb install pmbootstrap 0.git; then \
        mkdir -p /opt/user_interface/pmos && \
        git clone https://gitlab.postmarketos.org/postmarketOS/pmbootstrap.git
        ↪  /opt/user_interface/pmos/pmbootstrap && \
        cd /opt/user_interface/pmos/pmbootstrap && \
        git apply /tmp/pmbootstrap-osmo-bench-qemu.patch && \
        grep -q osmo_bench_args pmb/commands/qemu.py && \
        osmo-deb snapshot pmbootstrap 0.git /opt/user_interface/pmos/pmbootstrap; \
    fi && test -f /opt/user_interface/pmos/pmbootstrap/pmbootstrap.py
```

2.26.1.11 Etape 9 : osmocom-nitb

========== STAGE osmocom-nitb — L ASSEMBLAGE. C est l image publiee. ==========
Rien ne se compile ici : on repose les .deb produits par les stages, puis on ajoute ce qu aucun paquet ne porte (Node, le dashboard, la 4G, le depot, les configs du contexte). Le nom osmocom-nitb reste sur CETTE etape : build.sh (compose_build) et compose.yaml passent –target osmocom-nitb.

Dans Docker : `FROM base`. *Sur une machine, on continue dans le meme systeme.*

```
export ROOT=/opt/GSM
```

2.26.1.11.1 1. Les paquets de tous les stages

bb DERIVE de osmo-core et l1 DERIVE de qemu : leur cache porte deja celui de leur parent, d ou cinq COPY et non sept.

Copie entre etapes Docker (`--from=bb /var/cache/osmo-debs/ /var/cache/osmo-debs/`) : sur une seule machine, `/var/cache/osmo-debs/` est deja en place.

Copie entre etapes Docker (`--from=l1 /var/cache/osmo-debs/ /var/cache/osmo-debs/`) : sur une seule machine, `/var/cache/osmo-debs/` est deja en place.

Copie entre etapes Docker (`--from=grgsm-venv /var/cache/osmo-debs/ /var/cache/osmo-debs/`) : sur une seule machine, `/var/cache/osmo-debs/` est deja en place.

Copie entre etapes Docker (`--from=lte /var/cache/osmo-debs/ /var/cache/osmo-debs/`) : sur une seule machine, `/var/cache/osmo-debs/` est deja en place.

(pas de COPY depuis `node` : ce stage n appelle jamais osmo-deb - Node est un

```
 tarball et le dashboard un clone git. Son cache n est que celui de base.)
```

⚠ L ORDRE DE POSE COMPTE POUR LES QUATRE PAQUETS PATCHES. osmo-trx, osmo-bts, osmo-hlr et osmo-msc existent en DEUX versions portant le MEME nom de paquet : celle de la boucle (osmo-trx 1.7.2) et la patchee (1.7.2+ipc). Dans la chaine lineaire d avant, la patchee etait posee apres, donc elle gagnait. Ici le glob les pose dans l ordre ASCII, ou « + » (0x2B) precede « ~ » (0x7E) : osmo-trx_1.7.2+ipc passe AVANT osmo-trx_1.7.2, et c est la version NON patchee qui finissait installee - RACH sans table per-RA, BTS/HLR/MSC sans le forcage du RAND, et une LU qui ne passe plus. Silencieux : le binaire est la, il est juste le mauvais. On repose donc les quatre explicitement, en dernier.

```
set -eux; \
    dpkg -i --force-overwrite /var/cache/osmo-debs/osmo-build-*.deb; \
    dpkg -i --force-overwrite \
        /var/cache/osmo-debs/osmo-build-osmo-trx_1.7.2+ipc~*.deb \
        /var/cache/osmo-debs/osmo-build-osmo-bts_1.10.0+rand~*.deb \
        /var/cache/osmo-debs/osmo-build-osmo-hlr_1.9.2+rand~*.deb \
        /var/cache/osmo-debs/osmo-build-osmo-msc_1.15.0+rand~*.deb; \
    ldconfig
```

2.26.1.11.2 2. Ce qu aucun .deb ne porte

Node et le dashboard : le runtime est un tarball detarre dans /opt/node, le dashboard un clone git avec son node_modules. Ni l un ni l autre ne passe par osmo-deb, et install-web-service.sh a deja ete joue dans le stage node.

Copie entre etapes Docker (`--from=node /opt/node /opt/node`) : sur une seule machine, `/opt/node` est deja en place.

Copie entre etapes Docker (`--from=node /opt/GSM/osmo-egprs-web /opt/GSM/osmo-egprs-web`) : sur une seule machine, `/opt/GSM/osmo-egprs-web` est deja en place.

Copie entre etapes Docker (`--from=node /etc/systemd/system/osmo-egprs-web.service /etc/systemd/system/osmo-egprs-web.service`) : sur une seule machine, `/etc/systemd/system/osmo-egprs-web.service` est deja en place.

```
ln -sf /opt/node/bin/node /usr/local/bin/node \
    && ln -sf /opt/node/bin/npm  /usr/local/bin/npm \
    && ln -sf /opt/node/bin/npx  /usr/local/bin/npx \
    && ln -sf /etc/systemd/system/osmo-egprs-web.service \
        /etc/systemd/system/multi-user.target.wants/osmo-egprs-web.service \
    && node --version
```

Chemin EXPLICITE. Sans lui, ce clone heritait du WORKDIR /etc/osmocom (pose beaucoup plus haut, jamais remis a ${ROOT}) et atterrissait dans /etc/osmocom/osmo-operator — masque au runtime par le montage de start-docker.sh l.505, et surtout PAS la ou Dockerfile.run va faire son `git pull`. Deux arbres, deux HEAD, un seul utilise. /opt/GSM/osmo-operator est le chemin nominal, teste en premier par build-iso.sh l.964 et update.sh l.329. On reste sur la branche par defaut du depot (main) : pas de checkout explicite, donc pas de ref a maintenir ici, et HEAD est attache — ce dont le `git pull` de Dockerfile.run a besoin.

```
git clone https://github.com/bbaranoff/osmo-operator /opt/GSM/osmo-operator
```

```
mkdir -p "$(dirname /usr/local/sbin/osmo-lte-install)" && cp -r "$REPO"/tools/osmo-lte-install.sh
 ↪  /usr/local/sbin/osmo-lte-install
```

```
chmod 755 /usr/local/sbin/osmo-lte-install
```

2.26.1.11.3 La 4G

/opt/LTE en entier, et pas seulement ce que portent les .deb : le prefixe d installation d Open5GS vit SOUS l arbre de sources (/opt/LTE/open5gs/install), et /opt/LTE n est pas sous /opt/GSM - osmo-deb pack n y ajoute donc aucune source (SRC_ROOT). Sans cette copie, l image aurait les binaires 4G sans un seul arbre pour les recompiler.

Copie entre etapes Docker (`--from=lte /opt/LTE /opt/LTE`) : sur une seule machine, `/opt/LTE` est deja en place.

⚠ `--deps` DOIT ETRE REJOUE ICI. Ce sont AUSSI les paquets de RUNTIME (libzmq5, libmbedtls, libconfig++, boost_program_options, mongodb-org…) et les .deb d osmo-deb ne declarent aucune dependance - leur en-tete le dit. Les poser dans le seul stage `lte` donnait une image ou srsenb et open5gs-mmed existent mais ne demarrent pas, faute de .so. Le montage du cache apt fait que rien n est retelecharge : les paquets sont deja dans le cache BuildKit, poses par le stage lte.

`--configs --launchers` ensuite, dans le meme ordre que le stage lte : les yaml d Open5GS, enb.conf et les deux lanceurs s ecrivent HORS de tout DESTDIR, donc aucun .deb ne les porte. Ils lisent /opt/GSM/osmo-operator, clone juste au-dessus.

```
OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --deps && \
    OSMO_REPO=/opt/GSM/osmo-operator osmo-lte-install --configs --launchers && \
    ldconfig && test -x /usr/local/bin/srsenb \
    && test -x /opt/LTE/open5gs/install/bin/open5gs-mmed \
    && test -s /opt/LTE/open5gs/install/etc/open5gs/mme.yaml && grep -q '^  sgsap:'
     ↪  /opt/LTE/open5gs/install/etc/open5gs/mme.yaml \
    && test -s /opt/LTE/open5gs/install/etc/freeDiameter/mme.conf \
    && test -s /opt/LTE/open5gs/install/etc/open5gs/tls/mme.crt \
    && test -s /root/.config/srsran/enb.conf && test -x /usr/local/bin/osmo-lte && test -x /usr/local/bin/osmo-epc
```

Les alternatives gcc : elles ont ete DECLAREES dans le stage bb, qui ne transmet que ses .deb. On les redeclare ici pour que l image finale offre le meme choix, et on reste sur gcc-11 - le defaut d avant le decoupage.

```
update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-9 90 \
       --slave /usr/bin/g++ g++ /usr/bin/g++-9 \
    && update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-11 110 \
       --slave /usr/bin/g++ g++ /usr/bin/g++-11 \
    && update-alternatives --set gcc /usr/bin/gcc-11
```

2.26.1.11.4 4. Installation des fichiers du projet

```
cd /etc/osmocom


mkdir -p /etc/osmocom/ && cp -r "$REPO"/scripts/. /etc/osmocom/


mkdir -p /etc/osmocom/ && cp -r "$REPO"/configs/*cfg /etc/osmocom/


mv /etc/osmocom/run.sh /root/run.sh
```

Copie des binaires vers /usr/bin pour systemd et installation des .service (La `COPY scripts/gapk-start.sh /etc/osmocom/gapk-start.sh` qui etait ici est

```
 supprimee : `COPY scripts/. /etc/osmocom/` ci-dessus depose deja le fichier.
 Le RUN ci-dessous reste : le symlink et /var/lib/gapk sont uniques.)


chmod +x /etc/osmocom/gapk-start.sh && \
    ln -sf /etc/osmocom/gapk-start.sh /usr/local/bin/gapk-start.sh && \
    mkdir -p /var/lib/gapk


cp -f /usr/local/bin/osmo* /usr/bin/ || true && \
    cp -f /usr/local/bin/proto-smsc-* /usr/bin/ || true && \
    cp -f /usr/local/bin/sms-* /usr/bin/ || true && \
    cp -f /usr/local/bin/gen-sms-* /usr/bin/ || true && \
    # Si tu as des fichiers .service dans configs/
    cp /etc/osmocom/configs/*.service /lib/systemd/system/ 2>/dev/null || true
```

2.26.1.11.5 Vérification binaires proto-SMSC

Ex-Dockerfile.run. Premier point du build ou les deux chemins testes sont peuples (produits par gsup-smsc-proto, recopies dans /usr/bin juste au-dessus).

```
which proto-smsc-daemon && which proto-smsc-sendmt && \
    echo "proto-smsc binaries OK" || \
    echo "WARNING: proto-smsc binaries not found in PATH"
```

2.26.1.11.6 5. Fix Permissions & Systemd (Status 214/217)

```
sed -i 's/^CPUScheduling/#CPUScheduling/g' /lib/systemd/system/osmo-*.service && \
    sed -i 's/User=osmocom/User=root/g' /lib/systemd/system/osmo-ggsn.service && \
    sed -i 's/User=osmocom/User=root/g' /lib/systemd/system/osmo-sgsn.service && \
    chmod +x /etc/osmocom/*.sh
```

Activation du service et nettoyage Les unites systemd vivent dans services/ ; elles doivent atterrir dans /etc/systemd/system/, seul repertoire ou systemd les cherche.

```
mkdir -p "$(dirname /etc/systemd/system/osmo-bts-trx.service)" && cp -r "$REPO"/services/osmo-bts-trx.service
 ↪  /etc/systemd/system/osmo-bts-trx.service


systemctl enable osmo-bts-trx.service && \
    passwd -d root && \
    systemctl mask getty@tty1.service serial-getty@tty1.service
```

Binaires cote hote produits par osmocom-bb, deposes en une seule couche. `install -m755` plutot que `cp` : les droits sont explicites, pas herites.

```
mkdir -p /root/.osmocom/bb/ \
    && install -m755 ${ROOT}/osmocom-bb/src/host/trxcon/src/trxcon           /usr/local/bin/trxcon \
    && install -m755 ${ROOT}/osmocom-bb/src/host/layer23/src/mobile/mobile   /usr/local/bin/mobile \
    && install -m755 ${ROOT}/osmocom-bb/src/host/virt_phy/src/virtphy        /usr/local/bin/virtphy
```

ccch_scan n'est plus installe ici : la branche fixeria/burst_ind, en fin de fichier, ecrase de toute facon /usr/local/bin/ccch_scan par SA version (cp sans `|| true`, donc obligatoire). Le binaire de master ne survivait jamais — c'est la version burst_ind qui fait foi. Confort du shell interactif. Pas de `source` ici : chaque RUN est un shell neuf, ce serait sans effet — le fichier est lu a l'ouverture d'une session.

```
echo "alias faketrx='python3 ${ROOT}/osmocom-bb/src/target/trx_toolkit/fake_trx.py'" >> ~/.bashrc
```

Prompt : root(rouge)@(bleu)<nom-container=\h>(jaune) (vert)# \h = nom du container grâce à `docker run --hostname "$container_name"` (start-docker.sh). [refactor] LE PS1 ETAIT DEFINI DEUX FOIS, avec deux formats DIFFERENTS : ici et dans Dockerfile.run. Bash lit ~/.bashrc de haut en bas -> c'est celui de Dockerfile.run qui gagnait, et le prompt reellement affiche etait `user@host:cwd#`, pas celui decrit deux lignes plus haut. On ne garde QUE le gagnant, a l'identique, pour ne rien changer au prompt visible ; l'explication du `\h` ci-dessus reste vraie pour les deux formats.

2.26.1.11.7 Prompt interactif (root rouge / hostname bleu / cwd vert / #)

```
printf '%s\n' '' \
    '### calypso-prompt ###' \
    "export PS1='\[\033[1;31m\]\u\[\033[0m\]@\[\033[1;34m\]\h\[\033[0m\]:\[\033[1;32m\]\w\[\033[0m\]# '" \
    '### end calypso-prompt ###' >> /root/.bashrc


mkdir -p "$(dirname /root/.osmocom/bb/mobile.cfg)" && cp -r "$REPO"/configs/mobile.cfg
 ↪  /root/.osmocom/bb/mobile.cfg


chmod +x /root/run.sh
```

Répertoires pour le proto-SMSC + arborescence Asterisk (ex-Dockerfile.run, ou ce mkdir faisait doublon avec celui-ci ; seul le chmod 755 etait unique). Position : apres l'installation d'asterisk (liste apt) et apres WORKDIR /etc/osmocom.

```
mkdir -p /var/log/osmocom /var/run/smsc \
        /etc/asterisk /var/lib/asterisk /var/log/asterisk /var/run/asterisk \
        /root/.osmocom/bb && \
    chmod 755 /etc/asterisk /var/lib/asterisk /var/log/asterisk \
        /var/run/asterisk /var/log/osmocom
```

2.26.1.11.8 PulseAudio system-mode : config bakée au build

(ex-Dockerfile.run — depend du paquet pulseaudio de la liste apt en tete ;

```
 place ici, tard, pour ne pas invalider le cache des ~200 lignes de compilation.)
```

Cause racine du warning "[audio] PulseAudio injoignable — audio dégradé" : le démon system-mode lancé au runtime tournait SANS auth-anonymous (le patch de start-native.sh arrivait après son démarrage) → 'pactl' renvoyait "Access denied", et un 2e démon ne pouvait pas démarrer ("Daemon already running"). On bake ici la config pour que TOUT démon soit joignable dès son lancement : socket anonyme + null-sinks gsm_audio/gsm_mic + root dans pulse-access.

[2026-08-12] gsm_mic AJOUTÉ ICI. Il n'était créé que par scripts/pulse-gsm-setup.sh, appelé tard dans run.sh (après osmo-start.sh) : un HLR qui rate coupe run.sh (set -e) et le sink n'existe jamais. Or configs/asound.conf fait pointer la CAPTURE gsm_in sur gsm_mic.monitor, et gapk_io abandonne lecture ET capture si la capture échoue → appel muet des deux côtés. Les deux sinks doivent naître avec le démon, pas avec un script.

```
set -eux; \
    usermod -aG pulse-access root 2>/dev/null || true; \
    test -f /etc/pulse/system.pa; \
    sed -i 's|^#\?load-module module-native-protocol-unix.*|load-module module-native-protocol-unix
     ↪  auth-anonymous=1 socket=/var/run/pulse/native|' /etc/pulse/system.pa; \
    grep -q 'auth-anonymous=1' /etc/pulse/system.pa \
      || echo 'load-module module-native-protocol-unix auth-anonymous=1 socket=/var/run/pulse/native' >>
       ↪  /etc/pulse/system.pa; \
    grep -q 'sink_name=gsm_audio' /etc/pulse/system.pa \
      || echo 'load-module module-null-sink sink_name=gsm_audio format=s16le rate=8000 channels=1
       ↪  sink_properties=device.description=GSM_Audio' >> /etc/pulse/system.pa; \
    grep -q 'sink_name=gsm_mic' /etc/pulse/system.pa \
      || echo 'load-module module-null-sink sink_name=gsm_mic format=s16le rate=8000 channels=1
       ↪  sink_properties=device.description=GSM_Mic' >> /etc/pulse/system.pa; \
    grep -q 'sink_name=osmo_tts_off' /etc/pulse/system.pa \
      || echo 'load-module module-null-sink sink_name=osmo_tts_off format=s16le rate=8000 channels=1
       ↪  sink_properties=device.description=TTS_off' >> /etc/pulse/system.pa; \
    sed -i 's|^load-module module-suspend-on-idle|#load-module module-suspend-on-idle|' /etc/pulse/system.pa; \
    mkdir -p /var/run/pulse && chown -R pulse:pulse /var/run/pulse
```

Dernier maillon de la chaine venv/gr-gsm (ex-Dockerfile.run) : le profil de root active le venv.

```
echo 'source ~/.env/bin/activate' >> ~/.bashrc
```

2.26.1.11.9 scripts bridge camping -> /opt/GSM (sinon /opt/GSM/qosmo/run.sh casse)

si_bridge.py (full SI set -> 4730 -> shunt feed_si), si_bridge_loop.sh, record_drain.py (iq_record.fifo -> record.cfile), grgsm_fft_live.py.

```
mkdir -p /opt/GSM/ && cp -r "$REPO"/opt-gsm/. /opt/GSM/
```

2.26.1.11.10 Metadonnees de l'image

Regroupees a la fin : elles decrivent le conteneur qui tournera, pas une etape de construction. SIGRTMIN+3 est le signal d'arret propre de systemd.

Metadonnee Docker, sans equivalent sur une machine : `STOPSIGNAL SIGRTMIN+3`.

Metadonnee Docker, sans equivalent sur une machine : `ENTRYPOINT ["/etc/osmocom/entrypoint.sh"]`.

Metadonnee Docker, sans equivalent sur une machine : `CMD ["/bin/bash"]`.

2.26.1.12 Etape 10 : debs

2.26.1.12.1 L'ETAPE `debs` : le cache .deb seul, rien d'autre

ATTENTION - `osmocom-nitb` N'EST PLUS LA DERNIERE ETAPE de ce fichier. Tout ce qui construit l'image doit donc dire `--target osmocom-nitb` : build.sh (compose_build), compose.yaml (build.target du service nitb), iso_modules/21-docker-build.sh, .github/workflows/docker.yml. Sans ce –target, docker batit `debs` et sort une image VIDE de 2 Go de paquets.

Pourquoi cette etape existe. En CI, l'image de base n'est plus chargee dans le docker du runner : elle part directement vers GHCR (buildx –push), parce que la materialiser localement demandait de tenir DEUX fois 11 Go et tuait le job en « exporting layers » (exit 143). Mais le workflow doit quand meme recuperer les .deb que le build vient de compiler, pour les rendre a actions/cache - et sans image locale, plus de `docker create` ni de `docker cp` (l'equivalent de debs_from_image, build.sh:306).

D'ou cette etape : une seconde passe `--target debs --output type=local` reutilise INTEGRALEMENT le cache de la premiere (meme builder, memes couches, tout en hit) et n'ecrit sur le disque du runner que /var/cache/osmo-debs - quelques Go de .deb au lieu de l'image entiere. Sur l'hote, build.sh continue de passer par debs_from_image : il a l'image sous la main, lui.

Etape Docker sans contenu propre (image `scratch`) : rien a faire sur une machine.

Copie entre etapes Docker (`--from=osmocom-nitb /var/cache/osmo-debs/ /`) : sur une seule machine, `/var/cache/osmo-debs/` est deja en place.

2.26.2 Annexe : c54x_exe-depuis-ubuntu-vierge.md

Contenu de `doc/c54x_exe-depuis-ubuntu-vierge.md`, repris tel quel.

2.26.2.1 c54x_exe depuis une Ubuntu 24.04 vierge

Oct 20, 2026 · Bastien Baranoff

2.26.2.1.1 Ce que montre cette page

En une vingtaine de minutes, le cœur TMS320C54x émulé de `c54x_exe` charge la mask-ROM TI d'un vrai Calypso et l'exécute, sans QEMU, sans ARM, sans réseau GSM.

C'est le hello world du DSP : il prouve que le cœur exécute du vrai code TI. Il ne prouve pas que le DSP décode quoi que ce soit, puisque aucun burst n'est injecté. Le montage complet (ARM sous QEMU, BTS, pont, appel voix) est décrit en fin de page et vit dans osmo-operator.

Tout se fait en root ou avec `sudo` ; les chemins `/opt/GSM/…` sont ceux que les Makefile et les scripts attendent par défaut.

2.26.2.1.2 1. Prérequis (Ubuntu 24.04)

Une ligne `apt` suffit : le compilateur, `git`, `curl` pour les ROM, et `libosmocore` (qui contient `libosmocoding`, utilisée par `calypso_bsp.c`).

```
sudo apt update
sudo apt install -y build-essential git curl pkg-config libosmocore-dev
sudo mkdir -p /opt/GSM
```

Vérifier que `libosmocoding` est bien là :

```
pkg-config --modversion libosmocoding
```

Si `pkg-config` ne la trouve pas, la version de `libosmocore` du dépôt Ubuntu est trop ancienne ou incomplète ; installer alors `libosmocore-dev` depuis le dépôt nightly d'Osmocom (voir osmocom.org).

2.26.2.1.3 2. Les sources de qosmo

`c54x_exe` ne recopie pas le cœur C54x : il compile directement les sources de qosmo (`calypso_c54x.c` et ses voisins) avec ~120 lignes de doublures d'en-têtes QEMU. Il faut donc le dépôt sur disque, mais pas le construire.

```
cd /opt/GSM
git clone https://github.com/bbaranoff/qosmo
```

Aucun `configure`, aucun `make` ici. Si qosmo est ailleurs que `/opt/GSM/qosmo`, passer `QOSMO=/chemin` au `make` de l'étape suivante.

2.26.2.1.4 3. Cloner et compiler c54x_exe

La compilation prend moins d'une minute et produit un seul binaire, `c54x_exe`, à la racine du dépôt.

```
cd /opt/GSM
git clone https://github.com/bbaranoff/c54x_exe
cd c54x_exe
make QOSMO=/opt/GSM/qosmo \
     CFLAGS="-O3 -g -Wall -Werror=format -Werror=format-extra-args \
             -Wno-unused-function -Wno-unused-variable \
             -Wno-unused-but-set-variable -Wno-sign-compare"
```

Ce `CFLAGS` est celui du Dockerfile d'osmo-operator : il garde les erreurs de format bloquantes et rend muets les avertissements de variables inutilisées, nombreux dans les sources QEMU reprises telles quelles. Un `make` nu marche aussi mais sort beaucoup plus de bruit.

`make` reconstruit le binaire à chaque appel (cible `.PHONY`) ; pas besoin de `make clean` après un changement dans qosmo.

Vérifier :

```
./c54x_exe --help | head
```

2.26.2.1.5 4. Les ROM DSP, depuis FreeCalypso

La mask-ROM du C54x n'est pas dans le dépôt : elle vient du dump publié par le projet FreeCalypso (Mychaela Falconia), sur son FTP public. Le script `rom/fetch-rom.sh` la télécharge et la découpe en 7 sections au format attendu par `--rom-dir`.

```
cd /opt/GSM/c54x_exe
export FREECALYPSO_URL=ftp://ftp.freecalypso.org/pub/GSM/Calypso
bash rom/fetch-rom.sh --dest /opt/GSM --dest /opt/GSM/c54x_exe/rom
curl -fsS --retry 2 -o /opt/GSM/calypso_dsp.txt "$FREECALYPSO_URL/dsp-rom-3606-dump.txt"
cp /opt/GSM/calypso_dsp.txt rom/calypso_dsp.txt
```

`3606` est la version de ROM DSP du Calypso des Motorola C1xx ; c'est aussi la valeur que la L1 DSP écrit dans l'API RAM au boot, ce qui permettra de vérifier plus loin que la bonne ROM tourne.

Après le script, `/opt/GSM` doit contenir exactement ces 7 fichiers :

```
ls -l /opt/GSM/calypso_dsp.*.bin
## calypso_dsp.PROM0.bin  calypso_dsp.PROM1.bin  calypso_dsp.PROM2.bin
## calypso_dsp.PROM3.bin  calypso_dsp.DROM.bin   calypso_dsp.PDROM.bin
## calypso_dsp.Registers.bin
```

Si `curl` refuse le FTP (certaines builds le compilent sans), le dump se télécharge à la main depuis le même répertoire sur ftp.freecalypso.org et se passe à `fetch-rom.sh` en local.

2.26.2.1.6 5. Premier run : la ROM tourne

```
cd /opt/GSM/c54x_exe
./c54x_exe --trames 200
```

`--trames 200` fait tourner le DSP sur 200 trames TDMA (une trame = 4,615 ms de temps GSM), avec le budget d'instructions par défaut de 2300 par trame en mode autonome. Sans option, seules les erreurs du cœur passent sur stderr ; le bilan final dit combien de lignes ont été masquées et avec quel niveau `-v` les voir.

Ce qu'on doit voir, dans l'ordre :

1. Les 7 sections de ROM chargées depuis `/opt/GSM` (ou `--rom-dir`).
2. `c54x_reset()` : la ROM repart de `0xff80`, son vecteur de reset.

3. Des lignes `BRANCH-TRACE` autour de `PC=0xb41f` : c'est la boucle d'attente de la ROM TI, du vrai code que le cœur exécute.
4. Un bilan de fin avec `a_sch[3] = 0x771a`.

Pour voir une ligne par trame :

```
./c54x_exe --trames 50 --verbeux
```

Et pour les traces du cœur, `-v` à `-vvvvvv` (voir `--help`).

2.26.2.1.7 6. Lire le résultat

`a_sch[3] = 0x771a` est une valeur constante d'un run à l'autre : la ROM a booté, s'est parquée en IDLE et a laissé cet état dans sa fenêtre API. C'est la preuve que le cœur exécute fidèlement la ROM jusque-là.

Ce que ça ne prouve pas : le décodage. L'API RAM est vierge, aucun burst n'arrive, le DSP tourne à vide. Un décodeur dont la sortie ne dépend pas de l'entrée ne décode pas ; faire varier l'entrée demande le montage complet de la section 8.

Point de repère utile : le README de `qemu-calypso` donne `0xf8d8` pour ce même mot. La différence vient de l'entrée (QEMU fournit une API RAM initialisée par l'ARM, ici elle est vide), pas d'un bug : deux entrées différentes, deux constantes différentes.

2.26.2.1.8 7. Erreurs classiques

Symptôme	Cause	Remède
`make` : `qemu/osdep.h` ou `calypso_c54x.c` introuvable	qosmo n'est pas à `/opt/GSM/qosmo`	`make QOSMO=/chemin/vers/qosmo`
`make` : `osmocom/coding/...` introuvable	`libosmocore-dev` absent ou trop vieux	section 1, dépôt nightly Osmocom
`fetch-rom.sh` échoue ou `curl: (1) Protocol "ftp" not supported`	`curl` compilé sans FTP, ou pare-feu	télécharger à la main, puis `fetch-rom.sh` sur le fichier local
`c54x_exe` : ROM introuvable	les `.bin` ne sont pas dans `/opt/GSM`	`--rom-dir /chemin` ou refaire la section 4
Le bilan ne montre pas `0x771a`	mauvaise ROM, ou sections mal découpées	comparer `rom/calypso_dsp.txt` au dump 3606 ; refaire la section 4
`PC=0xb41f` n'apparaît jamais, le cœur sort en erreur	régression dans qosmo	`cd /opt/GSM/qosmo && git log -3`, remonter au commit du README

2.26.2.1.9 8. La suite : injecter des bursts

Pour que le DSP décode vraiment, il faut l'ARM (le firmware `layer1.highram.elf` d'osmocom-bb sous QEMU `qosmo`), une BTS et le pont qui lui livre des bursts. Deux chemins :

Le banc complet. Il pose tout, dans les mêmes chemins `/opt/GSM/…` que cette page :

```
cd /opt/GSM
git clone https://github.com/bbaranoff/osmo-operator
cd osmo-operator
sudo ./install.sh
sudo ./start-direct.sh --dsp
```

Le `--dsp` remplace la couche 1 gr-gsm par `c54x_exe` ; `--menu` pose la même question interactivement. Le wiki d'osmo-operator décrit les options, les ports et le plan de numérotation.

Exe par exe. Pour voir chaque étape produire son log : le DSP (`c54x_exe --arm -v`), puis QEMU avec `CALYPSO_DSP_EXTERN=1`, puis `osmocon`, puis `mobile`. La séquence, avec les lignes de journal attendues à chaque pas, est dans LAUNCH.md et le README de c54x_exe.

À partir de là, `a_sch` change avec l'entrée, et la question « est-ce que ça décode ? » a une réponse mesurable.

2.26.2.1.10 9. Le firmware Calypso : la toolchain arm-elf (gnuarm)

Le firmware d'osmocom-bb (`layer1.highram.elf`, `trx.highram.elf`…) se compile avec la toolchain historique `arm-elf` (gcc 4.5.2, binutils 2.21.1, newlib 1.19) du gnu-arm-installer, installée dans `/root/gnuarm/install`. Sur Ubuntu 24.04 son `cc1` réclame `libmpfr.so.4`, que le système ne fournit plus (il a `libmpfr.so.6`) ; un lien suffit :

```
ln -sf /usr/lib/x86_64-linux-gnu/libmpfr.so.6 /usr/local/lib/libmpfr.so.4 && ldconfig
/root/gnuarm/install/bin/arm-elf-gcc --version      # gcc 4.5.2
```

L'arbre `osmocom-bb-transceiver` (branche jolly/testing, appli `trx`) se compile aussi avec le `arm-none-eabi-gcc` du système, avec les adaptations de `patches/osmocom-bb-transceiver-qemu-bench.patch` (préfixe, édition de liens par gcc, `-fcommon`, `solve_envs.py` en python 3) :

```
cd /opt/GSM/osmocom-bb-transceiver/src/target/firmware
make BOARDS=compal_e88 APPLICATIONS=trx ENVIRONMENTS=highram CROSS_COMPILE=arm-none-eabi-
cp board/compal_e88/trx.highram.{elf,bin} /opt/GSM/firmware/board/compal_e88/
```

Tous les patchs appliqués aux arbres osmocom-bb (mobile, transceiver, burst-ind) sont dans `patches/` d'osmo-operator et appliqués par le Dockerfile et `install_modules/40-patches.sh`.

2.27 /opt/GSM/osmo-operator/doc/magnetite.md

8832 octets, 151 lignes

2.27.1 FreeCalypso Magnetite in qosmo (the original TI firmware, Motorola C139)

The original ARM firmware (TI TCS211, rebuilt by Mychaela Falconia's FreeCalypso Magnetite project) runs in the repository's Calypso emulator (qosmo) in place of the osmocom-bb `layer1`: Nucleus, the TI GSM L2/L3 stack, the BMI user interface and the RVTMUX traces. The emulated phone is a **C139** (96x64 colour screen on uWire, same Compal keypad as the C123), because it is the only Mot C1xx target for which Magnetite has a configuration with a user interface (`c139 hybrid-ui-vo`); the C11x/C123 target only has an AT pseudo-modem.

← README · Related: README-dsp, README-architecture

2.27.1.1 Install

```
sudo ./addition.sh --magnetite
```

This installs Wine and the TI TMS470 environment, clones FreeCalypso Magnetite and the FreeCalypso host tools, applies the bench patch, builds the firmware and assembles the flash image (logic in `tools/osmo-firmwares-install.sh`). `./addition.sh --status` shows whether the image is present.

2.27.1.2 Sources and toolchain

Magnetite does **not** build with gcc (nor with `/root/gnuarm`): the whole tree (sources, `.s` files in TI syntax, COFF `.lib` blobs, Nucleus) requires the TI TMS470 1.22e compiler, a Windows binary run under Wine.

path	content
`/opt/GSM/fc-magnetite`	Magnetite source tree (`freecalypso-env/` included)
`/opt/GSM/fc-host-tools`	FreeCalypso host tools: `rvinterf`, `fc-shell`, `fc-fsio`
`/opt/GSM/fc-wine-dl`	the TI Wine environment archive (`installed-env.tar.xz`), `nowhine.c`
`/opt/GSM/fc-dl`	the Compal boot loader `compal-flash-boot-for-fc.bin`
`/opt/GSM/firmware/magnetite/c139-flash.img`	the 4 MiB flash image used by the bench

Manual build, equivalent to what the installer does:

```
dpkg --add-architecture i386 && apt-get update && apt-get install -y wine wine64 wine32:i386
# TI environment (compiler, cygwin, perl, java): freecalypso-env/installed-env.tar.xz
mkdir -p ~/.wine/drive_c && tar xJf installed-env.tar.xz -C ~/.wine/drive_c && wineboot -u
gcc -O2 -o /usr/local/bin/nowhine nowhine.c      # wrapper that silences Wine's warnings
cd /opt/GSM/fc-magnetite && patch -p1 < /opt/GSM/osmo-operator/patches/fc-magnetite-banc.patch
(cd helpers && make) && ./configure.sh c139 hybrid-ui-vo
cd build-c139-hybrid-ui-vo && make               # no -j: parallel cl470 runs clobber each other's temp files
```

The installer builds the bench variant with `DWNLD 0` in `config/swconfig.cfg` (directory `build-c139-hybrid-ui-vo-nopatch`); `FC_MAGNETITE_DWNLD=1` builds the original configuration, and `OSMO_FW_REFAIRE=1` forces a rebuild. The output is `fwimage.bin` (2 MB, flashed at 0x10000) and `flash-script`.

Bench patches (in `patches/`):

patch	applies to	purpose
`fc-magnetite-banc.patch`	fc-magnetite	all bench changes in one file (forced ARFCN, automatic registration, SMS, MMI)
`fc-magnetite-alr-arfcn-forced.patch`	fc-magnetite	forced default ARFCN in ALR cell selection (`ARFCN_FORCED`)
`magnetite-sms-msginfo-doublefree.patch`	fc-magnetite	SMS message-info double free in the BMI
`magnetite-parole-ul-au-go.patch`	c54x_exe	uplink speech capture for the TI firmware on the DSP bench

2.27.1.3 The flash image

On a C1xx the boot ROM is out of circuit: the CPU starts at 0 in flash, where the Compal boot loader (FreeCalypso build `compal-flash-boot-for-fc.bin`, 8 KB) jumps to 0x10058. The 4 MiB image is laid out as:

```
img = 0xFF * 4 MiB ; img[0:8192] = compal-flash-boot-for-fc.bin ; img[0x10000:] = fwimage.bin
```

The firmware's flash file system (FFS: IMEI, afcparams, rfcap, configuration) lives at 0x3C0000 (3 sectors of 64 KB). When the installer updates an existing image it only rewrites the firmware area, keeps the boot loader and FFS, and saves the previous image as `c139-flash.img.avant-supplement`.

2.27.1.4 Emulation support in qosmo

file	what it provides for Magnetite
`hw/arm/calypso/calypso_uwire.c`	uWire controller at 0xFFFE4000 + C139 screen (9-bit words on CS0, window 0x10-0x15, RGB565); publishes `/dev/shm/calypso_lcd` with the magic `C139LCD`; property `c139-lcd`, set by `CALYPSO_LCD=c139`
`hw/arm/calypso/calypso_mb.c`	without `-kernel` and with `-drive if=pflash`: no fake vectors, boot at 0; 16-bit flash identified as Intel 0x0089 / 0x88C4 (28F320C3B, the C139 part recognised by `ffs/board/dev.c`)
`hw/arm/calypso/calypso_soc.c`	flash boot: writing CNTL_ARM_CLK does not remap IRAM at 0
`hw/ssi/calypso_spi.c`	real TWL3025 framing (bit 0 read, register in bits 5..1, data in 15..6, answer in RX_LSB), STATUS = RE_ST and WE_ST, VRPCSTS = ONBSTS (ON button released), VRPCSIM returns RSIMRSU
`hw/arm/calypso/calypso_trx.c`	the TDMA frame IRQ (Nucleus tick) starts on the first `TP_Enable`, since TI never writes TPU_INT_CTRL

file	what it provides for Magnetite
`hw/arm/calypso/calypso_sim.c`	ATR in manual mode, TX interrupt one character time after each byte (`CALYPSO_SIM_US_OCTET`, default 1040 µs), T=0 procedure byte, GSM 11.11 SELECT answers; a silence of more than 500 ms inside an APDU resynchronises it
`hw/char/calypso_uart.c`	RX wake-up interrupt (SCR[4] / SSR[1]) of the TI driver; no romload stub in flash boot; `CALYPSO_UART_DEBUG=1` traces IER/SCR/RX
`target/arm/tcg/translate.c`, `target/arm/tcg/hflags.c`	ARMv4T behaviour: BX to ARM state ignores bit 1; no alignment fault (the TI flash driver writes a halfword at odd address 0x5555)

The CPU must be **ti925t** (ARMv4T). With arm946 (ARMv5) the `pop {pc}` of TI's Thumb trampolines interworks to ARM state and crashes on the first call.

2.27.1.5 Running

Standalone, with the qosmo gr-gsm layer 1 (no DSP):

```
scripts/magnetite-c139.sh start        # QEMU + rvinterf; RUNDIR /dev/shm/magnetite; persistent working flash
scripts/magnetite-c139.sh gtk          # the window: C139 photo, colour screen, clickable keys
scripts/magnetite-c139.sh traces       # rvinterf.log (decoded L1, GPF, Riviera traces)
scripts/magnetite-c139.sh fsio ls /    # fc-fsio on the rvinterf socket (also: shell → fc-shell)
scripts/magnetite-c139.sh status | log
scripts/magnetite-c139.sh stop
```

variable	effect
`QEMU`	QEMU binary (default `/opt/GSM/qosmo/build/qemu-system-arm`)
`FLASH`	source flash image (default `/opt/GSM/firmware/magnetite/c139-flash.img`)
`RUNDIR`	run directory (default `/dev/shm/magnetite`)
`FLASH_RESET=1`	restart from the pristine image (empty FFS); a newer source image also replaces the working copy
`RVINTERF=0`	do not start rvinterf (otherwise traces go to `$RUNDIR/rvinterf.log`, socket `$RUNDIR/rvinterf.sock`)
`QEMU_CPU`	default `ti925t`
`QEMU_DEBUG=1`, `QEMU_DEBUG_FLAGS`	`-d` flags (default `guest_errors,unimp`) into `qemu-debug.log`
`QEMU_GDB=1`, `QEMU_GDB_PORT`, `QEMU_GDB_WAIT=1`	gdb stub on `tcp::1240` (gdb-multiarch, addresses from `fwimage.map`); `-S` to freeze at start
`QEMU_TRACE`	QEMU trace pattern, e.g. `'pflash_*'`
`CALYPSO_LCD_DEBUG=1`	LCD commands on stderr

On the DSP bench (c54x_exe, emulated DSP ROM): `c54x_exe/run.sh` with `FIRMWARE_FLASH=/opt/GSM/firmware/magnetite/c13` (CPU ti925t, `CALYPSO_LCD=c139`, rvinterf at step 3 instead of osmocon, no `mobile` at step 4; air link or bridge unchanged). `start-native.sh --ms-magnetite` sets this up: like `--ms`, but the phone is Magnetite. With `--bts --ms-magnetite` the BTS is instance 1 and the Magnetite C139 is instance 2 over the air; the flash image is only given to the phone, the BTS TRX keeps the `trx` firmware. With `--gapk`, voice is coded by the bridge (there is no L2/L3 mobile for GAPK).

The window: `tools/osmo-c123-gtk.py --modele c139 [--instance N]`; the model is otherwise picked automatically from the shared screen's magic, and the window switches to C139 by itself when it changes. Photo `data/c139.jpg` (Raimond Spekking, Wikimedia Commons, CC BY-SA 4.0, credit in `data/c139.txt`), key zones `data/c139-zones.json` (`--calibrate` to redo them).

2.27.1.6 Debugging

- Breakpoints disturb the firmware; prefer `QEMU_DEBUG_FLAGS=int,in_asm`, or dump the Nucleus task list from the monitor (`TCD_Created_Tasks_List` at 0x80cd60).
- `QEMU_TRACE='pflash_*'` traces flash programming.
- `fc-fsio` / `fc-shell` work over ETM through rvinterf.

2.27.1.7 Known issues

- Calls only work with ciphering off (`ENCRYPTION="a5 0" ./start.sh --regen …`).
- Formatting the FFS with `fc-fsio format /` erases and programs the 3 blocks, then the firmware ends in `os_SystemError`.
- The DSP patch download of the TI L1 (`DWNLD 1`) fails under emulation (`LA_LoadPage`), hence the `DWNLD 0` bench build. `L1_DYN_DSP_DWNLD` stays at 1 (`l1audio_async.c` does not compile with 0).

2.28 /opt/GSM/osmo-operator/environment/README.md

4961 octets, 105 lignes

2.28.1 `environment/` — bench configuration, by domain

The files in this directory define the environment of the native bench (`start-native.sh` / `start.sh`): which profile runs, where the dependencies live on this machine, and where runtime files go. Every file uses the `:=` idiom ("set only if not already set"), so **the first writer wins**, and the first writer is always the command line.

File	Domain	Edit it?
`load.env`	single entry point; sources the other files in order	no
`modes.env`	ready-made profiles (`CALYPSO_PROFILE` / `MODE`)	yes, pick one
`paths.env`	where the dependencies are on **this** machine (installation paths)	yes, first thing to adjust

Per-domain files `coeur.env`, `radio.env`, `reseau.env`, `debug.env` are sourced by `load.env` if present; the repository does not ship them.

2.28.1.1 How it is loaded

```
set -a ; . ./environment/load.env ; set +a      # done by startup_scripts/04-environnement.sh
```

Order, first writer wins:

1. the **command line** — `VAR=x ./start-native.sh` always wins;
2. `modes.env` — the chosen profile;
3. `paths.env` — where things are;
4. `coeur.env`, `radio.env`, `reseau.env`, `debug.env` (optional);
5. `$OQC_ROOT/environnement/paths.env` — the Calypso emulation tree's own paths (`FIRMWARE_ELF`, `DSP_PROM0..3`, `OSMOCON`, `QEMU_CFGS`…), sourced last by `paths.env` so nothing above is overridden.

`load.env` looks for the files in its own directory only: a copy of the tree without `environment/` loads nothing.

2.28.1.2 `modes.env` — profiles

`CALYPSO_PROFILE` is the profile; `MODE` (historical name) wins if set.

```
CALYPSO_PROFILE=faketrx ./run.sh
MODE=faketrx ./start-native.sh
```

Profile	Aliases	Content
`core`	`noproc`	STP + HLR + MSC + MGW + BSC (+ Asterisk), no radio (default)
`faketrx`		core + osmo-bts + fake_trx + trxcon + mobile, no QEMU
`virtphy`		core + osmo-bts-virtual + virtphy + mobile
`calypso`	`qemu`	core in no-process mode + the Calypso emulation (delegated to `OQC_ROOT`)
`hybrid`	`faketrx-qemu`	core + live fake_trx + attached Calypso
`hw`		physical SDR
`virtual`		SS7 multi-operator over `ip netns`; `N_OPERATORS` (default 1)

An unknown profile prints the list on stderr and continues.

2.28.1.3 `paths.env` — installation paths

All overridable from the environment; check with `./run.sh --check-paths`.

Variable	Default	Role
`GSM_ROOT`	`/opt/GSM`	installation root
`NITB_TREE`	this repository	`NITB_SCRIPTS`, `NITB_CONFIGS`, `NITB_HELPERS` derive from it
`CALYPSO_FORK`	`qosmo`	name of the Calypso emulation tree
`OQC_ROOT`	`../<fork>` next to this tree, else `$GSM_ROOT/<fork>`	the emulation tree (`run.sh`, `run_modules/`, `cfgs/`, `environnement/`)
`OSMO_STP`, `OSMO_HLR`, `OSMO_MSC`, `OSMO_MGW`, `OSMO_BSC`, `OSMO_SGSN`, `OSMO_GGSN`, `OSMO_PCU`, `OSMO_BTS_TRX`, `OSMO_BTS_VIRTUAL`, `OSMO_TRX_IPC`, `ASTERISK_BIN`	bare name	Osmocom binaries; a bare name is searched in `PATH` (package install)
`OSMOCOM_BB`	`$GSM_ROOT/osmocom-bb`	osmocom-bb tree; `MOBILE_BIN`, `TRXCON_BIN`, `VIRTPHY_BIN`, `FAKE_TRX` derive from it
`OSMOCOM_CFG`	`/etc/osmocom`	Osmocom configuration directory
`OSMOCOM_HOME`	`$HOME/.osmocom`	
`RUN_DIR`	`$XDG_RUNTIME_DIR/osmo-nitb`, else `/run/user/<real uid>/osmo-nitb`, else `/tmp/osmo-nitb`	PID files and monitor socket. Resolved from the real UID so that `sudo` and `sudo -i` launches see the same running stack
`LOG_DIR`	`$RUN_DIR/logs`	logs; must stay on a tmpfs (tmux follows them with `tail -F`, which needs inotify)
`QEMU_TREE`	`$OQC_ROOT`	the emulation tree as the shared modules name it

Variable	Default	Role
QEMU_BIN	$OQC_ROOT/build/qemu-system-arm	
QOSMO_LAUNCHER	/usr/local/bin/<fork>	C launcher (tools/qosmo-launch); 40-qemu.sh falls back to QEMU_BIN if absent

Two switches reduce the log volume if the tmpfs fills up: CALYPSO_INTM_TRANS=0 (about half of qemu.log) and CALYPSO_MAILBOX=0 (removes mailbox.log).

2.28.1.4 Calypso emulation variables

The CALYPSO_* variables read by QEMU and by c54x_exe are described in the emulation tree (/opt/GSM/qosmo/environnement/RE bench.env). The only reliable view of what QEMU received is the manifest it writes at boot:

```
grep "calypso-manifest" /var/log/osmocom/qemu.log
```

Four gate idioms coexist in that code, so =0 does not always turn a variable off:

Idiom	Active when	How to turn off
getenv("X") ? 1 : 0	the variable **exists**, even at 0	unset X
atoi(...) > 0	value > 0	X=0
*e == '1'	exactly "1"	any other value
default ON + X_OFF	by default	set X_OFF

2.28.1.5 := versus =

In these files : "${VAR:=value}" lets the command line win; VAR=value **locks** the value. Use = only for what must never be overridden.

2.29 /opt/GSM/osmo-operator/navigation/QUICKSTART.md

7083 octets, 159 lignes

2.29.1 SS7 console — quick start

Everything lives in navigation/. No dependency beyond the distribution's python3. Nothing is hard-coded: point codes, hub, ports and subscribers are read from the containers and from the repository's configurations.

2.29.1.1 1. In thirty seconds

```
cd /opt/GSM/osmo-operator
sudo ./launch/start-docker.sh --wan --operators 2 --hub-ip 192.168.1.49   # the lab
./ss7-console.py                                                          # the diagram
```

The diagram opens. Move with the **arrow keys**, open with **Enter**, go back with **Escape**. q quits.

Key	Effect

2.29.1.2 2. Keys

Key	Effect
arrows	move from box to box, in the space of the diagram
Enter	open the menu of the selected element
Escape	go back; **inside a VTY: close THIS VTY** and return to the diagram
Tab	next element
`v`	open the element's VTY directly
`s`	SS7 scripts (MAP operations)
`a`	SS7 audit (DAUD) of a point code
`c`	connect / disconnect the console's M3UA link
`j`	M3UA link journal
`r`	reload the topology
`?`	help

Inside a VTY: `Enter` sends, up/down replay the history, PageUp/PageDown scroll, `Escape` closes the session. Other VTYs stay open: one session per port, independent.

2.29.1.3 3. What the diagram shows

```
SS7 CONSOLE ---M3UA--- INTER-STP (hub)
                            |
               +--------------+--------------+
          OP1 - STP                       OP2 - STP
          1.11.2                          1.21.2
             |                               |
   STP  MSC/VLR  HLR  BSC  BTS  MGW  SGSN  GGSN  ...
```

Each service box carries its VTY port. Colours follow the family: SS7, core, subscribers, radio, media, data.

2.29.1.4 4. Without the interface: the command line

```
./ss7-console.py list                inventory (PC, hub, SCCP, VTY)
./ss7-console.py ops                 catalogue of MAP operations
./ss7-console.py vty 1 4258 "show subscribers all"
./ss7-console.py audit 1.11.1        the SS7 "ping" (DAUD -> DAVA/DUNA)
./ss7-console.py map sri-sm --pc 1.11.6 msisdn=600101 sc_addr=600999
./ss7-console.py serve               MAP responder (simulated HLR)
```

`vty <n>` accepts the operator number or the container name.

2.29.1.5 5. MAP operations

Key	Operation	Opcode	Target SSN
`sri-sm`	sendRoutingInfoForSM	45	6 (HLR)
`sri`	sendRoutingInfo	22	6
`ati`	anyTimeInterrogation	71	6
`sai`	sendAuthenticationInfo	56	6
`send-imsi`	sendIMSI	58	6
`psi`	provideSubscriberInfo	70	7 (VLR)
`ul`	updateLocation	2	6

Key	Operation	Opcode	Target SSN
raw	raw invoke (opcode + hex parameter)	-	-

The console is a **real** SS7 point: it attaches to the inter-STP over M3UA on SCTP, dynamically registers a routing key for its point code (RKM), goes ASP_ACTIVE, then sends TCAP/MAP that crosses the hub like the operators' traffic. Each send can be exported as a standalone Python script (navigation/scripts/<operation>-<timestamp>.py).

2.29.1.6 6. The responder: a complete MAP round trip

Osmocom makes MSC and HLR talk **GSUP**, not MAP: an SRI-SM sent to the real HLR finds nobody. The responder fills that gap by answering with the data of the REAL HLR (read over VTY), through the real hub.

```
# terminal A
./ss7-console.py serve
# terminal B
./ss7-console.py map sri-sm --pc 1.11.6 msisdn=600101 sc_addr=600999

result (opcode 45):
   IMSI                         001010001000001
   Service node                 +600101
```

2.29.1.7 7. Diagnostics and tests

```
./navigation/ss7-diag.py             full SS7 diagnostic
./navigation/ss7-diag.py --rapide    without M3UA or MAP
./navigation/test-console.sh         runs EVERY command, one log per test
```

ss7-diag.py checks, in the order a failure is read: environment (SCTP), topology inherited from the configs, STP/MSC/BSC/HLR of each operator, SCTP association to the hub, attachment of the console as ASP, audit of all known point codes, then a real MAP round trip. The exit code is the number of failures.

test-console.sh writes one file per command in navigation/logs/<timestamp>/ (override with LOGDIR), plus a rapport.txt.

2.29.1.8 8. Lab behaviour to know about

- **The M3UA link from the host only lives a few seconds.** The inter-STP stops serving an association coming from the host after ~3 s, and the SCTP association expires around 35 s. The console renews it by itself before each exchange (~0.3 s): invisible, but it explains an occasional "no answer" followed by a second attempt that works.
- **The hub refuses the "Service Indicators" parameter** in a routing key (registration status 9). It is therefore omitted.
- **The console proposes its own routing context**: otherwise the hub returns the same context to two different ASPs and, in traffic-mode override, the last one attached steals the first one's traffic (the first destination goes DUNA).
- **DATA goes out on SCTP stream 1**: stream 0 is reserved for management, and a DATA on it is refused ("Invalid Stream Identifier").
- **Point codes are in ITU 3-8-3 format**: a.b.c with a and c from 0 to 7, b from 0 to 255. 1.11.60 does not exist: it would be received as 1.15.4.
- The console's point codes (1.11.7, 1.11.6…) leave dynamic PROHIB routes in the operator STPs when it closes: this is normal, they go back to avail when it returns.

2.29.1.9 9. Modules

File	Role
`topo.py`	inventory: containers, env, `/etc/osmocom/*.cfg`, VTY ports
`vty.py`	persistent VTY sessions through `docker exec`
`diagram.py`	the diagram and arrow-key spatial navigation
`tui.py`	the curses interface (diagram, menus, forms, VTY)
`ber.py`	minimal BER/DER
`sccp.py`	SCCP UDT/UDTS, addresses, global titles, TBCD
`tcap.py`	TCAP begin/end, invoke, returnResult, returnError, AARQ/AARE
`mapops.py`	MAP catalogue: encoders, decoders, errors
`m3ua.py`	M3UA ASP over SCTP: ASPUP, RKM, ASPAC, DATA, heartbeat
`ss7.py`	the layer that ties everything together, and the readable report
`responder.py`	MAP responder backed by the real HLR
`quickcmd.py`	useful VTY commands, per daemon
`ss7-console.py`	entry point (diagram + sub-commands); `./ss7-console.py` at the repository root wraps it
`ss7-diag.py`	full diagnostic
`test-console.sh`	test of each command, with logs

2.29.1.10 10. Troubleshooting

Symptom	Probable cause
`aucun inter-STP connu` (no known inter-STP)	lab stopped, or `--hub-ip` never given to `start-docker.sh`
`le conteneur ... ne tourne pas` (container not running)	`docker ps` no longer sees it
`VTY ... injoignable` (VTY unreachable)	the daemon is not running in that container
`Aucune reponse` (no answer) to a MAP	nobody listens on that SSN: run `serve`
arrows have no effect	terminal in normal cursor mode (handled), otherwise `TERM=xterm`
`DUNA` on a point code	the ASP of that destination is not attached to the hub

2.30 /opt/GSM/osmo-operator/navigation/STATUS.md

4185 octets, 84 lignes

2.30.1 Known issues

Open limitations of the lab, grouped by area. Fixed items are not listed: what works is described in `navigation/QUICKSTART.md` (SS7 console), `pont/README.md` (radio bridge) and the wiki.

2.30.1.1 1. Radio bench (`--dsp`, c54x_exe)

Mounts: grgsm (`--grgsm`, gr-gsm layer 1 inside QEMU) and DSP (`--dsp`, the default: C54x mask ROM in c54x_exe). On the DSP bench the mobile camps, does its LU, uses A5/1 on the downlink through the modelled XIO coprocessor, completes TCH calls (echo 600, mobile to mobile) and MO/MT SMS. Open points, by importance:

1. **B_BFI set on every speech frame** delivered by the ROM, although the FR speech stays intelligible (the firmware does not report the BFI). Signal path (BSP, I/Q, equalisation) or C54x core (Viterbi, re-encoding) not decided; speech quality not measured. TODO: compare bit by bit the 33 bytes delivered by the ROM with the frames sent by the BTS (MGW RTP or bridge decoding).
2. **Occasional LOS on a TCH call**: the SACCH of the TCH is lost entirely for ~15 s (`LOSS counter for ACCH` down to 0, then "LOS during dedicated mode"); the following call is clean. Cause unknown.

3. **Downlink SACCH of the SDCCH/8**: about half of the SACCH blocks are discarded by the mobile during an LU. Suspect: the 45.002 table of the BSP for SACCH/8 (alternation over two 51-multiframes). TODO: compare `fn % 102` of discarded blocks against accepted ones.
4. **UI SAPI 0 on the TCH**: the BSC logs "SAPI=0 UNIT DATA INDICATION: unimplemented Abis RLL message" on a fresh TCH lchan. Not located, not blocking.
5. **Bridge clock**: the targeted advance occasionally hits the ceiling (`PONT_AVANCE_MAX` 40) and the real DL margin can touch 0 right after a LOS. No loss associated so far. The "marge DL reelle" line printed every 10 s in `pont.log` must stay > 0.
6. Minor: `MM_EVENT_NO_CELL_FOUND` after an MO SMS (resynchronisation), MSC "Duplicate DTAP" on a paging response.

Not anomalies: CRC failures of the bridge's downlink TCH monitor at the start and end of a call (the BTS puts nothing on the TCH while RTP does not flow; bridge decoding, independent of the DSP), and the `ko=` counter of the `[a_dd]` probe (B_FIRE1, no defined meaning on speech).

2.30.1.2 2. SS7 console (`navigation/`)

- Responder limits: a single operator and a fixed SSN. TODO: several operators, choice of the SSN.
- Exported scripts (`navigation/scripts/`) cannot be replayed from the diagram yet.
- The M3UA link from the host is only served for a few seconds by the hub; the console renews it before each exchange (see QUICKSTART § 8).

2.30.1.3 3. Numbering and multi-node (WAN) configuration

Open findings of the numbering audit (point codes, subscribers, WAN prefixes, network identifiers):

1. **ARFCN / BSIC / LAC / CI depend only on the operator**, not on the node (`generate_configs.sh`: `512 + op_id*2`, `(op_id*7) % 64`, `0x000<op>`): two nodes running the same operator number broadcast the same cell identity.
2. **`_gc_warn_perop` (`generate_configs.sh`) tests `N_OPERATORS`**, which `start-docker.sh` does not set, so the per-operator guard does not fire on the container path.
3. **The MGW RTP range (`rtp port-range 4002 16001`, `configs/osmo-mgw.cfg`) overlaps every service port** of the container, VTY ports included.
4. **`network/wan-nodes.sh`: `WAN_NOPS[id]` is always set** (default 1), so a peer declared without an operator count is always seen as carrying one operator.
5. Three divergent fallbacks for MCC/MNC (`start-native.sh`, `run_modules/21`, `scripts/sms-routing-setup.sh`).
6. Only two local SMS routes are generated whatever `OP_MS`.
7. No routing by global title: everything is PC/SSN 254.

TODO: index ARFCN / BSIC / LAC / CI by node in `generate_configs.sh`; set `WAN_NOPS` only when the field exists; stop fixing operator 1 in `checks/diag-stp-operator.sh`.

2.30.1.4 4. Verification commands

```
sudo ./launch/start-docker.sh --wan --operators 2 --hub-ip <hub>   # the lab
./navigation/ss7-diag.py        # expected: 2 distinct PCs and rctx per operator, no DUNA
./navigation/test-console.sh    # lab running: all sections must pass
```

2.31 /opt/GSM/osmo-operator/pont/README.md

13758 octets, 218 lignes

2.31.1 `pont/` — the TRX bridge between osmo-bts and the emulated Calypso

`pont` is a Python package that sits between `osmo-bts-trx` (TRXD/TRXC on UDP) and the emulated phone's layer 1. It decodes the downlink bursts, applies or undoes A5 ciphering, feeds the layer 1, and encodes and sends the uplink bursts back to the BTS at the right frame number. It also records I/Q for the spectrum panel and publishes GSMTAP for Wireshark.

2.31.1.1 Entry points

Entry point	Mount	Defaults set before the import
`pont.py`	grgsm (gr-gsm layer 1 inside QEMU, GSMTAP 4730/4731)	`PONT_UL_RETARD_MAX=0`; refuses `--dsp-port`
`pont_dsp.py` (`pont.dsp.main`)	DSP (`c54x_exe --arm`); `--dsp-port 6702` (or `PONT_DSP_PORT`) mandatory	`PONT_UL_RETARD_MAX=26`, `PONT_KC_RETENTION=1`, `PONT_DSP_COPIES=6762`

`pont_dsp.py` is launched by `c54x_exe/run.sh` (`PONT_PY`), hence by `start-native.sh` on the DSP bench (the default). A variable set by the operator always wins over these defaults (`setdefault`).

With `BRUIT_MODE=ber` or `relais`, `pont.py` targets `BRUIT_PORT_GRGSM` (14730) instead of 4730 so that `qosmo/tools/injecteur_bruit.py` can sit in between.

2.31.1.2 Command line (`pont/config.py`)

Option	Env var	Default	Role
`--arfcn`	`PONT_ARFCN`	514	
`--bsic`	`PONT_BSIC`	7	
`--trx-bind`	`PONT_TRX_BIND`	127.0.0.1	TRX side bind address
`--trx-base`	`PONT_TRX_BASE`	5700	TRX base port
`--dsp-port`	`PONT_DSP_PORT`	0	UDP port of the C54x BSP (6702); 0 = off
`--bsc-cfg`	`PONT_BSC_CFG`	`/etc/osmocom/osmo-bsc.cfg`	timeslot plan read from the BSC config
`--no-tap`	`PONT_TAP=0`	tap on	GSMTAP output (`PONT_TAP_PORT`, 4729)
`--no-record`	`PONT_AIRREC=0`	record on	I/Q recording (and the panel's FFT)

2.31.1.3 Package layout

Module	Role
`__init__.py`	`main()` of the grgsm mount: threads, signals, reporter
`config.py`	options and environment variables
`trx.py`	TRXD/TRXC sockets, `Clock` (DSP-slaved clock), `Transmitter` (uplink scheduling), copies to other BSPs
`downlink.py`	downlink decoding, `Feeder` (towards layer 1), IMMEDIATE ASSIGNMENT guard
`uplink.py`	uplink encoding, `TchScheduler`, FACCH handling
`cipher.py`	A5 keystream, Kc from `/dev/shm/calypso_kc_l1`, retention rule
`state.py`	`Dedicated` / `Tch` state machines
`gsm.py`	channel plans (SDCCH/8, TCH), xCCH coding
`record.py`	I/Q recording to files and FIFOs
`stats.py`	counters and the periodic STATS line
`dsp/`	subclasses for the DSP mount: `clock.py`, `dedicated.py`, `downlink.py`, `tch.py`, `trx.py`, `uplink.py`
`air.py`, `airmesh.py`	burst relays between Calypso instances (BTS mode, multi-node air mesh)
`voix_hote.py`, `vsp_audio.py`	host-side speech coding (`PONT_VOIX_HOTE=1`) and the modelled voice port (`CALYPSO_VSP=1`)
`pont.py`, `pont_dsp.py`	entry points

2.31.1.4 Shared memory interface

The bridge, the firmware shunt in QEMU and `c54x_exe` exchange state through `/dev/shm`:

File	Writer	Content
`calypso_kc_l1`	the layer 1 (from `d_a5mode` and `a_kc[4]` in the NDB, resolved from the firmware DWARF)	the Kc actually loaded into the DSP; the bridge reads it first and falls back on the historical file only if it is absent, never a mix of the two
`calypso_dcch_cfg`	layer 1	current dedicated channel (`chan_nr`, kind, sub-slot, `seq`; kinds 2/3 = TCH/F, TCH/H on the DSP mount); `0xFF` ends the connection
`calypso_tch_cfg`	the bridge	TCH announcement to `c54x_exe`; `seq+1, tn=0` returns to the SDCCH without dropping the Kc
`calypso_sdcch_ul`	layer 1	uplink SDCCH/SACCH L2 blocks; never a FACCH
`calypso_tch_facch_ul`	layer 1	uplink FACCH blocks (single slot with a process-wide `seq`; the bridge resyncs its cursor at each new channel epoch so a stale block is never replayed)
`calypso_xcch_ul_rom`	`c54x_exe`	ring of uplink SDCCH/SACCH bursts as emitted by the ROM, with the uplink A5 stream it XORed (header `w`(u32) `n`(u32), 1024-byte slots `seq fn l2[23] . bits[4x116] flux[4x116]`)
`calypso_tx_rom`	`c54x_exe`	final uplink bursts of the ROM (`PONT_UL_RAW=1`)

Kc byte order: the firmware stores the key reversed and by words (`a_kc[0] = key[7] | key[6]<<8` … `a_kc[3] = key[1] | key[0]<<8`); both inversions are undone when reading.

2.31.1.5 Ciphering rules

- The end of life of a Kc is a **network event**, not a local state. The key is released on IMMEDIATE ASSIGNMENT (the only grant of a new channel, which always starts in clear), CHANNEL RELEASE on SDCCH and CHANNEL RELEASE on FACCH. It is not released on SABM, on a layer-1 `algo=0` (the firmware reloads `dsp_load_ciph_param(0, NULL)` at every resynchronisation, including during the SDCCH to TCH switch) nor on `DM_EST_REQ` (also used to open SAPI 3 on an already ciphered channel).
- `PONT_KC_RETENTION` (default 1) keeps the key over the window where the local layer 1 is in clear while the network still ciphers.
- The C0 dummy burst (`osmo-bts` fills every idle timeslot of C0) is filtered before `a5_apply` on both the xCCH and the TCH paths: a dummy burst is never ciphered, and counting it as a CRC failure inflates the error rate on a healthy radio. `ne=0` with `rc=-1` on the TCH is the signature of a constant input, not of noise.
- A dedicated channel identity (`chan_nr`) is not an instance: the BSC reallocates the same sub-slot, so the identity is forgotten when the channel ends, and the next establishment is learnt even with an equal `chan_nr`.
- The DCCH guard (which removes the camp's SI from `a_cd` during a dedicated channel) is armed by the downlink (`set_dcch_active`) and kept alive by the dedicated uplink (`d_task_u` in {12 DUL, 13 TCHT, 14 TCHA}); its TTL (~2 s) returns the camp to the mobile at the end of the channel.
- `CALYPSO_CANNED` takes a token list (`FBDET,TOA,PM,SNR,ANGLE,CRC | FULL | ALL | NONE`) or a boolean (`1/ON/YES/TRUE`, `0/OFF/NO/FALSE`).

2.31.1.6 DSP mount (`pont_dsp.py`, `pont/dsp/`)

Same building blocks as grgsm, but the dedicated state follows the DSP instead of the bridge's own decoding, because the BSP plays frames at the C54x's time, 12 to ~200 frames after the bridge:

- **Downlink A5 is done by the DSP.** The mask ROM programs the Calypso XIO coprocessor (ports `0x2800..0x2818`, modelled in `qosmo/hw/arm/calypso/l1-dsp/calypso_a5.c`, compiled into c54x_exe) with `a_kc` and `a_a5fn`. The bridge does not apply the stream to the downlink; `PONT_DSP_DECHIFFRE=1` makes it do so again (with `CALYPSO_A5=0` on c54x_exe). The uplink stays ciphered by the bridge, at the air fn.
- **`TchDsp`**: the ASSIGNMENT COMMAND is an announcement; c54x_exe switches the BSP at the first TCHT/TCHA/TCHD task set by the firmware. `abandon()` returns to the SDCCH without dropping the Kc.
- **`UplinkDsp`**: uplink SDCCH/SACCH bursts come from the ROM (`calypso_xcch_ul_rom`); the bridge re-decodes them, undoes the ROM's XOR if needed (it ciphers at the DSP's time), compares with the L2 of `calypso_sdcch_ul`, then sends and re-ciphers at the air fn. Without ROM bursts after `PONT_UL_ROM_ATTENTE` (0.12 s): host coding. STATS line: `xCCH ul rom= hote= rom_ko=`. A block of `calypso_sdcch_ul` is never a FACCH; an ASSIGNMENT FAILURE, or an SDCCH block while the mobile was on the TCH, withdraws the announcement. The Kc is kept after the release until the next IMMEDIATE ASSIGNMENT.
- **`DedicatedDsp`**: `plan()` always returns the last SDCCH while the connection lives; only `0xFF` ends it.
- **`TrxDsp`**: the TCH timeslot counts as dedicated from the announcement (osmo-bts ciphers the TCH from activation), under the `cipher.dl_active` guard.
- **`ClockDsp`**, closed-loop clock: advance = `PONT_MARGE_DL` (16) minus the minimum BTS advance over `PONT_MARGE_FENETRE` (3 s), bounded to [`PONT_AVANCE_MIN` 10, `PONT_AVANCE_MAX` 40]. The BTS advance is measured on the targeted point (DSP + advance). A "marge DL reelle … min … moy" line every 10 s in `pont.log` must stay > 0. `PONT_MARGE_DL=0` returns to the fixed advance of `Clock`.

2.31.1.7 Environment variables

Variable	Default	Effect
`PONT_KC_RETENTION`	1	keep the Kc until the next network release event; 0 to compare
`PONT_DSP_DECHIFFRE`	0	DSP: the bridge deciphers the DL (with `CALYPSO_A5=0` on c54x_exe)
`PONT_DSP_COPIES`	`6762` (pont_dsp.py)	extra BSP UDP ports receiving a copy of the downlink (banc-rssi instance); empty = none
`PONT_DSP_COPIES_UL`	1	also copy the uplink to those ports (marker 0xA5 at byte 6)
`PONT_UL_ROM`	1	DSP: uplink SDCCH/SACCH bursts from the ROM; 0 = coded by the bridge
`PONT_UL_ROM_ATTENTE`	0.12	DSP: maximum wait (s) for a block's ROM bursts before host fallback
`PONT_UL_RAW`	0	DSP: uplink = the ROM's final bursts (`calypso_tx_rom`); host coding and A5 inactive
`PONT_UL_RAW_TCH`	0	with `PONT_UL_RAW=1`: 1 = TCH by the ROM too (uplink speech cut)
`PONT_UL_RAW_DCCH_HOTE`	0	with `PONT_UL_RAW=1`: SDCCH/SACCH still coded by the host
`PONT_UL_RAW_SACCH_T`, `PONT_UL_RAW_DECAL`	0	raw-mode SACCH/T handling and burst offset
`PONT_UL_TN_DEFAUT`	unset	default uplink timeslot
`PONT_PDCH_FILTRE`	1	filter PDCH bursts on the uplink
`PONT_UL_RETARD_MAX`	0 (grgsm), 26 (DSP)	maximum lateness (frames) of an uplink burst still sent; 0 = drop
`PONT_UL_FN_ADVANCE`	3	uplink frame advance

Variable	Default	Effect
PONT_WINDOW_TOL, PONT_WINDOW_ESSAIS	1, 12	tolerance and re-waits of an uplink burst woken too early
PONT_HORLOGE	1	0 = wall clock instead of the DSP-slaved clock
PONT_HORLOGE_AVANCE / _PERIODE / _KP / _PHASE_N / _MAX	12, 0.05 s, 1.0, 100, 4	clock servo: fixed advance, period, gain, phase, maximum factor
PONT_MARGE_DL, PONT_AVANCE_MIN, PONT_AVANCE_MAX, PONT_MARGE_FENETRE	16, 10, 40, 3 s	ClockDsp
PONT_CLCK_PERIOD_FN, PONT_CLCK_PAUSE_MAX, PONT_CLCK_PAUSE_PLAFOND, PONT_CLCK_SANS_IND_MAX, PONT_CLCK_SKEW_ZONE, PONT_BTS_FN_ADVANCE	51, 20, 38, 300, 60, 2	TRXC CLOCK handling
PONT_FACCH_SKIP	0	1 = also skip the pending FACCH when switching to the TCH
PONT_FACCH_TOUT	0	1 = send every uplink FACCH block without filtering
PONT_IMM_ASS_GARDE	2.0	s: guard after an IMMEDIATE ASSIGNMENT (Magnetite mobile, CALYPSO_PONT_IMM_DCCH=0 to disable)
PONT_DCCH_REL_DELAI	1.5	s: delay before a channel release takes effect
PONT_TCH_ABANDON_GRACE	2.0	s: grace before a TCH announcement is abandoned
PONT_GSMTAP_PORT	4730	gr-gsm layer 1 GSMTAP target (14730 with the noise injector)
PONT_TAP, PONT_TAP_PORT	1, 4729	GSMTAP for Wireshark
PONT_RSSI	60	RSSI reported to the BTS
PONT_AIRREC, PONT_AIRREC_UL	1, 1	I/Q recording DL / UL
PONT_AIRREC_DL_PATH, PONT_AIRREC_UL_PATH	/root/record.cfile, /root/record_ul.cfile	recording files
PONT_AIRREC_DL_FIFO, PONT_AIRREC_UL_FIFO	/tmp/iq_fft.fifo, /tmp/iq_fft_ms.fifo	FIFOs for the FFT panel
PONT_AIRREC_MAX_MB, PONT_AIRREC_MINFREE_MB, PONT_AIRREC_OSR	4096, 8192, 4	file recording stops below the free-space threshold
PONT_VOIX_HOTE, PONT_VOIX_SHM_UL, PONT_VOIX_SHM_DL	0, /dev/shm/pont_voix_ul, /dev/shm/pont_voix_dl	host-side speech coding for the Magnetite C139
CALYPSO_CANNED		NONE = nothing canned, everything real

Counters: ne is the number of bit errors, nb the **total** number of bits (378 for TCH/FS, constant). ne=0 with rc=-1 indicates a constant input, not a healthy radio.

2.31.1.8 Reproducible experiment

scripts/banc-repro.sh freezes the whole sequence: detached start (CALYPSO_NO_ATTACH=1), camp waited for by **querying** the VTY (show ms -> C3), N identical calls, a verdict per call computed on that call's slice of pont.log alone, timestamped collection in /root/repro-<stamp>/ with a resume.txt.

```
scripts/banc-repro.sh --calls 4                # with restart
scripts/banc-repro.sh --no-restart --calls 3   # on the running stack
scripts/banc-repro.sh --dest 600               # called number (default 600)
PONT_KC_RETENTION=0 scripts/banc-repro.sh      # A/B of a setting
```

Three calls are enough to tell an intermittent fault from a positional one.

2.31.1.9 Known issues

- **`unsupported SAPI` on the mobile side**: a few per run remain (not the SACCH L1 header, not the camp's SI; both checked). Does not block calls. Possible leads: another writer of `a_cd`, or the firmware re-reading a stale block for lack of refresh (`B_BLUD`).
- **Residual CRC failures** on the ciphered SDCCH and on the SACCH, not isolated.
- **B_BFI** set on every speech frame delivered by the ROM while FR stays intelligible; signal path or C54x core, not decided.
- **Occasional LOS** on a TCH call: the TCH's SACCH is lost entirely for ~15 s; the next call is clean. Cause unknown.
- **Downlink SACCH of the SDCCH/8**: about half of the blocks discarded by the mobile during an LU; suspect the BSP's SACCH/8 table (alternation over two 51-multiframes).
- **UI SAPI 0 on the TCH** ("unimplemented Abis RLL message" at the BSC), not blocking.
- **Clock**: the targeted advance sometimes reaches `PONT_AVANCE_MAX`; the real DL margin can touch 0 right after a LOS.
- `_TBL_SOFT` holds two encodings (`0x01` and `0xFF` both map to `-127`); harmless as long as the DL TRXD arrives as hard bits `0x00/0x01`.
- Not anomalies: CRC failures of the bridge's downlink TCH monitor at the start and end of a call (the BTS puts nothing on the TCH while RTP does not flow).

2.32 /opt/GSM/osmo-operator/services/README.md

6149 octets, 113 lignes

2.32.1 `services/` — systemd units

The project's units. They are installed by the `Dockerfile` (image), by `build-iso.sh` (bootable media) and by `addition.sh`; the native install (`install.sh`) does not enable them. `80-osmo-operator.preset` disables them all at boot: a freshly installed machine boots to its desktop, and the operator starts the bench (desktop icon, `launch.sh`, or `systemctl`).

Unit	Role
`osmo-banc.service`	the standalone GSM bench (`start.sh`); stack in the tmux session `calypso`
`osmo-multi.service`	the multi-operator bench (`launch/start-multi.sh`: native + containers + inter-STP); `After=osmo-banc`, docker required
`osmo-lte.service`	the bench's 4G (Open5GS + srsENB + srsUE over ZeroMQ, `tools/osmo-lte.sh start`)
`osmo-egprs-dsp.service`	the EGPRS workflow (`scripts/egprs-dsp.sh start`): virtual EGPRS cell + `osmo-pcu` + osmocom-bb GPRS modem on top of the DSP bench; `Requires=osmo-banc`
`osmo-dsp-rom-max.service`	on demand: the "ROM does the maximum" workflow (`scripts/dsp-rom-max.sh run`)
`osmo-bts-trx.service`	base station, TRX transceiver (`/etc/osmocom/osmo-bts-trx.cfg`)
`osmo-egprs-web.service`	web operations console (dashboard, port 8080)
`osmo-egprs-web-install.service`	installs/refreshes the dashboard runtime, unit and TLS certificate (`/opt/GSM/osmo-egprs-web/install-web-service.sh`)
`osmo-reglages.service`	settings web page for the Osmocom stack (`tools/osmo-reglages.py`, http://127.0.0.1:8090/)
`osmo-supplements.service`	first boot: installs the supplements chosen in the installer (`tools/osmo-supplements-firstboot.sh`)
`open5gs-webui.service`	Open5GS WebUI, 4G subscriber console (port 9999, MongoDB)

Unit	Role
`80-osmo-operator.preset`	systemd preset: all project units disabled at boot

The units under `contrib/systemd/` belong to upstream QEMU and are not part of this directory.

2.32.1.1 Starting the bench

```
systemctl start osmo-banc         # one session
systemctl enable --now osmo-banc  # and at every boot
```

To build an image that enables them anyway: `./build-iso.sh --banc --multi` (or `OSMO_ISO_BANC=1 OSMO_ISO_MULTI=1`); on an already installed machine, `OSMO_MULTI_ENABLE=1 ./addition.sh`.

Taking over a bench started as a service:

```
tmux attach -t calypso            # Ctrl-b d detaches, the bench keeps running
journalctl -u osmo-banc -f        # output of start.sh / run.sh
systemctl restart osmo-banc       # a fresh bench; launch.sh (icon) does the same then attaches
systemctl start osmo-multi        # adds the containers + inter-STP (docker required)
```

Persistent options: `OSMO_BANC_ARGS` in `/etc/default/osmo-banc` (`OSMO_MULTI_ARGS` in `/etc/default/osmo-multi`, `DSP_ROM_MAX_ARGS` in `/etc/default/osmo-dsp-rom-max`). The DSP bench is the default; `OSMO_BANC_ARGS="--grgsm"` selects the gr-gsm layer 1. `launch.sh --dsp` sets them for the session via `systemctl set-environment`. With the DSP bench, osmo-banc also runs `c54x_exe/run.sh` after the fork's plan (hand-off by `exec`); `systemctl stop` / `./start-native.sh --stop` stop the DSP bench as well.

Installing a unit by hand:

```
sudo cp services/osmo-bts-trx.service /etc/systemd/system/
sudo systemctl daemon-reload && sudo systemctl enable --now osmo-bts-trx
```

2.32.1.2 `osmo-egprs-dsp` — the EGPRS workflow

`scripts/egprs-dsp.sh` brings packet data up on the bench step by step: preflight (TI ROM 7/7, `c54x_exe`, `modem`), GPRS configs in `/etc/osmocom` (`configs/gprs/`, BTS 2 re-inserted into `osmo-bsc.cfg` after each `--regen`), `osmo-banc` (DSP chain), the modem's subscriber in the HLR, the virtual EGPRS cell (`osmo-bts-virtual` + `osmo-pcu` on `/tmp/pcu_bts3`), then the modem (`virtphy` + `modem`): GPRS attach, PDP context, ping to `apn0` from the `msgprs` netns. The daemons are transient units (`systemd-run`): `osmo-bts-virtual-gprs`, `osmo-pcu-gprs`, `osmo-virtphy-gprs`, `osmo-modem-gprs`.

```
systemctl enable --now osmo-egprs-dsp     # at boot, after osmo-banc
scripts/egprs-dsp.sh check                # preflight + configs, launches nothing
scripts/egprs-dsp.sh status               # units, SGSN mm-context and pdp-context, PCU TBFs
scripts/egprs-dsp.sh stop [--all]         # the cell and the modem (--all: the bench too)
scripts/egprs-dsp.sh start --l1 auto|virtphy|dsp [--no-banc] [--regen-gprs]
journalctl -u osmo-egprs-dsp -u osmo-modem-gprs -u osmo-pcu-gprs -f
```

Known limitation: the standard Calypso `layer1` firmware does not handle `L1CTL_GPRS_*` (no PDCH on the emulated ARM), so the script puts the modem on `virtphy`, in the same PLMN/LAC/RA as the DSP mobile camped on BTS 0. `EGPRS_L1=dsp` (or `--l1 dsp`) prepares the other path (`modem` instead of `mobile` in `c54x_exe/run.sh` through `MOBILE=`/`MOBILE_CFG=` in `/etc/default/osmo-banc`) for a firmware with PDCH support (`layer1.highram.gprs.elf`, built by `sudo ./addition.sh --calypso-fw`). The Calypso ROM only modulates GMSK: GPRS CS-1..4 at best on the C54x, 8-PSK stays with the virtual modem.

2.32.1.3 `osmo-dsp-rom-max` — the ROM does the maximum

`scripts/dsp-rom-max.sh run` writes the "ROM max" profile into `/etc/default/osmo-banc` (RACH, uplink SDCCH/SACCH and final bursts by the ROM, downlink A5 by the ROM, nothing canned), restarts `osmo-banc` on the DSP bench, runs the ladder of `/opt/GSM/tests/banc-max.sh --dsp --continue` (LU, SMS, call, coverage) and reads back `couverture.txt`: what the ROM did, what stayed on the host despite the profile, and what has no ROM path yet (uplink FACCH and SACCH-in-TCH, uplink speech, uplink A5 at the air fn, vocoding). `--fw-gprs` (default) uses the `layer1.highram.gprs.elf` firmware (PDCH, `L1CTL_GPRS_*`); the original firmware is used when it is absent. The unit is installed but not enabled: the profile persists, the campaign is launched on demand.

```
systemctl start osmo-dsp-rom-max          # or: scripts/dsp-rom-max.sh run
scripts/dsp-rom-max.sh report             # read the last report
scripts/dsp-rom-max.sh profile            # write the profile only
scripts/dsp-rom-max.sh run --no-restart --only=camp,attache,couverture
```

2.33 /opt/GSM/osmo-operator/wiki/Build.md

9178 octets, 196 lignes

2.33.1 Build — Docker image, ISO, `.deb` packages, native install

Four paths, one build engine. Everything that gets compiled (libosmocore, osmo-*, gapk, QEMU and its tree, osmocom-bb, the gr-gsm venv, the firmware) comes out as a `.deb` package and stays **on the host**, in `/var/cache/osmo-debs`. The first build compiles everything; later ones only compile what is missing.

```
build.sh ──► image osmocom-nitb ──► start-docker.sh / compose.sh      (Docker)
     │              │
     │              └──► build-iso.sh ──► ISO amd64 / .img arm64   (bootable)
     └──► /var/cache/osmo-debs ──► packaging/build-debs.sh ──► dpkg -i   (no git)
install.sh ──────────────────────────────────────────────────────►  native Ubuntu 24.04
```

2.33.1.1 1. `build.sh` — the Docker image

```
sudo ./build.sh [--no-cache] [--lite] [--stp] [--build-run] [--arch=arm64|--arm]
```

Option	Effect
(none)	`osmocom-nitb` image (Dockerfile): core + BTS + QEMU Calypso + gr-gsm + Asterisk
`--lite`	`osmocom-nitb:lite` image (Dockerfile.lite): without the workshops
`--stp`	`osmocom-stp` image (Dockerfile.stp): the inter-STP alone, arm64 possible
`--build-run`	also builds the `run` image (Dockerfile.run)
`--no-cache`	docker cache ignored **and** `OSMO_DEB_REFRESH=1`: everything is recompiled, the `.deb` cache rewritten
`--arch=arm64`, `--arm`	arm64 cross-build (Raspberry Pi 4)

What the script does, in order:

1. `cd` into the script's directory (the `.desktop` goes through pkexec, which starts in `/root`).
2. Puts `/usr/local/sbin` in the `PATH`: that is where `packaging/apt-fast-install.sh` installs `apt-fast` (aria2). Same installer on the host, in the images and in the ISO rootfs.

3. Installs Docker + compose v2 + buildx. `compose.yaml` describes the `nitb`, `run`, `lite`, `stp` services; `docker build` remains the fallback.
4. Copies `/var/cache/osmo-debs` into `.deb-cache/` (the docker context). The image installs these packages with `dpkg` instead of recompiling.
5. Builds. Every directory compiled in the image comes out as a `.deb` via `packaging/osmo-deb.sh`.
6. Brings what the image produced back into the host cache.

Variables:

Variable	Default	Role
`OSMO_DEB_CACHE`	`/var/cache/osmo-debs`	where the cache lives
`OSMO_DEB_REFRESH`	`0`	`1` = recompile and rewrite (what `--no-cache` sets)

```
./compose.sh debs                       # cache contents
sudo ./compose.sh build [--no-cache] [--lite] [--stp]   # = build.sh
sudo ./compose.sh up [--ms 2] [--phy faketrx] [--no-run]  # hub + osmo-operator-1, HLR populated
sudo ./compose.sh down | ps | logs | shell
```

First build time: 15-20 min (osmocom-bb + ARM toolchain + QEMU).

2.33.1.2 2. `build-iso.sh` — bootable ISO / SD image

```
sudo ./build-iso.sh [options]
```

No option: the four amd64 images — `interstp`, `operator` (normal), `lite`, `desktop`. A **single** docker build (`osmocom-nitb`), then each rootfs derives from it: interstp; normal, from scratch; lite = copy of normal with the workshops removed; desktop = normal + the desktop. Only exception: `--role=interstp` alone builds only `osmocom-stp`.

2.33.1.2.1 What to build

Option	Effect
`--role=operator|interstp`	a single image
`--node=N`	ISO pre-assigned to node N (otherwise the node is chosen at launch, `start-native.sh --node`)
`--lite`	image without workshops
`--desktop`	GNOME, bench status Conky, icons « Lancer le banc GSM », « multi-operator », « osmo-lte », « Pilotes graphiques »
`--all`	everything
`--arm`	arm64 SD image for Raspberry Pi 4, Armbian 24.04 base (`osmo-operator-<…>-rpi4.img`); `--lite` accepted, `--desktop` and `--all` refused
`--version=24.04|22.04`	Ubuntu base
`--kb=fr`	keyboard
`--output=fichier`	ISO file name

2.33.1.2.2 Where the Docker image comes from

By default (amd64) **the image is pulled, not built**, in this order: `ghcr.io/<repo>/osmocom-nitb:base-<tree fingerprint>` → `:latest` from the same repo → Docker Hub. `build.sh` only runs if none of them answers. When the fingerprint matches, the published image was built from exactly this tree; the other two references do not guarantee that, and the script says so. On arm64 nothing is published: it compiles by default.

Option	Variable	Effect
`--skip-build[=image:tag]`	`OSMO_ISO_SKIP_BUILD`	pull mandatory, no fallback to `build.sh`; `=image` forces the reference. `OSMO_ISO_SKIP_BUILD=0` = never pull
`--build-docker`	`OSMO_ISO_BUILD_DOCKER=1`	the opposite, **and it wins**: builds even if a `--skip-build` is set
`--no-cache`		passed to `build.sh`

2.33.1.2.3 What travels in the image

Option	Variable	Effect
(default)		the cached `.deb`s are installed into the rootfs with `dpkg`, then the rest of the image is copied; the packages stay in the ISO's `/var/cache/osmo-debs`
`--without-debs`	`OSMO_ISO_WITHOUT_DEBS=1`	the `.deb`s are used to install the stack but **do not stay**

2.33.1.2.4 What starts at boot

The `osmo-banc.service`, `osmo-multi.service`, `osmo-lte.service` units are **installed but not enabled**: a freshly installed machine boots to its desktop. The operator starts things (icon, `launch.sh`, `systemctl start osmo-banc`). See `services/README.md`.

Option	Variable	Effect
`--banc`	`OSMO_ISO_BANC=1`	enables `osmo-banc` at boot
`--multi`	`OSMO_ISO_MULTI=1`	enables `osmo-multi` (`Requires=osmo-banc`, docker)

On an already installed machine: `OSMO_MULTI_ENABLE=1 ./addition.sh`. Persistent service options: `OSMO_BANC_ARGS="--grgsm"` in `/etc/default/osmo-banc` (the DSP bench is the default).

2.33.1.2.5 WAN in the ISO

```
sudo ./build-iso.sh --wan --wan-nodes="1:IP:IND 2:IP:IND …" --wan-id=N --wan-ops=N --hub-ip=IP
```

Sets `WAN_AUTO=1` in `/etc/osmo-wan.conf`: the bench brings up the WAN without asking. Without `--node`, **a single ISO serves all nine nodes** — the number is deduced from the local IPs or passed with `start-native.sh --node N`.

2.33.1.2.6 Miscellaneous

Variable	Role
`OSMO_ISO_WORK`	working directory, **on disk** (not `/tmp`, often a tmpfs → « No space left on device »)
`OSMO_ISO_VERSION`	= `--version`

Variable	Role

The disk installed by Calamares: accounts from the « Utilisateurs » (Users) screen + root unlocked; LUKS offered (LVM in manual mode, `lvm2` included); the « Pilotes graphiques » (Graphics drivers) page = `apt install nvidia-driver-610` if `lspci` sees a card (network required), and the `tools/osmo-drivers.sh` icon afterwards. `/etc/osmocom/coeur.env` (written by the ISO) carries `N_MS`.

2.33.1.3 3. `packaging/build-debs.sh` — installing without git

```
./packaging/build-debs.sh          # osmo-operator, pont, qemu-calypso, calypso-firmware → packaging/dist/
sudo dpkg -i packaging/dist/*.deb  # refuses to install over a git clone at the same path
```

The `pont` package ships `pont.py` and `pont_dsp.py` (executables) and the `pont/dsp/` subpackage.

`packaging/snapshot-lte-debs.sh` freezes Open5GS / srsRAN; `packaging/build-pmos-kernel-deb.sh` the phone's postmarketOS kernel (PPP to the `ue1` netns).

2.33.1.4 4. `install.sh` — native, Ubuntu 24.04 (noble)

```
sudo ./install.sh                  # all steps
./install.sh --list                # the steps, without doing anything
./install.sh --check               # what is already in place
./install.sh --dry-run
sudo ./install.sh --reinstall      # replays everything: sources updated, configs and shortcuts rewritten
sudo ./install.sh --only bureau    # a single step
sudo ./install.sh --skip build     # all but one
```

The steps live in `install_modules/NN-<slug>.sh`: `prereqs`, `deps` (apt), `user` (osmocom user and group), `sources` (Osmocom repositories, never pulled if already cloned), `patches` (`tools/osmo-patches-apply.sh`, applies `patches/*.patch` to the trees under `/opt/GSM`, idempotent), `build`, `binaires`, `configs` (`/etc/osmocom`), `bureau`, `verify`. `GSM_ROOT` (default `/opt/GSM`) receives sources and binaries. `bureau` installs the same files as the ISO (`data/desktop/*.desktop`, `data/*.svg`) in the menu and on the desktop of root and of the `sudo` user. The native install **does not enable** the systemd units. Launching is still `./start-native.sh`, which delegates to `/opt/GSM/qosmo/run.sh`.

Optional components come from `addition.sh`: `--gnuarm` (ARM toolchain in `/root/gnuarm/install/bin`, needed to build osmocom-bb), `--calypso-fw` (recompiles the Calypso `layer1/rssi` firmware; `OSMO_FW_INSTALLER=1` copies the highram elf+bin into `/opt/GSM/firmware/board/compal_e88`, layer1 also as `layer1.highram.gprs.*`), `--magnetite`, `--trx`, `--firmwares` (all of the above), `--sim`, `--extras`, `--multi`, `--docker`, `--image`, `--status`. Plugging only the emulated phone (qosmo, c54x_exe, grgsm_exe) into your own Osmocom stack is described in Telephone-emule.

2.34 /opt/GSM/osmo-operator/wiki/Environnement.md

8620 octets, 163 lignes

2.34.1 Environment — who wins, and what reaches QEMU

The test bench is driven by environment variables, at three levels: `start-native.sh` (the preparer), `run.sh` + `run_modules/` (the orchestrator of the emulation tree), and QEMU itself (the `calypso` machine, which reads its `CALYPSO_*` at boot). A variable set in the wrong place, or with the wrong idiom, is silently reset to its default before reaching the last level. This page says where each one is read, by whom, and how to check what QEMU **actually** saw.

2.34.1.1 1. Load order

```
1. command line           VAR=x ./start-native.sh          always wins
2. /etc/osmocom/coeur.env N_MS                             `:=`, cannot override anything
3. options                --profile --dsp --node --wan …
4. globals.conf           26 network variables             `=`, but a non-empty caller wins (§ 2)
5. environment/load.env
     modes.env            the profile                      `:=`
     paths.env            where things are                 `:=`
     coeur/radio/reseau/debug.env                          optional, if present
     $OQC_ROOT/environnement/paths.env                     the emulation tree's paths
6. start-native.sh defaults     RUN_DIR, LOG_DIR, ENCRYPTION, CALYPSO_BRIDGE, MS_COUNT, HOST_IP
7. exec run.sh            the whole environment, intact
8. run_modules/40-qemu.sh → C launcher → QEMU   the manifest is written here
```

All environment files use `: "${VAR:=value}"` — "if not already set". So **the first one to set a value wins**, and that first one is the command line.

2.34.1.2 2. The exception: `globals.conf`

`globals.conf` uses `VAR=value` (not `:=`). On the 26 variables it declares — identity (MCC, MNC, OP_NAME), security (ENCRYPTION, SIM_ALGO, KI, IMSI, IMEI), radio (ARFCN, BAND, BSIC, LAC, CELL_ID, IPA_UNIT_ID, MS_MAX_POWER, RXLEV_ACCESS_MIN, CELL_RESEL_HYST), GPRS (BVCI, NSEI, NSVCI, APN), SMS_SC, MS_COUNT, N_OPERATORS, HOST_IP — it would overwrite the environment if it were sourced on its own. `start-native.sh` sets aside those of these variables that the caller has set **non-empty**, sources the file (`set -a`), then puts them back, and prints "globals.conf surchargé par l'environnement : …". The command line therefore also wins on these 26 variables; an empty value always means "computed" and overrides nothing. Only `start-native.sh` applies this rule (`generate_configs.sh` does not).

Everything it does not declare (`CALYPSO_*`, `MODE`, `LOG_DIR`, `QEMU_*`…) passes through intact. To change one of its 26 values:

```
vim globals.conf
./generate_configs.sh ARFCN=520      # rewrites the line
./generate_configs.sh --force        # regenerates the whole file
```

An empty field = computed (often from the operator number). Only pin the `[auto/operator]` fields if `N_OPERATORS=1`, otherwise the operators step on each other.

Example: `ENCRYPTION="a5 0" ./start-native.sh --dsp` gives a5 0. With nothing on the command line, the value comes from `globals.conf`; the script's `: "${ENCRYPTION:=a5 1}"` is only a fallback for the bare ISO or a container without `globals.conf`.

2.34.1.3 3. `:=` versus `=` in your own files

Syntax	Effect
`: "${VAR:=value}"`	default; the command line wins
`VAR=value`	**locks it**; no one can override

Use `=` only for what must never be overridden. A `=` slipped into a per-domain file is the classic cause of "my variable has no effect".

2.34.1.4 4. The four gate idioms — `=0` doesn't turn everything off

The QEMU code (`hw/arm/calypso/`) reads its variables in four ways. How to **turn off** a variable depends on the idiom, not on the value:

Idiom in the code	Active when	To turn off
`getenv("X") ? 1 : 0`	the variable **exists**, even at `0`	**`unset X`**
`atoi(getenv("X")) > 0`	value > 0	`X=0`
`*e == '1'`	exactly `"1"`	any other value
default ON + `X_OFF`	by default	set `X_OFF`

So `FB_ENERGY=0 ./start-native.sh` can leave `FB_ENERGY` **active**. Check the idiom at the read site: `grep -rn '"X"' /opt/GSM/qosmo/hw/arm/calypso/`.

2.34.1.5 5. Variables that set other variables

Some variables are **profiles**: `CALYPSO_NATIVE_HELPED=1` also turns on `FB_CORR_ENTRY`, `FB_ENERGY` and `FB_IQ_*`. Removing one of them from the command line does not remove it — it reverts to its default, set by the profile.

2.34.1.6 6. The truth is the manifest

Neither the command line, nor `env`, nor the files say what QEMU saw. At boot, the `calypso` machine writes a manifest to its log:

```
grep "calypso-manifest" /var/log/osmocom/qemu.log
```

This is the only reliable source. If a variable is not there at the expected value, it was reset somewhere between 1 and 8 — walk back up the chain in § 1.

2.34.1.7 7. What reaches QEMU

Variable	Default	Read by	Role
`CALYPSO_PROFILE` / `MODE`	`faketrx-qemu` (script), `core` (modes.env)	modes.env, run.sh	the profile; `MODE` wins if set
`CALYPSO_FORK`	`qosmo`	paths.env, start-native	name of the emulation tree
`OQC_ROOT`	`../<fork>` else `$GSM_ROOT/<fork>`	start-native	the emulation tree; `OQC_ROOT=/path ./start-native.sh`
`RUN_SH`	`$OQC_ROOT/run.sh`	start-native	the `run.sh` that is run
`QOSMO_LAUNCHER`	`/usr/local/bin/<fork>`	40-qemu.sh	the C launcher; absent → `qemu-system-arm`
`CALYPSO_BRIDGE`	`pont` (`none` with `--dsp`)	run.sh	`none` = QEMU + BTS only; with `--dsp`, `c54x_exe/run.sh` launches `pont_dsp.py`
`CALYPSO_NO_ATTACH`		start-native	`1` = do not attach to the tmux (`osmo-banc.service`)
`CALYPSO_LANG`		run.sh	language of the messages
`QEMU_BIN`	`$OQC_ROOT/build/qemu-system-arm`	40-qemu.sh	binary (without launcher)
`QEMU_FW`	`$GSM_ROOT/firmware/board/compal_e88/layer1.highram.elf`	launcher	ARM firmware
`QEMU_DSP_ROM`	`$GSM_ROOT/calypso_dsp.txt`	launcher (`-dsp`)	C54x mask ROM
`QEMU_L1CTL_SOCK`	`/tmp/osmocom_l2`	launcher, mobile	L1CTL socket
`QEMU_MON_SOCK`	`/tmp/qemu-calypso-mon.sock`	launcher	HMP monitor
`RUN_DIR` / `LOG_DIR`	`/run/user/<uid>/osmo-nitb` / `$RUN_DIR/logs`	everything	tmpfs — the tmux scrollback depends on it
`GSM_ROOT`	`/opt/GSM`	paths.env	root of the installation
`N_MS`	`1` (coeur.env)	21-abonnes-hlr.sh	subscribers provisioned in the HLR

Variable	Default	Read by	Role
`MS_COUNT`	`2`	start-native, globals	mobiles launched
`PHY_MODE`	`faketrx`	run.sh (Docker)	`faketrx` / `virtphy` / `qemu`; `qemu` forces `N_MS=1`
`OSMO_STP`, `OSMO_MSC`, …	bare name (PATH)	paths.env	Osmocom binaries; empty = packages

DSP bench (`--dsp`, the default): these variables are read by **c54x_exe** and its `run.sh`, not by QEMU — so they do not appear in the `calypso` manifest.

Variable	Default	Read by	Role
`BANC_DSP`	`$GSM_ROOT/c54x_exe/run.sh`	start-native	the bench launched after the emulation tree's plan; `none` to turn it off
`INSNS`	`120000`	c54x_exe (`--insns`)	DSP instruction ceiling per frame
`LOCKSTEP`	`1`	c54x_exe/run.sh	QEMU only advances the frame once the DSP has finished the previous one (`CALYPSO_PONT_LOCKSTEP=1`); `0` = wall clock
`IQ`	`none`	c54x_exe/run.sh	no synthetic cell: the bursts come from the BTS
`CALYPSO_BSP_STREAM`	`1` (start-native)	c54x_exe (`calypso_bsp.c` from `qosmo/hw/arm/calypso/l1-dsp`)	one TS0 burst per frame, in FN order; `IQ` other than `none` is refused with it
`CALYPSO_RHEA_DMA_XFER`	`1` (start-native)	nobody (the DMA2 transfer is unconditional); `src/pont.c` only prints it	Known limitation: dead variable

Workarounds for unimplemented emulator branches are annotated `@BEQUILLE` at their read site, with what they hide and the condition that would make them unnecessary: `grep -rn "@BEQUILLE" /opt/GSM/qosmo/hw/arm/calypso/`. The emulation tree's own variables are described in `/opt/GSM/qosmo/environnement/README.md`.

2.34.1.8 8. Verification recipe

```
./start-native.sh --list                          # the plan: profile, tree, run.sh, launcher
./start-native.sh --dry-run --verbose             # unrolls it, without launching
./start-native.sh --check-paths                   # the declared dependencies
X=1 ./start-native.sh && grep calypso-manifest /var/log/osmocom/qemu.log   # what QEMU saw
grep -rn '"X"' /opt/GSM/qosmo/hw/arm/calypso/     # the read idiom
```

If `X` is not in the manifest: either `globals.conf` declares it (§ 2), or a file sets it with `=` (§ 3), or a profile resets it (§ 5), or it is only read by c54x_exe (§ 7).

2.35 /opt/GSM/osmo-operator/wiki/Home.md

11984 octets, 214 lignes

2.35.1 osmo-operator — Quick start & reference

A complete **GSM + LTE** operator in a single image, with the radio emulated end to end down to the baseband (Calypso under QEMU), and a postmarketOS phone that does data over 4G and voice/SMS over 2G on the same identity — a real handset's CSFB, at the modem level. No SDR, no physical phone.

Page	Contents
Home (here)	recommended quick start (the ISO), launching, reference
Start-native	`start-native.sh` step by step, options manual
Resultats	reference `check_all.sh` runs, 1 and 3 operators, and how to read the deviations
Build	`build.sh`, `build-iso.sh`, `.deb`, `install.sh`
ISO	everything the desktop ISO contains, how it is built and installed
Environnement	which variable wins, what makes it all the way to QEMU
Reference	full reference: multi-PLMN architecture, SS7, SMS, voice, diagnostics
Telephone-emule	plug **only** the emulated phone (qosmo, c54x_exe, grgsm_exe) into **your own** Osmocom stack
Installer-qosmo · Installer-c54x_exe · Installer-grgsm_exe	per-repository install of the emulated phone's components
Lancer-a-la-main · lancer-pas-a-pas.ipynb · lancer-les-modes.ipynb	the phone launched process by process, no script, with proof at every step
SIM	the bench's card in an Omnikey HID reader (sysmoISIM programmed by pySim) or softSIM, served to the mobile over SAP: `start-native.sh --sap`, `tools/osmo-sim.sh`
WAN	linking two nodes (LAN or Internet via WireGuard): ports, SS7 hub, `--wan-nodes`, layer-by-layer checks, failures
Payload-ISM	SMS **text** between two nodes over an ISM-band radio link, everything else (SS7, SIP, addresses) over IP: `start-native.sh --wan --payload-ism`, `network/setup-payload-ism.sh`
README-notebook · installer-osmo-jusquau-dsp.ipynb	the install notebook: fresh Ubuntu 24.04 to a call on the DSP, step by step, and how to run it

---

2.35.1.1 Recommended method: the desktop ISO (release)

This is the path that works without compiling or configuring anything: the latest release ships `osmo-operator-desktop.iso`, a GNOME live system with Wireshark, Linphone, the I/Q spectra and the three launchers in the dock.

2.35.1.1.1 1. Download and reassemble

GitHub caps a release file at 2 GiB, so the image ships in pieces. **Download all the `part-*`**, then:

```
cat osmo-operator-desktop.iso.part-* > osmo-operator-desktop.iso
sha256sum -c SHA256SUMS
```

2.35.1.1.2 2. Requirements

x86_64, **8 GB of RAM and 4 cores minimum**. The live root is a tmpfs (~6 GB with `toram`): logs and captures there are capped and purged at every boot. The bench runs **two QEMUs** (the Calypso and the pmOS VM): inside a virtual machine, enable **nested virtualization**, otherwise the pmOS VM falls back to software emulation and becomes very slow. On real hardware KVM is available directly: that is the most comfortable setup.

2.35.1.1.3 3. Launch

QEMU

```
qemu-system-x86_64 -cdrom osmo-operator-desktop.iso -m 8G -enable-kvm \
  -cpu host -smp 4 -nic user,hostfwd=tcp::8080-:8080
```

VirtualBox — new Linux / Ubuntu 64-bit VM, 8 GB, 4 CPUs, 3D disabled. System → Processor: tick *Enable Nested VT-x/AMD-V* (if greyed out: `VBoxManage modifyvm "<name>" --nested-hw-virt on`). Attach the ISO to the optical drive, no disk. NAT network: port forward host 8080 → guest 8080.

VMware (Workstation / Player / Fusion) — VM from the ISO, Linux / Ubuntu 64-bit, 8 GB, 4 cores, tick *Virtualize Intel VT-x/EPT or AMD-V/RVI*, skip easy install.

USB stick — the ISO is hybrid:

```
sudo dd if=osmo-operator-desktop.iso of=/dev/sdX bs=4M status=progress conv=fsync
```

On Windows, Rufus in *DD Image* mode. UEFI or BIOS boot, either works.

2.35.1.1.4 4. Start the bench

Nothing starts at boot: the machine lands on its desktop. Three launchers in the dock, 2G and 4G independent and in any order, **the smartphone last**:

Launcher	Starts
2G — the red handset	osmo-stp / bsc / msc / hlr / sgsn / ggsn / pcu, the Calypso under QEMU, both BTSs, both osmocom-bb mobiles, Asterisk, the SMSC
4G — the signal bars	open5gs, srsENB + srsUE over ZMQ, the SGs interface to the MSC (CSFB)
Smartphone — phone icon	the postmarketOS VM: oFono on the virtual modem, data over 4G, voice and SMS over 2G

Passwords: live **`osmo`**, pmOS phone **`147147`**.

Check that everything is up: `checks/diag-stp-operator.sh`, or the full check `bash ./checks/check_all.sh` (reference result for a standalone native node in Resultats: everything passes, only the SS7 interconnect is *down* for lack of an inter-STP — that is expected).

2.35.1.1.5 5. Make a call

The two provisioned subscribers are **100101** and **100102**, one per cell; `100`/`200` are the Linphones, `600` the echo test. An incoming call to the smartphone is paged by the MME, the UE falls back to the BTS (CSFB), voice goes over 2G; the speedtest, on the other hand, goes over 4G. Everything is visible in Wireshark (GSMTAP, cleartext NAS, SGs, M3UA).

2.35.1.2 Other paths

Path	When	See
Docker — `docker pull bastienbaranoff/norf_gsm`, `sudo ./launch/start-docker.sh` (or `./start.sh --docker`)	N operators interconnected through an inter-STP, on an existing host	README-docker
Native — `sudo ./install.sh` then `./start-native.sh`	a dedicated Ubuntu 24.04 machine, no container	Build § install.sh
Build the ISO yourself — `sudo ./build-iso.sh`	modify the image, pre-wired WAN, arm64 for the hub	Build § build-iso.sh
Install to disk — Calamares from the live system	keep the bench, LUKS, NVIDIA drivers	Build

---

2.35.1.3 Reference

2.35.1.3.1 What is in the image

`/etc/os-release`: `IMAGE_VERSION="OSMO_EGPRS_V2"` — `/etc/osmo-role`: `OSMO_ROLE=operator`, `OSMO_LITE=0`.

2G / GSM — Osmocom

- Core: osmo-stp (M3UA, SCTP :2905), osmo-bsc, osmo-msc, osmo-hlr, osmo-sgsn, osmo-ggsn (GTPv1-C/U), osmo-pcu, osmo-mgw (MGCP :2427).
- Two DCS1800 cells, LAC 1: BTS 0 (Cell ID 6001, BSIC 7) and BTS 1 (Cell ID 6012, BSIC 8), each on its own osmo-bts-trx.
- Layer 1 (default, `./start.sh --bts-classique --ms`): `osmo-bts-trx` talks to `pont/pont_dsp.py`; the emulated phone (`qemu-system-arm -M calypso`, qosmo) runs osmocom-bb `layer1` + `mobile` on the C54x DSP mask ROM. Other modes: `--ms-magnetite` (FreeCalypso Magnetite on a C139), `--bts` (the Calypso as the BTS radio, `--ts 1 | 2 | 2/1`), `banc-rssi`.
- GPRS/EDGE: BSSGP/NS to the PCU, PDP contexts through osmo-ggsn.
- Voice: external MNCC → Asterisk (SIP :5060, FR/HR) via osmo-sip-connector.
- SMS: SMS-over-GSUP → proto-smsc-daemon, SMPP :2775 with a test ESME, inter-operator relay :7890.
- A3A8 authentication with deterministic RAND (`*-force-rand-toy` patches): reproducible from one boot to the next.
- Live GSMTAP in Wireshark.

4G / LTE — open5gs + srsRAN

- open5gs core (MME, SGW-C/U, SMF, UPF, PCRF, HSS…) on dedicated loopbacks; Prometheus :9090, WebUI :9999, MongoDB behind the HSS.
- PLMN 001/01, TAC 7. NAS security with null ciphering (EIA2/EIA1 + EEA0), on purpose: the NAS reads in cleartext.
- srsENB 10 MHz, EARFCN 3350, band 7; srsUE connected over ZMQ. Milenage subscriber IMSI 001010001000001. The UE has an IP in a netns and goes out through NAT.

CSFB

- SGs interface osmo-msc ↔ MME (:29118), TAI table (001-01, TAC 7) → LAI (001-01, LAC 1): the VLR files the subscriber under LAC 1 while it is on LTE.
- Combined EPS+IMSI attach; incoming call paged by the MME, the UE falls back to the BTS.

postmarketOS

- x86_64 VM, patched kernel (`CONFIG_PPP`, missing from upstream `linux-postmarketos-stable`) shipped prebuilt (Git LFS) in `/opt/user_interface/kernel/pmos/`.
- 4G data: NetworkManager → pppd (`ATD*99***1#`) → virtio-console `osmo.data` → `osmo-phonesim-banc.py` → UE netns → srsUE → srsENB → UPF → Internet.
- 2G voice/SMS: second virtio-console `osmo.modem`, `osmo-phonesim-banc.py` plays the 27.007 AT modem (state read from the bsc, msc and mobile VTYs), oFono drives it.

- Audio: two full-duplex PulseAudio chains (intel-hda) — a GAPK bridge to the osmocom-bb mobile, and a chain with echo cancellation. Slimmed down at every start (speech-dispatcher stopped, `osmo_rec` at 8 kHz mono, stream-restore without device restore). speex canceller by default; `AUDIO_AEC_METHOD=webrtc` selects the webrtc canceller (its `aec_args` are passed as a single argument).

2.35.1.3.2 Ports and access

Port	Service	Port	Service
4239	osmo-stp VTY	8080	bench dashboard
4242	osmo-bsc VTY	8081	I/Q spectra (fft-web)
4243	osmo-mgw VTY	8090	settings page (`osmo-reglages`)
4254	osmo-msc VTY	9090	open5gs Prometheus
4258	osmo-hlr VTY	9999	open5gs WebUI
4729	GSMTAP (Wireshark)	5060	Asterisk SIP
2905	local M3UA	2775	SMPP
2908	inter-STP M3UA	7890	inter-op SMS relay
		29118	SGs (MSC ↔ MME)

```
tmux attach -t calypso          # the 2G stack; Ctrl-b d detaches
journalctl -u osmo-banc -f      # if started as a service
./start-native.sh --status | --stop
./ss7-console.py                # browsable SS7 diagram, built-in VTYs
```

2.35.1.3.3 Numbering plan

Number	Use
`100101`, `100102`	the bench's two subscribers (one per cell)
`N0001…N9999`	subscribers of operator N (multi-op)
`100` / `200`	Linphone A / B
`600`	echo test
`9XXXXX`	inter-op breakout from a softphone

2.35.1.3.4 Most useful options

Command	Effect
`./start-native.sh --list`	the plan, without launching
`./start-native.sh --menu`	asks the questions instead of guessing
`./start-native.sh --regen`	regenerates the configs from the templates
`./start-native.sh`	C54x DSP bench (c54x_exe, TI mask ROM), **the default**: the mobile camps, does its LU, SMS and calls
`./start-native.sh --grgsm`	gr-gsm layer 1 inside QEMU
`./start-native.sh --node 2 --hub-ip IP`	this node = node 2 of a WAN, ASP towards the inter-STP
`./start-native.sh --wan`	N-node WAN, interactive questions
`./start-native.sh --sap`	SIM served over SAP (reader or softSIM)
`./generate_configs.sh ARFCN=520`	changes a value in `globals.conf`
`sudo ./build-iso.sh --desktop`	rebuild the desktop ISO
`bash ./checks/check_all.sh`	full check, log in `/tmp`

2.35.1.3.5 Files worth knowing

File	Role
`globals.conf`	the network configuration; a non-empty variable set on the command line overrides it
`environment/load.env`	the loading order of profiles and defaults (`environment/README.md`)
`/etc/osmocom/coeur.env`	`N_MS`, written by the ISO
`/etc/osmo-wan.conf`, `/etc/osmo-role`	WAN table, node role
`/etc/default/osmo-banc`	`OSMO_BANC_ARGS="--grgsm"` to make the service start in gr-gsm (DSP otherwise)
`/var/log/osmocom/{qemu,bridge,run.sh}.log`	QEMU, bridge, orchestration
`/tmp/c54x-pont/`, `/dev/shm/pont.log`	DSP bench: links to its logs (real files at the dashboard paths), bridge log
`pont/README.md`	the TRX bridge: interfaces, ciphering rules, variables, known issues
`services/README.md`	the systemd units and how to run the bench as a service
`patches/`	patches applied to the trees under `/opt/GSM` by `tools/osmo-patches-apply.sh`

2.36 /opt/GSM/osmo-operator/wiki/ISO.md

50229 octets, 765 lignes

2.36.1 The osmo-operator ISO: contents, build, installation, capabilities

This page describes **everything the ISO is**: what it contains, how it is built (`build-iso.sh` and its modules), how it boots, how it installs to a disk, and what it can do once booted. It points to the repository files for each item; the Build, Start-native and Environnement pages detail the scripts themselves. The reference image is the published **osmo-operator-desktop.iso**; what changed in each release is listed in `.github/release-notes.md`.



Figure 4: The ISO desktop: Telecom Operator, no RF, full stack

---

2.36.1.1 What's inside: a mobile operator on one USB stick

Boot it, click one icon, and you are running a **whole GSM network with its phones**: no radio, no SDR, no SIM card required. Everything below is in the image. Nothing starts by itself; you start it.

- **A real 2G core, end to end**: osmo-bsc, osmo-msc, osmo-hlr, osmo-stp, osmo-sgsn/ggsn, osmo-pcu (GPRS/EDGE), osmo-mgw, Asterisk, an SMSC. Two cells, two subscribers, A5/1, location update, SMS, **audible voice calls**, GSMTAP straight into Wireshark. See 5.1.
- **Real phone firmware on an emulated phone**: a QEMU fork (`qosmo`) runs the Calypso baseband of a Motorola C123 with the genuine OsmocomBB firmware, and the **TI TMS320C54x DSP runs on its own mask ROM**: the emulated phone syncs to the cell (FCCH/SCH), reads SI1-4 and camps. See 5.2, 5.3.
- **The phone can also *be* the base station**: `./start.sh --bts` turns the emulated Calypso into the transceiver of the BTS; `--bts --ms` adds a second Calypso that camps on it **over an emulated air interface**. A fourth one runs the RSSI app. One command each: `banc-trx`, `banc-mobile`, `banc-rssi`. See 5.13.
- **Hold the phone in your hand**: `c123` opens a photo of the Motorola C123 with its live LCD inlaid and clickable keys (or the PC keyboard). See 5.13.
- **4G next to it, with fallback to 2G**: Open5GS core, srsENB and srsUE over ZeroMQ, data out to the Internet, and **CSFB** over SGs back to the 2G MSC. See 5.6.
- **A smartphone on the network**: a postmarketOS/Phosh VM whose modem is the bench: SMS and voice through 2G, data through 4G. See 5.7.

- **Several operators, roaming between them**: SS7 with an inter-STP hub, 1 to 9 nodes across machines (`--node`, `--wan`), operators in Docker next to the native one. See 5.4, 5.5.
- **A desktop made for the bench**: GNOME with a city wallpaper that **raises its antenna when a network starts**, a live I/Q spectrum panel, a timeslot banner, a Conky status board, a **GLava spectrum circle driven by the speakers and the microphone**, Wireshark as root with ready-made filters, Linphone on extension 100, a web dashboard and a tutorial. Every launcher opens a progress window, never a bare terminal. See 5.8.
- **Tools to look inside**: gdb on the emulated baseband, VTYs for every daemon, `ss7-console.py`, `checks/check_all.sh`, a ring-buffer tcpdump. See 5.10, 5.11.
- **Live, in RAM, or installed**: hybrid ISO (UEFI/BIOS, Secure Boot), `toram` entry, persistence, and a Calamares installer with an optional LUKS2-encrypted `/home`. See 3, 6.
- **One click for more**: the *Supplements* icon adds the multi-operator Docker bench, OpenCL + deka (A5/1 tables), Claude Code, games and media, and extra phone firmwares (FreeCalypso Magnetite on a C139, MediaTek Fernvale, osmocom-bb-trx). See 5.5.
- **On ARM boards too**: the same stack as SD images built by armbian/build for the **Raspberry Pi 4** and **every Orange Pi 5** (5, 5B, 5 Plus, 5 Pro, 5 Max, 5 Ultra), with or without the bench desktop. See 2.

2.36.1.2 1. At a glance

Item	Value
Base	Ubuntu 24.04 (noble), kernel `linux-image-generic`
Size (desktop)	ISO about 6.5 GB, squashfs about 6.3 GB (zstd -19, about 41 % of the 15.9 GB rootfs)
Boot	hybrid ISO (dd to a USB stick, UEFI or BIOS), Secure Boot shim, grub, kernel; `toram` entry (needs about 9 GB of RAM)
Live accounts	`root` autologin (password `osmo`), `osmocom` sudoer (password `osmo`), pmOS phone `147147`
Build	`sudo ./build-iso.sh --desktop`, about 2 h 15 on 16 cores; Docker image pulled from ghcr.io, no recompilation; minimal debootstrap + the build's `.deb`s (about 41); about 1,650 packages in the chroot; dpkg integrity check before and after squashfs
2G	full bench (`launch.sh`, `osmo-banc`), 2 MS, A5/1, inter-operator SMS, MSC with SGs (CSFB)
Emulated baseband	QEMU `qosmo` (ARM7 + C54x DSP), DSP ROMs, layer1 firmware for the compal boards, gdb/telnet, `c54x_exe`, `grgsm_exe`
4G	portable srsENB/srsUE + srsGUI, Open5GS, freeDiameter TLS, MME SGsAP, HSS console `:9999`, subscribers in Mongo
Phone	postmarketOS VM (about 1.7 GB compressed, PPP kernel), patched pmbootstrap
Desktop	minimal GNOME, Firefox (Mozilla deb, uBlock), Wireshark, Linphone (extension 100), Kodi, dino, oFono, Conky, Plymouth `osmo-bts`, web dashboard + tutorial
Audio	system-wide PulseAudio, PipeWire for screencast only, PCM `gsm_out`/`gsm_in`
Not included	avahi, Chromium, sssd (masked), pipewire-pulse
Installation	Calamares (`osmo-install`), optional LUKS2-encrypted `/home`, `osmo-update` at boot
Services installed, not enabled	`osmo-banc`, `osmo-multi`, `osmo-lte` (`--banc`, `--multi`, `OSMO_ISO_LTE=1` at build time to enable them)

2.36.1.3 2. Images produced

A single chain (`build-iso.sh` + `iso_modules/`) produces the four amd64 images, built back to back with no option (`--all`); `--arm` hands over to the Armbian framework for the SD images.

Image	File	Role	Contents
SS7 hub	`interstp.iso`	`--role=interstp`: the inter-STP alone (PC 0.0.0), no operator; rejects `--lite` and `--desktop`	`osmocom-stp` image (`Dockerfile.stp`), libosmocore/netif/sigtran only, `osmo-interstp.service` at boot
Operator	`osmo-operator.iso`	`--role=operator` (default): the full bench, no desktop	2G stack, emulated baseband, 4G, dashboard, tools; text console
Lite	`osmo-operator-lite.iso`	`--lite`: the same minus the build workshops	`/opt/GSM` stripped of source trees (`iso_modules/87-lite.sh`, same rules as `Dockerfile.lite`)
Desktop	`osmo-operator-desktop.iso`	`--desktop`: + GNOME, Wireshark, Linphone, VLC, Calamares, Conky, Firefox (about 2.5 GB more)	the image published in releases
ARM boards	`Armbian-..._<Board>_noble_...img` (CI: `osmo-operator-<board>[-desktop].img`)	`--arm`: one arm64 SD image **per board**, built by armbian/build (`armbian/build-armbian.sh`): `rpi4b` and every Orange Pi 5 (`orangepi5`, `orangepi5b`, `orangepi5-plus`, `orangepi5pro`, `orangepi5-max`, `orangepi5-ultra`); `--board=rpi4|opi5|all|<board>`, default `all`	Armbian builds kernel, BSP, u-boot/firmware, rootfs and image; the stack comes from the arm64 Docker image through `armbian/userpatches/customize-image.`. `--desktop`: GNOME (Armbian configng, tier minimal) + the bench desktop, without GLava (OpenGL 4.2 missing on these GPUs). No 4G, no `--lite`. The first Docker build is emulated: about ten hours

With `--node=N` (1 to 9), the name becomes `osmo-operator-N[-lite][-desktop].iso` and the image carries the SS7 identity of node N. `--output=FILE` forces the name.

2.36.1.4 3. Getting it, booting, finding your way

2.36.1.4.1 3.1 Download

The latest release ships the desktop ISO **in 1,900 MB parts** (GitHub caps a release file at 2 GiB):

```
cat osmo-operator-desktop.iso.part-* > osmo-operator-desktop.iso
sha256sum -c SHA256SUMS
```

2.36.1.4.2 3.2 Machine requirements

x86_64, **8 GB of RAM and 4 cores minimum**. The bench runs two QEMUs (the Calypso and the postmarketOS VM): in a virtual machine, enable nested virtualization. On real hardware, KVM is available directly.

2.36.1.4.3 3.3 Booting

- **QEMU**: `qemu-system-x86_64 -cdrom osmo-operator-desktop.iso -m 8G -enable-kvm -cpu host -smp 4 -nic user,hostfwd=tcp::8080-:8080`
- **VirtualBox**: Linux/Ubuntu 64-bit VM, 8 GB, 4 CPUs, 3D disabled, *nested VT-x/AMD-V* checked, NAT with port forwarding 8080 to 8080.
- **VMware**: VM from the ISO, *Virtualize Intel VT-x/EPT* checked, skip Easy Install.
- **USB stick**: `sudo dd if=osmo-operator-desktop.iso of=/dev/sdX bs=4M status=progress conv=fsync` (Rufus in *DD Image* mode on Windows). UEFI or BIOS, either works.

2.36.1.4.4 3.4 The GRUB menu (`iso_modules/90-iso.sh`)

Entry	Kernel line	Use
Run from medium (default)	`boot=live quiet`	the stick stays plugged in, root on a tmpfs overlay
In RAM	`boot=live toram=filesystem.squashfs`	copies only the squashfs to RAM (about 9 GB), the stick can be removed
Persistence	`boot=live persistence persistence-encryption=none quiet`	`persistence` partition on the stick to keep changes

The live stick boots **without** `splash`: Plymouth does not show there. The `osmo-bts` theme (mast, handset and waves) appears on an installed disk, where GRUB boots with `quiet splash`.

2.36.1.4.5 3.5 Secure Boot (`iso_modules/91-secure-boot.sh`)

The ISO is assembled by xorriso with the **Canonical-signed shim** and the signed GRUB (`shim-signed`, `grub-efi-amd64-signed`), plus a `/.disk/info` so the signed GRUB can find the medium. The shim, grub, Ubuntu kernel chain is complete: the image boots with Secure Boot on. Without these packages on the build host, the ISO comes out unsigned and the build says so.

2.36.1.4.6 3.6 The live system: space, logs, network (`83-espace-ram.sh`, `81-cloture-systeme.sh`)

- `/dev/shm` and `/tmp` are capped as a percentage of RAM; I/Q cfiles, logs and captures live there and are purged at every boot. A `tcpdump` wrapper writes a pcap ring buffer.
- No network wait at boot: the `*-wait-online` units are masked, shutdown is capped at 30 s. DNS through the `systemd-resolved` stub with a fallback `resolv.conf`. NetworkManager ignores `apn*`, `tun*`, `veth*`, `docker*`.
- `osmo-ip-plan.service` derives the node's private addresses; `osmo-qemu-link` links `QEMU_BIN` and rebuilds the `qosmo` launcher if needed.
- `osmo-update.service` does a fast-forward `git fetch` of the embedded repositories at every boot (never a reclone, never apt), log in `/var/log/osmo-update.log`.
- When an interactive shell opens, an "SMS delivered" animation runs once (`/usr/local/sbin/osmo-sms.sh`) and reinstalls the icons.

Account	Password	Role

2.36.1.4.7 3.7 Accounts and sessions (`84-comptes.sh`)

Account	Password	Role
`root`	`osmo`	the bench's working account: autologin on tty1, ttyS0 (ttyAMA0 on Pi) and in GDM
`osmocom`	`osmo`	unprivileged, sudoer (sudo, adm, dialout, audio, video, pulse-access, docker); session selectable after logout
`linphone_A`	none	pre-provisioned SIP account: extension **100**, UDP to 127.0.0.1:5060
pmOS `user`	`147147`	Phosh screen, account and ssh of the postmarketOS VM

The bench runs as root. Icons elevate via `pkexec`, the command line via `sudo`.

2.36.1.5 4. What the image contains

2.36.1.5.1 4.1 Useful tree

Path	Contents
`/opt/GSM/osmo-operator`	**the full repository, with its `.git`** (clone of the `main` branch at build time, `51-depot.sh`): scripts, configs, services, tools, wiki
`/opt/GSM/qosmo`	Calypso QEMU fork (`run.sh`, `run_modules/`), C launcher `tools/qosmo-launch`
`/opt/GSM/c54x_exe`	the TMS320C54x DSP outside QEMU, TI mask ROM in `rom/`, its `run.sh` (5 steps)
`/opt/GSM/grgsm_exe`	the gr-gsm layer 1 outside QEMU
`/opt/GSM/firmware/board/compal_e88/`	OsmocomBB firmware: `layer1.highram.elf`, `layer1.highram.gprs.*`, `trx.highram.*`, `rssi.highram.*`, loaders
`/opt/GSM/osmocom-bb`	osmocon, mobile, trxcon
`/opt/user_interface/pmos/`	patched pmbootstrap, PPP kernel, VM image `image/qemu-amd64.img.zst` (if embedded), `bin/` launchers
`/etc/osmocom/`	generated Osmocom configs (PLMN 001-01, A5/1, 2 MS), `coeur.env` (`N_MS`, `OPERATOR_ID`), `osmo-multi.conf` after the add-ons
`/etc/asterisk/`, `/etc/asound.conf`	voice PBX, PCM `gsm_out`/`gsm_in`
`/var/cache/osmo-debs/`	the bench `.debs` (osmo-operator, bridge, qemu-calypso, calypso-firmware) for a git-less install
`/usr/share/osmo-operator/`	`info.html` (Info page), SVG icons
`/var/log/osmocom/`	`qemu.log`, `bridge.log`, `run.sh.log`

2.36.1.5.2 4.2 Packages and software chain (`80-chroot.sh`, `50-injection-image.sh`, `81-cloture-systeme.sh`)

- Base: `debootstrap --variant=minbase` noble, systemd, dbus, kmod, zstd, curl, iproute2, `live-boot`, generic kernel, Osmocom libs, tshark, openssh, cryptsetup.
- Bench: asterisk, pulseaudio, ffmpeg, MongoDB 8.0, NodeSource (dashboard), srsRAN and Open5GS dependencies, `build-dep gnuradio`, venv `/root/.env`.
- **The compiled binaries come from the `osmocom-nitb` Docker image** as `.debs` (`osmo-build-*~noble_amd64.deb`) installed with `dpkg -i`, then `/usr/local/*`, `/opt/GSM`, `/root/.env`, `/opt/node`, `/etc/osmocom`, `/etc/asterisk` and the `osmo-*.service` units are copied from the container.
- **`ldd` closure**: every `.so` used in the Docker image is copied into the rootfs; `/usr/local/lib` goes first in `ldconfig`. The runtime environment is the image's, not a different apt version.
- Desktop: `ubuntu-desktop-minimal`, wireshark, linphone-desktop, vlc, calamares, conky, `python3-gi`, `python3-gi-cairo`, `python3-cairo`, `gir1.2-gtk-3.0`, `gir1.2-vte-2.91`, GLava (compiled, `85-glava.sh`), Firefox from packages.mozilla.org, GDM with root autologin, no Wayland.
- Integrity check: the rootfs is verified against dpkg **before** compression, then the squashfs is read back and re-verified (`90-iso.sh`).

2.36.1.6 5. Capabilities, one by one

General rule: **nothing 2G/4G starts by itself at boot**. Starting is the operator's call, via the icon, the command line or `systemctl`. The units are installed and ready. The launcher is `start.sh`; `start-native.sh` is a symlink to it.

2.36.1.6.1 5.1 The standalone 2G bench

Full native Osmocom stack, one operator (PLMN 001/01), two DCS1800 cells (ARFCN 514, LAC 1; BTS0 Cell ID 6001 BSIC 7, BTS1 Cell ID 6012 BSIC 8), subscribers `100101` and `100102`.

- Components: osmo-stp, osmo-bsc, osmo-msc, osmo-hlr, osmo-sgsn, osmo-ggsn (GTPv1), osmo-pcu (GPRS/EDGE), osmo-mgw, Asterisk (MNCC voice via osmo-sip-connector, SIP :5060), SMSC (SMS-over-GSUP, SMPP :2775, inter-operator relay :7890). A3A8 auth with deterministic RAND. Live GSMTAP :4729 for Wireshark.
- Start: red icon **"Lancer le banc GSM"** (`launch.sh`, which starts `osmo-banc.service` and reports it by notification); `sudo ./start.sh` in the foreground; `systemctl start osmo-banc`. The stack lives in tmux: `tmux attach -t calypso`.
- Stop: right-click **"Arreter le banc"**, `./start.sh --stop` (also tears down the DSP bench), `systemctl stop osmo-banc`.
- Sources: `start.sh` + `startup_scripts/` (delegates to `/opt/GSM/qosmo/run.sh`), `launch.sh`, `services/osmo-banc.service`, `configs/*.cfg`, `generate_configs.sh`.

2.36.1.6.2 5.2 Two layer-1 chains: DSP (default) and gr-gsm

	DSP `--dsp` (default)	**gr-gsm** `--grgsm`
Who demodulates	the real TMS320C54x on the TI mask ROM, **outside** QEMU (`/opt/GSM/c54x_exe`)	a gr-gsm decoder in QEMU + `grgsm_exe` bridge
QEMU	ARM7 only (`CALYPSO_DSP_EXTERN=1`)	ARM7 + gr-gsm layer 1
TRX bridge	`pont/pont_dsp.py` started by `c54x_exe/run.sh` (TRXD 5700-5702, DL bursts UDP 6702)	`pont/pont.py`
Result	the mobile acquires FB/SB, decodes SI1-4, camps, does its location update; mobile VTY :4347	end to end: camp, LU, auth, A5/1, SMS, voice call; VTY :4247

The default (`DSP_MODE=1` in `startup_scripts/`) applies to everything that starts the bench (icon, service, multi's native operator). If `c54x_exe` is missing, the bench says so and falls back to gr-gsm with its bridge. Icon right-click: **"Lancer en mode gr-gsm"**. Persistent: `OSMO_BANC_ARGS="--grgsm"` in `/etc/default/osmo-banc`.

2.36.1.6.3 5.3 The emulated Calypso baseband

- QEMU fork **qosmo** (`-M calypso`, ARM7TDMI/arm946) runs the real OsmocomBB firmware `layer1.highram.elf`, loaded by osmocon as on a C123. DSP ROMs and firmware for the compal boards are shipped.
- C launcher `qosmo` (`/usr/local/bin/qosmo`): `-gdb tcp::1234`, `-serial pty` published as `<RUN_DIR>/modem.pty`, unix monitor, L1CTL `/tmp/osmocom_l2`.
- Debug: gdb on :1234 (`target remote :1234` against the layer1 ELF), monitor `socat - unix-connect:/tmp/qemu-calypso-mo` `--assembly-logs` for the ARM asm trace.
- Manual: `qosmo -k .../layer1.highram.elf`, `MODE=dsp PONT=1 /opt/GSM/c54x_exe/run.sh` (see Lancer-a-la-main).

2.36.1.6.4 5.4 `start.sh` profiles and options

- Profiles: `faketrx-qemu` (default, alias `hybrid`: core + BTS0 QEMU + BTS1 fake_trx + MS#2), `faketrx`, `qemu` (Calypso only), `noproc`/`core` (core only).
- Calypso set-ups: `--bts-classique` (osmo-bts-trx + bridge + emulated mobile, the default), `--bts` (the Calypso is the BTS radio), `--bts --ms` (plus a mobile camped over the air), `--ms-magnetite` (Magnetite firmware on the C139 model, add-on).
- `--menu` asks the questions (layer 1, standalone/hybrid topology, profile, node, WAN, regeneration) instead of guessing.
- `--node N` (1-9): the node's SS7 identity (point codes `1.<node><op>.<role>`, routing contexts), ASP towards the inter-STP; `--op N`, `--hub-ip ADDR`.
- `--wan`, `--wan FILE`, `--wan-nodes "1:IP:IND ..."`, `--wan-id`, `--gen-conf`, `--air-mesh[=PORT]` (shared inter-node radio medium): mesh of 1 to 9 nodes. `--virtualbox[=N]` sets up the WAN between the host and N-1 VirtualBox VMs (`tools/vbox-wan-lab.sh`). `--payload-ism[=backend]`: SMS text over ISM radio (Payload-ISM).
- `--regen`, `--list`, `--dry-run`, `--status`, `--check-paths`, `--force`, `--verbose`, `--launcher <bin>`, `--docker` (hands over to `launch/start-docker.sh`), exclusive L2 clients `--mobile`, `--ccch_scan`, `--bcch_scan`, `--cell_log`.
- Variables: `ENCRYPTION="a5 1"`, `MS_COUNT=2`, `CALYPSO_BRIDGE`, `BANC_DSP`, and any `globals.conf` variable set non-empty on the command line (see Environnement).

2.36.1.6.5 5.5 SS7 multi-operator (Docker) and the add-ons

Not in the ISO: the **"Supplements"** icon (`addition.sh`, checkbox window inside a bench GTK window) installs what is missing.

- **`--multi`**: docker.io, the operator image pulled from `ghcr.io/bbaranoff/osmo-operator/osmocom-nitb:latest` then derived into `osmocom-run` (`Dockerfile.run`), the topology `/etc/osmocom/osmo-multi.conf` (op1 **native** PC 1.1.2 + op2/op3 **docker** 172.20.0.12/.13 PC 1.2.2/1.3.2 + hub `osmo-inter-stp` 172.20.0.10, M3UA :2908), `osmo-multi.service` and the antenna icon **"multi-operator"**. It ends by starting a multi bench session via `launch.sh --multi`, the same path as the icon (`OSMO_MULTI_START=0` to install without starting). `addition.sh --status` shows docker, the image, the topology, the unit and the icon.
- Then start: antenna icon, `systemctl start osmo-multi`, `sudo ./launch/start-multi.sh` (`--status`, `--stop`, `--dry-run`). The native operator is held by `osmo-banc`: `start-multi.sh` keeps it if already running and starts it if stopped (it only restarts it if its point code was just realigned, or with `--restart-natif`).
- **`--opencl`**: OpenCL stack (ocl-icd, clinfo, pocl, detected Intel/Mesa/NVIDIA driver) and the tools cloned into `/root`: **deka**, a51_tools, dst80_reversing, tea1-cracker. **deka** icon.
- **`--claude`**: Claude Code (native installer, standalone binary). The **Claude** icon self-installs on first click.
- **`--extras`**: Doom (freedoom), Quake III (OpenArena), Kodi, YouTube (Firefox + uBlock), Wireshark as root, Linphone.
- Phone firmwares (`tools/osmo-firmwares-install.sh`): `--gnuarm` (arm-elf-gcc 4.5.2 toolchain in `/root/gnuarm`), `--magnetite` (FreeCalypso Magnetite for the C139 under Wine + TI TMS470), `--trx` (osmocom-bb-trx image and `trx` firmware), `--calypso-fw` (recompiles the bench osmocom-bb firmware: GPRS layer1, rssi; `OSMO_FW_INSTALLER=1` installs them into `/opt/GSM/firmware/board/compal_e88`), `--fernvale` (MediaTek MT6260), `--simcom` (documentation and SDKs), `--firmwares` (all of them).

2.36.1.6.6 5.6 4G (srsRAN + Open5GS) and CSFB

- Full **Open5GS** core on loopbacks (MME, SGW-C/U, SMF, UPF, PCRF, HSS, NRF…), MongoDB behind the HSS, **freeDiameter** with TLS certificates installed, Prometheus :9090, **WebUI :9999** (`open5gs-webui.service`, Next.js) for subscribers.
- **srsENB** radio (10 MHz, EARFCN 3350, band 7) and **srsUE** linked over **ZeroMQ** (:2000/:2001), portable binaries (no build-CPU AVX), **srsGUI** for real-time traces (`osmo-lte start-gui`). Milenage subscriber IMSI 001010001000001, UE in a `ue1` netns, data via `ogstun` (10.45.0.0/16) with NAT to the Internet.
- **CSFB**: SGs interface between osmo-msc and the MME (:29118), mapping TAI (001-01, TAC 7) to LAI (001-01, LAC 1).
- Start: **"4G du banc"** icon (`osmo-lte toggle`, `osmo-lte.service`); `osmo-lte start|stop|status|up|down|log|journal|sta` core only `osmo-epc start|stop|status`. Log and status in bench GTK windows.
- Sources: `tools/osmo-lte.sh`, `tools/osmo-epc.sh`, `tools/osmo-lte-install.sh`, `configs/srsran/`, `configs/open5gs/`, `services/osmo-lte.service`, `iso_modules/88-lte-pmos.sh`.

2.36.1.6.7 5.7 The postmarketOS phone

- x86_64 postmarketOS/Phosh VM, kernel patched with PPP, patched pmbootstrap (modem serial port, sound cards, screen size).
- **4G data**: NetworkManager, pppd (`ATD*99***1#`), virtio-console `osmo.data`, `osmo-phonesim-banc.py`, UE netns, srsUE, srsENB, UPF, Internet. **2G voice and SMS**: virtio-console `osmo.modem`; the 27.007 AT modem is played by `osmo-phonesim-banc.py` from the bsc/msc/mobile VTYs; oFono drives it on the VM side.
- Start: **"Telephone"** icon (`osmo-pmos up`), **smartphone** (720x1440) and **tablet** (1280x800) formats (`osmo-pmos-qemu`), status, stop and reconnect-modem actions (`osmo-pmos-setup`), VM alone without modem.
- **The image**: `osmo-pmos-build` decompresses `/opt/user_interface/pmos/image/qemu-amd64.img.zst` on first launch. If it is missing or corrupt (`zstd -t`), it is fetched from the GitHub release `pmos-image` (resumable, archive verified); otherwise `pmbootstrap install` builds it. `OSMO_PMOS_IMAGE_URL` changes the URL.
- Sources: `tools/osmo-pmos*.sh`, `tools/osmo-phonesim-banc.py`, `tools/osmo-ofono-bridge.py`, `iso_modules/88-lte-pmos.sh`

2.36.1.6.8 5.8 The desktop

- Minimal **GNOME**, root session, dock with the favourites: Firefox, Files, Claude, Lancer le banc, Installer, Tutoriel, Info, Supplements, multi-operator, Update, Paint, deka, Linphone, Wireshark, Telephone, 4G. The **multi-operator** icon is installed only by the add-ons (`addition.sh --multi`): before that it does not exist and the dock does not show it.
- **No favourite opens a terminal or a shell.** Everything goes through `tools/osmo-gtk-run.py`. Launches (2G, multi, 4G, add-ons, Update, deka, pmOS phone, drivers) open in `--progress` mode: a small window with a **progress bar** (driven by the `[n/m]` markers the scripts print: `run.sh` [n/10], `osmo-lte` [n/4], `osmo-pmos` [n/3]; otherwise the bar pulses), the current step, the last line, and a **"console"** button that expands the full output (VTE terminal, the command runs in a pseudo-terminal). A failure expands the console automatically. Only Claude and oFono, being interactive, keep a terminal in the window. The systemd unit's journal scrolls in the window for 2G and multi (`launch.sh --service-suivi`, `--multi-suivi`) and 4G (`osmo-lte up|toggle`).
- **Mobile VTY port**: 4247, or **4347 with `--dsp`**. `start.sh` publishes the actual port in `/run/osmo-banc/ms1-vty` and in the systemd environment (`OSMO_BB1_VTY_PORT`, read by the `osmo-egprs-web` dashboard on restart); `launch.sh --vty`, the panel and `vty-menu.sh` read it.
- **Bottom bar** (`tools/osmo-launcher.py`): not shown by default. `OSMO_LAUNCHER=1` in the session environment enables it (Banc, Jeux, Media, Telephone, Outils groups, "run" logs).
- **Panel** (`tools/osmo-panel.py`, `osmo-fft-snap.py`): the illustration becomes the I/Q spectrum and `mobile.log` while the bench runs; top banner (`osmo-topzone.py`, `configs/conky/labgsm.html`) fed by `osmo-ts-probe.py`: 8 timeslots, daemons, subscribers, calls, uptime.
- Status **Conky** at the top right (machine, selected operator, `Ctrl+AltGr+Left/Right` or `osmo-op --next`): dark translucent plate with rounded corners (`configs/conky/osmo-conky.lua`, Cairo), colours by kind of information, dots and a smiley per bench state. It is **on screen only while a bench runs**: `osmo-panel.py` fades it in and out (X11 window opacity, xdotool) together with the panel.
- **Central panel** (`tools/osmo-panel.py`, spectrum + log, Dashboard / tmux / VTY / ZMQ buttons): **appears only while a bench runs**, a 2G (native, container, hub) or a 4G (Open5GS, srsENB, srsUE). With the bench stopped, the desktop shows only the background. `OSMO_PANEL_TOUJOURS=1` keeps it on screen.

- **Web dashboard** :8080 (`osmo-egprs-web.service`, self-signed HTTPS on :80, GSMTAP/SCTP capture), **tutorial** (`osmo-tutorial`), **Info page** (`osmo-info`, `configs/info.html`), I/Q spectra `fft-web` :8081.
- Applications: Firefox (only browser, microphone and certificate policy set at boot), **Wireshark as root** (`osmo-wireshark-root`, 2G/4G/call/SMS/data filters), **Linphone** extension 100, Kodi, dino, oFono, Paint (overlay), graphics drivers (`tools/osmo-drivers.sh`, NVIDIA via `ubuntu-drivers`).
- **Plymouth** `osmo-bts`: mast, handset and waves (preview: `wiki/plymouth-osmo-bts.gif`). Generated at build time by `tools/plymouth-render.py` (`86-finitions.sh`).
- **Two-state** wallpaper (`tools/osmo-fond.py`, `osmo-fond ville|antenne`): at rest `/usr/share/backgrounds/gsm-lab-ville.j` (the waterfront city, no antenna); as soon as a 2G or 4G starts, a fade (about 2.4 s, Pillow intermediate frames + GNOME Shell cross-fade) to `gsm-lab-antenne.jpg` (the same city, mast raised), and back on stop. Triggered by `osmo-panel.py` when it sees the bench come up or go down, by `osmo-lte start` and, if the bottom bar is active, on clicking *run standalone*. Likewise, the top zone carries only the **Telecom Operator** title while nothing runs, and fades to the LAB GSM banner on launch (`$XDG_RUNTIME_DIR/osmo-banc.etat`, read by `osmo-topzone.py`). The low-poly landscapes `gsm-lab-fond.jpg` and `gsm-lab-wallpaper.jpg` (`tools/paysage-render.py`) are installed but not the default.

2.36.1.6.9 5.9 Audio

- **System-wide PulseAudio** (`osmo-pulse.service`, socket `/run/pulse/native`, sink `gsm_audio`): osmo-gapk, ALSA `gsm_out`, null sink, monitor, loopback; PCM `gsm_out`/`gsm_in` in `/etc/asound.conf`, which `mobile` opens. Two duplex chains for the pmOS phone (GAPK bridge, echo cancellation).
- **PipeWire** kept **for GNOME screencast only**; `pipewire-pulse` purged and masked.
- Sources: `iso_modules/82-services.sh`, `scripts/pulse-gsm-setup.sh`, `scripts/audio-chain.sh` (slimmed-down PulseAudio on every start).

2.36.1.6.10 5.10 Diagnostics

`bash ./checks/check_all.sh` runs everything and gives a verdict (log in `/tmp`): `global_check.sh` (STP/HLR/MSC/BSC per operator), `ss7_check.sh` (inter-STP matrix), `diag-stp-operator.sh`, `diag-interstp.sh`, `wan_ss7_check.sh`, `annuaire.sh` (subscribers), `operator_summary.sh`, `vty-debug-dump.sh`. `./ss7-console.py`: navigable SS7 diagram with built-in VTYs. `./start.sh --status` tells what is running.

2.36.1.6.11 5.11 Ports, VTY, network

Port	Service	Port	Service
4239	VTY osmo-stp	8080	bench dashboard
4242	VTY osmo-bsc	8081	I/Q spectra (fft-web)
4243	VTY osmo-mgw	9090	Prometheus Open5GS
4245	VTY osmo-sgsn	9999	WebUI Open5GS (HSS)
4254	VTY osmo-msc	5060	Asterisk SIP
4258	VTY osmo-hlr	2775	SMPP
4260	VTY osmo-ggsn	7890	inter-op SMS relay
4247 / 4248	VTY mobile 1 / 2 (gr-gsm)	29118	SGs (MSC to MME)
4347	VTY mobile (DSP bench)	2905 / 2908	M3UA local / inter-STP
4729	GSMTAP (Wireshark)	5700-5702	TRXD (bridge to osmo-bts-trx)
4730 / 4731	decoded SI / SCH	6702	DL bursts to the DSP
1234	gdb QEMU	2000 / 2001	ZeroMQ srsENB to srsUE

Network: docker interconnect `gsm-inter` 172.20.0.0/24 and `gsm-net-opN` 172.20.N.0/24; node private IP `192.168.<N+1>.10`; LTE `ogstun` 10.45.0.0/16. Numbering plan: `100101`/`100102` (the two subscribers), `N0001`…`N9999` (operator N in multi), `100`/`200` (Linphone), `600` (echo), `9XXXXX` (inter-op outgoing).

2.36.1.6.12 5.12 Not included

Item	Status	Source
avahi (daemon, utils, autoipd, libnss-mdns)	purged and masked: no mDNS on a bench	`80-chroot.sh`
Chromium	Firefox is the only browser, starts as root with no wrapper	`86-finitions.sh`
sssd	masked, not uninstalled (no domain): `systemctl unmask sssd` restores it	`86-finitions.sh`
pipewire-pulse	system-wide PulseAudio instead; pipewire kept for screencast	`80-chroot.sh`
`osmo-dsp` icon	DSP is the default; right-click on the bench icon offers gr-gsm	`85-installeur-bureau.sh`
`osmo-trust-desktop` and the desktop icons	launchers are in the dock and the menu	`85-installeur-bureau.sh`

2.36.1.6.13 5.13 The Calypso bench, the C123 window and GLava

The Calypso chains (`/opt/GSM/c54x_exe/run.sh`, `MODE=dsp`): each chain is an *instance* of four processes: `c54x_exe` (the DSP), `qosmo` (the ARM), osmocon, and the host application. The network itself (core, BSC, BTS) stays with `./start.sh`; these commands only add phones to it.

Command	Instance	What runs	Logs
`banc-trx`	1	firmware `trx.highram.elf` + host `transceiver`: the Calypso **is the BTS#0 radio** (TRXD 5700-5702)	`/tmp/c54x-pont`
`banc-mobile`	2	firmware layer1 + `mobile`, camped on instance 1 **over the air** (`pont/air.py`)	`/tmp/c54x-pont2`
`banc-mobile-seul`	1	the classic set-up: one mobile + `pont/pont_dsp.py` towards osmo-bts-trx	`/tmp/c54x-pont`
`banc-rssi`	4	firmware `rssi.highram.elf`: the RSSI app alone, screen and keys emulated; kept apart (`banc-stop` and `start.sh --stop` leave it running)	`/tmp/c54x-pont4`
`banc-stop` / `banc-status` / `banc-logs`		`run.sh --stop` / `--status` / `--logs`	
`banc-mobile-stop`/`-status`/`-logs`, `banc-rssi-stop`	2	the same for one instance	

In classic-BTS mode (`pont_dsp.py` running) `banc-rssi` takes its downlink from the bridge's copy (UDP 6762); with `--bts`, from the air. Realistic air levels are on by default (`CALYPSO_AIR_NIVEAUX=1`): the cell is heard only on its ARFCN (`PLAN_ARFCN` in `/etc/osmocom/radio-plan.env`), noise elsewhere.

The same set-ups in one go, network included:

```
./start.sh --bts                  # the Calypso is the BTS radio, alone
./start.sh --bts --ms             # + a mobile camped on it over the air
./start.sh --bts-classique --ms   # osmo-bts-trx + bridge + emulated mobile (the --dsp default)
```

These names are defined once, in `scripts/osmo-banc-aliases.sh`, installed as `/etc/profile.d/98-osmo-banc-aliases.sh` for login shells. They are **also real commands** in `/usr/local/bin`, links to `tools/osmo-banc-cmd.sh`, which sources that file and replays its own name: they work from Alt+F2, `sudo`, a script or `bash -c` (`tools/osmo-banc-cmd.sh --install` lays the links; `70-scripts.sh` does it in the ISO).

The C123 window (`tools/osmo-c123-gtk.py`, GTK3, system `python3` with `python3-gi`, `python3-gi-cairo`, `gir1.2-gtk-3.0`): a photo of the Motorola C123 (`data/c123.jpg`) with the ST7558 LCD that `qosmo` publishes in `/dev/shm/calypso[N]/calypso_lcd` redrawn into it, and keys you can click or type: `0-9`, `*`/`s`, `#`/`h`, arrows, Enter = menu, `o` = ok, F1/F2 = soft keys. Power (red key, Esc, `p`) needs a **long press** and is never sent to a TRX instance: it would switch the BTS radio off.

Command	Window on
`c123`	the first live instance that is **not** a TRX (the mobile); in a login shell, the alias `c123` opens instance 1
`c123-1`, `c123-2`, `c123-3`	instances 2 (mobile), 3 (second TRX phone), 4 (rssi)
`c123-tous`	one window per live instance, plus those that appear later
`c123 --modele c139`	the colour C139 screen, for the Magnetite firmware (add-on)
`c123 --calibrate`	redefine the key zones (`data/c123-zones.json`) with the mouse

GLava (`tools/glava-build.sh`, `iso_modules/85-glava.sh`, desktop ISO only): GLava with the *NCS Spectrum* module, compiled at build time (no noble package). A spectrum circle sits above the city of the wallpaper and a bar spectrum runs along the bottom edge, both sized from the screen at login (`tools/osmo-glava.sh`). They listen to **the speakers and the microphone together**: a null sink `osmo_glava` fed by two loopbacks, the monitor of the hardware output (everything that plays, bench voice included) and the default source (`osmo_mic_ec`, the echo-cancelled mic). The mic goes only to that sink, never to the speakers: no feedback. A systemd user service: `systemctl --user stop|start|disable glava`. It needs OpenGL 4.2 or newer.

2.36.1.7 6. Installing to a disk

2.36.1.7.1 6.1 Starting the installer

The dock's **"Installer"** icon, or `osmo-install` (`/usr/local/bin`, written by `85-installeur-bureau.sh`). It re-elevates to root via `pkexec`, finds the squashfs whatever the boot mode (medium, `toram`, loop) and mounts it at a stable path (`/run/osmo-install-src/live/filesystem.squashfs`), unmounts a target left mounted by a previous attempt, regenerates the apt mirror page ranked by speed (`packaging/apt-mirror.sh --list`), then opens Calamares with the repository's configuration (`installer/calamares/`, copied to `/etc/calamares` at build time).

2.36.1.7.2 6.2 The screens (`installer/calamares/settings.conf`)

Welcome (20 GB, 2 GB RAM, root required), Location (Europe/Paris), Keyboard (pc105), Partitions, Users, Graphics drivers, Ubuntu mirror, Summary, Install, Finish. The slideshow during the copy reproduces the image's spec sheet (`branding/osmo/show.qml`).

2.36.1.7.3 6.3 Partitioning (`modules/partition.conf`)

- **GPT** table, ext4, 300 MiB ESP on `/boot/efi`, no swap by default (`swapfile` optional).
- Automatic layout: **"osmo-egprs" on `/`** (50 %, 25 GiB minimum, **never encrypted**) and **"crypthome" on `/home`** (the rest, 5 GiB minimum).
- **Ticking "encrypt" encrypts `/home` only**, with LUKS2: the root stays readable, secrets (profiles, keyrings, cookies) live on the encrypted volume. Manual partitioning allowed.

2.36.1.7.4 6.4 Accounts and drivers

- Users: the created account is a sudoer (sudo, adm, dialout, audio, video, pulse-access, docker…), with GDM autologin; the root password is asked for (`setRootPassword`), no complexity requirement; the name `root` is forbidden.
- Graphics drivers: nouveau (free) by default; option **NVIDIA 610 proprietary**, installed in the target only if `lspci` sees a 10de card and the network responds (`apt-get install nvidia-driver-610`, then `update-initramfs`).
- Mirror: the live stick's one or a measured mirror.

2.36.1.7.5 6.5 What the installer does in the target (`modules/shellprocess-osmo.conf`)

1. Unlocks root (`passwd -u root`).
2. Deletes the live stick's `osmocom` account (unless it is the chosen name).
3. `crypthome-postinstall <user>`: wires up the encrypted `/home` (see 6.7).
4. Removes the root autologins on tty1 and ttyS0.
5. GDM: `AutomaticLogin=<user>`; the PAM rule refusing root at the login screen is restored.
6. Copies the launchers from `/etc/skel` to the user's desktop.
7. Installs and enables `osmo-egprs-web.service`.
8. Removes the `osmo-install` and `calamares` icons from the menu.
9. Generates `~/.config/pmbootstrap_v3.cfg` for the user.
10. Purges `live-boot`, rebuilds the initramfs.

GRUB: `/EFI/ubuntu` (prefix compiled into the signed GRUB), `GRUB_TIMEOUT 5`, os-prober enabled, `GRUB_ENABLE_CRYPTODISK=y`, kernel line inherited from the stick (`quiet splash`). fstab: `noatime`, `discard` on SSD.

2.36.1.7.6 6.6 After installation: accounts and services

	Live stick	Installed disk
GDM session	root, automatic	the created user, automatic
root	password `osmo`	password chosen at install, unlocked
`osmocom`	sudoer, password `osmo`	deleted
`osmo-egprs-web`	active	active
`osmo-banc`, `osmo-multi`, `osmo-lte`	installed, not enabled	installed, not enabled

Enable a bench at every boot: `systemctl enable --now osmo-banc` (or `osmo-multi`, `osmo-lte`). At build time: `--banc`, `--multi`, `OSMO_ISO_LTE=1`. On disk for multi: `OSMO_MULTI_ENABLE=1 ./addition.sh`.

2.36.1.7.7 6.7 Living with the encrypted `/home` (`configs/crypthome/`)

- **At boot**, `unlock-home.service` asks a single question, on the text console, after leaving Plymouth: `decrypt /home -passphrase- or Enter`. The passphrase (echoed as *) opens the volume and the user's session. **Empty Enter, 60 s timeout, three rejected passphrases, missing disk or failed mount: the machine boots anyway**, into an **ephemeral root** session (GDM switches to root for this boot only; `/root` is covered by an overlay whose writes go to a tmpfs). Boot is never blocked.
- `CRYPTHOME_PROMPT=plymouth` in `/etc/default/crypthome` puts the question in the Plymouth theme instead (only for machines where Plymouth keyboard input works).
- Afterwards: `unlock-home` / `lock-home` in a terminal; apps holding secrets (Firefox, Thunderbird, KeePassXC…) run as the volume owner via `run-as-owner` when it is open, as root otherwise (`refresh-owner-apps` redirects the launchers).
- Second disk added later: `/root/init-crypthome.sh` (LUKS2, `rsync` copy, crypttab/fstab with `noauto`, service enabled, the old `/home` set aside in `/var/backups/home-avant-chiffrement-*` at the next reboot).

2.36.1.7.8 6.8 Updating an installed machine

- `osmo-update` (at boot, or by hand): fast-forward `git fetch` of `/opt/GSM/osmo-operator` and the embedded repositories, then `tools/osmo-patches-apply.sh` re-applies `patches/*.patch` (idempotent). No packages.
- What the build installs **outside** the repository (units in `/etc/systemd/system`, `unlock-home` in `/usr/local/sbin`, `osmo-*-anim` launchers) is reinstalled by hand from `services/` and `configs/crypthome/`, or by reinstalling.
- `addition.sh` and `update.sh` rewrite the `.desktop` files they own.

2.36.1.8 7. Building the ISO

2.36.1.8.1 7.1 Where the pieces come from

```
Docker image osmocom-nitb (compiled by build.sh, or PULLED from ghcr.io)
        |  .deb osmo-build-*  +  /usr/local, /opt/GSM, venv, configs
        v
debootstrap minbase noble --> rootfs --> chroot (apt packages, desktop, 4G, pmOS...)
        |                                  |
        |   GitHub clone of osmo-operator --+  (51-depot.sh: main branch)
        v
squashfs zstd -19 --> hybrid xorriso ISO + signed shim/grub (or arm64 SD image)
```

- **The Docker image is not rebuilt by default** (amd64). Pull order: `ghcr.io/<repo>/osmocom-nitb:base-<fingerprint>` (the fingerprint is the sha256 of the tracked files in Dockerfile, packaging, configs, scripts, services, helpers, opt-gsm, tools, patches: when it matches, the published image was built from exactly this tree), then the repo's `:latest`, then the Docker Hub image. `build.sh` runs only if none responds, or with `--build-docker`.
- **The repository burned into the ISO is a clone of the `main` branch on GitHub**, not the local tree: push before burning. To burn the local tree, `OSMO_EGPRS_REPO=/path/to/repo OSMO_EGPRS_BRANCH=<branch>`.
- **The `iso_modules/` modules and `build-iso.sh` are read from the local tree** at build time.

2.36.1.8.2 7.2 Host requirements (`iso_modules/20-hote.sh`)

Ubuntu 24.04 as root, docker (installed if missing), and the packages the module installs: debootstrap, git, rsync, zstd, xz, cpio, gcc, make, python3-pil, python3-yaml, kpartx, dosfstools, mtools; on amd64 squashfs-tools, xorriso, grub-pc-bin, grub-efi-amd64-bin and -signed, shim-signed, isolinux, syslinux-utils; on arm64 qemu-user-static, binfmt-support, docker-buildx. `OSMO_ISO_HOST_READY=1` skips this step. No disk space check is done: leave plenty of room in `/var/tmp` (the work directory is `/var/tmp/iso-build-<pid>`, never `/tmp`, often a tmpfs: rootfs, squashfs and ISO must fit there together) and for the docker store (base image about 11 GB).

2.36.1.8.3 7.3 The command

```
sudo ./build-iso.sh --desktop                # the published image
sudo ./build-iso.sh                          # the four amd64 images
sudo ./build-iso.sh --role=operator --node=2  # the operator for node 2
sudo ./build-iso.sh --arm --lite             # Raspberry Pi 4 SD image
```

Option	Variable	Effect
`--role=operator|interstp`	`ISO_ROLE`	bench or SS7 hub
`--node=N`	`ISO_NODE`, `ISO_WAN_ID`	node identity 1-9, name `osmo-operator-N`
`--lite`, `--desktop`, `--all`	`ISO_LITE`, `ISO_DESKTOP`, `ISO_ALL`	see section 2
`--arm` / `--arch=arm64`	`ISO_ARCH`	ARM SD images through `armbian/build-armbian.sh` (section 2); `OSMO_ISO_ARM_DRY=1` prints the `compile.sh` commands only

Option	Variable	Effect
`--board=LIST`	`ISO_ARM_BOARD`, `OSMO_ISO_BOARD`	`--arm` boards: `rpi4`, `opi5`, `all` (default) or Armbian names
`--banc`, `--multi` (`--no-banc`, `--no-multi`)	`OSMO_ISO_BANC`, `OSMO_ISO_MULTI`	enable the unit at the image's boot
`--softsim` / `--no-softsim`		embed the softsim tools
`--version=24.04|22.04`	`ISO_UBUNTU`, `ISO_SUITE`	noble (default) or jammy
`--kb=fr`	`OSMO_ISO_KB`	keyboard layout (asked in the terminal otherwise)
`--output=FILE`	`OUTPUT`	output name
`--no-cache`	`NO_CACHE`	full recompilation, `.deb`s rebuilt
`--skip-build[=image:tag]`, `--image=`	`ISO_SKIP_BUILD`, `ISO_PULL_IMAGE`	pull required, no fallback to build.sh
`--build-docker` / `--build-image`	`ISO_FORCE_BUILD`	build the image locally
`--with-debs` / `--without-debs`	`ISO_WITH_DEBS`	bench `.deb`s in the image or not
`--wan`, `--wan-nodes=`, `--wan-id=`, `--wan-ops=`, `--hub-ip=`	`ISO_WAN*`, `ISO_HUB_IP`	WAN table burned into `/etc/osmo-wan.conf`
`--install-deps`, `--dry-run`, `--help`		host packages only, dry run, usage

Other variables: `OSMO_ISO_WORK` (work directory), `OSMO_DEB_CACHE` (`/var/cache/osmo-debs`), `OSMO_ISO_LTE=1`, `OSMO_ISO_PMOS_IMAGE` (`/path/to/qemu-amd64.img.zst` or `none`), `OSMO_PMOS_IMAGE_URL`, `OSMO_ISO_LTE_BUILD=0` (no 4G compilation if the `.deb`s are missing), `OSMO_EGPRS_REPO`/`OSMO_EGPRS_BRANCH`, `OSMO_WEB_REPO` (dashboard).

2.36.1.8.4 7.4 The module chain (`iso_modules/`)

All modules are sourced in the same shell, in numeric order; a module with nothing to do exits via `return` (that is how `--arm` skips a step).

Module	What it does
`00-options.sh`	options, help, suite, mirror, root check
`10-clavier.sh`	keyboard layout (menu if terminal, otherwise `fr`)
`20-hote.sh`	host packages, docker, tools, arm64 binfmt
`20-stockage-docker.sh`	docker storage location
`21-docker-build.sh`	pull or build of the Docker image, child passes of `--all`, output names
`22-wan-table.sh`	WAN table, `trap cleanup`, tool checks
`30-image-configs.sh`	Osmocom configs generated from templates (`start-docker.sh`), native patches
`31-image-source.sh`	source image, suite check, injection of `/etc/osmocom` and `/etc/asterisk`
`40-rootfs.sh`	`debootstrap --variant=minbase` (cache `~/.cache/osmo-iso-debs`), or inherited rootfs
`50-injection-image.sh`	build `.deb`s via `dpkg -i`, copy of `/usr/local`, `/opt/GSM`, venv, node, units
`51-depot.sh`	clone of osmo-operator and the tests, `coeur.env`
`51-softsim.sh`	softsim tools
`52-qemu.sh`	qosmo, c54x_exe, grgsm_exe, DSP ROMs, firmware, qemu-system-arm, `qosmo` launcher
`60-dashboard.sh`	osmo-egprs-web (native, no docker), tutorial, unit enabled
`70-scripts.sh`	`/usr/local/bin` links (`c123`, the `banc-*` commands), `98-osmo-banc-aliases.sh`, `osmo-wan.conf`, NetworkManager, `patches/*.patch` applied on the rootfs by `tools/osmo-patches-apply.sh`
`71-debs-banc.sh`	`packaging/build-debs.sh` to rootfs `/var/cache/osmo-debs`

Module	What it does
`80-chroot.sh`	apt repos (NodeSource, MongoDB, Armbian), kernel, packages, venv, desktop, Firefox, GDM, gsettings, `update-initramfs`
`81-cloture-systeme.sh`	`ldd` closure from the image, no-wait network, DNS, `osmo-ip-plan`, `osmo-qemu-link`, `osmo-update`, SMS animation
`82-arm-natif.sh`	legacy `--arm` path (native compilation in an arm64 chroot); not reached: `00-options.sh` hands `--arm` over to `armbian/build-armbian.sh`
`82-services.sh`	role marker, os-release, Asterisk, audio (system-wide PulseAudio, `asound.conf`), units installed not enabled, Open5GS WebUI
`83-espace-ram.sh`	fstab, `/tmp` and `/dev/shm` as % of RAM, journal, logrotate, ring-buffer tcpdump, ssh, purge
`84-comptes.sh`	root and osmocom, console and GDM autologin, Linphone account
`85-installeur-bureau.sh`	Calamares (`/etc/calamares`), `osmo-install`, icons, GTK launchers (`osmo-*-anim`), Info page
`85-extensions-gnome.sh`	GNOME Shell extensions
`85-glava.sh`	`--desktop`: GLava + NCS Spectrum compiled into the rootfs, `glava.service` (user)
`86-finitions.sh`	fixed live-boot `toram`, sssd masked, Chromium removed, Firefox microphone, screenfetch banner, Plymouth `osmo-bts`
`87-lite.sh`	`--lite` pruning of `/opt/GSM`
`88-lte-pmos.sh`	srsRAN + Open5GS (from the `.deb`s, otherwise compiled), freeDiameter, pmbootstrap, PPP kernel, VM image, final 2G + 4G + phone check
`89-home-chiffre.sh`	crypthome tools, `unlock-home.service`, root keyring on the encrypted volume, launchers under `run-as-owner`
`90-iso.sh`	dpkg check, squashfs zstd -19, kernel, GRUB menu, `/.disk/info`, squashfs read-back
`91-secure-boot.sh`	xorriso ISO with signed shim and GRUB (or unsigned `grub-mkrescue`)
`92-rpi-image.sh`	legacy `--arm` SD image assembly (Pi 4 only); not reached
`99-fin.sh`	rootfs handed to the next pass (`--all`), summary

2.36.1.8.5 7.5 CI (`.github/workflows/build-iso.yml`)

- Triggers: push of a `v*` or `iso-*` tag, or manual dispatch with the inputs `release`, `no_cache`, `build_docker`, `mode` (all, stp, base, lite, desktop) and `pmos_image` (embeds the postmarketOS VM, +1.7 GB per ISO).
- A single job on `ubuntu-24.04` (6 h max): runner cleanup, docker moved to `/mnt`, `.deb` cache keyed by fingerprint (`osmo-debs-noble-g2-<fingerprint>`). If `ghcr.io/.../osmocom-nitb:base-<fingerprint>` exists, it is pulled (`--skip-build=REF`); otherwise `--build-docker`.
- ARM: `.github/workflows/build-arm.yml`: the arm64 Docker image from `docker-arm.yml`, then **one job per board** (matrix, native `ubuntu-24.04-arm` runners) running `build-iso.sh --arm --board=<board> --skip-build=<ghcr image> [--desktop]`, and a release job gathering every board. Inputs `boards` (default `all`) and `desktop` (default on).
- Outputs: `SHA256SUMS`, `MD5SUMS`, files over 2 GiB split into `*.iso.part-NN` (1,900 MB) with `PARTS.sha256`, 14-day artifact, GitHub release (`softprops/action-gh-release`) with the notes from `.github/release-notes.md`, pre-release except on a `v*` tag.

2.36.1.8.6 7.6 Expected durations and sizes

Desktop build, Docker image pulled	about 2 h 15 on 16 cores

Full Docker compilation (`--build-docker`)	about 1 h 30 in CI; 15-20 min on a fast machine (see Build)
Desktop ISO	about 6.5 GB (4 parts in the release)
Desktop vs operator	+2.5 GB
Base Docker image	about 11 GB
`--arm` first time on x86	about ten hours (arm64 Docker image under emulation), then one Armbian build per board

---

2.36.1.9 8. Files to know

File	Role
`build-iso.sh`, `iso_modules/`	the build chain
`installer/calamares/`	the installer (modules, branding, slideshow)
`configs/crypthome/`	encrypted `/home`: `unlock-home`, `crypthome-postinstall`, `init-crypthome.sh`, `run-as-owner`
`configs/plymouth/osmo-bts/`, `tools/plymouth-render.py`	the boot theme, generated
`services/*.service`	`osmo-banc`, `osmo-multi`, `osmo-lte`, `osmo-egprs-web`, `open5gs-webui`
`data/desktop/*.desktop`	the dock icons; `tools/osmo-gtk-run.py` the windows
`patches/*.patch`, `tools/osmo-patches-apply.sh`	bench patches applied to the trees under `/opt/GSM` (install, ISO rootfs, `update.sh`, `addition.sh --calypso-fw`)
`/etc/default/osmo-banc`, `/etc/default/osmo-multi`, `/etc/default/osmo-lte`	persistent service options
`/etc/default/crypthome`	UUID, owner, timeout, `CRYPTHOME_PROMPT`
`/etc/osmocom/coeur.env`, `/etc/osmo-wan.conf`, `/etc/osmo-role`	node, WAN, image role
`/var/log/osmocom/`, `journalctl -u osmo-banc`	bench logs
`addition.sh`	the add-ons (docker/multi, OpenCL/deka, Claude, extras, phone firmwares)
`checks/check_all.sh`, `ss7-console.py`	diagnostics

2.37 /opt/GSM/osmo-operator/wiki/Installer-c54x_exe.md

9795 octets, 196 lignes

2.37.1 Installing c54x_exe (the Calypso DSP on the TI mask ROM) and the DSP bridge

Back: Telephone-emule. Prerequisite: qosmo.

`c54x_exe` (`https://github.com/bbaranoff/c54x_exe`) runs the **original mask ROM of the TMS320C54x** found in Calypso basebands as a Linux process. In the DSP setup, the osmocom-bb firmware under QEMU talks to it through the shared API RAM, frame by frame. The repository does not contain the C54x core itself: it **compiles qosmo's sources** (`qosmo/hw/arm/calypso/l1-dsp/`), so qosmo must be cloned first. The repository ships its own installer, `c54x_exe/install.sh`, which is also what the Docker images, the ISO build and `install.sh` call. The manual commands below are what the installer does, step by step.

2.37.1.1 With the installer

```
git clone https://github.com/bbaranoff/c54x_exe /opt/GSM/c54x_exe
cd /opt/GSM/c54x_exe
./install.sh --check      # qosmo cloned? libosmocoding seen by pkg-config? ROM? -- changes nothing
sudo ./install.sh         # make, ROM into /opt/GSM and rom/, then c54x_exe --trames 200
```

Step	Does	Already done if
`build`	`make QOSMO=...` (with `--portable`: `CFLAGS` without `-march=native`); `ldd` check	never skipped: the Makefile recompiles on every call
`rom`	copies an already verified ROM found on the machine, otherwise `rom/fetch-rom.sh --dest ...` (download, conversion, checksum)	all ROM files + `Registers` present and matching `rom/SHA256SUMS.3606` in every destination
`verify`	`c54x_exe --rom-dir ... --trames 200`: "N sections chargees" then "bilan sur 200 trames"	never skipped

Options (and equivalent variable):

Option	Variable	Default	Role
`--qosmo DIR`	`QOSMO`	`$GSM_ROOT/qosmo`	the sources to compile
`--rom-dir DIR` (repeatable)	`ROM_DIR`	`$GSM_ROOT` **and** the repo's `rom/`	where the ROM is written
`--portable`	`PORTABLE=1`	off	`CFLAGS` without `-march=native`, for a binary that travels
`--dest DIR`			copies the repo without `.git` or binaries into `DIR` and builds there: the binary in service is not overwritten
`--only STEPS`, `--skip STEPS`, `--reinstall`, `--with-deps`, `-v`			step selection, replay, apt packages, verbose

Logs: `/tmp/c54x_exe-install/<step>.log`.

Without root: `ROM_DIR=$HOME/calypso-rom ./install.sh`, then run `c54x_exe --rom-dir $HOME/calypso-rom` (the binary's default ROM directory is `/opt/GSM`).

The `--trames 200` check runs without `--arm`: private API RAM, no `/dev/shm`, no socket, no `/tmp/c54x-pont` (it runs in a private mount namespace), so it does not disturb a running bench.

Typical failure:

```
[FAIL] Compilation de c54x_exe (sources de qosmo) (manque : libosmocoding + libosmocore vus de pkg-config)
       -> export PKG_CONFIG_PATH=/usr/local/lib/pkgconfig
```

("manque" = missing: libosmocore is installed under `/usr/local`.)

2.37.1.2 What the installer does, by hand

2.37.1.2.1 Prerequisites

- qosmo cloned (default `/opt/GSM/qosmo`), see Installer-qosmo.
- gcc, make, pkg-config, **libosmocore + libosmocoding** visible to pkg-config. Check before compiling (the Makefile hides the pkg-config error):

  ```
  pkg-config --cflags --libs libosmocoding libosmocore
  ```

  If empty: `export PKG_CONFIG_PATH=/usr/local/lib/pkgconfig`.

- python3 + curl or wget for the ROM download.
- For the bridge: python3, `python3-numpy`, `libosmocoding.so` and `libosmogsm.so` loaded through ctypes (`pont/gsm.py`), so `ldconfig` must be up to date when libosmocore lives in `/usr/local`.

2.37.1.2.2 Build (`build` step)

```
git clone https://github.com/bbaranoff/c54x_exe /opt/GSM/c54x_exe
cd /opt/GSM/c54x_exe
make                                # or: make QOSMO=/path/to/qosmo
```

For a binary that must run on a different CPU than the build machine, drop `-march=native`:

```
make QOSMO=/opt/GSM/qosmo \
  CFLAGS="-O3 -g -Wall -Werror=format -Werror=format-extra-args -Wno-unused-function -Wno-unused-variable
↪  -Wno-unused-but-set-variable -Wno-sign-compare"
```

`make` recompiles on every call (a single cc command, no `.o`): this is intentional, the sources come from another repository.

2.37.1.2.3 The DSP ROM (`rom` step)

The ROM is not in the repository. `rom/fetch-rom.sh` downloads the FreeCalypso 3606 dump, converts it (`tools/dsp_txt2bin.py`) and checks `rom/SHA256SUMS.3606`. The binary reads it from `--rom-dir`, default `/opt/GSM`.

```
bash rom/fetch-rom.sh --dest /opt/GSM --dest /opt/GSM/c54x_exe/rom
```

Expected output:

```
telechargement : ftp://ftp.freecalypso.org/pub/GSM/Calypso/dsp-rom-3606-dump.txt
somme de controle verifiee (SHA256SUMS.3606)
ROM DSP 3606 ecrite dans /opt/GSM
ROM DSP 3606 ecrite dans /opt/GSM/c54x_exe/rom
```

(download / checksum verified / ROM written to.) Run again, it answers `ROM DSP 3606 deja en place et verifiee` (already in place and verified). Options: `--version 3311` (D-Sample), `--force`, mirror via `FREECALYPSO_URL=`. The installer first copies an already verified ROM found on the machine, so no network is needed when one is present.

2.37.1.2.4 Checking the DSP alone (`verify` step)

```
./c54x_exe --trames 200    # the ROM alone, no ARM: does it boot?
./c54x_exe --help
```

Optional ISA test: `make isa_test && ./isa_test tools/isa_tests.txt 2>/dev/null`.

2.37.1.3 The DSP bridge

The bridge connecting osmo-bts-trx to the DSP lives in **osmo-operator**, not in c54x_exe. Nothing to compile:

```
git clone https://github.com/bbaranoff/osmo-operator /opt/GSM/osmo-operator
cd /opt/GSM/osmo-operator && python3 pont/pont_dsp.py --help
```

`pont_dsp.py` requires `--dsp-port 6702` (see `pont/README.md`). It reads the timeslot table from `PONT_BSC_CFG` (default `/etc/osmocom/osmo-bsc.cfg`): point it at your BSC config.

2.37.1.4 Running

Against an already started stack (osmo-bts-trx included):

```
cd /opt/GSM/c54x_exe
PONT=1 CALYPSO_BSP_STREAM=1 CALYPSO_RHEA_DMA_XFER=1 \
  PONT_BSC_CFG=/etc/osmocom/osmo-bsc.cfg ./run.sh
./run.sh --status | --logs | --stop
```

- `PONT=1`: without it, no bridge, no burst arrives.
- `INSNS` (DSP instructions per frame): default 120000 in `run.sh`, which `start.sh` also sets in DSP mode. `CALYPSO_BSP_STREAM=1` (one TS0 burst per frame, in FN order) and `CALYPSO_RHEA_DMA_XFER=1` (without it the API page is never filled: no SB, no BCCH, no SI) are set by `start.sh` before it hands over to `run.sh`.
- `LOCKSTEP=1` (default): QEMU waits for the DSP on every frame (`CALYPSO_PONT_LOCKSTEP=1`).
- `GDB=1` (default): QEMU gdbstub on 1234 plus a telnet console on 44444 driven by `$QOSMO/tools/gdb-telnet.py` (`GDB_TELNET_PY` overrides the path; `GDB=0` disables both).
- Other variables: `MODE`, `VERB`, `IQ`, `AMP`, `QOSMO`, `FIRMWARE_ELF`, `FIRMWARE_BIN`, `OSMOCON`, `MOBILE`, `MOBILE_CFG`, `PONT_PY`, `RUNDIR`, `L2_SOCK`, `C54X_BIN`, `INSTANCE` (see the header of `run.sh` and `README.md`).
- Logs: `/tmp/c54x-pont/{dsp,qemu,osmocon,mobile,pont}.log`, sessions archived in `/tmp/c54x-pont/archives/<date>/`.

Process-by-process details: Lancer-a-la-main.

All-in-one variant for a **systemd** stack: `./run_real.sh [--secondes N]` starts `osmo-hlr osmo-stp osmo-msc osmo-mgw osmo-bsc`, restarts `osmo-bts-trx`, runs `run.sh`, watches for N seconds (120 by default) and prints a verdict. `./run_real.sh --stop` stops everything.

2.37.1.5 What proves it works

Line	File	Source
`pont : en attente de l'ARM sur /tmp/calypso_dsp.sock (API RAM : /dev/shm...)`	dsp.log	`src/pont.c`
`pont : RESET #1 (DL_STATUS=...)`	dsp.log	`src/pont.c`, `LAUNCH.md`
`fn=... pc=0xff80` then `pont : DSP boote (premier IDLE)`		
`[trx] pont DSP : API RAM partagee (...) + socket /tmp/calypso_dsp.sock`	qemu.log	`qosmo/hw/arm/calypso/calypso_trx.c`
`[montant] RACH ra=0x.. bsic=..;`	dsp.log / pont.log	`LAUNCH.md`
`pont.log` goes from `rach=0` to `rach=N`		
`[montant] TCH : le firmware poste la tache 13 a fn=..., BSP bascule sur TS2`	dsp.log (during a call)	`LAUNCH.md`

2.37.1.6 Troubleshooting

Symptom	Cause / fix
link fails on `gsm0503_*`	pkg-config cannot find libosmocoding (error hidden by the Makefile): set `PKG_CONFIG_PATH`
`Illegal instruction` at startup	`-march=native` binary from another machine: rebuild here (`./install.sh`), or `--portable` for a binary that travels
`c54x_exe n'a pas ouvert /tmp/calypso_dsp.sock` (did not open the socket)	ROM missing from `--rom-dir`: `./install.sh --only rom` (or `rom/fetch-rom.sh --dest /opt/GSM`); see dsp.log
rebuild without touching the binary of a running bench	`./install.sh --dest /tmp/c54x-neuf`, then compare / replace once stopped
`somme de controle differente` (checksum mismatch)	modified dump or wrong `--version`

Symptom	Cause / fix
`QEMU n'a pas rejoint le DSP` (QEMU did not join the DSP)	QEMU started without `CALYPSO_DSP_EXTERN=1`, or DSP not ready yet
`pont.py ne s'est pas annonce` (pont.py did not announce itself)	inherited `PONT_PY` pointing at `pont.py` (gr-gsm bridge): `env -u PONT_PY`
`FBSB RESP: result=255` in a loop	no bridge or no BTS: normal without a network
`DL bursts=0`	no BTS is transmitting to 5700-5702: check the `phy` section of osmo-bts-trx
`le port VTY 4347 ... deja pris` (already taken)	another mobile is running; change `bind 127.0.0.1` in the cfg
dirty state after a crash	`./run.sh --stop` purges sockets and `/dev/shm/calypso_*`

2.37.1.7 Known issues

- The SB is decoded only one time in three to five; `run.sh` sets `L23_SYNC_RETRIES_SELECTION=8` in DSP mode to compensate.
- `mobile_pont.cfg` sends GSMTAP to `172.20.0.1` (the docker gateway of the containerised bench): replace it with `127.0.0.1` when running outside a container.

2.38 /opt/GSM/osmo-operator/wiki/Installer-grgsm_exe.md

5249 octets, 106 lignes

2.38.1 Installing grgsm_exe, and the gr-gsm setup

Back: Telephone-emule. Prerequisite: qosmo.

The `grgsm_exe` repository ships its own installer, `grgsm_exe/install.sh`, also called by the Docker images, the ISO build and `install.sh`. This page covers that repository (a bench tool) and the gr-gsm layer-1 setup of the emulated phone, which does not need it.

2.38.1.1 Three things carry this name

Name	What it is	Makes the mobile camp?	Where
the **`grgsm_exe`** repository (`https://github.com/bbaranoff/grgsm_exe`)	qosmo's gr-gsm layer 1 (`fl1/grgsm_l1x5.c`) compiled **outside QEMU**, no ARM and no firmware: a bench and replay tool	no	`/opt/GSM/grgsm_exe`
the `/usr/local/bin/grgsm_exe` command	wrapper around `python3 pont/pont_dsp.py --no-record --dsp-port 6702`; `PONT_DSP_PORT=0 grgsm_exe` runs `pont/pont.py` instead	it is the **bridge**	outside the repositories, absent from a fresh install
the "grgsm_exe bridge" / `--grgsm` setup	gr-gsm L1 **inside** QEMU (qosmo built with `--enable-l1-grgsm`) + `pont/pont.py`	yes (the end-to-end chain: camp, LU, auth, A5/1, SMS, voice)	`osmo-operator/environment/paths.env`, `start.sh --grgsm`

To connect a phone to the network with the gr-gsm setup you need the third one: qosmo + firmware + osmocom-bb (see Installer-qosmo) and the `pont/pont.py` bridge from osmo-operator. The `grgsm_exe` repository is optional.

2.38.1.2 1. The grgsm_exe repository (bench tool)

Prerequisites: qosmo cloned (source), gcc, make. No Osmocom library (links only `-lpthread -lm -lrt`).

With the installer:

```
git clone https://github.com/bbaranoff/grgsm_exE /opt/GSM/grgsm_exe
cd /opt/GSM/grgsm_exe
./install.sh --check         # qosmo source, cc, make -- changes nothing
./install.sh                 # make -B, then grgsm_exe --trames 50 run safely
```

Step	Does
`build`	`make -B QOSMO=.... -B` is required because the binary is tracked by git: on a fresh clone it is newer than qosmo's sources and `make` alone would consider it up to date
`verify`	`grgsm_exe --trames 50`, expects "bilan sur 50 trames" (report over 50 frames)

The check does not disturb a running bench: `grgsm_exe` takes UDP 4730/4731 with `SO_REUSEADDR` and opens `/dev/shm/calypso_*` with `O_CREAT` + `ftruncate`, so as root the installer runs it in private mount and network namespaces (`unshare`, fresh `/dev/shm`); otherwise, if a bench is visible, the run is skipped and reported. Options: `--qosmo DIR` (`QOSMO`), `--dest DIR` (build a copy), `--only`, `--skip`, `-v`; logs in `/tmp/grgsm_exe-install/`.

By hand:

```
make -B                      # or: QOSMO=/path/to/qosmo make -B
./grgsm_exe --trames 5000
./grgsm_exe --verbeux
```

Check that it opens its inputs:

```
$ ss -ulnp | grep 473
127.0.0.1:4730   users:(("grgsm_exe",...))    # GSMTAP
127.0.0.1:4731   users:(("grgsm_exe",...))    # SCH
```

With nothing publishing on 4731 or in `/dev/shm`, `si_valid` stays false and the report says so. **Do not run it at the same time as QEMU in the gr-gsm setup**: both take UDP 4730/4731.

2.38.1.3 2. The gr-gsm setup (full phone)

Nothing more to compile than for qosmo: the `qemu-system-arm` binary built with `--enable-l1-grgsm` **is** layer 1. The bridge is `pont/pont.py` (python3, numpy, libosmocoding through ctypes; no GNU Radio).

Launch through the c54x_exe repository's script, which handles both setups:

```
cd /opt/GSM/c54x_exe
MODE=grgsm PONT=1 PONT_BSC_CFG=/etc/osmocom/osmo-bsc.cfg ./run.sh
```

In the grgsm setup, `run.sh` skips step 1 (no c54x_exe), launches QEMU **without** `CALYPSO_DSP_EXTERN`, waits for `backend gr-gsm` in qemu.log, then starts osmocon, mobile and `pont/pont.py`. Process by process: Lancer-a-la-main, gr-gsm variant.

Expected:

Line	File
`[l1] backend gr-gsm : GSMTAP udp/4730, SCH udp/4731`	qemu.log

Line	File
`pont TRX : ports 5700/5701/5702, ARFCN 514, BSIC 7` then `STATS fn=... | DL bursts=N`	pont.log

2.38.1.4 Troubleshooting

Symptom	Cause / fix
`la couche 1 gr-gsm ne s'est pas annoncee` (the gr-gsm layer 1 did not announce itself)	QEMU compiled without `--enable-l1-grgsm`, or launched with `CALYPSO_DSP_EXTERN=1`
`pont.py est le pont du montage grgsm : pour le DSP, lancer pont/pont_dsp.py`	`--dsp-port` passed to `pont.py`; it is the gr-gsm bridge, `pont_dsp.py` is the DSP one
`grgsm_exe` command not found	the wrapper is versioned nowhere; launch `python3 pont/pont.py` directly
79 % CRC errors, LU failing	a bridge copied outside the repository (an old copy); always launch the one from the osmo-operator tree

2.38.1.5 Known issues

- `run.sh` uses `c54x_exe/mobile_pont.cfg` in both setups, and that file imposes `io-tch-format ti` (DSP frames). On gr-gsm the bridge encodes speech in **rtp** format; with `ti` the frames go through but the sound comes out robotic. For the gr-gsm setup, pass a config with `io-tch-format rtp` via `MOBILE_CFG=`. The bench config `osmo-operator/configs/mobile.cfg` is a template (`__..._` placeholders filled by `generate_configs.sh`): it must be filled in by hand (IMSI, Ki, `stick`, VTY) before use.

2.39 /opt/GSM/osmo-operator/wiki/Installer-qosmo.md

10178 octets, 201 lignes

2.39.1 Installing qosmo (QEMU Calypso), and the osmocom-bb firmware

Back: Telephone-emule.

qosmo (`https://github.com/bbaranoff/qosm0`) is a QEMU fork that adds the `calypso` machine: the TI Calypso SoC (ARM946) on which the **original, unpatched osmocom-bb firmware** boots. The same binary serves both layer-1 setups: built with `--enable-l1-grgsm` it embeds the gr-gsm layer 1; started with `CALYPSO_DSP_EXTERN=1` it cuts that layer out and delegates to the external DSP `c54x_exe`. It also contains the **sources** that `c54x_exe` and `grgsm_exe` compile (`hw/arm/calypso/l1-dsp/`, `l1-grgsm/`, `contrib/hors-qemu/`), so install it first. The repository ships its own installer, `qosmo/install.sh`, also used by the Docker images and `install.sh`; the manual commands below are what it does, step by step.

2.39.1.1 With the installer

```
git clone https://github.com/bbaranoff/qosm0 /opt/GSM/qosmo
cd /opt/GSM/qosmo
./install.sh --check          # prerequisites and state of each step -- changes nothing
sudo ./install.sh --deps      # missing apt packages (the list: ./install.sh --print-deps)
sudo ./install.sh             # venv, configure, make, make install, /usr/local/bin, check
```

Steps (`./install.sh --list`), each with an "already done?" test, prerequisites, the work and an after-the-fact check (the contract of `osmo-operator/install_modules/_lib/inst.sh`):

Step	Does	Already done if
`venv`	`python3 -m venv ~/.venv-qemu` + `pip install tomli`	`tomli` importable in the venv

Step	Does	Already done if
`configure`	`mkdir -p build && cd build && ../configure --target-list=arm-softmmu --enable-l1-grgsm --prefix=... --disable-werror --disable-docs`	`build/config.log` has the same options
`build`	`make -j$(nproc)` in `build/` (incremental)	always rerun, only rebuilds what changed
`install`	`make install` (prefix `/opt/GSM/qemu-install`)	installed binary identical to `build/`
`bin`	copies `qemu-system-arm` and `qosmo` into `/usr/local/bin`	identical copies; *not applicable* if the directory is not writable
`verify`	`build/qemu-system-arm -M help` must list `calypso ... (couche 1 : grgsm)`	never skipped

Options (and equivalent variable):

Option	Variable	Default	Role
`--prefix DIR`	`PREFIX`	`$GSM_ROOT/qemu-install`	where `make install` puts QEMU
`--build-dir DIR`	`BUILD_DIR`	`$QOSMO/build`	build out of tree; never run `configure` from the source root, it wipes a `build/` it created
`--venv DIR|none`	`VENV`	`~/.venv-qemu`	venv for configure (`tomli`); `none` = system python
`--bin-dir DIR|none`	`BIN_DIR`	`/usr/local/bin`	copy of the binaries; `none` = no copy
`--destdir DIR`	`DESTDIR`	empty	install under another root (ISO rootfs)
`--cc NAME`	`QOSMO_CC`	empty	forced compiler, e.g. `gcc-13` when `build/` was configured with it
`-j N`, `--only STEPS`, `--skip STEPS`, `--reinstall`, `--with-deps`, `-v`			parallelism, step selection, replay, apt packages, output on screen

Without `-v`, one log per step goes to `/tmp/qosmo-install/`, whose tail is shown on failure. `GSM_ROOT` (default `/opt/GSM`) sets the default prefix.

Without root: `./install.sh` works if you own `/opt/GSM`; the `bin` step is then *not applicable* (binaries left in `$PREFIX/bin`), or use `--prefix ~/qemu-calypso --bin-dir none`.

2.39.1.2 What the installer does, by hand

2.39.1.2.1 Prerequisites

Ubuntu 24.04 (the bench base). Packages (`./install.sh --print-deps`):

```
sudo apt-get install -y --no-install-recommends \
  build-essential git pkg-config python3 python3-venv python3-pip \
  libglib2.0-dev libpixman-1-dev libslirp-dev ninja-build socat
```

`socat` is used to query the QEMU monitor (`qosmo/run_modules/40-qemu.sh`). `./install.sh --check` checks what matters for `configure` (cc, make, ninja, pkg-config, `glib-2.0` and `pixman-1` as seen by pkg-config, the `venv/ensurepip` module): a package installed some other way is only a note.

2.39.1.2.2 Build

```
sudo mkdir -p /opt/GSM && sudo chown "$USER" /opt/GSM
git clone https://github.com/bbaranoff/qosmO /opt/GSM/qosmo
cd /opt/GSM/qosmo
python3 -m venv ~/.venv-qemu && . ~/.venv-qemu/bin/activate
pip install --no-cache-dir tomli
mkdir -p build && cd build
../configure --target-list=arm-softmmu --enable-l1-grgsm \
    --prefix=/opt/GSM/qemu-install --disable-werror --disable-docs
make -j"$(nproc)"
make install
```

The `bin` step then copies the binaries into the PATH:

```
sudo cp /opt/GSM/qemu-install/bin/qemu-system-arm /usr/local/bin/qemu-system-arm
sudo cp /opt/GSM/qemu-install/bin/qosmo           /usr/local/bin/qosmo
```

> **Keep the tree and `build/`.** The launchers use `/opt/GSM/qosmo/build/qemu-system-arm` (`c54x_exe/run.sh`, `qosmo/environnement/bench.env`), and c54x_exe/grgsm_exe compile the sources from this tree.

Other configure variants (`qosmo/README.md`): no option (bare platform), `--enable-l1-dsp` (C54x **inside** QEMU, not used by the bench). The bench only uses `--enable-l1-grgsm`.

2.39.1.2.3 Check

```
./install.sh --only verify    # -M help must list "calypso ... (couche 1 : grgsm)"
ls -la /opt/GSM/qosmo/build/qemu-system-arm
/opt/GSM/qosmo/build/qemu-system-arm -M help | grep calypso
qosmo -V          # C launcher (tools/qosmo/qosmo.c): "qosmo <version> (<tree>)"
```

The machine description says which layer 1 is compiled in (`hw/arm/calypso/calypso_mb.c`): `couche 1 : grgsm` (layer 1: grgsm) is expected.

Starting the machine (needs the firmware, next section), grgsm setup (no `CALYPSO_DSP_EXTERN`):

```
/opt/GSM/qosmo/build/qemu-system-arm -M calypso -cpu arm946 \
  -display none -parallel none -serial pty -serial pty \
  -monitor unix:/tmp/qemu-monitor-pont.sock,server,nowait \
  -kernel /opt/GSM/firmware/board/compal_e88/layer1.highram.elf
```

Expected on stderr:

```
char device redirected to /dev/pts/N (label serial0)
[l1] backend gr-gsm : GSMTAP udp/4730, SCH udp/4731
```

and `printf 'info status\n' | socat - UNIX-CONNECT:/tmp/qemu-monitor-pont.sock` answers `VM status: running`. Stop with Ctrl-C.

2.39.1.3 Firmware and osmocom-bb

QEMU runs `layer1.highram.elf`; osmocon loads `layer1.highram.bin` via romload; the mobile speaks L1CTL through osmocon. All three pieces come from osmocom-bb.

Prebuilt firmware: the tree `https://github.com/bbaranoff/firmware` (packaged as the `calypso-firmware .deb` by `packaging/build-debs.sh`), nothing to compile:

```
git clone --depth 1 https://github.com/bbaranoff/firmware /opt/GSM/firmware
ls /opt/GSM/firmware/board/compal_e88/layer1.highram.{elf,bin}
```

Rebuilding the firmware (optional): `sudo ./addition.sh --gnuarm` installs the gnuarm toolchain (arm-elf-gcc 4.5.2 in `/root/gnuarm/install/bin`, added to PATH by `/etc/profile.d/97-osmo-gnuarm.sh`); `sudo ./addition.sh --calypso-fw` then recompiles osmocom-bb (pinned `eac4eed1` + `patches/osmocom-bb-*.patch`) with `cd /opt/GSM/osmocom-bb/src && make`. With `OSMO_FW_INSTALLER=1`, the layer1 and rssi highram elf+bin are copied into `/opt/GSM/firmware/board/compal_e88` (layer1 also as `layer1.highram.gprs.*`). The GPRS layer1 (PDCH) exists only as `layer1.highram.bin`, loaded through `loader.compalram.bin`: with GPRS it no longer fits the 64 KiB the Compal boot loader accepts, so no compalram variant is built.

osmo-gapk, before osmocom-bb (the bench mobile uses `tch-voice io-handler gapk`, `c54x_exe/mobile_pont.cfg`); package `libasound2-dev` required:

```
cd /opt/GSM && git clone https://gitea.osmocom.org/osmocom/gapk osmo-gapk
cd osmo-gapk && autoreconf -fi && ./configure --enable-alsa && make -j"$(nproc)" && sudo make install && sudo
↪  ldconfig
```

osmocom-bb, host tools only (requires libosmocore installed). The bench patches (`patches/osmocom-bb-*.patch`) are applied by `tools/osmo-patches-apply.sh` (idempotent; `OSMO_PATCHES_ONLY=osmocom-bb` restricts it to this tree):

```
cd /opt/GSM && git clone https://gitea.osmocom.org/phone-side/osmocom-bb
OSMO_PATCHES_ONLY=osmocom-bb /opt/GSM/osmo-operator/tools/osmo-patches-apply.sh
cd osmocom-bb/src && make nofirmware
sudo install -m755 host/layer23/src/mobile/mobile /usr/local/bin/mobile
```

`osmocon` stays in the tree: `/opt/GSM/osmocom-bb/src/host/osmocon/osmocon` (path expected by `c54x_exe/run.sh` and `qosmo/environnement/bench.env`).

Check: `ls /opt/GSM/osmocom-bb/src/host/osmocon/osmocon && mobile -h`.

2.39.1.4 Useful variables

Variable	Default	Role
`CALYPSO_DSP_EXTERN=1`	unset	cuts out the gr-gsm L1, shares the API RAM with `c54x_exe`
`CALYPSO_PONT_LOCKSTEP=1`	set by `run.sh` when `LOCKSTEP=1`	QEMU waits for the DSP on every frame
`CALYPSO_PONT_RETRY_DIV`	64 (`run.sh`)	retry rate towards the DSP
`CALYPSO_GDB_PORT`	1234	ARM gdbstub (`off` to disable)

The other `CALYPSO_*` variables are documented in `qosmo/environnement/README.md` and in the headers of `qosmo/run_modules/*.sh`.

Symptom	Cause / fix

2.39.1.5 Troubleshooting

Symptom	Cause / fix
CPU at 0, crash at `0x840000`, osmocon loads nothing	missing `-kernel`
`QEMU a demarre puis s'est arrete` (QEMU started then stopped)	invalid machine or firmware; see the tail of `qemu.log`
SDL “parallel0” window grabbing the keyboard	forgot `-display none -parallel none`
`binaire QEMU absent` (QEMU binary missing)	build not done: `./install.sh --only build` (or `ninja -C build qemu-system-arm`)
`[FAIL] configure ... manque : glib-2.0 vu de pkg-config`	missing `-dev` package: `sudo ./install.sh --deps`
`[SKIP] qemu-system-arm et qosmo dans le PATH (... non inscriptible)`	not root: binaries stay in `$PREFIX/bin`, or use `--bin-dir DIR`
ninja fails on the very first unit in a `build/` that came from an image	`build/` configured with gcc-13, `cc` = gcc-11: `./install.sh --only build --cc gcc-13`
`/usr/local/bin/qosmo-grgsm` instead of `qosmo`	the `make -C tools/qosmo-launch install` fallback of `start.sh` (unversioned directory); use `make install` in `build/`
`osmocon` stuck mid-download	an osmocon killed during romload leaves the UART stub mid-block: restart QEMU, then osmocon

2.40 /opt/GSM/osmo-operator/wiki/Lancer-a-la-main.md

10802 octets, 270 lignes

2.40.1 Launching the emulated phone by hand, process by process

Back: Telephone-emule. Runnable version, network included: lancer-pas-a-pas.ipynb.

Each command below is the one **built** by `c54x_exe/run.sh` (DSP setup, called by `osmo-operator/start.sh`), with the variables **expanded** to the values `start.sh` sets in DSP mode. The function names (`etape1` to `etape5`) are those of `run.sh`; details and expected lines are in `c54x_exe/LAUNCH.md`.

Convention: one terminal per process (or `&` + redirection as below). Logs go to `/tmp/c54x-pont/`, like `run.sh`.

```
mkdir -p /tmp/c54x-pont
```

2.40.1.1 Common variables (DSP setup)

`start.sh` exports them before handing over to `c54x_exe/run.sh`; `run.sh` derives `CALYPSO_PONT_LOCKSTEP=1` from `LOCKSTEP=1` (its default). They are inherited by **all** the following processes:

```
export CALYPSO_BSP_STREAM=1        # one TS0 burst per frame, in FN order
export CALYPSO_RHEA_DMA_XFER=1     # without it: no SB, no BCCH, no SI
export CALYPSO_PONT_LOCKSTEP=1     # QEMU only advances the frame once the DSP is done
```

2.40.1.2 Step 0: the network (assumed installed)

The phone launches none of it. Bench order: STP, HLR, MGW, MSC, BSC (`qosmo/run_modules/10...14`), then the BTS (`60-bts.sh`). With systemd, as `c54x_exe/run_real.sh` does:

```
for u in osmo-hlr osmo-stp osmo-msc osmo-mgw osmo-bsc; do systemctl is-active --quiet "$u" || systemctl start
↪  "$u"; done
systemctl restart osmo-bts-trx
```

Without systemd, start the BTS as `qosmo/run_modules/60-bts.sh` does:

```
osmo-bts-trx -c /etc/osmocom/osmo-bts-trx.cfg
```

Move on when: the BTS VTY answers (`telnet 127.0.0.1 4241`, the osmo-bts default VTY port; check your config) and the BSC has accepted OML (`show bts 0` on the BSC VTY, 4242). The BTS is already sending its TRXC commands to 5701; nobody answers yet, which is normal: the bridge (step 5) will receive them.

Check: `ss -ulnp | grep -E ':580[0-2]'` shows osmo-bts-trx on 5800-5802 (osmo-bts-trx default; `pont/trx.py` sends the clock back there).

2.40.1.3 Step 1: `c54x_exe --arm` (the DSP), `etape1`

Clean up first:

```
rm -f /dev/shm/calypso_api_ram /tmp/calypso_dsp.sock /dev/shm/calypso_tch_cfg
```

Command (`INSNS=120000`, `IQ=none`, `AMP=30000`, `VERB=-v`):

```
cd /opt/GSM/c54x_exe
CALYPSO_IQDUMP_FCCH=1 CALYPSO_BSP_ATTENTE_MS=40 \
  ./c54x_exe --arm --insns 120000 --iq none --amp 30000 -v > /tmp/c54x-pont/dsp.log 2>&1 &
echo $! > /tmp/c54x-pont/dsp.pid
```

- Publishes `/dev/shm/calypso_api_ram` and `/tmp/calypso_dsp.sock`; listens on **UDP 6702** (BSP).
- ROM read from `/opt/GSM` (`--rom-dir` default).

Move on when (`run.sh` waits 5 s):

```
test -S /tmp/calypso_dsp.sock && grep -a "en attente de l'ARM" /tmp/c54x-pont/dsp.log
# pont : en attente de l'ARM sur /tmp/calypso_dsp.sock (API RAM : /dev/shm...)
ss -ulnp | grep 6702
```

("en attente de l'ARM" = waiting for the ARM.)

2.40.1.4 Step 2: QEMU / qosmo (the ARM + osmocom-bb layer1), etape2

Command (`GDB=1`, `GDB_STUB=1234`, `MONITOR=/tmp/qemu-monitor-pont.sock`):

```
rm -f /tmp/qemu-monitor-pont.sock
CALYPSO_PONT_RETRY_DIV=64 CALYPSO_DSP_EXTERN=1 \
  /opt/GSM/qosmo/build/qemu-system-arm -M calypso -cpu arm946 -display none -parallel none \
    -serial pty -serial pty -monitor unix:/tmp/qemu-monitor-pont.sock,server,nowait \
    -gdb tcp:127.0.0.1:1234 \
    -kernel /opt/GSM/firmware/board/compal_e88/layer1.highram.elf > /tmp/c54x-pont/qemu.log 2>&1 &
echo $! > /tmp/c54x-pont/qemu.pid
```

Optional gdb telnet console (`run.sh` starts it when `GDB=1`):

```
python3 /opt/GSM/qosmo/tools/gdb-telnet.py --port 44444 --stub 1234 \
  --elf /opt/GSM/firmware/board/compal_e88/layer1.highram.elf > /tmp/c54x-pont/gdb.log 2>&1 &
```

Move on when (`run.sh` waits 10 s):

```
grep -a "label serial0" /tmp/c54x-pont/qemu.log          # char device redirected to /dev/pts/N (label serial0)
grep -a "pont DSP : API RAM partagee" /tmp/c54x-pont/qemu.log
```

("API RAM partagee" = shared API RAM.) Then extract the modem pty:

```
sed -n 's/.*redirected to \(\/dev\/pts\/[0-9]*\) (label serial0).*/\1/p' /tmp/c54x-pont/qemu.log | head -1 >
↪  /tmp/c54x-pont/modem.pty
cat /tmp/c54x-pont/modem.pty
```

On the DSP side, `RESET #1 ... pc=0xff80` then `DSP boote (premier IDLE)` (DSP booted, first IDLE) show up once the firmware is loaded (step 3). Other expected QEMU lines: `[trx] pont DSP : RESET_DSP relache par le firmware -> PONT_RESET`, `[trx] pont DSP : le TDMA du firmware prend le relais du timer de boot`.

Check the machine: `printf 'info status\n' | socat - UNIX-CONNECT:/tmp/qemu-monitor-pont.sock` answers `VM status: running`.

2.40.1.5 Step 3: osmocon (romload + L1CTL relay), etape3

```
rm -f /tmp/osmocom_l2
stdbuf -oL -eL /opt/GSM/osmocom-bb/src/host/osmocon/osmocon -m romload -i 100 \
  -p "$(cat /tmp/c54x-pont/modem.pty)" -s /tmp/osmocom_l2 \
  /opt/GSM/firmware/board/compal_e88/layer1.highram.bin > /tmp/c54x-pont/osmocon.log 2>&1 &
echo $! > /tmp/c54x-pont/osmocon.pid
```

Move on when (`run.sh` waits 30 s):

```
grep -a "your code is running now" /tmp/c54x-pont/osmocon.log
# Received ident ack / Progress: 100% / Received branch ack, your code is running now!
test -S /tmp/osmocom_l2
```

The firmware's last printf can also be read from the monitor: `printf 'xp /96bx 0x008305f0\n' | nc -U /tmp/qemu-monitor-pont.sock` shows `DSP API Version: 0x4e2a 0x491a`.

2.40.1.6 Step 4: `mobile` (layers 2/3), `etape4`

Command (`L23_SYNC_RETRIES_SELECTION=8` in DSP mode):

```
ss -ltn | grep -q ':4347 ' && echo "VTY 4347 deja prise"      # already taken
L23_SYNC_RETRIES_SELECTION=8 stdbuf -oL mobile -c /opt/GSM/c54x_exe/mobile_pont.cfg \
  > /tmp/c54x-pont/mobile.log 2>&1 &
echo $! > /tmp/c54x-pont/mobile.pid
```

Move on when: the process is still alive after 2 s; within 5 s, on the osmocon side: `L1CTL_PM_REQ`, `L1CTL_RESET_REQ: FULL!`, `L1CTL_FBSB_REQ (arfcn=514 ...)`. Without the bridge, `FBSB RESP: result=255` in a loop is expected.

Check: `telnet 127.0.0.1 4347` then `show ms`.

2.40.1.7 Step 5: `pont_dsp.py` (BTS to DSP), `etape5`

The bridge is started from the osmo-operator root with `--dsp-port 6702`. `run.sh` adds `--no-record` only when `PONT_AIRREC=0` (I/Q recording is on by default):

```
cd /opt/GSM/osmo-operator
PONT_BSC_CFG=/etc/osmocom/osmo-bsc.cfg PONT_ARFCN=514 PONT_BSIC=7 \
  python3 pont/pont_dsp.py --dsp-port 6702 > /tmp/c54x-pont/pont.log 2>&1 &
echo $! > /tmp/c54x-pont/pont.pid
```

`PONT_BSC_CFG`, `PONT_ARFCN`, `PONT_BSIC` are shown at their defaults (`pont/config.py`): adapt them to your cell. I/Q recording writes to `/root/record.cfile` by default; add `--no-record` if you do not want it.

Move on when (`run.sh` waits 10 s):

```
grep -a "pont TRX : ports" /tmp/c54x-pont/pont.log
# pont TRX : ports 5700/5701/5702, ARFCN 514, BSIC 7, avance UL 3 trames
grep -a "horloge armee" /tmp/c54x-pont/pont.log     # BTS 127.0.0.1 : horloge armee vers le port 5800
grep -a "CTRL " /tmp/c54x-pont/pont.log | head      # TRXC commands from the BTS (POWERON, RXTUNE...)
```

("horloge armee" = clock armed.) Then `STATS fn=... | DL bursts=N` lines with N growing.

> **Noise (optional)**: to degrade the downlink, start `c54x_exe` with `CALYPSO_BSP_PORT=16702` and `python3 /opt/GSM/qosmo/tools/injecteur_bruit.py --cible dsp --mode ber --ber 0.01` **before** this bridge (the injector takes 6702); `run.sh` does it with `BRUIT_MODE=...`.

2.40.1.8 Step 6: check the service

What	Command	Expected
SI read	`grep -a "New SYSTEM INFORMATION" /tmp/c54x-pont/mobile.log`	SI 1, 2, 3, 4…
LAI	`grep -aoE "lai=[0-9]+-[0-9]+-[0-9]+" /tmp/c54x-pont/mobile.log | tail -1`	`lai=001-01-<your LAC>`
RACH sent	`grep -a "\[montant\] RACH" /tmp/c54x-pont/dsp.log; grep -ao 'UL bursts=[0-9]* tard=[0-9]* rach=[0-9]*' /tmp/c54x-pont/pont.log`	rach > 0

What	Command	Expected
camp	`show ms` (VTY 4347)	`C3 camped normally` (`tests/modules/60-camp.sh`)
LU	`grep -a "LOCATION UPDATING ACCEPT" mobile.log`; `show ms`	`MM idle, normal service`; IMSI in `show subscriber cache` (MSC VTY 4254)
call	VTY 4347: `enable`, `call 1 600` (600 = the bench echo; otherwise your number)	`call control state: ACTIVE` in `show ms`; `call 1 hangup`
A5/1	`dsp.log`: `[montant] TCH : le firmware poste la tache 13 ...`	

2.40.1.9 Stop and clean up

Reverse order: bridge, mobile, osmocon, gdb, QEMU, DSP, then purge (what `run.sh --stop` does):

```
for n in pont mobile osmocon gdb qemu dsp; do kill "$(cat /tmp/c54x-pont/$n.pid 2>/dev/null)" 2>/dev/null; done
sleep 1
rm -f /dev/shm/calypso_api_ram /tmp/calypso_dsp.sock /tmp/osmocom_l2 /tmp/qemu-monitor-pont.sock \
      /tmp/c54x-pont/modem.pty /tmp/osmocom_sap /tmp/ms_data /dev/shm/calypso_horloge \
      /dev/shm/calypso_rach /dev/shm/calypso_sdcch_ul /dev/shm/calypso_tch_facch_ul \
      /dev/shm/calypso_tch_sacch_ul /dev/shm/calypso_tch_ul /dev/shm/calypso_tch_cfg
```

Why each file matters: a leftover `/tmp/ms_data` makes the next mobile's bind fail silently; a leftover `calypso_horloge` slaves the bridge to a dead session.

2.40.1.10 Variant: gr-gsm setup, by hand

Same steps **without step 1** and without the three `export`s at the top:

- **Step 2**: the same QEMU command **without** `CALYPSO_DSP_EXTERN=1` or `CALYPSO_PONT_*`; wait for `[l1] backend gr-gsm : GSMTAP udp/4730, SCH udp/4731` in qemu.log. QEMU then listens on UDP 4730/4731.
- **Step 3**: identical.
- **Step 4**: `mobile -c <cfg>` **without** `L23_SYNC_RETRIES_SELECTION`; config with `io-tch-format rtp` (see Installer-grgsm_exe, Known issues).
- **Step 5**: `cd /opt/GSM/osmo-operator && python3 pont/pont.py > /tmp/c54x-pont/pont.log 2>&1 &` (`pont.py` rejects `--dsp-port`). In the full bench, `start.sh` waits until osmo-bts-trx is (re)seen, then runs `setsid python3 -u pont/pont.py` logging to `/dev/shm/pont.log`.
- Expected on the bridge side: `pont TRX : ports 5700/5701/5702, ARFCN 514, BSIC 7`, then `STATS fn=... | DL bursts=N`.

2.40.1.11 Mapping to the scripts

Step	`c54x_exe/run.sh`	`qosmo/run_modules` (gr-gsm setup via `start.sh`)
DSP	`etape1`	none
QEMU	`etape2`	`40-qemu.sh` (via the `/usr/local/bin/qosmo` launcher if present)
osmocon	`etape3`	`50-osmocon.sh` (plus `-d tr`)
mobile	`etape4`	`70-l2.sh` (`mobile -c $cfg -d $CALYPSO_MOBILE_DEBUG`)
bridge	`etape5`	`start.sh`, `CALYPSO_BRIDGE=pont`
BTS	none (your network)	`60-bts.sh`

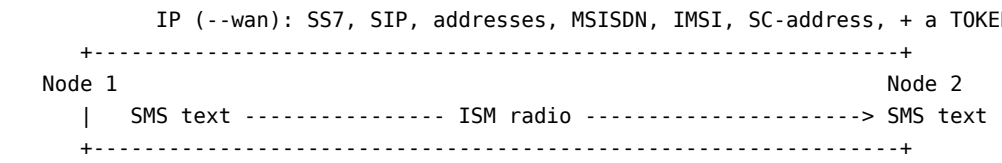
In DSP mode, `start.sh` plays qosmo's plan **without** `qemu,pty,osmocon,l2` (`DSP_MODULES_RETIRES`) then `execs c54x_exe/run.sh`: that is exactly step 0 (qosmo's plan brings up the network and the BTS) followed by steps 1 to 5.

2.41 /opt/GSM/osmo-operator/wiki/Payload-ISM.md

5800 octets, 115 lignes

2.41.1 SMS payload over ISM radio, the rest over IP (`--payload-ism`)

Between two machines running osmo-operator, `--wan` carries the whole inter-node link over IP: SS7 signalling, SIP voice, and the SMS relay (`scripts/sms-interop-relay.py`), which carries the message text in a JSON over TCP. The `--payload-ism` option keeps all of that over IP **except the SMS content**: the text travels over an **ISM-band radio link** (433 / 868 / 915 MHz, 2.4 GHz, licence-free), while the two relays still coordinate over TCP.



2.41.1.1 How it works

The SMS relay extracts the text from the TPDU of a mobile-originated SMS to reinject it as mobile-terminated on the other node. With `--payload-ism`:

1. **Transmission.** The relay draws a correlation token, transmits the text over the ISM radio stamped with that token, and sends over TCP only the token (plus the addresses). The TCP text field goes out empty.
2. **Reception.** The other relay receives the TCP JSON, sees `payload_via=ism`, and waits on the radio link for the text carrying that token before injecting the MT.

If the radio fails on transmission, the relay falls back to IP for that SMS (it never stays silent). If the text does not arrive over radio within the timeout (`recv_timeout`, 30 s by default), reception answers with an error.

The radio link is decoupled from the hardware: `scripts/ism_payload.py` talks to the **data interface** of the radio, frames the text into a single JSON line with a CRC32 (a truncated or corrupted frame is ignored), and correlates by token.

2.41.1.2 Backends

The backend is chosen with `network/setup-payload-ism.sh`, which writes the `[payload]` section of `sms-routing.conf` (read by the relay).

Backend	For what	Setup
`serial`	an ISM module on UART (LoRa, RFM69, CC1101, HC-12…) on `/dev/tty*`	`sudo ./network/setup-payload-ism.sh --serial /dev/ttyUSB0 --baud 9600`
`exec`	an SDR pipeline you provide (HackRF/LimeSDR/USRP + GNU Radio, rtl_433…)	`sudo ./network/setup-payload-ism.sh --exec --tx-cmd ... --rx-cmd ...`
`fifo`	two named pipes, to hook up the radio some other way (`socat`…)	`sudo ./network/setup-payload-ism.sh --fifo --tx /tmp/ism_tx --rx /tmp/ism_rx`
`loopback`	try the SMS chain **without radio**, on a single machine	`sudo ./network/setup-payload-ism.sh --loopback`

For the `exec` backend, `setup-payload-ism.sh --exemples` (alias `--examples`) drops two templates, `/opt/GSM/ism/ism_tx.sh` and `ism_rx.sh`: `ism_tx.sh` receives a frame (a JSON line) on its standard input and must transmit it; `ism_rx.sh` must write each received frame to its standard output. Until they are filled in with your radio chain, the text does not get through.

Other options: `--selftest` (runs the `ism_payload.py` self-test on the chosen backend and exits), `--off` (removes the `[payload]` section: text goes back over IP).

2.41.1.3 Launch

On **each** node, with the WAN already in place:

```
# 1. set up the radio link (once, or redone by start.sh)
sudo ./network/setup-payload-ism.sh --serial /dev/ttyUSB0 --baud 9600

# 2. launch the bench: WAN over IP + payload over ISM
sudo ./start.sh --wan --payload-ism
```

`--payload-ism=serial` (or `=exec`, `=fifo`, `=loopback`) additionally writes the `[payload]` section at launch, with the device/commands taken from the environment: `ISM_SERIAL`, `ISM_SERIAL_BAUD`, `ISM_TX`, `ISM_RX`, `ISM_TX_CMD`, `ISM_RX_CMD`. Without a backend, `--payload-ism` reuses the section already written.

To go back to text over IP: `sudo ./network/setup-payload-ism.sh --off`.

2.41.1.4 Verify without radio

```
python3 scripts/ism_payload.py --selftest                # in-memory loop
python3 scripts/ism_payload.py --selftest --link fifo \
        --tx /tmp/ism_a --rx /tmp/ism_b                  # loop over pipes
sudo ./network/setup-payload-ism.sh --loopback           # full SMS chain, one node
```

The self-test sends three texts (including 140 bytes and accented characters) and checks the rejection of a frame with a bad CRC. `ism_payload.py` options: `--selftest`, `--link loopback|fifo|serial|exec`, `--tx`, `--rx`, `--serial`, `--tx-cmd`, `--rx-cmd`.

2.41.1.5 The `[payload]` section of `sms-routing.conf`

```
[payload]
mode = ism                ; ism = text over radio; absent/other = text over IP
link = serial             ; serial | exec | fifo | loopback
serial = /dev/ttyUSB0     ; (serial) the UART device
; tx = /tmp/osmo_ism_tx   ; (fifo) transmit pipe
; rx = /tmp/osmo_ism_rx   ; (fifo) receive pipe
; tx_cmd = /opt/GSM/ism/ism_tx.sh   ; (exec) transmit command
; rx_cmd = /opt/GSM/ism/ism_rx.sh   ; (exec) receive command
recv_timeout = 30         ; wait for the text over radio before failing (s)
```

2.41.1.6 Regulatory

Only transmit on an ISM band and at a power allowed where you are (433 and 868 MHz in Europe, 902-928 MHz in region 2, 2.4 GHz), or into a dummy load / Faraday cage. It is your responsibility as the bench's operator.

File	Role

2.41.1.7 Files

File	Role
`scripts/ism_payload.py`	the ISM payload link: backends, frame + CRC, correlation by token, self-test
`scripts/sms-interop-relay.py`	SMS relay: token over TCP, text over radio when `[payload] mode=ism`
`network/setup-payload-ism.sh`	writes `[payload]`, sets the serial baud rate, drops the SDR templates, self-test
`start.sh`	the `--payload-ism[=backend]` option (`start-native.sh` is a symlink to it)

2.42 /opt/GSM/osmo-operator/wiki/README-notebook.md

7852 octets, 163 lignes

2.42.1 The install notebook

← README · Notebook: installer-osmo-jusquau-dsp.ipynb · See also: native, docker, tests

`wiki/installer-osmo-jusquau-dsp.ipynb` builds the whole bench **step by step, in your browser**: from a fresh Ubuntu 24.04 to a verified call on the emulated Calypso DSP (c54x_exe + TI mask ROM). Each step is one cell, followed by a check cell, so you see every component being built and can stop and look at any point.

It is a teaching path, not the fastest one. To simply get a working bench, use the ISO or the Docker images built by `build.sh`.

Target	Ubuntu 24.04 LTS (noble), x86_64, **fresh**
Disk	~12 GB free under `/opt`
Time	~1 h on 8 cores (Osmocom core 20–30 min, QEMU 10–15 min)
Network	gitea.osmocom.org, github.com, ftp.freecalypso.org, archive.ubuntu.com

> **Do not run it on a machine that already has a bench installed** (the ISO, a previous native install). Every step ends in `make install` into `/usr/local` followed by `ldconfig`: it would overwrite the libraries your existing bench uses. Use a disposable container (below) or a fresh VM. Step 0 warns when it finds Osmocom already in `/usr/local`.

2.42.1.1 Run it in a disposable container (recommended)

The container runs as root, so the notebook needs no sudo. `--privileged` is required by the later steps: `setcap` (step 2), SCTP for osmo-stp, tun/GTP for the GGSN, `ip_forward`. `--shm-size` keeps `/dev/shm/calypso_*` from filling Docker's 64 MB default.

On the host:

```
docker run -it --name nb-ubuntu --privileged --shm-size=2g -p 8888:8888 ubuntu:24.04 bash
```

In the container:

```
apt-get update && apt-get install -y python3-venv git
python3 -m venv /venv && /venv/bin/pip install -q jupyterlab
git clone https://github.com/bbaranoff/osmo-operator /nb
cd /nb && /venv/bin/jupyter lab --ip=0.0.0.0 --allow-root --no-browser \
  --LabApp.default_url=/lab/tree/wiki/installer-osmo-jusquau-dsp.ipynb
```

Open the `http://127.0.0.1:8888/lab/tree/…?token=…` URL it prints: the notebook opens directly. Run the cells in order with **Shift+Enter** and read each check before moving on.

If SCTP is refused, load the module on the **host**: `sudo modprobe sctp`.

A bare `ubuntu:24.04` container is addressed like a native machine: `network/osmo-ip-plan.sh` adds the node's private addresses itself (192.168.2.1 and 192.168.2.10, which osmo-ggsn and osmo-sgsn bind to). Only an image built from this repository (`OSMO_IMAGE=1`, set by the Dockerfile) leaves addressing to `start-docker.sh` / compose.

2.42.1.1.1 Stop and resume

The container is named and not `--rm`, so a closed terminal or a crashed Jupyter does not lose the build already done in `/opt/GSM`:

```
docker start -ai nb-ubuntu
cd /nb && /venv/bin/jupyter lab --ip=0.0.0.0 --allow-root --no-browser \
  --LabApp.default_url=/lab/tree/wiki/installer-osmo-jusquau-dsp.ipynb
```

Cells are idempotent: re-run the one you stopped at. When you are done: `docker rm nb-ubuntu`.

2.42.1.1.2 Testing local changes

Step 1 clones osmo-operator from GitHub, so the notebook installs the **pushed** version. To test an edited notebook before pushing, mount your clone instead of cloning it:

```
cd /path/to/osmo-operator
docker run -it --name nb-ubuntu --privileged --shm-size=2g -p 8888:8888 \
  -v "$PWD/wiki:/nb" ubuntu:24.04 bash
# then, in the container, the same commands without the git clone, and
#   --LabApp.default_url=/lab/tree/installer-osmo-jusquau-dsp.ipynb
```

The notebook file is then edited in place on the host, but the patches, configs and scripts it uses still come from GitHub.

2.42.1.2 Run it on a fresh Ubuntu 24.04

Follow the *Before opening the notebook* cell: Jupyter in a venv (Ubuntu 24.04 refuses `pip install` outside one), and sudo unlocked **in the terminal that starts Jupyter**, because a notebook cannot answer a password prompt:

```
sudo apt-get install -y python3-venv git
python3 -m venv ~/.venv-jupyter && ~/.venv-jupyter/bin/pip install jupyterlab
sudo -v
~/.venv-jupyter/bin/jupyter lab
```

`sudo -v` only lasts 15 minutes without a sudo call, and the QEMU build alone can run longer. For a full run, prefer a temporary `NOPASSWD` rule (`sudo visudo -f /etc/sudoers.d/osmo-nb`) and remove it afterwards.

2.42.1.3 Run it unattended and keep the outputs

To execute every cell without the browser and save the result with all its outputs:

```
/venv/bin/jupyter nbconvert --to notebook --execute \
  --ExecutePreprocessor.timeout=-1 --output executed.ipynb installer-osmo-jusquau-dsp.ipynb
/venv/bin/jupyter nbconvert --to html executed.ipynb     # readable in any browser
```

It prints nothing while it runs and stops at the first failing cell with a `CellExecutionError` showing that cell and its error.

2.42.1.4 The steps

Step	What it does	Needed for the DSP call
0	`/opt/GSM`, the shared environment file, sudo check	yes
1	clone osmo-operator, apt dependencies and the `osmocom` account through `install.sh`	yes
1b	GNU Radio build dependencies	no (gr-gsm chain only)
2	capture capabilities for `dumpcap`	yes
3	the Osmocom core, same versions and options as the Dockerfile, with the force-rand-toy patches	yes
4	osmo-gapk, before osmocom-bb	yes
5	the SMSC (gsup-smsc-proto, sms-coding-utils)	for SMS
6	osmocom-bb host tools: osmocon, mobile, trxcon	yes
7	prebuilt Calypso firmware	yes
7b	the transceiver tree, `trx` firmware and host `transceiver` (`./start.sh --bts`)	no
8	qosmo, the Calypso QEMU	yes
9	c54x_exe and the DSP ROM from FreeCalypso	yes
9b	FreeCalypso Magnetite for the C139 (TMS470 under Wine), its flash image, rvinterf (`--ms-magnetite`)	no
10	grgsm_exe	no
11	runtime setup: configs, run-time scripts, directories	yes
11b	audio sinks	only to hear the call
12	start the network, the BTS, then the DSP bench: `./start.sh --bts-classique --ms`, or the Magnetite / `--bts` variants	yes
13	camp, location update, call to 600 (echo test)	yes
14	full test ladder from bbaranoff/tests	no
15	stop	—

The launcher is `./start.sh`; `start-native.sh` is a symlink to it.

To start the bench **process by process** instead of through `start.sh`, run the companion notebook lancer-pas-a-pas.ipynb once the install is done: it installs nothing, starts the network daemons one by one, then the emulated phone (DSP, QEMU, osmocon, mobile, bridge), and checks each one before the next, with optional sections on the `--bts` variant (12b), the classic BTS seen from the bridge (12a) and the rssi application (`banc-rssi`, last section). The same steps as a page: Lancer-a-la-main.

2.42.1.5 Troubleshooting

A cell hangs with no output. It is probably waiting for a sudo password it can never receive. Interrupt the kernel, run `sudo -v` (or add the `NOPASSWD` rule) in the terminal that started Jupyter, and re-run the cell.

`configure: error: Package requirements (…) were not met.` A `-dev` package is missing (for example `sqlite3` → `libsqlite3-dev` for osmo-hlr). Install it, then re-run the cell. On a fresh system this means the apt list of step 1 is incomplete: please open an issue.

Step 11b prints "no audio server reachable". Expected in a container or on a server. The core, camp, location update, SMS and call set-up do not need audio.

Binder or Colab. They can display the notebook but cannot run it to the end: Binder gives no root and about 2 GB of RAM, Colab runs Ubuntu 22.04 without `--privileged` or SCTP.

2.43 /opt/GSM/osmo-operator/wiki/Reference.md

38561 octets, 923 lignes

2.43.1 osmo-operator — reference

osmo-operator is a complete GSM operator (Osmocom core, BTS, Asterisk, SMSC) plus an **emulated Calypso handset** on one machine: no radio hardware, no SIM card, no phone. The handset runs the real OsmocomBB `layer1` firmware on an emulated TI Calypso baseband, camps on the cell, authenticates, ciphers, sends SMS and holds a voice call. The same bench also ships as a live ISO with a 4G core (open5gs + srsRAN, CSFB) and a postmarketOS phone VM, and can be meshed with other machines over SS7. This page is the full reference; the top-level README is the index.

Contents

- 1. The live ISO
- 2. Quick start from source
- 3. The layer-1 chains
- 4. Profiles
- 5. Building the images
- 6. Daily use
- 7. Diagnosing the radio side
- 8. The qosmo launcher
- 9. One PLMN
- 10. PHY modes
- 11. Multi-operator and SS7
- 12. Other extensions
- 13. Related documents
- Known issues

2.43.1.1 1. The live ISO

Download links and checksums: doc/README-iso.md. The GitHub release ships the image in `.part-NN` pieces (release files are capped at 2 GiB):

```
cat osmo-operator-desktop.iso.part-* > osmo-operator-desktop.iso
sha256sum -c --ignore-missing SHA256SUMS
```

Live ISO password	`osmo`
postmarketOS phone password	`147147`
Minimum host	x86_64, 8 GB RAM, 4 cores, nested virtualization in a VM
Image identity	`IMAGE_VERSION="OSMO_EGPRS_V2"` · `OSMO_ROLE=operator` · `OSMO_LITE=0`

2.43.1.1.1 1.1 Run the image

The live root is a tmpfs (~6 GB in `toram` mode); logs and captures are bounded and purged at every boot. The bench runs two QEMUs (Calypso + pmOS): in a VM, **enable nested virtualization**, otherwise the phone VM falls back to software emulation and becomes very slow.

QEMU

```
qemu-system-x86_64 -cdrom osmo-operator-desktop.iso -m 8G -enable-kvm \
  -cpu host -smp 4 -nic user,hostfwd=tcp::8080-:8080
```

VirtualBox

1. New VM: Linux / Ubuntu 64-bit, 8 GB, 4 CPUs, 3D off.
2. System → Processor: tick *Nested VT-x/AMD-V* (if greyed out: `VBoxManage modifyvm "<name>" --nested-hw-virt on`).
3. ISO on the optical drive, no disk needed.
4. NAT port forward: host 8080 → guest 8080.

VMware: new VM from the ISO, Linux / Ubuntu 64-bit, 8 GB, 4 cores, tick *Virtualize Intel VT-x/EPT or AMD-V/RVI*, skip Easy Install.

USB stick (hybrid ISO, boots UEFI or BIOS; on Windows use Rufus in *DD Image* mode):

```
sudo dd if=osmo-operator-desktop.iso of=/dev/sdX bs=4M status=progress conv=fsync
```

2.43.1.1.2 1.2 Getting started

Three launchers are pinned in the dock. 2G and 4G are independent and can start in any order; **launch the smartphone last**, once both networks are up.

	Launcher	What it starts
	2G — the red handset	The GSM core and the emulated radio: osmo-stp / bsc / msc / hlr / sgsn / ggsn / pcu, the Calypso under QEMU, the two BTS and the two osmocom-bb mobiles, Asterisk, the SMSC.
	4G — the signal bars	open5gs, srsENB and srsUE over ZMQ, and the SGs interface to the MSC (CSFB).
☎	**Smartphone** — the Adwaita phone icon	The postmarketOS VM: oFono on the virtual modem, data on 4G, voice and SMS on 2G.

From a terminal:

```
cd /opt/GSM/osmo-operator && sudo ./start-native.sh --dsp   # 2G handset on the Calypso DSP
sudo ./start-native.sh --menu                               # ask interactively
checks/diag-stp-operator.sh                                 # check that everything is in place
```

2.43.1.1.3 1.3 What's inside

2G / GSM (Osmocom)

- Full core: osmo-stp (M3UA, SCTP :2905), osmo-bsc, osmo-msc, osmo-hlr, osmo-sgsn, osmo-ggsn (GTPv1-C/U), osmo-pcu, osmo-mgw (MGCP :2427).
- Two DCS1800 cells in LAC 1: BTS 0 (Cell ID 6001, BSIC 7) and BTS 1 (Cell ID 6012, BSIC 8), each on its own osmo-bts-trx.
- **A real emulated Calypso as layer 1**: `qemu-system-arm -M calypso -cpu arm946` (qosmo) runs `layer1.highram.elf`, loaded by `osmocon` exactly as on a C123. `pont/pont.py` turns bursts into I/Q for `fake_trx.py`, then `trxcon` and two osmocom-bb `mobile` instances on the terminal side.
- GPRS/EDGE: BSSGP/NS to the PCU, PDP contexts through osmo-ggsn.
- Voice: external MNCC → Asterisk (SIP :5060, FR/HR) via osmo-sip-connector.
- SMS: SMS-over-GSUP → proto-smsc-daemon, SMPP :2775 with a test ESME, inter-operator relay :7890.
- A3A8 with deterministic RAND (`*-force-rand-toy` patches): the bench is reproducible from one boot to the next.
- Live GSMTAP capture, ready for Wireshark.

The DSP (the Calypso's TMS320C54x on TI's real mask ROM)

- `./start-native.sh --dsp` (the default) hands layer 1 to the baseband's own DSP: a C54x core emulated outside QEMU (c54x_exe), running the mask ROM dumped from a real Calypso (PROM0-3, DROM, PDROM). `--grgsm` uses gr-gsm (grgsm_exe) instead.
- ARM (QEMU) and DSP share the API RAM through `/dev/shm/calypso_api_ram` and run in lockstep, one TDMA frame at a time, over `/tmp/calypso_dsp.sock`. Downlink bursts reach the DSP's serial port (BSP) over UDP 6702 via `pont/pont_dsp.py`.
- The ROM does FCCH/SCH acquisition, BCCH decoding (SI1-4), camping, location update, SMS MO/MT, MO/MT calls with A5/1, speech audible both ways. Limitations: see Known issues.

4G / LTE (open5gs + srsRAN)

- open5gs core (MME, SGW-C/U, SMF, UPF, PCRF, HSS…) on dedicated loopbacks, Prometheus :9090, WebUI :9999, MongoDB behind the HSS.
- PLMN 001/01, TAC 7. NAS null ciphering (EIA2/EIA1 + EEA0) on purpose: NAS reads in clear in Wireshark.
- srsENB (10 MHz, EARFCN 3350, band 7) and srsUE over ZMQ, no SDR. Milenage test subscriber IMSI 001010001000001; the UE gets an IP in a netns and reaches the Internet through NAT.

CSFB

- SGs between osmo-msc and the MME (:29118), mapping `TAI (001-01, TAC 7) → LAI (001-01, LAC 1)`.
- Combined EPS+IMSI attach: an incoming call is paged by the MME and the UE falls back to the BTS.
- Two subscribers (100101, 100102), one per cell, wired for an inter-cell call with fallback.

postmarketOS (the phone)

- x86_64 pmOS VM with a patched kernel (`CONFIG_PPP` enabled); prebuilt kernel in `/opt/user_interface/kernel/pmos/`.
- 4G data: NetworkManager → pppd (`ATD*99***1#`) → virtio-console `osmo.data` → `osmo-phonesim-banc.py` → UE netns → srsUE → srsENB → UPF → Internet.
- 2G voice and SMS: virtio-console `osmo.modem`, where `osmo-phonesim-banc.py` plays a 27.007 AT modem driven by oFono.
- Audio: two duplex PulseAudio chains (GAPK bridge to the osmocom-bb mobile, and an echo-cancelled chain).
- Result: data on 4G, voice and SMS on 2G, same identity — a CSFB handset's behaviour at modem level.

2.43.1.2 2. Quick start from source

2.43.1.2.1 2.1 From the published Docker image

```
sudo docker pull ghcr.io/bbaranoff/osmo-operator/osmocom-nitb:latest
git clone https://github.com/bbaranoff/osmo-operator
cd osmo-operator
sudo ./launch/start-docker.sh
```

Then enter the container and bring the stack up:

```
sudo docker exec -ti osmo-operator-1 bash

cd /opt/GSM/osmo-operator
./start-native.sh --regen     # regenerate configs from the templates (first run)
./start-native.sh --stop      # make sure nothing is left over
./start-native.sh             # start the bench
```

`start-native.sh` attaches to tmux at the end: a live GSM network with a mobile camped on it.

2.43.1.2.2 2.2 Native, without Docker

The repository installs directly on Ubuntu 24.04 (noble, the same base as the Dockerfile):

```
sudo ./install.sh                 # everything: deps, sources, build, binaries, configs, desktop
./install.sh --list               # show the steps, do nothing
./install.sh --check              # what is already in place
sudo ./install.sh --reinstall     # replay everything
sudo ./install.sh --only bureau   # desktop icons and shortcuts only
```

The launcher is the same `./start-native.sh`, which delegates to `/opt/GSM/qosmo/run.sh`. On a host with Docker, `./start-native.sh --docker …` hands over to `launch/start-docker.sh`.

2.43.1.2.3 2.3 Stopping

```
./start-native.sh --stop      # stops the stack AND the DSP bench, with or without --dsp
./start-native.sh --status    # what is running
```

2.43.1.3 3. The layer-1 chains

Both chains run the same core network, the same BTS and the same `mobile`; they differ in **who demodulates the air interface**.

	`--grgsm`	**`--dsp`** (default)
Layer 1	gr-gsm inside QEMU + `grgsm_exe` bridge	real TI mask-ROM C54x, **outside** QEMU
QEMU's job	ARM7 + gr-gsm layer 1	ARM7 only (`CALYPSO_DSP_EXTERN=1`)
Bridge	`pont/pont.py`	`pont/pont_dsp.py`
Status	end to end: camping, LU, auth, A5/1, SMS, voice call	camping, SI1-4, LU, SMS, calls; every speech frame is flagged `B_BFI`
Use it for	getting work done	studying the real DSP

```
./start-native.sh             # real C54x mask ROM (default)
./start-native.sh --dsp       # the same, explicitly
./start-native.sh --grgsm     # gr-gsm layer 1
```

2.43.1.3.1 3.1 Three trees, one QEMU

tree	what it is
/opt/GSM/qosmo	the QEMU fork: `-M calypso`, ARM7TDMI + platform. Carries `run.sh`, `run_modules/`, `cfgs/`. The layer 1 is a `configure` option: `--enable-l1-grgsm` or `--enable-l1-dsp`.
/opt/GSM/c54x_exe	the TMS320C54x running **outside** QEMU on the real TI mask ROM (`rom/calypso_dsp.*.bin`). Compiles `qosmo`'s sources, does not copy them.
/opt/GSM/grgsm_exe	the gr-gsm layer 1 outside QEMU.

`qemu-system-arm -M help` names the layer 1 actually linked in:

configure	`-M help` shows
(nothing)	`couche 1 : aucune`
`--enable-l1-grgsm`	`couche 1 : grgsm`
`--enable-l1-dsp`	`couche 1 : c54x`

2.43.1.3.2 3.2 The gr-gsm chain

```
flowchart LR
    mobile["mobile (layer23)"] -->|L1CTL| osmocon
    osmocon -->|romload, serial pty| QEMU
    subgraph QEMU["qosmo (-M calypso)"]
        ARM["ARM7<br/>layer1.highram.elf"]
        L1G["gr-gsm layer 1"]
        ARM <--> L1G
    end
    QEMU <-->|"GSMTAP 4730 / SCH 4731"| PONT["pont/pont.py"]
    PONT <-->|"TRXD 5700-5702"| BTS["osmo-bts-trx"]
    BTS -->|Abis/IP| BSC
    BSC --> CORE["MSC / HLR / MGW / Asterisk"]
```

2.43.1.3.3 3.3 The DSP chain (`--dsp`)

`--dsp` does **not** select a different QEMU. The stack comes up exactly as in gr-gsm mode (core, BTS, side-car, tmux) and only the four Calypso modules (`qemu`, `pty`, `osmocon`, `l2`) are handed over to `c54x_exe/run.sh`, which runs five steps of its own:

```
1. c54x_exe --arm     the DSP: /tmp/calypso_dsp.sock + shared API RAM,
                      DL bursts on UDP 6702
2. qemu-system-arm    the ARM + osmocom-bb layer1, gr-gsm layer 1 cut out
3. osmocon            romload on the serial pty, L1CTL relay
4. mobile             osmocom-bb layers 2/3
5. pont_dsp.py        TRXD 5700-5702 facing osmo-bts-trx, bursts down to the
                      DSP, uplink through the /dev/shm side-bands
```

```
./start-native.sh --dsp                  # the full stack, then the DSP bench
MODE=dsp PONT=1 /opt/GSM/c54x_exe/run.sh  # the mobile chain alone, no network
BANC_DSP=none ./start-native.sh --dsp     # the stack only, no DSP bench
```

`--dsp` starts **no bridge of its own**: `c54x_exe/run.sh` runs its own on 5700-5702 (step 5). A second one on the same ports would die on *Address already in use*. Step details and variables: Start-native § 8-dsp.

2.43.1.3.4 3.4 The Calypso as the BTS transceiver (`--bts` / `--osmo-trx`)

A third shape of the DSP chain: the emulated phone no longer camps on BTS#0, it *is* BTS#0's radio. QEMU boots `trx.highram.elf` (the `trx` app of the transceiver branch, tree `/opt/GSM/osmocom-bb-transceiver`), and the host app `transceiver` from the same tree replaces both `mobile` and the bridge: it talks L1CTL to the phone through osmocon and exposes the TRXD UDP interface (5700-5702) that `osmo-bts-trx` expects. The rest of the stack (BSC, MSC, side-car) comes up unchanged.

```
./start-native.sh --bts          # alias of --osmo-trx; implies --dsp; c54x_exe/run.sh runs with OSMO_TRX=1
./start-native.sh --bts --ms       # the BTS radio, then the second phone over the air (INSTANCE=2, see 3.6)
./start-native.sh --bts --ms --ts 2  # one Calypso TRX per timeslot: TS1 (TCH/F) served by a 3rd QEMU (OSMO_TRX_TS1=1, t
sudo armbian/build-armbian.sh      # SD image via github.com/armbian/build; --prepare, --no-stack, --board=
```

(`--ms` alone, or no option, is the usual mobile + bridge setup.)

What changes in `c54x_exe/run.sh` with `OSMO_TRX=1`: step 2 loads `trx.highram.elf`, step 4 launches `transceiver` instead of `mobile`, step 4b relaunches `osmo-bts-trx` when BTS#0 is absent (`OSMO_TRX_BTS_RELANCE=0` to disable; it exits whenever its transceiver disappears, so wait for the old process to finish its shutdown before `banc-trx`), step 5 (the bridge) is skipped because the transceiver holds the same ports, and step 5b starts the VSP audio bridge (`pont.vsp_audio`).

Variables: `OSMO_TRX_TS_MASK` / `OSMO_TRX_TX_MASK` (RX and TX timeslot masks; `-m` only sets RX, `-M` sets TX), `OSMO_TRX_TCH=1` (default: one phone serving TS0 and TS1, masks `0x03`, `CALYPSO_TRX_TX_SLOTS=2`), `OSMO_TRX_TS1=1` (a second phone serves TS1: `INSTANCE=3`, `OSMO_TRX_PHONE=2`, `transceiver -2 -L /tmp/osmocom_l2_3`), `OSMO_TRX_ARGS` and `TRANSCEIVER` (override the arguments and the binary), `CALYPSO_BSP_TRX`, `CALYPSO_BSP_TRX_DELTA`, `CALYPSO_BSP_TRX_OFFSET` (uplink frame-number calibration of the BSP), `CALYPSO_BSP_TRX_UL=0` (disable the uplink bit prosthesis), `MONTANT_ROM_TX_SB_RECODE` (SCH re-encoding, on by default in TRX mode).

The host app needs `patches/osmocom-bb-transceiver-qemu-bench.patch`, applied by the Dockerfile and by `install_modules/40-patches.sh`: `--no-sync` (BTS mode as soon as the phone resets, there is no cell to FBSB on inside QEMU), `--ts-mask`, `--l1ctl/--base-port`, `-L PATH`, and `SETTSC`, `SETFORMAT` (TRXD v0 answered) and `NOMTXPOWER` acknowledged, since `osmo-bts-trx` 1.10 treats them as critical. The firmware side of the patch: single TX window per mask, `BTS_RX_LAG` margin, TA/AFC loops removed from the BTS sync task, idempotent `BTS_MODE`.

Configuration that goes with it: `configs/osmo-bts-trx.cfg` has `osmotrx fn-advance 20` / `rts-advance 5`; `configs/osmo-bsc.cfg` gets `timeslot 1 phys_chan_config TCH/F` (`__TS1_PCHAN_BTS0__`, set by `generate_configs.sh` when `OSMO_TRX=1`; `SDCCH8` otherwise) so the SDCCH comes from the SDCCH/4 of TS0. Restart osmo-bsc for a config change to apply.

Logs: `/tmp/c54x-pont/dsp.log` (`[rom-tx] ... NB n, SB n, FB n`, `[ts0] mode TRX`, `[rom-tx] SB recode`, `[ts0] prothese montante`), `osmocon.log` (`STALE BURST` means bursts arriving outside their window).

2.43.1.3.5 3.5 The C123 screen and keypad under QEMU

`qosmo` emulates three SoC devices (`hw/arm/calypso/`): `calypso_armio.c` (the ARMIO block at 0xFFFE4800: the 5x5 key matrix, IRQ 8), `calypso_i2c_ctrl.c` (the I2C controller at 0xFFFE2800) and `calypso_st7558.c` (the 96x65 LCD, I2C slave 0x3c). Any firmware app that draws and reads keys works: `rssi`, `menu`, `hello_world`, `trx`.

- The screen is published in `/dev/shm/calypso_lcd` (`CALYPSO_LCD_SHM`) on every flush, and also on the QEMU graphic console (`-display sdl`).
- Keys arrive on the Unix datagram socket `/tmp/calypso_keys.sock` (`CALYPSO_KEYS_SOCK`): `press 5`, `release 5`, `tap menu 200`. Key names: `0-9`, `star`, `hash`, `menu`, `lsb`, `rsb`, `up`, `down`, `left`, `right`, `ok`, `power`. `CALYPSO_KEYS_DEBUG=1` traces them on stderr.
- `tools/osmo-c123-gtk.py` (run with `/usr/bin/python3`) shows the photo of the phone from `data/c123.jpg`, overlays the live LCD in the `lcd` zone of `data/c123-zones.json` and sends a press/release for each key clicked; the PC keyboard works too. `--calibrate` redraws the zones with the mouse.

```
FIRMWARE_APP=rssi bash /opt/GSM/c54x_exe/run.sh     # steps 1-3 only: DSP, QEMU, osmocon
/usr/bin/python3 tools/osmo-c123-gtk.py              # the phone window
echo "tap menu" | socat - UNIX-SENDTO:/tmp/calypso_keys.sock   # or by hand
```

2.43.1.3.6 3.6 Two Calypsos and the air between them

With `--bts` the emulated phone is the BTS radio, so a second emulated phone is needed to be the mobile. `c54x_exe/run.sh INSTANCE=2` runs a complete second chain (DSP, QEMU, osmocon, mobile) next to the first. It re-executes itself in a private mount namespace where `/dev/shm` is the host's `/dev/shm/calypso2`, so the hard-coded `/dev/shm/...` paths of c54x_exe and qosmo are unchanged; the host's real `/dev/shm` stays readable at `/run/calypso-shm-hote`. Everything else is shifted: `/tmp/c54x-pont2`, `/tmp/osmocom_l2_2`, `/tmp/calypso_dsp2.sock`, gdb 1235/44445, keypad socket `/tmp/calypso_keys2.sock`, BSP UDP port 6712, mobile VTY +100 (a derived copy of `mobile_pont.cfg`).

The air is `pont/air.py`: each c54x_exe publishes the bursts its ROM emits in the ring `calypso_tx_rom` (including the SB, dummy and FB bursts of a BTS, one per frame in TRX mode), and each BSP accepts bursts in TRXDv0 over UDP. The relay feeds A's ring to B's BSP and B's ring to A's BSP, tags bursts with their timeslot, decodes the ASSIGNMENT COMMAND on the TS0 SDCCH/4 (`pont.gsm.xcch_decode`) to announce the TCH to the mobile's c54x_exe (`calypso_tch_cfg`), and keeps relayed frame numbers continuous across a restart of `banc-trx`. It is step 5 of instance 2 (`AIR=1`), in place of the bridge, which must not touch 5700. With `OSMO_TRX_TS1=1`, `air.py` gets a third endpoint (`--c-ring`, `--c-port`) for phone 2.

```
banc-trx          # instance 1: the BTS radio (restart it after a rebuild of c54x_exe or trx.highram)
banc-mobile       # instance 2: the mobile, plus the air; banc-mobile-stop / -status / -logs
c123-2            # its screen and keypad
```

The two instances are not slaved to each other: each runs real time (4.62 ms per frame) and the mobile's BSP waits up to 40 ms for a late frame. Logs: `/tmp/c54x-pont2/air.log` (`A->B n (NB SB DUMMY FB)`), `/tmp/c54x-pont2/dsp.log`, `/tmp/c54x-pont2/mobile.log`. Over this path the mobile does its location update and holds a call (`call 1 600` from the mobile's VTY, echo test audible both ways).

2.43.1.4 4. Profiles

The layer-1 flag is independent of the **profile**, which chooses how many base stations come up and how:

profile	stack
`faketrx-qemu` (default)	core + BTS#0 QEMU + BTS#1 faketrx (alias `hybrid`)
`faketrx`	core + `fake_trx` + `trxcon` + `mobile`
`qemu`	the Calypso QEMU pipeline alone (alias `calypso`)
`noproc`	core only (alias `core`)
`virtphy`	core + osmo-bts-virtual + virtphy + mobile
`hw`	physical SDR
`virtual`	SS7 multi-operator via `ip netns`

```
./start-native.sh faketrx
./start-native.sh --profile qemu
./start-native.sh --list        # show the plan without running it
./start-native.sh --dry-run     # walk through it with no side effects
```

2.43.1.5 5. Building the images

```
sudo ./build.sh               # build osmocom-nitb (apt-fast, compose v2, .deb cache)
sudo ./build.sh --no-cache    # recompile everything and rewrite the cache
sudo ./build.sh --arch=arm64  # cross-build through buildx + qemu-user-static
```

2.43.1.5.1 5.1 The build is a graph

`Dockerfile` is split into BuildKit stages, so independent compilations run **at the same time**:

```
                base   (apt, apt-fast, osmo-deb, configs)
                  |
       +----------+--------+---------+
    osmo-core    qemu      lte      node        <- IN PARALLEL
        |          |
   +----+-----+    |
  bb  grgsm-venv  l1
   +----+-----+---+
             |
        osmocom-nitb   (assembly)
              |
           debs       (FROM scratch: the .deb cache, for CI)
```

`bb`, `l1` and `grgsm-venv` depend on `osmo-core`: `osmocom-bb` links `mobile/trxcon` against libosmocore, `c54x_exe` asks `pkg-config` for `libosmocoding libosmocore`, and gr-gsm's CMake looks for both.

`--target osmocom-nitb` is **mandatory**: `debs` is the last stage in the file, so without it Docker builds an empty image. `build.sh` and `compose.yaml` pass it.

2.43.1.5.2 5.2 The `.deb` cache

Everything the image compiles (libosmocore, osmo-*, gapk, QEMU and its tree, osmocom-bb, the gr-gsm venv, the firmware) comes out as a `.deb` through `packaging/osmo-deb.sh` and stays **on the host**, in `/var/cache/osmo-debs` (`OSMO_DEB_CACHE` to move it):

```
sudo ./build.sh              # 1st build: everything compiles, .deb go to the cache
sudo ./build.sh              # rebuild: dpkg -i, nothing compiles
sudo ./build.sh --no-cache   # recompile everything, rewrite the cache
./compose.sh debs            # what is in the cache
```

Each package carries **both** the installed files and the `/opt/GSM` source tree, so a cached rebuild still gives a workshop you can read, patch and `make` in.

Standalone `.deb`, to install without git:

```
./packaging/build-debs.sh          # -> packaging/dist/
sudo dpkg -i packaging/dist/*.deb  # refuses to install over a git clone at the same path
```

2.43.1.5.3 5.3 Docker Compose

```
sudo ./compose.sh build [--no-cache] [--lite] [--stp]
sudo ./compose.sh up [--ms 2] [--phy faketrx]    # hub + osmo-operator-1, HLR populated
sudo ./compose.sh down | ps | logs | shell
```

2.43.1.5.4 5.4 ISO images

`sudo ./build-iso.sh` does **one** docker build (`osmocom-nitb`) and derives each rootfs from it: interstp, normal, lite (the normal one with the workshops removed), desktop. On the installed disk the accounts are the ones from the Calamares *Users* screen plus an unlocked root; LUKS is offered in the partitioning choices; a *Graphics drivers* page offers `nvidia-driver-610` when `lspci` sees a card. Details: Build, ISO.

2.43.1.6 6. Daily use

2.43.1.6.1 6.1 tmux

```
docker exec -ti osmo-operator-1 tmux attach
docker exec -ti osmo-operator-1 tmux -S /tmp/osmocom_tmux attach -t osmocom
```

key	window
`Ctrl-b 0`	faketrx
`Ctrl-b 1`	MS1 (trxcon + mobile)
`Ctrl-b 2`	Asterisk
`Ctrl-b 3`	SMSC (proto-smsc-daemon + relay)
`Ctrl-b w`	list all windows
`Ctrl-b d`	detach

`--no-attach` starts without attaching; `osmo-banc.service` uses it.

2.43.1.6.2 6.2 VTY

port	component
`4239`	OsmoSTP
`4242`	OsmoBSC
`4243`	OsmoMGW
`4254`	OsmoMSC
`4258`	OsmoHLR
`4247` / `4347`	MS#1 `mobile` (`--grgsm` / `--dsp`)

```
OsmoMSC# show subscriber all
OsmoMSC# subscriber msisdn 10001 sms sender msisdn 10002 send Hello

OsmoHLR# subscriber imsi 001010000000001 show
OsmoHLR# show gsup-clients
```

Subscribers are provisioned automatically when the stack comes up, by `qosmo/run_modules/21-abonnes-hlr.sh`, following the numbering plan; `--regen` re-applies it.

2.43.1.6.3 6.3 Ports on the wire

port	carries
`5700-5702` UDP	TRXD, between `osmo-bts-trx` and the bridge
`4729` UDP	GSMTAP, the Um interface
`4730` UDP	full SI set into the shunt (`opt-gsm/si_bridge.py`)
`4731` UDP	SCH BSIC/FN out of the gr-gsm receiver
`6702` UDP	DL bursts into the DSP BSP (`--dsp`)
`8080` TCP	the web dashboard (`osmo-egprs-web`)
`8081` TCP	the I/Q spectra (`fft-web/fft_web.py`)

2.43.1.6.4 6.4 Wireshark

Started automatically on the Docker bridge with `sctp or udp port 4729`.

```
gsmtap                  # radio traffic (Um)
gsm_map                 # MAP messages
m3ua / sccp             # SS7, if you run multi-operator
```

2.43.1.6.5 6.5 Logs

Every component writes under `LOG_DIR`, which is `RUN_DIR/logs`: `/run/user/<uid>/osmo-nitb/logs` when the launching user has an `XDG_RUNTIME_DIR`, `/tmp/osmo-nitb/logs` otherwise (see `environment/paths.env`). The directory is on tmpfs on purpose: tmux and the desktop panel follow the files live, and tmpfs is the only filesystem here where inotify works reliably. `run.sh --stop` archives the directory under `../archives/<timestamp>`, truncates oversized archives and prunes old ones.

`mobile.log` is the journal of osmocom-bb `mobile` (MS#1), written by qosmo's `run_modules/70-l2.sh` or, with `--dsp`, by `c54x_exe/run.sh` under `/tmp/c54x-pont`. Its format is fixed by the `log stderr` block that `start-native.sh` guarantees in every generated `mobile.cfg`: colour per subsystem, category prefix, the compact 17-digit timestamp (`logging print extended-timestamp 1`, `YYYYMMDDhhmmssnnn`), and the seven categories the bench follows in debug (mm, rr, cc, sms, plmn, sim, sap). Readers strip the ANSI codes and the timestamp themselves: the desktop panel (`tools/osmo-fft-snap.py`), `qosmo/tools/rapport-run.sh`, `scripts/banc-repro.sh`, `/opt/GSM/tests/banc-max.sh`.

`start-native.sh` publishes at start and removes at stop:

path	holds
`/run/osmo-banc/ms1-vty`	the VTY port of MS#1 (4247, 4347 with `--dsp`)
`/run/osmo-banc/mobile-log`	the absolute path of `mobile.log`

Both are also pushed into systemd's environment (`OSMO_BB1_VTY_PORT`, `OSMO_MOBILE_LOG`). Setting `OSMO_MOBILE_LOG` by hand overrides the published path for the panel.

2.43.1.7 7. Diagnosing the radio side

Read `pont/README.md` before chasing a radio fault through a CRC counter. The causes documented there (overwritten Kc, the SACCH L1 header, C0 filling, an inert `CALYPSO_CANNED`) *all* present themselves as "the bridge has CRC errors".

```
./start-native.sh --check-paths     # verify the declared dependencies
./start-native.sh --verbose         # show the modules' output
```

Logs (`$LOG_DIR`, see 6.5; `/var/log/osmocom` holds the SMSC and side-car logs):

```
$LOG_DIR/qemu.log        # ARM/DSP emulation, BSP, IRQ
$LOG_DIR/bridge.log      # QEMU ticks, IND CLOCK, DL/UL bursts
$LOG_DIR/run.sh.log      # orchestration
```

symptom	likely cause	check
`bridge: timeout`	QEMU never started the TPU/DSP	`grep TINT0 $LOG_DIR/qemu.log`
`FBSB result=255` (no cell)	layer 1 is not detecting the FB	`grep "IMR change" $LOG_DIR/qemu.log | wc -l`
`osmo-bts-trx: PC clock skew too high`	the bridge stopped sending `IND CLOCK`	restart the bridge
`/tmp/osmocom_l2` never created	the firmware is not booting	`head -50 $LOG_DIR/qemu.log`
Address already in use on 5700	two bridges: a `--dsp` run left one behind	`./start-native.sh --stop`

QEMU monitor:

```
docker exec -ti osmo-operator-1 socat - unix-connect:/tmp/qemu-calypso-mon.sock
```

`--assembly-logs` gives a trace of the emulated ARM (`-d in_asm`: each block once, when it is translated). `ASSEMBLY_LOGS_FLAGS=in_asm,exec,nochain` also logs every execution; that makes the ARM ~150 times slower. The bridge clock then follows the DSP down to `PONT_HORLOGE_MAX` times real time (4 by default, 16 when `ASSEMBLY_LOGS=1`, see `pont/trx.py`; the trace itself goes to `/var/tmp/c54x-pont`, not the tmpfs), but the wall-clock timers of `mobile` (T3210) and of the core network (T3101) still run, so the attach may time out: use it to chase the firmware, not to measure the network.

2.43.1.8 8. The qosmo launcher

`40-qemu.sh` does not call `qemu-system-arm` directly: it calls the C launcher `qosmo` (`qosmo/tools/qosmo-launch`), installed next to it by `ninja install`. The launcher knows which layer 1 the QEMU it starts was built with, because `QOSMO_DSP` follows the `configure` option.

```
qosmo -k /opt/GSM/firmware/board/compal_e88/layer1.highram.elf
qosmo --help
```

Defaults: `-M calypso`, `-cpu arm946`, `-gdb tcp::1234`, `-serial pty -serial pty`, a unix monitor, L1CTL on `/tmp/osmocom_l2`, TRXDv0 on `0.0.0.0:6702`. It reads `l1s/last_rach` out of the ELF (no `nm` needed), relays QEMU's output unchanged, and publishes stable links to the ptys under `<RUN_DIR>/`.

`start-native.sh` resolves it (`--launcher <bin>`, or `QOSMO_LAUNCHER`), builds it if the source is there and the binary is not, and passes it to `run.sh`. Without a launcher, `40-qemu.sh` falls back to the plain `qemu-system-arm` command line.

2.43.1.9 9. One PLMN

Every simulated operator is the standard Osmocom stack, all components in a single container talking over `127.0.0.1`.

```
flowchart LR
    UE --> BTS
    BTS -->|Abis/IP| BSC
    BSC -->|SCCP via STP| MSC
    MSC -->|GSUP| HLR
    MSC -->|MGCP| MGW
    MSC -->|MNCC| Asterisk
```

The local STP is the intra-PLMN signalling hub. It listens on `127.0.0.1:2905` immediately, without waiting for the Docker interface.

2.43.1.9.1 9.1 SMS

```
sequenceDiagram
    participant MS_A as MS (sender)
    participant MSC as MSC (sms-over-gsup)
    participant HLR as HLR
    participant SMSC as proto-smsc-daemon
    participant MS_B as MS (recipient)

    MS_A->>MSC: SMS MO
    MSC->>HLR: GSUP MO-forwardSM
    HLR->>SMSC: GSUP MO-forwardSM
    SMSC->>HLR: GSUP MT-forwardSM
    HLR->>MSC: GSUP MT-forwardSM
    MSC->>MS_B: SMS MT
```

2.43.1.9.2 9.2 Voice

```
MS → BTS → BSC → MSC → MNCC → Asterisk → MNCC → MSC → BSC → BTS → MS
                                  ↕ MGCP
                               OsmoMGW (RTP)
```

2.43.1.9.3 9.3 Numbering

number	use
`N0001…N9999`	GSM subscribers of operator N
`100` / `200`	local softphones
`600`	echo test
`9XXXXX`	inter-operator out from a softphone

The first digit of `NXXXX` identifies the operator. The `[gsm_in]` dialplan (`configs/extensions.conf`) works out whether the destination is local or inter-op.

2.43.1.10 10. PHY modes

For the MS side, selected through `PHY_MODE` inside the container before calling `run.sh`:

`PHY_MODE`	stack	use
`faketrx` (default)	`fake_trx` → `trxcon` → `mobile`	multi-MS, fast, no DSP
`virtphy`	`osmo-bts-virtual` ↔ `virtphy` ↔ `mobile`	multi-MS over UDP multicast
`qemu`	`osmo-bts-trx` ↔ bridge ↔ Calypso QEMU ↔ `mobile`	emulated baseband, 1 MS

`PHY_MODE=qemu` forces `N_MS=1`: the bridge uses fixed UDP ports, so one emulated Calypso per container. For several virtual handsets, run several containers.

2.43.1.11 11. Multi-operator and SS7

The bench also runs **N operators (1 to 9)**, each with its own core network in its own container, joined by a central Inter-STP that routes M3UA/SCCP between them. No configuration file is edited by hand: pjsip, dialplan, SMS routing and the inter-STP config are all generated from N.

```
sudo ./launch/start-docker.sh          # choose bridge mode, enter N operators
sudo ./launch/start-docker.sh stop
```

2.43.1.11.1 11.1 Topology

```mermaid
flowchart LR
    subgraph "osmo-operator-1"
        BSC_A["BSC 1.23.3"]
        MSC_A["MSC 1.23.1"]
        STP_A["STP 1.23.2"]
        BSC_A -->|127.0.0.1:2905| STP_A
        MSC_A -->|127.0.0.1:2905| STP_A
    end
    subgraph "osmo-operator-2"
        BSC_B["BSC 2.23.3"]
        MSC_B["MSC 2.23.1"]
        STP_B["STP 2.23.2"]
        BSC_B -->|127.0.0.1:2905| STP_B
        MSC_B -->|127.0.0.1:2905| STP_B
    end
    subgraph "osmo-inter-stp"
        INTER["Inter-STP 0.23.0"]
    end
    STP_A <-->|"RCTX 150 / :2908"| INTER
    STP_B <-->|"RCTX 250 / :2908"| INTER
```

2.43.1.11.2 11.2 Addressing

Docker network	range	role
`gsm-inter`	`172.20.0.0/24`	interop backbone (inter-STP M3UA)
`gsm-net-opN`	`172.20.N.0/24`	operator N private network

Operator N → backbone `172.20.0.(10+N)`, private `172.20.N.10`. The Inter-STP is at `172.20.0.10`.

2.43.1.11.3 11.3 Point codes and routing contexts

ITU 14-bit, `zone.network.node`:

node	point code	RCTX	formula
Inter-STP	`0.23.0`	`rctx_msc`	`N×100+10`
MSC OpN	`N.23.1`	`rctx_stp`	`N×100+20`
STP OpN	`N.23.2`	`rctx_bsc`	`N×100+30`
BSC OpN	`N.23.3`	**`rctx_inter`**	**`N×100+50`**

`rctx_inter` must match in the operator's `osmo-stp.cfg` and be consistent with the connection to the inter-STP. The inter-STP uses **AS without a routing key** (catch-all per operator); routing is on the DPC alone.

2.43.1.11.4 11.4 Message path

```mermaid
flowchart LR
    MSC_A["MSC Op1<br/>1.23.1"] -->|"SCTP 127.0.0.1:2905"| STP_A["STP Op1<br/>1.23.2"]
    STP_A -->|"catch-all → as-inter<br/>172.20.0.10:2908"| INTER["Inter-STP<br/>0.23.0"]
    INTER -->|"DPC 2.23.x → as-op2"| STP_B["STP Op2<br/>2.23.2"]
    STP_B -->|"DPC 2.23.1 → dyn"| MSC_B["MSC Op2<br/>2.23.1"]
```

2.43.1.11.5 11.5 Generated configuration

`apply_config_templates()` in `launch/start-docker.sh` resolves every N-dependent placeholder in a **single `sed` pass** (`__PC_MSC__`, `__RCTX_INTER__`, `__ARFCN__` = `512+N×2`, `__CONTAINER_IP__`, …), then appends the sections that depend on the *total* number of operators: N-1 PJSIP trunks, the `[interop_out]` dialplan, the SMS routing table, and N AS + N×3 SS7 routes in the inter-STP config.

Startup order matters: the Inter-STP must be listening on `:2908` **before** the operator STPs try to connect.

2.43.1.11.6 11.6 Inter-operator SMS and voice

`scripts/sms-interop-relay.py` watches the `proto-smsc-daemon` MO log, parses the GSM 03.40 TPDU, looks up `sms-routing.conf` (longest-prefix match) and forwards over TCP 7890 to the target operator's relay, which resolves MSISDN→IMSI on the HLR VTY and calls `proto-smsc-sendmt`.

Voice crosses on a SIP trunk between the operators' Asterisk instances. The `[interop_out]` dialplan routes on the first digit of the dialled number.

2.43.1.11.7 11.7 Diagnosing PROHIB routes

```
sudo docker ps | grep inter-stp
sudo docker exec osmo-operator-1 sh -c 'echo "show cs7 instance 0 asp"   | telnet 127.0.0.1 4239'
sudo docker exec osmo-inter-stp  sh -c 'echo "show cs7 instance 0 asp"   | telnet 127.0.0.1 4239'
sudo docker exec osmo-operator-1 sh -c 'echo "show cs7 instance 0 route" | telnet 127.0.0.1 4239'
sudo docker exec osmo-operator-1 ping -c1 172.20.0.10
```

A healthy operator STP route table:

```
N.23.1/14   as-dyn-…  avail  avail  dyn   ← MSC (dynamic)
N.23.3/14   as-dyn-…  avail  avail  dyn   ← BSC (dynamic)
0.0.0/0     as-inter  avail  avail        ← catch-all to the inter-STP
```

symptom	likely cause	fix
route `0.0.0/0` PROHIB	inter-STP down, or wrong IP	check `INTER_STP_IP` and the backbone IP
no dynamic routes	MSC/BSC not reaching the local STP	check the ASPs point at `127.0.0.1`
ASP DOWN on the inter-STP	Docker race condition	`docker restart` the operator

2.43.1.11.8 11.8 WAN, mesh and multi-machine

Beyond a single host, `start-native.sh` also meshes several machines (procedure: WAN):

```
./start-native.sh --wan                  # interactive: N nodes, IPs, prefixes
./start-native.sh --wan mynode1.conf      # from other nodes' cards
./start-native.sh --wan-nodes "1:IP:IND …" # scriptable
./start-native.sh --gen-conf              # write this node's card, start nothing
./start-native.sh --virtualbox=3          # mesh with VirtualBox VMs
./start-native.sh --air-mesh              # mesh the BURSTS: a shared radio medium,
                                          # a mobile can hear and pick another node's BTS
```

`--node N` picks the node number **at launch**, not at build time, so one ISO serves all nine nodes.

2.43.1.12 12. Other extensions

- **4G (srsRAN + Open5GS)**: shipped in `/opt/LTE`, launched with `osmo-lte` and `osmo-epc`. CSFB to the 2G core through SGs.
- **Web dashboard**: `osmo-egprs-web` on `:8080`, started by `systemctl restart osmo-egprs-web`.
- **postmarketOS handset UI**: a patched `pmbootstrap` in `/opt/user_interface/pmos`.
- **SIM in a PC/SC reader or software SIM**: `--sap`, see SIM.

2.43.1.13 13. Related documents

document	subject
pont/README.md	A5 ciphering, Kc and SACCH: the measurement report behind the bridge's CRC counters
wiki/	Home, Build, Environment, Start-native, Results, SIM, WAN: using the bench day to day
environment/README.md	environment variables and path resolution
services/README.md	the systemd units shipped
navigation/QUICKSTART.md	getting started
qosmo/README.md	the QEMU fork
c54x_exe/README.md	the DSP emulator and its state

2.43.1.14 Known issues

- **DSP chain (`--dsp`)**: every TCH/F downlink frame is flagged BFI even though speech stays intelligible; SACCH occasionally drops a call; the SB window is rarely armed, so sync may take several tries; the DSP needs 4.3–4.6 ms of host time per 4.62 ms frame.
- **TRX mode (`--bts`)**: the SCH encoded by the DSP extension code comes out half wrong on `c54x_exe` (a C54x instruction of that loop is emulated wrongly, still to find); `c54x_exe` re-encodes the SCH before publishing it as a workaround (`MONTANT_ROM_TX_SB_RECODE`). The extcode's normal-burst demodulator likewise yields nothing on `c54x_exe`; the BSP writes the exact uplink bits into the result page instead (`CALYPSO_BSP_TRX_UL`). The trx firmware prints `DSP Error Status: 24` (DMA_PROG|DMA_TASK) every frame; harmless so far. SMS and calls between two Calypsos over `pont/air.py` beyond the echo test are not exercised.
- `osmo-bts-trx` exits whenever its transceiver disappears ("No more clock from transceiver"), i.e. after every `banc-stop`; `c54x_exe/run.sh` relaunches it at step 4b.
- `--assembly-logs` with `exec` tracing makes the attach time out on the network side (wall-clock timers keep running).

2.44 /opt/GSM/osmo-operator/wiki/Resultats.md

4534 octets, 80 lignes

2.44.1 Bench checks — `checks/check_all.sh`

`checks/check_all.sh` runs every read-only check of `checks/` in the right order, on the right node, and returns one verdict: a table with one line per check, a pass/warn/fail count and a return code. It modifies nothing and restarts nothing (only `--dump` raises the daemons' log level, and it is never on by default). This page describes what it runs and what a healthy bench looks like.

2.44.1.1 1. Usage

```
sudo ./checks/check_all.sh                          # ~2 min on a single node, ~6 min with three operators
./checks/check_all.sh --only=interstp --verbose     # one check, in detail
./checks/check_all.sh --dump                        # + vty-debug-dump and operator_summary
./checks/check_all.sh --list                        # the checks and whether each one applies here
```

option	effect
`--quick`, `-q`	passed to `global_check` / `ss7_check` (fast pass)
`--verbose`, `-v`	passed to the checks that understand it
`--op=N`	limit to one operator (`global_check`, `annuaire`)
`--docker` / `--native`	force the world to look at; auto-detected otherwise
`--dump`	add `vty-debug-dump.sh` then `operator_summary.sh`
`--only=a,b`	run only these checks
`--out FILE`	full log (default `/tmp/osmo-check-all-<date>.txt`)

2.44.1.2 2. Which checks run

check	script	runs when
`global`	`global_check.sh`	always: daemons, VTYs, BTS, PCU, SGSN, SIP connector, SMS relay
`ss7`	`ss7_check.sh`	always: ASP/AS state, routes, hub reachability, connectivity matrix
`interstp`	`diag-interstp.sh`	only if an inter-STP is queryable here (container `osmo-inter-stp`, or `OSMO_ROLE=interstp`)
`operator`	`diag-stp-operator.sh`	only when this node is not the hub and has `/etc/osmo-role` (a WAN setup)
`wan`	`wan_ss7_check.sh`	only if `/etc/osmo-wan.conf` exists
`annuaire`	`annuaire.sh`	only if subscriber directory entries exist (`/var/tmp` or `/etc/osmocom`)
`dump`, `resume`	`vty-debug-dump.sh`, `operator_summary.sh`	only with `--dump`

Verdicts: `global_check.sh` / `ss7_check.sh` return the number of failures, `wan_ss7_check.sh` returns 0 or 1, `diag-*.sh` and `annuaire.sh` always return 0 and their verdict is read from their last section (e.g. `INTERCO SS7 : DOWN`) with the colours stripped.

2.44.1.3 3. What a healthy run looks like

Single native node, no inter-STP: `global` OK (about 30 pass, 0 fail), `ss7` OK with a few skips, `interstp` skipped (no hub), `annuaire` skipped (no directory entry), `wan`/`dump`/`resume` skipped. `operator` reports `INTERCO SS7 : DOWN` (`asp-to-inter` in `shutdown`, `remote-ip 127.0.0.1`, no SCTP): expected, there is no inter-STP to reach.

Three operators (Op1 native + Op2/Op3 containers) + local inter-STP: `global` OK (about 90 pass), `ss7` OK with the full 3×3 connectivity matrix (each STP has 3 ASP / 3 AS active, the third being `as-inter`, and a default route to the inter-STP present and avail), `annuaire` OK (4 subscribers per operator, point codes `1.2.x` / `1.3.x`, RCTX 250 / 350, hub 172.20.0.10), `operator` skipped (this node is the hub).

Expected warnings:

warning	meaning	action
`GGSN: 0 APN configured`	2G data (SGSN → GGSN) cannot bring up a PDP context without an APN; 4G (Open5GS/ogstun) is not affected	`APN=internet` in `globals.conf`, then `--regen`
`SGSN: 0 NS entity` (container operators)	the containers' PCU did not open its NS-VC to the SGSN; no effect on voice/SMS/SS7	only matters if 2G data is wanted on those operators
subscribers without a directory entry	the HLR also knows IMSIs provisioned by another numbering scheme (bench restarted without `start-docker.sh`)	cosmetic

2.44.1.4 Known issues

- **`diag-interstp.sh` false negative on a hybrid node** (hub in the `osmo-inter-stp` container, operator 1 native on the same host): the check reports `interstp ECHEC – INTERCO SS7 : DOWN - aucun AS actif` and `[FAIL] service inactif` even though `ss7_check.sh` validates the interconnect end to end. The script queries `systemctl` for a hub unit that does not exist when the hub is a container launched by `launch/start-multi.sh`, and reads the VTY `127.0.0.1:4239`, which is operator 1's STP, so it sees `asp-to-inter` / `as-rkm-*` and no `as-opN`. TODO: target the container's VTY when `OSMO_ROLE=operator` and the hub is queryable here, and do not count the missing systemd unit as a FAIL when the hub is a container.

2.45 /opt/GSM/osmo-operator/wiki/SIM.md

6721 octets, 142 lignes

2.45.1 SIM: a real card in a PC/SC reader, or softSIM

The bench mobile (`mobile`, osmocom-bb) has no card reader: its SIM is either the **test SIM** from the config file (`sim test`, IMSI and Ki written in the clear, the default), or a SIM **served over a socket** by the SAP protocol (*SIM Access Profile*, `sim sap`). `start-native.sh --sap` sets up this second path with the SAP server from softSIM (Ruby), which can serve two things:

type	what the mobile sees	when
pcsc	the **real card** in the PC/SC reader (tested: sysmoISIM-SJA5 in a HID OMNIKEY 6121)	a card is in the reader
sim	the **software SIM** `sim-op1.xml` (IMSI of MS#1, COMP128v1 with the HLR Ki)	no card

`--sap` chooses on its own (pcsc if a card is present, otherwise sim); `--sap=pcsc` (`--omnikey`) and `--sap=sim` force it. Without `--sap` (or with `--no-sap`), nothing changes: test SIM.

2.45.1.1 1. Install

```
sudo tools/osmo-sim-install.sh        # or: sudo ./addition.sh --sim
```

Installs, without breaking anything already present: `pcscd` + `libccid` (the PC/SC daemon and the Omnikey's CCID driver), `pcsc-tools`/`opensc` (to see the reader and the ATR), Ruby + `ruby-libxml` + the `smartcard` gem (softSIM as a reader), Osmocom's **pySim** in `/opt/GSM/pysim-venv` (from its git: the "pySim" on PyPI is a different project), sysmocom's **sysmo-usim-tool** in `/opt/GSM/sysmo-usim-tool`, and softSIM itself in `/opt/GSM/softsim` with the patches from `tools/softsim/`. The ISO does the same (`iso_modules/51-softsim.sh` + `80-chroot.sh`, `--no-softsim` to skip it).

2.45.1.2 2. View the card

```
tools/osmo-sim.sh lecteur     # the readers, the ATR, the card type, ICCID, IMSI
tools/osmo-sim.sh lire        # everything (pySim-read): PLMN, SMSP, ACC, services…
tools/osmo-sim.sh shell       # interactive pySim-shell (ADM1 passed if known)
```

2.45.1.3 3. Program the card to the bench's plan

The bench provisions its subscribers in osmo-hlr with a fixed plan (`qosmo/run_modules/21-abonnes-hlr.sh`):

	MS#1 of operator 1
IMSI	`001 01 0001 000001` → `001010001000001`
Ki	`00112233445566778899aabbccdd` + MS + operator → `…dd0101`
algorithm	COMP128v1 (`aud2g comp128v1`)
MSISDN	node×100000 + operator×100 + MS → `100101`

For the card to register, it must carry **exactly** this identity:

```
tools/osmo-sim.sh programmer --dry-run          # shows, writes nothing
tools/osmo-sim.sh programmer --adm1 12345678    # writes (asks for confirmation)
tools/osmo-sim.sh programmer --ms 2 --op 1      # another MS of the plan
tools/osmo-sim.sh verifier                      # card = plan? HLR? ready for --sap?
```

Writing goes through **sysmo-usim-tool** (`/opt/GSM/sysmo-usim-tool/sysmo-isim-tool.sja5.py`): IMSI, MNC length, Ki (2G/3G/4G5G), **2G algorithm COMP128v1** (3G Milenage), OPc; then it reads back the algorithm and Ki (`-t -k`). It is the tool that knows how to set the algorithm: `pySim-prog` writes only the Ki, the card would stay in Milenage and the HLR would answer *authentication failed*. Without the tool (or with `OSMO_SIM_PYSIM_PROG=1`), the script falls back to `pySim-prog` (ICCID kept, IMSI, PLMN, name, SMSP, MSISDN, Ki) then `pySim-shell` for the algorithm byte `0x11` in `DF.SYSTEM/EF.SIM_AUTH_KEY` (6f20) and `ADF.USIM/EF.USIM_AUTH_KEY_2G` (af22).

2.45.1.3.1 The ADM1 key

Writing to a sysmoISIM requires the **ADM1 key of that card** (8 digits), provided by sysmocom with the card. **Three wrong tries block the card for good**: the script never guesses, it refuses without a key. Where it is looked for, in order:

1. `--adm1 K` on the command line;
2. `OSMO_SIM_ADM1=K` in the environment;
3. `/etc/osmocom/sim-adm1` (`chmod 600`): one line, or `ICCID key` per line for several cards;
4. `~/.config/osmo/sim-adm1`.

2.45.1.3.2 A card you don't want to reprogram

It keeps its IMSI and Ki: declare them to the HLR instead.

```
tools/osmo-sim.sh hlr --creer --ki <card's Ki>     # subscriber created via the VTY (4258)
```

(`hlr` without `--creer` only says whether the HLR knows the IMSI read on the card.)

2.45.1.4 4. Launch

```
./start-native.sh --sap            # card present → pcsc; otherwise software SIM
./start-native.sh --sap=pcsc       # the card, mandatory
./start-native.sh --dry-run --sap  # what would be done, without launching anything
```

`start_sap` runs **before** the stack: it launches `demo_server.rb` (softSIM) on `/tmp/osmocom_sap` (`SAP_SOCK`), with log `/tmp/osmo-sap.log` (`SAP_LOG`, `SAP_VERBOSITY=3`: connection, card ready, each timestamped APDU), and the mobile connects to it at startup. The server serves the next client after a disconnect, so a baseband restart requires nothing.

In DSP mode (the default), the MS#1 that runs is the one from c54x_exe (`mobile_pont.cfg`, `sim test`): `--sap` derives `/root/.osmocom/bb/mobile_pont_sap.cfg` from it (`sim sap`, `sap-socket`) and passes it to the bench via `MOBILE_CFG`. In grgsm mode, it is `mobile.cfg` generated from `configs/mobile.cfg` (`sim sap` in the template, reset to `sim test` without `--sap`) that carries it; MS#2 stays on the test SIM.

`./start-native.sh --stop` also stops the SAP server; a start always relaunches it fresh (it is killed with the rest when the stack is stopped before startup).

Serve the card by hand, without the stack (to watch the APDUs):

```
tools/osmo-sim.sh sap              # foreground, Ctrl-C to stop
```

2.45.1.5 5. The patches

file	what
`tools/softsim/0001-simos-comp128v1-a38.patch`	software SIM: COMP128v1 with `SOFTSIM_KI` (any RAND from the HLR), READ RECORD, `--file` passed through, acceptance loop
`tools/softsim/0002-sap-serve-next-client.patch`	the server serves the next client instead of exiting
`tools/softsim/0003-sap-log-timestamps.patch`	APDUs named and timestamped (ms) in the log
`tools/softsim/0004-pcsc-disconnect-idempotent.patch`	pcsc mode: `disconnect` may be called twice per client (DISCONNECT_REQ then socket close) without crashing the server
`tools/softsim/ruby_compat.rb`	Ruby 3.2: `File.exists?`, `Fixnum`
`tools/softsim/comp128.rb`	COMP128v1 for the software SIM
`tools/softsim/make_sim_op1.py`	`sim-op1.xml`: IMSI, valid ICCID, PLMN, SPN from the demo `sim.xml`
`patches/osmocom-bb-plmn-a6-reinsert.patch`	mobile: PLMN search handles a SIM reinserted without RPLMN (SAP reconnection)
`patches/osmocom-bb-sim-plmn-from-bcd.patch`	mobile: SIM PLMNsel/FPLMN decoded with the right BCD function

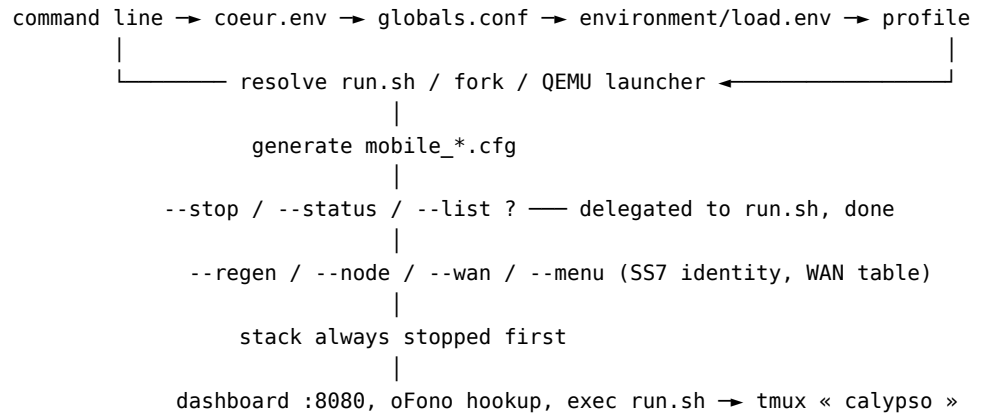
The two osmocom-bb patches are applied by `install_modules/40-patches.sh` (native) and the `Dockerfile`; `mobile` must be recompiled afterwards.

2.46 /opt/GSM/osmo-operator/wiki/Start-native.md

13649 octets, 285 lignes

2.46.1 `start-native.sh` — step by step

`start-native.sh` prepares, `run.sh` executes. The script itself is short: it handles `--docker` (hand-over to `launch/start-docker.sh`) and then sources `startup_scripts/*.sh` in order, in the same shell. This page follows those phases: what each one reads, decides and writes, and at what point a variable is taken into account or overwritten.



2.46.1.1 0. Where it runs

The script moves into its own directory (`HERE`). It is the **native** launcher (ISO, bare host, or inside a container); it does not check for Docker. Started by mistake on a Docker host without `/opt/GSM/qosmo`, it dies on « `run.sh` introuvable » (run.sh not found). On a host with Docker, use `./start-native.sh --docker …` or `./launch/start-docker.sh` directly.

2.46.1.2 1. Loading the environment

In this order, and **the order decides who wins** (details: Environnement.md):

1. **`/etc/osmocom/coeur.env`** (`OSMOCOM_CFG` to move it): written by `build-iso.sh`, carries `N_MS`. Loaded here, not by `~/.bashrc`, so that the desktop icon, systemd and `start-multi.sh` see the same value. The file uses `:=`: an `N_MS=3` set in the environment wins.
2. **Command-line options** (`--profile`, `--dsp`, `--node…`).
3. **`globals.conf`**: the reference for the network variables (MCC, MNC, OP_NAME, ENCRYPTION, ARFCN, BAND, LAC, APN, MS_COUNT, N_OPERATORS, HOST_IP…), **except those the caller has already set to a non-empty value**: `start-native.sh` records them, reads the file, then sets them again, and prints « globals.conf surchargé par l'environnement : … ». `ENCRYPTION="a5 0" ./start-native.sh --dsp` does give a5 0. An empty value overrides nothing (empty = computed, often from the operator number). To change a value: `vim globals.conf` or `./generate_configs.sh ARFCN=520`.
4. **`environment/load.env`**: `modes.env` (the profile), `paths.env` (where things are), then the per-domain files. All with `:=`: the first one to set a value wins, hence the command line.
5. Minimal fallback if `load.env` is missing (bare ISO, bare container): `GSM_ROOT=/opt/GSM`, `CALYPSO_FORK=qosmo`, resolution of `OQC_ROOT`.
6. Remaining defaults: `RUN_DIR=/run/osmo-direct`, `LOG_DIR=$RUN_DIR/logs` (tmpfs: tmux scrollback depends on it), `ENCRYPTION="a5 1"` (fallback: the real value comes from `globals.conf`), `CALYPSO_BRIDGE=pont` (`none` with `--dsp`), `MS_COUNT=2`, `HOST_IP=127.0.0.1`.

2.46.1.3 2. Profile

Default: **`faketrx-qemu`** (alias `hybrid`): core + BTS#0 QEMU + BTS#1 faketrx. `--profile <name>` or the positional mode sets it; `CALYPSO_PROFILE` or the legacy `MODE` set it from the environment (`MODE` wins if set). `--dsp` does not touch the profile. An explicitly named profile always wins.

Profile	What runs
`faketrx-qemu` / `hybrid`	core + osmo-bts-trx ← QEMU Calypso + fake_trx/trxcon/mobile
`faketrx`	core + fake_trx + trxcon + mobile; no QEMU
`virtphy`	core + osmo-bts-virtual + virtphy + mobile
`qemu` / `calypso`	the Calypso pipeline alone, core in no-process
`core` / `noproc`	STP + HLR + MSC + MGW + BSC (+ Asterisk), no radio
`hw`	physical SDR
`virtual`	SS7 multi-operator via `ip netns` (`N_OPERATORS`)

2.46.1.4 3. Resolving `run.sh` and the QEMU launcher

Order: explicit `RUN_SH` > resolved `OQC_ROOT` > `$GSM_ROOT/<fork>/run.sh`. `OQC_ROOT` is looked up **next to the repository** (`../qosmo`) then under `/opt/GSM`. If `run.sh` is not executable: FAIL, with the path tried.

The C launcher (`--launcher <bin>`, `QOSMO_LAUNCHER`, default `/usr/local/bin/<fork>`) is compiled if missing and `<fork>/tools/qosmo-launch` is present. Without a launcher, `40-qemu.sh` falls back to the plain `qemu-system-arm` command line.

2.46.1.5 4. Path validation

`--check-paths` stops here after checking every declared dependency (Osmocom binaries, `layer1.highram.elf` firmware, `calypso_dsp.txt` ROM, bridge).

2.46.1.6 5. Generating `mobile_*.cfg`

`mobile_group1.cfg` (MS#1, behind QEMU) and MS#2 (faketrx) are written to `$OQC_ROOT/cfgs/` from the templates, with IMSI/Ki/IMEI from `globals.conf` (or computed per operator). The layer 2 client is `mobile` by default; `--ccch_scan`, `--bcch_scan`, `--cell_log` **replace** it (they take its L1CTL socket and its VTY: mutually exclusive).

2.46.1.7 6. Summary

The plan is printed: profile, fork, `run.sh`, launcher, bridge, number of MSs, ciphering, ARFCN. `--list` stops there (delegated to `run.sh --list`); `--dry-run` continues without side effects.

2.46.1.8 7. Delegated actions

`--stop` and `--status` are passed to `run.sh` and the script exits.

2.46.1.8.1 7-menu. `--menu`

Asks **once** for the choices the script would otherwise have guessed: profile, node, operator, inter-STP, WAN, regeneration. The proposed values are those of the automatic launch: accepting without changing anything = not passing the option. Ignored without a terminal.

2.46.1.8.2 7-regen. `--regen`

By default the existing configs are **kept** (regenerating them at startup would erase the SS7 identity set just before). `--regen` stops the stack, then rewrites everything from the templates (current node and operator).

2.46.1.8.3 7-node. `--node N` / `--op N` / `--hub-ip`

The node number (1 to 9) is **deduced**: environment, then `/etc/osmo-role`, then the WAN table compared against the local addresses. `--node` forces it. The script rewrites the SS7 identity (point codes `1.<node><op>.<role>`, routing contexts) and points the ASP at the inter-STP (`--hub-ip`, otherwise deduced: docker or VM). This is what makes a single ISO possible for all nine nodes.

2.46.1.8.4 7-wan. `--wan`

Never on by default. A `--wan` on the command line applies the WAN **for that run only**: the table it writes keeps `WAN_AUTO=0`, so the next plain start has no WAN. Only an ISO built with `--wan` sets `WAN_AUTO=1` in `/etc/osmo-wan.conf`, which brings the WAN up at every start (and stays 1 when the table is rewritten). To make a machine a permanent node by hand, set `WAN_AUTO=1` in that file; to stop it, set `WAN_AUTO=0`. Three forms:

```
./start-native.sh --wan                                  # questions: nodes, IPs, prefixes, my number
./start-native.sh --wan-nodes "1:IP:IND 2:IP:IND" --wan-id 2   # scriptable
./start-native.sh --wan mynode1.conf --wan mynode3.conf  # from the other nodes' cards
./start-native.sh --gen-conf [FILE]                      # writes MY card, launches nothing
```

`--regen-conf` rebuilds the table from scratch (the old one is backed up next to it): use it when a powered-off peer leaves a PROHIB route on the STP. Full procedure for two nodes, LAN or Internet: WAN.

The table (`--wan-conf`, default `/etc/osmo-wan.conf`) is applied, the SS7 addressing towards the inter-STP set up, the SIP + SMS mesh generated. Dialing `<prefix><number>` reaches that number on the matching node. The profile stays `faketrx-qemu`.

`--air-mesh[=PORT]` additionally meshes the **bursts**: the radio medium becomes shared, a mobile can hear and pick another node's BTS (peers from the WAN table, range in `data/air-mesh.txt`, two extra mobiles, `pont/airmesh.py`).

`--virtualbox[=N]` sets up the segment and N−1 VMs before the WAN; refuses to run **inside** a VM (`--vbox-node N` = this machine's number).

2.46.1.8.5 7-tcpdump

A tcpdump wrapper is installed for GSMTAP (`udp/4729`) and M3UA captures.

2.46.1.8.6 7-stop

The stack is always stopped before starting (`run.sh --stop`). There is no hot restart: `--force` only restarts modules that are already running instead of skipping them.

2.46.1.9 8. Launch

1. The node's private addresses are printed.
2. **Dashboard** on `:8080` (Firefox opens it from `launch.sh`); the I/Q spectra on `:8081` (`fft-web/fft_web.py`).
3. **oFono mobile hookup**: the physical phone (postmarketOS, PPP) or the Android QEMU sees the bench as a modem.
4. **Audio (slimmed-down PulseAudio)**: runs `scripts/audio-chain.sh 5` (log `/tmp/osmo-audio-chain.log`): stops speech-dispatcher, switches `osmo_rec` to 8 kHz mono, loads `module-stream-restore` without `restore_device`. `AUDIO_ALLEGER=0`: don't slim down (the chain is set up anyway). `AUDIO_AEC_METHOD`: `webrtc` by default (`speex` possible). `AUDIO_PENDANT_APPEL=1`: the heavy chain is only set up between gapk-auto's « Endpoint actif » and « Endpoint disparu » (default 0). `audio-chain.sh --reinit`: start again from a clean chain without restarting PulseAudio.
5. **Hand-off to `run.sh`**: `exec`, with the profile and the whole environment. Any `CALYPSO_*` variable passed as a prefix reaches `run.sh`, its `run_modules/` and QEMU **untouched** (`CALYPSO_LANG=en`, `CALYPSO_NO_ATTACH=1`, `CALYPSO_BRIDGE=none`…).

`run.sh` opens the `calypso` tmux: `osmo-bts-trx` → QEMU (PTY + monitor) → wait for `/tmp/osmocom_l2` → bridge → wait for UDP 6700 ticks → `mobile`, then the core, Asterisk, SMSC. Unless `--no-attach` / `CALYPSO_NO_ATTACH=1` (which `osmo-banc.service` sets), the script attaches to the tmux.

2.46.1.9.1 8-dsp. The DSP bench

With `--dsp` (the default), `RUN_SH` stays the fork's one and receives `--skip qemu,pty,osmocon,l2 --no-attach`. Then `exec c54x_exe/run.sh` (`BANC_DSP`, if executable) takes over with `MODE=dsp PONT=1`, `IQ=none`, `INSNS=120000`, `LOCKSTEP=1`, `CALYPSO_BSP_STREAM=1`, set with `:=` so they can be overridden. The five steps:

1. `c54x_exe --arm`: the DSP, socket `/tmp/calypso_dsp.sock`, shared API RAM, DL bursts on `udp/6702`;
2. `qemu-system-arm` from `/opt/GSM/qosmo` (`CALYPSO_DSP_EXTERN=1`): the ARM alone;
3. `osmocon`;
4. `mobile` (`mobile_pont.cfg`, VTY **4347**);
5. `pont/pont_dsp.py`.

`CALYPSO_BRIDGE` is `none`: it is not `start-native.sh` that starts the bridge. `--stop`, `--regen` and the stop before starting also stop the DSP bench, even without `--dsp`; `--status` and `--check-paths` show it (with `--dsp`). The summary and oFono target VTY 4347 (`OSMO_MOB_VTY_PORT`). `BANC_DSP=none`: the stack alone.

`--bts` (alias `--osmo-trx`) implies `--dsp` and runs `c54x_exe/run.sh` with `OSMO_TRX=1`: the Calypso becomes BTS#0's transceiver; `--ms` adds the second emulated phone and the air relay, `--ts 1|2` the timeslots served. See Reference § 3.4-3.6.

Logs: the real files are at the dashboard paths (`/run/user/0/osmo-nitb/logs/{qemu,osmocon,mobile}.log`, `/dev/shm/pont.log`), with links in `/tmp/c54x-pont`. `--assembly-logs` (`ASSEMBLY_LOGS=1`, `ASSEMBLY_LOGS_FLAGS`, `ASSEMBLY_LOGS_FILTRE`) writes `/tmp/c54x-pont/qemu-asm.log`, but the bench then loses real time.

2.46.1.10 Short manual

```
./start-native.sh [options] [mode]

Modes: faketrx-qemu (default) | faketrx | qemu | noproc | core | hybrid

Stack selection
  --dsp                (default) C54x DSP bench: the C54x runs outside QEMU (/opt/GSM/c54x_exe,
                       TI mask ROM) and decodes FCCH/SCH/BCCH itself; same profile and same
                       stack as grgsm, only qemu,pty,osmocon,l2 move to c54x_exe/run.sh
                       (BANC_DSP=none: the stack alone)
  --grgsm              gr-gsm layer 1 inside qosmo
  --bts | --osmo-trx   the Calypso as BTS#0's transceiver (implies --dsp)
  --ms                 with --bts: a second emulated phone as the mobile, over the air relay
  --ts N               with --bts: timeslots served (1 = TS0 only, 2 = TS0+TS1)
  --assembly-logs      ARM asm trace (/tmp/c54x-pont/qemu-asm.log); no more real time
  --sap, --softsim     SIM served over SAP (softSIM): the card in the PC/SC reader
                       (HID Omnikey) if one is inserted, otherwise the software SIM;
                       --sap=pcsc / --sap=sim force it, --no-sap (default): test SIM.
                       See [SIM](SIM.md).
  --launcher <bin>     QEMU C launcher (default /usr/local/bin/<fork>)
  --profile <name>     forces the profile
  --mobile | --ccch_scan | --bcch_scan | --cell_log   layer 2 client (mutually exclusive)

Lifecycle
  --list  --dry-run  --stop  --status  --force  --verbose  --no-attach  --check-paths
  --regen              regenerates the configs from the templates (kept otherwise)
  --menu               asks the questions instead of guessing
  --docker             hand over to launch/start-docker.sh with the remaining arguments

Topology
  --node N  --op N  --hub-ip IP  --interstp | --no-interstp
  --wan | --wan CARD… | --wan-nodes "…" | --wan-id N | --wan-conf FILE | --gen-conf [F] | --regen-conf
  --air-mesh[=PORT]  --virtualbox[=N]  --vbox-node N
```

Getting back in:

```
tmux attach -t calypso          # Ctrl-b d detaches, the bench keeps running
journalctl -u osmo-banc -f      # if started as a service
./start-native.sh --status
./start-native.sh --stop
```

Logs: `$LOG_DIR` (`/run/osmo-direct/logs`, tmpfs): `qemu.log`, `bridge.log`, `run.sh.log`; with `--dsp`, `/tmp/c54x-pont/`.

2.47 /opt/GSM/osmo-operator/wiki/Telephone-emule.md

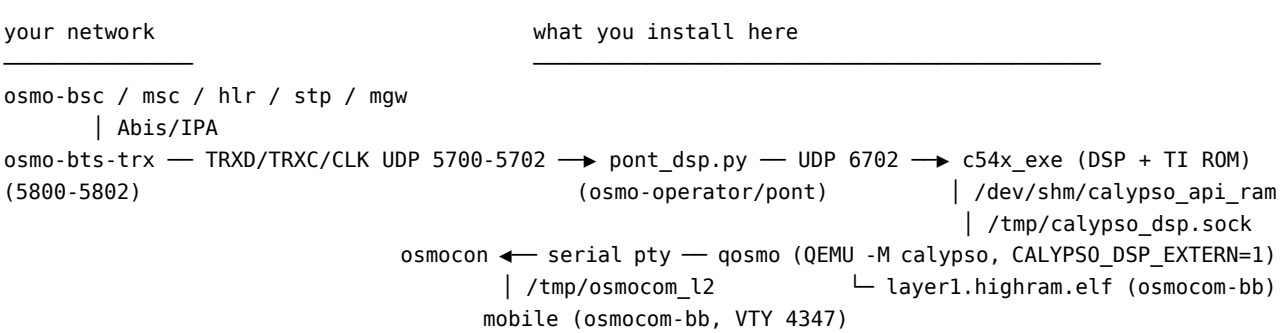
9430 octets, 157 lignes

2.47.1 Installing the emulated phone on your own Osmocom stack

This page is for anyone who **already** has an Osmocom network (osmo-bsc, osmo-msc, osmo-hlr, osmo-stp, osmo-mgw, installed from packages or from source) and wants to connect an **emulated Calypso phone** to it, without the ISO, without Docker and without the osmo-operator stack.

Page	Contents
Telephone-emule (here)	overview, requirements for **your** network, installation order
Installer-qosmo	QEMU, the Calypso machine (the ARM + osmocom-bb firmware)
Installer-c54x_exe	the C54x DSP outside QEMU, its TI ROM, the `pont_dsp.py` bridge
Installer-grgsm_exe	the gr-gsm setup, and what "grgsm_exe" really means
Lancer-a-la-main	the chain launched process by process, with commands unfolded
installer-osmo-jusquau-dsp.ipynb	notebook: the whole Dockerfile replayed natively, from apt down to a call in the DSP

2.47.1.1 What runs on the phone side



Two setups exist (`c54x_exe/LAUNCH.md`):

setup	layer 1	bridge	state
dsp (default)	`c54x_exe --arm`, real TI mask-ROM	`pont/pont_dsp.py --dsp-port 6702`	camps, LU, SMS MO/MT, MO/MT calls, A5/1; SB sync sometimes takes several tries
grgsm	gr-gsm compiled into QEMU	`pont/pont.py`	end to end, up to the voice call

2.47.1.2 What your network must provide

The phone is pinned to a specific cell and subscriber. Either your network conforms to it, or you change the values on the phone side (right-hand column).

Element	Expected	Where to change it on the phone side
BTS	**osmo-bts-trx** on the same machine (tested with osmo-bts 1.10.0)	—
TRX link	BTS config: `phy 0` / `osmotrx ip local 127.0.0.1` / `osmotrx ip remote 127.0.0.1`, **without** `base-port` (defaults 5800 local, 5700 remote); see `osmo-operator/configs/osmo-bts-trx.cfg`	`PONT_TRX_BIND`, `PONT_TRX_BASE` (`pont/config.py`)
Band, ARFCN	DCS1800; the bridge defaults to ARFCN **514** (`PONT_ARFCN`), the bench's `globals.conf` uses `ARFCN="1"`; the three (BSC, bridge, mobile) must agree	`PONT_ARFCN` + the `stick <ARFCN>` line of `c54x_exe/mobile_pont.cfg`
BSIC	**7**	`PONT_BSIC` (`pont/config.py`)
Timeslots of the BSC's first `bts`	TS0 `CCCH+SDCCH4`, TS1 `SDCCH8`, TS2-5 `TCH/F`, TS6-7 `PDCH` (`osmo-operator/configs/osmo-bsc.cfg`)	the bridge **reads** them from `PONT_BSC_CFG` (default `/etc/osmocom/osmo-bsc.cfg`, `pont/gsm.py`); if the file is unreadable, the table at left is assumed
PLMN	MCC 001, MNC 01	`rplmn 001 01` in `mobile_pont.cfg`
Subscriber	IMSI `001010001000001`, COMP128v1, Ki `00112233445566778899aabbccdd0101`	`test-sim` in `mobile_pont.cfg`

Provision the subscriber in osmo-hlr (same VTY commands as `qosmo/run_modules/21-abonnes-hlr.sh`):

```
telnet 127.0.0.1 4258
enable
subscriber imsi 001010001000001 create
subscriber imsi 001010001000001 update msisdn 100101
subscriber imsi 001010001000001 update aud2g comp128v1 ki 00112233445566778899aabbccdd0101
```

The MSISDN `100101` is the bench's (numbering plan in Home); use your own.

Ports and files the phone occupies on the machine (do not use them in your stack): UDP 5700-5702 (bridge), UDP 6702 (DSP), UDP 4730/4731 (gr-gsm L1, grgsm setup), TCP 4347 (mobile VTY), TCP 1234 (QEMU gdbstub), TCP 44444 (gdb console), `/tmp/osmocom_l2`, `/tmp/osmocom_sap`, `/tmp/ms_data`, `/tmp/calypso_dsp.sock`, `/tmp/qemu-monitor-pont.sock`, `/dev/shm/calypso_*`, `/dev/shm/pont.log`, `/tmp/c54x-pont/` (sources: `c54x_exe/run.sh`, `c54x_exe/mobile_pont.cfg`, `pont/config.py`).

2.47.1.3 Installation order

Everything goes under a common root, `/opt/GSM` by default (paths hard-coded in the Makefiles and `run.sh`, all overridable: `GSM_ROOT=`, `QOSMO=`, `FIRMWARE_ELF=`, `OSMOCON=`…).

Each repo carries its own installer (`install.sh`), with the same contract as `osmo-operator/install_modules/_lib/inst.sh`: `--check` (prerequisites, changes nothing), `--list`, `--deps` / `--print-deps` (apt packages), `--only` / `--skip`, a log per step, a post-check. These are **the same scripts** the Dockerfile (stages `qemu` and `l1`), `Dockerfile.run`, `start-docker.sh` and the ISO call: a single list of commands per component.

2.47.1.3.1 Component by component

1. **System dependencies**: each installer has its list: `./install.sh --print-deps`, `sudo ./install.sh --deps`.
2. **libosmocore** (with libosmocoding): you already have it if your stack is compiled; otherwise the `libosmocore-dev` packages from the Osmocom repo (the bench uses 1.12.1). `c54x_exe/install.sh --check` says right away whether pkg-config sees it.
3. **osmo-gapk**, then **osmocom-bb** (osmocon, mobile) and the prebuilt **firmware**: see Installer-qosmo. The bench's `patches/osmocom-bb-*.patch` are applied by `tools/osmo-patches-apply.sh` (`install_modules/40-patches.sh`); with the gnuarm toolchain in PATH (`sudo ./addition.sh --gnuarm`), the firmware rebuilds with `cd /opt/GSM/osmocom-bb/src && make` (`sudo ./addition.sh --calypso-fw`).
4. **qosmo**: `qosmo/install.sh`, Installer-qosmo.
5. **c54x_exe + DSP ROM + bridge**: `c54x_exe/install.sh`, Installer-c54x_exe.
6. (optional) **gr-gsm / grgsm_exe setup**: `grgsm_exe/install.sh`, Installer-grgsm_exe.
7. **Launch**: `PONT=1 ./run.sh` in `c54x_exe` (`c54x_exe/README.md`) or by hand, Lancer-a-la-main.

```
cd /opt/GSM
git clone https://github.com/bbaranoff/qosm0    qosmo     && sudo qosmo/install.sh --with-deps
git clone https://github.com/bbaranoff/c54x_exe c54x_exe  && sudo c54x_exe/install.sh --with-deps
git clone https://github.com/bbaranoff/grgsm_exE grgsm_exe && grgsm_exe/install.sh      # optional
git clone --depth 1 https://github.com/bbaranoff/firmware firmware
```

2.47.1.4 Proof that it works

In order (details in Lancer-a-la-main):

Step	Expected line	File
DSP ready	`pont : en attente de l'ARM sur /tmp/calypso_dsp.sock` (bridge: waiting for the ARM)	`/tmp/c54x-pont/dsp.log`
QEMU connected to the DSP	`[trx] pont DSP : API RAM partagee ...` (shared API RAM)	`/tmp/c54x-pont/qemu.log`
firmware loaded	`Received branch ack, your code is running now!`	`/tmp/c54x-pont/osmocon.log`
bridge ready	`pont TRX : ports 5700/5701/5702, ARFCN 514, BSIC 7`	`/tmp/c54x-pont/pont.log` (link to `/dev/shm/pont.log`)
BTS seen by the bridge	`BTS 127.0.0.1 : horloge armee vers le port 5800` (clock armed toward port 5800, `pont/trx.py`)	same
cell read	`New SYSTEM INFORMATION ...`, `lai=001-01-<LAC>`	`/tmp/c54x-pont/mobile.log`
camp	`C3 camped normally` in `show ms` (`/opt/GSM/tests/modules/60-camp.sh`)	VTY 4347
LU	`LOCATION UPDATING ACCEPT`; `MM idle, normal service`	`mobile.log` / `show ms`
call	`call 1 600` then `call control state: ACTIVE` (`/opt/GSM/tests/modules/_lib.sh`)	VTY 4347

`c54x_exe/run_real.sh` automates these checks against a stack launched by **systemd** (`systemctl start osmo-hlr osmo-stp osmo-msc osmo-mgw osmo-bsc`, `systemctl restart osmo-bts-trx`) and prints a verdict `VRAI : lai=… = la config du reseau` (TRUE: lai=… = the network config). It reads MCC/MNC from `/etc/osmocom/osmo-msc.cfg` and LAC/BSIC from `/etc/osmocom/osmo-bsc.cfg`: adapt if your configs are elsewhere.

2.47.1.5 Known issues and untested cases

- Compatibility with an osmo-bts-trx **other than 1.10.0** is untested (TRXD negotiation: the bridge answers `SETFORMAT` with an echo and only reads TRXD v0, `pont/trx.py`).
- The bench's `osmo-bts/hlr/msc-force-rand-toy` patches (`patches/`) are **not** needed by the phone (they make the RAND reproducible), but the phone has not been tested without them.

- An osmo-bts-trx **on a different machine** from the phone is not planned for: the bridge returns the clock to the source IP of the CTRL command (`pont/trx.py`); it might work with `PONT_TRX_BIND=0.0.0.0`.
- Audio during a call: `mobile_pont.cfg` uses ALSA `gsm_out`/`gsm_in`, defined by `osmo-operator/configs/asound.conf` (the `libasound2-plugins` plugin + PulseAudio). Without them, the mobile's behaviour during a call is unverified.

2.48 /opt/GSM/osmo-operator/wiki/WAN.md

10217 octets, 233 lignes

2.48.1 WAN — linking two nodes

Two machines, each running its own stack (`start-native.sh`, one operator per node), linked so that the mobile on node 1 can call and text the mobile on node 2. The method works the same on a LAN and over the Internet. Over the Internet it goes through a **WireGuard** tunnel (§2), which is the only part that changes.

2.48.1.1 1. What crosses between the nodes

Flow	Protocol / port	Who listens	Used for
SS7 (M3UA)	**SCTP 2908**	the inter-STP (the hub), one for the whole WAN	SS7 interconnect, point codes `1.<node><op>.<role>`
Voice signalling	UDP/TCP **5060** (single-operator node); 5080, 5082… (N-operator node)	Asterisk	PJSIP trunks between nodes
Voice media	UDP **20000–20499** (op 1)	Asterisk	RTP
SMS	TCP **7890** (op 1; 789x for op x)	`sms-interop-relay.py`	SMS between nodes

Numbering works like a country code. Node N has prefix `NN` (11, 22…), and a mobile's MSISDN is `<node>00<op><ms>`. The first mobile of node 2 is therefore `20011`. From node 1 you reach it by dialling `22` + `20011` = **`2220011`**. The sending node strips the prefix before going out on the WAN.

2.48.1.2 2. Why a tunnel over the Internet

Two nodes on the same LAN (or on the VirtualBox host-only segment) reach each other directly: skip to §4. Over the Internet, without a tunnel, four things break:

1. **SCTP does not cross the NAT** of a consumer router. The SS7 link to the hub never comes up.
2. The address in the WAN table must be **both** reachable by the peer **and** held by a local interface. The node uses it to recognise itself (`wan_nodes_detect_self`), and the SCTP ASP binds to it (`local-ip`). A NATed public IP cannot be both.
3. Asterisk's SDP announces `external_media_address` = the table address. Behind NAT, RTP goes to a private address.
4. Ports 2908/5060/789x/RTP would be exposed to the whole Internet.

With WireGuard, each node holds a private address that is **local and routable** (`10.99.0.N`). SCTP travels inside the tunnel's UDP, and the WAN case becomes the LAN case.

2.48.1.3 3. Step 1 — the WireGuard tunnel

Example plan:

Node	Machine	Tunnel address	Extra role
1	A	`10.99.0.1`	runs the inter-STP (co-located hub)
2	B	`10.99.0.2`	—

At least one side must be reachable on UDP 51820: public IP, or a port forward on the router. Here, that is A.

On each machine:

```
sudo apt install wireguard
wg genkey | sudo tee /etc/wireguard/priv | wg pubkey | sudo tee /etc/wireguard/pub
sudo chmod 600 /etc/wireguard/priv
```

`/etc/wireguard/wg0.conf` on **A**:

```
[Interface]
Address    = 10.99.0.1/24
ListenPort = 51820
PrivateKey = <contents of /etc/wireguard/priv on A>

[Peer]                      # node 2
PublicKey  = <pub of B>
AllowedIPs = 10.99.0.2/32
```

`/etc/wireguard/wg0.conf` on **B**:

```
[Interface]
Address    = 10.99.0.2/24
PrivateKey = <contents of /etc/wireguard/priv on B>

[Peer]                      # node 1 (+ hub)
PublicKey  = <pub of A>
Endpoint   = <public IP or DNS name of A>:51820
AllowedIPs = 10.99.0.1/32
PersistentKeepalive = 25    # keeps the NAT mapping open on B's side
```

```
sudo systemctl enable --now wg-quick@wg0
```

> **Do not use a `192.168.x` range** for the tunnel. With no WAN table yet, `wan_detect_local_ip` prefers the first `192.168.*` address, which is the LAN one. With `10.99.0.0/24` there is no ambiguity. The command in §5 also gives the addresses explicitly.

2.48.1.4 4. Step 2 — check the link and open the firewall

```
ping -c3 10.99.0.2          # from A
ping -c3 10.99.0.1          # from B
sudo wg show                # a recent "latest handshake" on both sides
```

Firewall. `setup-wan-mesh.sh` calls `network/firewall-wan.sh`, which opens **neither SCTP 2908** (the hub) **nor 5060** (direct SIP to a single-operator node: only 5080 and up are opened). With an `INPUT` policy of ACCEPT (the iptables default) this has no effect. With `ufw` active, SS7 and voice are dropped. On the tunnel, the simplest fix is to trust the interface, on **both** machines:

```
# ufw
sudo ufw allow in on wg0 from 10.99.0.0/24
# or iptables
sudo iptables -C INPUT -i wg0 -s 10.99.0.0/24 -j ACCEPT 2>/dev/null \
  || sudo iptables -I INPUT -i wg0 -s 10.99.0.0/24 -j ACCEPT
```

On a LAN without a tunnel, open the ports by hand, from the peer's IP: SCTP 2908 on the hub, then UDP/TCP 5060, UDP 20000–20499 and TCP 7890 on every node.

2.48.1.5 5. Step 3 — start the nodes (node 1 first)

Both nodes get the same table. Each one gets its own number, and all of them point at the same hub. This is the scriptable form, with no questions and no detection, and it is better than node cards for a first setup (see §7).

A — node 1, runs the hub:

```
sudo ./start-native.sh --regen-conf \
     --wan-nodes "1:10.99.0.1:11 2:10.99.0.2:22" --wan-id 1 --node 1 \
     --hub-ip 10.99.0.1 --interstp
```

B — node 2, once A shows `Inter-STP … OK`:

```
sudo ./start-native.sh --regen-conf \
     --wan-nodes "1:10.99.0.1:11 2:10.99.0.2:22" --wan-id 2 --node 2 \
     --hub-ip 10.99.0.1 --no-interstp
```

- `--wan-nodes` applies the WAN **for this run only**: the next plain `./start-native.sh` starts without it. Pass the same options again (or set `WAN_AUTO=1` in `/etc/osmo-wan.conf`) for the node to mesh at every start.
- `--regen-conf` starts from an empty table. The old one is saved as `/etc/osmo-wan.conf.avant-regen-<date>`. Without this option, a peer seen once and then switched off stays in the table and leaves a **PROHIB** route at the STP. On the very first start the option is not needed, but it is harmless.
- The hub is **co-located** on node 1's address (`--hub-ip 10.99.0.1`). `set_stp_ip.sh` supports this: node 1's ASP leaves from port 2910 towards the neighbouring hub on 2908. There is no extra address to add and no `AllowedIPs` to widen.
- Order matters. If B starts before the hub, its ASP waits and `start-native.sh` prints `hub 10.99.0.1 muet` (hub silent). Restart B once the hub is up.

What each start does, in order: write `/etc/osmo-wan.conf` → `setup-wan-mesh.sh` (PJSIP trunks, `[wan_out]`/`[wan_in]` dialplan, SMS routes, REDIRECT, firewall) → inter-STP (A only) → `set_stp_ip.sh` (point codes, ASP `asp-to-inter` → `10.99.0.1:2908`) → start the stack.

```
sequenceDiagram
    participant A as node 1 (A)
    participant HUB as inter-STP on A
    participant B as node 2 (B)
    A->>A: write /etc/osmo-wan.conf, setup-wan-mesh.sh
    A->>HUB: start-interstp.sh (listen 2908)
    A->>HUB: ASP asp-to-inter (from 2910)
    B->>B: write /etc/osmo-wan.conf, setup-wan-mesh.sh
    B->>HUB: ASP asp-to-inter, SCTP over wg0
    A-->>B: SIP qualify 5060, both ways
```

2.48.1.6 6. Step 4 — check, layer by layer

Layer	Command	Expected
Tunnel	`sudo wg show`	handshake less than 2 min old
SS7, from the hub (A)	`sudo checks/diag-interstp.sh`	both ASs `AS_ACTIVE`, no PROHIB/UNAVAIL route, no flapping
SS7, from a node	`sudo checks/ss7_check.sh`	`asp-to-inter` ACTIVE towards `10.99.0.1:2908`
SIP	`sudo asterisk -rx 'pjsip show endpoints'`	trunk to the peer `Avail` (qualify OK)
SMS	`cat /etc/osmocom/sms-routing.conf`	an entry for the remote node → `10.99.0.x:7890`
End to end	from mobile 1 on node 1, call `2220011`, then send an SMS to the same number	it rings and the SMS arrives on node 2; same the other way with `1110011`

Useful capture: `tcpdump -ni wg0 sctp or port 5060 or port 7890`. The bench's tcpdump wrapper also captures GSMTAP and M3UA.

Each step's logs are in `$LOG_DIR` (default `/tmp`): `wan-mesh.log`, `interstp.log`, `set-stp-ip.log`.

2.48.1.7 7. Variants

Node cards (`--gen-conf` / `--wan CARD`). Each node writes its card, the cards are exchanged, and each node imports the other's:

```
sudo ./start-native.sh --gen-conf mynode2.conf      # on B
sudo ./start-native.sh --wan mynode2.conf --node 1 --hub-ip 10.99.0.1   # on A
```

Before sending a card, check its `OSMO_NODE_IP` line. On a machine with no table yet, the detected address is the LAN one (`192.168.x`), not the tunnel's. Either fix the card by hand, or generate it **after** a first start with `--wan-nodes`: the table is authoritative once its address is up.

Same LAN, no tunnel. Same command with the LAN addresses (`--wan-nodes "1:192.168.1.10:11 2:192.168.1.20:22"`). `--hub-ip` is either node 1's address (co-located) or a free address in the /24, which `start-interstp.sh` will add.

More than two nodes. Add one entry per node to `--wan-nodes` (up to 9, prefixes 11 to 99), and one WireGuard `[Peer]` per node on A. With the hub on A, a star around A is enough for SS7. Voice and SMS go node to node, so every node must reach every other: either a full WireGuard mesh, or `AllowedIPs = 10.99.0.0/24` on the satellites with `net.ipv4.ip_forward=1` on A.

2.48.1.8 8. Common failures

Symptom	Cause	Fix
PROHIB route at the STP towards a node that is off	old entry in `/etc/osmo-wan.conf`	restart with `--regen-conf` **on the node that runs the hub**: a hub already running elsewhere is not rebuilt
`aucune IP locale dans la table - passez --wan-id N` (no local IP in the table)	the table address is not on any interface (public IP, tunnel down)	`--wan-id N --node N`, check `wg show`
`… repond au ping mais pas en SCTP 2908` (answers ping but not SCTP 2908)	the hub is not running on the machine that holds the address	start A first, or `sudo ./launch/start-interstp.sh --ip 10.99.0.1` on A
ASP going up and down (flapping in `diag-interstp.sh`)	firewall on 2908, or two hubs	§4; only one node with `--interstp`, the others `--no-interstp`
SIP trunk `Unavail`	5060 filtered (ufw), or wrong port targeted	§4; `pjsip show endpoints`, `tcpdump -ni wg0 port 5060`
The call rings but there is no audio	RTP filtered, or SDP with an address the peer cannot reach	open 20000–20499; the table address must be the tunnel's
SMS lost between nodes	TCP 7890 filtered, or no route	`sms-routing.conf`, `nc -zv 10.99.0.2 7890`

3 Fichiers

3.1 /opt/GSM/osmo-operator/.gitignore

580 octets, 28 lignes → 28 lignes

```
# Cache des .deb compiles (voir packaging/osmo-deb.sh) : recopie depuis
# /var/cache/osmo-debs par build.sh avant chaque build, jamais versionne.
.deb-cache/*.deb
# Configs generees pour docker compose (compose.sh up)
data/compose/
packaging/dist/
__pycache__/
*.pyc
*.iso
vty-debug-dump-*.txt

# depot git a part (github.com/bbaranoff/dino), deploye par wrangler
dino/
*.bak-*
vty-debug-dump.txt
navigation/logs/
mynode*.conf
.claude/

# Jupyter
.ipynb_checkpoints/
*-[0-9]*.pdf
osmo-operator-qmd/
bundle-qmd/

# notebook scratch output (wiki/)
wiki/sms.db*
wiki/Untitled*.ipynb
```

3.2 /opt/GSM/osmo-operator/compose.yaml

6990 octets, 174 lignes → 174 lignes

```
# ============================================================================
#  compose.yaml - the bench images and the one-operator bench, in compose v2
# ============================================================================
#
#  TWO PROFILES, TWO USES.
#
#  build   : the four images, each from its Dockerfile. This is what build.sh
#            (and behind it build-iso.sh, start-docker.sh) calls:
#                docker compose build nitb          Dockerfile      osmocom-nitb
#                docker compose build run           Dockerfile.run  osmocom-run
#                docker compose build lite          Dockerfile.lite osmocom-run:lite
#                docker compose build stp           Dockerfile.stp  osmocom-stp
#            They are named on the command line: no need to enable the
#            profile. `run` starts from the osmocom-nitb image ALREADY built
#            (or pulled from the hub: docker pull bastienbaranoff/norf_gsm);
#            it is deliberately not declared as additional_context, otherwise
#            compose would rebuild it every time.
#
#  bench   : the minimal bench - the SS7 hub (osmo-inter-stp) and ONE operator
#            (osmo-operator-1), with the same capabilities, networks, mounts
#            and variables as the `docker run` of start-docker.sh. The configs
#            are GENERATED by ./compose.sh up (apply_config_templates of this
#            repository, create_interop.sh) in data/compose/; the matching
#            .env file carries the computed addresses. For N operators, the
#            WAN and published SIP ports, start-docker.sh remains the orchestrator.
#                ./compose.sh up      ./compose.sh down     ./compose.sh ps
#
#  Variables (data/compose/.env, written by compose.sh; defaults here):
#    IMAGE_RUN IMAGE_STP        the images to run
#    OSMO_COMPOSE_DIR           where the generated configs are (./data/compose)
#    OP1_BACKBONE_IP            172.20.0.11   (op_backbone_ip 1)
```

```
#    OP1_PRIV_NET/GW/IP         192.168.2.0/24 .1 .10  (op_private_* 1)
#    N_MS PHY_MODE HOST_IP WAN_NODE_ID PULSE_SERVER
# ============================================================================
name: osmo-operator

services:
  # ── Images ──────────────────────────────────────────────────────────────
  nitb:
    image: osmocom-nitb
    build:
      context: .
      dockerfile: Dockerfile
      # REQUIRED: Dockerfile ends with the `debs` stage (the .deb cache alone,
      # for the CI), which would be the default target. Without this target,
      # `docker compose build nitb` produces an empty image.
      target: osmocom-nitb
      args:
        OSMO_DEB_REFRESH: ${OSMO_DEB_REFRESH:-0}
        OSMO_UBUNTU_MIRROR: ${OSMO_UBUNTU_MIRROR:-}
    profiles: [build]

  run:
    image: ${IMAGE_RUN:-osmocom-run}
    build:
      context: .
      dockerfile: Dockerfile.run
      args:
        QEMU_CACHE_BUST: ${QEMU_CACHE_BUST:-0}
        BASE: ${RUN_BASE:-osmocom-nitb}
    profiles: [build]

  lite:
    image: ${LITE_IMAGE:-osmocom-run:lite}
    build:
      context: .
      dockerfile: Dockerfile.lite
      args:
        BASE: ${LITE_BASE:-osmocom-run}
    profiles: [build]

  stp:
    image: ${IMAGE_STP:-osmocom-stp}
    build:
      context: .
      dockerfile: Dockerfile.stp
      args:
        BASE_IMAGE: ${STP_BASE_IMAGE:-ubuntu:24.04}
        HUB_NODES: ${HUB_NODES:-3}
        HUB_OPS: ${HUB_OPS:-1}
        OSMO_DEB_REFRESH: ${OSMO_DEB_REFRESH:-0}
        OSMO_UBUNTU_MIRROR: ${OSMO_UBUNTU_MIRROR:-}
    profiles: [build]

  # ── The bench: hub + operator 1 ─────────────────────────────────────────
  inter-stp:
    image: ${IMAGE_STP:-osmocom-stp}
    container_name: osmo-inter-stp
    hostname: osmo-inter-stp
    restart: unless-stopped
    cap_add: [NET_ADMIN]
    networks:
      gsm-inter:
        ipv4_address: 172.20.0.10
    # 2908 on the loopback, in SCTP: the attachment point of the NATIVE
    # operator (start-native.sh), see start-docker.sh start_inter_stp.
    ports:
```

```
      - "127.0.0.1:2908:2908/sctp"
    volumes:
      - ${OSMO_COMPOSE_DIR:-./data/compose}/hub/osmo-stp-interop.cfg:/etc/osmocom/osmo-stp-interop.cfg:ro
    profiles: [bench]

  operator-1:
    image: ${IMAGE_RUN:-osmocom-run}
    container_name: osmo-operator-1
    hostname: osmo-operator-1
    restart: unless-stopped
    command: ["sleep", "infinity"]
    depends_on: [inter-stp]
    cap_add: [NET_ADMIN, SYS_ADMIN, NET_RAW]
    ulimits:
      rtprio: 18
    shm_size: 8g
    cgroup: host
    devices:
      - /dev/net/tun:/dev/net/tun
    tmpfs:
      - /tmp:exec,rw,mode=1777,size=2g
      - /run:exec,size=64m
      - /run/lock
    volumes:
      - /sys/fs/cgroup:/sys/fs/cgroup:rw
      - /tmp/osmocom-logs/op1:/var/log/osmocom
      - ${OSMO_COMPOSE_DIR:-./data/compose}/op1/osmocom:/etc/osmocom
      - ${OSMO_COMPOSE_DIR:-./data/compose}/op1/asterisk/extensions.conf:/etc/asterisk/extensions.conf
      - ${OSMO_COMPOSE_DIR:-./data/compose}/op1/asterisk/pjsip.conf:/etc/asterisk/pjsip.conf
      - ${OSMO_COMPOSE_DIR:-./data/compose}/op1/asterisk/rtp.conf:/etc/asterisk/rtp.conf
      - ${OSMO_COMPOSE_DIR:-./data/compose}/op1/asterisk/modules.conf:/etc/asterisk/modules.conf
      - ${OSMO_COMPOSE_DIR:-./data/compose}/op1/asterisk/annuaire.conf:/etc/asterisk/annuaire.conf
      - ${OSMO_COMPOSE_DIR:-./data/compose}/op1/bb/mobile.cfg:/root/.osmocom/bb/mobile.cfg
      - ${OSMO_COMPOSE_DIR:-./data/compose}/asound.conf:/etc/asound.conf:rw
    environment:
      OPERATOR_ID: "1"
      OSMO_ROLE: operator
      N_MS: ${N_MS:-2}
      CONTAINER_IP: ${OP1_PRIV_IP:-192.168.2.10}
      GATEWAY_IP: ${OP1_PRIV_GW:-192.168.2.1}
      INTER_STP_IP: 172.20.0.10
      OSMO_HUB_IP: 172.20.0.10
      WAN_NODE_ID: ${WAN_NODE_ID:-1}
      OSMO_WAN_NODE: ${WAN_NODE_ID:-1}
      HOST_IP: ${HOST_IP:-}
      SIP_HOST_PORT: "5060"
      PHY_MODE: ${PHY_MODE:-faketrx}
      HOST_AUDIO_RELAY: ${HOST_AUDIO_RELAY:-}
      PULSE_SERVER: ${PULSE_SERVER:-tcp:172.20.0.1:4713}
      ALSA_OUTPUT: gsm_out
      ALSA_INPUT: gsm_in
      ALSA_CARD: gsm_out
      GAPK_ALSA_DEV: gsm_out
    networks:
      gsm-inter:
        ipv4_address: ${OP1_BACKBONE_IP:-172.20.0.11}
      gsm-net-op1:
        ipv4_address: ${OP1_PRIV_IP:-192.168.2.10}
    profiles: [bench]

networks:
  # The SS7 backbone: hub at .10, operator N at .(10+N) - INTER_NET of start-docker.sh.
  gsm-inter:
    name: gsm-inter
    driver: bridge
    ipam:
```

```
      config:
        - subnet: 172.20.0.0/24
          gateway: 172.20.0.1
  # Operator 1 private network - gsm-net-op1 of start-docker.sh (op_private_*).
  gsm-net-op1:
    name: gsm-net-op1
    driver: bridge
    ipam:
      config:
        - subnet: ${OP1_PRIV_NET:-192.168.2.0/24}
          gateway: ${OP1_PRIV_GW:-192.168.2.1}
```

3.3 /opt/GSM/osmo-operator/globals.conf

15656 octets, 454 lignes → 454 lignes

```
# ═══════════════════════════════════════════════════════════
#  globals.conf — the network configuration, in one place.
# ═══════════════════════════════════════════════════════════
#
#  Edit this file by hand: a run NEVER rewrites it.
#  Only « ./generate_configs.sh --force » regenerates it.
#
#  THIS FILE IS AUTHORITATIVE — FOR THE VARIABLES IT DECLARES, AND FOR
#  THOSE ONLY. It is read last: the 26 settings below override their
#  namesake in the environment. Everything else (CALYPSO_*, LOG_DIR,
#  MODE…) passes through untouched. Two ways to change it:
#      vim globals.conf
#      ./generate_configs.sh ARFCN=520
#
#  An EMPTY field = value computed automatically. Fields marked
#  [auto/operator] derive from the operator number: pin them only if
#  N_OPERATORS=1, otherwise the operators collide with each other.
# ═══════════════════════════════════════════════════════════

# ── Network identity ─────────────────────────────────────

# Mobile Country Code (3 digits)
MCC="001"
# Mobile Network Code (2 digits)
MNC="01"
# Short and long network name
OP_NAME="OsmoQEMU"

# ── Security and SIM card ────────────────────────────────

# Ciphering: « a5 0 » (none), « a5 1 », « a5 0 1 3 »…
ENCRYPTION="a5 1"
# Authentication algorithm (comp128, xor…)
SIM_ALGO="comp128"
# SIM Ki, 16 space-separated hex bytes  [auto/operator]
KI=""
# Mobile IMSI  [auto/operator]
IMSI=""
# Mobile IMEI  [auto/operator]
IMEI=""

# ── Radio ────────────────────────────────────────────────

# Radio channel (must match BAND)  [auto/operator]
ARFCN="1"
# Band of BTS 1 ONLY (BTS 0 is GSM900 in osmo-bsc.cfg): GSM900, DCS1800, PCS1900, GSM850
# Must match the ARFCN of BTS 1 (614 by default, DCS1800), otherwise the BSC
# refuses its OML.
```

```
BAND="DCS1800"
# Base Station Identity Code, 0..63  [auto/operator]
BSIC="7"
# Location Area Code  [auto/operator]
LAC="1"
# Cell identity  [auto/operator]
CELL_ID="6007"
# IPA unit id of the BTS  [auto/operator]
IPA_UNIT_ID=""
# Maximum power allowed to the mobile (0..15)
MS_MAX_POWER="15"
# Minimum received level to access the cell
RXLEV_ACCESS_MIN="0"
# Cell reselection hysteresis (dB)
CELL_RESEL_HYST="4"
# BSC timer T3109 (s) — channel release
T3109="5"

# ── GPRS ──────────────────────────────────────────────────

# BSSGP Virtual Connection Id  [auto/operator]
BVCI=""
# Network Service Entity Id  [auto/operator]
NSEI=""
# NS Virtual Connection Id  [auto/operator]
NSVCI=""
# Access point name
APN="internet"

# ── Services ──────────────────────────────────────────────

# SMS centre number  [auto/operator]
SMS_SC=""

# ── Sizing ────────────────────────────────────────────────

# Number of simulated mobiles
MS_COUNT="2"
# Number of operators (« virtual » mode)
N_OPERATORS="1"
# Host address as seen by the stack
HOST_IP="127.0.0.1"

# >>> VARIABLES DU LANCEUR : bloc genere par tools/globals-complet.py (ne pas editer les marqueurs) >>>
#
#  Valeur VIDE = defaut du script (en commentaire). Regenerer : python3 tools/globals-complet.py
#

# ── startup_scripts/01-defauts-options.sh ─────────────────

# defaut : /etc/osmocom   (01-defauts-options.sh:14)
#OSMOCOM_CFG=""
# defaut : faketrx-qemu   (01-defauts-options.sh:18)
#CALYPSO_PROFILE=""
# defaut : 0   (01-defauts-options.sh:25)
#SAP_MODE=""
# defaut : auto   (01-defauts-options.sh:26)
#SAP_TYPE=""
# defaut : /tmp/osmocom_sap   (01-defauts-options.sh:27)
#SAP_SOCK=""
# defaut : 1   (01-defauts-options.sh:80)
#DSP_MODE=""
# defaut : /opt/GSM/firmware/magnetite/c139-flash.img   (01-defauts-options.sh:327)
#MAGNETITE_FLASH=""
# defaut : 0   (01-defauts-options.sh:401)
#BTS_CLASSIC=""
```

```
# defaut : (vide)   (01-defauts-options.sh:414)
#OSMO_TRX_TS=""
# defaut : 0   (01-defauts-options.sh:421)
#OSMO_TRX_TS1=""
# defaut : 3   (01-defauts-options.sh:422)
#OSMO_TRX_TS1_INSTANCE=""
# defaut : 0   (01-defauts-options.sh:423)
#OSMO_TRX_TCH=""
# defaut : 0   (01-defauts-options.sh:434)
#MODEM_GPRS=""
# defaut : ${OSMO_WAN_NODE:-}   (01-defauts-options.sh:497)
#WAN_NODE_ID=""
# defaut : (vide)   (01-defauts-options.sh:497)
#OSMO_WAN_NODE=""
# defaut : /etc/osmo-wan.conf   (01-defauts-options.sh:507)
#WAN_CONF_FILE=""

# ── startup_scripts/02-noeud-couleurs.sh ──────────────────────

# defaut : ${INTER_STP_IP:-}   (02-noeud-couleurs.sh:19)
#OSMO_HUB_IP=""
# defaut : (vide)   (02-noeud-couleurs.sh:19)
#INTER_STP_IP=""
# defaut : (vide)   (02-noeud-couleurs.sh:78)
#CALYPSO_PY_BANC_EXTRA=""

# ── startup_scripts/04-environnement.sh ───────────────────────

# defaut : /opt/GSM   (04-environnement.sh:12)
#GSM_ROOT=""
# defaut : qosmo   (04-environnement.sh:17)
#CALYPSO_FORK=""
# defaut : /run/osmo-direct   (04-environnement.sh:25)
#RUN_DIR=""
# defaut : $RUN_DIR/logs   (04-environnement.sh:28)
#LOG_DIR=""

# ── startup_scripts/05-run-sh.sh ──────────────────────────────

# defaut : $GSM_ROOT/c54x_exe/run.sh   (05-run-sh.sh:25)
#BANC_DSP=""
# defaut : none   (05-run-sh.sh:34)
#CALYPSO_BRIDGE=""

# ── startup_scripts/06-banc-dsp-menu.sh ───────────────────────

# defaut : /usr/local/bin/${CALYPSO_FORK:-qosmo}   (06-banc-dsp-menu.sh:58)
#QOSMO_LAUNCHER=""
# defaut : 0   (06-banc-dsp-menu.sh:83)
#MENU_MODE=""
# defaut : 0   (06-banc-dsp-menu.sh:84)
#NO_MENU=""
# defaut : 1   (06-banc-dsp-menu.sh:192)
#NODE_ID=""
# defaut : 1   (06-banc-dsp-menu.sh:193)
#NODE_OP=""
# defaut : /etc/osmo-role   (06-banc-dsp-menu.sh:196)
#ROLE_FILE=""
# defaut : 0   (06-banc-dsp-menu.sh:198)
#WAN_MESH=""
# defaut : 0   (06-banc-dsp-menu.sh:203)
#REGEN_GABARITS=""
# defaut : 0   (06-banc-dsp-menu.sh:221)
#RUN_NO_PROCESS=""
# defaut : mobile   (06-banc-dsp-menu.sh:263)
#CALYPSO_L2_CLIENT=""
```

```
# defaut : 320   (06-banc-dsp-menu.sh:282)
#CALYPSO_PULSE_LATENCY_MSEC=""
# defaut : (vide)   (06-banc-dsp-menu.sh:293)
#FIRMWARE_SRC=""
# defaut : $FIRMWARE_SRC   (06-banc-dsp-menu.sh:295)
#FIRMWARE_ELF=""
# defaut : $FIRMWARE_SRC   (06-banc-dsp-menu.sh:297)
#FIRMWARE_DIR=""
# defaut : (vide)   (06-banc-dsp-menu.sh:306)
#FIRMWARE_BIN=""
# defaut : rtp   (06-banc-dsp-menu.sh:443)
#MS_TCH_FORMAT=""
# defaut : 1   (06-banc-dsp-menu.sh:567)
#OPERATOR_ID=""
# defaut : (vide)   (06-banc-dsp-menu.sh:637)
#PLAN_ARFCN=""
# defaut : ${PLAN_ARFCN_BTS1:-$(( 612 + MS_OP_ID * 2 ))}   (06-banc-dsp-menu.sh:648)
#SC_ARFCN=""
# defaut : $(( 612 + MS_OP_ID * 2 ))   (06-banc-dsp-menu.sh:648)
#PLAN_ARFCN_BTS1=""
# defaut : ${PLAN_UNIT_ID_BTS1:-$(( 6000 + MS_OP_ID * 10 + 2 ))}   (06-banc-dsp-menu.sh:649)
#SC_UNIT_ID=""
# defaut : $(( 6000 + MS_OP_ID * 10 + 2 ))   (06-banc-dsp-menu.sh:649)
#PLAN_UNIT_ID_BTS1=""

# ── startup_scripts/08-mobile-ms1.sh ──────────────────────────

# defaut : $BB_DIR/mobile.cfg   (08-mobile-ms1.sh:4)
#MOBILE_CFG_MS1_PATH=""

# ── startup_scripts/09-mobile-cfg-tmux.sh ─────────────────────

# defaut : $HERE/pont/pont.py   (09-mobile-cfg-tmux.sh:76)
#PONT_PY=""
# defaut : ${PLAN_ARFCN:-$MS_ARFCN1}   (09-mobile-cfg-tmux.sh:97)
#PONT_ARFCN=""
# defaut : ${PLAN_BSIC:-$(( MS_OP_ID * 7 % 64 ))}   (09-mobile-cfg-tmux.sh:98)
#PONT_BSIC=""
# defaut : $(( MS_OP_ID * 7 % 64 ))   (09-mobile-cfg-tmux.sh:98)
#PLAN_BSIC=""
# defaut : 0   (09-mobile-cfg-tmux.sh:129)
#CALYPSO_NO_KILLALL=""
# defaut : 0   (09-mobile-cfg-tmux.sh:193)
#AIR_MESH=""
# defaut : ${HERE}/pont/airmesh.py   (09-mobile-cfg-tmux.sh:194)
#AIRMESH_PY=""
# defaut : 2   (09-mobile-cfg-tmux.sh:203)
#N_MS=""
# defaut : 6700   (09-mobile-cfg-tmux.sh:205)
#BB_BASE_PORT=""
# defaut : 3   (09-mobile-cfg-tmux.sh:205)
#BB_PORT_STEP=""
# defaut : (vide)   (09-mobile-cfg-tmux.sh:211)
#WAN_NODES=""
# defaut : 4739   (09-mobile-cfg-tmux.sh:215)
#AIRMESH_PORT=""
# defaut : 0   (09-mobile-cfg-tmux.sh:266)
#OSMO_TRX=""
# defaut : $(dirname "$BANC_DSP")/mobile_pont.cfg   (09-mobile-cfg-tmux.sh:270)
#MOBILE_CFG=""
# defaut : 0   (09-mobile-cfg-tmux.sh:324)
#CALYPSO_NO_RESTART=""
# defaut : 0   (09-mobile-cfg-tmux.sh:328)
#CALYPSO_NO_ATTACH=""
# defaut : (vide)   (09-mobile-cfg-tmux.sh:332)
#QEMU_ICOUNT=""
```

```
# defaut : 0   (09-mobile-cfg-tmux.sh:333)
#ASSEMBLY_LOGS=""
# defaut : in_asm   (09-mobile-cfg-tmux.sh:338)
#ASSEMBLY_LOGS_FLAGS=""
# defaut : (vide)   (09-mobile-cfg-tmux.sh:339)
#ASSEMBLY_LOGS_FILTRE=""

# ── startup_scripts/10-identite-noeud.sh ──────────────────────

# defaut : 1   (10-identite-noeud.sh:73)
#CALYPSO_STOP_KILL_PYTHON=""

# ── startup_scripts/11-regen-configs.sh ──────────────────────

# defaut : 127.0.0.2   (11-regen-configs.sh:13)
#OSMO_NATIVE_CORE_IP=""
# defaut : 127.0.0.1   (11-regen-configs.sh:14)
#OSMO_NATIVE_GW_IP=""

# ── startup_scripts/12-pcap-softsim.sh ──────────────────────

# defaut : 0   (12-pcap-softsim.sh:81)
#PAYLOAD_ISM=""
# defaut : /dev/ttyUSB0   (12-pcap-softsim.sh:88)
#ISM_SERIAL=""
# defaut : 9600   (12-pcap-softsim.sh:97)
#ISM_SERIAL_BAUD=""
# defaut : /tmp/osmo_ism_tx   (12-pcap-softsim.sh:98)
#ISM_TX=""
# defaut : /tmp/osmo_ism_rx   (12-pcap-softsim.sh:98)
#ISM_RX=""
# defaut : 0   (12-pcap-softsim.sh:202)
#WAN_NODE_COUNT=""
# defaut : 0   (12-pcap-softsim.sh:441)
#OSMO_NO_PCAP_RING=""
# defaut : 32   (12-pcap-softsim.sh:504)
#OSMO_PCAP_RING_MB=""
# defaut : 5   (12-pcap-softsim.sh:504)
#OSMO_PCAP_RING_FILES=""

# ── startup_scripts/13-adresses.sh ──────────────────────

# defaut : 0   (13-adresses.sh:3)
#CALYPSO_NO_AUTOSTOP=""
# defaut : 1   (13-adresses.sh:13)
#CALYPSO_STOP_KILL_SOFTSIM=""
# defaut : 0   (13-adresses.sh:88)
#CALYPSO_NO_DASHBOARD=""
# defaut : 1   (13-adresses.sh:144)
#OSMO_RACCORD_MOBILE=""
# defaut : /opt/GSM/pysim-venv   (13-adresses.sh:274)
#PYSIM_VENV=""
# defaut : /opt/GSM/softsim   (13-adresses.sh:299)
#SOFTSIM_DIR=""
# defaut : /tmp/osmo-sap.log   (13-adresses.sh:300)
#SAP_LOG=""
# defaut : 0   (13-adresses.sh:332)
#DSP_BANC=""
# defaut : 3   (13-adresses.sh:360)
#SAP_VERBOSITY=""

# ── startup_scripts/14-demarrage.sh ──────────────────────

# defaut : none   (14-demarrage.sh:33)
#IQ=""
# defaut : 120000   (14-demarrage.sh:41)
```

```
#INSNS=""
# defaut : 1   (14-demarrage.sh:42)
#LOCKSTEP=""
# defaut : 1   (14-demarrage.sh:43)
#CALYPSO_BSP_STREAM=""
# defaut : 1   (14-demarrage.sh:44)
#CALYPSO_RHEA_DMA_XFER=""
# defaut : rom   (14-demarrage.sh:60)
#BANC_VOCODEUR=""
# defaut : 1   (14-demarrage.sh:61)
#CALYPSO_VSP=""
# defaut : (vide)   (14-demarrage.sh:90)
#MS_MAGNETITE=""
# defaut : 0   (14-demarrage.sh:91)
#MS_AIR=""
# defaut : 150   (14-demarrage.sh:114)
#OSMO_TRX_BTS_ATTENTE=""

# ── /opt/GSM/c54x_exe/run.sh ──────────────────────────────

# defaut : dsp   (run.sh:52)
#MODE=""
# defaut : 0   (run.sh:53)
#PONT=""
# defaut : /tmp/osmocom_l2_$INSTANCE   (run.sh:68)
#L2_SOCK=""
# defaut : /tmp/qemu-monitor-pont$INSTANCE.sock   (run.sh:69)
#MONITOR=""
# defaut : /tmp/calypso_dsp$INSTANCE.sock   (run.sh:70)
#DSP_SOCK=""
# defaut : $((1233 + INSTANCE))   (run.sh:71)
#GDB_STUB=""
# defaut : $((44443 + INSTANCE))   (run.sh:72)
#GDB_TELNET=""
# defaut : none   (run.sh:73)
#PANNEAU_LOGS=""
# defaut : $((6702 + 10 * (INSTANCE - 1) + (INSTANCE >= 3 ? 30 : 0)))   (run.sh:76)
#CALYPSO_BSP_PORT=""
# defaut : /tmp/calypso_keys$INSTANCE.sock   (run.sh:77)
#CALYPSO_KEYS_SOCK=""
# defaut : 1   (run.sh:78)
#AIR=""
# defaut : /opt/GSM/osmo-operator/pont/air.py   (run.sh:81)
#AIR_PY=""
# defaut : /opt/GSM/qosmo   (run.sh:82)
#QOSMO=""
# defaut : $QOSMO/build/qemu-system-arm   (run.sh:83)
#QEMU=""
# defaut : (vide)   (run.sh:91)
#FIRMWARE_APP=""
# defaut : 1   (run.sh:108)
#OSMO_TRX_PHONE=""
# defaut : $(awk '/^ *bts 0$/{f=1} f&&/base_station_id_code/{print $2; exit}' /etc/osmocom/osmo-bsc.cfg
 ↪  2>/dev/null)   (run.sh:124)
#OSMO_TRX_BSIC=""
# defaut : /opt/GSM/osmocom-bb-transceiver/src/host/layer23/src/transceiver/transceiver   (run.sh:125)
#TRANSCEIVER=""
# defaut : 0x03   (run.sh:134)
#OSMO_TRX_TS_MASK=""
# defaut : 0x03   (run.sh:135)
#OSMO_TRX_TX_MASK=""
# defaut : 2   (run.sh:136)
#CALYPSO_TRX_TX_SLOTS=""
# defaut : --no-sync -e 3   (run.sh:146)
#OSMO_TRX_ARGS=""
# defaut : 1   (run.sh:149)
```

```
#MONTANT_ROM_TX_CHAQUE=""
# defaut : 1   (run.sh:153)
#CALYPSO_BSP_TRX=""
# defaut : -11   (run.sh:157)
#CALYPSO_BSP_TRX_DELTA=""
# defaut : (vide)   (run.sh:171)
#FIRMWARE_FLASH=""
# defaut : 0   (run.sh:180)
#CALYPSO_PACER_RATTRAPAGE=""
# defaut : 1   (run.sh:181)
#CALYPSO_PONT_ARM_LOCK=""
# defaut : 1   (run.sh:184)
#CALYPSO_BSP_SB_FIN_TRAME=""
# defaut : 1   (run.sh:185)
#CALYPSO_BSP_NB_MARGE=""
# defaut : (vide)   (run.sh:188)
#PONT_CAN_NB_TOA=""
# defaut : 0.5   (run.sh:193)
#CALYPSO_BSP_SB_DEC=""
# defaut : 1   (run.sh:194)
#CALYPSO_BSP_BRUIT_FOND=""
# defaut : $AIR   (run.sh:199)
#CALYPSO_BSP_RATTRAPE=""
# defaut : 1   (run.sh:200)
#CALYPSO_RACH_VERS_AIR=""
# defaut : 0   (run.sh:205)
#PONT_UL_TN_DEFAUT=""
# defaut : 0   (run.sh:209)
#AIR_B_FN_DELTA=""
# defaut : 0   (run.sh:210)
#MONTANT_RACH_DIFFERE=""
# defaut : $( . /etc/osmocom/radio-plan.env 2>/dev/null; echo "${PLAN_ARFCN:-}")   (run.sh:211)
#CALYPSO_FAUX_AIR=""
# defaut : ti925t   (run.sh:213)
#QEMU_CPU=""
# defaut : c139   (run.sh:214)
#CALYPSO_LCD=""
# defaut : /opt/GSM/fc-host-tools/rvinterf/lowlevel/rvinterf   (run.sh:215)
#RVINTERF_BIN=""
# defaut : /opt/GSM/osmocom-bb/src/host/osmocon/osmocon   (run.sh:217)
#OSMOCON=""
# defaut : $(command -v mobile || echo /usr/local/bin/mobile)   (run.sh:218)
#MOBILE=""
# defaut : (vide)   (run.sh:239)
#RT_FIFO=""
# defaut : (vide)   (run.sh:240)
#RT_CPUS=""
# defaut : (vide)   (run.sh:241)
#RT_RENICE_AUTRES=""
# defaut : 1   (run.sh:263)
#GDB=""
# defaut : $QOSMO/tools/gdb-telnet.py   (run.sh:266)
#GDB_TELNET_PY=""
# defaut : -v   (run.sh:278)
#VERB=""
# defaut : 30000   (run.sh:280)
#AMP=""
# defaut : 1   (run.sh:294)
#PONT_AIRREC=""
# defaut : 4194304   (run.sh:385)
#QEMU_LOG_BUFFERED=""
# defaut : 64   (run.sh:422)
#CALYPSO_PONT_RETRY_DIV=""
# defaut : 10   (run.sh:722)
#JOURNAUX_GARDES=""
# defaut : (vide)   (run.sh:763)
```

```
#BRUIT_MODE=""
# defaut : (vide)   (run.sh:764)
#BRUIT_SNR_DB=""
# defaut : 16702   (run.sh:765)
#BRUIT_PORT_DSP=""
# defaut : 14730   (run.sh:766)
#BRUIT_PORT_GRGSM=""
# defaut : $BRUIT_SNR_DB   (run.sh:811)
#CALYPSO_BSP_SNR_DB=""

# <<< FIN DES VARIABLES DU LANCEUR <<<
```

3.4 /opt/GSM/osmo-operator/globals.conf.example

12832 octets, 371 lignes → 371 lignes

```
# ═══════════════════════════════════════════════════════════
#  globals.conf — la configuration du réseau, en un seul endroit.
# ═══════════════════════════════════════════════════════════
#
#  Édite ce fichier à la main : il ne sera JAMAIS réécrit par un run.
#  Seul « ./generate_configs.sh --force » le régénère.
#
#  CE FICHIER FAIT AUTORITÉ — SUR LES VARIABLES QU'IL DÉCLARE, ET SUR
#  ELLES SEULES. Il est lu en dernier : les 26 réglages ci-dessous
#  écrasent leur homonyme dans l'environnement. Tout le reste (CALYPSO_*,
#  LOG_DIR, MODE…) passe au travers, intact. Deux façons de le modifier :
#      vim globals.conf
#      ./generate_configs.sh ARFCN=520
#
#  Un champ VIDE = valeur calculée automatiquement. Les champs notés
#  [auto/opérateur] se déduisent du numéro d'opérateur : ne les fige
#  que si N_OPERATORS=1, sinon tous les opérateurs se marchent dessus.
# ═══════════════════════════════════════════════════════════

# ── Identité du réseau ──────────────────────────────────────

# Mobile Country Code (3 chiffres)
MCC="001"
# Mobile Network Code (2 chiffres)
MNC="01"
# Nom court et long du réseau
OP_NAME="OsmoQEMU"

# ── Sécurité et carte SIM ──────────────────────────────────

# Chiffrement : « a5 0 » (aucun), « a5 1 », « a5 0 1 3 »…
# choix : a5 0 | a5 1 | a5 3 | a5 0 1 | a5 0 1 3
ENCRYPTION="a5 1"
# Algorithme d'authentification (comp128, xor…)
# choix : comp128 | comp128v2 | comp128v3 | xor | milenage
SIM_ALGO="comp128"
# Ki de la SIM, 16 octets hexa espacés  [auto/opérateur]
KI=""
# IMSI du mobile  [auto/opérateur]
IMSI=""
# IMEI du mobile  [auto/opérateur]
IMEI=""

# ── Radio ──────────────────────────────────────────────────

# Canal radio (à accorder avec BAND)  [auto/opérateur]
ARFCN="1"
# Bande de la BTS n°1 SEULEMENT (la BTS 0 est en GSM900 dans osmo-bsc.cfg) : GSM900, DCS1800, PCS1900, GSM850
```

```
# Doit correspondre a l ARFCN de la BTS n°1 (614 par defaut, DCS1800) sinon le BSC refuse son OML.
# choix : GSM900 | DCS1800 | PCS1900 | GSM850
BAND="DCS1800"
# Base Station Identity Code, 0..63  [auto/opérateur]
BSIC=""
# Location Area Code  [auto/opérateur]
LAC=""
# Cell identity  [auto/opérateur]
CELL_ID=""
# Identifiant IPA du BTS  [auto/opérateur]
IPA_UNIT_ID=""
# Puissance max autorisée au mobile (0..15)
MS_MAX_POWER="15"
# Niveau reçu minimal pour accéder à la cellule
RXLEV_ACCESS_MIN="0"
# Hystérésis de resélection de cellule (dB)
CELL_RESEL_HYST="4"
# Timer T3109 du BSC (s) — libération de canal
T3109="5"

# ── GPRS ──────────────────────────────────────────────────

# BSSGP Virtual Connection Id  [auto/opérateur]
BVCI=""
# Network Service Entity Id  [auto/opérateur]
NSEI=""
# NS Virtual Connection Id  [auto/opérateur]
NSVCI=""
# Nom du point d'accès
APN="internet"

# ── Services ──────────────────────────────────────────────

# Numéro du centre SMS  [auto/opérateur]
SMS_SC=""

# ── Dimensionnement ───────────────────────────────────────

# Nombre de mobiles simulés
MS_COUNT="2"
# Nombre d'opérateurs (mode « virtual »)
N_OPERATORS="1"
# Adresse de l'hôte vue par la pile
HOST_IP="127.0.0.1"

# >>> VARIABLES DU LANCEUR : bloc genere par tools/globals-complet.py (ne pas editer les marqueurs) >>>
#
#  Valeur VIDE = defaut du script (en commentaire). Regenerer : python3 tools/globals-complet.py
#

# ── startup_scripts/01-defauts-options.sh ─────────────────

# defaut : /etc/osmocom   (01-defauts-options.sh:14)
OSMOCOM_CFG=""
# defaut : faketrx-qemu   (01-defauts-options.sh:18)
# choix : faketrx-qemu | faketrx | qemu | noproc
CALYPSO_PROFILE=""
# defaut : 0   (01-defauts-options.sh:25)
# choix : 0 | 1
SAP_MODE=""
# defaut : auto   (01-defauts-options.sh:26)
# choix : auto | pcsc | softsim
SAP_TYPE=""
# defaut : /tmp/osmocom_sap   (01-defauts-options.sh:27)
SAP_SOCK=""
# defaut : 1   (01-defauts-options.sh:80)
```

```
# choix : 0 | 1
DSP_MODE=""
# defaut : /opt/GSM/firmware/magnetite/c139-flash.img   (01-defauts-options.sh:327)
MAGNETITE_FLASH=""
# defaut : 0   (01-defauts-options.sh:401)
# choix : 0 | 1
BTS_CLASSIC=""
# defaut : (vide)   (01-defauts-options.sh:414)
# choix : 1 | 2 | 2/1
OSMO_TRX_TS=""
# defaut : 0   (01-defauts-options.sh:421)
# choix : 0 | 1
OSMO_TRX_TS1=""
# defaut : 3   (01-defauts-options.sh:422)
OSMO_TRX_TS1_INSTANCE=""
# defaut : 0   (01-defauts-options.sh:423)
# choix : 0 | 1
OSMO_TRX_TCH=""
# defaut : 0   (01-defauts-options.sh:434)
# choix : 0 | 1
MODEM_GPRS=""
# defaut : ${OSMO_WAN_NODE:-}   (01-defauts-options.sh:497)
# choix : 0 | 1
WAN_NODE_ID=""
# defaut : (vide)   (01-defauts-options.sh:497)
OSMO_WAN_NODE=""
# defaut : /etc/osmo-wan.conf   (01-defauts-options.sh:507)
WAN_CONF_FILE=""

# ── startup_scripts/02-noeud-couleurs.sh ──────────────────────

# defaut : ${INTER_STP_IP:-}   (02-noeud-couleurs.sh:19)
OSMO_HUB_IP=""
# defaut : (vide)   (02-noeud-couleurs.sh:19)
INTER_STP_IP=""
# defaut : (vide)   (02-noeud-couleurs.sh:78)
CALYPSO_PY_BANC_EXTRA=""

# ── startup_scripts/04-environnement.sh ───────────────────────

# defaut : /opt/GSM   (04-environnement.sh:12)
GSM_ROOT=""
# defaut : qosmo   (04-environnement.sh:17)
CALYPSO_FORK=""
# defaut : /run/osmo-direct   (04-environnement.sh:25)
RUN_DIR=""
# defaut : $RUN_DIR/logs   (04-environnement.sh:28)
LOG_DIR=""

# ── startup_scripts/05-run-sh.sh ──────────────────────────────

# defaut : $GSM_ROOT/c54x_exe/run.sh   (05-run-sh.sh:25)
BANC_DSP=""
# defaut : none   (05-run-sh.sh:34)
CALYPSO_BRIDGE=""

# ── startup_scripts/06-banc-dsp-menu.sh ───────────────────────

# defaut : /usr/local/bin/${CALYPSO_FORK:-qosmo}   (06-banc-dsp-menu.sh:58)
QOSMO_LAUNCHER=""
# defaut : 0   (06-banc-dsp-menu.sh:83)
# choix : 0 | 1
MENU_MODE=""
# defaut : 0   (06-banc-dsp-menu.sh:84)
# choix : 0 | 1
NO_MENU=""
```

```
# defaut : 1   (06-banc-dsp-menu.sh:192)
# choix : 0 | 1
NODE_ID=""
# defaut : 1   (06-banc-dsp-menu.sh:193)
# choix : 0 | 1
NODE_OP=""
# defaut : /etc/osmo-role   (06-banc-dsp-menu.sh:196)
ROLE_FILE=""
# defaut : 0   (06-banc-dsp-menu.sh:198)
# choix : 0 | 1
WAN_MESH=""
# defaut : 0   (06-banc-dsp-menu.sh:203)
# choix : 0 | 1
REGEN_GABARITS=""
# defaut : 0   (06-banc-dsp-menu.sh:221)
# choix : 0 | 1
RUN_NO_PROCESS=""
# defaut : mobile   (06-banc-dsp-menu.sh:263)
CALYPSO_L2_CLIENT=""
# defaut : 320   (06-banc-dsp-menu.sh:282)
CALYPSO_PULSE_LATENCY_MSEC=""
# defaut : (vide)   (06-banc-dsp-menu.sh:293)
FIRMWARE_SRC=""
# defaut : $FIRMWARE_SRC   (06-banc-dsp-menu.sh:295)
FIRMWARE_ELF=""
# defaut : $FIRMWARE_SRC   (06-banc-dsp-menu.sh:297)
FIRMWARE_DIR=""
# defaut : (vide)   (06-banc-dsp-menu.sh:306)
FIRMWARE_BIN=""
# defaut : rtp   (06-banc-dsp-menu.sh:443)
MS_TCH_FORMAT=""
# defaut : 1   (06-banc-dsp-menu.sh:567)
# choix : 0 | 1
OPERATOR_ID=""
# defaut : (vide)   (06-banc-dsp-menu.sh:637)
PLAN_ARFCN=""
# defaut : ${PLAN_ARFCN_BTS1:-$(( 612 + MS_OP_ID * 2 ))}   (06-banc-dsp-menu.sh:648)
SC_ARFCN=""
# defaut : $(( 612 + MS_OP_ID * 2 ))   (06-banc-dsp-menu.sh:648)
PLAN_ARFCN_BTS1=""
# defaut : ${PLAN_UNIT_ID_BTS1:-$(( 6000 + MS_OP_ID * 10 + 2 ))}   (06-banc-dsp-menu.sh:649)
SC_UNIT_ID=""
# defaut : $(( 6000 + MS_OP_ID * 10 + 2 ))   (06-banc-dsp-menu.sh:649)
PLAN_UNIT_ID_BTS1=""

# ── startup_scripts/08-mobile-ms1.sh ──────────────────────────

# defaut : $BB_DIR/mobile.cfg   (08-mobile-ms1.sh:4)
MOBILE_CFG_MS1_PATH=""

# ── startup_scripts/09-mobile-cfg-tmux.sh ─────────────────────

# defaut : $HERE/pont/pont.py   (09-mobile-cfg-tmux.sh:76)
PONT_PY=""
# defaut : ${PLAN_ARFCN:-$MS_ARFCN1}   (09-mobile-cfg-tmux.sh:97)
PONT_ARFCN=""
# defaut : ${PLAN_BSIC:-$(( MS_OP_ID * 7 % 64 ))}   (09-mobile-cfg-tmux.sh:98)
PONT_BSIC=""
# defaut : $(( MS_OP_ID * 7 % 64 ))   (09-mobile-cfg-tmux.sh:98)
PLAN_BSIC=""
# defaut : 0   (09-mobile-cfg-tmux.sh:129)
# choix : 0 | 1
CALYPSO_NO_KILLALL=""
# defaut : 0   (09-mobile-cfg-tmux.sh:193)
# choix : 0 | 1
AIR_MESH=""
```

```
# defaut : ${HERE}/pont/airmesh.py   (09-mobile-cfg-tmux.sh:194)
AIRMESH_PY=""
# defaut : 2   (09-mobile-cfg-tmux.sh:203)
N_MS=""
# defaut : 6700   (09-mobile-cfg-tmux.sh:205)
BB_BASE_PORT=""
# defaut : 3   (09-mobile-cfg-tmux.sh:205)
BB_PORT_STEP=""
# defaut : (vide)   (09-mobile-cfg-tmux.sh:211)
WAN_NODES=""
# defaut : 4739   (09-mobile-cfg-tmux.sh:215)
AIRMESH_PORT=""
# defaut : 0   (09-mobile-cfg-tmux.sh:266)
# choix : 0 | 1
OSMO_TRX=""
# defaut : $(dirname "$BANC_DSP")/mobile_pont.cfg   (09-mobile-cfg-tmux.sh:270)
MOBILE_CFG=""
# defaut : 0   (09-mobile-cfg-tmux.sh:324)
# choix : 0 | 1
CALYPSO_NO_RESTART=""
# defaut : 0   (09-mobile-cfg-tmux.sh:328)
# choix : 0 | 1
CALYPSO_NO_ATTACH=""
# defaut : (vide)   (09-mobile-cfg-tmux.sh:332)
QEMU_ICOUNT=""
# defaut : 0   (09-mobile-cfg-tmux.sh:333)
# choix : 0 | 1
ASSEMBLY_LOGS=""
# defaut : in_asm   (09-mobile-cfg-tmux.sh:338)
ASSEMBLY_LOGS_FLAGS=""
# defaut : (vide)   (09-mobile-cfg-tmux.sh:339)
ASSEMBLY_LOGS_FILTRE=""

# ── startup_scripts/10-identite-noeud.sh ──────────────────────

# defaut : 1   (10-identite-noeud.sh:73)
# choix : 0 | 1
CALYPSO_STOP_KILL_PYTHON=""

# ── startup_scripts/11-regen-configs.sh ──────────────────────

# defaut : 127.0.0.2   (11-regen-configs.sh:13)
OSMO_NATIVE_CORE_IP=""
# defaut : 127.0.0.1   (11-regen-configs.sh:14)
OSMO_NATIVE_GW_IP=""

# ── startup_scripts/12-pcap-softsim.sh ──────────────────────

# defaut : 0   (12-pcap-softsim.sh:81)
# choix : 0 | 1
PAYLOAD_ISM=""
# defaut : /dev/ttyUSB0   (12-pcap-softsim.sh:88)
ISM_SERIAL=""
# defaut : 9600   (12-pcap-softsim.sh:97)
ISM_SERIAL_BAUD=""
# defaut : /tmp/osmo_ism_tx   (12-pcap-softsim.sh:98)
ISM_TX=""
# defaut : /tmp/osmo_ism_rx   (12-pcap-softsim.sh:98)
ISM_RX=""
# defaut : 0   (12-pcap-softsim.sh:202)
# choix : 0 | 1
WAN_NODE_COUNT=""
# defaut : 0   (12-pcap-softsim.sh:441)
# choix : 0 | 1
OSMO_NO_PCAP_RING=""
# defaut : 32   (12-pcap-softsim.sh:504)
```

```
OSMO_PCAP_RING_MB=""
# defaut : 5   (12-pcap-softsim.sh:504)
OSMO_PCAP_RING_FILES=""

# ── startup_scripts/13-adresses.sh ──────────────────────────

# defaut : 0   (13-adresses.sh:3)
# choix : 0 | 1
CALYPSO_NO_AUTOSTOP=""
# defaut : 1   (13-adresses.sh:13)
# choix : 0 | 1
CALYPSO_STOP_KILL_SOFTSIM=""
# defaut : 0   (13-adresses.sh:88)
# choix : 0 | 1
CALYPSO_NO_DASHBOARD=""
# defaut : 1   (13-adresses.sh:144)
# choix : 0 | 1
OSMO_RACCORD_MOBILE=""
# defaut : /opt/GSM/pysim-venv   (13-adresses.sh:274)
PYSIM_VENV=""
# defaut : /opt/GSM/softsim   (13-adresses.sh:299)
SOFTSIM_DIR=""
# defaut : /tmp/osmo-sap.log   (13-adresses.sh:300)
SAP_LOG=""
# defaut : 0   (13-adresses.sh:332)
# choix : 0 | 1
DSP_BANC=""
# defaut : 3   (13-adresses.sh:360)
SAP_VERBOSITY=""

# ── startup_scripts/14-demarrage.sh ─────────────────────────

# defaut : none   (14-demarrage.sh:33)
IQ=""
# defaut : 120000   (14-demarrage.sh:41)
INSNS=""
# defaut : 1   (14-demarrage.sh:42)
# choix : 0 | 1
LOCKSTEP=""
# defaut : 1   (14-demarrage.sh:43)
# choix : 0 | 1
CALYPSO_BSP_STREAM=""
# defaut : 1   (14-demarrage.sh:44)
# choix : 0 | 1
CALYPSO_RHEA_DMA_XFER=""
# defaut : rom   (14-demarrage.sh:60)
BANC_VOCODEUR=""
# defaut : 1   (14-demarrage.sh:61)
# choix : 0 | 1
CALYPSO_VSP=""
# defaut : (vide)   (14-demarrage.sh:90)
MS_MAGNETITE=""
# defaut : 0   (14-demarrage.sh:91)
# choix : 0 | 1
MS_AIR=""
# defaut : 150   (14-demarrage.sh:114)
OSMO_TRX_BTS_ATTENTE=""

# <<< FIN DES VARIABLES DU LANCEUR <<<
```

3.5 /opt/GSM/osmo-operator/.deb-cache/.keep

0 octets, 0 lignes → 0 lignes

3.6 /opt/GSM/osmo-operator/.github/actions/docker-disk/action.yml

3397 octets, 74 lignes → 74 lignes

```yaml
name: "Disque pour docker"
description: >
  Fait de la place sur le runner, puis installe /var/lib/docker sur le plus
  grand disque disponible. A utiliser dans tout job qui manipule une image
  osmocom-nitb (11 Go).

inputs:
  need:
    description: "Go libres exiges pour /var/lib/docker"
    required: false
    default: "40"

runs:
  using: composite
  steps:
    # Le runner arrive avec ~14 Go d'outils dont aucun build de ce depot n'a
    # l'usage, et plusieurs images deja tirees dans /var/lib/docker.
    - name: Menage du runner
      shell: bash
      run: |
        sudo rm -rf /usr/share/dotnet /usr/local/lib/android \
          /opt/ghc /opt/hostedtoolcache/CodeQL \
          /usr/local/share/boost /usr/share/swift \
          /usr/local/graalvm /usr/local/share/chromium \
          /usr/local/lib/node_modules
        sudo apt-get remove -y '^dotnet-.*' 'php.*' 'mongodb.*' \
          'mysql.*' azure-cli google-cloud-cli 2>/dev/null || true
        sudo apt-get autoremove -y && sudo apt-get clean
        docker system prune -af --volumes
        df -h / /mnt || true

    # / n'offre qu'une trentaine de Go libres. /mnt est un second disque
    # d'environ 70 Go quasiment vide : c'est la que docker doit vivre.
    #
    # [2026-09-13] Le repli silencieux « on reste sur / » est SUPPRIME. Sur /,
    # le build va au bout de ses 75 etapes puis meurt en « exporting layers »,
    # SIGTERM du runner a court d'espace : exit 143, une heure et demie perdue,
    # et pour seule trace une ligne noyee ici. S'il n'y a nulle part la place,
    # l'echec est certain : autant le dire en trois secondes, avec son motif.
    - name: Docker data-root sur /mnt
      shell: bash
      env:
        NEED: ${{ inputs.need }}
      run: |
        AVAIL=$(df -BG --output=avail /mnt 2>/dev/null | tail -1 | tr -dc '0-9')
        ROOT_AVAIL=$(df -BG --output=avail / | tail -1 | tr -dc '0-9')
        if [ "${AVAIL:-0}" -lt "$NEED" ]; then
          if [ "${ROOT_AVAIL:-0}" -ge "$NEED" ]; then
            echo "::warning::/mnt inutilisable (${AVAIL:-0} Go) mais / offre ${ROOT_AVAIL} Go - on reste sur /."
            df -h /; exit 0
          fi
          echo "::error::Aucun disque de ${NEED} Go pour /var/lib/docker (/mnt ${AVAIL:-0} Go, / ${ROOT_AVAIL:-0}
↪ Go). L'image de base en pese 11 : ce job echouerait en exit 143 apres plus d'une heure."
          df -h / /mnt || true
          exit 1
        fi
        sudo systemctl stop docker.service docker.socket
        sudo mkdir -p /mnt/docker
```

```
        sudo mv /var/lib/docker/* /mnt/docker/ 2>/dev/null || true
        sudo jq '. + {"data-root":"/mnt/docker"}' /etc/docker/daemon.json \
          > /tmp/daemon.json 2>/dev/null \
          || echo '{"data-root":"/mnt/docker"}' > /tmp/daemon.json
        sudo mv /tmp/daemon.json /etc/docker/daemon.json
        sudo systemctl start docker.service
        # Le demon a pu redemarrer sur l'ancienne racine (daemon.json refuse,
        # mv partiel) : sans cette verification on ne l'apprendrait qu'a
        # l'export, sous la forme d'un 143 muet.
        for _ in $(seq 30); do docker info >/dev/null 2>&1 && break; sleep 2; done
        DR=$(docker info -f '{{.DockerRootDir}}')
        if [ "$DR" != "/mnt/docker" ]; then
          echo "::error::docker a redemarre sur ${DR}, pas /mnt/docker."
          exit 1
        fi
        echo "data-root = $DR"
        df -h / /mnt
```

3.7 /opt/GSM/osmo-operator/.github/workflows/build-arm.yml

13854 octets, 305 lignes → 305 lignes

```
name: Build images SD (arm64 / Armbian - Raspberry Pi 4, Orange Pi 5)

# La MEME pile que les ISO x86, construite pour arm64 et livree en IMAGES SD
# par le framework officiel github.com/armbian/build : c'est le chemin --arm de
# build-iso.sh, qui passe la main a armbian/build-armbian.sh. Armbian fabrique
# noyau, BSP, u-boot/firmware, rootfs et image pour chaque carte ; la pile y
# est posee par armbian/userpatches/customize-image.sh, depuis l'image Docker
# arm64 publiee par docker-arm.yml (job amont).
#
# [2026-10-24] Une image PAR CARTE, un job par carte (matrice) : Armbian ne
# construit qu'une carte par compile.sh, et sept cartes a la suite ne tiendraient
# pas dans les 6 h d'un job heberge. Les cartes :
#   rpi4b                      Raspberry Pi 4
#   orangepi5, orangepi5b, orangepi5-plus, orangepi5pro,
#   orangepi5-max, orangepi5-ultra      toutes les Orange Pi 5 (la Max et
#                              l'Ultra ont leur propre fichier de carte)
# La branche noyau est choisie par carte (build-armbian.sh : current, sinon la
# premiere proposee - edge pour la 5B).
#
# ── POURQUOI UN RUNNER arm64 ET PAS L'EMULATION ──────────────────────────
# Sur un hote x86, le chroot de customize-image.sh tourne sous qemu-user :
# c'est lent, et un job heberge est TUE a 6 heures. Les runners arm64 heberges
# (ubuntu-24.04-arm) font tout NATIVEMENT. C'est le defaut ; l'entree `runner`
# permet de demander un hote x86 pour verifier le chemin emule.
#
# ── LE BUREAU ────────────────────────────────────────────────────────────
# desktop=true : GNOME (DESKTOP_ENVIRONMENT=gnome, DESKTOP_TIER=minimal, via
# armbian/configng) + le bureau du banc (fond, Conky, encart, icones, c123).
# GLava n'y est pas : le cercle NCS veut OpenGL >= 4.2, absent des GPU de ces
# cartes sous Mesa. Pas de 4G non plus (srsRAN/Open5GS non compiles en aarch64).

on:
  push:
    tags: ["rpi-*", "arm-*", "opi-*"]
  workflow_dispatch:
    inputs:
      version:
        description: "Release tag (v0.1-8 kept in reserve)"
        type: string
        default: "v0.1-9"
      boards:
       description: "Cartes : all, rpi4, opi5, ou des noms Armbian separes par des virgules (orangepi5-max,rpi4b)"
```

```
        type: string
        default: "all"
      desktop:
        description: "Bureau GNOME + bureau du banc (--desktop)"
        type: boolean
        default: true
      runner:
        description: "Hote de construction"
        type: choice
        default: ubuntu-24.04-arm
        options:
          - ubuntu-24.04-arm   # natif aarch64 : le seul qui tienne dans les 6 h
          - ubuntu-24.04       # x86 + qemu-user : lent, sera coupe a 6 h
      release:
        description: "Create GitHub Release"
        type: boolean
        default: true
      no_cache:
        description: "Rebuild Docker images without cache (--no-cache)"
        type: boolean
        default: false

concurrency:
  group: build-arm-${{ github.ref }}
  cancel-in-progress: false

jobs:
  # ── EN AMONT : l'image Docker arm64 de la pile (docker-arm.yml) ──────────
  docker-arm:
    permissions:
      contents: read
      packages: write
    uses: ./.github/workflows/docker-arm.yml
    with:
      rebuild: false
      no_cache: ${{ github.event.inputs.no_cache == 'true' }}

  # ── La liste des cartes -> la matrice ────────────────────────────────────
  # Memes groupes que build-armbian.sh (--board=rpi4|opi5|all). Sur un push de
  # tag les entrees sont VIDES : toutes les cartes, avec bureau.
  boards:
    runs-on: ubuntu-24.04
    outputs:
      matrix: ${{ steps.m.outputs.matrix }}
      desktop: ${{ steps.m.outputs.desktop }}
    steps:
      - id: m
        run: |
          IN='${{ github.event.inputs.boards }}'; IN=${IN:-all}
          OPI5="orangepi5 orangepi5b orangepi5-plus orangepi5pro orangepi5-max orangepi5-ultra"
          L=""
          for b in ${IN//,/ }; do
            case "$b" in
              rpi4|rpi4b) L="$L rpi4b" ;;
              opi5)       L="$L $OPI5" ;;
              all)        L="$L rpi4b $OPI5" ;;
              *)          L="$L $b" ;;
            esac
          done
          M=$(echo $L | tr ' ' '\n' | awk 'NF && !vu[$0]++' | jq -R . | jq -cs .)
          echo "matrix=$M" >> "$GITHUB_OUTPUT"
          D='${{ github.event.inputs.desktop }}'
          echo "desktop=${D:-true}" >> "$GITHUB_OUTPUT"
          echo "cartes : $M ; bureau : ${D:-true}"

  build:
```

```yaml
    needs: [docker-arm, boards]
    runs-on: ${{ github.event.inputs.runner || 'ubuntu-24.04-arm' }}
    timeout-minutes: 360
    strategy:
      fail-fast: false            # une carte qui casse n'emporte pas les autres
      matrix:
        board: ${{ fromJSON(needs.boards.outputs.matrix) }}
    permissions:
      contents: read
      packages: read

    steps:
      - name: Checkout
        uses: actions/checkout@v5
        with:
          fetch-depth: 0

      - name: Architecture de l'hote
        run: |
          HOST=$(dpkg --print-architecture)
          echo "hote : $HOST  ($(uname -m), $(nproc) coeurs, $(free -g | awk '/^Mem:/{print $2}') Go) ; carte ${{
↪ matrix.board }}"
          if [ "$HOST" = "arm64" ]; then
            echo "::notice::Hote aarch64 : construction Armbian NATIVE."
          else
            echo "::warning::Hote $HOST : le chroot arm64 sera EMULE (qemu-user-static). Ce job est coupe a 6 h."
          fi

      # Menage + /var/lib/docker sur le plus grand disque.
      - uses: ./.github/actions/docker-disk
        with:
          need: "40"

      # Le clone armbian/build (cache noyau, rootfs, image) sur le disque qui a
      # le plus de place : une image Armbian avec bureau en demande ~30 Go.
      - name: Repertoire Armbian
        run: |
          pick() { df -P "$1" 2>/dev/null | awk 'NR==2{print $4}'; }
          BASE=/; [ "$(pick /mnt || echo 0)" -gt "$(pick /)" ] 2>/dev/null && BASE=/mnt
          D="$BASE/armbian-build"
          sudo mkdir -p "$D"
          echo "ARMBIAN_DIR=$D" >> "$GITHUB_ENV"
          echo "armbian/build -> $D ($(df -h --output=avail "$BASE" | tail -1 | tr -d ' ') libres)"

      - name: Swap anti-OOM
        run: |
          pick() { df -P "$1" 2>/dev/null | awk 'NR==2{print $4}'; }
          AV_ROOT=$(pick /); AV_MNT=$(pick /mnt)
          BASE=/; [ -n "$AV_MNT" ] && [ "${AV_MNT:-0}" -gt "${AV_ROOT:-0}" ] && BASE=/mnt
          FREE_GB=$(( $(pick "$BASE") / 1024 / 1024 ))
          SWAP_GB=8; THIRD=$(( FREE_GB / 4 ))
          [ "$SWAP_GB" -gt "$THIRD" ] && SWAP_GB=$THIRD
          [ "$SWAP_GB" -lt 2 ] && SWAP_GB=2
          SF="$BASE/osmo-swapfile"
          sudo swapoff "$SF" 2>/dev/null || true; sudo rm -f "$SF"
          sudo fallocate -l ${SWAP_GB}G "$SF" 2>/dev/null \
            || sudo dd if=/dev/zero of="$SF" bs=1M count=$((SWAP_GB*1024)) status=none
          sudo chmod 600 "$SF"; sudo mkswap "$SF" >/dev/null; sudo swapon "$SF"
          echo "swap +${SWAP_GB} Go sur $BASE"

      # La base arm64 vient de GHCR (job docker-arm) : tiree ICI, sous le compte
      # runner qui porte les identifiants GHCR (build-iso.sh tourne sous sudo).
      - name: Log in to GHCR
        uses: docker/login-action@v4
        with:
          registry: ghcr.io
```

```
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}

    - name: Image de base arm64 depuis GHCR
      run: |
        REF='${{ needs.docker-arm.outputs.image }}'
        docker pull --platform linux/arm64 "$REF"
        echo "BASE_REF=$REF" >> "$GITHUB_ENV"
        echo "::notice::Base arm64 : $REF"

    # build-iso.sh --arm -> armbian/build-armbian.sh (iso_modules/00-options.sh).
    # --skip-build=REF : la pile vient de l'image tiree, rien n'est compile ici.
    - name: Build image Armbian (${{ matrix.board }})
      run: |
        chmod +x build-iso.sh armbian/build-armbian.sh
        ARGS="--arm --board=${{ matrix.board }} --skip-build=$BASE_REF"
        [ "${{ needs.boards.outputs.desktop }}" = "true" ] && ARGS="$ARGS --desktop"
        echo "::notice::build-iso.sh $ARGS"
        # sudo env ... : sudoers est en env_reset sans SETENV sur un runner.
        sudo env ARMBIAN_DIR="$ARMBIAN_DIR" ./build-iso.sh $ARGS
        df -h / /mnt || true

    # Le nom d'Armbian (Armbian-unofficial_<version>_<Carte>_noble_<branche>_<noyau>[_gnome_desktop].img)
    # devient un nom stable par carte, pour les sommes et la Release.
    - name: Renommer, sommes, compresser
      run: |
        IMG=$(ls -t "$ARMBIAN_DIR"/output/images/*.img 2>/dev/null | head -1)
        [ -n "$IMG" ] || { echo "::error::pas d image dans $ARMBIAN_DIR/output/images/"; exit 1; }
        SUF=""; [ "${{ needs.boards.outputs.desktop }}" = "true" ] && SUF="-desktop"
        OUT="osmo-operator-${{ matrix.board }}${SUF}.img"
        sudo mv "$IMG" "$OUT"; sudo chown "$(id -u):$(id -g)" "$OUT"
        echo "$(basename "$IMG") -> $OUT ($(du -h "$OUT" | cut -f1))"
        sha256sum "$OUT" > "SHA256SUMS-${{ matrix.board }}"
        md5sum    "$OUT" > "MD5SUMS-${{ matrix.board }}"
        zstd -T0 -9 --sparse --rm -q -o "$OUT.zst" "$OUT"
        ls -lh "$OUT.zst"

    # Une Release GitHub refuse tout fichier de plus de 2 Gio : on ne decoupe
    # QUE ce qui depasse. SHA256SUMS porte sur l'image brute.
    - name: Split over 2 GiB
      run: |
        LIMIT=$((2 * 1024 * 1024 * 1024))
        : > "PARTS-${{ matrix.board }}.sha256"
        for z in ./osmo-operator-*.img.zst; do
          [ -f "$z" ] || continue
          if [ "$(stat -c%s "$z")" -gt "$LIMIT" ]; then
            split -b 1900M -d -a 2 "$z" "${z}.part-"
            sha256sum "${z}".part-* >> "PARTS-${{ matrix.board }}.sha256"
            rm -f "$z"
          fi
        done
        ls -lh ./osmo-operator-*.img.zst* || true

    - name: Upload artifacts
      uses: actions/upload-artifact@v6
      with:
        name: osmo_egprs-${{ matrix.board }}
        path: |
          osmo-operator-*.img.zst*
          SHA256SUMS-*
          MD5SUMS-*
          PARTS-*.sha256
        retention-days: 14
        compression-level: 0

# ── La Release : toutes les cartes reussies, sommes fusionnees ──────────────
```

```
 release:
   needs: [boards, build]
   if: always() && needs.build.result != 'cancelled' && (startsWith(github.ref, 'refs/tags/') ||
↪  (github.event_name == 'workflow_dispatch' && github.event.inputs.release == 'true'))
   runs-on: ubuntu-24.04
   permissions:
     contents: write
   steps:
     - uses: actions/download-artifact@v5
       with:
         pattern: osmo_egprs-*
         merge-multiple: true
     - name: Fusionner les sommes
       run: |
         ls osmo-operator-*.img.zst* >/dev/null 2>&1 || { echo "::error::aucune image produite"; exit 1; }
         cat SHA256SUMS-* > SHA256SUMS; cat MD5SUMS-* > MD5SUMS; cat PARTS-*.sha256 > PARTS.sha256 2>/dev/null ||
↪  : > PARTS.sha256
         cat SHA256SUMS
     - name: Create GitHub Release
       uses: softprops/action-gh-release@v3
       with:
         # Pas github.ref_name tout seul : sur un workflow_dispatch il vaut « main »
         # et l'action creerait un TAG main, ambigu avec la branche.
         tag_name: ${{ startsWith(github.ref, 'refs/tags/') && github.ref_name || github.event.inputs.version ||
↪  'v0.1-9' }}
         name: "osmo_egprs — images SD arm64 (Armbian : Raspberry Pi 4, Orange Pi 5) — ${{ startsWith(github.ref,
↪  'refs/tags/') && github.ref_name || github.event.inputs.version || 'v0.1-9' }}"
         draft: false
         prerelease: ${{ !(startsWith(github.ref, 'refs/tags/v') || startsWith(github.event.inputs.version, 'v'))
↪  }}
         body: |
           ## osmo_egprs — images SD arm64 (Armbian)

           La meme pile que les ISO x86, construite pour aarch64 par le framework
           [armbian/build](https://github.com/armbian/build) (Ubuntu 24.04 noble).
           Une image **persistante** par carte, ${{ needs.boards.outputs.desktop == 'true' && 'avec le bureau
↪  GNOME du banc (fond GSM LAB, Conky, icones, `c123`)' || 'sans bureau (console et ssh)' }}.

           | Image | Carte |
           |---|---|
           | `osmo-operator-rpi4b*.img.zst` | Raspberry Pi 4 |
           | `osmo-operator-orangepi5*.img.zst` | Orange Pi 5 |
           | `osmo-operator-orangepi5b*.img.zst` | Orange Pi 5B (noyau edge) |
           | `osmo-operator-orangepi5-plus*.img.zst` | Orange Pi 5 Plus |
           | `osmo-operator-orangepi5pro*.img.zst` | Orange Pi 5 Pro |
           | `osmo-operator-orangepi5-max*.img.zst` | Orange Pi 5 Max |
           | `osmo-operator-orangepi5-ultra*.img.zst` | Orange Pi 5 Ultra |

           ### Graver

           ```bash
           zstd -dc osmo-operator-<carte>.img.zst \
             | sudo dd of=/dev/sdX bs=4M status=progress conv=fsync
           ```

           Au premier demarrage Armbian etend la racine a la taille de la carte.
           Comptes : `root:osmo`, `osmocom:osmo`.

           ### Verifier

           `SHA256SUMS` porte sur l'image **brute** : `zstd -d <image>.zst && sha256sum -c SHA256SUMS
↪ --ignore-missing`.
           Ce qui depasse 2 Gio est decoupe (`PARTS.sha256`) : `cat <image>.zst.part-* > <image>.zst`.

           ### Ce que ces images n'ont pas
```

```
          Pas de 4G (srsRAN et Open5GS ne sont pas compiles pour aarch64) ni de
          GLava (OpenGL >= 4.2 requis). La 2G/EDGE est entiere.
        files: |
          osmo-operator-*.img.zst*
          SHA256SUMS
          MD5SUMS
          PARTS.sha256
```

3.8 /opt/GSM/osmo-operator/.github/workflows/build-iso.yml

25285 octets, 462 lignes → 462 lignes

```yaml
name: Build ISO

# Trois images, une seule chaine :
#   interstp.iso            le hub SS7 (PC 0.0.0), aucun operateur
#   osmo-operator.iso       un noeud - UNE image pour les 9 : le numero se
#                           choisit au demarrage (start-native.sh --node N)
#   osmo-operator-lite.iso  le meme noeud, construit depuis l'image d'execution
#                           elaguee (Dockerfile.lite) : les ateliers de
#                           compilation de /opt/GSM ne partent pas dans le
#                           squashfs. Ce qui tourne est identique.
#
# Les trois dans UN SEUL job, et dans cet ordre : la lite se greffe sur l'image
# osmocom-run que la passe operateur vient de construire. Dans un job separe, le
# runner repartirait de zero - deux heures de compilation pour le meme resultat.
#
# --kb=us : build-iso.sh pose sa question du clavier AU DEBUT. Sans terminal il
# prend deja son defaut sans attendre - le job ne peut pas se bloquer dessus -
# mais on l'ecrit pour que l'image de CI ne dependent pas de ce defaut.

on:
  push:
    # [2026-10-01] "v*" et pas "v*.*.*" : les versions du depot s ecrivent
    # v0.1-6 (UN point) - l ancien motif ne declenchait jamais sur leur push.
    tags: ["v*", "iso-*"]
  workflow_dispatch:
    inputs:
      # [2026-10-18] Version de la release, en entree : on la choisit au lancement
      # au lieu de modifier le workflow. v0.1-8 est gardee en reserve : on passe
      # directement de v0.1-7 a v0.1-9 (la saisir ici si besoin).
      version:
        description: "Release tag (v0.1-8 kept in reserve)"
        type: string
        default: "v0.1-9"
      release:
        description: "Create GitHub Release"
        type: boolean
        default: true
      no_cache:
        description: "Rebuild Docker images without cache (--no-cache)"
        type: boolean
        default: true
      build_docker:
        description: "Construire l'image de base ici, au lieu de la tirer de GHCR (--build-docker)"
        type: boolean
        default: false
      mode:
        description: "Quelles images construire"
        type: choice
        default: all
        options:
          - all       # les quatre en une passe
          - stp       # le hub SS7 seul (interstp.iso)
```

```
          - base      # le noeud complet (osmo-operator.iso)
          - lite      # le noeud sans les ateliers (osmo-operator-lite.iso)
          - desktop   # le noeud avec GNOME (osmo-operator-desktop.iso)
      pmos_image:
        description: "Embed the postmarketOS VM image in the DESKTOP ISO only (~1.7 GB, from the pmos-image
↪  release)"
        type: boolean
        default: false

jobs:
  build-iso:
    # [2026-09-13] noble, PAS jammy. L'ISO et le Dockerfile sont en 24.04
    # (ISO_UBUNTU par defaut dans iso_modules/00-options.sh, FROM ubuntu:24.04),
    # mais le runner etait en 22.04 : l'hote du build etait donc d'une suite
    # differente de ce qu'il construit. Deux consequences concretes -
    #   - le debootstrap de jammy ne connait pas « noble » : 40-rootfs.sh s'en
    #     tire en liant le script gutsy, mais c'est un contournement ;
    #   - packaging/osmo-deb.sh lit VERSION_CODENAME de l'HOTE pour nommer les
    #     .deb : tout ce qui se compile cote hote sortait en « ~jammy », et un
    #     .deb jammy n'est jamais pose sur un rootfs noble (osmo-deb.sh:43).
    # Un runner noble aligne les trois.
    runs-on: ubuntu-24.04
    timeout-minutes: 360
    permissions:
      contents: write
      packages: read

    steps:
      - name: Checkout
        uses: actions/checkout@v5
        with:
          fetch-depth: 0

      # L'EMPREINTE DU CONTEXTE. La meme commande que docker.yml (job `base`) :
      # elle nomme l'image de base publiee sur GHCR, et elle sert de cle au
      # cache .deb. Calculee ici, une seule fois, parce que les deux en ont
      # besoin et qu'elles doivent parler de la meme chose.
      - name: Empreinte du contexte
        id: key
        run: |
          KEY=$(git ls-files -- Dockerfile packaging configs scripts services \
                                helpers opt-gsm tools patches \
                ; sort | xargs sha256sum | sha256sum | cut -c1-16)
          echo "key=$KEY" >> "$GITHUB_OUTPUT"
          echo "Empreinte du contexte : $KEY"

      # Les trois ISO plus les images Docker intermediaires tiennent tout juste
      # dans un runner : on fait la place avant, pas apres l'echec.
      - name: Free disk space
        run: |
          sudo rm -rf /usr/share/dotnet /usr/local/lib/android \
            /opt/ghc /opt/hostedtoolcache/CodeQL \
            /usr/local/share/boost /usr/share/swift \
            /usr/local/graalvm /usr/local/share/chromium \
            /usr/local/lib/node_modules
          sudo apt-get remove -y '^dotnet-.*' 'php.*' 'mongodb.*' \
            'mysql.*' azure-cli google-cloud-cli 2>/dev/null || true
          sudo apt-get autoremove -y
          sudo apt-get clean
          sudo docker image prune -af || true
          df -h / /mnt

      # [2026-09-13] Le job mourait en « exporting layers » sur exit 143 : pas
      # une erreur de docker (il n'ecrit rien), un SIGTERM recu de l'exterieur -
      # la partition / du runner arrivait a saturation et le runner lui-meme
      # tombait. Trois gros consommateurs partageaient / :
```

```
#   /var/lib/docker    base ~11 Go, et on en construit QUATRE variantes
#   /var/tmp/iso-build-$$   le rootfs decompresse (build-iso.sh:33)
#   le workspace       les quatre .iso
# Le runner a un SECOND disque, /mnt, ~70 Go, inutilise. On y met le store
# docker : le rootfs et les ISO gardent / pour eux seuls.
- name: Docker data-root sur /mnt
  run: |
    sudo systemctl stop docker docker.socket containerd || true
    # Deplacer, pas recopier : le store contient deja les images de base
    # tirees par le runner. Si /mnt/docker existe (re-run sur le meme
    # runner), on le garde plutot que d'ecraser un store valide.
    [ -d /mnt/docker ] || sudo mv /var/lib/docker /mnt/docker
    echo '{ "data-root": "/mnt/docker" }' | sudo tee /etc/docker/daemon.json
    sudo systemctl start docker
    docker info 2>/dev/null | grep -i 'docker root dir'
    df -h / /mnt

# Un exit 143 ne dit rien de ce qui manquait. Ce temoin ecrit disque et
# memoire toutes les 30 s dans un fichier relu en fin de job (if: always),
# y compris quand l'etape de build est tuee sans un mot.
- name: Temoin disque/memoire
  run: |
    nohup bash -c 'while true; do
      printf "[%s] / %s  /mnt %s  mem %s\n" \
        "$(date -u +%H:%M:%S)" \
        "$(df -h --output=avail / | tail -1 | tr -d " ")" \
        "$(df -h --output=avail /mnt | tail -1 | tr -d " ")" \
        "$(free -m | awk "/^Mem:/{print \$3\"/\"\$2\" Mo\"}")"
      sleep 30
    done' > "$RUNNER_TEMP/watchdog.log" 2>&1 &
    echo "temoin demarre -> $RUNNER_TEMP/watchdog.log"

# ── Le cache .deb entre deux runs ──────────────────────────────
# Ce que le Dockerfile compile (libosmocore, osmo-*, gnuradio, gr-gsm,
# QEMU, osmocom-bb, pmbootstrap...) sort en paquet .deb dans
# /var/cache/osmo-debs (packaging/osmo-deb.sh). build.sh recopie ce
# dossier dans .deb-cache/ AVANT le build - le Dockerfile le prend par
# `COPY .deb-cache/ /var/cache/osmo-debs/` - et y remet APRES les paquets
# que l'image vient de produire (debs_from_image). Sur l'hote c'est ce
# qui fait passer un rebuild de 1h23 a quelques minutes ; un runner,
# lui, repart vierge a chaque fois. On lui rend la memoire.
#
# Le dossier appartient a root (build.sh tourne sous sudo) : il faut le
# creer AVANT que actions/cache y ecrive, et le rendre au runner APRES
# le build pour que le post-step puisse le relire.
# SUR /mnt, PAS SUR /. Ce job est le seul des deux ou / est la partition
# serree : le rootfs decompresse, le squashfs et les quatre ISO y tiennent
# tout juste. Les 2-4 Go du cache .deb iraient s'ajouter a ca, alors que
# /mnt en a 68 de libres. build.sh, build-iso.sh et 50-injection-image.sh
# lisent tous OSMO_DEB_CACHE : il suffit de le leur dire.
# Repli sur / si le deplacement de docker plus haut a renonce a /mnt.
- name: Preparer le cache .deb
  run: |
    # Pas de repli conditionnel : `path:` du step suivant est statique,
    # un chemin qui divergerait ici ne serait pas celui qu'on sauvegarde.
    # /mnt existe sur tout runner heberge ; ailleurs, mkdir -p le cree sur
    # / et on retombe simplement sur l'ancien comportement.
    CACHE=/mnt/osmo-debs
    sudo mkdir -p "$CACHE"
    sudo chown -R "$(id -u):$(id -g)" "$CACHE"
    echo "OSMO_DEB_CACHE=$CACHE" >> "$GITHUB_ENV"
    echo "cache .deb : $CACHE"

- name: Cache .deb (osmo-deb)
  uses: actions/cache@v5
  with:
```

```
    path: /mnt/osmo-debs
    # CLE = L'EMPREINTE DU CONTEXTE, pas le numero du run.
    # [2026-09-13] C'etait `${{ github.run_id }}`, donc une entree NEUVE a
    # chaque passage. A 2-4 Go l'entree et 10 Go de quota par depot,
    # l'eviction LRU detruisait le cache au bout de trois runs. Les .deb
    # portent leur nom et leur version, osmo-deb.sh trie : le prefixe
    # reprend la plus recente, et ce qui n'a pas bouge reste valable.
    # [2026-09-14] LA GENERATION (-g2-) EXISTE POUR JETER UN CACHE DEVENU
    # FAUX. `restore-keys` repeche la derniere entree du prefixe meme
    # quand la cle exacte a change : un osmo-build-srsran compile en
    # -march=native AVANT le 14/09 revenait donc a chaque run, passait le
    # `osmo-deb install srsran`, et l ISO repartait avec un srsenb qui
    # fait « Illegal instruction » ailleurs - jusqu a ce que le controle
    # objdump de iso_modules/88-lte-pmos.sh l ecarte et recompile une
    # heure, A CHAQUE FOIS. La version des .deb porte desormais son ISA
    # (tools/osmo-lte-install.sh, srs_deb_version), ce qui empeche la
    # confusion pour de bon ; ce marqueur coupe en plus le lien avec les
    # entrees d avant. L INCREMENTER quand un cache doit etre abandonne.
    key: osmo-debs-noble-g2-${{ steps.key.outputs.key }}
    restore-keys: |
      osmo-debs-noble-g2-

- name: Log in to GHCR
  uses: docker/login-action@v4
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}

# ── L'image de base vient de GHCR quand elle y est deja ──────────────
# Le workflow Docker (docker.yml) publie osmocom-nitb:base-<empreinte>,
# l'empreinte etant le SHA de tout ce que le Dockerfile prend dans le
# contexte. MEME COMMANDE ici, au caractere pres : si elle donne la meme
# valeur, l'image publiee correspond exactement a ce commit, et il n'y a
# aucune raison de recompiler 1h23 pour la refabriquer a l'identique.
# build-iso.sh la tague osmocom-nitb et la chaine continue sans rien
# savoir de sa provenance (--skip-build, iso_modules/21-docker-build.sh).
#
# Le pull se fait ICI, sous le compte runner qui porte les identifiants
# GHCR - build-iso.sh tourne sous sudo, et le ~/.docker de root n'en a
# pas. L'image etant alors locale, le module ne retire rien.
- name: Image de base (Docker Hub, puis GHCR)
  run: |
    if [ "${{ github.event.inputs.build_docker }}" = "true" ]; then
      echo "ISO_DOCKER_SRC=--build-docker" >> "$GITHUB_ENV"
      echo "::notice::--build-docker demande : l'image de base sera construite ici."
      exit 0
    fi
    KEY='${{ steps.key.outputs.key }}'
    REPO=$(echo '${{ github.repository }}' | tr '[:upper:]' '[:lower:]')
    # [2026-10-10] ORDRE : Docker Hub (bastienbaranoff/norf_gsm:latest)
    # d'abord, l'image GHCR de l'empreinte ensuite, la compilation en
    # dernier. L'image retenue est taguee osmocom-nitb.
    HUB="bastienbaranoff/norf_gsm:latest"
    GHCR="ghcr.io/${REPO}/osmocom-nitb:base-${KEY}"
    REF=""
    if docker pull "$HUB"; then
      REF="$HUB"
      echo "::notice::Base tiree de Docker Hub : $HUB - ne suit pas forcement ce commit."
    elif docker manifest inspect "$GHCR" >/dev/null 2>&1 && docker pull "$GHCR"; then
      REF="$GHCR"
      echo "::notice::Docker Hub indisponible, base tiree de GHCR : $GHCR - pas de compilation."
    fi
    if [ -n "$REF" ]; then
      docker tag "$REF" osmocom-nitb
      echo "ISO_DOCKER_SRC=--skip-build=$REF" >> "$GITHUB_ENV"
```

```
      # Les .deb de l'image rejoignent le cache de l'hote : c'est la que
      # 50-injection-image.sh va les chercher pour poser la pile dans le
      # rootfs (il sait retomber sur l'image, mais un dpkg vaut mieux
      # qu'un docker cp integral).
      mkdir -p /tmp/debs-out
      cid=$(docker create "$REF" /bin/true)
      docker cp "$cid:/var/cache/osmo-debs/." /tmp/debs-out/ 2>/dev/null || true
      docker rm "$cid" >/dev/null
      cp -f /tmp/debs-out/osmo-build-*.deb "$OSMO_DEB_CACHE/" 2>/dev/null || true
      rm -rf /tmp/debs-out
      echo "cache .deb : $(find "$OSMO_DEB_CACHE" -name 'osmo-build-*.deb' | wc -l) paquets"
    else
      # --build-docker EXPLICITE, et pas une chaine vide. [2026-09-14]
      # build-iso.sh tire desormais par defaut, et son dernier recours est
      # l'image Docker Hub, qui ne suit pas le commit : laisser vide
      # publierait ici une ISO batie sur une pile qui n'est pas celle de
      # ce commit, sans que rien ne le dise. Quand l'empreinte n'a pas
      # d'image, la seule reponse juste est de compiler.
      echo "ISO_DOCKER_SRC=--build-docker" >> "$GITHUB_ENV"
      echo "::notice::Ni Docker Hub ni base GHCR pour l'empreinte $KEY - construction locale."
    fi
    df -h / /mnt

# Les paquets hote (squashfs-tools, xorriso, grub*, mtools, debootstrap...)
# sont installes par build-iso.sh lui-meme.

# Le hub d'abord : il ne depend ni de build.sh ni de l'image osmocom-run,
# donc un echec de son cote se voit en minutes plutot qu'en heures.
# --all : interstp + operator + operator-lite + operator-desktop en UNE
# passe. build-iso.sh greffe lite et desktop sur l'image osmocom-run que la
# passe operateur vient de produire, et retire --no-cache pour ces passes
# greffees : inutile de separer en plusieurs etapes. On rend le cache buildx
# avant - QUATRE ISO plus les images intermediaires, le disque du runner
# decide si le job passe.
- name: Build ISOs
  run: |
    chmod +x build-iso.sh
    df -h /
    # ── QUELLES IMAGES ──────────────────────────────────────────
    # Sur un push de tag les entrees de workflow_dispatch sont VIDES :
    # le defaut reste "all", c'est-a-dire le comportement d'avant ce
    # choix. Les autres modes servent a reprendre UNE image sans
    # repayer les trois autres - un theme plymouth a revoir, un
    # installeur a retester, le hub qui sort en minutes.
    #
    # CE QUE COUTE UN MODE SEUL : hors "all", build-iso.sh n'a pas de
    # rootfs frere a reprendre (le greffage lite/desktop sur le rootfs
    # de la normale n'existe que dans la passe --all). Chaque image
    # refait donc son rootfs de zero. Pour les quatre, "all" reste
    # nettement moins cher que quatre runs.
    case "${{ github.event.inputs.mode }}" in
      ""|all)  ISO_ARGS="--all" ;;
      stp)     ISO_ARGS="--role=interstp" ;;
      base)    ISO_ARGS="--role=operator --output=osmo-operator.iso" ;;
      lite)    ISO_ARGS="--role=operator --lite --output=osmo-operator-lite.iso" ;;
      desktop) ISO_ARGS="--role=operator --desktop --output=osmo-operator-desktop.iso" ;;
      *) echo "::error::mode inconnu : ${{ github.event.inputs.mode }}"; exit 2 ;;
    esac
    echo "::notice::mode ${{ github.event.inputs.mode || 'all' }} -> build-iso.sh $ISO_ARGS"
    # [2026-09-13] DEFAUT INVERSE. C'etait NO_CACHE="--no-cache" avec un
    # repli sur "" quand l'entree valait "false" - mais sur un push les
    # entrees de workflow_dispatch sont VIDES, jamais "false" : chaque
    # push forcait donc --no-cache, donc OSMO_DEB_REFRESH=1, donc les 51
    # RUN de compilation refaits integralement. 1h23 a chaque fois, pour
    # des sources qui n'avaient pas bouge. Le plein rebuild reste
    # accessible, mais il se demande.
```

```
        NO_CACHE=""
        [ "${{ github.event.inputs.no_cache }}" = "true" ] && NO_CACHE="--no-cache"
        # L'image de la VM postmarketOS (qemu-amd64.img.zst, 1,7 Go) n'existe
        # pas sur un runner : 88-lte-pmos.sh la telecharge depuis l'asset de
        # Release quand on lui donne l'URL. [2026-10-09] Elle n'est embarquee
        # que dans l'ISO desktop (88-lte-pmos.sh gate sur ISO_DESKTOP), pas
        # dans interstp, operator ni lite. Sans elle, le premier clic sur le
        # telephone fait un "pmbootstrap install" complet.
        # sudo env ... : sur un runner, sudoers est en env_reset sans SETENV,
        # « sudo -E » et « sudo VAR=x » peuvent etre refuses ; env(1) lance en
        # root, lui, passe toujours.
        sudo env OSMO_PMOS_IMAGE_URL="$OSMO_PMOS_IMAGE_URL" \
                 OSMO_HOST_DOCKER="$OSMO_HOST_DOCKER" \
                 OSMO_DEB_CACHE="$OSMO_DEB_CACHE" \
             ./build-iso.sh $NO_CACHE $ISO_DOCKER_SRC --kb=us $ISO_ARGS
        # `|| true` : hors mode "all" les autres .iso n'existent pas, et un
        # ls qui ne trouve pas un de ses arguments sort en statut 2 - le
        # shell des steps tourne avec -e, la derniere commande decide.
        ls -lh ./*.iso 2>/dev/null || true
        df -h /
      env:
        OSMO_PMOS_IMAGE_URL: ${{ github.event.inputs.pmos_image == 'true' && (vars.PMOS_IMAGE_URL ||
↪ 'https://github.com/bbaranoff/osmo-operator/releases/download/pmos-image/qemu-amd64.img.zst') || '' }}
        # Le runner a deja docker, compose v2 et buildx (paquets moby-*), et
        # docker.io entre en conflit avec eux : apt repond « pkgProblemResolver
        # generated breaks » et build.sh s'arretait la. Rien a installer ici.
        OSMO_HOST_DOCKER: "0"

    # build.sh a tourne sous sudo : les .deb qu'il vient de deposer sont a
    # root, et le post-step d'actions/cache tourne en runner. Sans ce chown,
    # la sauvegarde du cache echoue en silence et le run suivant recompile.
    - name: Rendre le cache .deb au runner
      if: always()
      run: |
        CACHE="${OSMO_DEB_CACHE:-/var/cache/osmo-debs}"
        sudo chown -R "$(id -u):$(id -g)" "$CACHE" 2>/dev/null || true
        echo "$(find "$CACHE" -name 'osmo-build-*.deb' | wc -l) paquets, $(du -sh "$CACHE" | cut -f1)"

    # Apres le build, tue ou non : les dernieres mesures avant la fin.
    - name: Temoin - dernieres mesures
      if: always()
      run: tail -40 "$RUNNER_TEMP/watchdog.log" 2>/dev/null || echo "(pas de temoin)"

    - name: Checksums
      run: |
        # Aucune image n'est tenue pour acquise : avec --role=... une seule
        # est produite, et nommer les trois autres ferait echouer sha256sum.
        ISOS=""
        for iso in interstp.iso osmo-operator.iso \
                   osmo-operator-lite.iso osmo-operator-desktop.iso; do
            [ -f "$iso" ] && ISOS="$ISOS $iso"
        done
        [ -n "$ISOS" ] || { echo "::error::aucune ISO produite"; exit 1; }
        sha256sum $ISOS > SHA256SUMS
        md5sum    $ISOS > MD5SUMS
        cat SHA256SUMS

    # Une Release GitHub refuse tout fichier de plus de 2 Gio. On ne decoupe
    # QUE ce qui depasse : decouper une image de 1,3 Go obligerait a la
    # recombiner pour rien. Les sommes ci-dessus portent sur les ISO ENTIERES,
    # elles restent donc verifiables apres recombinaison.
    - name: Split ISOs over 2 GiB
      run: |
        LIMIT=$((2 * 1024 * 1024 * 1024))
        for iso in interstp.iso osmo-operator.iso osmo-operator-lite.iso osmo-operator-desktop.iso; do
          [ -f "$iso" ] || continue
```

```
          size=$(stat -c%s "$iso")
          if [ "$size" -gt "$LIMIT" ]; then
            echo "$iso : $size octets > 2 Gio -> decoupe"
            split -b 1900M -d -a 2 "$iso" "${iso}.part-"
            sha256sum "${iso}".part-* >> PARTS.sha256
            rm -f "$iso"          # seules les parties sont publiees
          else
            echo "$iso : $size octets -> publie tel quel"
          fi
        done
        touch PARTS.sha256
        ls -lh ./*.iso ./*.iso.part-* 2>/dev/null || true

    - name: Upload artifacts
      uses: actions/upload-artifact@v6
      with:
        name: osmo_egprs-iso
        path: |
          interstp.iso*
          osmo-operator.iso*
          osmo-operator-lite.iso*
          osmo-operator-desktop.iso*
          SHA256SUMS
          MD5SUMS
        retention-days: 14
        compression-level: 0

    - name: Create GitHub Release
      if: startsWith(github.ref, 'refs/tags/') || (github.event_name == 'workflow_dispatch' &&
↪ github.event.inputs.release == 'true')
      uses: softprops/action-gh-release@v3
      with:
        # [2026-09-14] PAS github.ref_name tout seul. Sur un workflow_dispatch
        # depuis la branche, ref_name vaut « main » : l'action creait donc un
        # TAG nomme « main », a cote de la branche du meme nom. D'ou le
        # « warning: refname 'main' is ambiguous » que git sort ensuite a
        # chaque commande - il ne sait plus si « main » designe la branche ou
        # le tag - et un release qui ne disait pas quelle version il portait.
        # Un push de tag garde son propre tag ; tout le reste sort sous la
        # version saisie au lancement (entree version, defaut ci-dessous).
        tag_name: ${{ startsWith(github.ref, 'refs/tags/') && github.ref_name || github.event.inputs.version ||
↪ 'v0.1-9' }}
        # [2026-09-25] Le nom et le texte sont ceux des releases publiees a la
        # main (« Osmo-Operator v0.1-3 ») : le texte vit dans
        # .github/release-notes.md, qui se modifie sans toucher au workflow.
        name: "Osmo-Operator ${{ startsWith(github.ref, 'refs/tags/') && github.ref_name ||
↪ github.event.inputs.version || 'v0.1-9' }}"
        # [2026-10-09] Brouillon le temps des envois : sans cela la release
        # devient publique avec quelques fichiers seulement (un timeout d'upload
        # laissait ainsi v0.1-7 a moitie complete). Publiee par l'etape suivante.
        draft: true
        prerelease: ${{ !(startsWith(github.ref, 'refs/tags/v') || startsWith(github.event.inputs.version, 'v'))
↪ }}
        body_path: .github/release-notes.md

    # [2026-10-09] softprops envoyait tous les fichiers en parallele ; les
    # parties de 1 a 1,7 Go coupaient avec « Upload body timed out due to
    # inactivity ». Ici : un fichier a la fois, relances sur echec.
    - name: Upload release assets (one by one, retried)
      if: startsWith(github.ref, 'refs/tags/') || (github.event_name == 'workflow_dispatch' &&
↪ github.event.inputs.release == 'true')
      env:
        GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        TAG: ${{ startsWith(github.ref, 'refs/tags/') && github.ref_name || github.event.inputs.version ||
↪ 'v0.1-9' }}
      run: |
```

```
          set -u
          shopt -s nullglob
          files=(interstp.iso* osmo-operator.iso* osmo-operator-lite.iso* osmo-operator-desktop.iso* SHA256SUMS
↪  MD5SUMS PARTS.sha256)
          for f in "${files[@]}"; do
            [ -f "$f" ] || { echo "absent: $f"; continue; }
            ok=0
            for attempt in 1 2 3 4 5; do
              echo "== $f (tentative $attempt)"
              if gh release upload "$TAG" "$f" --clobber; then ok=1; break; fi
              sleep $((attempt * 20))
            done
            [ "$ok" = 1 ] || { echo "ECHEC definitif : $f"; exit 1; }
          done

      - name: Publish release
        if: startsWith(github.ref, 'refs/tags/') || (github.event_name == 'workflow_dispatch' &&
↪  github.event.inputs.release == 'true')
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          TAG: ${{ startsWith(github.ref, 'refs/tags/') && github.ref_name || github.event.inputs.version ||
↪  'v0.1-9' }}
          PRE: ${{ !(startsWith(github.ref, 'refs/tags/v') || startsWith(github.event.inputs.version, 'v')) }}
        run: |
          gh release edit "$TAG" --draft=false --prerelease="$PRE"
```

3.9 /opt/GSM/osmo-operator/.github/workflows/docker-arm.yml

7661 octets, 191 lignes → 191 lignes

```
name: Docker arm64 (GHCR)

# L'image de BASE (Dockerfile -> osmocom-nitb) construite NATIVEMENT en aarch64
# et publiee sur GHCR sous osmocom-nitb:base-arm64-<empreinte>. C'est le pendant
# arm64 du job `base` de docker.yml (qui ne fait que de l'amd64).
#
# Pourquoi un workflow a part, en amont de build-arm.yml : jusqu'ici l'image SD
# compilait toute la pile DANS son propre job (« rien n'est publie en arm64 »,
# donc pas de --skip-build) - c'etait l'essentiel des 6 h. Ici la compilation
# sort dans un job dedie, avec ses caches (couches via :cache-arm64, .deb via
# actions/cache), et build-arm.yml n'a plus qu'a TIRER l'image. Une base
# inchangee (meme empreinte) n'est pas reconstruite.
#
# L'empreinte est la MEME commande que docker.yml / build-iso.yml : memes
# fichiers, donc memes contenus. Le prefixe `base-arm64-` evite qu'elle
# collisionne avec la base amd64 du meme commit.

on:
  workflow_call:
    inputs:
      rebuild:
        description: "Reconstruire meme si l'empreinte existe deja"
        type: boolean
        default: false
      no_cache:
        description: "Sans cache de couches (--no-cache)"
        type: boolean
        default: false
    outputs:
      image:
        description: "Reference GHCR de la base arm64"
        value: ${{ jobs.base-arm64.outputs.image }}
  workflow_dispatch:
    inputs:
```

```
      rebuild:
        description: "Reconstruire meme si l'empreinte existe deja"
        type: boolean
        default: false
      no_cache:
        description: "Sans cache de couches (--no-cache)"
        type: boolean
        default: false

env:
  REGISTRY: ghcr.io

jobs:
  base-arm64:
    runs-on: ubuntu-24.04-arm
    timeout-minutes: 360
    permissions:
      contents: read
      packages: write
    outputs:
      image: ${{ steps.key.outputs.image }}
      hit:   ${{ steps.probe.outputs.hit }}
    steps:
      - name: Checkout
        uses: actions/checkout@v5

      - name: Empreinte du contexte
        id: key
        run: |
          REPO=$(echo '${{ github.repository }}' | tr '[:upper:]' '[:lower:]')
          KEY=$(git ls-files -- Dockerfile packaging configs scripts services \
                                helpers opt-gsm tools patches \
                | sort | xargs sha256sum | sha256sum | cut -c1-16)
          echo "key=$KEY" >> "$GITHUB_OUTPUT"
          echo "image=${REGISTRY}/${REPO}/osmocom-nitb:base-arm64-${KEY}" >> "$GITHUB_OUTPUT"
          echo "cache=${REGISTRY}/${REPO}/osmocom-nitb:cache-arm64"       >> "$GITHUB_OUTPUT"
          echo "Empreinte du contexte : $KEY"

      - name: Log in to GHCR
        uses: docker/login-action@v4
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: La base arm64 existe-t-elle deja ?
        id: probe
        run: |
          HIT=false
          if [ "${{ inputs.rebuild }}" != "true" ] \
             && docker manifest inspect '${{ steps.key.outputs.image }}' >/dev/null 2>&1; then
            HIT=true
          fi
          echo "hit=$HIT" >> "$GITHUB_OUTPUT"
          if [ "$HIT" = true ]; then
            echo "::notice::Base arm64 inchangee, deja publiee : ${{ steps.key.outputs.image }}"
          else
            echo "::notice::Base arm64 absente ou forcee : construction native."
          fi

      - name: Disque pour docker
        if: steps.probe.outputs.hit != 'true'
        uses: ./.github/actions/docker-disk
        with:
          need: "40"
```

```
    - name: Buildx
      if: steps.probe.outputs.hit != 'true'
      uses: docker/setup-buildx-action@v4

    - name: Preparer le cache .deb
      if: steps.probe.outputs.hit != 'true'
      run: |
        sudo mkdir -p /var/cache/osmo-debs
        sudo chown -R "$(id -u):$(id -g)" /var/cache/osmo-debs

    # Cle distincte de celle de build-arm.yml (osmo-debs-noble-arm64-) : meme
    # contenu, mais ce job est desormais celui qui l'alimente.
    - name: Cache .deb (osmo-deb, arm64)
      if: steps.probe.outputs.hit != 'true'
      uses: actions/cache@v5
      with:
        path: /var/cache/osmo-debs
        key: osmo-debs-noble-arm64-${{ steps.key.outputs.key }}
        restore-keys: |
          osmo-debs-noble-arm64-

    # Meme filet anti-OOM que build-arm.yml : buildkit batit plusieurs stages
    # a la fois, le pic cumule depasse la RAM du runner arm64.
    - name: Swap anti-OOM
      if: steps.probe.outputs.hit != 'true'
      run: |
        pick() { df -P "$1" 2>/dev/null | awk 'NR==2{print $4}'; }
        AV_ROOT=$(pick /); AV_MNT=$(pick /mnt)
        BASE=/; [ -n "$AV_MNT" ] && [ "${AV_MNT:-0}" -gt "${AV_ROOT:-0}" ] && BASE=/mnt
        FREE_GB=$(( $(pick "$BASE") / 1024 / 1024 ))
        SWAP_GB=12; THIRD=$(( FREE_GB / 3 ))
        [ "$SWAP_GB" -gt "$THIRD" ] && SWAP_GB=$THIRD
        [ "$SWAP_GB" -lt 2 ] && SWAP_GB=2
        SF="$BASE/osmo-swapfile"
        sudo swapoff "$SF" 2>/dev/null || true; sudo rm -f "$SF"
        sudo fallocate -l ${SWAP_GB}G "$SF" 2>/dev/null \
          || sudo dd if=/dev/zero of="$SF" bs=1M count=$((SWAP_GB*1024)) status=none
        sudo chmod 600 "$SF"; sudo mkswap "$SF" >/dev/null; sudo swapon "$SF"
        echo "swap +${SWAP_GB} Go sur $BASE"

    - name: Temoin disque/memoire
      if: steps.probe.outputs.hit != 'true'
      run: |
        nohup bash -c 'while true; do
          echo "$(date -u +%H:%M:%S)  / $(df -h --output=avail / | tail -1 | tr -d " ")  /mnt $(df -h
↪  --output=avail /mnt 2>/dev/null | tail -1 | tr -d " ")  mem $(free -m | awk "/^Mem:/{print \$7}")Mo"
          sleep 30
        done' > "$RUNNER_TEMP/watchdog.log" 2>&1 &

    # Hote aarch64 : buildx construit pour la plateforme native, sans
    # --platform ni qemu. --push comme docker.yml : rien dans le store local.
    - name: Build et push osmocom-nitb (base arm64)
      if: steps.probe.outputs.hit != 'true'
      env:
        IMAGE: ${{ steps.key.outputs.image }}
        CACHE: ${{ steps.key.outputs.cache }}
        NO_CACHE: ${{ inputs.no_cache }}
      run: |
        mkdir -p .deb-cache
        cp -f /var/cache/osmo-debs/osmo-build-*_arm64.deb .deb-cache/ 2>/dev/null || true
        echo "cache .deb en entree : $(find .deb-cache -name 'osmo-build-*.deb' | wc -l) paquets"

        CACHE_ARGS=(--cache-from "type=registry,ref=$CACHE")
        REFRESH=0
        if [ "$NO_CACHE" = true ]; then CACHE_ARGS=(--no-cache); REFRESH=1; fi
```

```
          docker buildx build \
            --platform linux/arm64 \
            --target osmocom-nitb \
            --build-arg "OSMO_DEB_REFRESH=$REFRESH" \
            "${CACHE_ARGS[@]}" \
            --cache-to type=inline \
            -t "$IMAGE" -t "$CACHE" --push \
            -f Dockerfile .

          # Recolte des .deb : seconde passe sur `debs`, tout en cache.
          rm -rf /tmp/debs-out && mkdir -p /tmp/debs-out
          docker buildx build \
            --platform linux/arm64 \
            --target debs \
            --build-arg "OSMO_DEB_REFRESH=$REFRESH" \
            --cache-from "type=registry,ref=$CACHE" \
            --output "type=local,dest=/tmp/debs-out" \
            -f Dockerfile .
          cp -f /tmp/debs-out/osmo-build-*.deb /var/cache/osmo-debs/ 2>/dev/null || true
          rm -rf /tmp/debs-out .deb-cache/*.deb
          echo "cache .deb en sortie : $(find /var/cache/osmo-debs -name 'osmo-build-*.deb' | wc -l) paquets"
          df -h / /mnt || true

      - name: Temoin - dernieres mesures
        if: always()
        run: tail -40 "$RUNNER_TEMP/watchdog.log" 2>/dev/null || echo "(pas de temoin)"
```

3.10 /opt/GSM/osmo-operator/.github/workflows/docker.yml

19026 octets, 411 lignes → 411 lignes

```
name: Docker (GHCR)

# Quatre images publiees :
#
#   osmocom-nitb        l'image de BASE (Dockerfile) : toute la compilation,
#                       ~11 Go. Elle n'est PLUS rebatie a chaque push - voir le
#                       job `base` plus bas.
#   interstp            le hub SS7 seul (Dockerfile.stp) - osmo-stp et sa table
#                       de routage, rien d'autre. Independante : elle ne part
#                       pas de osmocom-nitb, donc elle sort en minutes.
#   osmo-operator       un noeud complet (Dockerfile.run, qui part de
#                       osmocom-nitb) : HLR, MSC, BSC, PCU, BTS, Asterisk, QEMU.
#   osmo-operator-lite  la meme pile, sans les arbres de sources de /opt/GSM
#                       (Dockerfile.lite) - ~4 Go de moins sur 11.
#
# ── Pourquoi cette structure ──────────────────────────────────────────
# [2026-09-13] Avant, un seul job enchainait les trois builds sur un runner, et
# recompilait donc l'image de base a chaque push : 1h23, plus un disque sature
# en « exporting layers » (exit 143, un SIGTERM du runner a bout d'espace).
# Maintenant :
#   1. `base` calcule une EMPREINTE de tout ce que le Dockerfile prend dans le
#      contexte. Si ghcr.io/<depot>/osmocom-nitb:base-<empreinte> existe deja,
#      il ne construit RIEN et se contente de la publier comme reference.
#   2. Sinon il construit, avec deux caches : les couches de la derniere base
#      publiee (--cache-from registry + --cache-to inline) et les .deb deja
#      compiles (actions/cache sur /var/cache/osmo-debs, le mecanisme de
#      packaging/osmo-deb.sh). Un changement en fin de Dockerfile ne recompile
#      plus que sa queue.
#   3. `operator` et `lite` partent de cette image, en parallele, chacun sur
#      son runner - donc chacun avec un disque propre.
#
# ── Et pourquoi buildx --push partout ─────────────────────────────────
# [2026-09-13] Les trois jobs poussent leur image SANS jamais la charger dans
```

```yaml
# le docker du runner. `docker build -t X . && docker push X` demande a
# buildkit d'exporter l'image dans le store local : il detient alors ses
# snapshots ET ecrit par-dessus les archives de couches, soit deux fois 11 Go.
# C'est la, en « exporting layers » - build reussi, 75 etapes sur 75 - que le
# runner saturait et se faisait tuer (exit 143). Avec
# `docker buildx build --push`, les couches sont compressees et poussees au
# fil de l'eau ; il ne reste sur le disque que le cache du builder.
#
# Consequence a ne pas perdre de vue : plus aucune image locale apres le
# build, donc plus de `docker create` + `docker cp` pour recuperer les .deb
# compiles. C'est le role de l'etape `debs` du Dockerfile (une seconde passe
# `--target debs --output type=local`, tout en cache). Et parce que `debs` est
# desormais la DERNIERE etape du Dockerfile, tout build de la base doit dire
# `--target osmocom-nitb` : ici, dans build.sh, dans compose.yaml et dans
# iso_modules/21-docker-build.sh.
#
# Dockerfile.run et Dockerfile.lite ont un `ARG BASE` : en local ils valent
# toujours `osmocom-nitb` (l'image du docker de l'hote), en CI ils recoivent la
# reference GHCR. build.sh n'est pas concerne.
#
# Sur une pull_request, rien n'est publie (le GITHUB_TOKEN d'un fork n'a pas
# packages:write) : `base` construit pour valider le Dockerfile, et les deux
# jobs derives ne tournent pas.

on:
  push:
    branches: ["main"]
    tags: ["v*.*.*"]
  pull_request:
    branches: ["main"]
  workflow_dispatch:
    inputs:
      rebuild_base:
        description: "Reconstruire l'image de base meme si l'empreinte existe deja"
        type: boolean
        default: false

env:
  REGISTRY: ghcr.io

jobs:
  # ── L'image de base : construite seulement si son contexte a bouge ──────────
  base:
    runs-on: ubuntu-24.04
    timeout-minutes: 360
    permissions:
      contents: read
      packages: write
      id-token: write
    outputs:
      repo:  ${{ steps.img.outputs.repo }}
      image: ${{ steps.key.outputs.image }}
      hit:   ${{ steps.probe.outputs.hit }}
    steps:
      - name: Checkout
        uses: actions/checkout@v5

      # GHCR n'accepte que des noms d'image en minuscules ; github.repository
      # garde la casse du proprietaire.
      - name: Image name
        id: img
        run: echo "repo=$(echo '${{ github.repository }}' | tr '[:upper:]' '[:lower:]')" >> "$GITHUB_OUTPUT"

      # L'EMPREINTE. Tout ce que Dockerfile prend dans le contexte de build -
      # lui-meme, packaging/, patches/, et les dossiers qu'il COPY (configs,
      # scripts, services, helpers, opt-gsm, tools). Pas .deb-cache/ : c'est un
```

```
# cache, son contenu ne definit pas l'image.
# `git ls-files` et non `find` : ce qui n'est pas versionne n'entre pas
# dans l'image de toute facon, et l'ordre est stable.
- name: Empreinte du contexte de base
  id: key
  run: |
    KEY=$(git ls-files -- Dockerfile packaging configs scripts services \
                          helpers opt-gsm tools patches \
          | sort | xargs sha256sum | sha256sum | cut -c1-16)
    REPO='${{ steps.img.outputs.repo }}'
    echo "key=$KEY"                                              >> "$GITHUB_OUTPUT"
    echo "image=${REGISTRY}/${REPO}/osmocom-nitb:base-${KEY}"    >> "$GITHUB_OUTPUT"
    echo "cache=${REGISTRY}/${REPO}/osmocom-nitb:cache"          >> "$GITHUB_OUTPUT"
    echo "Empreinte du contexte : $KEY"

- name: Log in to GHCR
  if: github.event_name != 'pull_request'
  uses: docker/login-action@v4
  with:
    registry: ${{ env.REGISTRY }}
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}

# Le manifeste suffit : pas besoin de tirer 11 Go pour savoir si l'image
# est la. Un `rebuild_base` coche force la reconstruction.
- name: L'image de base existe-t-elle deja ?
  id: probe
  run: |
    HIT=false
    if [ "${{ github.event.inputs.rebuild_base }}" != "true" ] \
       && [ "${{ github.event_name }}" != "pull_request" ] \
       && docker manifest inspect '${{ steps.key.outputs.image }}' >/dev/null 2>&1; then
      HIT=true
    fi
    echo "hit=$HIT" >> "$GITHUB_OUTPUT"
    if [ "$HIT" = true ]; then
      echo "::notice::Base inchangee, deja publiee : ${{ steps.key.outputs.image }} - rien a construire."
    else
      echo "::notice::Base absente ou forcee : construction."
    fi

# ── A partir d'ici, uniquement quand il faut vraiment construire ────────

# Menage du runner + /var/lib/docker sur /mnt, avec verification et
# echec immediat s'il n'y a la place nulle part. Trois jobs en ont
# besoin, d'ou l'action locale plutot qu'un bloc recopie trois fois.
- name: Disque pour docker
  if: steps.probe.outputs.hit != 'true'
  uses: ./.github/actions/docker-disk

# buildx/docker-container : le driver qui sait ecrire AILLEURS que dans
# le store d'images local - voir le step de build. Apres le deplacement
# du data-root, jamais avant : le redemarrage du demon laisserait le
# conteneur du builder orphelin.
- name: Buildx
  if: steps.probe.outputs.hit != 'true'
  uses: docker/setup-buildx-action@v4

# ── Le cache .deb entre deux runs ──────────────────────────────────────
# Ce que le Dockerfile compile sort en paquet .deb dans
# /var/cache/osmo-debs (packaging/osmo-deb.sh), et il le relit par
# `COPY .deb-cache/ /var/cache/osmo-debs/`. Sur l'hote, build.sh fait la
# navette entre les deux ; ici on n'appelle pas build.sh, donc on la fait
# a la main dans le step de build. Les .deb sont indexes par nom et
# version : contrairement au cache de couches, ce cache-la survit a une
# modification du Dockerfile.
```

```
      # OSMO_DEB_REFRESH n'est pas passe : son defaut (Dockerfile:166) est 0,
      # donc le cache est utilise. Ne pas y toucher ici.
      - name: Preparer le cache .deb
        if: steps.probe.outputs.hit != 'true'
        run: |
          sudo mkdir -p /var/cache/osmo-debs
          sudo chown -R "$(id -u):$(id -g)" /var/cache/osmo-debs

      - name: Cache .deb (osmo-deb)
        if: steps.probe.outputs.hit != 'true'
        uses: actions/cache@v5
        with:
          path: /var/cache/osmo-debs
          # CLE = L'EMPREINTE DU CONTEXTE, pas le numero du run.
          # [2026-09-13] C'etait `${{ github.run_id }}`, donc une entree NEUVE a
          # chaque passage. A 2-4 Go l'entree et 10 Go de quota par depot,
          # l'eviction LRU detruisait le cache au bout de trois runs - et tous
          # les autres caches du depot avec. Avec l'empreinte : contexte
          # inchange, la cle est deja la, rien n'est reecrit ; contexte change,
          # une seule entree s'ajoute. Le prefixe reprend la plus recente.
          key: osmo-debs-noble-${{ steps.key.outputs.key }}
          restore-keys: |
            osmo-debs-noble-

      - name: Temoin disque
        if: steps.probe.outputs.hit != 'true'
        run: |
          nohup bash -c 'while true; do
            echo "$(date -u +%H:%M:%S)  / $(df -h --output=avail / | tail -1 | tr -d " ")  /mnt $(df -h
↪ --output=avail /mnt | tail -1 | tr -d " ")  mem $(free -m | awk "/^Mem:/{print \$7}")Mo"
            sleep 30
          done' > "$RUNNER_TEMP/watchdog.log" 2>&1 &
          echo "temoin -> $RUNNER_TEMP/watchdog.log"

      # —— LE BUILD. Vers GHCR directement, jamais dans le docker local ————
      # [2026-09-13] C'etait `docker build -t ... .` puis `docker push`. Le
      # `-t` impose a buildkit d'EXPORTER l'image dans le store local : a ce
      # moment il detient ses snapshots (11 Go) et ecrit par-dessus les
      # archives de couches (11 Go de plus). C'est exactement la, en
      # « exporting layers », que le runner saturait et se faisait tuer -
      # exit 143, apres 1h20 de compilation reussie.
      #
      # `--push` (soit --output type=image,push=true) supprime cette moitie-la
      # du probleme : les couches sont compressees et poussees au fil de l'eau,
      # rien n'atterrit dans le store local. Il ne reste que le cache du
      # builder. Corollaire : plus aucune image `osmocom-nitb` sur le runner
      # apres le build - d'ou l'etape `debs` du Dockerfile pour la recolte.
      - name: Build et push osmocom-nitb (base)
        if: steps.probe.outputs.hit != 'true'
        env:
          IMAGE: ${{ steps.key.outputs.image }}
          CACHE: ${{ steps.key.outputs.cache }}
          PR: ${{ github.event_name == 'pull_request' }}
        run: |
          # Semer le contexte avec les .deb du cache : chaque etape qui les
          # trouve pose le paquet au lieu de cloner et compiler.
          mkdir -p .deb-cache
          cp -f /var/cache/osmo-debs/osmo-build-*.deb .deb-cache/ 2>/dev/null || true
          echo "cache .deb en entree : $(find .deb-cache -name 'osmo-build-*.deb' | wc -l) paquets"

          # base-<empreinte> est ce que les jobs derives tirent ; :cache et
          # :latest sont les deux commodites - l'un pour le --cache-from du
          # prochain build, l'autre pour qui veut « la derniere ». Les trois
          # noms sortent d'UN push : memes digests, les couches ne montent
          # qu'une fois.
          # Sur une pull_request rien n'est publie (le GITHUB_TOKEN d'un fork
```

```
        # n'a pas packages:write) : on batit pour valider le Dockerfile et on
        # garde le resultat dans le cache du builder, sans export ni nom.
        if [ "$PR" = true ]; then
          TAGS=(); OUT=(--output type=cacheonly)
        else
          BASE_REPO="${CACHE%:cache}"
          TAGS=(-t "$IMAGE" -t "$CACHE")
          [ "$GITHUB_REF_NAME" = "main" ] && TAGS+=(-t "$BASE_REPO:latest")
          OUT=(--push)
        fi

        # --target : Dockerfile finit par l'etape `debs`, pas par osmocom-nitb.
        # --cache-to inline : les metadonnees de cache voyagent DANS le
        # manifeste pousse, ce qui rend le --cache-from du prochain run
        # possible sans second depot de cache a gerer.
        docker buildx build \
          --target osmocom-nitb \
          --cache-from "type=registry,ref=$CACHE" \
          --cache-to   type=inline \
          "${TAGS[@]+"${TAGS[@]}"}" "${OUT[@]}" \
          -f Dockerfile .

        # Recolte des .deb : seconde passe sur l'etape `debs`. Le CONTEXTE
        # N'A PAS BOUGE (on ne vide .deb-cache/ qu'apres) donc les 75 etapes
        # sont toutes en cache dans le builder ; seul le COPY final tourne,
        # et n'ecrit sur le disque que les paquets. Equivalent sans image
        # locale de debs_from_image() (build.sh:306).
        rm -rf /tmp/debs-out && mkdir -p /tmp/debs-out
        docker buildx build \
          --target debs \
          --cache-from "type=registry,ref=$CACHE" \
          --output "type=local,dest=/tmp/debs-out" \
          -f Dockerfile .
        cp -f /tmp/debs-out/osmo-build-*.deb /var/cache/osmo-debs/ 2>/dev/null || true
        rm -rf /tmp/debs-out .deb-cache/*.deb
        echo "cache .deb en sortie : $(find /var/cache/osmo-debs -name 'osmo-build-*.deb' | wc -l) paquets, $(du
↪  -sh /var/cache/osmo-debs | cut -f1)"
        df -h / /mnt

    - name: Temoin - dernieres mesures
      if: always()
      run: tail -40 "$RUNNER_TEMP/watchdog.log" 2>/dev/null || echo "(pas de temoin)"

# ── Le hub : rapide, independant, donc en parallele du reste ──────────────
interstp:
  runs-on: ubuntu-24.04
  timeout-minutes: 120
  permissions:
    contents: read
    packages: write
    id-token: write
  steps:
    - name: Checkout
      uses: actions/checkout@v5

    - name: Image name
      id: img
      run: echo "repo=$(echo '${{ github.repository }}' | tr '[:upper:]' '[:lower:]')" >> "$GITHUB_OUTPUT"

    - name: Log in to GHCR
      if: github.event_name != 'pull_request'
      uses: docker/login-action@v4
      with:
        registry: ${{ env.REGISTRY }}
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}
```

```
    - name: Build interstp
      run: |
        docker build -f Dockerfile.stp -t interstp:ci .
        docker images interstp:ci --format '  {{.Repository}}:{{.Tag}} {{.Size}}'

    - name: Push interstp
      if: github.event_name != 'pull_request'
      env:
        IMG: ${{ env.REGISTRY }}/${{ steps.img.outputs.repo }}/interstp
      run: |
        TAGS="sha-${GITHUB_SHA::7}"
        [ "${GITHUB_REF_TYPE}" = "tag" ]    && TAGS="$TAGS ${GITHUB_REF_NAME}"
        [ "${GITHUB_REF_NAME}" = "main" ]   && TAGS="$TAGS latest"
        for t in $TAGS; do
          docker tag interstp:ci "$IMG:$t"
          docker push "$IMG:$t"
        done

# ── Les deux derivees : en parallele, chacune sur un runner neuf ──────────
# Quelques minutes chacune - elles ne compilent rien, elles rafraichissent des
# COPY par-dessus une base deja construite.
operator:
  needs: base
  if: github.event_name != 'pull_request'
  runs-on: ubuntu-24.04
  timeout-minutes: 90
  permissions:
    contents: read
    packages: write
    id-token: write
  steps:
    - name: Checkout
      uses: actions/checkout@v5

    - name: Disque pour docker
      uses: ./.github/actions/docker-disk

    - name: Buildx
      uses: docker/setup-buildx-action@v4

    - name: Log in to GHCR
      uses: docker/login-action@v4
      with:
        registry: ${{ env.REGISTRY }}
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}

    # Meme principe que la base : buildx pousse les couches vers GHCR sans
    # jamais materialiser l'image dans le store local. Le `docker pull` du
    # BASE disparait avec, buildkit ne tire que ce dont le build a besoin ;
    # et les couches heritees de la base sont MONTEES d'un depot a l'autre
    # (cross-repo blob mount, meme registre), donc la poussee ne renvoie que
    # les couches propres a Dockerfile.run.
    - name: Build et push osmo-operator
      env:
        BASE_IMAGE: ${{ needs.base.outputs.image }}
        IMG: ${{ env.REGISTRY }}/${{ needs.base.outputs.repo }}/osmo-operator
      run: |
        TAGS=(-t "$IMG:sha-${GITHUB_SHA::7}")
        [ "${GITHUB_REF_TYPE}" = "tag" ]  && TAGS+=(-t "$IMG:${GITHUB_REF_NAME}")
        [ "${GITHUB_REF_NAME}" = "main" ] && TAGS+=(-t "$IMG:latest")
        docker buildx build -f Dockerfile.run \
          --build-arg BASE="$BASE_IMAGE" \
          "${TAGS[@]}" --push .
        docker buildx imagetools inspect "$IMG:sha-${GITHUB_SHA::7}" \
```

```
          --format '{{.Manifest.Config.Digest}}  {{len .Manifest.Layers}} couches' || true
        df -h / /mnt

  # L'image lite se deduit de la BASE, pas de osmo-operator : Dockerfile.lite
  # elague /opt/GSM puis aplatit le resultat (FROM scratch + COPY / - effacer
  # dans une couche de plus ne rend rien). C'est ce que fait build.sh:339.
  lite:
    needs: base
    if: github.event_name != 'pull_request'
    runs-on: ubuntu-24.04
    timeout-minutes: 90
    permissions:
      contents: read
      packages: write
      id-token: write
    steps:
      - name: Checkout
        uses: actions/checkout@v5

      - name: Disque pour docker
        uses: ./.github/actions/docker-disk

      - name: Buildx
        uses: docker/setup-buildx-action@v4

      - name: Log in to GHCR
        uses: docker/login-action@v4
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      # L'aplatissement (FROM scratch + COPY /) recopie la racine entiere :
      # c'est le job qui a le plus a ecrire, pour un job qui ne compile rien.
      # Raison de plus pour que la sortie parte au registre et non dans le
      # store local.
      - name: Build et push osmo-operator-lite
        env:
          BASE_IMAGE: ${{ needs.base.outputs.image }}
          IMG: ${{ env.REGISTRY }}/${{ needs.base.outputs.repo }}/osmo-operator-lite
        run: |
          TAGS=(-t "$IMG:sha-${GITHUB_SHA::7}")
          [ "${GITHUB_REF_TYPE}" = "tag" ]  && TAGS+=(-t "$IMG:${GITHUB_REF_NAME}")
          [ "${GITHUB_REF_NAME}" = "main" ] && TAGS+=(-t "$IMG:latest")
          docker buildx build -f Dockerfile.lite \
            --build-arg BASE="$BASE_IMAGE" \
            "${TAGS[@]}" --push .
          echo "--- taille publiee ---"
          docker buildx imagetools inspect "$IMG:sha-${GITHUB_SHA::7}" || true
          df -h / /mnt
```

3.11 /opt/GSM/osmo-operator/configs/asound.conf

1099 octets, 42 lignes → 42 lignes

```
# asound.conf - ALSA devices for Osmocom GSM audio (mobile/osmo-gapk) via
# PulseAudio. Installed as /etc/asound.conf.
#
# gsm_out : osmo-gapk playback -> PulseAudio sink "gsm_audio"
# gsm_in  : osmo-gapk capture  -> monitor of "gsm_mic" (silence)
#
# gsm_in must point at a sink nobody feeds: capturing the default source
# (gsm_audio.monitor) loops the mobiles' own output back into their uplink.
```

```
pcm.gsm_out {
    # Do not add period_size / buffer_size in a `slave` block: ALSA rejects
    # them ("snd_pcm_slave_conf: Unknown field period_size") and the device
    # no longer opens. Buffer geometry is negotiated by the application
    # (gapk, snd_pcm_set_params).
    #
    # Known limitation: on a silent call the mobile's playback loop spins
    # (~6300 writes/s on the ALSA-PulseAudio plugin wake-up pipe, peak 0,
    # no error reported). Cause not identified.
    type pulse
    device gsm_audio
}

ctl.gsm_out {
    type pulse
}

pcm.gsm_in {
    type pulse
    device gsm_mic.monitor
}

ctl.gsm_in {
    type pulse
}

pcm.!default {
    type pulse
}

ctl.!default {
    type pulse
}
```

3.12 /opt/GSM/osmo-operator/configs/extensions.conf

21332 octets, 402 lignes → 402 lignes

```
; ===========================================================================
; extensions.conf - Asterisk dialplan template, one instance per operator.
; Placeholders __OPERATOR_ID__ and __MSISDN_PFX__ are filled in by
; generate_configs.sh / start-docker.sh.
;
; NUMBERING PLAN
;   100          Linphone A (local softphone)
;   200          Linphone B (local softphone)
;   500          plays /root/lithium.wav
;   600          echo test
;   700          welcome announcement
;   <pfx><op>XX  local GSM subscribers (MSISDN = node prefix, operator id, rank)
;   9<number>    inter-operator call from a softphone
;   00<number>   international call via Zadarma (E.164)
;   0[1-9]+8dig  French national number via Zadarma
;
; CONTEXTS
;   [sub-record]   timestamped MixMonitor recording of every call
;   [sub-annuaire] caller name lookup (annuaire.conf, generated)
;   [internal]     calls from local softphones
;   [gsm_in]       calls from OsmoMSC (GSM -> Asterisk)
;   [interop_in]   calls from a remote operator (SIP trunk)
;   [zadarma_in]   calls from the external operator (Zadarma)
;   [interop_out]  routing to remote operators; extensions are GENERATED and
;                  appended by start-docker.sh / generate_configs.sh (one
;                  extension per remote operator -> interop_trunk_opN)
```

```
;
; CALL SCENARIOS
;   1. MS -> MS (same operator)   : MS -> MSC -> MNCC -> Asterisk -> MNCC -> MSC -> MS
;   2. MS -> Linphone             : MS -> MSC -> Asterisk SIP -> Linphone
;   3. Linphone -> MS             : Linphone SIP -> Asterisk -> MNCC -> MSC -> MS
;   4. Linphone A -> Linphone B   : local SIP
;   5. MS(OpX) -> MS(OpY)         : MNCC -> AstX -> SIP trunk -> AstY -> MNCC -> MS
;   6. MS -> echo / announcement  : audio test
; ============================================================================

[general]
static=yes
writeprotect=no
clearglobalvars=no

[globals]
MSC_LOCAL=127.0.0.1
; Operator id, used as label in recording file names (sub-record).
OP_PREFIX=__OPERATOR_ID__
; —— Monitoring: timestamped recording of ALL calls (yes/no) ——
; The MixMonitor audiohook takes the bridge out of native RTP, forcing
; gsm<->alaw transcoding on external (Zadarma) legs. Do not add an adaptive
; JITTERBUFFER on the calling (MS) leg: it only buffers the uplink and adds
; latency; put it on the trunk leg if needed.
CALL_RECORDING=yes
RECORDINGS_DIR=/var/spool/asterisk/monitor

; —— Inbound from Zadarma: who rings on a call from outside ——————————
; Value = an MSISDN of the HLR (see run_modules/21-abonnes-hlr.sh):
;   100101  the bench phone (osmocom-bb / postmarketOS VM)
;   100102  the second mobile
; To ring a softphone instead, target [internal] rather than gsm_msc in the
; Dial line of [zadarma_in] (end of file).
DEST_ZADARMA_IN=100101

; ============================================================================
; [sub-record] - starts the timestamped MixMonitor recording of the call.
;   Called at the top of each call: Gosub(sub-record,s,1(<called_number>))
;   File: ${RECORDINGS_DIR}/<date>_<time>_op<N>_<caller>_to_<called>_<uid>.wav
;   plus -ul.wav (uplink, r()) and -dl.wav (downlink, t()) on the calling
;   (MS) channel, so a degradation can be attributed to one direction.
;   Option 'b': record only while the call is bridged.
;   Disabled globally with CALL_RECORDING=no. Requires app_mixmonitor.so.
;   Edit THIS file: /etc/asterisk/extensions.conf is a copy regenerated at
;   start-up.
; ============================================================================
[sub-record]
exten => s,1,GotoIf($["${CALL_RECORDING}" != "yes"]?skip)
 ; Adaptive jitterbuffer intentionally not set here: on the calling (MS)
 ; channel it would buffer the uplink only (see [globals]).
 ; same => n,Set(JITTERBUFFER(adaptive)=default)
 same => n,Set(REC_TS=${STRFTIME(${EPOCH},,%Y%m%d_%H%M%S)})
 same => n,Set(REC_NAME=${FILTER(A-Za-z0-9_-,${REC_TS}_op${OP_PREFIX}_${CALLERID(num)}_to_${ARG1}_${UNIQUEID})})
 same => n,MixMonitor(${RECORDINGS_DIR}/${REC_NAME}.wav,br(${RECORDINGS_DIR}/${REC_NAME}-
 ↪  ul.wav)t(${RECORDINGS_DIR}/${REC_NAME}-dl.wav))
 same => n,NoOp(=== REC → ${RECORDINGS_DIR}/${REC_NAME}.wav ===)
 same => n(skip),Return()

; ============================================================================
; [sub-annuaire] - caller name lookup
; ============================================================================
; The bench directory (data/noms.txt, drawn by start-docker.sh, written to
; /etc/osmocom/annuaire.csv) gives each subscriber a name, so the called
; mobile displays "Aline Boulanger" rather than "20001".
;
; Must be called AFTER Set(CALLERID(all)=<...>), which rebuilds the CallerID
```

```
; without a name.
;
; The entries live in annuaire.conf, written by start-docker.sh at launch
; (one extension per MSISDN). The _X. pattern below is the fallback: an exact
; extension wins over a pattern, so a known entry wins and an unknown number
; returns unchanged (also when annuaire.conf does not exist).
[sub-annuaire]
exten => _X.,1,NoOp(=== ANNUAIRE : ${EXTEN} inconnu - CallerID inchange ===)
 same => n,Return()
exten => 0,1,NoOp(=== ANNUAIRE : appelant anonyme ===)
 same => n,Return()

#include "annuaire.conf"

; ===========================================================================
; [internal] - calls FROM local softphones
; ===========================================================================
[internal]

; ── 500: lithium audio playback ──────────────────────────────────────
;   Plays /root/lithium.wav of the container (8 kHz mono WAV recommended).
;   Playback(/root/lithium) without extension resolves to /root/lithium.wav.
;   The file must be provided in each container.
exten => 500,1,NoOp(=== LITHIUM - CallerID: ${CALLERID(all)} ===)
 same => n,Answer()
 same => n,Wait(1)
 same => n,Playback(/root/lithium)
 same => n,Hangup()

; ── Audio tests ──────────────────────────────────────────────────────
exten => 600,1,NoOp(=== ECHO TEST - CallerID: ${CALLERID(all)} ===)
 same => n,Answer()
 ; No announcement before the echo for bench calls (ECHO_NO_PROMPT=1, set by
 ; osmo-phonesim-banc.py on its Originate): with MixMonitor in the bridge,
 ; the Playback -> Echo transition emits a burst of silence RTP packets that
 ; desynchronises the BTS jitter buffer. The bench plays the announcement on
 ; the mobile leg instead (DIAL_OPTS=A(demo-echotest) in the Dial below).
 same => n,GotoIf($["${ECHO_NO_PROMPT}" = "1"]?echo)
 same => n,Playback(demo-echotest)
 same => n(echo),Echo()
 same => n,Playback(demo-echodone)
 same => n,Hangup()

exten => 700,1,NoOp(=== ANNONCE - CallerID: ${CALLERID(all)} ===)
 same => n,Answer()
 same => n,Playback(hello-world)
 same => n,Hangup()

; ── Softphone -> Softphone ───────────────────────────────────────────
exten => 100,1,NoOp(=== SIP: → Linphone A - CallerID: ${CALLERID(all)} ===)
 same => n,Gosub(sub-record,s,1(${EXTEN}))
 same => n,Dial(PJSIP/linphone_A,,rT)
 same => n,Congestion()
 same => n,Hangup()

exten => 200,1,NoOp(=== SIP: → Linphone B - CallerID: ${CALLERID(all)} ===)
 same => n,Gosub(sub-record,s,1(${EXTEN}))
 same => n,Dial(PJSIP/linphone_B,,rT)
 same => n,Congestion()
 same => n,Hangup()

; ── Softphone -> local GSM ───────────────────────────────────────────
; Local operator numbers: MSISDN prefix, operator id, then 2 digits (rank of
; the mobile), following the HLR plan (start-docker.sh,
; run_modules/21-abonnes-hlr.sh, scripts/sms-routing-setup.sh).
```

```
exten => ___MSISDN_PFX____OPERATOR_ID__XX,1,NoOp(=== SIP -> GSM local (Op__OPERATOR_ID__): ${EXTEN} - CallerID:
 ↪  ${CALLERID(all)} ===)
 same => n,Set(CALLERID(all)=<${CALLERID(num)}>)
 ; Caller name from the bench directory. ANNU_NUM rather than CALLERID(num)
 ; directly: an anonymous caller would give an empty extension and a failing
 ; Gosub on every call.
 same => n,Set(ANNU_NUM=${IF($["${CALLERID(num)}" != ""]?${CALLERID(num)}:0)})
 same => n,Gosub(sub-annuaire,${ANNU_NUM},1)
 same => n,Gosub(sub-record,s,1(${EXTEN}))
 same => n,Dial(PJSIP/${EXTEN}@gsm_msc,,rT${DIAL_OPTS})
 same => n,Congestion()
 same => n,Hangup()

; ── Softphone -> inter-operator GSM ──────────────────────────────────
; Dial 9 + full number (e.g. 920001 to reach 20001 on Op2)
exten => _9.,1,NoOp(=== SIP -> INTER-OP: ${EXTEN:1} - CallerID: ${CALLERID(all)} ===)
 same => n,Gosub(sub-record,s,1(${EXTEN:1}))
 same => n,Goto(interop_out,${EXTEN:1},1)

; ── Outbound to Zadarma: 00 + international number ──────────────────
; More specific than the catch-all _X. below, hence matched first.
; The 00 is replaced by the E.164 "+" form expected by Zadarma.
exten => _00.,1,NoOp(=== SIP -> ZADARMA: +${EXTEN:2} - CallerID: ${CALLERID(all)} ===)
 same => n,Gosub(sub-record,s,1(zadarma_${EXTEN:2}))
; The "+" is mandatory: Zadarma answers 404 Not Found to a bare E.164 number
; (sip:33656678074@...) and accepts sip:+33656678074@...
 same => n,Dial(PJSIP/+${EXTEN:2}@zadarma,60,rT)
 same => n,NoOp(=== ZADARMA OUT: échec ${DIALSTATUS} ===)
 same => n,Congestion()
 same => n,Hangup()


; ── Outbound to Zadarma: French NATIONAL number, dialled as is ───────
; Pattern: 0, one digit 1-9, then 8 digits (the ten digits of the French
; plan). The second digit excludes 0, so there is no overlap with _00.
; (international) above; against the catch-all _X. the fixed-length pattern
; wins. Conversion: 0612345678 -> +33612345678 (the leading 0 is the national
; prefix and is replaced by the country code; the "+" is mandatory).
exten => _0[1-9]XXXXXXXX,1,NoOp(=== SIP -> ZADARMA (national): +33${EXTEN:1} - CallerID: ${CALLERID(all)} ===)
 same => n,Gosub(sub-record,s,1(zadarma_33${EXTEN:1}))
 same => n,Dial(PJSIP/+33${EXTEN:1}@zadarma,60,rT)
 same => n,NoOp(=== ZADARMA OUT (national): échec ${DIALSTATUS} ===)
 same => n,Congestion()
 same => n,Hangup()

; ── Catch-all: try the local MSC ─────────────────────────────────────
exten => _X.,1,NoOp(=== SIP -> GSM fallback: ${EXTEN} - CallerID: ${CALLERID(all)} ===)
 same => n,Gosub(sub-record,s,1(${EXTEN}))
 same => n,Dial(PJSIP/${EXTEN}@gsm_msc,,rT${DIAL_OPTS})
 same => n,Congestion()
 same => n,Hangup()

; ============================================================================
; [gsm_in] - calls FROM OsmoMSC (GSM -> Asterisk)
;
; Pattern layout (most specific first; Asterisk matches the most specific
; pattern, no priority needed):
;   1. fixed numbers (100, 200, 500, 600)        -> local services
;   2. _<pfx><op>XX (fixed length)               -> local MS, same operator
;   3. _00. / _0[1-9]XXXXXXXX / _+X.             -> Zadarma
;   4. _<pfx><op>.                               -> malformed local number
;   5. _X. (catch-all)                           -> inter-operator routing
; Do not add two patterns of the same length for the same numbers: Asterisk
; keeps only one of the duplicates.
; ============================================================================
[gsm_in]
```

```
; ── 500: lithium audio playback (/root/lithium.wav of the container) ──
exten => 500,1,NoOp(=== GSM -> LITHIUM - CallerID: ${CALLERID(all)} ===)
 same => n,Answer()
 same => n,Wait(1)
 same => n,Playback(/root/lithium)
 same => n,Hangup()

; ── Audio tests ──────────────────────────────────────────────────────
exten => 600,1,NoOp(=== GSM -> ECHO TEST - CallerID: ${CALLERID(all)} ===)
 same => n,Answer()
 same => n,Playback(demo-echotest)
 same => n,Echo()
 same => n,Playback(demo-echodone)
 same => n,Hangup()

; ── MS -> Linphone ───────────────────────────────────────────────────
exten => 100,1,NoOp(=== GSM -> Linphone A - CallerID: ${CALLERID(all)} ===)
 same => n,Gosub(sub-record,s,1(${EXTEN}))
 same => n,Dial(PJSIP/linphone_A,,rT)
 same => n,Congestion()
 same => n,Hangup()

exten => 200,1,NoOp(=== GSM -> Linphone B - CallerID: ${CALLERID(all)} ===)
 same => n,Gosub(sub-record,s,1(${EXTEN}))
 same => n,Dial(PJSIP/linphone_B,,rT)
 same => n,Congestion()
 same => n,Hangup()

; ── MS -> local MS (same operator) ──────────────────────────────────
; Pattern: MSISDN prefix, operator id, then 2 digits.
exten => ___MSISDN_PFX____OPERATOR_ID__XX,1,NoOp(=== GSM -> GSM local (Op__OPERATOR_ID__): ${EXTEN} - CallerID:
 ↪  ${CALLERID(all)} ===)
 same => n,Set(CALLERID(all)=<${CALLERID(num)}>)
 ; Caller name from the bench directory. ANNU_NUM rather than CALLERID(num)
 ; directly: an anonymous caller would give an empty extension and a failing
 ; Gosub on every call.
 same => n,Set(ANNU_NUM=${IF($["${CALLERID(num)}" != ""]?${CALLERID(num)}:0)})
 same => n,Gosub(sub-annuaire,${ANNU_NUM},1)
 same => n,Gosub(sub-record,s,1(${EXTEN}))
 same => n,Dial(PJSIP/${EXTEN}@gsm_msc,,rT)
 same => n,Congestion()
 same => n,Hangup()

; ── Outbound to Zadarma: 00 + international number ──────────────────
; More specific than the catch-all _X. below, hence matched first.
; The 00 is replaced by the E.164 "+" form expected by Zadarma.
exten => _00.,1,NoOp(=== GSM -> ZADARMA: +${EXTEN:2} - CallerID: ${CALLERID(all)} ===)
 same => n,Gosub(sub-record,s,1(zadarma_${EXTEN:2}))
; The "+" is mandatory: Zadarma answers 404 Not Found to a bare E.164 number
; (sip:33656678074@...) and accepts sip:+33656678074@...
 same => n,Dial(PJSIP/+${EXTEN:2}@zadarma,60,rT)
 same => n,NoOp(=== ZADARMA OUT: échec ${DIALSTATUS} ===)
 same => n,Congestion()
 same => n,Hangup()

; ── Outbound to Zadarma: French NATIONAL number, dialled as is ───────
; Pattern: 0, one digit 1-9, then 8 digits (the ten digits of the French
; plan). The second digit excludes 0, so there is no overlap with _00.
; (international) above; against the catch-all _X. the fixed-length pattern
; wins. Conversion: 0612345678 -> +33612345678 (the leading 0 is the national
; prefix and is replaced by the country code; the "+" is mandatory).
exten => _0[1-9]XXXXXXXX,1,NoOp(=== GSM -> ZADARMA (national): +33${EXTEN:1} - CallerID: ${CALLERID(all)} ===)
 same => n,Gosub(sub-record,s,1(zadarma_33${EXTEN:1}))
 same => n,Dial(PJSIP/+33${EXTEN:1}@zadarma,60,rT)
 same => n,NoOp(=== ZADARMA OUT (national): échec ${DIALSTATUS} ===)
```

```
 same => n,Congestion()
 same => n,Hangup()

; ── E.164 with "+": what the MSC actually presents ──────────────────────────
; osmo-msc normalises an international 00 number to E.164 with a leading "+"
; (0033656678074 dialled on the MS arrives here as +33656678074). The "+" is
; kept in the Dial (Zadarma returns 404 without it). The trunk itself needs
; valid credentials in pjsip.conf, otherwise the call ends in Congestion().
exten => _+X.,1,NoOp(=== GSM -> ZADARMA (E.164): ${EXTEN} - CallerID: ${CALLERID(all)} ===)
 same => n,Gosub(sub-record,s,1(zadarma_${EXTEN:1}))
 same => n,Dial(PJSIP/${EXTEN}@zadarma,60,rT)
 same => n,NoOp(=== ZADARMA OUT (E.164): échec ${DIALSTATUS} ===)
 same => n,Congestion()
 same => n,Hangup()

; ── Malformed local number ──────────────────────────────────────────────────
; _<pfx><op>. is more specific than _X. and less than the _<pfx><op>XX above:
; an exact local number goes to the MSC, a malformed local number is rejected
; here (avoids a routing loop), everything else goes inter-operator.
exten => ___MSISDN_PFX____OPERATOR_ID__.,1,NoOp(=== GSM -> numéro local inexistant: ${EXTEN} ===)
 same => n,Congestion()
 same => n,Hangup()

; ── Inter-operator routing (catch-all) via [interop_out] ────────────────────
exten => _X.,1,NoOp(=== GSM -> routage inter-op: ${EXTEN} - CallerID: ${CALLERID(all)} ===)
 same => n,Gosub(sub-record,s,1(${EXTEN}))
 same => n,Goto(interop_out,${EXTEN},1)

; ============================================================================
; [interop_in] - calls FROM a remote operator (SIP trunk)
; ============================================================================
[interop_in]

; Remote MS -> local MS
exten => ___MSISDN_PFX____OPERATOR_ID__XX,1,NoOp(=== INTER-OP -> GSM local: ${EXTEN} - CallerID distant:
 ↪  ${CALLERID(all)} ===)
 same => n,Set(CALLERID(all)=<${CALLERID(num)}>)
 ; Caller name from the bench directory. ANNU_NUM rather than CALLERID(num)
 ; directly: an anonymous caller would give an empty extension and a failing
 ; Gosub on every call.
 same => n,Set(ANNU_NUM=${IF($["${CALLERID(num)}" != ""]?${CALLERID(num)}:0)})
 same => n,Gosub(sub-annuaire,${ANNU_NUM},1)
 same => n,Gosub(sub-record,s,1(${EXTEN}))
 same => n,Dial(PJSIP/${EXTEN}@gsm_msc,,rT)
 same => n,Congestion()
 same => n,Hangup()

; Remote operator -> local Linphone
exten => 100,1,NoOp(=== INTER-OP -> Linphone A - CallerID distant: ${CALLERID(all)} ===)
 same => n,Gosub(sub-record,s,1(${EXTEN}))
 same => n,Dial(PJSIP/linphone_A,,rT)
 same => n,Hangup()

exten => 200,1,NoOp(=== INTER-OP -> Linphone B - CallerID distant: ${CALLERID(all)} ===)
 same => n,Gosub(sub-record,s,1(${EXTEN}))
 same => n,Dial(PJSIP/linphone_B,,rT)
 same => n,Hangup()

; Clean rejection
exten => _X.,1,NoOp(=== INTER-OP -> numéro inconnu: ${EXTEN} - CallerID distant: ${CALLERID(all)} ===)
 same => n,Congestion()
 same => n,Hangup()

; ============================================================================
; [zadarma_in] - calls FROM the external operator (Zadarma)
;
```

```
; Zadarma presents the call on the SIP login or on a DID. Everything is
; routed to the GSM mobile DEST_ZADARMA_IN ([globals]); change it to ring a
; softphone (100 / 200) or an IVR instead.
; ============================================================================
[zadarma_in]
; The channel must keep the destination number as its extension: the
; DestExten of the PJSIP channel created by Dial inherits the extension of
; the calling channel, and the bench modem (tools/osmo-phonesim-banc.py)
; rings the phone only on a DialBegin whose DestExten equals its MSISDN.
; Routing through an "s" extension would make the inbound call invisible to
; the handset. Hence the Goto to ${DEST_ZADARMA_IN}, which carries the
; processing.
;
; The entry pattern must never match a single character: "h" (hangup), "i",
; "t" and "T" are special extensions, and catching "h" would replay the whole
; processing on a dead channel in a loop. A called number has at least two
; characters and starts with "+" or a digit, so no special extension matches.
exten => _[+0-9].,1,Goto(${DEST_ZADARMA_IN},1)
exten => s,1,Goto(${DEST_ZADARMA_IN},1)
exten => i,1,NoOp(=== ZADARMA IN: numéro appelé invalide (${INVALID_EXTEN}) ===)
 same => n,Goto(${DEST_ZADARMA_IN},1)

; The processing, carried by the local MSISDN itself. Fixed-length pattern,
; hence more specific than _[+0-9]. above. If DEST_ZADARMA_IN leaves this
; plan, widen this pattern.
exten => ___MSISDN_PFX____OPERATOR_ID__XX,1,NoOp(=== ZADARMA IN: appelé=${EXTEN} - CallerID: ${CALLERID(all)} ===)
 ; Strip the E.164 "+" from the CALLING number: the MSC cannot carry it to
 ; the mobile display. FILTER rather than ${CALLERID(num):1}, so a caller
 ; already without "+" does not lose its first digit.
 same => n,Set(CALLERID(num)=${FILTER(0-9,${CALLERID(num)})})
 same => n,Set(CALLERID(name)=${CALLERID(num)})
 same => n,Gosub(sub-record,s,1(zadarma_in_${CALLERID(num)}))
 ; 45 s, not 30: a mobile-terminated call goes through paging over the air
 ; and the QEMU/Calypso chain takes 10-15 s before the first ring.
 same => n,Dial(PJSIP/${EXTEN}@gsm_msc,45,rT)
 same => n,NoOp(=== ZADARMA IN: échec ${DIALSTATUS} vers ${EXTEN} ===)
 same => n,Hangup()

; ============================================================================
; [interop_out] - outbound routing to remote operators
;
; The per-operator extensions are GENERATED and appended by start-docker.sh /
; generate_configs.sh (generate_extensions_interop_out in lib/gabarits.sh):
; one extension per remote operator N, routed to interop_trunk_opN via PJSIP.
; Only the catch-all is declared here; do not declare it twice.
; ============================================================================
[interop_out]

exten => _X.,1,NoOp(=== INTEROP OUT: inconnu ${EXTEN} ===)
 same => n,Congestion()
 same => n,Hangup()
```

3.13 /opt/GSM/osmo-operator/configs/linphonerc

4556 octets, 121 lignes → 121 lignes

```
# ============================================================================
# linphonerc - compte SIP pre-provisionne du client Linphone (ISO --desktop)
#
# Depose par build-iso.sh dans ~/.config/linphone/linphonerc des comptes root
# et osmocom, et dans /etc/skel. Sans ce fichier l'ISO livrait Linphone VIERGE :
# l'operateur creait le compte a la main, a chaque demarrage, et tombait sur les
# deux pieges ci-dessous.
#
```

```
#   PIEGE 1 - LINPHONE NE RELIT PAS SA CONFIG A CHAUD.
#   Le compte ecrit dans linphonerc pendant que Linphone TOURNE n'est jamais
#   charge : l'instance en cours garde son etat en memoire et REECRIT le fichier
#   en sortant. Symptome constate le 2026-08-31 sur ce banc : linphonerc correct
#   (linphone_A / tester / bon proxy, reg_sendregister=1), process bien vivant,
#   socket SIP UDP ouverte -- et pas UN REGISTER en 90 s de capture. Cote
#   Asterisk : endpoint "Unavailable", zero contact, et RIEN dans les logs (pas
#   de 401, pas de "No matching endpoint") puisque aucun paquet n'arrivait.
#   Le serveur etait sain : un REGISTER injecte a la main sur 127.0.0.1:5060
#   recevait bien son "401 Unauthorized" avec challenge digest.
#   -> Il faut REDEMARRER Linphone. D'ou ce provisionnement au build, pose
#      avant le premier lancement.
#
#   PIEGE 2 - publish=0, ET PAS 1 (voir la section [proxy_0]).
#
#   L'identifiant SIP est "linphone_A", PAS "100". Le 100 est l'extension
#   composee dans le dialplan (configs/extensions.conf), pas le nom du compte.
# ============================================================================

[sip]
# -2 = port UDP aleatoire. Surtout PAS 5060 : sur l'ISO, Linphone et Asterisk
# partagent le meme hote, et Asterisk tient deja 0.0.0.0:5060 (pjsip.conf
# [transport-udp] bind=0.0.0.0). Un port fixe a 5060 ici et le client ne
# demarre pas sa pile SIP - sans le dire clairement.
sip_port=-2
# TCP et TLS coupes : pjsip.conf ne declare QU'UN transport, [transport-udp].
# Rien n'ecoute en TCP 5060 (et chan_sip n'est meme pas charge). Un Linphone
# laisse sur son transport par defaut echoue instantanement, sans qu'aucun
# paquet n'arrive jusqu'a Asterisk - donc sans la moindre trace cote serveur.
sip_tcp_port=-1
sip_tls_port=-1
media_encryption=none
default_proxy=0

# LES DEUX POSTES SONT PROVISIONNES DANS LA MEME INSTANCE. Linphone gere
# plusieurs comptes de front : les deux s enregistrent au demarrage, et l on
# appelle de l un vers l autre (100 <-> 200) sans rien reconfigurer. Le compte
# par defaut (celui qui emet si l on ne choisit pas) est proxy_0, linphone_A,
# fixe par default_proxy=0 dans [sip] ci-dessus.
[auth_info_0]
username=linphone_A
passwd=tester
# 127.0.0.1 est valable dans LES DEUX topologies - conteneur comme ISO :
# Linphone et Asterisk partagent l'hote. Une IP de LAN (192.168.1.x) marcherait
# sur ce banc mais serait fausse des le prochain bail DHCP.
domain=127.0.0.1

[auth_info_1]
username=linphone_B
passwd=testerB
domain=127.0.0.1

[proxy_0]
reg_proxy=<sip:127.0.0.1;transport=udp>
reg_identity=sip:linphone_A@127.0.0.1
reg_expires=3600
reg_sendregister=1
# publish=0 IMPERATIF. A 1, Linphone emet un PUBLISH de presence a chaque
# enregistrement et Asterisk 18 le refuse - il n'embarque aucun gestionnaire
# pour l'evenement "presence" (c'est une brique absente de la version libre) :
#     WARNING res_pjsip_pubsub.c: pubsub_on_rx_publish_request:
#     No registered publish handler for event presence from linphone_A
# Le warning est inoffensif mais il revient a chaque REGISTER et noie
# /var/log/asterisk/messages. Rien a corriger cote Asterisk : c'est le client
# qui doit se taire.
publish=0
```

```
avpf=0
quality_reporting_enabled=0

[proxy_1]
reg_proxy=<sip:127.0.0.1;transport=udp>
reg_identity=sip:linphone_B@127.0.0.1
reg_expires=3600
reg_sendregister=1
# publish=0 pour la meme raison que proxy_0 : sans ce reglage, CHAQUE compte
# emet son PUBLISH de presence a chaque enregistrement, et le warning
# "No registered publish handler for event presence" arrive alors en DOUBLE.
publish=0
avpf=0
quality_reporting_enabled=0

# Codecs alignes sur l'endpoint Asterisk (pjsip.conf : disallow=all, puis
# allow=alaw/ulaw/gsm). Opus et speex sont coupes : proposes par defaut, ils
# ne sont jamais retenus et allongent le SDP pour rien.
[audio_codec_0]
mime=PCMA
rate=8000
channels=1
enabled=1

[audio_codec_1]
mime=PCMU
rate=8000
channels=1
enabled=1

[audio_codec_2]
mime=GSM
rate=8000
channels=1
enabled=1

[audio_codec_3]
mime=opus
rate=48000
channels=2
enabled=0

[audio_codec_4]
mime=speex
rate=16000
channels=1
enabled=0
```

3.14 /opt/GSM/osmo-operator/configs/manager-osmo.conf

658 octets, 15 lignes → 15 lignes

```
; manager-osmo.conf - Asterisk AMI account for the mobile bridge.
;
; Installed as /etc/asterisk/manager.d/osmo.conf by start-native.sh. Used by
; tools/osmo-phonesim-banc.py to originate the calls requested by the Android
; phone (the bench modem receives ATD..., the AMI Originate action creates the
; call in Asterisk). The CLI "channel originate" is not available because
; res_clioriginate is not loaded (modules.conf uses autoload = no).
;
; Access is restricted to the local machine: deny all, permit loopback.
[osmo]
secret = osmocom1
deny = 0.0.0.0/0.0.0.0
```

```
permit = 127.0.0.1/255.255.255.255
read = system,call,verbose
write = system,call,originate
```

3.15 /opt/GSM/osmo-operator/configs/mobile.cfg

3007 octets, 134 lignes → 134 lignes

```
# mobile.cfg - osmocom-bb "mobile" configuration template (placeholders
# __IMEI__, __IMSI__, __KI__, __SIM_ALGO__, __MCC__, __MNC__, __ARFCN__,
# __SMS_SC__, __INTER_NET_GATEWAY__ are filled in by the config generators).
# KI: "ki comp128 XX XX ... XX", 16 bytes; byte 15 = ms_idx, byte 16 = op_id.
#
# Audio: fake_trx carries the TCH payload -> trxcon -> mobile -> ALSA (gapk).
# Required in the container: libasound2-plugins + /dev/snd + PULSE_SERVER.
!
line vty
 no login
 bind 127.0.0.1 4247
!
gps device /dev/ttyACM0
gps baudrate default
no gps enable
!
no hide-default
!
!
log gsmtap 127.0.0.1
 logging filter all 1
 logging color 0
 logging timestamp 0
 logging print category 1
 logging level mm debug
 logging level rr debug
 logging level cc debug
 logging level sms debug
 logging level plmn debug
 logging level sim debug
 logging level sap debug
!
ms 1
 layer2-socket /tmp/osmocom_l2
! SIM: "sim test" (the test-sim block below) by default. "sim sap" (SIM served
! over the SAP socket by softSIM: PC/SC reader or software SIM) is only set
! when --sap / --softsim is passed to start-native.sh (SAP_MODE=1); both
! generate_mobile_cfg (06-banc-dsp-menu.sh) and generate_configs.sh apply it.
 sap-socket /tmp/osmocom_sap
 sim test
 network-selection-mode auto
 imei __IMEI__ 0
 imei-fixed
 no emergency-imsi
 sms-service-center __SMS_SC__
 no call-waiting
 no auto-answer
 no force-rekey
 no clip
 no clir
 tx-power auto
 no simulated-delay
 stick __ARFCN__
 location-updating
 neighbour-measurement
 codec full-speed prefer
```

```
 codec half-speed
 no abbrev
 support
  sms
  a5/1
  a5/2
  p-gsm
  e-gsm
  r-gsm
  no gsm-850
  dcs
  no pcs
  class-900 4
  class-850 4
  class-dcs 1
  class-pcs 1
  channel-capability sdcch+tchf+tchh
  full-speech-v1
  full-speech-v2
  half-speech-v1
  min-rxlev -106
  dsc-max 90
  no skip-max-per-band
 test-sim
  imsi __IMSI__
  ki __SIM_ALGO__ __KI__
  no barred-access
  rplmn __MCC__ __MNC__
 tch-voice
  io-handler gapk
! TCH frame format must be rtp, not ti: "ti" is the Texas Instruments layout
! of real Calypso phones (DSP vocoder). Here channel coding is done by pont.py
! with libosmocoding, which uses the RTP/RFC3551 layout (33 bytes, 0xD
! signature). A mismatch gives robotic audio without any counter error.
  io-tch-format rtp
  alsa-output-dev gsm_out
  alsa-input-dev gsm_in
 no shutdown
!
! GSMTAP configuration
!
gsmtap
 remote-host __INTER_NET_GATEWAY__
 no local-host
 lchan sacch
 lchan lsacch
 lchan sacch/4
 lchan sacch/8
 lchan sacch/f
 lchan sacch/h
 lchan unknown
 lchan bcch
 lchan ccch
 lchan rach
 lchan agch
 lchan pch
 lchan sdcch
 lchan sdcch/4
 lchan sdcch/8
 lchan facch/f
 lchan facch/h
 lchan pacch
 lchan cbch
 lchan pdch
 lchan pttch
 lchan tch/f
```

```
 lchan tch/h
 category gprs dl-unknown
 category gprs dl-dummy
 category gprs dl-ctrl
 category gprs dl-data-gprs
 category gprs dl-data-egprs
 category gprs ul-unknown
 category gprs ul-dummy
 category gprs ul-ctrl
 category gprs ul-data-gprs
 category gprs ul-data-egprs
end
```

3.16 /opt/GSM/osmo-operator/configs/modules.conf

5885 octets, 130 lignes → 130 lignes

```
; modules.conf - Asterisk module list for the GSM core (autoload = no:
; only the modules listed below are loaded).
[modules]
autoload = no

; ── Blocked - no SRTP/DTLS in a GSM core ──────────────────────────────
; res_srtp is pulled in as a dependency by res_rtp_asterisk even with
; autoload=no. It inspects incoming RTP and some GSM-FR frames trigger
; "Received SSL traffic on RTP instance without an SSL session", after which
; the audio is dropped. noload prevents loading even as a dependency.
noload = res_srtp.so
noload = res_hep.so
noload = res_hep_pjsip.so
noload = res_hep_rtcp.so

; ── Applications ──────────────────────────────────────────────────────
load = app_bridgewait.so
load = app_dial.so
load = app_playback.so
load = app_stack.so
load = app_verbose.so
load = app_voicemail.so
load = app_directory.so
load = app_confbridge.so
load = app_queue.so
load = app_echo.so
load = app_mixmonitor.so

; ── Audio functions (jitterbuffer: smooths the transcoding forced by recording) ──
load = func_jitterbuffer.so

; ── Bridging ──────────────────────────────────────────────────────────
load = bridge_builtin_features.so
load = bridge_builtin_interval_features.so
load = bridge_holding.so
load = bridge_native_rtp.so
load = bridge_simple.so
load = bridge_softmix.so

; ── CDR ───────────────────────────────────────────────────────────────
load = cdr_custom.so
; cdr_csv writes /var/log/asterisk/cdr-csv/Master.csv, one line per call
; (cdr_custom alone writes nothing without a cdr_custom.conf).
load = cdr_csv.so

; ── Channels ──────────────────────────────────────────────────────────
load = chan_bridge_media.so
```

```
load = chan_pjsip.so
; chan_iax2 is unused (everything goes through PJSIP). Side effect: it is the
; only module that calls jb_setoutput, i.e. the only one that wires the log
; output of the jitterbuf engine used by func_jitterbuffer. Without it the
; jitterbuffer still runs but logs no "Resyncing the jb ..." lines; set
; `load` again to get that probe back.
noload = chan_iax2.so

; ── Codecs ──────────────────────────────────────────────────────────────
load = codec_gsm.so
load = codec_resample.so
load = codec_alaw.so
load = codec_ulaw.so
load = codec_g722.so

; ── Formats ─────────────────────────────────────────────────────────────
load = format_gsm.so
load = format_pcm.so
load = format_wav_gsm.so
load = format_wav.so

; ── Functions ───────────────────────────────────────────────────────────
load = func_callerid.so
load = func_cdr.so
load = func_pjsip_endpoint.so
load = func_sorcery.so
load = func_devstate.so
load = func_strings.so
; func_logic provides ${IF()}, used by extensions.conf ([internal], [gsm_in],
; [interop_in]); without it calls to local mobiles fail with
; "Function IF not registered". Check: asterisk -rx "core show function IF".
load = func_logic.so

; ── PBX ─────────────────────────────────────────────────────────────────
load = pbx_config.so

; ── Resources ───────────────────────────────────────────────────────────
load = res_http_websocket.so
load = res_musiconhold.so
load = res_pjproject.so
load = res_pjsip_acl.so
load = res_pjsip_authenticator_digest.so
load = res_pjsip_caller_id.so
load = res_pjsip_dialog_info_body_generator.so
load = res_pjsip_diversion.so
load = res_pjsip_dtmf_info.so
load = res_pjsip_endpoint_identifier_anonymous.so
load = res_pjsip_endpoint_identifier_ip.so
load = res_pjsip_endpoint_identifier_user.so
load = res_pjsip_exten_state.so
load = res_pjsip_header_funcs.so
load = res_pjsip_logger.so
load = res_pjsip_messaging.so
load = res_pjsip_mwi_body_generator.so
load = res_pjsip_mwi.so
load = res_pjsip_nat.so
load = res_pjsip_notify.so
load = res_pjsip_one_touch_record_info.so
load = res_pjsip_outbound_authenticator_digest.so
load = res_pjsip_outbound_publish.so
load = res_pjsip_outbound_registration.so
load = res_pjsip_path.so
load = res_pjsip_pidf_body_generator.so
load = res_pjsip_pidf_digium_body_supplement.so
load = res_pjsip_pidf_eyebeam_body_supplement.so
load = res_pjsip_publish_asterisk.so
```

```
load = res_pjsip_pubsub.so
load = res_pjsip_refer.so
load = res_pjsip_registrar.so
load = res_pjsip_rfc3326.so
load = res_pjsip_sdp_rtp.so
load = res_pjsip_send_to_voicemail.so
load = res_pjsip_session.so
load = res_pjsip.so
load = res_pjsip_t38.so
load = res_pjsip_transport_websocket.so
load = res_pjsip_xpidf_body_generator.so
load = res_rtp_asterisk.so
load = res_sorcery_astdb.so
load = res_sorcery_config.so
load = res_sorcery_memory.so
load = res_sorcery_realtime.so
load = res_timing_timerfd.so
```

3.17 /opt/GSM/osmo-operator/configs/osmo-bsc.cfg

10542 octets, 379 lignes → 373 lignes (1 groupes compactés)

```
!
! osmo-bsc.cfg - OsmoBSC template for operator __OPERATOR_ID__ (MCC __MCC__ /
! MNC __MNC__). Placeholders are filled in by generate_configs.sh.
!
! __CONTAINER_IP__ = 172.20.0.11 for operator 1, 172.20.0.12 for operator 2.
!
log file /var/log/osmocom/osmo-bsc.log
 logging color 1
 logging timestamp 1
 logging filter all 1
 logging print category-hex 0
 logging print category 1
 logging print thread-id 0
 logging print level 1
 logging print file basename last
log stderr
 logging filter all 1
 logging color 1
 logging print category-hex 0
 logging print category 1
 logging print thread-id 0
 logging timestamp 0
 logging print level 1
 logging print file basename last
!
log gsmtap __GATEWAY_IP__
 logging filter all 1
 logging color 0
 logging timestamp 0
 logging print category 1
 logging print level 1
 logging level rll notice
 logging level mm notice
 logging level rr notice
 logging level rsl notice
 logging level nm notice
 logging level pag notice
 logging level meas notice
 logging level msc notice
 logging level ho notice
 logging level hodec notice
 logging level ref notice
```

```
 logging level ctrl notice
 logging level filter notice
 logging level pcu notice
 logging level lcls notice
 logging level chan notice
 logging level ts notice
 logging level as notice
 logging level cbs notice
 logging level lcs notice
 logging level asci notice
 logging level reset notice
 logging level loop notice
 logging level lglobal notice
 logging level llapd notice
 logging level linp notice
 logging level lmux notice
 logging level lmi notice
 logging level lmib notice
 logging level lsms notice
 logging level lctrl notice
 logging level lgtp notice
 logging level lstats notice
 logging level lgsup notice
 logging level loap notice
 logging level lss7 notice
 logging level lsccp notice
 logging level lsua notice
 logging level lm3ua notice
 logging level lmgcp notice
 logging level ljibuf notice
 logging level lrspro notice
 logging level lns notice
 logging level lbssgp notice
 logging level lnsdata notice
 logging level lnssignal notice
 logging level liuup notice
 logging level lpfcp notice
 logging level lcsn1 notice
 logging level lio notice
 logging level ltcap notice
!
stats interval 5
!
line vty
 no login
!
e1_input
 e1_line 0 driver ipa
 e1_line 0 port 0
!
cs7 instance 0
 network-indicator international
 point-code __PC_BSC__

 asp asp-to-stp 2905 2907 m3ua
  remote-ip 127.0.0.1
  local-ip  127.0.0.1
  role asp
  sctp-role client
  no shutdown

 as as-bsc m3ua
  asp asp-to-stp
  routing-key __RCTX_BSC__ __PC_BSC__
  traffic-mode override
```

```
 sccp-address addr-msc
  routing-indicator PC
  point-code __PC_MSC__
  subsystem-number 254

network
 network country code __MCC__
 mobile network code __MNC__
 encryption __ENCRYPTION__
 neci 1
 paging any use tch 0
 handover 0
 handover algorithm 1
 handover1 window rxlev averaging 10
 handover1 window rxqual averaging 1
 handover1 window rxlev neighbor averaging 10
 handover1 power budget interval 6
 handover1 power budget hysteresis 3
 handover1 maximum distance 9999
 timer t3109 __T3109__
 mgw 0
  local-port 2727
  remote-ip 127.0.0.1
  remote-port 2427
 bts 0
  type osmo-bts
  band GSM900
  cell_identity __CELL_ID__
  location_area_code __LAC__
  base_station_id_code __BSIC__
  ms max power __MS_MAX_POWER__
  cell reselection hysteresis __CELL_RESEL_HYST__
  rxlev access min __RXLEV_ACCESS_MIN__
  radio-link-timeout 32
  channel allocator mode chan-req descending
  channel allocator mode assignment ascending
  channel allocator mode handover ascending
  channel allocator mode vgcs-vbs ascending
  rach tx integer 9
  rach max transmission 7
  rach max-delay 63
  rach expiry-timeout 32
  channel-description attach 1
  channel-description bs-pa-mfrms 5
  channel-description bs-ag-blks-res 1
  no nch-position
  no access-control-class-ramping
  early-classmark-sending forbidden
  early-classmark-sending-3g allowed
  ipa unit-id __IPA_UNIT_ID__ 0
  oml ipa stream-id 255 line 0
  ! NEIGHBOR_MODE_BTS0 (generate_configs.sh): "manual" (no neighbour) with
  ! --ms-magnetite, so the BTS1 side-car ARFCN does not enter the BA list and
  ! trigger a neighbour sync / DSP RECOVERY on the emulated L1;
  ! MAGNETITE_VOISINS=1 keeps "automatic".
  neighbor-list mode __NEIGHBOR_MODE_BTS0__
  codec-support fr
  amr tch-f modes 0 2 5 7
  amr tch-f threshold ms 13 25 37
  amr tch-f hysteresis ms 4 4 4
  amr tch-f threshold bts 13 25 37
  amr tch-f hysteresis bts 4 4 4
  amr tch-f start-mode auto
  amr tch-h modes 0 2 3 5
  amr tch-h threshold ms 16 24 32
  amr tch-h hysteresis ms 4 4 4
```

```
amr tch-h threshold bts 16 24 32
amr tch-h hysteresis bts 4 4 4
amr tch-h start-mode auto
gprs mode egprs
gprs routing area 0
gprs network-control-order nc0
gprs power-control alpha 0
gprs cell bvci __BVCI__
gprs cell timer blocking-timer 3
gprs cell timer blocking-retries 3
gprs cell timer unblocking-retries 3
gprs cell timer reset-timer 3
gprs cell timer reset-retries 3
gprs cell timer suspend-timer 10
gprs cell timer suspend-retries 3
gprs cell timer resume-timer 10
gprs cell timer resume-retries 3
gprs cell timer capability-update-timer 10
gprs cell timer capability-update-retries 3
gprs nsei __NSEI__
gprs ns timer tns-block 3
gprs ns timer tns-block-retries 3
gprs ns timer tns-reset 3
gprs ns timer tns-reset-retries 3
gprs ns timer tns-test 30
gprs ns timer tns-alive 3
gprs ns timer tns-alive-retries 10
gprs nsvc 0 nsvci __NSVCI__
gprs nsvc 0 local udp port 23001
gprs nsvc 0 remote ip __CONTAINER_IP__
gprs nsvc 0 remote udp port 23000
gprs nsvc 1 nsvci 0
gprs nsvc 1 local udp port 0
gprs nsvc 1 remote ip 0.0.0.0
gprs nsvc 1 remote udp port 0
bs-power-control
 mode static
ms-power-control
 mode dyn-bts
 ctrl-interval 2
 step-size inc 4 red 2
 rxlev-thresh lower 32 upper 38
 rxlev-thresh-comp lower 10 12 upper 19 20
 rxqual-thresh lower 3 upper 0
 rxqual-thresh-comp lower 5 7 upper 15 18
 ci-thresh fr-efr disable
 ci-thresh fr-efr lower 13 upper 17
 ci-thresh-comp fr-efr lower 5 7 upper 15 18
 ci-thresh hr disable
 ci-thresh hr lower 16 upper 21
 ci-thresh-comp hr lower 5 7 upper 15 18
 ci-thresh amr-fr disable
 ci-thresh amr-fr lower 7 upper 11
 ci-thresh-comp amr-fr lower 5 7 upper 15 18
 ci-thresh amr-hr disable
 ci-thresh amr-hr lower 13 upper 17
 ci-thresh-comp amr-hr lower 5 7 upper 15 18
 ci-thresh sdcch disable
 ci-thresh sdcch lower 12 upper 16
 ci-thresh-comp sdcch lower 5 7 upper 15 18
 ci-thresh gprs disable
 ci-thresh gprs lower 18 upper 24
 ci-thresh-comp gprs lower 5 7 upper 15 18
trx 0
 rf_locked 0
 arfcn __ARFCN__
```

```
   nominal power 23
   max_power_red 0
   rsl e1 tei 0
   timeslot 0
    phys_chan_config CCCH+SDCCH4
    hopping enabled 0
   timeslot 1
    ! TS1_PCHAN_BTS0 (generate_configs.sh): SDCCH8 with the bridge (pont), TCH/F
    ! with the Calypso TRX (--bts / OSMO_TRX=1), which serves a single TS and
    ! must not get the SDCCH/8 preferred by the BSC for the LU.
    phys_chan_config __TS1_PCHAN_BTS0__
    hopping enabled 0
   timeslot 2
    phys_chan_config TCH/F
    hopping enabled 0
   timeslot 3
    phys_chan_config TCH/F
    hopping enabled 0
   timeslot 4
    phys_chan_config TCH/F
    hopping enabled 0
   timeslot 5
    phys_chan_config TCH/F
    hopping enabled 0
   timeslot 6
    phys_chan_config PDCH
    hopping enabled 0
   timeslot 7
    phys_chan_config PDCH
    hopping enabled 0
!
! ── Second BTS: the fake_trx side-car (osmo-bts-trx-bts1.cfg) ──────────────────
! unit-id (6000 + operator*10 + 2) and ARFCN are offset so they collide
! neither with the main BTS nor with neighbouring operators.
!
! Placement: "bts 1" must stay INSIDE the "network" node, after "bts 0" and
! BEFORE "msc 0" ("msc 0" closes the network node; a "bts 1" after it is a
! fatal VTY parse error).
 bts 1
  type osmo-bts
  band __BAND__
  cell_identity __IPA_UNIT_ID_BTS1__
  location_area_code __LAC__
  base_station_id_code __BSIC_BTS1__
  ms max power __MS_MAX_POWER__
  cell reselection hysteresis __CELL_RESEL_HYST__
  rxlev access min __RXLEV_ACCESS_MIN__
  radio-link-timeout 32
  channel allocator mode chan-req descending
  channel allocator mode assignment ascending
  channel allocator mode handover ascending
  rach tx integer 9
  rach max transmission 7
  ipa unit-id __IPA_UNIT_ID_BTS1__ 0
  oml ipa stream-id 255 line 0
  neighbor-list mode automatic
  codec-support fr
  gprs mode egprs
  gprs routing area 0
  gprs network-control-order nc0
  gprs power-control alpha 0
  gprs cell bvci 13
  gprs cell timer blocking-timer 3
  gprs cell timer blocking-retries 3
  gprs cell timer unblocking-retries 3
  gprs cell timer reset-timer 3
```

```
  gprs cell timer reset-retries 3
  gprs cell timer suspend-timer 10
  gprs cell timer suspend-retries 3
  gprs cell timer resume-timer 10
  gprs cell timer resume-retries 3
  gprs cell timer capability-update-timer 10
  gprs cell timer capability-update-retries 3
  gprs nsei 11
  gprs ns timer tns-block 3
  gprs ns timer tns-block-retries 3
  gprs ns timer tns-reset 3
  gprs ns timer tns-reset-retries 3
  gprs ns timer tns-test 30
  gprs ns timer tns-alive 3
  gprs ns timer tns-alive-retries 10
  gprs nsvc 0 nsvci 11
  gprs nsvc 0 local udp port 23002
  gprs nsvc 0 remote ip 192.168.2.10
  gprs nsvc 0 remote udp port 23000
  gprs nsvc 1 nsvci 0
  gprs nsvc 1 local udp port 0
  gprs nsvc 1 remote ip 0.0.0.0
  gprs nsvc 1 remote udp port 0
  trx 0
   rf_locked 0
   arfcn __ARFCN_BTS1__
   nominal power 23
   max_power_red 0
   rsl e1 tei 0
   timeslot 0
    phys_chan_config CCCH+SDCCH4
    hopping enabled 0
   timeslot 1
    phys_chan_config SDCCH8
    hopping enabled 0
   timeslot 2
    phys_chan_config TCH/F
    hopping enabled 0
   timeslot 3
    phys_chan_config TCH/F
    hopping enabled 0
   timeslot 4
    phys_chan_config TCH/F
    hopping enabled 0
   timeslot 5
    phys_chan_config PDCH
    hopping enabled 0
   timeslot 6
    phys_chan_config PDCH
    hopping enabled 0
   timeslot 7
    phys_chan_config PDCH
    hopping enabled 0
msc 0
 codec-list fr1 hr1 fr2 fr3 hr3
 allow-emergency deny
 amr-config ⟨1⟩_⟨2⟩k allowed  ×8
    ⟨⟩ = (12,2) (10,2) (7,95) (7,40) (6,70) (5,90) (5,15) (4,75)
 amr-payload octet-aligned
 msc-addr addr-msc
 asp-protocol m3ua
 lcls-mode bts-loop
 lcls-codec-mismatch allowed
bsc
 mid-call-timeout 0
end
```

3.18 /opt/GSM/osmo-operator/configs/osmo-bts-trx-bts1.cfg

1615 octets, 70 lignes → 70 lignes

```
!
! osmo-bts-trx-bts1.cfg - OsmoBTS BTS#1 (fake_trx side-car), generated by
! start-native.sh (faketrx-qemu mode).
! Not to be confused with /etc/osmocom/osmo-bts-trx.cfg (BTS#0, QEMU).
!
log stderr
 logging color 1
 logging print category-hex 0
 logging print category 1
 logging timestamp 0
 logging print file basename last
 logging print level 1
!
log gsmtap __GATEWAY_IP__
 logging filter all 1
 logging color 0
 logging timestamp 0
 logging print category 1
 logging print level 1
 logging level rsl info
 logging level oml info
 logging level rr notice
 logging level trx notice
 logging level abis debug
!
vty telnet-port 4250
!
ctrl
 bind 127.0.0.2 4239
!
line vty
 no login
!
e1_input
 e1_line 0 driver ipa
 e1_line 0 port 0
 no e1_line 0 keepalive
!
phy 0
 instance 0
  osmotrx rx-gain 40
  osmotrx tx-attenuation 50
 osmotrx ip local 127.0.0.1
 osmotrx ip remote 127.0.0.1
 osmotrx base-port local 5820
 osmotrx base-port remote 5720
 ! Advance on the fake_trx clock: with the default fn-advance 2 the bursts
 ! arrive late and fake_trx drops them ("Stale TRXD message"). 10 frames are
 ! enough for the fake TRX (the bridge BTS#0 uses 20, see osmo-bts-trx.cfg).
 osmotrx fn-advance 10
 osmotrx rts-advance 5
!
bts 0
 band DCS1800
 ipa unit-id __IPA_UNIT_ID_BTS1__ 0
 oml remote-ip 127.0.0.1
 pcu-socket /tmp/pcu_bts2
 rtp jitter-buffer 100
 paging queue-size 200
 paging lifetime 0
 min-qual-rach 50
 min-qual-norm -5
```

```
 gsmtap-sapi ccch
 gsmtap-sapi pdtch
 trx 0
  power-ramp max-initial 23000 mdBm
  power-ramp step-size 2000 mdB
  power-ramp step-interval 1
  ms-power-control osmo
  phy 0 instance 0
```

3.19 /opt/GSM/osmo-operator/configs/osmo-bts-trx.cfg

1427 octets, 60 lignes → 60 lignes

```
!
! osmo-bts-trx.cfg - OsmoBTS BTS#0 (osmo-bts-trx, TRXD towards fake_trx /
! Calypso TRX). Base: OsmoBTS (192.168.1.15-bc49-dirty) configuration saved
! from vty.
!!
!
log file /var/log/osmocom/osmo-bts-trx.log
 logging color 1
 logging timestamp 1
 logging print category-hex 0
 logging print category 1
 logging print file basename last
 logging print level 1
log stderr
 logging color 1
 logging print category-hex 0
 logging print category 1
 logging timestamp 0
 logging print file basename last
 logging print level 1
!
line vty
 no login
!
e1_input
 e1_line 0 driver ipa
 e1_line 0 port 0
 no e1_line 0 keepalive
!
phy 0
 instance 0
  osmotrx rx-gain 40
  osmotrx tx-attenuation 50
 osmotrx ip local 127.0.0.1
 osmotrx ip remote 127.0.0.1
 no osmotrx timing-advance-loop
 ! Calypso TRX (transceiver firmware, --osmo-trx): IND CLOCK carries the
 ! firmware fn; with the defaults (fn-advance 2, rts-advance 3) bursts reach
 ! the firmware too late and are dropped (STALE BURST). Real osmo-trx values:
 ! 20 and 5.
 osmotrx fn-advance 20
 osmotrx rts-advance 5

bts 0
 band GSM900
 ipa unit-id __IPA_UNIT_ID__ 0
 oml remote-ip 127.0.0.1
 rtp jitter-buffer 100
 paging queue-size 200
 paging lifetime 0
 min-qual-rach 50
```

```
 min-qual-norm -5
 gsmtap-sapi ccch
 gsmtap-sapi pdtch
 trx 0
  power-ramp max-initial 23000 mdBm
  power-ramp step-size 2000 mdB
  power-ramp step-interval 1
  ms-power-control osmo
  phy 0 instance 0
```

3.20 /opt/GSM/osmo-operator/configs/osmo-bts-virtual.cfg

3223 octets, 133 lignes → 133 lignes

```
!
! osmo-bts-virtual.cfg - virtual BTS (virtphy mode), operator __OPERATOR_ID__
!
! Unlike osmo-bts-trx.cfg (TRXD/fake_trx), osmo-bts-virtual talks to virtphy
! over multicast UDP; no trxcon or fake_trx needed.
!
! Chain: mobile <-> virtphy <-> (multicast) <-> osmo-bts-virtual <-> BSC
!
log file /var/log/osmocom/osmo-bts-virtual.log
 logging color 1
 logging timestamp 1
 logging print category-hex 0
 logging print category 1
 logging print file basename last
 logging print level 1
 logging level rsl info
 logging level oml info
 logging level rll notice
 logging level rr notice
 logging level meas notice
 logging level pag info
 logging level l1c info
 logging level l1p info
 logging level dsp error
 logging level pcu notice
 logging level ho debug
 logging level trx notice
 logging level loop notice
 logging level abis debug
 logging level rtp notice
 logging level lglobal notice
 logging level llapd notice
 logging level linp notice
 logging level lmux notice
 logging level lmi notice
 logging level lmib notice
 logging level lsms notice
 logging level lctrl notice
 logging level lgtp notice
 logging level lstats error
log stderr
 logging color 1
 logging print category-hex 0
 logging print category 1
 logging timestamp 0
 logging print file basename last
 logging print level 1
 logging level rsl info
 logging level oml info
 logging level rll notice
```

```
 logging level rr notice
 logging level meas notice
 logging level pag info
 logging level l1c info
 logging level l1p info
 logging level dsp error
 logging level pcu notice
 logging level ho debug
 logging level trx notice
 logging level loop notice
 logging level abis debug
 logging level rtp notice
 logging level lglobal notice
 logging level llapd notice
 logging level linp notice
 logging level lmux notice
 logging level lmi notice
 logging level lmib notice
 logging level lsms notice
 logging level lctrl notice
 logging level lgtp notice
 logging level lstats error
!
log gsmtap __GATEWAY_IP__
 logging filter all 1
 logging color 0
 logging timestamp 0
 logging print category 1
 logging print level 1
 logging level rsl info
 logging level oml info
 logging level rll notice
 logging level rr notice
 logging level meas notice
 logging level pag info
 logging level l1c info
 logging level l1p info
 logging level dsp error
 logging level pcu notice
 logging level ho debug
 logging level trx notice
 logging level loop notice
 logging level abis debug
 logging level rtp notice
 logging level lglobal notice
 logging level llapd notice
 logging level linp notice
 logging level lmux notice
 logging level lmi notice
 logging level lmib notice
 logging level lsms notice
 logging level lctrl notice
 logging level lgtp notice
 logging level lstats error
!
line vty
 no login
!
e1_input
 e1_line 0 driver ipa
 e1_line 0 port 0
 no e1_line 0 keepalive
!
phy 0
 instance 0
!
```

```
bts 0
 band DCS1800
 ipa unit-id __IPA_UNIT_ID__ 0
 oml remote-ip 127.0.0.1
 rtp jitter-buffer 100
 paging queue-size 200
 paging lifetime 0
 min-qual-rach 50
 min-qual-norm -5
 gsmtap-sapi ccch
 gsmtap-sapi pdtch
 trx 0
  power-ramp max-initial 23000 mdBm
  power-ramp step-size 2000 mdB
  power-ramp step-interval 1
  ms-power-control osmo
  phy 0 instance 0
```

3.21 /opt/GSM/osmo-operator/configs/osmo-bts.cfg

1974 octets, 90 lignes → 90 lignes

```
!
! osmo-bts.cfg - generic OsmoBTS configuration. Base: OsmoBTS
! (192.168.1.15-bc49-dirty) configuration saved from vty.
!!
!
log file /var/log/osmocom/osmo-bts.log
 logging color 1
 logging timestamp 1
 logging print category-hex 0
 logging print category 1
 logging print file basename last
 logging print level 1
log stderr
 logging color 1
 logging print category-hex 0
 logging print category 1
 logging timestamp 0
 logging print file basename last
 logging print level 1
!
log gsmtap __GATEWAY_IP__
 logging filter all 1
 logging color 0
 logging timestamp 0
 logging print category 1
 logging print level 1
 logging level rsl info
 logging level oml info
 logging level rll notice
 logging level rr notice
 logging level meas notice
 logging level pag info
 logging level l1c info
 logging level l1p info
 logging level dsp error
 logging level pcu notice
 logging level ho debug
 logging level trx notice
 logging level loop notice
 logging level abis debug
 logging level rtp notice
 logging level lglobal notice
```

```
 logging level llapd notice
 logging level linp notice
 logging level lmux notice
 logging level lmi notice
 logging level lmib notice
 logging level lsms notice
 logging level lctrl notice
 logging level lgtp notice
 logging level lstats error
!
line vty
 no login
!
e1_input
 e1_line 0 driver ipa
 e1_line 0 port 0
 no e1_line 0 keepalive
!
phy 0
 instance 0
  osmotrx rx-gain 40
  osmotrx tx-attenuation 50
 osmotrx ip local 127.0.0.1
 osmotrx ip remote 127.0.0.1
 no osmotrx timing-advance-loop

bts 0
 band GSM900
 ipa unit-id __IPA_UNIT_ID__ 0
 oml remote-ip 127.0.0.1
 rtp jitter-buffer 100
 paging queue-size 200
 paging lifetime 0
 min-qual-rach 50
 min-qual-norm -5
 gsmtap-sapi ccch
 gsmtap-sapi bcch
 gsmtap-sapi rach
 gsmtap-sapi agch
 gsmtap-sapi pch
 gsmtap-sapi sdcch
 gsmtap-sapi sacch
 trx 0
  power-ramp max-initial 23000 mdBm
  power-ramp step-size 2000 mdB
  power-ramp step-interval 1
  ms-power-control osmo
  phy 0 instance 0
```

3.22 /opt/GSM/osmo-operator/configs/osmo-ggsn.cfg

1952 octets, 81 lignes → 81 lignes

```
!
! osmo-ggsn.cfg - OsmoGGSN (GPRS gateway) configuration
!
log file /var/log/osmocom/osmo-ggsn.log
 logging color 1
 logging timestamp 1
 logging print category-hex 0
 logging print category 1
 logging print file basename last
 logging print level 1
 logging level ip info
```

```
 logging level tun info
 logging level ggsn info
 logging level sgsn notice
 logging level icmp6 notice
 logging level lglobal notice
 logging level llapd notice
 logging level linp notice
 logging level lmux notice
 logging level lmi notice
 logging level lmib notice
 logging level lsms notice
 logging level lctrl notice
 logging level lgtp info
 logging level lstats notice
 logging level lgsup notice
 logging level loap notice
 logging level lss7 notice
 logging level lsccp notice
 logging level lsua notice
 logging level lm3ua notice
 logging level lmgcp notice
log stderr
 logging color 1
 logging print category-hex 0
 logging print category 1
 logging timestamp 0
 logging print file basename last
 logging print level 1
 logging level ip info
 logging level tun info
 logging level ggsn info
 logging level sgsn notice
 logging level icmp6 notice
 logging level lglobal notice
 logging level llapd notice
 logging level linp notice
 logging level lmux notice
 logging level lmi notice
 logging level lmib notice
 logging level lsms notice
 logging level lctrl notice
 logging level lgtp info
 logging level lstats notice
 logging level lgsup notice
 logging level loap notice
 logging level lss7 notice
 logging level lsccp notice
 logging level lsua notice
 logging level lm3ua notice
 logging level lmgcp notice
!
stats interval 5
!
line vty
 no login
!
ggsn ggsn0
 gtp state-dir /var/lib/osmocom/osmo-ggsn
 gtp bind-ip __CONTAINER_IP__
 apn __APN__
  gtpu-mode tun
  tun-device apn0
  type-support v4
  ip prefix dynamic 176.16.32.0/24
  ip dns 0 192.168.1.254
  ip dns 1 1.1.1.1
```

```
  ip ifconfig 176.16.32.0/24
  no shutdown
 default-apn __APN__
 no shutdown ggsn
```

3.23 /opt/GSM/osmo-operator/configs/osmo-hlr.cfg

3910 octets, 145 lignes → 145 lignes

```
!
! osmo-hlr.cfg - OsmoHLR template for operator __OPERATOR_ID__
! SMS-over-GSUP enabled via gsup-smsc-proto
!
log file /var/log/osmocom/osmo-hlr.log
 logging color 1
 logging timestamp 1
 logging filter all 1
 logging print category-hex 0
 logging print category 1
 logging print extended-timestamp 1
 logging print level 1
 logging print file basename
 logging level main debug
 logging level db debug
 logging level auc debug
 logging level ss debug
 logging level lglobal debug
 logging level llapd debug
 logging level linp debug
 logging level lmux debug
 logging level lmi debug
 logging level lmib debug
 logging level lsms debug
 logging level lctrl debug
 logging level lgtp debug
 logging level lstats debug
 logging level lgsup debug
 logging level loap debug
 logging level lss7 debug
 logging level lsccp debug
 logging level lsua debug
 logging level lm3ua debug
 logging level lmgcp debug
 logging level ljibuf debug
 logging level lrspro debug
 logging filter all 0
 logging print category-hex 1
 logging print category 0
 logging print file 1
 logging level main notice
 logging level db notice
 logging level auc notice
 logging level ss notice
 logging level lglobal notice
 logging level llapd notice
 logging level linp notice
 logging level lmux notice
 logging level lmi notice
 logging level lmib notice
 logging level lsms notice
 logging level lctrl notice
 logging level lgtp notice
 logging level lstats notice
 logging level lgsup notice
```

```
 logging level loap notice
 logging level lss7 notice
 logging level lsccp notice
 logging level lsua notice
 logging level lm3ua notice
 logging level lmgcp notice
 logging level ljibuf notice
 logging level lrspro notice
log stderr
 logging filter all 1
 logging color 1
 logging print category-hex 0
 logging print category 1
 logging print extended-timestamp 1
 logging print level 1
 logging print file basename
 logging level main debug
 logging level db debug
 logging level auc debug
 logging level ss debug
 logging level lglobal debug
 logging level llapd debug
 logging level linp debug
 logging level lmux debug
 logging level lmi debug
 logging level lmib debug
 logging level lsms debug
 logging level lctrl debug
 logging level lgtp debug
 logging level lstats debug
 logging level lgsup debug
 logging level loap debug
 logging level lss7 debug
 logging level lsccp debug
 logging level lsua debug
 logging level lm3ua debug
 logging level lmgcp debug
 logging level ljibuf debug
 logging level lrspro debug
 logging filter all 0
 logging color 1
 logging print category-hex 1
 logging print category 0
 logging timestamp 0
 logging print file 1
 logging level main notice
 logging level db notice
 logging level auc notice
 logging level ss notice
 logging level lglobal notice
 logging level llapd notice
 logging level linp notice
 logging level lmux notice
 logging level lmi notice
 logging level lmib notice
 logging level lsms notice
 logging level lctrl notice
 logging level lgtp notice
 logging level lstats notice
 logging level lgsup notice
 logging level loap notice
 logging level lss7 notice
 logging level lsccp notice
 logging level lsua notice
 logging level lm3ua notice
 logging level lmgcp notice
```

```
 logging level ljibuf notice
 logging level lrspro notice
!
stats interval 5
!
line vty
 no login
!
hlr
 subscriber-create-on-demand 5 cs+ps
 store-imei
 database /var/lib/osmocom/hlr.db
 gsup
  bind ip 127.0.0.2
  ipa-name HLR-OP__OPERATOR_ID__
 ussd route prefix *#100# internal own-msisdn
 ussd route prefix *#101# internal own-imsi
 !
 ! ── SMS-over-GSUP: proto-SMSC connected via GSUP ──────────────────────────
 ! The proto-smsc-daemon connects to the HLR with the IPA name SMSC-OP<N>.
 ! SC-address = fictitious number, unique per operator (reserved NANP range).
 !
 smsc entity SMSC-OP__OPERATOR_ID__
 smsc default-route SMSC-OP__OPERATOR_ID__
```

3.24 /opt/GSM/osmo-operator/configs/osmo-mgw.cfg

2304 octets, 90 lignes → 90 lignes

```
!
! osmo-mgw.cfg - OsmoMGW media gateway (RTP bridge between BSC/MSC and Asterisk).
! Template: placeholders are substituted by apply_config_templates()
! (generate_configs.sh) at container/ISO start.
!
log file /var/log/osmocom/osmo-mgw.log
 logging color 1
 logging timestamp 1
 logging filter all 1
 logging print category-hex 1
 logging print category 0
 logging print file 1
 logging level rtp notice
 logging level lglobal notice
 logging level llapd notice
 logging level linp notice
 logging level lmux notice
 logging level lmi notice
 logging level lmib notice
 logging level lsms notice
 logging level lctrl notice
 logging level lgtp notice
 logging level lstats notice
 logging level lgsup notice
 logging level loap notice
 logging level lss7 notice
 logging level lsccp notice
 logging level lsua notice
 logging level lm3ua notice
 logging level lmgcp notice
 logging level ljibuf notice
 logging level lrspro notice
log stderr
 logging filter all 1
 logging color 0
```

```
 logging print category-hex 1
 logging print category 0
 logging timestamp 0
 logging print file 1
 logging level rtp notice
 logging level lglobal notice
 logging level llapd notice
 logging level linp notice
 logging level lmux notice
 logging level lmi notice
 logging level lmib notice
 logging level lsms notice
 logging level lctrl notice
 logging level lgtp notice
 logging level lstats notice
 logging level lgsup notice
 logging level loap notice
 logging level lss7 notice
 logging level lsccp notice
 logging level lsua notice
 logging level lm3ua notice
 logging level lmgcp notice
 logging level ljibuf notice
 logging level lrspro notice
!
stats interval 5
!
line vty
 no login
!
!
mgcp
  domain mgw
  bind ip 127.0.0.1
  bind port 2427
  rtp port-range 4002 16001
  rtp bind-ip 127.0.0.1
  rtp ip-probing
!
! RTP keep-alive stays off: the 1-byte NAT-punching dummy packet (0x23) is
! classified by Asterisk as a DTLS record ("Received SSL traffic on RTP
! instance without an SSL session"), and the peer is on loopback anyway.
  no rtp keep-alive
  rtcp-omit
  rtp-patch ssrc
  rtp-patch timestamp
  no rtp-patch rfc5993hr
  sdp audio-payload number 98
  sdp audio-payload name GSM
  sdp audio-payload send-ptime
  sdp audio-payload send-name
  loop 0
  number endpoints 31
  allow-transcoding
  osmux off
```

3.25 /opt/GSM/osmo-operator/configs/osmo-msc.cfg

5582 octets, 210 lignes → 210 lignes

```
!
! osmo-msc.cfg - OsmoMSC for operator __OPERATOR_ID__ (MCC __MCC__ / MNC __MNC__).
! Template: placeholders (__PC_*__, __RCTX_*__, __HLR_IP__, __OP_NAME__, ...)
! are substituted by apply_config_templates() (generate_configs.sh).
```

```
!
log file /var/log/osmocom/osmo-msc.log
# cc, mm, mncc and lsms are logged at "info" so that DISCONNECT / RELEASE and
# the fate of each RP-DATA are visible. For a given category the last
# "logging level" line wins, so keep the "info" lines after the "notice" ones.
 logging level mncc info
 logging level cc info
 logging color 1
 logging timestamp 1
 logging filter all 1
 logging print category-hex 1
 logging print category 0
 logging print thread-id 0
 logging print file 1
 logging level rll notice
 logging level cc info
 logging level bcc notice
 logging level gcc notice
 logging level mm info
 logging level rr notice
 logging level mncc info
 logging level pag notice
 logging level msc notice
 logging level mgcp notice
 logging level ho notice
 logging level db notice
 logging level ref notice
 logging level ctrl notice
 logging level smpp notice
 logging level ranap notice
 logging level vlr notice
 logging level iucs notice
 logging level bssap notice
 logging level sgs notice
 logging level ss notice
 logging level asci notice
 logging level lglobal notice
 logging level llapd notice
 logging level linp notice
 logging level lmux notice
 logging level lmi notice
 logging level lmib notice
 logging level lsms info
 logging level lctrl notice
 logging level lgtp notice
 logging level lstats notice
 logging level lgsup notice
 logging level loap notice
 logging level lss7 info
 logging level lsccp notice
 logging level lsua notice
 logging level lm3ua info
 logging level lmgcp notice
 logging level ljibuf notice
 logging level lrspro notice
 logging level lns notice
 logging level lbssgp notice
 logging level lnsdata notice
 logging level lnssignal notice
 logging level liuup notice
 logging level lpfcp notice
 logging level lcsn1 notice
 logging level lio notice
 logging level ltcap notice
log stderr
 logging filter all 1
```

```
 logging color 1
 logging print category-hex 1
 logging print category 0
 logging print thread-id 0
 logging timestamp 0
 logging print file 1
 logging level rll notice
 logging level cc notice
 logging level bcc notice
 logging level gcc notice
 logging level mm notice
 logging level rr notice
 logging level mncc notice
 logging level pag notice
 logging level msc notice
 logging level mgcp notice
 logging level ho notice
 logging level db notice
 logging level ref notice
 logging level ctrl notice
 logging level smpp notice
 logging level ranap notice
 logging level vlr notice
 logging level iucs notice
 logging level bssap notice
 logging level sgs notice
 logging level ss notice
 logging level asci notice
 logging level lglobal notice
 logging level llapd notice
 logging level linp notice
 logging level lmux notice
 logging level lmi notice
 logging level lmib notice
 logging level lsms notice
 logging level lctrl notice
 logging level lgtp notice
 logging level lstats notice
 logging level lgsup notice
 logging level loap notice
 logging level lss7 info
 logging level lsccp notice
 logging level lsua notice
 logging level lm3ua info
 logging level lmgcp notice
 logging level ljibuf notice
 logging level lrspro notice
 logging level lns notice
 logging level lbssgp notice
 logging level lnsdata notice
 logging level lnssignal notice
 logging level liuup notice
 logging level lpfcp notice
 logging level lcsn1 notice
 logging level lio notice
 logging level ltcap notice
!
stats interval 5
!
line vty
 no login
!
cs7 instance 0
 network-indicator international
 point-code __PC_MSC__
!
```

```
 ! MSC -> STP: same container, loopback.
 asp asp-to-stp 2905 2906 m3ua
  remote-ip 127.0.0.2
  local-ip  127.0.0.1
  role asp
  sctp-role client
  no shutdown
!
 as as-msc m3ua
  asp asp-to-stp
  routing-key __RCTX_MSC__ __PC_MSC__
  traffic-mode override
!
 sccp-address addr-bsc
  routing-indicator PC
  point-code __PC_BSC__
  subsystem-number 254
!
network
 network country code __MCC__
 mobile network code __MNC__
 short name __OP_NAME__
 long name __OP_NAME__
 encryption __ENCRYPTION__
 authentication optional
 rrlp mode none
 mm info 0
 mgw 0
  remote-ip 127.0.0.1
msc
 mncc external /tmp/msc_mncc
 mncc guard-timeout 180
 ncss guard-timeout 30
 assign-tmsi
 ! X1 raised from the 5 s default: a full LU (auth, ciphering, TMSI REALLOC
 ! COMPLETE) on the slower-than-real-time DSP bench would otherwise be rejected.
 timer geran X1 30
 lcls-permitted
 check-imei-rqd early
 ! SMS-over-GSUP: SM-RP goes to the HLR (proto-smsc-daemon) instead of the
 ! internal SMS store.
 sms-over-gsup
 auth-tuple-max-reuse-count 3
 auth-tuple-reuse-on-error 1

mncc-int
 default-codec tch-f fr
 default-codec tch-h hr
smpp
 local-tcp-port 2775
 policy closed
 # smpp-first: the SMS is offered to the ESME before internal delivery.
 # tools/osmo-phonesim-banc.py binds here as ESME (bench phone = subscriber
 # 100101) and re-injects a copy to the mobile through the VTY so both ring.
 smpp-first
 esme msc_tester
  password osmocom1
  route prefix national isdn 33
  # The bench phone's number, in the four numbering forms an SMS may carry.
  route prefix unknown unknown 100101
  route prefix unknown isdn 100101
  route prefix national isdn 100101
  route prefix international isdn 100101
hlr
 remote-ip __HLR_IP__
 remote-port 4222
```

```
 ipa-name VLR-SDR-OP__OPERATOR_ID__
sgs
 local-port 29118
 local-ip 127.0.0.1
 vlr-name vlr.msc001.mnc001.mcc001.3gppnetwork.org
asci
 disable
 gcr
```

3.26 /opt/GSM/osmo-operator/configs/osmo-pcu.cfg

557 octets, 25 lignes → 25 lignes

```
!
! osmo-pcu.cfg - OsmoPCU (GPRS packet control unit, Gb towards the SGSN).
! Template: __GATEWAY_IP__ (GSMTAP sink) is substituted by
! apply_config_templates() (generate_configs.sh).
!
log file /var/log/osmocom/osmo-pcu.log
 logging color 1
 logging timestamp 1
 logging filter all 1
 logging print category 1
 logging print level 1
log gsmtap __GATEWAY_IP__
 logging filter all 1
 logging color 0
 logging timestamp 0
 logging print category 1
 logging print level 1
!
pcu
 flow-control-interval 10
 cs 1
 cs max 1 1
 alloc-algorithm a
 alpha 0
 gamma 0
```

3.27 /opt/GSM/osmo-operator/configs/osmo-sgsn.cfg

1040 octets, 47 lignes → 47 lignes

```
!
! osmo-sgsn.cfg - OsmoSGSN (GPRS core, Gb towards the PCU, GTP towards the GGSN).
! Template: placeholders are substituted by apply_config_templates()
! (generate_configs.sh).
!
log file /var/log/osmocom/osmo-sgsn.log
 logging color 1
 logging timestamp 1
 logging print category-hex 0
 logging print category 1
 logging print file basename last
 logging print level 1
log stderr
 logging color 1
 logging print category-hex 0
 logging print category 1
 logging timestamp 0
 logging print file basename last
 logging print level 1
```

```
line vty
 no login
!
sgsn
 gtp state-dir /var/lib/osmocom/osmo-sgsn
 gtp local-ip 127.0.0.1
 ggsn 0 remote-ip __CONTAINER_IP__
 ggsn 0 gtp-version 1
 ggsn 0 echo-interval 60
 authentication optional
 auth-policy remote
 gsup remote-ip __HLR_IP__
 gsup remote-port 4222
!
ns
 timer tns-block 3
 timer tns-block-retries 3
 timer tns-reset 3
 timer tns-reset-retries 3
 timer tns-test 30
 timer tns-alive 3
 timer tns-alive-retries 10
 bind udp local
  listen __CONTAINER_IP__ 23000
  accept-ipaccess
!
bssgp
```

3.28 /opt/GSM/osmo-operator/configs/osmo-sip-connector.cfg

870 octets, 24 lignes → 24 lignes

```
!
! osmo-sip-connector.cfg - MNCC <-> SIP bridge between OsmoMSC and Asterisk.
! Template: __SIP_REMOTE_IP__ is substituted by apply_config_templates()
! (generate_configs.sh); it defaults to 127.0.0.1 and can be overridden with
! the SIP_REMOTE_IP_OVERRIDE environment variable.
!
log file /var/log/osmocom/osmo-sip-connector.log
 logging level sip info
 logging level mncc info
 logging color 1
 logging timestamp 1
 logging filter all 1
 logging print category 1
 logging print level 1
!
mncc
  socket-path /tmp/msc_mncc
sip
  local 127.0.0.1 5061
  ! The SIP leg is local in both topologies (container and ISO): Asterisk
  ! listens on 0.0.0.0:5060 and only identifies this leg on loopback
  ! (pjsip.conf [gsm_msc-identify] match=127.0.0.1, contact sip:127.0.0.1:5061).
  ! The inter-operator backbone address must NOT be used here.
  remote __SIP_REMOTE_IP__ 5060
```

3.29 /opt/GSM/osmo-operator/configs/osmo-stp.cfg

2155 octets, 76 lignes → 76 lignes

```
!
! osmo-stp.cfg - OsmoSTP (SS7 Signal Transfer Point) for operator __OPERATOR_ID__.
! Template: placeholders are substituted by apply_config_templates()
! (generate_configs.sh).
!
! Topology:
!   [BSC __PC_BSC__] -+
!                     +- STP __PC_STP__ @ 127.0.0.1:2905 (local listener)
!   [MSC __PC_MSC__] -+        |
!                              | ASP client -> inter-STP __INTER_STP_IP__:2908
!                              v
!                       [Inter-STP 0.0.0]
!
! BSC and MSC connect to the STP over loopback (same container). The listener
! is bound to loopback only, so it is up before Docker attaches the private
! network.
!
log file /var/log/osmocom/osmo-stp.log
 logging color 1
 logging timestamp 1
 logging filter all 1
 logging print category-hex 0
 logging print category 1
 logging print extended-timestamp 1
 logging print level 1
 logging print file basename
 logging level lss7 info
 logging level lsccp info
 logging level lm3ua info
 logging level linp notice
log stderr
 logging filter all 1
 logging color 1
 logging print category-hex 0
 logging print category 1
 logging print extended-timestamp 1
 logging print level 1
 logging print file basename
 logging level lss7 info
 logging level lsccp info
 logging level lm3ua info
 logging level linp notice
!
stats interval 5
!
line vty
 no login
!
cs7 instance 0
 network-indicator international
 point-code __PC_STP__
!
 ! -- Local listener (BSC/MSC connect via 127.0.0.1 / 127.0.0.2) -----------
 xua rkm routing-key-allocation dynamic-permitted
 listen m3ua 2905
  accept-asp-connections dynamic-permitted
  local-ip 127.0.0.1
  local-ip 127.0.0.2
!
 ! -- Client link to the central inter-STP ----------------------------------
 asp asp-to-inter 2908 2910 m3ua
  remote-ip __INTER_STP_IP__
  local-ip __INTER_LOCAL_IP__
  role asp
  sctp-role client
  __INTER_STP_SHUTDOWN__
```

```
!
 as as-inter m3ua
  asp asp-to-inter
  routing-key __RCTX_INTER__ __PC_STP__
  traffic-mode override
!
 ! -- Routing: local routes (BSC/MSC) are created dynamically by accept-asp;
 ! -- everything else goes to the inter-STP.
 route-table system
  update route 0.0.0 0.0.0 linkset as-inter
```

3.30 /opt/GSM/osmo-operator/configs/osmo-trx.cfg

568 octets, 30 lignes → 30 lignes

```
!
! osmo-trx.cfg - OsmoTRX transceiver (TRX interface on 127.0.0.1, base port 5700).
!
log file /var/log/osmocom/osmo-trx.log
 logging color 1
 logging timestamp 1
 logging filter all 1
 logging print category 1
 logging print file basename
 logging level set-all info
log stderr
 logging filter all 1
 logging color 1
 logging print category 1
 logging timestamp 1
 logging print file basename
 logging level set-all info
!
line vty
 no login
!
trx
 bind-ip 127.0.0.1
 remote-ip 127.0.0.1
 base-port 5700
 egprs enable
 tx-sps 4
 rx-sps 4
 clock-ref internal
 chan 0
```

3.31 /opt/GSM/osmo-operator/configs/osmo-upf.cfg

413 octets, 16 lignes → 16 lignes

```
!
! osmo-upf.cfg - OsmoUPF (GTP user plane, PFCP on the container address).
! Template: __CONTAINER_IP__ is substituted by apply_config_templates()
! (generate_configs.sh).
!
log file /var/log/osmocom/osmo-upf.log
 logging color 1
 logging timestamp 1
 logging filter all 1
 logging print category 1
 logging print level 1
pfcp
```

```
 local-addr __CONTAINER_IP__
nft
 # netfilter requires no specific configuration
gtp
```

3.32 /opt/GSM/osmo-operator/configs/pjsip.conf

9455 octets, 309 lignes → 309 lignes

```
; ============================================================================
; pjsip.conf - Asterisk PJSIP configuration for operator __OPERATOR_ID__
;
; Template: __OPERATOR_ID__ / __CONTAINER_IP__ are substituted by
; apply_config_templates() (generate_configs.sh). Inter-operator trunks are
; appended at start by launch/start-docker.sh (generate_pjsip_interop_trunks).
;
; Linphone from the host:
;   Direct: __CONTAINER_IP__:5060  (Docker bridge)
;   linphone_A / tester -> 100  |  linphone_B / testerB -> 200
;   (100/200 are local SIP extensions, outside the MSISDN plan
;    <node>00<op><ms>: the dialplan handles them in [internal], next to
;    500/600/700.)
; ============================================================================

[global]
; debug=no: debug=yes logs every SIP message (REGISTER included) and floods
; /var/log/asterisk/messages. For a one-off capture use
; `asterisk -rx "pjsip set logger on"` (lost on restart); for a capture that
; survives a restart set debug=yes here, then set it back to no.
debug=no
type=global
user_agent=OsmoAsterisk-Op__OPERATOR_ID__
; Linphone clients and the WAN trunk all arrive through the Docker gateway
; 172.20.0.1 (NAT). Identify by username first so that the IP-only WAN trunk
; identify (network/setup-wan-interop.sh, match=172.20.0.1) does not swallow
; the Linphone REGISTERs.
endpoint_identifier_order=username,ip,anonymous

; ============================================================================
; TRANSPORT
; ============================================================================

[transport-udp]
type=transport
protocol=udp
bind=0.0.0.0
; No external_media_address / external_signaling_address: Asterisk announces
; the real address of its interface, which is correct here. Only add them
; behind a real NAT, with the real public address (they are only used for
; peers outside local_net).
local_net=172.20.0.0/16
local_net=127.0.0.0/8

; ============================================================================
; LINPHONE A - extension 100
; ============================================================================

[linphone_A]
type=endpoint
transport=transport-udp
context=internal
disallow=all
allow=alaw
allow=ulaw
```

```
allow=gsm
auth=linphone_A
aors=linphone_A
direct_media=no
rtp_symmetric=yes
force_rport=yes
rewrite_contact=yes
media_encryption=no
callerid=Linphone A <100>

[linphone_A]
type=auth
auth_type=userpass
username=linphone_A
password=tester

[linphone_A]
type=aor
max_contacts=3
remove_existing=yes
qualify_frequency=30
qualify_timeout=5.0

; ============================================================================
; LINPHONE B - extension 200
; ============================================================================

[linphone_B]
type=endpoint
transport=transport-udp
context=internal
disallow=all
allow=alaw
allow=ulaw
allow=gsm
auth=linphone_B
aors=linphone_B
direct_media=no
rtp_symmetric=yes
force_rport=yes
rewrite_contact=yes
media_encryption=no
callerid=Linphone B <200>

[linphone_B]
type=auth
auth_type=userpass
username=linphone_B
password=testerB

[linphone_B]
type=aor
max_contacts=3
remove_existing=yes
qualify_frequency=30
qualify_timeout=5.0

; ============================================================================
; GSM TRUNK <- OsmoMSC via osmo-sip-connector (127.0.0.1:5061)
;
; media_encryption=no is required: osmo-sip-connector sends plain RTP, not
;   SRTP/DTLS. Without it Asterisk may take RTP packets for DTLS ("Received
;   SSL traffic without SSL session") and drop the audio.
; ============================================================================

[gsm_msc-identify]
```

```
type=identify
endpoint=gsm_msc
match=127.0.0.1

[gsm_msc]
type=endpoint
transport=transport-udp
context=gsm_in
disallow=all
allow=alaw
allow=ulaw
allow=gsm
aors=gsm_msc
direct_media=no
rtp_symmetric=yes
force_rport=yes
rewrite_contact=yes
media_encryption=no

[gsm_msc]
type=aor
contact=sip:127.0.0.1:5061
qualify_frequency=10

; ============================================================================
; INTER-OPERATOR TRUNKS - generated by launch/start-docker.sh
; ============================================================================
;
; ============================================================================
; ZADARMA TRUNK (external SIP operator) - https://zadarma.com
;
; Credentials: the SIP login and SIP password from the Zadarma customer area,
; section "My SIP". They are dedicated SIP credentials, not the web account
; login/password.
;
; After editing:  asterisk -rx "pjsip reload"
; Check:          asterisk -rx "pjsip show registrations"   -> Registered
;
; Dialing:
;   outgoing -> prefix 00 + international number   (e.g. 0033612345678)
;   incoming -> dialplan context [zadarma_in]
; ============================================================================

[zadarma_auth]
type=auth
auth_type=userpass
username=533110
password=3eahh8IWZC

[zadarma]
type=aor
contact=sip:sip.zadarma.com
qualify_frequency=60
qualify_timeout=5.0

[zadarma_reg]
type=registration
transport=transport-udp
outbound_auth=zadarma_auth
server_uri=sip:sip.zadarma.com
client_uri=sip:533110@sip.zadarma.com
contact_user=533110
retry_interval=60
forbidden_retry_interval=600
expiration=300
line=yes
```

```
endpoint=zadarma

[zadarma]
type=endpoint
transport=transport-udp
context=zadarma_in
disallow=all
; Codec order: gsm first. PJSIP offers codecs in this order and the peer takes
; the first it accepts; the MS leg is GSM-FR, so a gsm answer lets
; bridge_native_rtp relay RTP without a gsm<->slin<->alaw double transcode.
; If the peer refuses gsm it falls back to ulaw/alaw and the call still works.
; Check with `core show channel PJSIP/zadarma-...` during a call:
; NativeFormats (gsm) = accepted, (alaw) = refused.
; Known limitation: while [sub-record] installs MixMonitor + JITTERBUFFER the
; channel leaves the native bridge and is transcoded to slin anyway.
allow=gsm
allow=ulaw
allow=alaw
outbound_auth=zadarma_auth
aors=zadarma
from_user=533110
from_domain=sip.zadarma.com
; NAT: Asterisk sits behind the Docker gateway. Without these four lines RTP
; is sent to a private address and the call is one-way silent.
direct_media=no
rtp_symmetric=yes
force_rport=yes
rewrite_contact=yes
; Zadarma does not use SRTP on this account type. Keep 'no', otherwise
; Asterisk may take plain RTP for DTLS and drop the audio (same trap as the
; gsm_msc trunk above).
media_encryption=no
dtmf_mode=rfc4733

[zadarma-identify]
type=identify
endpoint=zadarma
match=sip.zadarma.com
; endpoint_identifier_order (top of this file) is username,ip,anonymous: IP
; identification only runs after username, so this match cannot swallow the
; Linphone REGISTERs.

; ============================================================================
; ZADARMA PBX TRUNK (PBX extension) - pbx.zadarma.com
;
; Second registration, as extension 100 of PBX 578163. Incoming calls on the
; number normally arrive through [zadarma] (the number is connected to SIP
; account 533110); this trunk is a fallback, both land in [zadarma_in].
;   Server: pbx.zadarma.com   (not sip.zadarma.com: two separate platforms)
;   Login:  578163-100        (PBX number + dash + extension)
;
; Three things must be set in the Zadarma customer area (not covered here):
;   1. The extension password must be SAVED ("PBX extensions settings" page);
;      a generated but unsaved password gives a 401 loop.
;   2. "Restriction by IP" must be the bench's public IP; a changed IP gives
;      403 Forbidden on REGISTER.
;   3. The number's incoming call scenario must route to extension 100.
;
; Check:  asterisk -rx "pjsip show registrations"
;         -> zadarma_pbx_reg/sip:pbx.zadarma.com ... Registered
;         and the PBX page shows the extension "online".
; ============================================================================

[zadarma_pbx_auth]
type=auth
auth_type=userpass
```

```
username=578163-100
; The extension password as saved in the Zadarma PBX page (see above).
password=QsK3m8fmNE

[zadarma_pbx]
type=aor
contact=sip:pbx.zadarma.com
qualify_frequency=60
qualify_timeout=5.0

[zadarma_pbx_reg]
type=registration
transport=transport-udp
outbound_auth=zadarma_pbx_auth
server_uri=sip:pbx.zadarma.com
client_uri=sip:578163-100@pbx.zadarma.com
contact_user=578163-100
retry_interval=60
; 600 s after a 403/404: the extension is limited to 3 lines and Zadarma
; blacklists an IP that hammers refused REGISTERs.
forbidden_retry_interval=600
expiration=300
; line=yes: Asterisk tags the registered Contact and matches the incoming
; INVITE on that tag before any IP identification, so PBX calls land in
; [zadarma_in] and not in "anonymous".
line=yes
endpoint=zadarma_pbx

[zadarma_pbx]
type=endpoint
transport=transport-udp
context=zadarma_in
disallow=all
; Same codec order as [zadarma]: gsm first to avoid the gsm<->slin<->alaw
; double transcode with the MS leg; falls back to ulaw/alaw if refused.
allow=gsm
allow=ulaw
allow=alaw
outbound_auth=zadarma_pbx_auth
aors=zadarma_pbx
from_user=578163-100
from_domain=pbx.zadarma.com
; NAT: same four lines as the [zadarma] trunk, same reasons.
direct_media=no
rtp_symmetric=yes
force_rport=yes
rewrite_contact=yes
media_encryption=no
dtmf_mode=rfc4733

[zadarma_pbx-identify]
type=identify
endpoint=zadarma_pbx
; pbx.zadarma.com and sip.zadarma.com resolve to different addresses, so the
; two identify blocks do not overlap. Safety net behind line=yes.
match=pbx.zadarma.com
```

3.33 /opt/GSM/osmo-operator/configs/rtp.conf

373 octets, 13 lignes → 13 lignes

```
; rtp.conf - Asterisk RTP configuration: port range for voice media.
; Template: __RTP_START__ / __RTP_END__ are substituted by
; apply_config_templates() (generate_configs.sh).
```

```
;
; With an external Linphone these ports must be published by Docker (-p) and
; opened in the firewall.

[general]
rtpstart=__RTP_START__
rtpend=__RTP_END__
strictrtp=no
icesupport=no
stunaddr=
```

3.34 /opt/GSM/osmo-operator/configs/sms-routing.conf

1245 octets, 49 lignes → 39 lignes (2 groupes compactés)

```
# ============================================================================
# sms-routing.conf - inter-operator SMS routing table
#
# Template: __XXX__ placeholders are substituted by apply_config_templates()
# (generate_configs.sh) at container start.
#
# To add an operator:
#   1. add its IP to [operators]
#   2. add its MSISDN prefixes to [routes]
# ============================================================================

[local]
# ID of this operator (substituted at start)
operator_id = __OPERATOR_ID__

[operators]
# operator_id = container_ip
# Addresses on the Docker bridge network (172.20.0.0/24)
1 = 172.20.0.11
2 = 172.20.0.12
# 3 = 172.20.0.13

[routes]
# prefix = operator_id
# Longest-prefix match: the longest matching prefix wins.
#
# -- MSISDN prefixes per operator (adapt to the numbering plan) --------------
#
# Operator 1: extensions 1xxx and MSISDN +33601xxxxxx
⟨1⟩ = 1  ×7
    ⟨⟩ = (10000) (1001) (1002) (1003) (1004) (1005) (33601)
#
# Operator 2: extensions 2xxx and MSISDN +33602xxxxxx
⟨1⟩ = 2  ×7
    ⟨⟩ = (20000) (2001) (2002) (2003) (2004) (2005) (33602)
#
# -- Default route (optional) ------------------------------------------------
# If no prefix matches, the SMS is not routed. Uncomment for a fallback:
# * = 1
```

3.35 /opt/GSM/osmo-operator/configs/conky/osmo-conky.conf

5660 octets, 88 lignes → 88 lignes

```
-- ============================================================================
--  osmo-conky.conf - Conky dashboard of the bench (live ISO and installed system)
-- ============================================================================
--  Started by /etc/xdg/autostart/osmo-conky.desktop (installed by build-iso.sh
--  in the desktop block) for every GNOME session: root on the live USB key,
--  the user created by Calamares on an installed disk.
--      conky -c /opt/GSM/osmo-operator/configs/conky/osmo-conky.conf
--
--  The bench lines come from tools/conky-osmo-status.sh (execpi: its output is
--  conky text with its own colours). The path /opt/GSM/osmo-operator is fixed:
--  the repository lives there on both the live key and the installed disk.
--
--  The SELECTOR (◄ ... ►): the central block follows the chosen stop - op1,
--  the running docker operators, the inter-STP hub, 4G. Conky receives no
--  clicks: the arrows are driven with Ctrl+AltGr+Left/Right, with the arrows
--  of the panel (tools/osmo-panel.py) or with "osmo-op --next" (tools/osmo-op.sh).
--  The list of stops is that of "osmo-op --list-vue".
--
--  Appearance: Ubuntu font for all text, Ubuntu Mono for what aligns in
--  columns. The dots ● ○ and arrows ↓ ↑ ◄ ►, which Ubuntu lacks, are drawn
--  glyph by glyph in DejaVu Sans 9 by the status script (OK/KO/WARN, FS/FU):
--  Conky does not fall back to another font. osmo-conky.lua (Cairo) paints a
--  translucent dark rounded panel behind the text. Colours by kind of
--  information: titles and CPU blue (color1), OK green (color2), missing red
--  (color3), warnings and upload orange (color4), labels grey (color5),
--  hostname and disk violet (color6), time and highlighted values yellow
--  (color7).
--  Conky is on screen only while a bench runs: osmo-panel.py fades it in and
--  out (window opacity) together with the panel.
conky.config = {
    alignment = 'top_right',
    gap_x = 20, gap_y = 44,
    minimum_width = 400, maximum_width = 400,
    update_interval = 2.0,
    total_run_times = 0,
    background = false,
    double_buffer = true,
    no_buffers = true,
    cpu_avg_samples = 2, net_avg_samples = 2,
    own_window = true,
    own_window_type = 'desktop',
    own_window_transparent = false,
    own_window_argb_visual = true,
    -- the panel is painted by osmo-conky.lua (rounded corners): the window
    -- itself stays fully transparent
    own_window_argb_value = 0,
    own_window_colour = '0b1020',
    own_window_hints = 'undecorated,below,sticky,skip_taskbar,skip_pager',
    border_inner_margin = 18, border_outer_margin = 0,
    draw_borders = false, draw_shades = false, draw_outline = false, draw_graph_borders = false,
    lua_load = '/opt/GSM/osmo-operator/configs/conky/osmo-conky.lua',
    lua_draw_hook_pre = 'panneau',
    use_xft = true, font = 'Ubuntu:size=9', xftalpha = 1,
    override_utf8_locale = true,
    default_color = 'e8eef6',
    color1 = '5ac8fa',   -- blue    : titles, CPU, values
    color2 = '3fe08a',   -- green   : OK, RAM, download rate
    color3 = 'ff5c6c',   -- red     : missing, failed
    color4 = 'ffb347',   -- orange  : warnings, upload rate, load
    color5 = '9aa7b8',   -- grey    : labels, rules
    color6 = 'b48cff',   -- violet  : the machine, the disk, the operator
    color7 = 'ffd166',   -- yellow  : time, highlighted values
    default_bar_width = 180, default_bar_height = 5,
    default_graph_width = 364, default_graph_height = 26,
    text_buffer_size = 4096,
};
```

```
conky.text = [[
${font Ubuntu:bold:size=16}${color1}OSMO${color5}-${color6}OPERATOR${font}${color}${alignr}${font
 ↪  Ubuntu:size=8}${color5}${execi 60 . /etc/os-release 2>/dev/null; echo
 ↪  "${VARIANT:-$PRETTY_NAME}"}${font}${color}
${voffset -2}${color5}${hr 1}${color}
${voffset 2}${font Ubuntu Mono:size=9}${color5}hôte   ${color6}${nodename}${color}${alignr}${execpi 30
 ↪  /opt/GSM/osmo-operator/tools/conky-osmo-status.sh role}
${color5}noyau  ${color}${kernel}${alignr}${color5}up ${color7}${uptime_short} ${color5}· charge
 ↪  ${color4}${loadavg 1}${color}${font}
${voffset 8}${font Ubuntu:bold:size=10}${color1}Système${font}${color}${alignr}${font
 ↪  Ubuntu:size=8}${color5}${freq_g} GHz · ${color4}${execi 10 t=$(cat /sys/class/thermal/thermal_zone0/temp
 ↪  2>/dev/null); [ -n "$t" ] && echo "$((t/1000))°C"}${font}${color}
${voffset 3}${color5}CPU${color}     ${color1}${cpu cpu0}%${color}${alignr}${color1}${cpubar 5,220 cpu0}${color}
${voffset 2}${cpugraph cpu0 26,364 3fe08a 5ac8fa}
${voffset -2}${color5}RAM${color}     ${color2}${mem}${color5} / ${memmax}${color}${alignr}${color2}${membar
 ↪  5,220}${color}
${voffset 2}${color5}Disque${color}  ${color6}${fs_used /}${color5} / ${fs_size
 ↪  /}${color}${alignr}${color6}${fs_bar 5,220 /}${color}
${voffset 2}${color5}Procs${color}   ${processes} ${color5}· ${color2}${running_processes}${color5}
 ↪  actifs${color}${alignr}${color5}top ${color7}${top name 1} ${top cpu 1}%${color}
${voffset 8}${font Ubuntu:bold:size=10}${color1}Réseau${font}${color}
${voffset 3}${font Ubuntu Mono:size=9}${execpi 10 /opt/GSM/osmo-operator/tools/conky-osmo-status.sh net}${font}
${voffset 6}${color5}${hr 1}${color}
${voffset 3}${execpi 5 /opt/GSM/osmo-operator/tools/conky-osmo-status.sh vue}
${voffset 6}${color5}${hr 1}${color}
${voffset 3}${font Ubuntu:bold:size=10}${color1}Services${font}${color}
${voffset 3}${execpi 10 /opt/GSM/osmo-operator/tools/conky-osmo-status.sh services}
${voffset 6}${color5}${hr 1}${color}
${voffset 3}${alignc}${font Ubuntu:size=8}${color7}sudo -i · ./start-native.sh${color}   ${color5}web
 ↪  ${color1}:8443${color}   ${color5}VTY ${color1}nc 127.0.0.1 4239${color}${font}
]]
```

3.36 /opt/GSM/osmo-operator/configs/conky/osmo-conky.lua

2436 octets, 59 lignes → 59 lignes

```
-- osmo-conky.lua - Cairo panel of the bench's Conky dashboard.
--
-- Loaded by osmo-conky.conf (lua_load / lua_draw_hook_pre = 'panneau'). Conky
-- can only draw a solid rectangle (own_window_colour + argb), so this hook
-- paints, before the text:
--   - a translucent dark panel with rounded corners (the wallpaper,
--     configs/gsm-lab-ville.jpg, shows through);
--   - a one-pixel light outline all around;
--   - an accent line (the bench blue) along the top edge.
-- Conky 1.19 (conky-all, Ubuntu 24.04): the Cairo bindings are in "cairo"
-- and, for the X11 surface, "cairo_xlib" (absent from older versions, so it
-- is loaded without being required).
require 'cairo'
pcall(require, 'cairo_xlib')

-- Panel settings (background, outline, accent): rgba 0..1.
local FOND     = {0.03, 0.06, 0.12, 0.42}
local FILET    = {1.00, 1.00, 1.00, 0.16}
local ACCENT   = {0.35, 0.78, 0.98, 0.95}   -- #5ac8fa, the bench blue
local RAYON    = 16
local MARGE    = 1                          -- the outline must not touch the edge

local function arrondi(cr, x, y, w, h, r)
    cairo_new_path(cr)
    cairo_arc(cr, x + w - r, y + r, r, -math.pi / 2, 0)
    cairo_arc(cr, x + w - r, y + h - r, r, 0, math.pi / 2)
    cairo_arc(cr, x + r, y + h - r, r, math.pi / 2, math.pi)
```

```
    cairo_arc(cr, x + r, y + r, r, math.pi, 3 * math.pi / 2)
    cairo_close_path(cr)
end

function conky_panneau()
    if conky_window == nil then return end
    local w, h = conky_window.width, conky_window.height
    if w < 2 or h < 2 then return end
    local cs = cairo_xlib_surface_create(conky_window.display, conky_window.drawable,
                                         conky_window.visual, w, h)
    local cr = cairo_create(cs)

    -- the panel
    arrondi(cr, MARGE + 0.5, MARGE + 0.5, w - 2 * MARGE - 1, h - 2 * MARGE - 1, RAYON)
    cairo_set_source_rgba(cr, FOND[1], FOND[2], FOND[3], FOND[4])
    cairo_fill_preserve(cr)
    -- the outline
    cairo_set_source_rgba(cr, FILET[1], FILET[2], FILET[3], FILET[4])
    cairo_set_line_width(cr, 1)
    cairo_stroke(cr)

    -- the accent: a line under the top edge, between the two rounded corners
    cairo_new_path(cr)
    cairo_move_to(cr, MARGE + RAYON, MARGE + 2)
    cairo_line_to(cr, w - MARGE - RAYON, MARGE + 2)
    cairo_set_source_rgba(cr, ACCENT[1], ACCENT[2], ACCENT[3], ACCENT[4])
    cairo_set_line_width(cr, 2)
    cairo_stroke(cr)

    cairo_destroy(cr)
    cairo_surface_destroy(cs)
end
```

3.37 /opt/GSM/osmo-operator/configs/crypthome/crypthome-session.desktop

480 octets, 10 lignes → 10 lignes

```
# crypthome-session.desktop - GNOME autostart entry (installed in ~/.config/autostart by
# iso_modules/89-home-chiffre.sh): runs crypthome-session-setup once /home is decrypted.
[Desktop Entry]
Type=Application
Name=Preparation du compte chiffre
Comment=Autorise le compte proprietaire sur l'affichage et demarre sa session utilisateur, quand /home est
 ↪  dechiffre
Exec=/usr/local/bin/crypthome-session-setup
NoDisplay=true
X-GNOME-Autostart-enabled=true
X-Crypthome-Generated=true
```

3.38 /opt/GSM/osmo-operator/configs/crypthome/crypthome.default

4441 octets, 84 lignes → 84 lignes

```
# /etc/default/crypthome - encrypted second disk carrying /home
#
# MODEL. The system disk is NOT encrypted: everything that is system is
# readable - the OS, /etc, and /root, the WORKING account of the bench. What
# is encrypted is the second disk, mounted on /home: profiles, personal data
# and everything that belongs to the keyring.
#
# WHO WRITES THIS FILE. Two writers, for two paths:
#   - crypthome-postinstall, at INSTALLATION time, when Calamares has just
```

```
#     encrypted the /home partition (the normal case);
#   - init-crypthome.sh, afterwards, when encrypting a SECOND disk added to an
#     already installed machine.
# CRYPTHOME_UUID stays empty until one of them has run; unlock-home then does
# nothing and the boot is not affected.

# UUID of the LUKS container (= UUID shown by blkid on the encrypted partition).
CRYPTHOME_UUID=""

# Mapping name in /dev/mapper and mount point.
#
# "crypthome" is the name chosen by init-crypthome.sh. A Calamares installation
# inherits the name forged by kpmcore - "luks-<uuid>" - and crypthome-postinstall
# writes THAT one here. Both are fine: every component reads this setting
# instead of assuming a name.
CRYPTHOME_NAME="crypthome"
CRYPTHOME_MOUNT="/home"

# The account whose home lives on the encrypted disk: it carries the keyring
# and runs the applications holding secrets (see run-as-owner).
#
# On the live key it is osmocom, and the session runs as root. On an installed
# disk, osmocom no longer exists (removed by shellprocess-osmo.conf) and
# crypthome-postinstall writes here the account chosen at installation - which
# is also the session account. run-as-owner then has nothing to switch and
# starts the application as is: the home of that account IS already on the
# encrypted volume.
#
# TWO SESSIONS, DEPENDING ON THE ANSWER AT BOOT (unlock-home):
#   yes -> the account above, PERSISTENT: its home is on the encrypted volume;
#   no  -> ROOT, EPHEMERAL: GDM opens on root for this boot only, and whatever
#          the session writes in /root - and in /opt/GSM, see
#          CRYPTHOME_RAM_DIRS - stays in /tmp/crypthome-root (tmpfs, in RAM):
#          nothing survives a shutdown.
CRYPTHOME_OWNER="__OWNER__"

# Boot: number of passphrase attempts, and delay (s) after which an unanswered
# prompt counts as "no". The boot is NEVER blocked: at worst the machine
# starts without /home.
CRYPTHOME_TRIES=3
CRYPTHOME_TIMEOUT=60

# Where the prompt "decrypt /home -passphrase- or Enter" is asked at boot:
#   console  (default) on the text console, read by unlock-home itself;
#            plymouth, if running, is quit just before. This path works
#            everywhere: it depends neither on a systemd agent nor on the
#            plymouth keyboard (dead on some machines, Intel + NVIDIA laptops
#            in particular).
#   plymouth in the boot theme (osmo-bts), through systemd-ask-password:
#            nicer, for a machine where keyboard input in plymouth has been
#            VERIFIED to work.
CRYPTHOME_PROMPT="console"

# Image painted behind the prompt (console mode, /dev/fb0) by crypthome-fond.
# "none": bare text console.
CRYPTHOME_IMAGE="/usr/share/backgrounds/gsm-lab-antenne.jpg"

# Directories covered by a RAM overlay during a ROOT session (/home closed),
# IN ADDITION to /root, which always is. The whole bench lives in /opt/GSM
# (repository, configs, hlr.db, logs): in a root session it is "live" -
# everything written there evaporates at shutdown, and the disk keeps its
# previous state. Content stays readable and executable; only writes go to
# RAM. Space separated; a missing directory is ignored. Empty = /root only.
CRYPTHOME_RAM_DIRS="/opt/GSM"

# Directories of root that must not exist in clear on the system disk:
```

```
# init-crypthome.sh replaces them with links to the encrypted disk. Disk not
# decrypted = dangling link = nothing is written in clear.
CRYPTHOME_ROOT_LINKS=".local/share/keyrings"

# Applications holding secrets: their launchers are redirected to run-as-owner
# by refresh-owner-apps. Names of .desktop files without the extension; those
# not installed are silently ignored. Run "refresh-owner-apps" again after
# adding some.
CRYPTHOME_OWNER_APPS="firefox firefox-esr thunderbird chromium chromium-browser google-chrome keepassxc seahorse
 ↪  org.gnome.seahorse.Application signal-desktop element-desktop telegramdesktop org.gnome.Geary
 ↪  org.gnome.Evolution evolution nextcloud org.keepassxc.KeePassXC"
```

3.39 /opt/GSM/osmo-operator/configs/crypthome/unlock-home.service

1379 octets, 30 lignes → 30 lignes

```
[Unit]
Description=Dechiffrement optionnel de /home (second disque, LUKS)
Documentation=file:/etc/default/crypthome
ConditionPathExists=/etc/default/crypthome
# unlock-home.service - optional decryption of /home at boot (runs unlock-home --boot).
# AFTER local file systems and udev (the /dev/disk/by-uuid link must exist),
# AFTER plymouth (unlock-home quits it before asking on the console, so it
# must already be up, not starting), and AFTER keyboard setup: the layout of
# /etc/default/keyboard must be the one loaded when the passphrase is typed.
After=local-fs.target systemd-udev-settle.service plymouth-start.service
After=keyboard-setup.service console-setup.service
Wants=systemd-udev-settle.service
# BEFORE anything that opens a session: GDM must not start on an empty /home
# when the operator has just answered "yes".
Before=systemd-user-sessions.service display-manager.service multi-user.target graphical.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/local/sbin/unlock-home --boot
# The prompt and the input go through /dev/console (or, with CRYPTHOME_PROMPT=
# plymouth, through systemd-ask-password), never through stdin.
StandardInput=null
StandardOutput=journal+console
StandardError=journal+console
# Generous but bounded: unlock-home already applies its own timeout per prompt.
TimeoutStartSec=600

[Install]
WantedBy=multi-user.target
```

3.40 /opt/GSM/osmo-operator/configs/gprs/modem-gprs.cfg

902 octets, 35 lignes → 35 lignes

```
! modem-gprs.cfg - GPRS MS of the bench (osmocom-bb modem), installed by scripts/egprs-dsp.sh
! IMSI 001010001000001 · MSISDN 100101 (the mobile's: only one of the two runs on the Calypso) · APN internet ->
 ↪  tun_modem · ARFCN 714
line vty
 no login
 bind 127.0.0.1 4299
log stderr
 logging filter all 1
 logging color 0
 logging timestamp 1
 logging print category 1
```

```
 logging print level 1
 logging level all notice
 logging level cs info
 logging level rr debug
 logging level rlcmac info
 logging level gmm info
 logging level sm info
 logging level llc notice
 logging level sndcp notice
ms 1
 layer2-socket /tmp/modem_l2
 sim test
 imei 358925008659483 0
 imei-fixed
 test-sim
  imsi 001010001000001
  ki comp128 00 11 22 33 44 55 66 77 88 99 aa bb cc dd 01 01
  no barred-access
  rplmn 001 01
 apn internet
  tun-device tun_modem
  tun-netns msgprs
  type-support v4
  no shutdown
 no shutdown
```

3.41 /opt/GSM/osmo-operator/configs/gprs/osmo-bsc-bts2-egprs.frag

2102 octets, 81 lignes → 81 lignes

```
 bts 2
  type osmo-bts
  band DCS1800
  cell_identity 6013
  location_area_code 0x0001
  base_station_id_code 9
  ms max power 15
  cell reselection hysteresis 4
  rxlev access min 0
  radio-link-timeout 32
  channel allocator mode chan-req descending
  channel allocator mode assignment ascending
  channel allocator mode handover ascending
  rach tx integer 9
  rach max transmission 7
  ipa unit-id 6013 0
  oml ipa stream-id 255 line 0
  neighbor-list mode automatic
  codec-support fr
  gprs mode egprs
  gprs routing area 0
  gprs network-control-order nc0
  gprs power-control alpha 0
  gprs cell bvci 13
  gprs cell timer blocking-timer 3
  gprs cell timer blocking-retries 3
  gprs cell timer unblocking-retries 3
  gprs cell timer reset-timer 3
  gprs cell timer reset-retries 3
  gprs cell timer suspend-timer 10
  gprs cell timer suspend-retries 3
  gprs cell timer resume-timer 10
  gprs cell timer resume-retries 3
  gprs cell timer capability-update-timer 10
```

```
  gprs cell timer capability-update-retries 3
  gprs nsei 11
  gprs ns timer tns-block 3
  gprs ns timer tns-block-retries 3
  gprs ns timer tns-reset 3
  gprs ns timer tns-reset-retries 3
  gprs ns timer tns-test 30
  gprs ns timer tns-alive 3
  gprs ns timer tns-alive-retries 10
  gprs nsvc 0 nsvci 11
  gprs nsvc 0 local udp port 23002
  gprs nsvc 0 remote ip __SGSN_IP__
  gprs nsvc 0 remote udp port 23000
  gprs nsvc 1 nsvci 0
  gprs nsvc 1 local udp port 0
  gprs nsvc 1 remote ip 0.0.0.0
  gprs nsvc 1 remote udp port 0
  trx 0
   rf_locked 0
   arfcn 714
   nominal power 23
   max_power_red 0
   rsl e1 tei 0
   timeslot 0
    phys_chan_config CCCH+SDCCH4
    hopping enabled 0
   timeslot 1
    phys_chan_config SDCCH8
    hopping enabled 0
   timeslot 2
    phys_chan_config TCH/F
    hopping enabled 0
   timeslot 3
    phys_chan_config TCH/F
    hopping enabled 0
   timeslot 4
    phys_chan_config TCH/F
    hopping enabled 0
   timeslot 5
    phys_chan_config PDCH
    hopping enabled 0
   timeslot 6
    phys_chan_config PDCH
    hopping enabled 0
   timeslot 7
    phys_chan_config PDCH
    hopping enabled 0
```

3.42 /opt/GSM/osmo-operator/configs/gprs/osmo-bts-virtual-gprs.cfg

1044 octets, 42 lignes → 42 lignes

```
! osmo-bts-virtual-gprs.cfg - virtual BTS 2 (Um multicast on lo), EGPRS cell of the modem.
! Installed in /etc/osmocom by scripts/egprs-dsp.sh. PCU: /tmp/pcu_bts3 (osmo-pcu-gprs.cfg).
! Virtual Um: GSMTAP multicast 239.193.23.1/2 udp 4729 on lo (virtphy -D lo)
log file /var/log/osmocom/osmo-bts-virtual-gprs.log
 logging filter all 1
 logging color 0
 logging timestamp 1
 logging print category 1
 logging print level 1
 logging level all notice
 logging level pcu info
line vty
```

```
 no login
 bind 127.0.0.1 4253
ctrl
 bind 127.0.0.3
e1_input
 e1_line 0 driver ipa
 e1_line 0 port 0
 no e1_line 0 keepalive
phy 0
 virtual-um net-device lo
 virtual-um ttl 1
 instance 0
bts 0
 band DCS1800
 ipa unit-id 6013 0
 oml remote-ip 127.0.0.1
 pcu-socket /tmp/pcu_bts3
 rtp jitter-buffer 100
 paging queue-size 200
 paging lifetime 0
 min-qual-rach 50
 min-qual-norm -5
 gsmtap-sapi ccch
 gsmtap-sapi pdtch
 trx 0
  power-ramp max-initial 23000 mdBm
  power-ramp step-size 2000 mdB
  power-ramp step-interval 1
  ms-power-control osmo
  phy 0 instance 0
```

3.43 /opt/GSM/osmo-operator/configs/gprs/osmo-pcu-gprs.cfg

610 octets, 25 lignes → 25 lignes

```
! osmo-pcu-gprs.cfg - PCU of virtual BTS 2 (socket /tmp/pcu_bts3, VTY 127.0.0.2:4240).
! Installed in /etc/osmocom by scripts/egprs-dsp.sh; __GSMTAP_IP__ = the one of osmo-pcu.cfg.
log file /var/log/osmocom/osmo-pcu-gprs.log
 logging color 1
 logging timestamp 1
 logging filter all 1
 logging print category 1
 logging print level 1
log gsmtap __GSMTAP_IP__
 logging filter all 1
 logging color 0
 logging timestamp 0
 logging print category 1
 logging print level 1
!
pcu
 flow-control-interval 10
 cs 2
 alloc-algorithm dynamic
 alpha 0
 gamma 0
 pcu-socket /tmp/pcu_bts3
line vty
 no login
 bind 127.0.0.2
```

3.44 /opt/GSM/osmo-operator/configs/open5gs/hss.yaml

626 octets, 20 lignes → 20 lignes

```
# hss.yaml - Open5GS HSS of the bench (/opt/LTE/open5gs/install); Diameter S6a on 127.0.0.8.
db_uri: mongodb://localhost/open5gs
logger:
  file:
    path: /opt/LTE/open5gs/install/var/log/open5gs/hss.log
#  level: info   # fatal|error|warn|info(default)|debug|trace

global:
  max:
    ue: 1024  # The number of UE can be increased depending on memory size.
#    peer: 64

hss:
  freeDiameter: /opt/LTE/open5gs/install/etc/freeDiameter/hss.conf
  metrics:
    server:
      - address: 127.0.0.8
        port: 9090
#  sms_over_ims: "sip:smsc.mnc001.mcc001.3gppnetwork.org:7060;transport=tcp"
#  use_mongodb_change_stream: true
```

3.45 /opt/GSM/osmo-operator/configs/open5gs/mme.yaml

3477 octets, 103 lignes → 103 lignes

```
# mme.yaml - Open5GS MME of the bench (Open5GS v2.8.0, /opt/LTE/open5gs/install).
#
# Only what runs on this bench is kept (no roaming, DNS, SGSN or IPv6
# examples). The source is configs/open5gs/mme.yaml in the repository; the
# copy in service is /opt/LTE/open5gs/install/etc/open5gs/mme.yaml (cp, then
# restart open5gs-mmed).
#
# ADDRESS PLAN: the whole 4G core lives on the loopback, one address per
# function, so that ports 2123/2152/8805 do not collide:
#   127.0.0.2  MME (S1AP sctp 36412, GTP-C 2123, Diameter S6a, SGs to the MSC)
#   127.0.0.3  SGW-C     127.0.0.4  SMF (= PGW-C)     127.0.0.6  SGW-U
#   127.0.0.7  UPF       127.0.0.8  HSS               127.0.0.9  PCRF
#   127.0.0.1  osmo-msc (SGs 29118), osmo-stp, osmo-sgsn - the osmocom 2G.
# srsENB (ZeroMQ, 127.0.1.1) connects to the S1AP of 127.0.0.2 (enb.conf: mme_addr).
#
# Known limitation: the PGW is NOT chosen by gtpc.client.smf alone. If the HSS
# subscription (MongoDB, open5gs.subscribers, slice[].session[].smf) carries an
# address, it wins (the WebUI puts 127.0.0.200 there by default) and the SGW-C
# waits for a Create Session Response from nobody (GTP timeout, RRC release,
# "Attach failed" on the srsUE side). Leave that field empty in the database.

logger:
  file:
    path: /opt/LTE/open5gs/install/var/log/open5gs/mme.log

global:
  max:
    ue: 1024          # plenty: one srsUE and the bench phone

mme:
  freeDiameter: /opt/LTE/open5gs/install/etc/freeDiameter/mme.conf   # S6a to the HSS (127.0.0.8)

  # S1AP: what the MME listens on for the eNBs. srsENB connects to it over SCTP.
  s1ap:
```

```
  server:
    - address: 127.0.0.2

# GTP-C (S11 to the SGW-C, S5 to the SMF when the HSS says nothing).
gtpc:
  server:
    - address: 127.0.0.2
  client:
    sgwc:
      - address: 127.0.0.3
    smf:
      - address: 127.0.0.4

metrics:
  server:
    - address: 127.0.0.2
      port: 9090

# SGs: THE LINK WITH THE 2G. Through it the UE camped on 4G is also
# registered in the osmo-msc VLR (combined EPS + IMSI attach), and CSFB
# works: on a call or SMS the UE falls back to the BTS; meanwhile the MSC
# knows where to reach it (paging by the MME). osmo-msc listens over SCTP
# on 127.0.0.1:29118 (osmo-msc.cfg, "sgs" section). The "map" table
# translates the 4G area (TAI: PLMN 001-01, TAC 7 - the one of srsENB's
# rr.conf) into the 2G area (LAI: PLMN 001-01, LAC 1 - location_area_code
# 0x0001 in osmo-bsc.cfg): the VLR files the subscriber in LAC 1 while it is
# on LTE.
sgsap:
  client:
    - address: 127.0.0.1
      local_address: 127.0.0.2
      map:
        tai:
          plmn_id:
            mcc: 001
            mnc: 01
          tac: 7
        lai:
          plmn_id:
            mcc: 001
            mnc: 01
          lac: 1

# MME identity (GUMMEI): carried by the GUTI given to the UE.
gummei:
  - plmn_id:
      mcc: 001
      mnc: 01
    mme_gid: 2
    mme_code: 1

# Served areas: a single one, the one srsENB broadcasts.
tai:
  - plmn_id:
      mcc: 001
      mnc: 01
    tac: 7

# NAS security: mandatory integrity (EIA2 first), null ciphering (EEA0)
# first so that the NAS can be READ in Wireshark on this bench.
security:
  integrity_order : [ EIA2, EIA1, EIA0 ]
  ciphering_order : [ EEA0, EEA1, EEA2 ]

# The name the phone displays for 4G (EMM Information).
network_name:
```

```
    full: OsmoDirect
    short: OsmoDirect
  mme_name: open5gs-mme0
```

3.46 /opt/GSM/osmo-operator/configs/open5gs/nrf.yaml

3531 octets, 106 lignes → 106 lignes

```
# nrf.yaml - Open5GS NRF (5G core SBI registry, 127.0.0.10) for the lab core.
# Installed by tools/osmo-lte-install.sh --configs into $PREFIX/etc/open5gs;
# the /opt/LTE/open5gs/install prefix is substituted at install time (OPEN5GS_PREFIX).
# Upstream Open5GS template: the commented blocks below are the upstream examples.

logger:
  file:
    path: /opt/LTE/open5gs/install/var/log/open5gs/nrf.log
#  level: info   # fatal|error|warn|info(default)|debug|trace

global:
  max:
    ue: 1024  # The number of UE can be increased depending on memory size.
#    peer: 64

nrf:
  serving:  # 5G roaming requires PLMN in NRF
    - plmn_id:
        mcc: 999
        mnc: 70
  sbi:
    server:
      - address: 127.0.0.10
        port: 7777

###########################################################################
# SBI Server
###########################################################################
#  o Bind to the address on the eth0 and advertise as open5gs-nrf.svc.local
#  sbi:
#    server:
#      - dev:eth0
#        advertise: open5gs-nrf.svc.local
#
#  o Specify a custom port number 7777 while binding to the given address
#  sbi:
#    server:
#      - address: nrf.localdomain
#        port: 7777
#
#  o Bind to 127.0.0.10 and advertise as open5gs-nrf.svc.local
#  sbi:
#    server:
#      - address: 127.0.0.10
#        port: 7777
#        advertise: open5gs-nrf.svc.local
#
#  o Bind to port 7777 but advertise with a different port number 8888
#  sbi:
#    server:
#      - address: 127.0.0.10
#        port: 7777
#        advertise: open5gs-nrf.svc.local:8888
#
###########################################################################
# HTTPS scheme with TLS
```

```
################################################################################
#  o Set as default if not individually set
#  default:
#    tls:
#      server:
#        scheme: https
#        private_key: /opt/LTE/open5gs/install/etc/open5gs/tls/nrf.key
#        cert: /opt/LTE/open5gs/install/etc/open5gs/tls/nrf.crt
#      client:
#        scheme: https
#        cacert: /opt/LTE/open5gs/install/etc/open5gs/tls/ca.crt
#  sbi:
#    server:
#      - address: nrf.localdomain
#
#  o Enable SSL key logging for Wireshark
#    - This configuration allows capturing SSL/TLS session keys
#      for debugging or analysis purposes using Wireshark.
#  default:
#    tls:
#      server:
#        scheme: https
#        private_key: /opt/LTE/open5gs/install/etc/open5gs/tls/nrf.key
#        cert: /opt/LTE/open5gs/install/etc/open5gs/tls/nrf.crt
#        sslkeylogfile: /opt/LTE/open5gs/install/var/log/open5gs/tls/nrf-server-sslkeylog.log
#      client:
#        scheme: https
#        cacert: /opt/LTE/open5gs/install/etc/open5gs/tls/ca.crt
#        client_sslkeylogfile: /opt/LTE/open5gs/install/var/log/open5gs/tls/nrf-client-sslkeylog.log
#  sbi:
#    server:
#      - address: nrf.localdomain
#
#  o Add client TLS verification
#  default:
#    tls:
#      server:
#        scheme: https
#        private_key: /opt/LTE/open5gs/install/etc/open5gs/tls/nrf.key
#        cert: /opt/LTE/open5gs/install/etc/open5gs/tls/nrf.crt
#        verify_client: true
#        verify_client_cacert: /opt/LTE/open5gs/install/etc/open5gs/tls/ca.crt
#      client:
#        scheme: https
#        cacert: /opt/LTE/open5gs/install/etc/open5gs/tls/ca.crt
#        client_private_key: /opt/LTE/open5gs/install/etc/open5gs/tls/nrf.key
#        client_cert: /opt/LTE/open5gs/install/etc/open5gs/tls/nrf.crt
#  sbi:
#    server:
#      - address: nrf.localdomain
```

3.47 /opt/GSM/osmo-operator/configs/open5gs/pcrf.yaml

5188 octets, 139 lignes → 139 lignes

```
# pcrf.yaml - Open5GS PCRF (Gx policy, 127.0.0.9) for the lab EPC.
# Installed by tools/osmo-lte-install.sh --configs into $PREFIX/etc/open5gs;
# the /opt/LTE/open5gs/install prefix is substituted at install time (OPEN5GS_PREFIX).
# Upstream Open5GS template: the commented blocks below are the upstream examples.

db_uri: mongodb://localhost/open5gs
logger:
  file:
    path: /opt/LTE/open5gs/install/var/log/open5gs/pcrf.log
```

```
#  level: info   # fatal|error|warn|info(default)|debug|trace

global:
  max:
    ue: 1024  # The number of UE can be increased depending on memory size.
#    peer: 64
pcrf:
  freeDiameter: /opt/LTE/open5gs/install/etc/freeDiameter/pcrf.conf
  metrics:
    server:
      - address: 127.0.0.9
        port: 9090

################################################################################
# PCRF Policy Configuration: SUPI Range Based Policies
################################################################################
#
# This configuration applies policies based solely on the UE's SUPI range.
#
# supi_range: Specifies one or more ranges of SUPIs.
# session:   Defines the session configuration for each DNN.
#
# Example:
#
#  policy:
#    - supi_range:          # Filter policies by SUPI
#        - 999700000000001-999709999999999
#        - 315010000000001-315010999999999
#      session:              # Define session based on DNN
#        - name: internet
#          type: 3         # 1: IPv4, 2: IPv6, 3: IPv4v6
#          ambr:
#            downlink:
#              value: 1
#              unit: 3     # 0: bps, 1: Kbps, 2: Mbps, 3: Gbps, 4: Tbps
#            uplink:
#              value: 1
#              unit: 3
#          qos:
#            index: 9        # Allowed values: 1,2,3,4,65,66,67,75,71,72,
#                             # 73,74,76,5,6,7,8,9,69,70,79,80,82,83,
#                             # 84,85,86
#            arp:
#              priority_level: 8   # Allowed values: 1 to 15
#              pre_emption_vulnerability: 1  # 1: Disabled, 2: Enabled
#              pre_emption_capability: 1     # 1: Disabled, 2: Enabled
#        - name: ims
#          type: 3         # 1: IPv4, 2: IPv6, 3: IPv4v6
#          ambr:
#            downlink:
#              value: 1
#              unit: 3
#            uplink:
#              value: 1
#              unit: 3
#          qos:
#            index: 5        # Allowed values as above
#            arp:
#              priority_level: 1   # Allowed values: 1 to 15
#              pre_emption_vulnerability: 1  # 1: Disabled, 2: Enabled
#              pre_emption_capability: 1     # 1: Disabled, 2: Enabled
#          pcc_rule:
#            - qos:
#                index: 1    # Allowed values as above
#                arp:
#                  priority_level: 1
```

```
#                 pre_emption_vulnerability: 1
#                 pre_emption_capability: 1
#               mbr:
#                 downlink:
#                   value: 82
#                   unit: 1   # 0: bps, 1: Kbps, 2: Mbps, 3: Gbps, 4: Tbps
#                 uplink:
#                   value: 82
#                   unit: 1
#               gbr:
#                 downlink:
#                   value: 82
#                   unit: 1
#                 uplink:
#                   value: 82
#                   unit: 1
#             flow:
#               - direction: 2
#                 description: "permit out icmp from any to assigned"
#               - direction: 1
#                 description: "permit out icmp from any to assigned"
#               - direction: 2
#                 description: "permit out udp from 10.200.136.98/32 23455 to assigned 1-65535"
#               - direction: 1
#                 description: "permit out udp from 10.200.136.98/32 1-65535 to assigned 50021"
#           - qos:
#               index: 2    # Allowed values as above
#               arp:
#                 priority_level: 4   # Allowed values: 1 to 15
#                 pre_emption_vulnerability: 2  # 1: Disabled, 2: Enabled
#                 pre_emption_capability: 2     # 1: Disabled, 2: Enabled
#               mbr:
#                 downlink:
#                   value: 802
#                   unit: 1
#                 uplink:
#                   value: 802
#                   unit: 1
#               gbr:
#                 downlink:
#                   value: 802
#                   unit: 1
#                 uplink:
#                   value: 802
#                   unit: 1
#
#   - supi_range:         # Filter policies by SUPI only
#       - 001010000000001-001019999999999
#     session:              # Define session based on DNN
#       - name: internet
#         type: 3         # 1: IPv4, 2: IPv6, 3: IPv4v6
#         ambr:
#           downlink:
#             value: 1
#             unit: 3
#           uplink:
#             value: 1
#             unit: 3
#         qos:
#           index: 9        # Allowed values as above
#           arp:
#             priority_level: 8   # Allowed values: 1 to 15
#             pre_emption_vulnerability: 1  # 1: Disabled, 2: Enabled
#             pre_emption_capability: 1     # 1: Disabled, 2: Enabled
```

3.48 /opt/GSM/osmo-operator/configs/open5gs/sgwc.yaml

2252 octets, 75 lignes → 75 lignes

```
# sgwc.yaml - Open5GS SGW-C (127.0.0.3) for the lab EPC.
# Installed by tools/osmo-lte-install.sh --configs into $PREFIX/etc/open5gs;
# the /opt/LTE/open5gs/install prefix is substituted at install time (OPEN5GS_PREFIX).
# Upstream Open5GS template: the commented blocks below are the upstream examples.

logger:
  file:
    path: /opt/LTE/open5gs/install/var/log/open5gs/sgwc.log
#  level: info   # fatal|error|warn|info(default)|debug|trace

global:
  max:
    ue: 1024  # The number of UE can be increased depending on memory size.
#    peer: 64

sgwc:
  gtpc:
    server:
      - address: 127.0.0.3
  pfcp:
    server:
      - address: 127.0.0.3
    client:
      sgwu:
        - address: 127.0.0.6

#########################################################################################
# GTP-C Server
#########################################################################################
#  o Listen on IPv4 and IPv6
#  gtpc:
#    server:
#      - address: 127.0.0.3
#      - address: fd69:f21d:873c:fa::2
#
#########################################################################################
# PFCP Server
#########################################################################################
#  o Override PFCP address to be advertised to SGW-U in PFCP association
#  pfcp:
#    server:
#      - dev: eth0
#        advertise: open5gs-sgwc.svc.local
#
#########################################################################################
# PFCP Client
#########################################################################################
#  o SGWU selection by eNodeB TAC
#   (either single TAC or multiple TACs, DECIMAL representation)
#  pfcp:
#    client:
#      sgwu:
#        - address: 127.0.0.6
#          tac: 1
#        - address: 127.0.0.12
#          tac: [3,5,8]
#
#  o SGWU selection by UE's APN (either single APN or multiple APNs)
#  pfcp:
#    client:
#      sgwu:
#        - address: 127.0.0.6
```

```
#          apn: ims
#        - address: 127.0.0.12
#          apn: [internet, web]
#
#  o SGWU selection by CellID(e_cell_id: 28bit)
#   (either single e_cell_id or multiple e_cell_id, HEX representation)
#  pfcp:
#    client:
#      sgwu:
#        - address: 127.0.0.6
#          e_cell_id: 463
#        - address: 127.0.0.12
#          e_cell_id: [123456789, 9413]
```

3.49 /opt/GSM/osmo-operator/configs/open5gs/sgwu.yaml

1810 octets, 57 lignes → 57 lignes

```
# sgwu.yaml - Open5GS SGW-U (127.0.0.6) for the lab EPC.
# Installed by tools/osmo-lte-install.sh --configs into $PREFIX/etc/open5gs;
# the /opt/LTE/open5gs/install prefix is substituted at install time (OPEN5GS_PREFIX).
# Upstream Open5GS template: the commented blocks below are the upstream examples.

logger:
  file:
    path: /opt/LTE/open5gs/install/var/log/open5gs/sgwu.log
#  level: info   # fatal|error|warn|info(default)|debug|trace

global:
  max:
    ue: 1024  # The number of UE can be increased depending on memory size.
#    peer: 64

sgwu:
  pfcp:
    server:
      - address: 127.0.0.6
    client:
#      sgwc:    # SGW-U PFCP Client try to associate SGW-C PFCP Server
#        - address: 127.0.0.3
  gtpu:
    server:
      - address: 127.0.0.6

################################################################################
# PFCP Server
################################################################################
#  o Override PFCP address to be advertised to SGW-C in PFCP association
#  pfcp:
#    server:
#      - dev: eth0
#        advertise: open5gs-sgwu.svc.local
#
################################################################################
# GTP-U Server
################################################################################
#  o Override SGW-U GTP-U address to be advertised inside S1AP messages
#  gtpu:
#    server:
#      - dev: ens3
#        advertise: sgw1.epc.mnc001.mcc001.3gppnetwork.org
#
#  o User Plane IP Resource information
#  gtpu:
```

```
#    server:
#      - address: 127.0.0.6
#        teid_range_indication: 4
#        teid_range: 10
#        network_instance: internet
#        source_interface: 0
#      - address: 127.0.10.4
#        teid_range_indication: 4
#        teid_range: 5
#        network_instance: ims
#        source_interface: 1
```

3.50 /opt/GSM/osmo-operator/configs/open5gs/smf.yaml

11974 octets, 428 lignes → 427 lignes (1 groupes compactés)

```
# smf.yaml - Open5GS SMF / PGW-C (127.0.0.4) for the lab EPC.
# Installed by tools/osmo-lte-install.sh --configs into $PREFIX/etc/open5gs;
# the /opt/LTE/open5gs/install prefix is substituted at install time (OPEN5GS_PREFIX).
# Upstream Open5GS template: the commented blocks below are the upstream examples.

logger:
  file:
    path: /opt/LTE/open5gs/install/var/log/open5gs/smf.log
#  level: info   # fatal|error|warn|info(default)|debug|trace

global:
  max:
    ue: 1024  # The number of UE can be increased depending on memory size.
#    peer: 64

smf:
  sbi:
    server:
      - address: 127.0.0.4
        port: 7777
    client:
      nrf:
        - uri: http://127.0.0.10:7777
#      scp:
#        - uri: http://127.0.0.200:7777
  pfcp:
    server:
      - address: 127.0.0.4
    client:
      upf:
        - address: 127.0.0.7
  gtpc:
    server:
      - address: 127.0.0.4
  gtpu:
    server:
      - address: 127.0.0.4
  metrics:
    server:
      - address: 127.0.0.4
        port: 9090
  session:
    - subnet: 10.45.0.0/16
      gateway: 10.45.0.1
    - subnet: 2001:db8:cafe::/48
      gateway: 2001:db8:cafe::1
  dns:
    - 8.8.8.8
```

```
    - 8.8.4.4
    - 2001:4860:4860::8888
    - 2001:4860:4860::8844
  mtu: 1400
#  p-cscf:
#    - 127.0.0.1
#    - ::1
#  ctf:
#    enabled: auto   # auto(default)|yes|no
  freeDiameter: /opt/LTE/open5gs/install/etc/freeDiameter/smf.conf

################################################################################
# SMF Info
################################################################################
#  <SMF Selection - 5G Core only>
#  1. SMF sends SmfInfo(S-NSSAI, DNN, TAI) to the NRF
#  2. NRF responds to AMF with SmfInfo during NF-Discovery.
#  3. AMF selects SMF based on S-NSSAI, DNN and TAI in SmfInfo.
#
#  Note that if there is no SmfInfo, any AMF can select this SMF.
#
#  o S-NSSAI[SST:1] and DNN[internet] - At least 1 DNN is required in S-NSSAI
#  info:
#    - s_nssai:
#        - sst: 1
#          dnn:
#            - internet
#
#  o S-NSSAI[SST:1 SD:009000] and DNN[internet or ims]
#  info:
#    - s_nssai:
#        - sst: 1
#          sd: 009000
#          dnn:
#            - internet
#            - ims
#
#  o S-NSSAI[SST:1] and DNN[internet] and TAI[PLMN-ID:99970 TAC:1]
#  info:
#    - s_nssai:
#        - sst: 1
#          dnn:
#            - internet
#      tai:
#        - plmn_id:
#            mcc: 999
#            mnc: 70
#          tac: 1
#
#  o If any of conditions below are met:
#   - S-NSSAI[SST:1] and DNN[internet] and TAI[PLMN-ID:99970 TAC:1-9]
#   - S-NSSAI[SST:2 SD:000080] and DNN[internet or ims]
#   - S-NSSAI[SST:4] and DNN[internet] and TAI[PLMN-ID:99970 TAC:10-20,30-40]
#  info:
#    - s_nssai:
#        - sst: 1
#          dnn:
#            - internet
#      tai:
#        - plmn_id:
#            mcc: 999
#            mnc: 70
#          tac:
#            - 1-9
#    - s_nssai:
#        - sst: 2
```

```
#          sd: 000080
#          dnn:
#            - internet
#            - ims
#    - s_nssai:
#        - sst: 4
#          dnn:
#            - internet
#      tai:
#        - plmn_id:
#            mcc: 999
#            mnc: 70
#          tac:
#            - 10-20
#            - 30-40
#
#  o Complex Example
#  info:
#    - s_nssai:
#        - sst: 1
#          dnn:
#            - internet
#        - sst: 1
#          sd: 000080
#          dnn:
#            - internet
#            - ims
#        - sst: 1
#          sd: 009000
#          dnn:
#            [internet, ims]
#        - sst: 2
#          dnn:
#            - internet
#        - sst: 3
#          sd: 123456
#          dnn:
#            - internet
#      tai:
#        - plmn_id:
#            mcc: 999
#            mnc: 70
#          tac: [1, 2, 3]
#        - plmn_id:
#            mcc: 999
#            mnc: 70
#          tac: 4
#        - plmn_id:
#            mcc: 999
#            mnc: 70
#          tac:
#            - 5
#            - 6
#        - plmn_id:
#            mcc: 999
#            mnc: 70
#          tac:
#            - 100-200
#            - 300-400
#        - plmn_id:
#            mcc: 999
#            mnc: 70
#          tac:
#            - ⟨1⟩-⟨2⟩  ×3
    ⟨⟩ = (500,600) (700,800) (900,1000)
#    - s_nssai:
```

```
#        - sst: 4
#          dnn:
#            - internet
#      tai:
#        - plmn_id:
#            mcc: 999
#            mnc: 70
#          tac: 99
#
################################################################################
# SBI Server
################################################################################
#  o Bind to the address on the eth0 and advertise as open5gs-smf.svc.local
#  sbi:
#    server:
#      - dev:eth0
#        advertise: open5gs-smf.svc.local
#
#  o Specify a custom port number 7777 while binding to the given address
#  sbi:
#    server:
#      - address: smf.localdomain
#        port: 7777
#
#  o Bind to 127.0.0.4 and advertise as open5gs-smf.svc.local
#  sbi:
#    server:
#      - address: 127.0.0.4
#        port: 7777
#        advertise: open5gs-smf.svc.local
#
#  o Bind to port 7777 but advertise with a different port number 8888
#  sbi:
#    server:
#      - address: 127.0.0.4
#        port: 7777
#        advertise: open5gs-smf.svc.local:8888
#
################################################################################
# SBI Client
################################################################################
#  o Direct Communication with NRF
#  sbi:
#    client:
#      nrf:
#        - uri: http://127.0.0.10:7777
#
#  o Indirect Communication by Delegating to SCP
#  sbi:
#    client:
#      scp:
#        - uri: http://127.0.0.200:7777
#
#  o Indirect Communication without Delegation
#  sbi:
#    client:
#      nrf:
#        - uri: http://127.0.0.10:7777
#      scp:
#        - uri: http://127.0.0.200:7777
#      delegated:
#        nrf:
#          nfm: no    # Directly communicate NRF management functions
#          disc: no   # Directly communicate NRF discovery
#        scp:
#          next: no   # Do not delegate to SCP for next-hop
```

```
#
#  o Indirect Communication with Delegated Discovery
#  sbi:
#    client:
#      nrf:
#        - uri: http://127.0.0.10:7777
#      scp:
#        - uri: http://127.0.0.200:7777
#      delegated:
#        nrf:
#          nfm: no    # Directly communicate NRF management functions
#          disc: yes  # Delegate discovery to SCP
#        scp:
#          next: yes  # Delegate to SCP for next-hop communications
#
#  o Default delegation: all communications are delegated to the SCP
#  sbi:
#    client:
#      nrf:
#        - uri: http://127.0.0.10:7777
#      scp:
#        - uri: http://127.0.0.200:7777
#      # No 'delegated' section; defaults to AUTO delegation
#
################################################################################
# HTTPS scheme with TLS
################################################################################
#  o Set as default if not individually set
#  default:
#    tls:
#      server:
#        scheme: https
#        private_key: /opt/LTE/open5gs/install/etc/open5gs/tls/smf.key
#        cert: /opt/LTE/open5gs/install/etc/open5gs/tls/smf.crt
#      client:
#        scheme: https
#        cacert: /opt/LTE/open5gs/install/etc/open5gs/tls/ca.crt
#  sbi:
#    server:
#      - address: smf.localdomain
#    client:
#      nrf:
#        - uri: https://nrf.localdomain
#
#  o Enable SSL key logging for Wireshark
#    - This configuration allows capturing SSL/TLS session keys
#      for debugging or analysis purposes using Wireshark.
#  default:
#    tls:
#      server:
#        scheme: https
#        private_key: /opt/LTE/open5gs/install/etc/open5gs/tls/smf.key
#        cert: /opt/LTE/open5gs/install/etc/open5gs/tls/smf.crt
#        sslkeylogfile: /opt/LTE/open5gs/install/var/log/open5gs/tls/smf-server-sslkeylog.log
#      client:
#        scheme: https
#        cacert: /opt/LTE/open5gs/install/etc/open5gs/tls/ca.crt
#        client_sslkeylogfile: /opt/LTE/open5gs/install/var/log/open5gs/tls/smf-client-sslkeylog.log
#  sbi:
#    server:
#      - address: smf.localdomain
#    client:
#      nrf:
#        - uri: https://nrf.localdomain
#
#  o Add client TLS verification
```

```
#  default:
#    tls:
#      server:
#        scheme: https
#        private_key: /opt/LTE/open5gs/install/etc/open5gs/tls/smf.key
#        cert: /opt/LTE/open5gs/install/etc/open5gs/tls/smf.crt
#        verify_client: true
#        verify_client_cacert: /opt/LTE/open5gs/install/etc/open5gs/tls/ca.crt
#      client:
#        scheme: https
#        cacert: /opt/LTE/open5gs/install/etc/open5gs/tls/ca.crt
#        client_private_key: /opt/LTE/open5gs/install/etc/open5gs/tls/smf.key
#        client_cert: /opt/LTE/open5gs/install/etc/open5gs/tls/smf.crt
#  sbi:
#    server:
#      - address: smf.localdomain
#    client:
#      nrf:
#        - uri: https://nrf.localdomain
#
################################################################################
# PFCP Server
################################################################################
#  o Override PFCP address to be advertised to UPF in PFCP association
#  pfcp:
#    server:
#      - dev: eth0
#        advertise: open5gs-smf.svc.local
#
################################################################################
# PFCP Client
################################################################################
#  o UPF selection by eNodeB TAC
#   (either single TAC or multiple TACs, DECIMAL representation)
#  pfcp:
#    client:
#      upf:
#        - address: 127.0.0.7
#          tac: 1
#        - address: 127.0.0.12
#          tac: [3,5,8]
#
#  o UPF selection by UE's DNN/APN (either single DNN/APN or multiple DNNs/APNs)
#  pfcp:
#    client:
#      upf:
#        - address: 127.0.0.7
#          dnn: ims
#        - address: 127.0.0.12
#          dnn: [internet, web]
#
#  o UPF selection by CellID(e_cell_id: 28bit, nr_cell_id: 36bit)
#    (either single enb_id or multiple enb_ids, HEX representation)
#  pfcp:
#    client:
#      upf:
#        - address: 127.0.0.7
#          e_cell_id: 463
#        - address: 127.0.0.12
#          nr_cell_id: [123456789, 9413]
#
################################################################################
# GTP-C Server
################################################################################
#  o Listen on IPv4 and IPv6
#  gtpc:
```

```
#    server:
#      - address: 127.0.0.4
#      - address: fd69:f21d:873c:fa::3
#
###########################################################################
# GTP-U Server
###########################################################################
#  o Listen on IPv4 and IPv6
#  gtpu:
#    server:
#      - address: 127.0.0.4
#      - address: ::1
#
###########################################################################
# 3GPP Specification
###########################################################################
#  o Specific DNN/APN(e.g 'ims') uses 10.46.0.1/16, 2001:db8:babe::1/48
#   (If the UE has unknown DNN/APN(not internet/ims), SMF/UPF will crash.)
#  session:
#    - subnet: 10.45.0.0/16
#      gateway: 10.45.0.1
#      dnn: internet
#    - subnet: 2001:db8:cafe::/48
#      dnn: internet
#    - subnet: 10.46.0.0/16
#      gateway: 10.46.0.1
#      dnn: ims
#    - subnet: 2001:db8:babe::/48
#      dnn: ims
#
#  o Pool Range
#  session:
#    - subnet: 10.45.0.0/16
#      gateway: 10.45.0.1
#      range:
#        - 10.45.0.100-10.45.0.200
#        - 10.45.1.100-
#        - -10.45.0.200
#    - subnet: 2001:db8:cafe::/48
#      range:
#        - 2001:db8:cafe:a0::0-2001:db8:cafe:b0::0
#        - 2001:db8:cafe:c0::0-2001:db8:cafe:d0::0
#
#  o Security Indication(5G Core only)
#  security_indication:
#    integrity_protection_indication: required|preferred|not-needed
#    confidentiality_protection_indication: required|preferred|not-needed
#    maximum_integrity_protected_data_rate_uplink: bitrate64kbs|maximum-UE-rate
#    maximum_integrity_protected_data_rate_downlink: bitrate64kbs|maximum-UE-rate
```

3.51 /opt/GSM/osmo-operator/configs/open5gs/subscribers.json

1423 octets, 79 lignes → 79 lignes

```
[
 {
  "ambr": {
   "downlink": {
    "value": 1,
    "unit": 3
   },
   "uplink": {
    "value": 1,
    "unit": 3
```

```
 }
},
"schema_version": 1,
"msisdn": [],
"imeisv": "3589250042736753",
"mme_host": "mme.localdomain",
"mme_realm": "localdomain",
"purge_flag": false,
"access_restriction_data": 32,
"subscriber_status": 0,
"operator_determined_barring": 0,
"network_access_mode": 0,
"subscribed_rau_tau_timer": 12,
"imsi": "001010001000001",
"security": {
 "k": "00112233445566778899aabbccdd0101",
 "amf": "8000",
 "op": null,
 "opc": "63BFA50EE6523365FF14C1F45F88737D",
 "sqn": {
  "high": 0,
  "low": 1152,
  "unsigned": false
 }
},
"slice": [
 {
  "_id": "6a9f17d9436b8b2e35f39745",
  "sst": 1,
  "default_indicator": true,
  "session": [
   {
    "qos": {
     "arp": {
      "priority_level": 8,
      "pre_emption_capability": 1,
      "pre_emption_vulnerability": 1
     },
     "index": 9
    },
    "ambr": {
     "downlink": {
      "value": 1,
      "unit": 3
     },
     "uplink": {
      "value": 1,
      "unit": 3
     }
    },
    "_id": "6a9f17d9436b8b2e35f39746",
    "name": "srsapn",
    "type": 1,
    "pcc_rule": [],
    "ue": {
     "ipv4": "10.45.0.2"
    }
   }
  ]
 }
],
"__v": 0,
"mme_timestamp": {
 "high": 416507,
 "low": -2117099773,
 "unsigned": false
```

```
  }
 }
]
```

3.52 /opt/GSM/osmo-operator/configs/open5gs/upf.yaml

2845 octets, 93 lignes → 93 lignes

```
# upf.yaml - Open5GS UPF / PGW-U (127.0.0.7, UE pool 10.45.0.0/16 on ogstun) for the lab EPC.
# Installed by tools/osmo-lte-install.sh --configs into $PREFIX/etc/open5gs;
# the /opt/LTE/open5gs/install prefix is substituted at install time (OPEN5GS_PREFIX).
# Upstream Open5GS template: the commented blocks below are the upstream examples.

logger:
  file:
    path: /opt/LTE/open5gs/install/var/log/open5gs/upf.log
#  level: info   # fatal|error|warn|info(default)|debug|trace

global:
  max:
    ue: 1024  # The number of UE can be increased depending on memory size.
#    peer: 64

upf:
  pfcp:
    server:
      - address: 127.0.0.7
    client:
#      smf:     #  UPF PFCP Client try to associate SMF PFCP Server
#        - address: 127.0.0.4
  gtpu:
    server:
      - address: 127.0.0.7
  session:
    - subnet: 10.45.0.0/16
      gateway: 10.45.0.1
    - subnet: 2001:db8:cafe::/48
      gateway: 2001:db8:cafe::1
  metrics:
    server:
      - address: 127.0.0.7
        port: 9090

################################################################################
# PFCP Server
################################################################################
# o Override PFCP address to be advertised to SMF in PFCP association
#  pfcp:
#    server:
#      - dev: eth0
#        advertise: open5gs-upf.svc.local
#
################################################################################
# GTP-U Server
################################################################################
#  o Override SGW-U GTP-U address to be advertised inside S1AP messages
#  gtpu:
#    server:
#      - dev: ens3
#        advertise: upf1.5gc.mnc001.mcc001.3gppnetwork.org
#
#  o User Plane IP Resource information
#  gtpu:
#    server:
```

```
#      - address:
#        - 127.0.0.7
#        - ::1
#        teid_range_indication: 4
#        teid_range: 10
#        network_instance: internet
#        source_interface: 0
#      - address: 127.0.10.4
#        teid_range_indication: 4
#        teid_range: 5
#        network_instance: ims
#        source_interface: 1
#
################################################################################
# 3GPP Specification
################################################################################
#
#  o Specific DNN/APN(e.g 'ims') uses 10.46.0.1/16, 2001:db8:babe::1/48
#  $ sudo ip addr add 10.45.0.1/16 dev ogstun
#  $ sudo ip addr add 2001:db8:cafe::1/48 dev ogstun2
#  $ sudo ip addr add 10.46.0.1/16 dev ogstun3
#  $ sudo ip addr add 2001:db8:babe::1/48 dev ogstun3
#
#  session:
#    - subnet: 10.45.0.0/16
#      gateway: 10.45.0.1
#      dnn: internet
#    - subnet: 2001:db8:cafe::/48
#      dnn: internet
#      dev: ogstun2
#    - subnet: 10.46.0.0/16
#      gateway: 10.46.0.1
#      dnn: ims
#      dev: ogstun3
#    - subnet: 2001:db8:babe::/48
#      dnn: ims
#      dev: ogstun3
```

3.53 /opt/GSM/osmo-operator/configs/open5gs/dump/open5gs/accounts.metadata.json

290 octets, 1 lignes → 1 lignes


```
{"indexes":[{"v":{"$numberInt":"2"},"key":{"_id":{"$numberInt":"1"}},"name":"_id_"},{"v":{"$numberInt":"2"},
 ↪  "key":{"username":{"$numberInt":"1"}},"name":"username_1","background":true,"unique":true}],"uuid":
 ↪  "735ad18ffaf64fd5bfb7d1ad99635967","collectionName":"accounts","type":"collection"}
```


3.54 /opt/GSM/osmo-operator/configs/open5gs/dump/open5gs/prelude.json

51 octets, 1 lignes → 1 lignes


```
{"ServerVersion":"8.0.29","ToolVersion":"100.18.0"}
```


3.55 /opt/GSM/osmo-operator/configs/open5gs/dump/open5gs/sessions.metadata.json

314 octets, 1 lignes → 1 lignes

```
{"indexes":[{"v":{"$numberInt":"2"},"key":{"_id":{"$numberInt":"1"}},"name":"_id_"},{"v":{"$numberInt":"2"},
↪  "key":{"expires":{"$numberInt":"1"}},"name":"expires_1","background":true,"expireAfterSeconds":{"$numberInt"
↪  :"0"}}],"uuid":"5c280e6a26b144d0a2832f3a97dde0a3","collectionName":"sessions","type":"collection"}
```

3.56 /opt/GSM/osmo-operator/configs/open5gs/dump/open5gs/subscribers.metadata.json

285 octets, 1 lignes → 1 lignes

```
{"indexes":[{"v":{"$numberInt":"2"},"key":{"_id":{"$numberInt":"1"}},"name":"_id_"},{"v":{"$numberInt":"2"},
↪  "key":{"imsi":{"$numberInt":"1"}},"name":"imsi_1","background":true,"unique":true}],"uuid":
↪  "cdee4054c5954e61a45edc7a6c67a5bf","collectionName":"subscribers","type":"collection"}
```

3.57 /opt/GSM/osmo-operator/configs/open5gs/freeDiameter/hss.conf

1272 octets, 20 lignes → 20 lignes

```
# hss.conf - freeDiameter config of the HSS: S6a peer of the MME (Open5GS v2.8.0, /opt/LTE/open5gs/install).
# Repository osmo-operator, configs/open5gs/freeDiameter; installed by
# tools/osmo-lte-install.sh --configs into $PREFIX/etc/freeDiameter. See mme.conf.

Identity = "hss.localdomain";
Realm = "localdomain";
ListenOn = "127.0.0.8";
TLS_Cred = "/opt/LTE/open5gs/install/etc/open5gs/tls/hss.crt",
↪  "/opt/LTE/open5gs/install/etc/open5gs/tls/hss.key";
TLS_CA = "/opt/LTE/open5gs/install/etc/open5gs/tls/ca.crt";
NoRelay;

LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dbg_msg_dumps.fdx" : "0x8888";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_rfc5777.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_mip6i.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_nasreq.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_nas_mipv6.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_dcca.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_dcca_3gpp.fdx";

ConnectPeer = "mme.localdomain" { ConnectTo = "127.0.0.2"; No_TLS; };
```

3.58 /opt/GSM/osmo-operator/configs/open5gs/freeDiameter/mme.conf

1998 octets, 32 lignes → 32 lignes

```
# mme.conf - freeDiameter config of the MME: S6a client of the HSS (Open5GS v2.8.0, /opt/LTE/open5gs/install).
# Repository osmo-operator, configs/open5gs/freeDiameter; installed by
# tools/osmo-lte-install.sh --configs into $PREFIX/etc/freeDiameter.
#
# The four .conf files (mme, hss, pcrf, smf) live in the repository because the
# Open5GS meson install skips its config files under DESTDIR: the
# osmo-build-open5gs .deb ships only bin/ and lib/, and each daemon aborts at
# init without its freeDiameter config. The TLS certificates are generated
# with openssl at install time ($PREFIX/etc/open5gs/tls).
#
# Peers use No_TLS (loopback only, so the Diameter traffic is readable in
# Wireshark); freeDiameter still requires a valid TLS_Cred at init.
# Addresses follow configs/open5gs/*.yaml (MME 127.0.0.2, HSS 127.0.0.8,
# SMF 127.0.0.4, PCRF 127.0.0.9). The install prefix (OPEN5GS_PREFIX) and the
# extension directory (lib/<triplet>/freeDiameter) are substituted at install time.
```

```
Identity = "mme.localdomain";
Realm = "localdomain";
ListenOn = "127.0.0.2";
TLS_Cred = "/opt/LTE/open5gs/install/etc/open5gs/tls/mme.crt",
 ↪  "/opt/LTE/open5gs/install/etc/open5gs/tls/mme.key";
TLS_CA = "/opt/LTE/open5gs/install/etc/open5gs/tls/ca.crt";
NoRelay;

LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dbg_msg_dumps.fdx" : "0x8888";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_rfc5777.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_mip6i.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_nasreq.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_nas_mipv6.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_dcca.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_dcca_3gpp.fdx";

ConnectPeer = "hss.localdomain" { ConnectTo = "127.0.0.8"; No_TLS; };
```

3.59 /opt/GSM/osmo-operator/configs/open5gs/freeDiameter/pcrf.conf

1276 octets, 20 lignes → 20 lignes

```
# pcrf.conf - freeDiameter config of the PCRF: Gx peer of the SMF (Open5GS v2.8.0, /opt/LTE/open5gs/install).
# Repository osmo-operator, configs/open5gs/freeDiameter; installed by
# tools/osmo-lte-install.sh --configs into $PREFIX/etc/freeDiameter. See mme.conf.

Identity = "pcrf.localdomain";
Realm = "localdomain";
ListenOn = "127.0.0.9";
TLS_Cred = "/opt/LTE/open5gs/install/etc/open5gs/tls/pcrf.crt",
 ↪  "/opt/LTE/open5gs/install/etc/open5gs/tls/pcrf.key";
TLS_CA = "/opt/LTE/open5gs/install/etc/open5gs/tls/ca.crt";
NoRelay;

LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dbg_msg_dumps.fdx" : "0x8888";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_rfc5777.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_mip6i.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_nasreq.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_nas_mipv6.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_dcca.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_dcca_3gpp.fdx";

ConnectPeer = "smf.localdomain" { ConnectTo = "127.0.0.4"; No_TLS; };
```

3.60 /opt/GSM/osmo-operator/configs/open5gs/freeDiameter/smf.conf

1283 octets, 20 lignes → 20 lignes

```
# smf.conf - freeDiameter config of the SMF (PGW-C): Gx client of the PCRF (Open5GS v2.8.0,
 ↪  /opt/LTE/open5gs/install).
# Repository osmo-operator, configs/open5gs/freeDiameter; installed by
# tools/osmo-lte-install.sh --configs into $PREFIX/etc/freeDiameter. See mme.conf.

Identity = "smf.localdomain";
Realm = "localdomain";
ListenOn = "127.0.0.4";
TLS_Cred = "/opt/LTE/open5gs/install/etc/open5gs/tls/smf.crt",
 ↪  "/opt/LTE/open5gs/install/etc/open5gs/tls/smf.key";
TLS_CA = "/opt/LTE/open5gs/install/etc/open5gs/tls/ca.crt";
NoRelay;

LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dbg_msg_dumps.fdx" : "0x8888";
```

```
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_rfc5777.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_mip6i.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_nasreq.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_nas_mipv6.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_dcca.fdx";
LoadExtension = "/opt/LTE/open5gs/install/lib/x86_64-linux-gnu/freeDiameter/dict_dcca_3gpp.fdx";

ConnectPeer = "pcrf.localdomain" { ConnectTo = "127.0.0.9"; No_TLS; };
```

3.61 /opt/GSM/osmo-operator/configs/plymouth/osmo-bts/osmo-bts.plymouth

477 octets, 12 lignes → 12 lignes

```
# osmo-bts - le theme Plymouth du banc.
# Les images sont GENEREES par tools/plymouth-render.py au build de l ISO
# (iso_modules/86-finitions.sh) : ce depot ne porte que le dessin, en Python,
# et le script d animation ci-contre.
[Plymouth Theme]
Name=osmo-operator (antenne)
Description=Une antenne et des cercles qui s ouvrent, en 2D pure
ModuleName=script

[script]
ImageDir=/usr/share/plymouth/themes/osmo-bts
ScriptFile=/usr/share/plymouth/themes/osmo-bts/osmo-bts.script
```

3.62 /opt/GSM/osmo-operator/configs/plymouth/osmo-bts/osmo-bts.script

6743 octets, 164 lignes → 164 lignes

```
# osmo-bts.script - Plymouth boot animation of the "osmo-bts" theme.
#
# Plymouth calls the refresh function ~50 times per second; the script only
# advances a counter and swaps the image of two sprites: the wave strip
# (24 frames of rings opening from the tower and from the phone) and the
# knob of the progress bar. Tower, phone and title are painted once in
# background.png so the initrd does not carry 24 copies of the scenery.
# Animated preview: wiki/plymouth-osmo-bts.gif (tools/plymouth-render.py --gif).
#
# All positions are FRACTIONS of the real screen size (Window.GetWidth()/
# GetHeight()), so the layout holds on 1366x768 as on 4K. THIS FILE IS A
# TEMPLATE: the @@...@@ placeholders are filled in by tools/plymouth-render.py,
# which draws the background and therefore knows its geometry.

sw = Window.GetWidth();
sh = Window.GetHeight();

# ── BACKGROUND ──────────────────────────────────────────────────────
bg_image = Image("background.png");
bg = Sprite(bg_image.Scale(sw, sh));
bg.SetX(0);
bg.SetY(0);
bg.SetZ(-100);

# ── WAVES BETWEEN THE TWO ANTENNAS ──────────────────────────────────
# 24 frames played at ~12 fps (one frame every 4 refreshes), files burst-NN.png.
n_frames = 24;
for (i = 0; i < n_frames; i++) {
    if (i < 10) {
        burst_image[i] = Image("burst-0" + i + ".png");
    } else {
        burst_image[i] = Image("burst-" + i + ".png");
```

```
    }
    burst_scaled[i] = burst_image[i].Scale(sw * @@BW@@, sh * @@BH@@);
}
# @@BX@@ / @@BY@@ / @@BW@@ / @@BH@@: the strip between the two antennas, as
# screen FRACTIONS injected by tools/plymouth-render.py (it places the tower
# and the phone in background.png, so moving them in the drawing moves the waves).
burst = Sprite();
burst.SetX(sw * @@BX@@);
burst.SetY(sh * @@BY@@);
burst.SetZ(10);

# ── PROGRESS BAR ──────────────────────────────────────────────────────
box_image = Image("progress-box.png");
bar_image = Image("progress-bar.png");
box_w = sw * 0.27;
box_h = sh * 0.008;
box = Sprite(box_image.Scale(box_w, box_h));
box.SetX((sw - box_w) / 2);
box.SetY(sh * 0.90);
box.SetZ(10);
bar_w = box_w * 0.16;
bar = Sprite(bar_image.Scale(bar_w, box_h));
bar.SetY(sh * 0.90);
bar.SetZ(11);

tick = 0;

# ── PASSWORD ENTRY: THE LUKS PASSPHRASE OF /home ──────────────────────
# Single prompt ("decrypt /home -passphrase- or Enter"): the passphrase opens
# /home, an empty Enter boots as root without /home. A blinking cursor shows
# in the field while the question is pending and each typed character is
# echoed as "*".
#
# This entry is NOT the default path: unlock-home asks on the text console
# after leaving plymouth. It is used only when /etc/default/crypthome sets
# CRYPTHOME_PROMPT=plymouth; unlock-home then goes through
# systemd-ask-password, whose plymouth agent calls the callbacks below.
# Known limitation: on some Intel + NVIDIA laptops the plymouth keyboard
# input does not work (cursor blinks, no key accepted, timeout).
#
# Everything is declared HERE, before refresh_callback: it blinks the cursor
# and runs from the first frame.
entry_image = Image("entry.png");
entry = Sprite();
entry.SetZ(20);
prompt_sprite = Sprite();
prompt_sprite.SetZ(20);
stars_sprite = Sprite();
stars_sprite.SetZ(21);
# Cursor: a terminal-style underscore (Ubuntu Mono is in the initrd; otherwise
# fontconfig falls back to the default font).
cursor_image = Image.Text("_", 0.95, 0.96, 0.98, 1, "Ubuntu Mono 16");
cursor_sprite = Sprite(cursor_image);
cursor_sprite.SetZ(22);
cursor_sprite.SetOpacity(0);
saisie_visible = 0;

fun refresh_callback() {
    tick++;
    # Waves: one frame every 4 refreshes (~12 fps).
    frame = Math.Int(tick / 4) % n_frames;
    burst.SetImage(burst_scaled[frame]);
    # Knob: a steady back-and-forth in the bar (no boot progress is available
    # to a script theme without a progress plugin, so no fake percentage).
    u = (tick % 200) / 200;
    if (u > 0.5) {
```

```
        u = 1 - u;
    }
    bar.SetX((sw - box_w) / 2 + u * 2 * (box_w - bar_w));
    # Cursor: half a second on, half a second off (~50 Hz refresh).
    if (saisie_visible == 1) {
        if (Math.Int(tick / 25) % 2 == 0) {
            cursor_sprite.SetOpacity(1);
        } else {
            cursor_sprite.SetOpacity(0);
        }
    } else {
        cursor_sprite.SetOpacity(0);
    }
}
Plymouth.SetRefreshFunction(refresh_callback);

fun display_password_callback(prompt, bullets) {
    ew = sw * 0.32;
    eh = sh * 0.052;
    entry.SetImage(entry_image.Scale(ew, eh));
    entry.SetX((sw - ew) / 2);
    entry.SetY(sh * 0.72);
    entry.SetOpacity(1);
    text = Image.Text(prompt, 0.90, 0.93, 0.95, 1, "Ubuntu 15");
    prompt_sprite.SetImage(text);
    prompt_sprite.SetX((sw - text.GetWidth()) / 2);
    prompt_sprite.SetY(sh * 0.66);
    prompt_sprite.SetOpacity(1);
    # Stars: one "*" per typed character, left-aligned in the field.
    marge = sh * 0.016;
    stars_w = 0;
    if (bullets > 0) {
        stars = "";
        for (i = 0; i < bullets; i++) {
            stars = stars + "*";
        }
        simg = Image.Text(stars, 0.95, 0.96, 0.98, 1, "Ubuntu Mono 16");
        stars_sprite.SetImage(simg);
        stars_sprite.SetX((sw - ew) / 2 + marge);
        stars_sprite.SetY(sh * 0.72 + (eh - simg.GetHeight()) / 2);
        stars_sprite.SetOpacity(1);
        stars_w = simg.GetWidth();
    } else {
        stars_sprite.SetOpacity(0);
    }
    # The cursor follows the last "*"; refresh_callback blinks it.
    cursor_sprite.SetX((sw - ew) / 2 + marge + stars_w);
    cursor_sprite.SetY(sh * 0.72 + (eh - cursor_image.GetHeight()) / 2);
    saisie_visible = 1;
}

fun display_normal_callback() {
    entry.SetOpacity(0);
    prompt_sprite.SetOpacity(0);
    stars_sprite.SetOpacity(0);
    cursor_sprite.SetOpacity(0);
    saisie_visible = 0;
}

Plymouth.SetDisplayPasswordFunction(display_password_callback);
Plymouth.SetDisplayNormalFunction(display_normal_callback);

# ── NO MESSAGES ─────────────────────────────────────────────────────
# No message callback: what systemd and fsck send to plymouth is not shown
# (it stays in journalctl -b). Anything that really needs to talk during boot
# uses the text console, as unlock-home does for the passphrase.
```

3.63 /opt/GSM/osmo-operator/configs/plymouth/vortex-ubuntu/SOURCE

295 octets, 4 lignes → 4 lignes

```
Theme Plymouth « Vortex Ubuntu », repris tel quel (GPL-2, voir LICENSE).
Origine : https://github.com/emanuele-scarsella/vortex-ubuntu-plymouth-theme
Commit   : 3072445ee35f10a0268baa9aaa326c0abd54af3e (2025-06-20)
Installe par iso_modules/86-finitions.sh comme theme de demarrage par defaut.
```

3.64 /opt/GSM/osmo-operator/configs/plymouth/vortex-ubuntu/logo.svg

3272 octets, 90 lignes → 90 lignes

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!-- Created with Inkscape (http://www.inkscape.org/) -->

<svg
   width="376"
   height="376"
   viewBox="0 0 99.483332 99.483333"
   version="1.1"
   id="svg308"
   inkscape:version="1.2.2 (b0a8486541, 2022-12-01)"
   sodipodi:docname="Ubuntu People.svg"
   inkscape:export-filename="../../../Code/Themes/vortex-ubuntu-plymouth-theme/vortex-ubuntu/logo.png"
   inkscape:export-xdpi="96"
   inkscape:export-ydpi="96"
   xmlns:inkscape="http://www.inkscape.org/namespaces/inkscape"
   xmlns:sodipodi="http://sodipodi.sourceforge.net/DTD/sodipodi-0.dtd"
   xmlns="http://www.w3.org/2000/svg"
   xmlns:svg="http://www.w3.org/2000/svg">
  <sodipodi:namedview
     id="namedview310"
     pagecolor="#ffffff"
     bordercolor="#666666"
     borderopacity="1.0"
     inkscape:showpageshadow="2"
     inkscape:pageopacity="0"
     inkscape:pagecheckerboard="true"
     inkscape:deskcolor="#d1d1d1"
     inkscape:document-units="px"
     showgrid="false"
     inkscape:zoom="0.49219345"
     inkscape:cx="-428.69323"
     inkscape:cy="358.59884"
     inkscape:window-width="1920"
     inkscape:window-height="1011"
     inkscape:window-x="0"
     inkscape:window-y="32"
     inkscape:window-maximized="1"
     inkscape:current-layer="g277" />
  <defs
     id="defs305" />
  <g
     inkscape:label="Layer 1"
     inkscape:groupmode="layer"
     id="layer1">
    <g
       id="g277"
       transform="matrix(0.45734222,0,0,0.45734222,-7.1305763,-69.626942)"
       style="fill:#ebebeb;fill-opacity:1">
      <g
         id="g300">
```

```
        <circle
           class="cls-3"
           cx="58.09"
           cy="255.92"
           r="26.299999"
           id="circle44"
           style="fill:#ebebeb;fill-opacity:1" />
        <circle
           class="cls-3"
           cx="167.63"
           cy="198.25"
           r="26.299999"
           id="circle46"
           style="fill:#ebebeb;fill-opacity:1" />
        <path
           class="cls-3"
           d="M 117.41,323.62 C 98.46,319.56 82.63,307.46 73.73,290.3 c -7.01,3.19 -14.9,4.16 -22.49,2.76
↪  10.77,26.45 33.61,45.66 61.65,51.67 6.15,1.32 12.42,1.96 18.68,1.92 -4.83,-6.35 -7.52,-14.01 -7.7,-21.99
↪  -2.17,-0.24 -4.33,-0.59 -6.45,-1.05 z"
           id="path48"
           style="fill:#ebebeb;fill-opacity:1" />
        <circle
           class="cls-3"
           cx="161.71001"
           cy="323.76001"
           r="26.299999"
           id="circle50"
           style="fill:#ebebeb;fill-opacity:1" />
        <path
           class="cls-3"
           d="m 198.31,314.1 c 8.18,-10.31 13.94,-22.5 16.71,-35.45 4.84,-22.59 0.32,-46.29 -12.4,-65.51
↪  -3.03,7.14 -8.17,13.17 -14.79,17.32 7.1,13.38 9.26,28.82 6.08,43.67 -1.56,7.27 -4.31,14.12 -8.18,20.38
↪  6.19,5.08 10.56,11.91 12.59,19.6 z"
           id="path52"
           style="fill:#ebebeb;fill-opacity:1" />
        <path
           class="cls-3"
           d="m 56.06,218.2 c 0.67,-0.04 1.34,-0.05 2,-0.05 2.66,0 5.31,0.28 7.94,0.85 4.29,0.92 8.32,2.54
↪  12.01,4.84 11.84,-17.03 30.95,-27.25 51.65,-27.63 0.11,-1.99 0.38,-3.97 0.79,-5.92 1.21,-5.63 3.67,-10.9
↪  7.19,-15.41 -33.08,-2.62 -65.22,14.45 -81.59,43.33 z"
           id="path54"
           style="fill:#ebebeb;fill-opacity:1" />
      </g>
    </g>
  </g>
</svg>
```

3.65 /opt/GSM/osmo-operator/configs/plymouth/vortex-ubuntu/vortex-ubuntu.grub

46 octets, 3 lignes → 3 lignes

```
if background_color 00,00,00; then
  clear
fi
```

3.66 /opt/GSM/osmo-operator/configs/plymouth/vortex-ubuntu/vortex-ubuntu.plymouth

269 octets, 9 lignes → 9 lignes

```
[Plymouth Theme]
Name=Vortex Ubuntu
Description=Ubuntu logo with spinning colored stripes
ModuleName=script

[script]
ImageDir=/usr/share/plymouth/themes/vortex-ubuntu
ScriptFile=/usr/share/plymouth/themes/vortex-ubuntu/vortex-ubuntu.script
UseFirmwareBackground=false
```

3.67 /opt/GSM/osmo-operator/configs/plymouth/vortex-ubuntu/vortex-ubuntu.script

5502 octets, 190 lignes → 190 lignes

```
/*
 * Configuration
 */

logo_spin = -0.009;

/*
 * Screen dimensions
 */

// Screen size
screen.w = Window.GetWidth(0);
screen.h = Window.GetHeight(0);
screen.half.w = screen.w / 2;
screen.half.h = screen.h / 2;

/*
 * Background
 */

Window.SetBackgroundTopColor(0, 0, 0);
Window.SetBackgroundBottomColor(0, 0, 0);
bg = Sprite(Image("bg.png").Scale(screen.w, screen.h));
bg.SetZ(-10);

/*
 * Global state
 */

// Message
message = null;

// Question and password prompts
prompt = null;
response = null;
bullet.image = Image("dot.png").Scale(10, 10);

text_line_height = Image.Text("Dummy text abcxyz").GetHeight();
prompt_y = screen.h * 0.75;
response_y = prompt_y + text_line_height * 2;
message_y = prompt_y + text_line_height * 4;

// Flow
time_counter = 0.0;

// Stop animation flag
stop_animation = 0;

/*
 * Splash image
```

```
 */

splash_scale = 0.75;
splash_y_position = 0.4;
if (Plymouth.GetMode() == "shutdown") {
    splash_y_position = 0.5;
}

fun make_splash_sprite_from_image(img) {
    local.splash_sprite.image = img;
    local.splash_sprite.sprite = make_splash_sprite_sprite(local.splash_sprite.image);
    return local.splash_sprite;
}

fun make_splash_sprite_sprite(img) {
    local.sprite = Sprite(local.img);
    local.sprite.SetX(screen.half.w - local.img.GetWidth() / 2);
    local.sprite.SetY((screen.h * splash_y_position) - local.img.GetHeight() / 2);
    return local.sprite;
}

fun make_splash_sprite(fn) {
    local.img = Image(local.fn);
    local.splash_sprite.image = local.img.Scale(img.GetWidth() * splash_scale, local.img.GetHeight() *
 ↪  splash_scale);
    local.splash_sprite.sprite = make_splash_sprite_sprite(local.splash_sprite.image);
    return local.splash_sprite;
}

spins = [0.009, 0.007, 0.006, 0.0053, 0.0048, 0.004, 0.0035, 0.003, 0.0025, 0.002];
spin_sprite_count = 10;

logo_image = Image("logo.png").Scale(120 * splash_scale, 120 * splash_scale);
logo_sprite = make_splash_sprite_from_image(logo_image);

splash_static_bg_image = Image("static.png");
splash_static_bg_image = splash_static_bg_image.Scale(splash_static_bg_image.GetWidth() * splash_scale * 0.3,
 ↪  splash_static_bg_image.GetHeight() * splash_scale * 0.3);
splash_static_bg = make_splash_sprite_from_image(splash_static_bg_image);

for (i = 0; i < spin_sprite_count; i++) {
    splash_sprites[i] = make_splash_sprite((i + 1) + ".png");
}

fun rotate_splash_sprite(splash_sprite, rotation) {
    splash_sprite.sprite.SetImage(splash_sprite.image.Rotate(rotation));
}

fun update_splash_sprites() {
    if (stop_animation == 0) {
      rotate_splash_sprite(logo_sprite, time_counter * logo_spin);
      for (i = 0; i < spin_sprite_count; i++) {
          rotate_splash_sprite(splash_sprites[i], time_counter * spins[i]);
      }
    }
}

/*
 * Callback functions
 */

fun refresh_callback() {
    time_counter++;
    update_splash_sprites();
}
Plymouth.SetRefreshFunction(refresh_callback);
```

```
// Progress bar
if (Plymouth.GetMode () == "boot") {
    ipb = Image("pb.png");
    pb = Sprite(ipb);
    pb.SetX(screen.half.w - 50);
    pb.SetY(screen.h * 0.6 - ipb.GetHeight() / 2);

    fun boot_progress_callback(duration, progress) {
         pb.SetImage(ipb.Scale(progress * 100, 3));
    }
    Plymouth.SetBootProgressFunction(boot_progress_callback);
}

fun display_prompt(prompt_text) {
    prompt.image = Image.Text(prompt_text, 1, 1, 1);
    prompt.sprite = Sprite(prompt.image);
    prompt.sprite.SetX(screen.half.w - prompt.image.GetWidth() / 2);
    prompt.sprite.SetY(prompt_y);
}

fun display_question_callback(prompt_text, entry) {
    stop_animation = 1;
    display_prompt(prompt_text);

    local.entry_config.alpha = 1.0;
    local.entry_config.font = "Sans Bold 12";
    if (entry == "") {
        entry = "Type your response";
        local.entry_config.alpha = 0.5;
        local.entry_config.font = "Sans 12";
    }

    response = null;
    response.image = Image.Text(entry, 1, 1, 1, local.entry_config.alpha, local.entry_config.font);
    response.sprite = Sprite(response.image);
    response.sprite.SetX(screen.half.w - response.image.GetWidth() / 2);
    response.sprite.SetY(response_y);
}
Plymouth.SetDisplayQuestionFunction(display_question_callback);

fun display_password_callback(prompt_text, dot_count) {
    stop_animation = 1;
    display_prompt(prompt_text);

    response = null;
    if (dot_count > 0) {
        dot_start_position = screen.half.w - (dot_count * bullet.image.GetWidth()) / 2;
        for (i = 0; i < dot_count; i++) {
            response[i].sprite = Sprite(bullet.image);
            response[i].sprite.SetX(dot_start_position + i * bullet.image.GetWidth());
            response[i].sprite.SetY(response_y + (text_line_height - bullet.image.GetHeight()) / 2);
        }
    } else {
        response.image = Image.Text("Enter passphrase", 1, 1, 1, 0.5);
        response.sprite = Sprite(response.image);
        response.sprite.SetX(screen.half.w - response.image.GetWidth() / 2);
        response.sprite.SetY(response_y);
    }
}
Plymouth.SetDisplayPasswordFunction(display_password_callback);

fun display_normal_callback() {
    prompt = null;
    message = null;
    response = null;
```

```
    stop_animation = 0;
}
Plymouth.SetDisplayNormalFunction(display_normal_callback);

fun message_callback(text) {
    message.image = Image.Text(text, 1, 1, 1, 0.75, "Sans 10");
    message.sprite = Sprite(message.image);
    message.sprite.SetPosition(screen.half.w - message.image.GetWidth() / 2, message_y);
}
Plymouth.SetMessageFunction(message_callback);
```

3.68 /opt/GSM/osmo-operator/configs/pmos/pmbootstrap_v3.cfg

651 octets, 21 lignes → 21 lignes

```
# pmbootstrap_v3.cfg - pmbootstrap template for the postmarketOS phone image
# (Phosh UI, qemu-amd64 device, modem/audio packages). Installed by
# tools/osmo-pmos-install.sh; tools/osmo-pmos-build.sh substitutes @WORK@ and
# @APORTS@ and writes the result to ~/.config/pmbootstrap_v3.cfg.

[pmbootstrap]
aports = @APORTS@
build_pkgs_on_install = False
extra_packages = alsa-utils,alsa-ucm-conf,pipewire,pipewire-
 ↪  alsa,wireplumber,socat,minicom,picocom,networkmanager,networkmanager-ppp,ppp
hostname = pmos-gsm
is_default_channel = False
locale = fr_FR.UTF-8
timezone = Europe/Paris
ui = phosh
device = qemu-amd64
work = @WORK@

[providers]

[mirrors]
```

3.69 /opt/GSM/osmo-operator/configs/srsran/enb.conf

20543 octets, 478 lignes → 476 lignes (1 groupes compactés)

```
#####################################################################
#                   srsENB configuration file
#####################################################################

#####################################################################
# eNB configuration
#
# enb_id:               20-bit eNB identifier.
# mcc:                  Mobile Country Code
# mnc:                  Mobile Network Code
# mme_addr:             IP address of MME for S1 connnection
# gtp_bind_addr:        Local IP address to bind for GTP connection
# gtp_advertise_addr:   IP address of eNB to advertise for DL GTP-U Traffic
# s1c_bind_addr:        Local IP address to bind for S1AP connection
# s1c_bind_port:        Source port for S1AP connection (0 means any)
# n_prb:                Number of Physical Resource Blocks (6,15,25,50,75,100)
# tm:                   Transmission mode 1-4 (TM1 default)
# nof_ports:            Number of Tx ports (1 port default, set to 2 for TM2/3/4)
#
#####################################################################
[enb]
```

```
enb_id = 0x19B
mcc = 001
mnc = 01
mme_addr = 127.0.0.2
gtp_bind_addr = 127.0.1.1
s1c_bind_addr = 127.0.1.1
s1c_bind_port = 0
n_prb = 50
#tm = 4
#nof_ports = 2

#####################################################################
# eNB configuration files
#
# sib_config:  SIB1, SIB2 and SIB3 configuration file
# note: When enabling MBMS, use the sib.conf.mbsfn configuration file which includes SIB13
# rr_config:   *Radio Resources configuration file
# rb_config:   SRB/DRB configuration file
#####################################################################
[enb_files]
sib_config = sib.conf
rr_config  = rr.conf
rb_config = rb.conf

#####################################################################
# RF configuration
#
# dl_earfcn: EARFCN code for DL (only valid if a single cell is configured in rr.conf)
# tx_gain: Transmit gain (dB).
# rx_gain: Optional receive gain (dB). If disabled, AGC if enabled
#
# Optional parameters:
# dl_freq:            Override DL frequency corresponding to dl_earfcn
# ul_freq:            Override UL frequency corresponding to dl_earfcn (must be set if dl_freq is set)
# device_name:        Device driver family
#                     Supported options: "auto" (uses first driver found), "UHD", "bladeRF", "soapy" or "zmq"
# device_args:        Arguments for the device driver. Options are "auto" or any string.
#                     Default for UHD: "recv_frame_size=9232,send_frame_size=9232"
#                     Default for bladeRF: ""
# time_adv_nsamples:  Transmission time advance (in number of samples) to compensate for RF delay
#                     from antenna to timestamp insertion.
#                     Default "auto". B210 USRP: 100 samples, bladeRF: 27
#####################################################################
[rf]
dl_earfcn = 3350
tx_gain = 80
rx_gain = 40

#device_name = zmq

# For best performance in 2x2 MIMO and >= 15 MHz use the following device_args settings:
#     USRP B210: num_recv_frames=64,num_send_frames=64
#     And for 75 PRBs, also append ",master_clock_rate=15.36e6" to the device args

# For best performance when BW<5 MHz (25 PRB), use the following device_args settings:
#     USRP B210: send_frame_size=512,recv_frame_size=512

#device_args = auto
#time_adv_nsamples = auto

# Example for ZMQ-based operation with TCP transport for I/Q samples
device_name = zmq
device_args = fail_on_disconnect=true,tx_port=tcp://*:2000,rx_port=tcp://localhost:2001,id=enb,base_srate=11.52e6

#####################################################################
# Packet capture configuration
```

```
#
# MAC-layer packets are captured to a file in the compact format which can
# be decoded by Wireshark. For decoding, use the UDP dissector and the UDP
# heuristic dissection. Edit the preferences (Edit > Preferences >
# Protocols > DLT_USER) for DLT_USER to add an entry for DLT=149 with
# Protocol=udp. Further, enable the heuristic dissection in UDP under:
# Analyze > Enabled Protocols > MAC-LTE > mac_lte_udp and MAC-NR > mac_nr_udp
# For more information see: https://wiki.wireshark.org/MAC-LTE
# Configuring this Wireshark preferences is needed for decoding the MAC PCAP
# files as well as for the live network capture option.
#
# Please note that this setting will by default only capture MAC
# frames on dedicated channels, and not SIB.  You have to build with
# WRITE_SIB_PCAP enabled in srsenb/src/stack/mac/mac.cc if you want
# SIB to be part of the MAC pcap file.
#
# S1AP Packets are captured to a file in the compact format which can
# be decoded by the Wireshark s1ap dissector with DLT 150.
# To use the dissector, edit the preferences for DLT_USER to
# add an entry with DLT=150, Payload Protocol=s1ap.
#
# enable:        Enable MAC layer packet captures (true/false)
# filename:      File path to use for LTE MAC packet captures
# nr_filename:   File path to use for NR MAC packet captures
# s1ap_enable:   Enable or disable the PCAP.
# s1ap_filename: File name where to save the PCAP.
#
# mac_net_enable: Enable MAC layer packet captures sent over the network (true/false default: false)
# bind_ip: Bind IP address for MAC network trace (default: "0.0.0.0")
# bind_port: Bind port for MAC network trace (default: 5687)
# client_ip: Client IP address for MAC network trace (default: "127.0.0.1")
# client_port Client IP address for MAC network trace (default: 5847)
############################################################################
[pcap]
#enable = false
#filename = /tmp/enb_mac.pcap
#nr_filename = /tmp/enb_mac_nr.pcap
#s1ap_enable = false
#s1ap_filename = /tmp/enb_s1ap.pcap

# Lab setting: every MAC PDU (DL and UL, hence RRC and NAS; open5gs uses EEA0
# so everything is readable) is sent over UDP to 127.0.0.1:5847 in Wireshark
# "mac-lte-framed" format. osmo-egprs-web captures it alongside GSMTAP and
# SCTP, and the pmOS phone's Network Monitor app displays it. Without a
# listener the UDP is simply dropped.
mac_net_enable = true
bind_ip = 0.0.0.0
bind_port = 5687
client_ip = 127.0.0.1
client_port = 5847
#bind_ip = 0.0.0.0
#bind_port = 5687
#client_ip = 127.0.0.1
#client_port = 5847

############################################################################
# Log configuration
#
# Log levels can be set for individual layers. "all_level" sets log
# level for all layers unless otherwise configured.
# Format: e.g. phy_level = info
#
# In the same way, packet hex dumps can be limited for each level.
# "all_hex_limit" sets the hex limit for all layers unless otherwise
# configured.
# Format: e.g. phy_hex_limit = 32
```

```
#
# Logging layers: rf, phy, phy_lib, mac, rlc, pdcp, rrc, gtpu, s1ap, stack, all
# Logging levels: debug, info, warning, error, none
#
# filename: File path to use for log output. Can be set to stdout
#           to print logs to standard output
# file_max_size: Maximum file size (in kilobytes). When passed, multiple files are created.
#                If set to negative, a single log file will be created.
##########################################################################
[log]
all_level = warning
all_hex_limit = 32
filename = /tmp/enb.log
file_max_size = -1

[gui]
enable = false

##########################################################################
# Scheduler configuration options
#
# sched_policy:      User MAC scheduling policy (E.g. time_rr, time_pf)
# min_aggr_level:    Optional minimum aggregation level index (l=log2(L) can be 0, 1, 2 or 3)
# max_aggr_level:    Optional maximum aggregation level index (l=log2(L) can be 0, 1, 2 or 3)
# adaptive_aggr_level: Boolean flag to enable/disable adaptive aggregation level based on target BLER
# pdsch_mcs:         Optional fixed PDSCH MCS (ignores reported CQIs if specified)
# pdsch_max_mcs:     Optional PDSCH MCS limit
# pusch_mcs:         Optional fixed PUSCH MCS (ignores reported CQIs if specified)
# pusch_max_mcs:     Optional PUSCH MCS limit
# min_nof_ctrl_symbols: Minimum number of control symbols
# max_nof_ctrl_symbols: Maximum number of control symbols
# pucch_multiplex_enable: Allow PUCCH HARQ to collide with PUSCH and other PUCCH
# pucch_harq_max_rb: Maximum number of RB to be used for PUCCH on the edges of the grid.
#                    If defined and greater than 0, the scheduler will avoid DL PDCCH allocations if
#                    PUCCH HARQ falls outside this region
# target_bler:       Target BLER (in decimal) to achieve via adaptive link
# max_delta_dl_cqi:  Maximum shift in CQI for adaptive DL link
# max_delta_ul_snr:  Maximum shift in UL SNR for adaptive UL link
# adaptive_dl_mcs_step_size: Step size or learning rate used in adaptive DL MCS link
# adaptive_ul_mcs_step_size: Step size or learning rate used in adaptive UL MCS link
# min_tpc_tti_interval: Minimum TTI interval between TPCs different than 1
# ul_snr_avg_alpha:  Exponential Average alpha coefficient used in estimation of UL SNR
# init_ul_snr_value: Initial UL SNR value used for computing MCS in the first UL grant
# init_dl_cqi:       DL CQI value used before any CQI report is available to the eNB
# max_sib_coderate:  Upper bound on SIB and RAR grants coderate
# pdcch_cqi_offset:  CQI offset in derivation of PDCCH aggregation level
# nr_pdsch_mcs:      Optional fixed NR PDSCH MCS (ignores reported CQIs if specified)
# nr_pusch_mcs:      Optional fixed NR PUSCH MCS (ignores reported CQIs if specified)
#
##########################################################################
[scheduler]
#policy     = time_pf
#policy_args = 2
#min_aggr_level   = 0
#max_aggr_level   = 3
#adaptive_aggr_level = false
#pdsch_mcs        = -1
#pdsch_max_mcs    = -1
#pusch_mcs        = -1
#pusch_max_mcs    = 16
#min_nof_ctrl_symbols = 1
#max_nof_ctrl_symbols = 3
#pucch_multiplex_enable = false
#pucch_harq_max_rb = 0
#target_bler = 0.05
#max_delta_dl_cqi = 5
```

```
#max_delta_ul_snr = 5
#adaptive_dl_mcs_step_size = 0.001
#adaptive_ul_mcs_step_size = 0.001
#min_tpc_tti_interval = 1
#ul_snr_avg_alpha=0.05
#init_ul_snr_value=5
#init_dl_cqi=5
#max_sib_coderate=0.3
#pdcch_cqi_offset=0
#nr_pdsch_mcs=28
#nr_pusch_mcs=28

###########################################################################
# Slicing configuration
# enable_eMBB           Enables enhanced mobile broadband (eMBB) slice in the gNodeB
# enable_URLLC          Enables Ultra Reliable Low Latency Communications (URLLC) slice in the gNodeB
# enable_MIoT           Enables Massive Internet of Things (MIoT) slice in the gNodeB
# eMBB_sd               eMBB slice differentiator
# URLLC_sd              URLLC slice differentiator
# MIoT_sd               MIoT slice differentiator
###########################################################################
[slicing]
#enable_eMBB = false
#enable_URLLC = false
#enable_MIoT = false
#eMBB_sd = 1
#URLLC_sd = 1
#MIoT_sd = 1

###########################################################################
# eMBMS configuration options
#
# enable:               Enable MBMS transmission in the eNB
# m1u_multiaddr:        Multicast address the M1-U socket will register to
# m1u_if_addr:          Address of the interface the M1-U interface will listen to for multicast packets
# mcs:                  Modulation and Coding scheme for MBMS traffic
#
###########################################################################
[embms]
#enable = false
#m1u_multiaddr = 239.255.0.1
#m1u_if_addr = 127.0.1.201
#mcs = 20
  ×3
###########################################################################
# Channel emulator options:
# enable:            Enable/disable internal Downlink/Uplink channel emulator
#
# -- AWGN Generator
# awgn.enable:       Enable/disable AWGN generator
# awgn.snr:          Target SNR in dB
#
# -- Fading emulator
# fading.enable:     Enable/disable fading simulator
# fading.model:      Fading model + maximum doppler (E.g. none, epa5, eva70, etu300, etc)
#
# -- Delay Emulator     delay(t) = delay_min + (delay_max - delay_min) * (1 + sin(2pi*t/period)) / 2
#                       Maximum speed [m/s]: (delay_max - delay_min) * pi * 300 / period
# delay.enable:      Enable/disable delay simulator
# delay.period_s:    Delay period in seconds
# delay.init_time_s: Delay initial time in seconds
# delay.maximum_us:  Maximum delay in microseconds
# delay.minumum_us:  Minimum delay in microseconds
#
# -- Radio-Link Failure (RLF) Emulator
# rlf.enable:        Enable/disable RLF simulator
```

```
# rlf.t_on_ms:       Time for On state of the channel (ms)
# rlf.t_off_ms:      Time for Off state of the channel (ms)
#
# -- High Speed Train Doppler model simulator
# hst.enable:        Enable/disable HST simulator
# hst.period_s:      HST simulation period in seconds
# hst.fd_hz:         Doppler frequency in Hz
# hst.init_time_s:   Initial time in seconds
######################################################################
[channel.dl]
#enable        = false

[channel.dl.awgn]
#enable        = false
#snr            = 30

[channel.dl.fading]
#enable        = false
#model         = none

[channel.dl.delay]
#enable        = false
#period_s      = 3600
#init_time_s   = 0
#maximum_us    = 100
#minimum_us    = 10

[channel.dl.rlf]
#enable        = false
#t_on_ms       = 10000
#t_off_ms      = 2000

[channel.dl.hst]
#enable        = false
#period_s      = 7.2
#fd_hz         = 750.0
#init_time_s   = 0.0

[channel.ul]
#enable        = false

[channel.ul.awgn]
#enable        = false
#n0            = -30

[channel.ul.fading]
#enable        = false
#model         = none

[channel.ul.delay]
#enable        = false
#period_s      = 3600
#init_time_s   = 0
#maximum_us    = 100
#minimum_us    = 10

[channel.ul.rlf]
#enable        = false
#t_on_ms       = 10000
#t_off_ms      = 2000

[channel.ul.hst]
#enable        = false
#period_s      = 7.2
#fd_hz         = -750.0
#init_time_s   = 0.0
```

```
#####################################################################
# CFR configuration options
#
# The CFR module provides crest factor reduction for the transmitted signal.
#
# enable:           Enable or disable the CFR. Default: disabled
#
# mode:             manual:   CFR threshold is set by cfr_manual_thres (default).
#                   auto_ema: CFR threshold is adaptive based on the signal PAPR. Power avg. with Exponential
 ↪  Moving Average.
#                             The time constant of the averaging can be tweaked with the ema_alpha parameter.
#                  auto_cma: CFR threshold is adaptive based on the signal PAPR. Power avg. with Cumulative Moving
 ↪  Average.
#                           Use with care, as CMA's increasingly slow response may be unsuitable for most use cases.
#
# strength:         Ratio between amplitude-limited vs unprocessed signal (0 to 1). Default: 1
# manual_thres:     Fixed manual clipping threshold for CFR manual mode. Default: 0.5
# auto_target_papr: Signal PAPR target (in dB) in CFR auto modes. output PAPR can be higher due to peak smoothing.
 ↪  Default: 8
# ema_alpha:        Alpha coefficient for the power average in auto_ema mode. Default: 1/7
#
#####################################################################
[cfr]
#enable           = false
#mode             = manual
#manual_thres     = 0.5
#strength         = 1
#auto_target_papr = 8
#ema_alpha        = 0.0143

# E2 Agent configuration options
#
# ric_ip:                IP address of the RIC controller
# ric_port:              Port of the RIC controller
# ric_bind_ip:           Local IP address to bind for RIC connection
# ric_bind_port:         Local port to bind for RIC connection
# max_ric_setup_retries: Maximum amount of retries to setup the RIC connection. If this value is exceeded, an
 ↪  alarm is written to the log. -1 means infinity.
# ric_connect_timer:     Connection Retry Timer for RIC connection (seconds)
#####################################################################
[e2_agent]
#enable = false
#ric_ip =  127.0.0.1
#ric_port = 36421
#ric_bind_ip =  127.0.0.1
#ric_bind_port = 36425
#max_ric_setup_retries = -1
#ric_connect_timer = 10

#####################################################################
# Expert configuration options
#
# pusch_max_its:        Maximum number of turbo decoder iterations (default: 4)
# nr_pusch_max_its:     Maximum number of LDPC iterations for NR (Default 10)
# pusch_8bit_decoder:   Use 8-bit for LLR representation and turbo decoder trellis computation (experimental)
# nof_phy_threads:      Selects the number of PHY threads (maximum: 4, minimum: 1, default: 3)
# metrics_period_secs:  Sets the period at which metrics are requested from the eNB
# metrics_csv_enable:   Write eNB metrics to CSV file.
# metrics_csv_filename: File path to use for CSV metrics
# report_json_enable:   Write eNB report to JSON file (default: disabled)
# report_json_filename: Report JSON filename (default: /tmp/enb_report.json)
# report_json_asn1_oct: Prints ASN1 messages encoded as an octet string instead of plain text in the JSON report
 ↪  file
# alarms_log_enable:    Enable Alarms logging (default: disabled)
# alarms_filename:      Alarms logging filename (default: /tmp/alarms.log)
```

```
# tracing_enable:       Write source code tracing information to a file
# tracing_filename:     File path to use for tracing information
# tracing_buffcapacity: Maximum capacity in bytes the tracing framework can store
# stdout_ts_enable:     Prints once per second the timestamp into stdout
# tx_amplitude:         Transmit amplitude factor (set 0-1 to reduce PAPR)
# rrc_inactivity_timer  Inactivity timeout used to remove UE context from RRC (in milliseconds)
# max_mac_dl_kos:       Maximum number of consecutive KOs in DL before triggering the UE's release (default: 100)
# max_mac_ul_kos:       Maximum number of consecutive KOs in UL before triggering the UE's release (default: 100)
# max_prach_offset_us:  Maximum allowed RACH offset (in us)
# nof_prealloc_ues:     Number of UE memory resources to preallocate during eNB initialization for faster UE
 ↪  creation (default: 8)
# rlf_release_timer_ms: Time taken by eNB to release UE context after it detects an RLF
# eea_pref_list:        Ordered preference list for the selection of encryption algorithm (EEA) (default: EEA0,
 ↪  EEA2, EEA1)
# eia_pref_list:        Ordered preference list for the selection of integrity algorithm (EIA) (default: EIA2,
 ↪  EIA1, EIA0)
# gtpu_tunnel_timeout:  Time that GTPU takes to release indirect forwarding tunnel since the last received GTPU
 ↪  PDU (0 for no timer)
# ts1_reloc_prep_timeout: S1AP TS 36.413 TS1RelocPrep Expiry Timeout value in milliseconds
# ts1_reloc_overall_timeout: S1AP TS 36.413 TS1RelocOverall Expiry Timeout value in milliseconds
# rlf_release_timer_ms: Time taken by eNB to release UE context after it detects a RLF
# rlf_min_ul_snr_estim: SNR threshold in dB below which the enb is notified with RLF ko
# s1_setup_max_retries: Maximum amount of retries to setup the S1AP connection. If this value is exceeded, an
 ↪  alarm is written to the log. -1 means infinity.
# s1_connect_timer:     Connection Retry Timer for S1 connection (seconds)
# rx_gain_offset:       RX Gain offset to add to rx_gain to calibrate RSRP readings
# use_cedron_f_est_alg: Whether to use Cedron algorithm for TA estimation or not (Default: false)
#########################################################################
[expert]
#pusch_max_its        = 8 # These are half iterations
#nr_pusch_max_its     = 10
#pusch_8bit_decoder   = false
#nof_phy_threads      = 3
#metrics_period_secs  = 1
#metrics_csv_enable   = false
#metrics_csv_filename = /tmp/enb_metrics.csv
#report_json_enable   = true
#report_json_filename = /tmp/enb_report.json
#report_json_asn1_oct = false
#alarms_log_enable    = true
#alarms_filename      = /tmp/enb_alarms.log
#tracing_enable       = true
#tracing_filename     = /tmp/enb_tracing.log
#tracing_buffcapacity = 1000000
#stdout_ts_enable     = false
#tx_amplitude         = 0.6
# Lab setting: srsENB releases the RRC context after 30 s without data by
# default (UE Context Release Request, then the UE re-attaches in a loop);
# 2 h keeps the session up for a whole experiment.
rrc_inactivity_timer = 7200000
#max_mac_dl_kos       = 100
#max_mac_ul_kos       = 100
#max_prach_offset_us  = 30
#nof_prealloc_ues     = 8
#rlf_release_timer_ms = 4000
#lcid_padding         = 3
#eea_pref_list = EEA0, EEA2, EEA1
#eia_pref_list = EIA2, EIA1, EIA0
#gtpu_tunnel_timeout = 0
#extended_cp         = false
#ts1_reloc_prep_timeout = 10000
#ts1_reloc_overall_timeout = 10000
#rlf_release_timer_ms = 4000
#rlf_min_ul_snr_estim = -2
#s1_setup_max_retries = -1
#s1_connect_timer = 10
```

```
#rx_gain_offset = 62
#mac_prach_bi         = 0
#use_cedron_f_est_alg = false
```

3.70 /opt/GSM/osmo-operator/configs/srsran/rb.conf

3539 octets, 190 lignes → 190 lignes

```
// rb.conf - srsENB radio bearer (SRB/DRB, QCI/5QI) configuration, libconfig
// syntax. Installed to /root/.config/srsran by tools/osmo-lte-install.sh.

// All times are in ms. Use -1 for infinity, where available

// 4G Section

// srb1_config = {
//   rlc_config = {
//     ul_am = {
//       t_poll_retx = 45;
//       poll_pdu = -1;
//       poll_byte = -1;
//       max_retx_thresh = 4;
//     };
//     dl_am = {
//       t_reordering = 35;
//       t_status_prohibit = 0;
//     };
//     enb_specific = {
//      dl_max_retx_thresh = 32;
//     };
//   };
// }

// srb2_config = {
//   rlc_config = {
//     ul_am = {
//       t_poll_retx = 45;
//       poll_pdu = -1;
//       poll_byte = -1;
//       max_retx_thresh = 4;
//     };
//     dl_am = {
//       t_reordering = 35;
//       t_status_prohibit = 0;
//     };
//     enb_specific = {
//      dl_max_retx_thresh = 32;
//     };
//   };
// }

qci_config = (
{
  qci = 7;
  pdcp_config = {
    discard_timer = -1;
    pdcp_sn_size = 12;
  }
  rlc_config = {
    ul_um = {
      sn_field_length = 10;
    };
    dl_um = {
      sn_field_length = 10;
```

```
      t_reordering    = 45;
    };
  };
  logical_channel_config = {
    priority = 13;
    prioritized_bit_rate   = -1;
    bucket_size_duration  = 100;
    log_chan_group = 2;
  };
  enb_specific = {
    dl_max_retx_thresh = 32;
  };
},
{
  qci = 9;
  pdcp_config = {
    discard_timer = 150;
    status_report_required = true;
  }
  rlc_config = {
    ul_am = {
      t_poll_retx = 120;
      poll_pdu = 64;
      poll_byte = 750;
      max_retx_thresh = 16;
    };
    dl_am = {
      t_reordering = 50;
      t_status_prohibit = 50;
    };
  };
  logical_channel_config = {
    priority = 11;
    prioritized_bit_rate   = -1;
    bucket_size_duration  = 100;
    log_chan_group = 3;
  };
  enb_specific = {
    dl_max_retx_thresh = 32;
  };
}
);

// 5G Section
srb1_5g_config = {
 rlc_config = {
   ul_am = {
     sn_field_len = 12;
     t_poll_retx = 45;
     poll_pdu = -1;
     poll_byte = -1;
     max_retx_thres = 8;
   };
   dl_am = {
     sn_field_len = 12;
     t_reassembly = 35;
     t_status_prohibit = 10;
   };
 };
}

srb2_5g_config = {
 rlc_config = {
   ul_am = {
     sn_field_len = 12;
     t_poll_retx = 45;
```

```
      poll_pdu = -1;
      poll_byte = -1;
      max_retx_thres = 8;
    };
    dl_am = {
      sn_field_len = 12;
      t_reassembly = 35;
      t_status_prohibit = 10;
    };
  };
}

five_qi_config = (
{
  five_qi = 7;
  pdcp_nr_config = {
    drb = {
      pdcp_sn_size_ul = 18;
      pdcp_sn_size_dl = 18;
      discard_timer = 50;
      integrity_protection = false;
      status_report = false;
    };
    t_reordering = 50;
  };
  rlc_config = {
    um_bi_dir = {
      ul_um = {
        sn_field_len = 12;
      };
      dl_um = {
        sn_field_len = 12;
        t_reassembly = 50;
      };
    };
  };
},
{
  five_qi = 9;
  pdcp_nr_config = {
    drb = {
      pdcp_sn_size_ul = 18;
      pdcp_sn_size_dl = 18;
      discard_timer = 50;
      integrity_protection = false;
      status_report = false;
    };
    t_reordering = 50;
  };
  rlc_config = {
    am = {
      ul_am = {
        sn_field_len = 12;
        t_poll_retx = 50;
        poll_pdu = 4;
        poll_byte = 3000;
        max_retx_thres = 4;
      };
      dl_am = {
        sn_field_len = 12;
        t_reassembly = 50;
        t_status_prohibit = 50;
      };
    };
  };
}
```

```
);
```

3.71 /opt/GSM/osmo-operator/configs/srsran/rr.conf

3189 octets, 121 lignes → 121 lignes

```
// rr.conf - srsENB radio resource configuration (MAC/PHY common settings and
// cell list). Installed to /root/.config/srsran by tools/osmo-lte-install.sh.

mac_cnfg =
{
  phr_cnfg =
  {
    dl_pathloss_change = "dB3"; // Valid: 1, 3, 6 or INFINITY
    periodic_phr_timer = 50;
    prohibit_phr_timer = 0;
  };
  ulsch_cnfg =
  {
    max_harq_tx = 4;
    periodic_bsr_timer = 20; // in ms
    retx_bsr_timer = 320;   // in ms
  };

  time_alignment_timer = -1; // -1 is infinity
};

phy_cnfg =
{
  phich_cnfg =
  {
    duration  = "Normal";
    resources = "1/6";
  };

  pusch_cnfg_ded =
  {
    beta_offset_ack_idx = 6;
    beta_offset_ri_idx  = 6;
    beta_offset_cqi_idx = 6;
  };

  // PUCCH-SR resources are scheduled on time-frequeny domain first, then multiplexed in the same resource.
  sched_request_cnfg =
  {
    dsr_trans_max = 64;
    period = 20;          // in ms
    //subframe = [1, 11]; // Optional vector of subframe indices allowed for SR transmissions (default uses all)
    nof_prb = 1;          // number of PRBs on each extreme used for SR (total prb is twice this number)
  };
  cqi_report_cnfg =
  {
    mode = "periodic";
    simultaneousAckCQI = true;
    period = 40;                   // in ms
    //subframe = [0, 10, 20, 30];  // Optional vector of subframe indices every period where CQI resources will be
 ↪  allocated (default uses all)
    m_ri = 8; // RI period in CQI period
    //subband_k = 1; // If enabled and > 0, configures sub-band CQI reporting and defines K (see 36.213 7.2.2). If
 ↪  disabled, configures wideband CQI
  };
};

cell_list =
```

```
(
  {
    // rf_port = 0;
    cell_id = 0x01;
    tac = 0x0007;
    pci = 1;
    // root_seq_idx = 204;
    dl_earfcn = 3350;
    //ul_earfcn = 21400;
    ho_active = false;
    //meas_gap_period = 0; // 0 (inactive), 40 or 80
    //meas_gap_offset_subframe = [6, 12, 18, 24, 30];
    // target_pusch_sinr = -1;
    // target_pucch_sinr = -1;
    // enable_phr_handling = false;
    // min_phr_thres = 0;
    // allowed_meas_bw = 6;
    // t304 = 2000; // in msec. possible values: 50, 100, 150, 200, 500, 1000, 2000
    // tx_gain = 20.0; // in dB. This gain is set by scaling the source signal.

    // CA cells
    scell_list = (
      // {cell_id = 0x02; cross_carrier_scheduling = false; scheduling_cell_id = 0x02; ul_allowed = true}
    )

    // Cells available for handover
    meas_cell_list =
    (
      {
        eci = 0x19C02;
        dl_earfcn = 2850;
        pci = 2;
        //direct_forward_path_available = false;
        //allowed_meas_bw = 6;
        //cell_individual_offset = 0;
      }
    );

    // Select measurement report configuration (all reports are combined with all measurement objects)
    meas_report_desc =
    (
        {
          eventA = 3
          a3_offset = 6;
          hysteresis = 0;
          time_to_trigger = 480;
          trigger_quant = "RSRP";
          max_report_cells = 1;
          report_interv = 120;
          report_amount = 1;
        }
    );
    meas_quant_desc = {
        // averaging filter coefficient
        rsrq_config = 4;
        rsrp_config = 4;
     };
  }
  // Add here more cells
);

nr_cell_list =
(
  // no NR cells
);
```

3.72 /opt/GSM/osmo-operator/configs/srsran/sib.conf

10850 octets, 308 lignes → 308 lignes

```
#####################################################################
# sib.conf - srsENB system information blocks (SIB1..SIB7). Installed to
# /root/.config/srsran by tools/osmo-lte-install.sh.
#####################################################################

#####################################################################
# sib1 configuration options (See TS 36.331)
#
# additional_plmns: A list of additional PLMN identities.
#     mcc: MCC
#     mnc: MNC
#     cell_reserved_for_oper:  One of "reserved" or "notReserved", default is "notReserved"
#
#####################################################################
sib1 =
{
    intra_freq_reselection = "Allowed";
    q_rx_lev_min = -65;
    //p_max = 3;
    cell_barred = "NotBarred"
    si_window_length = 20;
    sched_info =
    (
        {
            si_periodicity = 16;

           // comma-separated array of SIB-indexes (from 3 to 13), leave empty or commented to just scheduler sib2
            si_mapping_info = [ 3, 7 ];
        }
    );
    system_info_value_tag = 0;
};

sib2 =
{
    rr_config_common_sib =
    {
        rach_cnfg =
        {
            num_ra_preambles = 52;
            preamble_init_rx_target_pwr = -104;
            pwr_ramping_step = 6;  // in dB
            preamble_trans_max = 10;
            ra_resp_win_size = 10;  // in ms
            mac_con_res_timer = 64; // in ms
            max_harq_msg3_tx = 4;
        };
        bcch_cnfg =
        {
            modification_period_coeff = 16; // in ms
        };
        pcch_cnfg =
        {
            default_paging_cycle = 32; // in rf
            nB = "1";
        };
        prach_cnfg =
        {
            root_sequence_index = 128;
            prach_cnfg_info =
            {
                high_speed_flag = false;
```

```
            prach_config_index = 3;
            prach_freq_offset = 4;
            zero_correlation_zone_config = 5;
        };
    };
    pdsch_cnfg =
    {
        /* Warning: Currently disabled and forced to p_b=1 for TM2/3/4 and p_b=0 for TM1
         */
        p_b = 1;
        rs_power = 0;
    };
    pusch_cnfg =
    {
        n_sb = 1;
        hopping_mode = "inter-subframe";
        pusch_hopping_offset = 2;
        enable_64_qam = false; // 64QAM PUSCH is not currently enabled
        ul_rs =
        {
            cyclic_shift = 0;
            group_assignment_pusch = 0;
            group_hopping_enabled = false;
            sequence_hopping_enabled = false;
        };
    };
    pucch_cnfg =
    {
        delta_pucch_shift = 1;
        n_rb_cqi = 1;
        n_cs_an = 0;
        n1_pucch_an = 12;
    };
    ul_pwr_ctrl =
    {
        p0_nominal_pusch = -85;
        alpha = 0.7;
        p0_nominal_pucch = -107;
        delta_flist_pucch =
        {
            format_1  = 0;
            format_1b = 3;
            format_2  = 1;
            format_2a = 2;
            format_2b = 2;
        };
        delta_preamble_msg3 = 6;
    };
    ul_cp_length = "len1";
};

ue_timers_and_constants =
{
    t300 = 2000; // in ms
    t301 = 100;  // in ms
    t310 = 200; // in ms
    n310 = 1;
    t311 = 10000; // in ms
    n311 = 1;
};

freqInfo =
{
    ul_carrier_freq_present = true;
    ul_bw_present = true;
    additional_spectrum_emission = 1;
```

```
    };

    time_alignment_timer = "INFINITY"; // use "sf500", "sf750", etc.
};

sib3 =
{
    cell_reselection_common = {
        q_hyst = 2; // in dB
    },
    cell_reselection_serving = {
        s_non_intra_search = 3,
        thresh_serving_low = 2,
        cell_resel_prio = 6
    },
    intra_freq_reselection = {
        q_rx_lev_min = -61,
        p_max = 23,
        s_intra_search = 5,
        presence_ant_port_1 = true,
        neigh_cell_cnfg = 1,
        t_resel_eutra = 1
    }
};

#########################################################################
# sib5 configuration options (See TS 36.331)
# Contains information relevant for inter-frequency cell re-selection.
# Must be added to sib1::sched_info::si_mapping_info array parameter to be transmitted
#
# inter_freq_carrier_freq_list: A list of neighbouring inter-frequencies.
#     dl_carrier_freq: The EARFCN for the EUTRA carrier frequency.
#     q_rx_lev_min: Minimum received RSRP level in the E-UTRA cell, ([field_val] * 2) = [level in dBm].
#     p_max: Optional maximum allowed transmission power for the neighbouring E-UTRA cells on this carrier
 ↪  frequency.
#     t_resel_eutra: Cell reselection timer (seconds).
#     t_resel_eutra_sf: Optional speed dependent ScalingFactor for t_resel_eutra.
#         sf_medium: Scaling factor if the UE is in Medium Mobility state, one of "0.25", "0.5", "0.75" or "1.0".
#         sf_high: Scaling factor if the UE is in High Mobility state, one of "0.25", "0.5", "0.75" or "1.0".
#     thresh_x_high: Srclev threshold (dB) to select to a higher-priority RAT/Frequency.
#     thresh_x_low: Srclev threshold (dB) to select to a lower-priority RAT/Frequency.
#     allowed_meas_bw: Maximum allowed measurement bandwidth on a carrier frequency .
#     presence_ant_port_1: whether all the neighbouring cells use Antenna Port 1.
#     cell_resel_prio: Optional absolute priority of the carrier frequency group.
#     neigh_cell_cfg: Information related to MBSFN and TDD UL/DL configuration of neighbour cells.
#     q_offset_freq: Frequency specific offset for equal priority E-UTRAN frequencies.
#     inter_freq_neigh_cell_list: A List of inter-frequency neighbouring cells with specific cell re-selection
 ↪  parameters.
#         phys_cell_id: Physical layer identity of the cell.
#         q_offset_cell: Cell spcific offset.
#     inter_freq_black_cell_list: A List of blacklisted inter-frequency neighbouring cells.
#         start: The lowest physical cell identity in the range.
#         range: The number of physical cell identities in the range.
#
#########################################################################
sib5 =
{
    inter_freq_carrier_freq_list =
    (
        {
            dl_carrier_freq = 1450;
            q_rx_lev_min = -70;
            t_resel_eutra = 2;
            t_resel_eutra_sf = {
                sf_medium = "0.25";
                sf_high = "1.0";
```

```
            };
            thresh_x_high = 3;
            thresh_x_low = 2;
            allowed_meas_bw = 75;
            presence_ant_port_1 = True;
            cell_resel_prio = 4;
            neigh_cell_cfg = 2;
            q_offset_freq = -6;
            inter_freq_neigh_cell_list =
            (
                {
                    phys_cell_id = 500;
                    q_offset_cell = 2;
                }
            );
            inter_freq_black_cell_list =
            (
                {
                    start = 123;
                    range = 4;
                }
            );
        }
    );
};

######################################################################
# sib6 configuration options (See TS 36.331)
# Contains UTRA neighbor information for inter-rat handover.
# Must be added to sib1::sched_info::si_mapping_info array parameter to be transmitted
#
# t_resel_utra: Cell reselection timer (seconds)
# t_resel_utra_sf: Optional speed dependent ScalingFactor for t_resel_utra.
#     sf_medium: Scaling factor if the UE is in Medium Mobility state, one of "0.25", "0.5", "0.75" or "1.0".
#     sf_high: Scaling factor if the UE is in High Mobility state, one of "0.25", "0.5", "0.75" or "1.0".
# carrier_freq_list_utra_fdd / carrier_freq_list_utra_tdd: A list of carrier frequencies of UTRA FDD / TDD.
#     carrier_freq: The UARFCN for the UTRA carrier frequency.
#     cell_resel_prio: Optional absolute priority of the carrier frequency group.
#     thresh_x_high: Srclev threshold (dB) to select to a higher-priority RAT/Frequency.
#     thresh_x_low: Srclev threshold (dB) to select to a lower-priority RAT/Frequency.
#     q_rx_lev_min: Minimum receive level in UTRA cell, ([field_val] * 2) + 1 = [level in dBm].
#     p_max_utra: The maximum allowed transmission power on the (uplink) carrier frequency.
#     q_qual_min: Minimum required quality leve in UTRA cell, applicable only for FDD cells.
#
######################################################################
sib6 =
{
    t_resel_utra = 1;
    t_resel_utra_sf = {
        sf_medium = "0.25";
        sf_high = "1.0";
    }
    carrier_freq_list_utra_fdd =
    (
        {
            carrier_freq = 9613;
            cell_resel_prio = 6;
            thresh_x_high = 3;
            thresh_x_low = 2;
            q_rx_lev_min = -50;
            p_max_utra = 4;
            q_qual_min = -10;
        }
    );
    carrier_freq_list_utra_tdd =
    (
```

```
        {
            carrier_freq = 9505;
            thresh_x_high = 1;
            thresh_x_low = 2;
            q_rx_lev_min = -50;
            p_max_utra = -3;
        }
    );
};

########################################################################
# sib7 configuration options (See TS 36.331)
# Contains GERAN neighbor information for CSFB and inter-rat handover.
# Must be added to sib1::sched_info::si_mapping_info array parameter to be transmitted
#
# t_resel_geran: Cell reselection timer (seconds)
# t_resel_geran_sf: Optional speed dependent ScalingFactor for t_resel_geran.
#     sf_medium: Scaling factor if the UE is in Medium Mobility state, one of "0.25", "0.5", "0.75" or "1.0".
#     sf_high: Scaling factor if the UE is in High Mobility state, one of "0.25", "0.5", "0.75" or "1.0".
# carrier_freqs_info_list: A list of carrier frequency groups.
#     cell_resel_prio: Absolute priority of the carrier frequency group
#     ncc_permitted: 8-bit bitmap of NCC carriers permitted for monitoring
#     q_rx_lev_min: Minimum receive level in gsm cell, ([field_val] * 2) - 115 = [level in dBm]
#     thresh_x_high: Srclev threshold (dB) to select to a higher-priority RAT/Frequency
#     thresh_x_low: Srclev threshold (dB) to select to a lower-priority RAT/Frequency
#     start_arfcn: Initial search ARFCN value
#    band_ind: One of "dcs1800" or "pcs1900" Disambiguates ARFCNs in these bands, has no meaning for other ARFCNs.
#     explicit_list_of_arfcns: List of ARFCN numbers in the group
#
########################################################################
sib7 =
{
    t_resel_geran = 1;
    carrier_freqs_info_list =
    (
        {
            cell_resel_prio = 0;
            ncc_permitted = 255;
            q_rx_lev_min = 0;
            thresh_x_high = 2;
            thresh_x_low = 2;

            start_arfcn = 1;
            band_ind = "dcs1800";
            explicit_list_of_arfcns = (
                1
            );
        }
    );
};
```

3.73 /opt/GSM/osmo-operator/configs/srsran/ue.conf

19832 octets, 486 lignes → 486 lignes

```
########################################################################
#                   srsUE configuration file
########################################################################

########################################################################
# RF configuration
#
# freq_offset: Uplink and Downlink optional frequency offset (in Hz)
# tx_gain: Transmit gain (dB).
```

```
# rx_gain: Optional receive gain (dB). If disabled, AGC if enabled
# srate: Optional fixed sampling rate (Hz), corresponding to cell bandwidth. Must be set for 5G-SA.
#
# nof_antennas:       Number of antennas per carrier (all carriers have the same number of antennas)
# device_name:        Device driver family. Supported options: "auto" (uses first found), "UHD" or "bladeRF"
# device_args:        Arguments for the device driver. Options are "auto" or any string.
#                     Default for UHD: "recv_frame_size=9232,send_frame_size=9232"
#                     Default for bladeRF: ""
# device_args_2:      Arguments for the RF device driver 2.
# device_args_3:      Arguments for the RF device driver 3.
# time_adv_nsamples:  Transmission time advance (in number of samples) to compensate for RF delay
#                     from antenna to timestamp insertion.
#                     Default "auto". B210 USRP: 100 samples, bladeRF: 27.
# continuous_tx:      Transmit samples continuously to the radio or on bursts (auto/yes/no).
#                     Default is auto (yes for UHD, no for rest)
###################################################################
[rf]
freq_offset = 0
tx_gain = 80
rx_gain = 40
srate = 11.52e6

nof_antennas = 1

# For best performance in 2x2 MIMO and >= 15 MHz use the following device_args settings:
#     USRP B210: num_recv_frames=64,num_send_frames=64

# For best performance when BW<5 MHz (25 PRB), use the following device_args settings:
#     USRP B210: send_frame_size=512,recv_frame_size=512

#device_args = auto
time_adv_nsamples = auto
continuous_tx     = auto

# Example for ZMQ-based operation with TCP transport for I/Q samples
device_name = zmq
device_args = tx_port=tcp://*:2001,rx_port=tcp://localhost:2000,id=ue,base_srate=11.52e6

###################################################################
# EUTRA RAT configuration
#
# dl_earfcn:   Downlink EARFCN list.
#
# Optional parameters:
# dl_freq:            Override DL frequency corresponding to dl_earfcn
# ul_freq:            Override UL frequency corresponding to dl_earfcn
# nof_carriers:       Number of carriers
###################################################################
[rat.eutra]
dl_earfcn = 3350
nof_carriers = 1

###################################################################
# NR RAT configuration
#
# Optional parameters:
# bands:           List of support NR bands seperated by a comma (default 78)
# nof_carriers:    Number of NR carriers (must be at least 1 for NR support)
###################################################################
[rat.nr]
# bands = 78
# nof_carriers = 0

###################################################################
# Packet capture configuration
#
```

```
# Packet capture is supported at the MAC, MAC_NR, and NAS layer.
# MAC-layer packets are captured to file a the compact format decoded
# by the Wireshark. For decoding, use the UDP dissector and the UDP
# heuristic dissection. Edit the preferences (Edit > Preferences >
# Protocols > DLT_USER) for DLT_USER to add an entry for DLT=149 with
# Protocol=udp. Further, enable the heuristic dissection in UDP under:
# Analyze > Enabled Protocols > MAC-LTE > mac_lte_udp and MAC-NR > mac_nr_udp
# For more information see: https://wiki.wireshark.org/MAC-LTE
# Using the same filename for mac_filename and mac_nr_filename writes both
# MAC-LTE and MAC-NR to the same file allowing a better analysis.
# NAS-layer packets are dissected with DLT=148, and Protocol = nas-eps.
#
# enable:            Enable packet captures of layers (mac/mac_nr/nas/none) multiple option list
# mac_filename:      File path to use for MAC packet capture
# mac_nr_filename:   File path to use for MAC NR packet capture
# nas_filename:      File path to use for NAS packet capture
##########################################################################
[pcap]
enable = none
mac_filename = /tmp/ue_mac.pcap
mac_nr_filename = /tmp/ue_mac_nr.pcap
nas_filename = /tmp/ue_nas.pcap

##########################################################################
# Log configuration
#
# Log levels can be set for individual layers. "all_level" sets log
# level for all layers unless otherwise configured.
# Format: e.g. phy_level = info
#
# In the same way, packet hex dumps can be limited for each level.
# "all_hex_limit" sets the hex limit for all layers unless otherwise
# configured.
# Format: e.g. phy_hex_limit = 32
#
# Logging layers: rf, phy, mac, rlc, pdcp, rrc, nas, gw, usim, stack, all
# Logging levels: debug, info, warning, error, none
#
# filename: File path to use for log output. Can be set to stdout
#           to print logs to standard output
# file_max_size: Maximum file size (in kilobytes). When passed, multiple files are created.
#                If set to negative, a single log file will be created.
##########################################################################
[log]
all_level = warning
# NAS at info level without hex dumps: this is what the desktop srsUE console
# (tools/osmo-fft-snap.py) shows (attach, EMM, TAU, PDN). RRC stays at warning:
# at info it logs a cell measurement every 20 ms and drowns everything else.
nas_level = info
nas_hex_limit = 0
phy_lib_level = none
all_hex_limit = 32
# tools/osmo-lte.sh forces --log.filename=/tmp/osmo-lte-ue.log; this value only
# applies to an srsue started by hand.
filename = /tmp/osmo-lte-ue.log
file_max_size = -1

##########################################################################
# USIM configuration
#
# mode:   USIM mode (soft/pcsc)
# algo:   Authentication algorithm (xor/milenage)
# op/opc: 128-bit Operator Variant Algorithm Configuration Field (hex)
#         - Specify either op or opc (only used in milenage)
# k:      128-bit subscriber key (hex)
# imsi:   15 digit International Mobile Subscriber Identity
```

```
# imei:   15 digit International Mobile Station Equipment Identity
# pin:    PIN in case real SIM card is used
# reader: Specify card reader by it's name as listed by 'pcsc_scan'. If empty, try all available readers.
#####################################################################
[usim]
mode = soft
algo = milenage
opc  = 63BFA50EE6523365FF14C1F45F88737D
k    = 00112233445566778899aabbccdd0101
imsi = 001010001000001
imei = 358925004273675
#reader =
#pin  = 1234

#####################################################################
# RRC configuration
#
# ue_category:          Sets UE category (range 1-5). Default: 4
# release:              UE Release (8 to 15)
# feature_group:        Hex value of the featureGroupIndicators field in the
#                       UECapabilityInformation message. Default 0xe6041000
# mbms_service_id:      MBMS service id for autostarting MBMS reception
#                       (default -1 means disabled)
# mbms_service_port:    Port of the MBMS service
# nr_measurement_pci:   NR PCI for the simulated NR measurement. Default: 500
# nr_short_sn_support:  Announce PDCP short SN support. Default: true
#####################################################################
[rrc]
#ue_category       = 4
#release           = 8
#feature_group     = 0xe6041000
#mbms_service_id   = -1
#mbms_service_port = 4321

#####################################################################
# NAS configuration
#
# apn:               Set Access Point Name (APN)
# apn_protocol:      Set APN protocol (IPv4, IPv6 or IPv4v6.)
# user:              Username for CHAP authentication
# pass:              Password for CHAP authentication
# force_imsi_attach: Whether to always perform an IMSI attach
# eia:               List of integrity algorithms included in UE capabilities
#                      Supported: 1 - Snow3G, 2 - AES, 3 - ZUC
# eea:               List of ciphering algorithms included in UE capabilities
#                      Supported: 0 - NULL, 1 - Snow3G, 2 - AES, 3 - ZUC
#####################################################################
[nas]
apn = srsapn
#apn_protocol = ipv4
#user = srsuser
#pass = srspass
#force_imsi_attach = false
#eia = 1,2,3
#eea = 0,1,2,3

#####################################################################
# Slice configuration
#
# enable:           Enable a specific slice
# nssai-sst:        Specfic Slice Type
# nssai-sd:         Slice diffentiator
#####################################################################
[slicing]
#enable = false
#nssai-sst = 1
```

```
#nssai-sd = 1

#############################################################################
# GW configuration
#
# netns:                Network namespace to create TUN device. Default: empty
# ip_devname:           Name of the tun_srsue device. Default: tun_srsue
# ip_netmask:           Netmask of the tun_srsue device. Default: 255.255.255.0
#############################################################################
[gw]
#netns =
#ip_devname = tun_srsue
#ip_netmask = 255.255.255.0

#############################################################################
# GUI configuration
#
# Simple GUI displaying PDSCH constellation and channel freq response.
# (Requires building with srsGUI)
# enable:               Enable the graphical interface (true/false)
#############################################################################
[gui]
enable = false

#############################################################################
# Channel emulator options:
# enable:            Enable/Disable internal Downlink/Uplink channel emulator
#
# -- AWGN Generator
# awgn.enable:       Enable/disable AWGN generator
# awgn.snr:          SNR in dB
# awgn.signal_power: Received signal power in decibels full scale (dBfs)
#
# -- Fading emulator
# fading.enable:     Enable/disable fading simulator
# fading.model:      Fading model + maximum doppler (E.g. none, epa5, eva70, etu300, etc)
#
# -- Delay Emulator     delay(t) = delay_min + (delay_max - delay_min) * (1 + sin(2pi*t/period)) / 2
#                       Maximum speed [m/s]: (delay_max - delay_min) * pi * 300 / period
# delay.enable:      Enable/disable delay simulator
# delay.period_s:    Delay period in seconds.
# delay.init_time_s: Delay initial time in seconds.
# delay.maximum_us:  Maximum delay in microseconds
# delay.minumum_us:  Minimum delay in microseconds
#
# -- Radio-Link Failure (RLF) Emulator
# rlf.enable:        Enable/disable RLF simulator
# rlf.t_on_ms:       Time for On state of the channel (ms)
# rlf.t_off_ms:      Time for Off state of the channel (ms)
#
# -- High Speed Train Doppler model simulator
# hst.enable:        Enable/Disable HST simulator
# hst.period_s:      HST simulation period in seconds
# hst.fd_hz:         Doppler frequency in Hz
# hst.init_time_s:   Initial time in seconds
#############################################################################
[channel.dl]
#enable        = false

[channel.dl.awgn]
#enable        = false
#snr           = 30

[channel.dl.fading]
#enable        = false
#model         = none
```

```
[channel.dl.delay]
#enable        = false
#period_s      = 3600
#init_time_s   = 0
#maximum_us    = 100
#minimum_us    = 10

[channel.dl.rlf]
#enable        = false
#t_on_ms       = 10000
#t_off_ms      = 2000

[channel.dl.hst]
#enable        = false
#period_s      = 7.2
#fd_hz         = 750.0
#init_time_s   = 0.0

[channel.ul]
#enable        = false

[channel.ul.awgn]
#enable        = false
#n0            = -30

[channel.ul.fading]
#enable        = false
#model         = none

[channel.ul.delay]
#enable        = false
#period_s      = 3600
#init_time_s   = 0
#maximum_us    = 100
#minimum_us    = 10

[channel.ul.rlf]
#enable        = false
#t_on_ms       = 10000
#t_off_ms      = 2000

[channel.ul.hst]
#enable        = false
#period_s      = 7.2
#fd_hz         = -750.0
#init_time_s   = 0.0

##############################################################################
# PHY configuration options
#
# rx_gain_offset:       RX Gain offset to add to rx_gain to calibrate RSRP readings
# prach_gain:           PRACH gain (dB). If defined, forces a gain for the tranmsission of PRACH only.,
#                       Default is to use tx_gain in [rf] section.
# cqi_max:              Upper bound on the maximum CQI to be reported. Default 15.
# cqi_fixed:            Fixes the reported CQI to a constant value. Default disabled.
# snr_ema_coeff:        Sets the SNR exponential moving average coefficient (Default 0.1)
# snr_estim_alg:        Sets the noise estimation algorithm. (Default refs)
#                          Options: pss:   use difference between received and known pss signal,
#                                   refs:  use difference between noise references and noiseless (after filtering)
#                                   empty: use empty subcarriers in the boarder of pss/sss signal
# pdsch_max_its:        Maximum number of turbo decoder iterations (Default 4)
# pdsch_meas_evm:       Measure PDSCH EVM, increases CPU load (default false)
# nof_phy_threads:      Selects the number of PHY threads (maximum 4, minimum 1, default 3)
# equalizer_mode:       Selects equalizer mode. Valid modes are: "mmse", "zf" or any
#                       non-negative real number to indicate a regularized zf coefficient.
```

```
#                       Default is MMSE.
# correct_sync_error:   Channel estimator measures and pre-compensates time synchronization error. Increases CPU
 ↪  usage,
#                       improves PDSCH decoding in high SFO and high speed UE scenarios.
# sfo_ema:              EMA coefficient to average sample offsets used to compute SFO
# sfo_correct_period:   Period in ms to correct sample time to adjust for SFO
# sss_algorithm:        Selects the SSS estimation algorithm. Can choose between
#                       {full, partial, diff}.
# estimator_fil_auto:   The channel estimator smooths the channel estimate with an adaptative filter.
# estimator_fil_stddev: Sets the channel estimator smooth gaussian filter standard deviation.
# estimator_fil_order:  Sets the channel estimator smooth gaussian filter order (even values perform better).
#                       The taps are [w, 1-2w, w]
#
# snr_to_cqi_offset:    Sets an offset in the SNR to CQI table. This is used to adjust the reported CQI.
#
# interpolate_subframe_enabled: Interpolates in the time domain the channel estimates within 1 subframe. Default
 ↪  is to average.
#
# pdsch_csi_enabled:     Stores the Channel State Information and uses it for weightening the softbits. It is only
#                        used in TM1. It is True by default.
#
# pdsch_8bit_decoder:    Use 8-bit for LLR representation and turbo decoder trellis computation (Experimental)
# force_ul_amplitude:    Forces the peak amplitude in the PUCCH, PUSCH and SRS (set 0.0 to 1.0, set to 0 or
 ↪  negative for disabling)
#
# in_sync_rsrp_dbm_th:    RSRP threshold (in dBm) above which the UE considers to be in-sync
# in_sync_snr_db_th:      SNR threshold (in dB) above which the UE considers to be in-sync
# nof_in_sync_events:     Number of PHY in-sync events before sending an in-sync event to RRC
# nof_out_of_sync_events: Number of PHY out-sync events before sending an out-sync event to RRC
#
# force_N_id_2: Force using a specific PSS (set to -1 to allow all PSSs).
# force_N_id_1: Force using a specific SSS (set to -1 to allow all SSSs).
#
#####################################################################
[phy]
#rx_gain_offset      = 62
#prach_gain          = 30
#cqi_max             = 15
#cqi_fixed           = 10
#snr_ema_coeff       = 0.1
#snr_estim_alg       = refs
#pdsch_max_its       = 8    # These are half iterations
#pdsch_meas_evm      = false
#nof_phy_threads     = 3
#equalizer_mode      = mmse
#correct_sync_error  = false
#sfo_ema             = 0.1
#sfo_correct_period  = 10
#sss_algorithm       = full
#estimator_fil_auto  = false
#estimator_fil_stddev  = 1.0
#estimator_fil_order  = 4
#snr_to_cqi_offset   = 0.0
#interpolate_subframe_enabled = false
#pdsch_csi_enabled  = true
#pdsch_8bit_decoder = false
#force_ul_amplitude = 0
#detect_cp          = false

#in_sync_rsrp_dbm_th    = -130.0
#in_sync_snr_db_th      = 3.0
#nof_in_sync_events     = 10
#nof_out_of_sync_events = 20

#force_N_id_2           = 1
#force_N_id_1           = 10
```

```
###########################################################################
# PHY NR specific configuration options
#
# store_pdsch_ko:       Dumps the PDSCH baseband samples into a file on KO reception
#
###########################################################################
[phy.nr]
#store_pdsch_ko = false

###########################################################################
# CFR configuration options
#
# The CFR module provides crest factor reduction for the transmitted signal.
#
# enable:           Enable or disable the CFR. Default: disabled
#
# mode:             manual:   CFR threshold is set by cfr_manual_thres (default).
#                   auto_ema: CFR threshold is adaptive based on the signal PAPR. Power avg. with Exponential
 ↪  Moving Average.
#                             The time constant of the averaging can be tweaked with the ema_alpha parameter.
#                  auto_cma: CFR threshold is adaptive based on the signal PAPR. Power avg. with Cumulative Moving
 ↪  Average.
#                            Use with care, as CMA's increasingly slow response may be unsuitable for most use cases.
#
# strength:         Ratio between amplitude-limited vs unprocessed signal (0 to 1). Default: 1
# manual_thres:     Fixed manual clipping threshold for CFR manual mode. Default: 2
# auto_target_papr: Signal PAPR target (in dB) in CFR auto modes. output PAPR can be higher due to peak smoothing.
 ↪  Default: 7
# ema_alpha:        Alpha coefficient for the power average in auto_ema mode. Default: 1/7
#
###########################################################################
[cfr]
#enable           = false
#mode             = manual
#manual_thres     = 2.0
#strength         = 1.0
#auto_target_papr = 7.0
#ema_alpha        = 0.0143

###########################################################################
# Simulation configuration options
#
# The UE simulation supports turning on and off airplane mode in the UE.
# The actions are carried periodically until the UE is stopped.
#
# airplane_t_on_ms:   Time to leave airplane mode turned on (in ms)
#
# airplane_t_off_ms:  Time to leave airplane mode turned off (in ms)
#
###########################################################################
[sim]
#airplane_t_on_ms  = -1
#airplane_t_off_ms = -1

###########################################################################
# General configuration options
#
# metrics_csv_enable:    Write UE metrics to CSV file.
#
# metrics_period_secs:   Sets the period at which metrics are requested from the UE.
#
# metrics_csv_filename:  File path to use for CSV metrics.
#
# tracing_enable:        Write source code tracing information to a file.
#
```

```
# tracing_filename:      File path to use for tracing information.
#
# tracing_buffcapacity:  Maximum capacity in bytes the tracing framework can store.
#
# have_tti_time_stats:   Calculate TTI execution statistics using system clock
#
# metrics_json_enable:   Write UE metrics to JSON file.
#
# metrics_json_filename: File path to use for JSON metrics.
#
######################################################################
[general]
#metrics_csv_enable    = false
#metrics_period_secs   = 1
#metrics_csv_filename  = /tmp/ue_metrics.csv
#have_tti_time_stats   = true
#tracing_enable        = true
#tracing_filename      = /tmp/ue_tracing.log
#tracing_buffcapacity  = 1000000
#metrics_json_enable   = false
#metrics_json_filename = /tmp/ue_metrics.json
```

3.74 /opt/GSM/osmo-operator/data/c123-zones.json

1528 octets, 135 lignes → 113 lignes (22 groupes compactés)

```
{
  "_doc": "Zones cliquables de la photo du C123 (data/c123.jpg, 150x331), en coordonnees normalisees [x, y,
   ↪  largeur, hauteur] (0..1 de l image). Recalees a la main sur la photo le 21/10 (les touches etaient une
   ↪  demi-touche trop bas). `osmo-c123-gtk.py --calibrate` les reecrit a la souris.",
  "lcd": [
    0.⟨1⟩,  ×3
    ⟨⟩ = (16) (195) (67)
    0.215
  ],
  "lsb": [
    0.⟨1⟩,  ×3
    ⟨⟩ = (15) (495) (27)
    0.045
  ],
  "rsb": [
    0.⟨1⟩,  ×3
    ⟨⟩ = (58) (495) (27)
    0.045
  ],
  "up": [
    0.⟨1⟩,  ×3
    ⟨⟩ = (4) (52) (2)
    0.048
  ],
  "left": [
    0.⟨1⟩,  ×3
    ⟨⟩ = (335) (565) (09)
    0.06
  ],
  "menu": [
    0.⟨1⟩,  ×3
    ⟨⟩ = (435) (56) (13)
    0.07
  ],
  "right": [
    0.⟨1⟩,  ×3
    ⟨⟩ = (575) (565) (09)
    0.06
```

```
],
"down": [
  0.(1),  ×3
  () = (4) (618) (2)
  0.048
],
"ok": [
  0.(1),  ×3
  () = (115) (555) (175)
  0.09
],
"power": [
  0.(1),  ×3
  () = (705) (555) (175)
  0.09
],
"1": [
  0.(1),  ×3
  () = (11) (662) (23)
  0.052
],
"2": [
  0.(1),  ×3
  () = (36) (662) (27)
  0.052
],
"3": [
  0.(1),  ×3
  () = (66) (662) (23)
  0.052
],
"4": [
  0.(1),  ×3
  () = (11) (726) (23)
  0.052
],
"5": [
  0.(1),  ×3
  () = (36) (726) (27)
  0.052
],
"6": [
  0.(1),  ×3
  () = (66) (726) (23)
  0.052
],
"7": [
  0.(1),  ×3
  () = (11) (79) (23)
  0.052
],
"8": [
  0.(1),  ×3
  () = (36) (79) (27)
  0.052
],
"9": [
  0.(1),  ×3
  () = (66) (79) (23)
  0.052
],
"star": [
  0.(1),  ×3
  () = (11) (852) (23)
  0.052
],
```

```
  "0": [
    0.⟨1⟩,  ×3
    ⟨⟩ = (36) (852) (27)
    0.052
  ],
  "hash": [
    0.⟨1⟩,  ×3
    ⟨⟩ = (66) (852) (23)
    0.052
  ]
}
```

3.75 /opt/GSM/osmo-operator/data/c139-zones.json

1122 octets, 25 lignes → 19 lignes (1 groupes compactés)

```
{
  "_doc": "Zones cliquables de la photo du C139 (data/c139.jpg, 720x1561, recadree de « Motorola C139-1170.jpg »
  ↪  de Raimond Spekking, Wikimedia Commons, CC BY-SA 4.0), en coordonnees normalisees [x, y, largeur, hauteur].
  ↪  Posees le 21/10/2026 sur une grille ; `osmo-c123-gtk.py --modele c139 --calibrate` les reecrit a la souris.",
  "lcd": [0.15, 0.20, 0.69, 0.22],
  "lsb": [0.11, 0.50, 0.22, 0.045],
  "rsb": [0.67, 0.50, 0.22, 0.045],
  "up": [0.44, 0.535, 0.12, 0.04],
  "left": [0.34, 0.575, 0.08, 0.055],
  "menu": [0.43, 0.575, 0.14, 0.055],
  "right": [0.58, 0.575, 0.08, 0.055],
  "down": [0.44, 0.63, 0.12, 0.04],
  "ok": [0.12, 0.565, 0.18, 0.07],
  "power": [0.70, 0.565, 0.18, 0.07],
  "⟨1⟩": [0.⟨2⟩, 0.⟨3⟩, 0.⟨4⟩, 0.045],  ×9
    ⟨⟩ = (1,12,655,23) (2,40,69,20) (3,65,655,23) (4,12,72,23) (5,40,755,20) (6,65,72,23) (7,12,785,23)
         (8,40,82,20) (9,65,785,23)
  "star": [0.12, 0.85, 0.23, 0.045],
  "0": [0.40, 0.885, 0.20, 0.045],
  "hash": [0.65, 0.85, 0.23, 0.045]
}
```

3.76 /opt/GSM/osmo-operator/data/claude.svg

1075 octets, 23 lignes → 23 lignes

```
<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 256 256" width="256" height="256">
  <!-- Icone Claude : tuile clay (#D97757) + sunburst Anthropic creme -->
  <rect x="8" y="8" width="240" height="240" rx="56" fill="#D97757"/>
  <g transform="translate(128,128)" fill="#F5EFE6">
    <!-- rayon long et rayon court, alternes, pointe vers l exterieur -->
    <path id="rl" d="M0,-96 L7,-16 L-7,-16 Z"/>
    <path id="rs" d="M0,-74 L6,-14 L-6,-14 Z"/>
    <use href="#rl" transform="rotate(0)"/>
    <use href="#rs" transform="rotate(30)"/>
    <use href="#rl" transform="rotate(60)"/>
    <use href="#rs" transform="rotate(90)"/>
    <use href="#rl" transform="rotate(120)"/>
    <use href="#rs" transform="rotate(150)"/>
    <use href="#rl" transform="rotate(180)"/>
    <use href="#rs" transform="rotate(210)"/>
    <use href="#rl" transform="rotate(240)"/>
    <use href="#rs" transform="rotate(270)"/>
    <use href="#rl" transform="rotate(300)"/>
    <use href="#rs" transform="rotate(330)"/>
```

```
    <circle r="17"/>
  </g>
</svg>
```

3.77 /opt/GSM/osmo-operator/data/deka.svg

7511 octets, 96 lignes → 32 lignes (6 groupes compactés)

```
<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 256 256" width="256" height="256">
<rect x="8" y="8" width="240" height="240" rx="54" fill="#0a0d10"/>
<g fill="none" stroke="#9db8c9" stroke-width="2">
<circle cx="128" cy="128" r="9"/>
<circle cx="128" cy="128" r="15"/>
<circle cx="⟨1⟩.⟨2⟩" cy="⟨3⟩.⟨4⟩" r="1.6"/>  ×8
    ⟨⟩ = (143,0,128,0) (138,6,138,6) (128,0,143,0) (117,4,138,6) (113,0,128,0) (117,4,117,4) (128,0,113,0)
         (138,6,117,4)
<circle cx="⟨1⟩.0" cy="⟨2⟩.0" r="3"/>  ×12
    ⟨⟩ = (158,128) (154,143) (143,154) (128,158) (113,154) (102,143) (98,128) (102,113) (113,102) (128,98)
         (143,102) (154,113)
<rect x="⟨1⟩.⟨2⟩" y="⟨3⟩.⟨4⟩" width="5" height="5" rx="1" transform="rotate(⟨5⟩.⟨6⟩ ⟨7⟩.⟨8⟩ ⟨9⟩.⟨10⟩)"/>  ×16
    ⟨⟩ = (168,7,134,1,11,2,171,2,136,6) (162,1,149,9,33,8,164,6,152,4) (149,9,162,1,56,2,152,4,164,6)
         (134,1,168,7,78,8,136,6,171,2) (116,9,168,7,101,2,119,4,171,2) (101,1,162,1,123,7,103,6,164,6)
         (88,9,149,9,146,2,91,4,152,4) (82,3,134,1,168,8,84,8,136,6) (82,3,116,9,191,2,84,8,119,4)
         (88,9,101,1,213,8,91,4,103,6) (101,1,88,9,236,2,103,6,91,4) (116,9,82,3,258,8,119,4,84,8)
         (134,1,82,3,281,2,136,6,84,8) (149,9,88,9,303,8,152,4,91,4) (162,1,101,1,326,2,164,6,103,6)
         (168,7,116,9,348,8,171,2,119,4)
<circle cx="⟨1⟩.⟨2⟩" cy="⟨3⟩.⟨4⟩" r="⟨5⟩"/>  ×36
    ⟨⟩ = (186,0,128,0,3) (183,2,145,9,3) (174,9,162,1,3) (162,1,174,9,3) (145,9,183,2,3) (128,0,186,0,3)
         (110,1,183,2,3) (93,9,174,9,3) (81,1,162,1,3) (72,8,145,9,3) (70,0,128,0,3) (72,8,110,1,3)
         (81,1,93,9,3) (93,9,81,1,3) (110,1,72,8,3) (128,0,70,0,3) (145,9,72,8,3) (162,1,81,1,3) (174,9,93,9,3)
         (183,2,110,1,3) (200,6,142,4,4) (189,5,169,1,4) (169,1,189,5,4) (142,4,200,6,4) (113,6,200,6,4)
         (86,9,189,5,4) (66,5,169,1,4) (55,4,142,4,4) (55,4,113,6,4) (66,5,86,9,4) (86,9,66,5,4) (113,6,55,4,4)
         (142,4,55,4,4) (169,1,66,5,4) (189,5,86,9,4) (200,6,113,6,4)
<g transform="rotate(⟨1⟩.0 128 128)"><rect x="120.0" y="32.0" width="16.0" height="8" rx="1.5"/><line x1="122.0"
↪  y1="36.0" x2="134.0" y2="36.0"/><rect x="121.5" y="22.0" width="13.0" height="8" rx="1.5"/><line x1="123.5"
↪  y1="26.0" x2="132.5" y2="26.0"/><rect x="123.0" y="12.0" width="10.0" height="8" rx="1.5"/><line x1="125.0"
↪  y1="16.0" x2="131.0" y2="16.0"/></g>  ×8
    ⟨⟩ = (0) (45) (90) (135) (180) (225) (270) (315)
<g transform="rotate(⟨1⟩.5 128 128)"><line x1="128" y1="48" x2="128" y2="38"/></g>  ×8
    ⟨⟩ = (22) (67) (112) (157) (202) (247) (292) (337)
</g>
</svg>
```

3.78 /opt/GSM/osmo-operator/data/osmo-dino.svg

1144 octets, 25 lignes → 25 lignes

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64" width="64" height="64">
  <!-- osmo-dino.svg - le coureur, en pixels de 4 px sur une grille de 16.
       Gris #535353 : le meme sur un menu clair et sur un dock sombre, sans
       avoir a fournir deux variantes. Le cadre reste transparent. -->
  <g fill="#535353" shape-rendering="crispEdges">
    <!-- tete et museau -->
    <rect x="36" y="8"  width="20" height="16"/>
    <rect x="52" y="20" width="8"  height="4"/>
    <!-- cou -->
    <rect x="36" y="24" width="12" height="8"/>
    <!-- corps -->
    <rect x="16" y="28" width="32" height="16"/>
    <!-- queue -->
    <rect x="8"  y="24" width="12" height="8"/>
```

```
    <rect x="4"  y="28" width="8"  height="4"/>
    <!-- bras -->
    <rect x="32" y="36" width="8"  height="4"/>
    <!-- pattes : l'une posee, l'autre levee - il court -->
    <rect x="20" y="44" width="8"  height="12"/>
    <rect x="16" y="52" width="12" height="4"/>
    <rect x="36" y="44" width="8"  height="8"/>
  </g>
  <!-- l'oeil : un trou dans la tete, pas un point clair pose dessus -->
  <rect x="48" y="12" width="4" height="4" fill="#fff"/>
</svg>
```

3.79 /opt/GSM/osmo-operator/data/osmo-info.svg

288 octets, 1 lignes → 1 lignes

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 128 128"><circle cx="64" cy="64" r="58" fill="#d62828"
↪  stroke="#ff6b6b" stroke-width="4"/><text x="64" y="94" font-family="Ubuntu,DejaVu Sans,sans-serif"
↪  font-weight="700" font-size="92" text-anchor="middle" fill="#fff">?</text></svg>
```

3.80 /opt/GSM/osmo-operator/data/osmo-launch.svg

2521 octets, 51 lignes → 51 lignes

```
<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64" width="64" height="64">
<!-- ⚠ CE COMMENTAIRE EST *DANS* <svg>, ET IL DOIT Y RESTER.
     gdk-pixbuf ne RENIFLE le type d un fichier que sur ses ~256 premiers
     octets : il y cherche la balise <svg. Un commentaire d en-tete un peu
     long la repousse au-dela, et le chargeur repond alors
         Couldn't recognize the image file format
     sur un SVG parfaitement valide - `file` le reconnait, un parseur XML
     l accepte, et l icone se remplace quand meme par la page blanche
     generique sur le bureau. Constate le 31/08 sur les trois icones.
     Tout texte d explication va donc APRES la balise ouvrante. -->
<!-- osmo-launch.svg - icone du lanceur du banc osmo-operator.
     Combine dessine a la main plutot qu un nom d icone du theme : "call-start"
     est VERT dans Adwaita et n a pas de variante rouge, et une icone de theme
     absente se remplace en silence par un rectangle gris - le lanceur devient
     alors introuvable sur le bureau. Un SVG du depot ne peut pas manquer. -->
  <defs>
    <linearGradient id="corps" x1="0" y1="0" x2="0" y2="1">
      <stop offset="0%"   stop-color="#e8462f"/>
      <stop offset="100%" stop-color="#b3160b"/>
    </linearGradient>
    <linearGradient id="brillance" x1="0" y1="0" x2="0" y2="1">
      <stop offset="0%"   stop-color="#ffffff" stop-opacity="0.30"/>
      <stop offset="55%"  stop-color="#ffffff" stop-opacity="0.05"/>
      <stop offset="100%" stop-color="#ffffff" stop-opacity="0"/>
    </linearGradient>
  </defs>

  <!-- Pastille de fond -->
  <rect x="3" y="3" width="58" height="58" rx="14" ry="14" fill="url(#corps)"/>
  <rect x="3" y="3" width="58" height="58" rx="14" ry="14" fill="url(#brillance)"/>
  <rect x="3.75" y="3.75" width="56.5" height="56.5" rx="13.5" ry="13.5"
        fill="none" stroke="#7d0d05" stroke-width="1.5" opacity="0.55"/>

  <!-- Combine : ecouteur haut-gauche, micro bas-droite, cordon en arc.
       Trace en une seule forme pour rester net a 24 px comme a 256 px. -->
  <g transform="rotate(-32 32 32)">
    <path fill="#ffffff" d="
```

```
      M 20.2 14.0
      c 3.9 0 7.0 3.1 7.0 7.0
      c 0 2.6 -1.4 4.8 -3.4 6.1
      c -1.1 0.7 -1.6 2.0 -1.2 3.2
      c 1.6 4.9 5.4 8.7 10.3 10.3
      c 1.2 0.4 2.5 -0.1 3.2 -1.2
      c 1.3 -2.0 3.5 -3.4 6.1 -3.4
      c 3.9 0 7.0 3.1 7.0 7.0
      c 0 3.9 -3.1 7.0 -7.0 7.0
      C 27.4 50.0 14.0 36.6 14.0 21.0
      c 0 -3.9 3.1 -7.0 6.2 -7.0 z"/>
  </g>
</svg>
```

3.81 /opt/GSM/osmo-operator/data/osmo-lte.svg

755 octets, 11 lignes → 11 lignes

```
<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64" width="64" height="64">
<!-- osmo-lte : la 4G du banc. Commentaire court, APRES <svg (gdk-pixbuf ne
     renifle que les ~256 premiers octets ; voir osmo-launch.svg). -->
<rect x="4" y="4" width="56" height="56" rx="12" fill="#1e3a8a"/>
<rect x="12" y="38" width="6" height="12" rx="1.5" fill="#93c5fd"/>
<rect x="21" y="31" width="6" height="19" rx="1.5" fill="#bfdbfe"/>
<rect x="30" y="24" width="6" height="26" rx="1.5" fill="#dbeafe"/>
<rect x="39" y="17" width="6" height="33" rx="1.5" fill="#ffffff"/>
<text x="55" y="19" font-family="DejaVu Sans,Arial,sans-serif" font-size="13" font-weight="bold" fill="#fbbf24"
 ↪  text-anchor="end">4G</text>
</svg>
```

3.82 /opt/GSM/osmo-operator/data/osmo-multi.svg

2546 octets, 46 lignes → 46 lignes

```
<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64" width="64" height="64">
<!-- ⚠ CE COMMENTAIRE EST *DANS* <svg>, ET IL DOIT Y RESTER.
     gdk-pixbuf ne RENIFLE le type d un fichier que sur ses ~256 premiers
     octets : il y cherche la balise <svg. Un commentaire d en-tete un peu
     long la repousse au-dela, et le chargeur repond alors
         Couldn't recognize the image file format
     sur un SVG parfaitement valide - `file` le reconnait, un parseur XML
     l accepte, et l icone se remplace quand meme par la page blanche
     generique sur le bureau. Constate le 31/08 sur les trois icones.
     Tout texte d explication va donc APRES la balise ouvrante. -->
<!-- osmo-multi.svg - icone du lancement multi-operateur (start-multi.sh).
     Meme facture que osmo-launch.svg / osmo-tutorial.svg : pastille + glyphe
     blanc, pour que les trois se lisent comme une famille sur le bureau.
     Antenne bleue : c est le SS7 entre noeuds, pas un appel. -->
  <defs>
    <linearGradient id="corpsM" x1="0" y1="0" x2="0" y2="1">
      <stop offset="0%"   stop-color="#2f9fd6"/>
      <stop offset="100%" stop-color="#0d5f8f"/>
    </linearGradient>
    <linearGradient id="brillanceM" x1="0" y1="0" x2="0" y2="1">
      <stop offset="0%"   stop-color="#ffffff" stop-opacity="0.30"/>
      <stop offset="55%"  stop-color="#ffffff" stop-opacity="0.05"/>
      <stop offset="100%" stop-color="#ffffff" stop-opacity="0"/>
    </linearGradient>
  </defs>
```

```
  <rect x="3" y="3" width="58" height="58" rx="14" ry="14" fill="url(#corpsM)"/>
  <rect x="3" y="3" width="58" height="58" rx="14" ry="14" fill="url(#brillanceM)"/>
  <rect x="3.75" y="3.75" width="56.5" height="56.5" rx="13.5" ry="13.5"
        fill="none" stroke="#08405f" stroke-width="1.5" opacity="0.55"/>

  <g fill="none" stroke="#ffffff" stroke-linecap="round">
    <!-- Ondes, deux paires : l antenne rayonne des DEUX cotes, c est ce qui
         la distingue d un simple pylone et dit le lien entre noeuds. -->
    <path d="M 22.5 20.5 a 13 13 0 0 0 0 17"  stroke-width="3"/>
    <path d="M 41.5 20.5 a 13 13 0 0 1 0 17"  stroke-width="3"/>
    <path d="M 16.5 15.5 a 21 21 0 0 0 0 27"  stroke-width="3" opacity="0.55"/>
    <path d="M 47.5 15.5 a 21 21 0 0 1 0 27"  stroke-width="3" opacity="0.55"/>
  </g>

  <!-- Mat + embase -->
  <circle cx="32" cy="26" r="4.2" fill="#ffffff"/>
  <path fill="#ffffff" d="M 30.4 30.2 h 3.2 l 3.0 20.4 h -4.0 l -0.6 -5.6 h -0.0
                          l -0.6 5.6 h -4.0 z"/>
</svg>
```

3.83 /opt/GSM/osmo-operator/data/osmo-paint.svg

448 octets, 9 lignes → 9 lignes

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64" width="64" height="64">
  <rect x="5" y="5" width="54" height="54" rx="13" fill="#1b1d22"/>
  <path d="M13 47 C24 25, 31 53, 42 29" fill="none" stroke="#00d1ff"
        stroke-width="5" stroke-linecap="round"/>
  <g transform="rotate(45 45 43)">
    <rect x="40" y="18" width="10" height="24" rx="2" fill="#ff9500"/>
    <polygon points="40,42 50,42 45,53" fill="#ffd199"/>
  </g>
</svg>
```

3.84 /opt/GSM/osmo-operator/data/osmo-tutorial.svg

2545 octets, 56 lignes → 56 lignes

```
<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64" width="64" height="64">
<!-- ⚠ CE COMMENTAIRE EST *DANS* <svg>, ET IL DOIT Y RESTER.
     gdk-pixbuf ne RENIFLE le type d un fichier que sur ses ~256 premiers
     octets : il y cherche la balise <svg. Un commentaire d en-tete un peu
     long la repousse au-dela, et le chargeur repond alors
         Couldn't recognize the image file format
     sur un SVG parfaitement valide - `file` le reconnait, un parseur XML
     l accepte, et l icone se remplace quand meme par la page blanche
     generique sur le bureau. Constate le 31/08 sur les trois icones.
     Tout texte d explication va donc APRES la balise ouvrante. -->
<!-- osmo-tutorial.svg - icone du tutoriel. Meme facture que osmo-launch.svg
     (pastille + glyphe blanc) pour que les deux se lisent comme une paire sur
     le bureau ; teinte bleue pour qu on les distingue d un coup d oeil. -->
  <defs>
    <linearGradient id="corpsL" x1="0" y1="0" x2="0" y2="1">
      <stop offset="0%"   stop-color="#3f7fd0"/>
      <stop offset="100%" stop-color="#1b4788"/>
    </linearGradient>
    <linearGradient id="brillanceL" x1="0" y1="0" x2="0" y2="1">
      <stop offset="0%"   stop-color="#ffffff" stop-opacity="0.30"/>
      <stop offset="55%"  stop-color="#ffffff" stop-opacity="0.05"/>
      <stop offset="100%" stop-color="#ffffff" stop-opacity="0"/>
    </linearGradient>
```

```
  </defs>

  <rect x="3" y="3" width="58" height="58" rx="14" ry="14" fill="url(#corpsL)"/>
  <rect x="3" y="3" width="58" height="58" rx="14" ry="14" fill="url(#brillanceL)"/>
  <rect x="3.75" y="3.75" width="56.5" height="56.5" rx="13.5" ry="13.5"
        fill="none" stroke="#12305d" stroke-width="1.5" opacity="0.55"/>

  <!-- Livre ouvert : deux pages et la reliure centrale. -->
  <g>
    <path fill="#ffffff" d="
      M 31.0 20.4
      c -3.6 -2.6 -8.0 -3.9 -13.1 -3.9
      c -1.6 0 -3.1 0.1 -4.6 0.4
      c -0.8 0.1 -1.3 0.8 -1.3 1.6
      v 24.6
      c 0 1.0 0.9 1.8 1.9 1.6
      c 1.3 -0.2 2.7 -0.3 4.0 -0.3
      c 5.0 0 9.2 1.3 12.6 3.8
      c 0.2 0.2 0.4 0.2 0.5 0.2 z"/>
    <path fill="#ffffff" d="
      M 33.0 20.4
      c 3.6 -2.6 8.0 -3.9 13.1 -3.9
      c 1.6 0 3.1 0.1 4.6 0.4
      c 0.8 0.1 1.3 0.8 1.3 1.6
      v 24.6
      c 0 1.0 -0.9 1.8 -1.9 1.6
      c -1.3 -0.2 -2.7 -0.3 -4.0 -0.3
      c -5.0 0 -9.2 1.3 -12.6 3.8
      c -0.2 0.2 -0.4 0.2 -0.5 0.2 z"/>
    <rect x="31.1" y="20.0" width="1.8" height="28.6" rx="0.9" fill="#ffffff" opacity="0.55"/>
  </g>
</svg>
```

3.85 /opt/GSM/osmo-operator/data/desktop/claude.desktop

451 octets, 13 lignes → 13 lignes

```
# claude.desktop - launcher: Claude Code, the terminal AI assistant.
# Runs /usr/local/bin/osmo-claude-anim.
[Desktop Entry]
Type=Application
Name=Claude
Name[fr]=Claude
Comment=Claude Code - l assistant IA en terminal
Comment[fr]=Claude Code - l assistant IA en terminal
Exec=/usr/local/bin/osmo-claude-anim
Icon=/usr/share/osmo-operator/icons/claude.svg
Terminal=false
Categories=Development;Utility;
Keywords=claude;ia;ai;assistant;code;anthropic;
```

3.86 /opt/GSM/osmo-operator/data/desktop/osmo-addition.desktop

526 octets, 13 lignes → 13 lignes

```
# osmo-addition.desktop - launcher: optional add-ons (Docker container,
# SS7 multi-operator) installed by addition.sh. Runs /usr/local/bin/osmo-addition-anim.
[Desktop Entry]
Type=Application
Name=Supplements
Name[fr]=Supplements
Comment=Docker container and SS7 multioperator - installe via build.sh
```

```
Comment[fr]=Docker container and SS7 multioperator - installe via build.sh
Exec=/usr/local/bin/osmo-addition-anim
Icon=system-software-install
Terminal=false
Categories=System;
Keywords=docker;supplement;multi;ss7;addition;
```

3.87 /opt/GSM/osmo-operator/data/desktop/osmo-dino.desktop

478 octets, 13 lignes → 13 lignes

```
# osmo-dino.desktop - launcher: the dino game (Sun, Moon and computed shadows).
# Runs tools/osmo-dino.py --app.
[Desktop Entry]
Type=Application
Name=Dino (Coureur d'ombres)
Name[fr]=Dino (Coureur d'ombres)
Comment=Le jeu du dino : Soleil, Lune et ombres calculees
Comment[fr]=Le jeu du dino : Soleil, Lune et ombres calculees
Exec=/opt/GSM/osmo-operator/tools/osmo-dino.py --app
Icon=osmo-dino
Terminal=false
Categories=Game;ArcadeGame;
Keywords=dino;jeu;game;coureur;ombres;
```

3.88 /opt/GSM/osmo-operator/data/desktop/osmo-drivers.desktop

795 octets, 15 lignes → 15 lignes

```
# osmo-drivers.desktop - launcher: NVIDIA driver status / install / update
# (tools/osmo-drivers.sh in a bench GTK progress window; no terminal).
[Desktop Entry]
Type=Application
Name=Pilotes graphiques
Name[fr]=Pilotes graphiques
Comment=Pilote NVIDIA : etat, installation, mise a jour (ubuntu-drivers)
Comment[fr]=Pilote NVIDIA : etat, installation, mise a jour (ubuntu-drivers)
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --progress --root --title "Pilotes graphiques" --
 ↪  /opt/GSM/osmo-operator/tools/osmo-drivers.sh
Icon=preferences-desktop-display
# The whiptail menu runs in a VTE terminal inside a bench GTK window
# (osmo-gtk-run --tty): no launcher opens a system terminal.
Terminal=false
Categories=System;Settings;HardwareSettings;
Keywords=nvidia;pilote;driver;ubuntu-drivers;
```

3.89 /opt/GSM/osmo-operator/data/desktop/osmo-info.desktop

396 octets, 10 lignes → 10 lignes

```
# osmo-info.desktop - launcher: bench help page (every module, its icon,
# launcher bar entry and CLI). Runs /usr/local/bin/osmo-info.
[Desktop Entry]
Type=Application
Name=Help - how to run the bench
Comment=Every bench module: icon, launcher bar and CLI, with the code behind it
Exec=/usr/local/bin/osmo-info
Icon=/usr/share/osmo-operator/icons/osmo-info.svg
Terminal=false
Categories=Utility;
```

3.90 /opt/GSM/osmo-operator/data/desktop/osmo-install.desktop

478 octets, 13 lignes → 13 lignes

```
# osmo-install.desktop - launcher: install osmo-operator (the GSM/EGPRS bench)
# to disk from the live ISO. Runs /usr/local/bin/osmo-install (Calamares).
[Desktop Entry]
Type=Application
Name=Installer osmo-operator
Name[fr]=Installer osmo-operator
Comment=Installer le banc GSM/EGPRS sur le disque
Comment[fr]=Installer le banc GSM/EGPRS sur le disque
Exec=/usr/local/bin/osmo-install
Icon=drive-harddisk
Terminal=false
Categories=System;
Keywords=install;installer;calamares;
```

3.91 /opt/GSM/osmo-operator/data/desktop/osmo-launch.desktop

2536 octets, 60 lignes → 60 lignes

```
# osmo-launch.desktop - launcher: start the GSM bench (osmo-bts-trx +
# pont_dsp.py DSP chain, osmocom-bb mobile, Wireshark, Linphone, web console).
# Every action is one launch.sh command line; launch.sh passes "$@" to start.sh.
[Desktop Entry]
Type=Application
Name=Lancer le banc GSM
Name[fr]=Lancer le banc GSM
Comment=Demarre le banc, Wireshark, Linphone et la console web
Comment[fr]=Demarre le banc, Wireshark, Linphone et la console web
# Single click: the classic BTS (osmo-bts-trx <-> pont_dsp.py) and the osmocom-bb mobile.
Exec=/opt/GSM/osmo-operator/launch.sh --bts-classique --ms
Icon=/usr/share/osmo-operator/icons/osmo-launch.svg
Terminal=false
Categories=System;Network;
Keywords=gsm;osmocom;banc;lancer;

# ── RIGHT-CLICK ACTIONS ──────────────────────────────────────────────
# GNOME/DING reads `Actions=` and adds one context-menu entry per action
# (desktop AND dock). Group names must match the `Actions=` tokens EXACTLY,
# case included: an entry that matches nothing is silently ignored.
Actions=Service;Dashboard;Console;Vty;Grgsm;Stop;

# "As a service": systemctl, no terminal, the bench survives the end of the
# session (launch.sh --service-suivi talks to systemd).
[Desktop Action Service]
Name=Demarrer le banc en service
Name[fr]=Demarrer le banc en service (sans terminal)

Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --progress --root --title "Banc GSM 2G" --icon
 ↪  /usr/share/osmo-operator/icons/osmo-launch.svg -- /opt/GSM/osmo-operator/launch.sh --service-suivi
 ↪  --bts-classique --ms

[Desktop Action Dashboard]
Name=Tableau de bord (127.0.0.1:8080)
Name[fr]=Tableau de bord (127.0.0.1:8080)

Exec=/opt/GSM/osmo-operator/launch.sh --dashboard

# The only action that opens a terminal: a tmux session is interactive.
[Desktop Action Console]
Name=Console du banc (tmux attach)
```

```
Name[fr]=Console du banc (tmux attach)

Exec=/opt/GSM/osmo-operator/launch.sh --console

[Desktop Action Vty]
Name=Console du mobile (VTY 4247)
Name[fr]=Console du mobile (VTY 4247)

Exec=/opt/GSM/osmo-operator/launch.sh --vty

# The DSP (C54x) chain is the start.sh default (single click); this action
# starts the OTHER chain, gr-gsm.
[Desktop Action Grgsm]
Name=Lancer en mode gr-gsm (sans DSP)
Name[fr]=Lancer en mode gr-gsm (sans DSP)
Exec=/opt/GSM/osmo-operator/launch.sh --grgsm

[Desktop Action Stop]
Name=Arreter le banc
Name[fr]=Arreter le banc
Exec=/opt/GSM/osmo-operator/launch.sh --stop
```

3.92 /opt/GSM/osmo-operator/data/desktop/osmo-lte.desktop

2978 octets, 55 lignes → 55 lignes

```
# osmo-lte.desktop - launcher: the bench 4G (Open5GS core, srsRAN eNB and UE).
# Single click toggles the 4G through `osmo-lte toggle` (tools/osmo-lte.sh,
# service services/osmo-lte.service, root via pkexec). Everything runs in the
# bench progress window (osmo-gtk-run.py --progress): step bar, folded console,
# no shell. Right-click actions: start, start with srsGUI traces, traces only,
# stop, status, journal.
[Desktop Entry]
Type=Application
Name=4G du banc (srsRAN + Open5GS)
Name[fr]=4G du banc (srsRAN + Open5GS)
Comment=Un clic demarre la 4G (coeur Open5GS, eNB, UE), un second l arrete - osmo-lte toggle
Comment[fr]=Un clic demarre la 4G (coeur Open5GS, eNB, UE), un second l arrete - osmo-lte toggle
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --progress --title "4G du banc" --icon
 ↪  /usr/share/osmo-operator/icons/osmo-lte.svg -- /usr/local/bin/osmo-lte toggle
Icon=/usr/share/osmo-operator/icons/osmo-lte.svg
Terminal=false
Categories=System;Network;
Keywords=4g;lte;srsran;open5gs;epc;enb;ue;

Actions=Start;StartGui;Traces;Stop;Status;Journal;

[Desktop Action Start]
Name=Demarrer la 4G
Name[fr]=Demarrer la 4G
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --progress --title "4G du banc" --icon
 ↪  /usr/share/osmo-operator/icons/osmo-lte.svg -- /usr/local/bin/osmo-lte up

# 4G with its traces: srsRAN is built with srsGUI; `osmo-lte start-gui` adds
# --gui.enable on the eNB and the UE and gives them a DISPLAY (pkexec env
# DISPLAY=... XAUTHORITY=...). This is NOT the systemd service (a system unit
# has no display).
[Desktop Action StartGui]
Name=Demarrer la 4G avec les traces (srsGUI)
Name[fr]=Demarrer la 4G avec les traces (srsGUI)
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --progress --title "4G du banc (srsGUI)" --
 ↪  /usr/local/bin/osmo-lte start-gui

# Traces only: srsGUI lives in srsenb/srsue (--gui.enable), so the radio is
```

```
# restarted, but the Open5GS core is kept.
[Desktop Action Traces]
Name=Traces srsGUI seules (relance eNB et UE, garde le coeur)
Name[fr]=Traces srsGUI seules (relance eNB et UE, garde le coeur)
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --progress --title "Traces srsGUI" -- /usr/local/bin/osmo-lte
 ↪  traces

[Desktop Action Stop]
Name=Arreter la 4G
Name[fr]=Arreter la 4G
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --progress --close-on-exit --title "Arret de la 4G" --icon
 ↪  /usr/share/osmo-operator/icons/osmo-lte.svg -- /usr/local/bin/osmo-lte down

[Desktop Action Status]
Name=Etat (osmo-lte status, osmo-epc status)
Name[fr]=Etat (osmo-lte status, osmo-epc status)
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --root --keep-open --title "Etat 4G" -- bash -c
 ↪  "/usr/local/bin/osmo-lte status; /usr/local/bin/osmo-epc status"

[Desktop Action Journal]
Name=Voir le journal (journalctl -u osmo-lte)
Name[fr]=Voir le journal (journalctl -u osmo-lte)
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --root --title "journal osmo-lte" -- journalctl -u osmo-lte -f
 ↪  -n 200 --no-pager -o cat
```

3.93 /opt/GSM/osmo-operator/data/desktop/osmo-multi.desktop

1855 octets, 35 lignes → 35 lignes

```
# osmo-multi.desktop - launcher: the multi-operator bench (SS7 between the
# native operator, two docker operators and the inter-STP hub).
# Single click runs `launch.sh --multi-suivi` in the bench progress window:
# launch.sh checks the ConditionPathExists of osmo-multi.service itself
# (/etc/osmocom/osmo-multi.conf, written by addition.sh), names what is
# missing, starts the unit and reports by notification; the unit's journal
# scrolls in the window. The bench runs as a systemd unit (osmo-multi.service,
# like osmo-banc.service for the mono-operator): one journal, one status, a
# clean stop. Its ExecStop only stops the containers and the hub; the native
# operator stays with osmo-banc.
[Desktop Entry]
Type=Application
Name=multi-operator
Name[fr]=multi-operator
Comment=SS7 entre l operateur natif, deux operateurs docker et l inter-STP
Comment[fr]=SS7 entre l operateur natif, deux operateurs docker et l inter-STP
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --progress --root --title "Multi-operateur" --icon
 ↪  /usr/share/osmo-operator/icons/osmo-multi.svg -- /opt/GSM/osmo-operator/launch.sh --multi-suivi
Icon=/usr/share/osmo-operator/icons/osmo-multi.svg
Terminal=false
Categories=System;Network;
Keywords=ss7;m3ua;multi;operateur;interstp;

Actions=Stop;Journal;

[Desktop Action Stop]
Name=Arreter le multi-operateur
Name[fr]=Arreter le multi-operateur
Exec=/opt/GSM/osmo-operator/launch.sh --multi-stop

# Startup can take minutes (images, containers, HLR): this action follows the
# unit's journal in a bench GTK window (tools/osmo-gtk-run.py), not a terminal.
[Desktop Action Journal]
Name=Voir le journal (journalctl -u osmo-multi)
```

```
Name[fr]=Voir le journal (journalctl -u osmo-multi)
Exec=/opt/GSM/osmo-operator/tools/osmo-gtk-run.py --root --title "journal osmo-multi" -- journalctl -u osmo-multi
 ↪  -f -n 200 --no-pager -o cat
```

3.94 /opt/GSM/osmo-operator/data/desktop/osmo-paint.desktop

529 octets, 13 lignes → 13 lignes

```
# osmo-paint.desktop - launcher: on-screen drawing overlay (pen, shapes,
# eraser) over the desktop. Runs tools/overlay-draw.py.
[Desktop Entry]
Type=Application
Name=Peinture (overlay)
Name[fr]=Peinture (overlay)
Comment=Dessin au premier plan par-dessus le bureau : feutre, formes, gomme
Comment[fr]=Dessin au premier plan par-dessus le bureau : feutre, formes, gomme
Exec=/opt/GSM/osmo-operator/tools/overlay-draw.py
Icon=osmo-paint
Terminal=false
Categories=Graphics;Utility;
Keywords=peinture;dessin;overlay;feutre;annoter;
```

3.95 /opt/GSM/osmo-operator/data/desktop/osmo-tutorial.desktop

500 octets, 13 lignes → 13 lignes

```
# osmo-tutorial.desktop - launcher: five-minute quick-start tutorial of the
# bench. Runs /usr/local/bin/osmo-tutorial.
[Desktop Entry]
Type=Application
Name=Tutoriel - demarrage rapide
Name[fr]=Tutoriel - demarrage rapide
Comment=Prise en main du banc en cinq minutes
Comment[fr]=Prise en main du banc en cinq minutes
Exec=/usr/local/bin/osmo-tutorial
Icon=/usr/share/osmo-operator/icons/osmo-tutorial.svg
Terminal=false
Categories=System;Documentation;
Keywords=tutoriel;aide;demarrage;quickstart;
```

3.96 /opt/GSM/osmo-operator/data/desktop/osmo-update.desktop

445 octets, 13 lignes → 13 lignes

```
# osmo-update.desktop - launcher: replay update.sh of the osmo-operator
# repository. Runs /usr/local/bin/osmo-update-anim.
[Desktop Entry]
Type=Application
Name=Update osmo-operator
Name[fr]=Mise a jour osmo-operator
Comment=Rejouer update.sh du depot osmo-operator
Comment[fr]=Rejouer update.sh du depot osmo-operator
Exec=/usr/local/bin/osmo-update-anim
Icon=system-software-update
Terminal=false
Categories=System;
Keywords=update;sms;osmo;
```

3.97 /opt/GSM/osmo-operator/environment/load.env

985 octets, 24 lignes → 24 lignes

```
# ============================================================================
#  environment/load.env - configuration loader
# ============================================================================
#  Single entry point. Sourced by startup_scripts/04-environnement.sh with
#  `set -a`, so everything is exported.
#
#  ORDER matters. Every file uses `:=` ("unless already set"), so THE FIRST
#  ONE TO SET A VALUE WINS:
#      1. the command line       VAR=x ./start.sh     <- always wins
#      2. modes.env              the chosen profile
#      3. paths.env              where the dependencies are
#      4. the per-domain files   coeur / radio / reseau / debug (optional)
# ----------------------------------------------------------------------------

_ENVDIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

. "$_ENVDIR/modes.env"
. "$_ENVDIR/paths.env"

for _f in coeur radio reseau debug; do
    [ -f "$_ENVDIR/$_f.env" ] && . "$_ENVDIR/$_f.env"
done

unset _f _ENVDIR
```

3.98 /opt/GSM/osmo-operator/environment/modes.env

1545 octets, 43 lignes → 43 lignes

```
# ============================================================================
#  environment/modes.env - ready-made profiles
# ============================================================================
#  The start-native.sh modes, as profiles of the engine (run.sh). Pick ONE:
#      CALYPSO_PROFILE=faketrx ./start.sh
#  or, through the legacy interface:
#      MODE=faketrx ./start-native.sh
# ----------------------------------------------------------------------------

# MODE (legacy) has priority when set; otherwise CALYPSO_PROFILE.
: "${CALYPSO_PROFILE:=${MODE:-core}}"

case "$CALYPSO_PROFILE" in

  # Core only: STP + HLR + MSC + MGW + BSC (+ Asterisk). No radio.
  core|noproc)      CALYPSO_PROFILE=core ;;

  # Core + osmo-bts + fake_trx + trxcon + mobile. No QEMU.
  faketrx)          CALYPSO_PROFILE=faketrx ;;

  # Core + osmo-bts-virtual + virtphy + mobile.
  virtphy)          CALYPSO_PROFILE=virtphy ;;

  # Core in no-process mode + the Calypso emulation (delegated to OQC_ROOT).
  qemu|calypso)     CALYPSO_PROFILE=calypso ;;

  # Core + live fake_trx + attached Calypso.
  faketrx-qemu|hybrid) CALYPSO_PROFILE=hybrid ;;

  # Physical SDR.
```

```
  hw)               CALYPSO_PROFILE=hw ;;

  # SS7 multi-operator through ip netns.
  virtual)          CALYPSO_PROFILE=virtual ;;

  *)
      echo "[environment/modes.env] profil inconnu : '$CALYPSO_PROFILE'" >&2
      echo "   core | faketrx | virtphy | calypso | hybrid | hw | virtual" >&2
      ;;
esac

# Number of operators ("virtual" profile only).
: "${N_OPERATORS:=1}"
```

3.99 /opt/GSM/osmo-operator/environment/paths.env

5680 octets, 112 lignes → 112 lignes

```
# ============================================================================
#  environment/paths.env - where the dependencies are (INSTALLATION settings)
# ============================================================================
#  Answers one question: WHERE things are on THIS machine.
#  Every value can be overridden from the environment (`:=` idiom):
#      GSM_ROOT=/home/me/gsm ./start.sh
#  Check:   ./start.sh --check-paths
# ----------------------------------------------------------------------------

# --- Installation root ------------------------------------------------------
: "${GSM_ROOT:=/opt/GSM}"

# --- This repository --------------------------------------------------------
: "${NITB_TREE:=$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)}"
: "${NITB_SCRIPTS:=$NITB_TREE/scripts}"
: "${NITB_CONFIGS:=$NITB_TREE/configs}"
: "${NITB_HELPERS:=$NITB_TREE/helpers}"

# --- The Calypso emulation (neighbour repository) ---------------------------
# The "qemu" mode delegates to it. A single QEMU tree, /opt/GSM/qosmo (run.sh,
# run_modules, cfgs, environnement), and two layer-1 chains:
#
#   grgsm_exe    DEFAULT. gr-gsm layer 1 inside QEMU (qosmo --enable-l1-grgsm)
#                and the grgsm_exe bridge: the mobile camps, does its LU,
#                authenticates (COMP128v1), ciphers (A5/1), exchanges SMS and
#                holds a voice call.
#   c54x_exe     The TMS320C54x DSP runs OUTSIDE QEMU (/opt/GSM/c54x_exe) on
#                the real TI mask ROM (ROM: /opt/GSM/calypso_dsp.*.bin, copied
#                into c54x_exe/rom); QEMU runs with CALYPSO_DSP_EXTERN=1.
#
# Select:  ./start-native.sh --dsp        (c54x_exe; --grgsm for the default)
# Or point directly at a tree:  OQC_ROOT=/path/to/qosmo ./start-native.sh
: "${CALYPSO_FORK:=qosmo}"

# Looked up next to this repository first, then at the installation location.
if [ -x "$NITB_TREE/../$CALYPSO_FORK/run.sh" ]; then
    : "${OQC_ROOT:=$(cd "$NITB_TREE/../$CALYPSO_FORK" && pwd)}"
else
    : "${OQC_ROOT:=$GSM_ROOT/$CALYPSO_FORK}"
fi

# --- Osmocom binaries -------------------------------------------------------
# Bare names = looked up in PATH, which is the case of a package installation.
: "${OSMO_STP:=osmo-stp}"
: "${OSMO_HLR:=osmo-hlr}"
: "${OSMO_MSC:=osmo-msc}"
```

```
: "${OSMO_MGW:=osmo-mgw}"
: "${OSMO_BSC:=osmo-bsc}"
: "${OSMO_SGSN:=osmo-sgsn}"
: "${OSMO_GGSN:=osmo-ggsn}"
: "${OSMO_PCU:=osmo-pcu}"
: "${OSMO_BTS_TRX:=osmo-bts-trx}"
: "${OSMO_BTS_VIRTUAL:=osmo-bts-virtual}"
: "${OSMO_TRX_IPC:=osmo-trx-ipc}"
: "${ASTERISK_BIN:=asterisk}"

# --- osmocom-bb (mobile, trxcon, virtphy, fake_trx) ---------------------------
: "${OSMOCOM_BB:=$GSM_ROOT/osmocom-bb}"
: "${MOBILE_BIN:=$OSMOCOM_BB/src/host/layer23/src/mobile/mobile}"
: "${TRXCON_BIN:=$OSMOCOM_BB/src/host/trxcon/trxcon}"
: "${VIRTPHY_BIN:=$OSMOCOM_BB/src/host/virt_phy/src/virtphy}"
: "${FAKE_TRX:=$OSMOCOM_BB/src/target/trx_toolkit/fake_trx.py}"

# --- Osmocom configuration ----------------------------------------------------
: "${OSMOCOM_CFG:=/etc/osmocom}"
: "${OSMOCOM_HOME:=$HOME/.osmocom}"

# --- Outputs ------------------------------------------------------------------
# RUN_DIR holds the monitor socket and the *.pid files: it is the bench's only
# memory of what already runs, so every way of launching MUST resolve the same
# directory. It is derived from the REAL uid (which survives sudo), not from
# XDG_RUNTIME_DIR alone (sudo's env_reset clears it, which would give two
# bench identities and stacked duplicate processes fighting over the global
# resources: /tmp/osmocom_l2, UDP 5720/6720, gdb 1234). Falls back to /tmp
# only when /run/user/<uid> does not exist. A RUN_DIR set by the caller wins.
if [ -z "${RUN_DIR:-}" ]; then
    _oq_rt="${XDG_RUNTIME_DIR:-}"
    { [ -n "$_oq_rt" ] && [ -d "$_oq_rt" ]; } || _oq_rt="/run/user/$(id -u)"
    [ -d "$_oq_rt" ] || _oq_rt="/tmp"
    RUN_DIR="${_oq_rt}/osmo-nitb"
    unset _oq_rt
fi

# Logs stay on the tmpfs: tmux follows them live (tail -F) and the tmpfs is the
# only filesystem here where inotify works (on the overlay root, tail falls
# back to 1 s polling and the tmux panes stutter). If volume becomes a problem,
# reduce the SOURCE instead of moving the target:
#   CALYPSO_INTM_TRANS=0   removes ~56 % of the qemu.log flow
#   CALYPSO_MAILBOX=0      removes mailbox.log (~230 kB/s, ~800 MB/h)
# Overridable like everything else: LOG_DIR=/elsewhere ./start-native.sh
: "${LOG_DIR:=$RUN_DIR/logs}"

# --- Naming bridge with the Calypso repository --------------------------------
# Modules shared between the two repositories call the Calypso emulation
# QEMU_TREE (its name seen from inside). Here that repository is the NEIGHBOUR,
# OQC_ROOT. Both names are exposed so the same modules work on both sides.
: "${QEMU_TREE:=$OQC_ROOT}"
: "${QEMU_BIN:=$OQC_ROOT/build/qemu-system-arm}"
# C launcher of the fork (qosmo/tools/qosmo-launch -> /usr/local/bin/<fork>).
# 40-qemu.sh calls it instead of qemu-system-arm and falls back to QEMU_BIN
# when it is absent. See start-native.sh "Lanceur QEMU" and `qosmo --help`.
: "${QOSMO_LAUNCHER:=/usr/local/bin/$CALYPSO_FORK}"

# --- Calypso emulation paths (firmware, DSP ROM, osmocon...) ------------------
# The "qemu" mode modules are shared with the neighbour repository and expect
# FIRMWARE_ELF, DSP_PROM0..3, OSMOCON, QEMU_CFGS... Those paths are described
# by the neighbour's own paths.env (optional): it is sourced rather than
# duplicated here. LAST on purpose: everything above has already set its
# value and `:=` lets the first one win, so RUN_DIR/LOG_DIR stay ours.
if [ -r "$OQC_ROOT/environnement/paths.env" ]; then
    . "$OQC_ROOT/environnement/paths.env"
fi
```

3.100 /opt/GSM/osmo-operator/installer/calamares/settings.conf

4547 octets, 123 lignes → 123 lignes

```
# ============================================================================
#  settings.conf - Calamares, the installer of the osmo-operator ISO
# ============================================================================
#  Calamares rather than Ubiquity: Ubiquity reads the live system state
#  through casper, while this ISO boots with Debian's live-boot (build-iso.sh
#  installs live-boot and live-boot-initramfs-tools, not casper), so Ubiquity
#  would find neither the squashfs nor the medium mount point. Calamares is
#  simply TOLD where the squashfs is (modules/unpackfs.conf).
#
#  The MODULE ORDER MATTERS. "exec" runs top to bottom and each step assumes
#  the previous one: partition creates the filesystems, mount mounts them,
#  unpackfs extracts the squashfs into them, and everything after (machineid,
#  fstab, locale, users, grub) writes INTO that mounted target. A module
#  placed before unpackfs writes into an empty directory that is then
#  overwritten.
# ----------------------------------------------------------------------------
---
modules-search: [ local, /usr/lib/x86_64-linux-gnu/calamares/modules ]

# A named INSTANCE of the shellprocess module: it carries our own guarantees
# (root by default, osmocom second) and runs after grub, inside the target.
instances:
- id:     osmo
  module: shellprocess
  config: shellprocess-osmo.conf
# NVIDIA driver: the choice page, then the conditional installation.
- id:     nvidia
  module: packagechooser
  config: packagechooser-nvidia.conf
- id:     nvidia
  module: contextualprocess
  config: contextualprocess-nvidia.conf
# Ubuntu mirror of the installed system: the choice page (measured by
# osmo-install at startup, fastest one as default), then the write.
- id:     mirror
  module: packagechooser
  config: packagechooser-mirror.conf
- id:     mirror
  module: contextualprocess
  config: contextualprocess-mirror.conf
# Installs grub2-common into the target BEFORE bootloader, otherwise
# grub-install does not exist there and the module fails with "error code 127".
# The .deb packages arrive through the /run/osmo-debs bind of mount.conf.
- id:     grub
  module: shellprocess
  config: shellprocess-grub.conf
# addition.sh supplements: check boxes, then the origin of the operator image;
# the osmosupplements job writes them into the target for the first boot.
- id:     supplements
  module: packagechooser
  config: packagechooser-supplements.conf
- id:     image
  module: packagechooser
  config: packagechooser-image.conf

sequence:
- show:
  - welcome
  - locale
  - keyboard
  - partition
  - users
```

```
  - packagechooser@nvidia
  - packagechooser@mirror
  - packagechooser@supplements
  - packagechooser@image
  - summary
- exec:
  - partition
  - mount
  - unpackfs
  # The chosen mirror, BEFORE anything that could run apt in the target.
  - contextualprocess@mirror
  - machineid
  - fstab
  - locale
  - keyboard
  - localecfg
  # LUKS (see partition.conf): only /home is encrypted, the root carries
  # noEncrypt. luksbootkeyfile and initramfscfg therefore find no encrypted
  # root and do nothing; initramfs regenerates the initrd, which
  # shellprocess@osmo does again anyway. They stay for a MANUAL partitioning
  # where the operator encrypts the root himself. The crypttab line of /home
  # is handled by crypthome-postinstall (shellprocess@osmo): "noauto",
  # unlock-home asks for the passphrase.
  - luksbootkeyfile
  - initramfscfg
  - initramfs
  - users
  - displaymanager
  - networkcfg
  - hwclock
  - services-systemd
  - shellprocess@grub
  - grubcfg
  - bootloader
  - shellprocess@osmo
  - osmosupplements
  - contextualprocess@nvidia
  - umount
- show:
  - finished

branding: osmo

# Ask for confirmation before writing: this installer runs on a bench, next
# to disks that hold captures and HLR databases.
prompt-install: true
dont-chroot: false
oem-setup: false
disable-cancel: false
disable-cancel-during-exec: true

# These two keys are REQUIRED by calamares: without them it logs
# "Required settings.conf key ... is missing" at every start and falls back
# to an unwritten default.
#   quit-at-end: false                -> the "finished" screen stays, the
#                                        operator reads the result before closing.
#   hide-back-and-next-during-exec    -> during the copy (exec), Back/Next
#                                        are hidden: nothing to navigate
#                                        while the installation runs.
quit-at-end: false
hide-back-and-next-during-exec: true
```

3.101 /opt/GSM/osmo-operator/installer/calamares/branding/osmo/branding.desc

1637 octets, 41 lignes → 41 lignes

```
# branding.desc - Calamares branding of the osmo-operator ISO (names, images, slideshow, colours).
---
componentName: osmo

# The empty strings are INTENTIONAL: they remove the links (support, release
# notes) that the welcome screen would otherwise show, pointing to pages that
# do not exist for this project.
strings:
    productName:         osmo-operator
    shortProductName:    osmo-operator
    version:             0.1-9
    shortVersion:        0.1-9
    versionedName:       osmo-operator v0.1-9 - banc GSM/EGPRS
    shortVersionedName:  osmo-operator v0.1-9
    bootloaderEntryName: osmo-operator
    productUrl:          ""
    supportUrl:          ""
    knownIssuesUrl:      ""
    releaseNotesUrl:     ""

images:
    productLogo:   "logo.png"
    productIcon:   "logo.png"
    productWelcome: "welcome.jpg"

slideshow: "show.qml"
# show.qml is written for slideshow API 1 (a Timer running from start to
# end). Without this key, Calamares logs "Invalid or missing *slideshowAPI*"
# and guesses.
slideshowAPI: 1

# ── SIDEBAR COLOURS ──────────────────────────────────────────────────
# Calamares 3.3 accepts only these four names (initial capital, as read in
# libcalamaresui); any other name is rejected with "Unknown branding *style*
# entry" and the sidebar falls back to the Qt palette. "Current" = the step
# in progress.
style:
    SidebarBackground:         "#0d1b2a"
    SidebarText:               "#ffffff"
    SidebarBackgroundCurrent:  "#1b98e0"
    SidebarTextCurrent:        "#13293d"
```

3.102 /opt/GSM/osmo-operator/installer/calamares/branding/osmo/show.qml

6673 octets, 157 lignes → 157 lignes

```
import QtQuick 2.0;
import calamares.slideshow 1.0;

// show.qml - Calamares slideshow shown while the system is being copied.
// Two screenshots of the running bench (slide-banc.png: desktop, dashboard
// and tmux; slide-calypso-gdb.png: gdb attached to the emulated Calypso
// firmware) between text slides that summarise the image release notes
// (osmo-operator-desktop.iso v0.1-9, .github/release-notes.md): what the
// machine is receiving, component by component. Update at every tag.
// The "accounts" slide describes what the installation really does
// (see modules/shellprocess-osmo.conf): no osmocom account on the disk.
Presentation {
    id: presentation
```

```
Timer {
    interval: 9000
    running: true
    repeat: true
    onTriggered: presentation.goToNextSlide()
}
// Live desktop wallpaper, copied to welcome.jpg by iso_modules/85-installeur-bureau.sh.
Slide {
    Image {
        anchors.fill: parent
        anchors.margins: 12
        source: "welcome.jpg"
        fillMode: Image.PreserveAspectFit
        smooth: true
    }
}
Slide {
    Text {
        anchors.centerIn: parent
        horizontalAlignment: Text.AlignHCenter
        font.pixelSize: 20
        color: "#13293d"
        text: "osmo-operator-desktop.iso  v0.1-9\n\n" +
              "Un operateur GSM + LTE complet, sans SDR ni telephone :\n" +
              "la radio est emulee de bout en bout, jusqu'a la bande de base.\n" +
              "L'ARM du Calypso tourne sous QEMU (qosmo),\n" +
              "son DSP C54x execute la vraie ROM masque (c54x_exe).\n\n" +
              "Base Ubuntu 24.04, noyau 6.8 generic\n" +
              "ISO hybride (dd sur cle), Secure Boot shim > grub > noyau, option toram\n" +
              "8 Go de RAM et 4 coeurs minimum"
    }
}
Slide {
    Image {
        anchors.fill: parent
        anchors.margins: 12
        source: "slide-banc.png"
        fillMode: Image.PreserveAspectFit
        smooth: true
    }
    Text {
        anchors.bottom: parent.bottom
        anchors.horizontalCenter: parent.horizontalCenter
        font.pixelSize: 14
        color: "#13293d"
        text: "Le banc en marche : tableau de bord, mobile emule, chiffrement A5/1 au VTY"
    }
}
Slide {
    Text {
        anchors.centerIn: parent
        horizontalAlignment: Text.AlignHCenter
        font.pixelSize: 19
        color: "#13293d"
        text: "2G\n\n" +
              "osmo-stp, bsc, msc, hlr, sgsn, ggsn, pcu, mgw ; 2 cellules en LAC 1,\n" +
              "GPRS/EDGE, voix par osmo-sip-connector + Asterisk, SMS GSUP/SMPP.\n\n" +
              "Nouveau en 0.1-9\n\n" +
              "BTS classique par defaut : osmo-bts-trx + pont_dsp.py, layer1 + mobile sur la ROM DSP.\n" +
              "--ms-magnetite : Motorola C139 sous FreeCalypso Magnetite (firmware TI).\n" +
              "--bts : le Calypso devient la radio de la BTS ; --ts 1 | 2 | 2/1.\n" +
              "banc-rssi : l'appli rssi d'osmocom-bb sur un quatrieme Calypso.\n" +
              "Voix : --vocoder-rom (FR de la ROM, defaut) ou --gapk (hote)."
    }
}
Slide {
```

```
    Image {
        anchors.fill: parent
        anchors.margins: 12
        source: "slide-calypso-gdb.png"
        fillMode: Image.PreserveAspectFit
        smooth: true
    }
    Text {
        anchors.bottom: parent.bottom
        anchors.horizontalCenter: parent.horizontalCenter
        font.pixelSize: 14
        color: "#13293d"
        text: "gdb attache au firmware Calypso (layer1) qui tourne dans QEMU"
    }
}
Slide {
    Text {
        anchors.centerIn: parent
        horizontalAlignment: Text.AlignHCenter
        font.pixelSize: 19
        color: "#13293d"
        text: "4G\n\n" +
              "Open5GS + srsRAN sur ZMQ, PLMN 001/01, chiffrement NAS nul pour l'analyse,\n" +
              "CSFB par SGs entre la MME et osmo-msc.\n\n" +
              "Telephone\n\n" +
              "VM postmarketOS : data en 4G, voix et SMS en 2G sous une seule identite,\n" +
              "oFono sur un modem AT virtuel - code de deverrouillage 147147."
    }
}
Slide {
    Text {
        anchors.centerIn: parent
        horizontalAlignment: Text.AlignHCenter
        font.pixelSize: 19
        color: "#13293d"
        text: "Bureau\n\n" +
              "GNOME, Firefox (deb Mozilla), Wireshark, Linphone (poste 100), Conky.\n" +
              "Le ? du dock ouvre l'aide : modes, modules, notebooks, depots.\n" +
              "Reglages de toute la pile : http://127.0.0.1:8090\n" +
              "Nouvel ecran de demarrage Plymouth : une antenne et ses cercles.\n\n" +
              "Audio\n\n" +
              "PulseAudio systeme, PipeWire pour le screencast seul,\n" +
              "PCM gsm_out / gsm_in.\n\n" +
              "Retire : avahi, Chromium ; sssd masque."
    }
}
Slide {
    Text {
        anchors.centerIn: parent
        horizontalAlignment: Text.AlignHCenter
        font.pixelSize: 19
        color: "#13293d"
        text: "Vos comptes\n\n" +
              "L'utilisateur que vous venez de creer ouvre la session\n" +
              "(sudoer : il pilote le banc avec sudo).\n" +
              "root - le compte de travail, deverrouille,\n" +
              "avec le mot de passe demande a l'installation.\n" +
              "Le compte osmocom de la cle live n'est pas installe.\n\n" +
              "/home chiffre (LUKS) si vous l'avez coche : au demarrage,\n" +
              "« decrypt /home -passphrase- or Enter » - Entree a vide demarre sans.\n" +
              "Rien de telecom ne demarre au boot (preset 80-osmo-operator) :\n" +
              "banc, coeur Osmocom, BTS, Asterisk, oFono, Open5GS et srsRAN vous attendent."
    }
}
Slide {
    Text {
```

```
            anchors.centerIn: parent
            horizontalAlignment: Text.AlignHCenter
            font.pixelSize: 20
            color: "#13293d"
            text: "Pour demarrer le banc\n\n" +
                  "    cd /opt/GSM/osmo-operator\n" +
                  "    sudo ./start.sh --bts-classique --ms\n\n" +
                  "(ou le telephone rouge du dock ; sudo ./start.sh --help pour les autres modes)\n" +
                  "Le tableau de bord web ecoute deja sur cette machine."
        }
    }
}
```

3.103 /opt/GSM/osmo-operator/installer/calamares/jobs/osmosupplements/module.desc

371 octets, 9 lignes → 9 lignes

```
---
# module.desc - Calamares job osmosupplements: writes the choices of the
# "Supplements" and "Image operateur" pages to /etc/osmo-supplements in the
# target and enables osmo-supplements.service (first boot).
# Installed by iso_modules/85-installeur-bureau.sh in the system modules directory.
type: "job"
name: "osmosupplements"
interface: "python"
script: "main.py"
```

3.104 /opt/GSM/osmo-operator/installer/calamares/modules/bootloader.conf

692 octets, 17 lignes → 17 lignes

```
# bootloader.conf - GRUB, in EFI as well as BIOS mode.
#
# efiBootloaderId: the directory created under /boot/efi/EFI/. Keep "ubuntu",
# NOT "osmo-operator": the GRUB binaries signed by Canonical
# (grub-efi-amd64-signed) have /EFI/ubuntu compiled in as prefix. Installed
# anywhere else, the signed GRUB does not find its configuration and drops to
# the "grub>" prompt. The installed system thus stays Secure Boot bootable,
# like the ISO.
---
efiBootLoader:      grub
grubInstall:        grub-install
grubMkconfig:       grub-mkconfig
grubCfg:            /boot/grub/grub.cfg
grubProbe:          grub-probe
efiBootMgr:         efibootmgr
efiBootloaderId:    ubuntu
installEFIFallback: true
```

3.105 /opt/GSM/osmo-operator/installer/calamares/modules/contextualprocess-mirror.conf

820 octets, 16 lignes → 16 lignes

```
# ============================================================================
#  contextualprocess-mirror.conf - contextualprocess@mirror: writes the chosen
#  apt mirror into the target
# ============================================================================
#  Reads the globalstorage key packagechooser_mirror (page packagechooser@mirror).
#  Runs right after unpackfs, inside the target (dontChroot: false): every apt
#  that follows (NVIDIA driver, first-boot update) already reads the right
#  mirror. "keep" changes nothing. The other keys (m1, m2... in measurement
#  order) are generated by the osmo-install launcher at each start: a sed on
#  /etc/apt/sources.list, and on ubuntu.sources (deb822) if present.
---
dontChroot: false
timeout: 60
packagechooser_mirror:
    "keep":
        - "-/bin/true"
```

3.106 /opt/GSM/osmo-operator/installer/calamares/modules/contextualprocess-nvidia.conf

2006 octets, 26 lignes → 26 lignes

```
# ============================================================================
#  contextualprocess-nvidia.conf - contextualprocess@nvidia: installs the
#  NVIDIA driver when that entry is selected
# ============================================================================
#  Reads the globalstorage key packagechooser_nvidia (page packagechooser@nvidia).
#  Value "nvidia": installs nvidia-driver-610 INSIDE THE TARGET, a pinned
#  version from noble-updates/multiverse (no `ubuntu-drivers`, whose choice
#  varies between machines and needs network at the right moment):
#      sudo apt install nvidia-driver-610
#
#  Two guards: an NVIDIA card must be visible (lspci, vendor 10de) and the
#  network must answer; otherwise it is logged and the installation continues
#  (a disk without the proprietary driver is better than a failed install).
#
#  /run of the target is an empty tmpfs (mount.conf): /etc/resolv.conf, a link
#  to /run/systemd/resolve/stub-resolv.conf, points nowhere. Public resolvers
#  are written there for the duration of apt; the tmpfs vanishes at unmount.
---
dontChroot: false
timeout: 1800
packagechooser_nvidia:
    "nvidia":
        - "-/bin/bash -c \"if ! lspci -n 2>/dev/null | grep -qi ' 10de:'; then echo '[nvidia] aucune carte NVIDIA
 ↪  (10de) : pilote non installe'; exit 0; fi; mkdir -p /run/systemd/resolve; [ -s
 ↪  /run/systemd/resolve/stub-resolv.conf ] || printf 'nameserver 1.1.1.1\\\\nnameserver 8.8.8.8\\\\n' >
 ↪  /run/systemd/resolve/stub-resolv.conf; export DEBIAN_FRONTEND=noninteractive; apt-get update -qq || { echo
 ↪  '[nvidia] pas de reseau : pilote non installe'; exit 0; }; if apt-get install -y nvidia-driver-610; then echo
 ↪  '[nvidia] pilote installe :'; dpkg -l 'nvidia-driver-*' 2>/dev/null | awk '/^ii/{print \\$2, \\$3}'; else echo
 ↪  '[nvidia] nvidia-driver-610 non installable (absent du miroir ?) - le pilote libre nouveau reste en place';
 ↪  fi\""
        - "-/bin/bash -c \"update-initramfs -u >/dev/null 2>&1 || true\""
    "none":
        - "-/bin/true"
```

3.107 /opt/GSM/osmo-operator/installer/calamares/modules/displaymanager.conf

164 octets, 6 lignes → 6 lignes

```
# displaymanager.conf - GDM only: the display manager ubuntu-desktop-minimal installs.
---
displaymanagers:
    - gdm
basicSetup:      false
sysconfigSetup:  false
```

3.108 /opt/GSM/osmo-operator/installer/calamares/modules/finished.conf

154 octets, 4 lignes → 4 lignes

```
# finished.conf - last Calamares page: reboot proposed, unchecked by default.
---
restartNowMode: user-unchecked
restartNowCommand: "systemctl -i reboot"
```

3.109 /opt/GSM/osmo-operator/installer/calamares/modules/fstab.conf

267 octets, 9 lignes → 9 lignes

```
# fstab.conf - mount options written to /etc/fstab and /etc/crypttab of the target.
---
mountOptions:
    default:  defaults,noatime
    btrfs:    defaults,noatime,space_cache=v2
ssdExtraMountOptions:
    ext4:     discard
    btrfs:    discard
crypttabOptions: luks
```

3.110 /opt/GSM/osmo-operator/installer/calamares/modules/grubcfg.conf

1074 octets, 19 lignes → 19 lignes

```
# ================================================================================
#  grubcfg.conf - /etc/default/grub of the installed system
# ================================================================================
#  overwrite: false - the target file is completed instead of replaced: the
#  values dpkg put there (GRUB_CMDLINE_LINUX_DEFAULT from grub-common) remain.
---
overwrite: false
defaults:
    GRUB_TIMEOUT: 5
    GRUB_DISABLE_OS_PROBER: false
    # LUKS: without this key an encrypted installation DOES NOT BOOT.
    # partition.conf declares a single partition (root, 100%): with "Encrypt
    # system" checked, /boot lives INSIDE the LUKS container, there is no
    # cleartext /boot. GRUB must open the volume itself to read its kernel,
    # and grub-mkconfig only builds cryptomount/cryptodisk into the image when
    # it sees this "y" (otherwise: "error: disk `cryptouuid/...' not found"
    # then the grub> prompt). GRUB 2.12 (noble) opens LUKS2 argon2id.
    # On an unencrypted installation the key has no effect.
    GRUB_ENABLE_CRYPTODISK: "y"
```

3.111 /opt/GSM/osmo-operator/installer/calamares/modules/keyboard.conf

943 octets, 18 lignes → 18 lignes

```
# ============================================================================
#  keyboard.conf - keyboard configuration written to the target
# ============================================================================
#  Without this file Calamares logs "No config file for keyboard found" and
#  applies a default (us) keyboard. The values below are only the starting
#  point; the keyboard page still lets the operator change them.
# ----------------------------------------------------------------------------
---
# Generic 105-key keyboard model: the right default for a bench PC.
keyboardModel: pc105

# X11 keyboard configuration file written to the target and keymap conversion path.
xOrgConfFileName:   "00-keyboard.conf"
convertedKeymapPath: "/usr/share/X11/xkb/symbols/pc"

# Also write the console configuration (vconsole/keymap) to the target, not
# only X: a bench is often driven from a tty.
writeEtcDefaultKeyboard: true
```

3.112 /opt/GSM/osmo-operator/installer/calamares/modules/locale.conf

265 octets, 7 lignes → 7 lignes

```
---
# locale.conf - fixed time zone; the installer must not depend on the network
# for a question whose answer is known.
region:  Europe
zone:    Paris
# No geoip block: "style: none" is not a recognised value; leaving the key
# out is the real "no geoip lookup".
```

3.113 /opt/GSM/osmo-operator/installer/calamares/modules/mount.conf

716 octets, 25 lignes → 25 lignes

```
---
# mount.conf - extra mounts set up inside the target before the chroot steps.
# "options" is a LIST for Calamares 3.3 (noble: 3.3.5): main.py does
# ",".join(options). The 3.2 form "options: bind" mounts with -o b,i,n,d and
# grub-install then fails in the target.
extraMounts:
    - device: proc
      fs:     proc
      mountPoint: /proc
    - device: sys
      fs:     sysfs
      mountPoint: /sys
    - device: /dev
      mountPoint: /dev
      options: [ bind ]
    - device: tmpfs
      fs:     tmpfs
      mountPoint: /run
    - device: /run/udev
      mountPoint: /run/udev
      options: [ bind ]
extraMountsEfi:
```

```
    - device: efivarfs
      fs:     efivarfs
      mountPoint: /sys/firmware/efi/efivars
```

3.114 /opt/GSM/osmo-operator/installer/calamares/modules/packagechooser-image.conf

685 octets, 17 lignes → 17 lignes

```
---
# packagechooser-image.conf - origin of the operator Docker image, like the
# second window (radio buttons) of addition.sh. No effect unless
# "Docker et multi-operateur SS7" is checked on the supplements page.
mode: required
method: legacy
labels:
    step: "Image operateur"
    step[fr]: "Image operateur"
default: ghcr
items:
    - id: ghcr
      name: "Le depot (GHCR)"
      description: "ghcr.io/bbaranoff/osmo-operator/osmocom-nitb:latest, publiee par la CI a chaque push. Ignore
↪  sans le supplement multi-operateur."
    - id: hub
      name: "Docker Hub"
      description: "bastienbaranoff/norf_gsm, la pile publiee a la main. Ignore sans le supplement
↪  multi-operateur."
```

3.115 /opt/GSM/osmo-operator/installer/calamares/modules/packagechooser-mirror.conf

1045 octets, 22 lignes → 22 lignes

```
# ============================================================================
#  packagechooser@mirror - Ubuntu mirror used by the installed system
# ============================================================================
#  The installed system keeps the live key's mirror (chosen by build-iso.sh at
#  build time) unless the operator picks another one on this page.
#  Calamares measures nothing: osmo-install (the live launcher, build-iso.sh)
#  REWRITES this page on each launch from packaging/apt-mirror.sh --list: one
#  entry per responding mirror with its response time, the fastest one
#  pre-selected, plus a "keep" entry that changes nothing. The actual
#  replacement is done in contextualprocess-mirror.conf, after unpackfs, in the
#  target.
---
mode: required
method: legacy
labels:
    step: "Miroir Ubuntu"
    step[fr]: "Miroir Ubuntu"
default: keep
items:
    - id: keep
      name: "Celui de la cle live"
      description: "Aucune mesure disponible : le systeme installe garde le miroir de la cle. @@MIRROR_STATE@@"
```

3.116 /opt/GSM/osmo-operator/installer/calamares/modules/packagechooser-nvidia.conf

1449 octets, 29 lignes → 29 lignes

```
# ============================================================================
#  packagechooser@nvidia - optional NVIDIA driver page
# ============================================================================
#  Two entries: no extra driver (default) or the proprietary NVIDIA driver,
#  pinned to nvidia-driver-610 (no ubuntu-drivers selection).
#  Calamares cannot grey out an entry based on hardware, so osmo-install (the
#  live launcher, build-iso.sh) REWRITES this file just before starting
#  Calamares:
#    - NVIDIA card seen by lspci -> the NVIDIA entry shows the card name and is
#                                   pre-selected;
#    - no card                   -> the entry stays but is marked "no card
#                                   detected" and installs nothing.
#  The installation itself is in contextualprocess-nvidia.conf: it runs only
#  if the entry is selected AND a card is present (lspci in the target), and it
#  needs network access (packages come from the Ubuntu archive).
---
mode: optional
method: legacy
labels:
    step: "Pilotes graphiques"
    step[fr]: "Pilotes graphiques"
default: none
items:
    - id: none
      name: "Aucun pilote supplementaire"
      description: "Le pilote libre (nouveau) du noyau, comme sur la cle live."
    - id: nvidia
      name: "Pilote NVIDIA 610 (proprietaire)"
      description: "Installe nvidia-driver-610 depuis les depots Ubuntu (reseau requis). @@NVIDIA_STATE@@"
```

3.117 /opt/GSM/osmo-operator/installer/calamares/modules/packagechooser-supplements.conf

1507 octets, 28 lignes → 28 lignes

```
---
# packagechooser@supplements - addition.sh supplements in the installer: the same
# five choices as its zenity dialog (same keys), as checkboxes. They are installed
# at FIRST BOOT once the network is up (osmo-supplements.service), not during
# Calamares (the operator image is ~11 GB and the firmwares take long to build).
# The osmosupplements job (installer/calamares/jobs) writes the choices to
# /etc/osmo-supplements in the target.
mode: optionalmultiple
method: legacy
labels:
    step: "Supplements"
    step[fr]: "Supplements"
items:
    - id: multi
      name: "Docker et multi-operateur SS7"
      description: "docker.io, l image operateur (~11 Go, origine choisie a la page suivante) et la topologie op1
↪  natif + op2/op3 docker + inter-STP."
    - id: opencl
      name: "OpenCL (calcul GPU)"
      description: "Runtime ICD, clinfo, le pilote de la carte detectee (Intel / Mesa-AMD, pocl en repli) et les
↪  outils de calcul GPU du banc."
    - id: claude
      name: "Claude Code"
```

```
      description: "La CLI de l assistant IA (installeur natif claude.ai/install.sh), lancee ensuite avec la
↪  commande claude."
    - id: firmwares
      name: "Firmwares telephone"
      description: "gnuarm (arm-elf-gcc 4.5.2), FreeCalypso Magnetite, osmocom-bb-trx, Fernvale MT6260,
↪  documentation SIMCom. Long : plusieurs Go et des compilations."
    - id: extras
      name: "Jeux et media"
      description: "Doom, Quake III (OpenArena), Kodi, YouTube (Firefox + uBlock Origin), Wireshark root,
↪  Linphone."
```

3.118 /opt/GSM/osmo-operator/installer/calamares/modules/partition.conf

3933 octets, 84 lignes → 84 lignes

```
# ================================================================================
#  partition.conf - partitioning of the target disk
# ================================================================================
#  Erase-and-install (or manual partitioning), ext4, GPT when booting UEFI.
# --------------------------------------------------------------------------------
---
# Mount point of the EFI system partition. Must match what bootloader/grubcfg
# expect (GRUB EFI installs grubx64/shimx64 there). /boot/efi is the
# Debian/Ubuntu convention.
efiSystemPartition:     "/boot/efi"

# ESP size: room for shim + grub + a fallback kernel.
efiSystemPartitionSize: 300MiB
efiSystemPartitionName: EFI

# Default root filesystem.
defaultFileSystemType:  "ext4"

# Partition table when nothing forces otherwise: GPT on UEFI (this ISO boots
# through signed shim), msdos on legacy BIOS.
defaultPartitionTableType: gpt

# Swap: none by default; "swapfile" can be selected for small machines.
initialSwapChoice: none
userSwapChoices:
    - none
    - swapfile

# Full erase AND manual partitioning are allowed; no "alongside" install.
allowManualPartitioning: true

# ── LUKS: ONLY /home IS ENCRYPTED, NOT THE ROOT ──────────────────────────
# The "Encrypt system" checkbox (automatic modes) does NOT encrypt everything:
# the root partition has "noEncrypt" below, only /home becomes LUKS2.
#
# Model: everything system-related stays readable (the bench is driven as root
# and must be repairable from a live key without decrypting anything); secrets
# (keyring, cookies, tokens, profiles) live in /home and are encrypted.
#
# Consequences: GRUB reads the root directly, no separate /boot, no passphrase
# in the initramfs, no /crypto_keyfile.bin (luksbootkeyfile sees no encrypted
# root and does nothing). /home is opened later by unlock-home.
#
# Calamares writes a regular crypttab line for /home, which would make systemd
# block at boot waiting for the passphrase. crypthome-postinstall
# (shellprocess@osmo) switches crypttab and fstab to "noauto" and fills
# /etc/default/crypthome so that unlock-home.service asks instead, with a
# timeout, and boot completes whatever the answer.
#
```

```
# In MANUAL partitioning nothing is enforced: encryption is chosen per
# partition, and crypthome-postinstall wires up whatever it finds under /home,
# encrypted or not.
enableLuksAutomatedPartitioning: true
luksGeneration: luks2
# LVM: not offered in automatic mode; available in MANUAL partitioning
# (create a volume group, put the root on it, optionally encrypt the physical
# volume for LUKS-on-LVM). lvm2 is in the rootfs (build-iso.sh, desktop block),
# so the installer, the target initrd (initramfscfg + initramfs) and the
# installed system support it.

# ── LAYOUT ──────────────────────────────────────────────────────────────
# Two partitions; a separate /home is what makes encrypting only /home possible.
#
# Sizes: half/half with two limits. The root holds /opt/GSM, docker images and
# the bench .debs, hence a 25 GiB floor (above the 20 GiB required by
# welcome.conf). /home holds profiles AND pmbootstrap disk images (tens of GB),
# so it gets the rest (100% = whatever is left).
#
# On a disk too small for both, the root takes everything and there is no
# separate /home: crypthome-postinstall detects and reports it.
partitionLayout:
    - name: "osmo-egprs"
      filesystem: "ext4"
      mountPoint: "/"
      size: 50%
      minSize: 25GiB
      # The root stays READABLE even when the operator ticks "encrypt"; without
      # this line the whole system would be encrypted and live-key repair lost.
      noEncrypt: true
    - name: "crypthome"
      filesystem: "ext4"
      mountPoint: "/home"
      size: 100%
      minSize: 5GiB
```

3.119 /opt/GSM/osmo-operator/installer/calamares/modules/shellprocess-grub.conf

1039 octets, 17 lignes → 17 lignes

```
# ============================================================================
#  shellprocess-grub - check that grub-install exists in the target before the
#                      bootloader module runs
# ============================================================================
#  The bootloader module runs inside the installed system (dontChroot: false),
#  so grub-install (package grub2-common) must be in the squashfs; build-iso.sh
#  installs it in the rootfs and refuses to build an ISO without it. If it is
#  missing anyway, bootloader fails with exit code 127 and the disk is left
#  unbootable.
#  This module is a single non-fatal check ("-" prefix): it logs a warning in
#  the installer log, before the bootloader module, if grub-install is missing.
---
dontChroot: false
timeout: 600
script:
    # Log a warning if grub-install is missing in the target.
    - "-/bin/bash -c \"command -v grub-install >/dev/null || echo 'ATTENTION: grub-install absent de la cible - le
 ↪  module bootloader va echouer (code 127)' >&2\""
```

3.120 /opt/GSM/osmo-operator/installer/calamares/modules/shellprocess-osmo.conf

7720 octets, 95 lignes → 95 lignes

```
# ============================================================================
#  shellprocess-osmo - turn the copied live system into an INSTALLED system
# ============================================================================
#  The installation settings take precedence over the live key defaults:
#    - the user is the one from the "Users" page (${USER}), with its password,
#      its autologin if ticked, and its hostname;
#    - root stays a working account: unlocked, password = the one entered
#      during installation (users.conf, setRootPassword);
#    - the live key's osmocom account is removed (unless it is the chosen name);
#    - the live key's root autologins (console, GDM) and the PAM rule change
#      that allowed them are reverted: GDM logs into ${USER}, as on Ubuntu.
#  The bench is still driven as root (sudo -i or root login): desktop
#  launchers, units and /opt/GSM are set up for ${USER}, who is a sudoer
#  (users.conf, defaultGroups).
#
#  dontChroot: false - commands run INSIDE the mounted target, not on the live
#  system (otherwise the USB key's root account would be modified).
#  Generous timeout: update-initramfs is slow on slow disks.
#  ${USER} and ${ROOT} are substituted by Calamares (shellprocess module).
---
dontChroot: false
timeout: 600
script:
    # Unlock root; its password is the one entered on the "Users" page
    # (users.conf, setRootPassword).
    - "-/usr/bin/passwd -u root"

    # ── Remove the live osmocom account ──────────────────────────────────
    # Unless the installer chose that name: then it is the user's own account,
    # created by the users module, and is left alone.
    - "-/bin/bash -c \"[ \\\"${USER}\\\" = osmocom ] || { pkill -u osmocom 2>/dev/null; userdel -r osmocom
    2>/dev/null; groupdel osmocom 2>/dev/null; rm -rf /home/osmocom; }\""

    # ── Encrypted /home: wire up what the installer produced ─────────────
    # Must run AFTER the osmocom removal: it writes CRYPTHOME_OWNER=${USER}, the
    # session account that owns the encrypted home.
    # It encrypts nothing (Calamares did, see partition.conf: the root has
    # noEncrypt, only /home is LUKS2). It switches crypttab and fstab to "noauto"
    # so that unlock-home asks at boot without blocking it, and puts root's
    # keyring on the volume. If /home is not encrypted, it says so and does nothing.
    # At boot: "yes" opens GDM on ${USER} (persistent), "no" on root, ephemeral,
    # writing to /tmp (unlock-home, session_root). The GDM lines below are the
    # "yes" configuration, which unlock-home restores at every boot.
    - "-/usr/local/sbin/crypthome-postinstall ${USER}"

    # ── Remove the live key's root autologins ────────────────────────────
    # Console: tty1 and ttyS0 drop-ins installed by build-iso.sh.
    - "-/bin/bash -c \"rm -f /etc/systemd/system/getty@tty1.service.d/10-autologin-root.conf
    /etc/systemd/system/serial-getty@ttyS0.service.d/10-autologin-root.conf\""
    - "-/bin/bash -c \"rmdir /etc/systemd/system/getty@tty1.service.d
    /etc/systemd/system/serial-getty@ttyS0.service.d 2>/dev/null || true\""
    # GDM: the displaymanager module (just before, in settings.conf) wrote the
    # ${USER} autologin if ticked. A leftover AutomaticLogin=root line from the
    # live key is switched to the installed user.
    - "-/bin/bash -c \"[ -f /etc/gdm3/custom.conf ] && sed -i 's/^AutomaticLogin=root$/AutomaticLogin=${USER}/'
    /etc/gdm3/custom.conf || true\""
    # Restore Ubuntu's PAM rule 'user != root' (commented out on the live key so
    # GDM could open a root session).
    - "-/bin/bash -c \"sed -i 's/^#\\\\(auth.*pam_succeed_if.so user != root quiet_success\\\\)/\\\\1/'
    /etc/pam.d/gdm-password /etc/pam.d/gdm-autologin 2>/dev/null || true\""

    # ── ${USER}'s desktop: same launchers as the live key ────────────────
    # The home comes from /etc/skel, filled by build-iso.sh (launchers, favourites,
    # gnome-initial-setup-done). Make sure a home created before skel was read
    # (users module) has the launchers and the right owner.
```

```
    - "-/bin/bash -c \"h=\\$(getent passwd ${USER} | cut -d: -f6); [ -n \\\"\\$h\\\" ] && for d in Desktop Bureau;
↪   do [ -d /etc/skel/\\$d ] && mkdir -p \\$h/\\$d && cp -n /etc/skel/\\$d/*.desktop \\$h/\\$d/ 2>/dev/null; done;
↪   [ -n \\\"\\$h\\\" ] && chown -R ${USER}: \\$h || true\""

    # Web dashboard unit, taken from the repository shipped in the target
    # (/opt/GSM/osmo-operator), so nothing is downloaded. The TLS certificate is
    # generated at build time (Dockerfile).
    - "-/bin/bash -c \"cp -f /opt/GSM/osmo-operator/services/osmo-egprs-web.service /etc/systemd/system/
↪   2>/dev/null || true\""
    - "-/bin/bash -c \"mkdir -p /etc/systemd/system/multi-user.target.wants && ln -sf
↪   /etc/systemd/system/osmo-egprs-web.service
↪   /etc/systemd/system/multi-user.target.wants/osmo-egprs-web.service\""

    # ── Remove the "Install" icon ──────────────────────────────────────
    # build-iso.sh puts it on root's desktop and in /etc/skel. On an installed
    # disk it would open Calamares with the machine's own disk as a target.
    # Remove it from every desktop under both names ("Desktop" and "Bureau" both
    # exist; the session locale decides which one is shown) and from
    # /usr/share/applications (application menu). /usr/local/bin/osmo-install is
    # kept: it can install ANOTHER machine from this one with a live key plugged in.
    - "-/bin/bash -c \"rm -f /root/Desktop/osmo-install.desktop /root/Bureau/osmo-install.desktop
↪   /usr/share/applications/osmo-install.desktop /etc/skel/Desktop/osmo-install.desktop
↪   /etc/skel/Bureau/osmo-install.desktop\""
    - "-/bin/bash -c \"for h in /home/*; do rm -f \\$h/Desktop/osmo-install.desktop
↪   \\$h/Bureau/osmo-install.desktop; done\""
    # The calamares package's own entry (NoDisplay on the live key) is removed:
    # it starts /usr/bin/calamares directly, bypassing osmo-install's safeguards.
    - "-/bin/bash -c \"rm -f /usr/share/applications/calamares.desktop\""
    - "-/bin/bash -c \"update-desktop-database /usr/share/applications >/dev/null 2>&1 || true\""

    # ── Smartphone UI: ${USER}'s pmbootstrap config ────────────────────
    # pmbootstrap runs as the session user (it refuses root) and reads
    # ~/.config/pmbootstrap_v3.cfg. 88-lte-pmos.sh installed the template in
    # /etc/skel (pmbootstrap_v3.cfg.osmo, with @WORK@/@APORTS@); instantiate it
    # here with ${USER}'s home so the icon works immediately (osmo-pmos-build
    # would do the same on first launch). Work dir: ~/.local/var/pmbootstrap.
    - "-/bin/bash -c \"h=\\$(getent passwd ${USER} | cut -d: -f6); t=/etc/skel/.config/pmbootstrap_v3.cfg.osmo; [
↪   -n \\\"\\$h\\\" ] && [ -f \\$t ] && [ ! -f \\$h/.config/pmbootstrap_v3.cfg ] && mkdir -p \\$h/.config && sed
↪   \\\"s#@WORK@#\\$h/.local/var/pmbootstrap#g; s#@APORTS@#\\$h/.local/var/pmbootstrap/cache_git/pmaports#g\\\"
↪   \\$t > \\$h/.config/pmbootstrap_v3.cfg && chown -R ${USER}: \\$h/.config || true\""
    - "-/bin/bash -c \"chmod -R a+rX /opt/user_interface 2>/dev/null || true\""

    # Purge live-boot: its initramfs scripts would look for a live medium at
    # every boot.
    - "-/bin/bash -c \"apt-get purge -y live-boot live-boot-initramfs-tools >/dev/null 2>&1 || true\""
    - "-/bin/bash -c \"update-initramfs -u >/dev/null 2>&1 || true\""
```

3.121 /opt/GSM/osmo-operator/installer/calamares/modules/unpackfs.conf

827 octets, 17 lignes → 17 lignes

```
# unpackfs.conf - source of the system copied onto the target disk.
#
# /run/osmo-install-src/live/filesystem.squashfs is a path that
# /usr/local/bin/osmo-install guarantees before starting Calamares: it finds the
# squashfs actually mounted (via the loop device backing the running root) and
# bind-mounts it there. This makes the path independent of the boot mode:
# live-boot's /run/live/medium/live/filesystem.squashfs only exists in normal
# mode ("toram" puts the file at /run/live/medium/filesystem.squashfs,
# "persistent" may leave /run/live/medium empty).
#
# Consequence: starting /usr/bin/calamares directly does not work; always use
# osmo-install (which also cleans up targets left mounted).
---
```

```
unpack:
    - source: /run/osmo-install-src/live/filesystem.squashfs
      sourcefs: squashfs
      destination: ""
```

3.122 /opt/GSM/osmo-operator/installer/calamares/modules/users.conf

1953 octets, 53 lignes → 53 lignes

```
# ====================================================================================
#  users.conf - account created during installation
# ====================================================================================
#  The installer creates the account entered on this page (sudoer).
#  shellprocess@osmo, which runs afterwards, removes the live key's osmocom
#  account (unless it is the chosen name) and keeps root as an unlocked working
#  account.
#
#  setRootPassword: true - root is the bench's working account, so "su" needs
#  a root password. doAutologin: true - the installer sets up autologin for
#  the created account if ticked.
---
defaultGroups:
    - adm
    - cdrom
    - sudo
    - dip
    - plugdev
    - lpadmin
    - dialout
    - audio
    - video
    # pulse-access: bench audio goes through a SYSTEM PulseAudio
    # (osmo-pulse.service). Its socket accepts anonymous clients (auth-anonymous=1,
    # set by iso_modules/82-services.sh), but this group is PulseAudio's documented
    # access path in system mode.
    - pulse-access
    # docker: Conky (tools/conky-osmo-status.sh) and the panel
    # (tools/osmo-panel.py) run under THIS account and probe each containerised
    # operator with "docker exec"; without the group every probe fails with
    # "permission denied" and the operator is shown as stopped.
    # The group exists only once docker is installed (addition.sh); Calamares
    # silently ignores a missing group, and addition.sh adds the session account
    # itself (_docker_groupe_session).
    - docker

autologinGroup:  autologin
doAutologin:     true
sudoersGroup:    sudo
setRootPassword: true
doReusePassword: false

# No password policy on a lab bench.
passwordRequirements:
    nonempty:  false
    minLength: -1

user:
    shell: /bin/bash
    # Only root is forbidden. Choosing "osmocom" makes it the user's own account
    # (shellprocess-osmo.conf then keeps it).
    forbidden_names:
        - root
```

3.123 /opt/GSM/osmo-operator/installer/calamares/modules/welcome.conf

401 octets, 16 lignes → 16 lignes

```
# welcome.conf - Calamares welcome page: installation requirements
# (at least 20 GiB of disk and 2 GiB of RAM; disk space and root are mandatory).
---
showSupportUrl:      false
showKnownIssuesUrl:  false
showReleaseNotesUrl: false
requirements:
    requiredStorage:  20
    requiredRam:      2
    check:
        - storage
        - ram
        - root
    required:
        - storage
        - root
```

3.124 /opt/GSM/osmo-operator/instances/instance.env.example

5219 octets, 173 lignes → 173 lignes

```
# instances/instance.env.example — patron des reglages PAR INSTANCE Calypso (c54x_exe/run.sh).
# Genere par tools/globals-complet.py ; la page web (tools/osmo-reglages.py) ecrit instance-N.env :
#   1 = la radio de la BTS (TRX Calypso, --bts) ou le mobile sur le pont (--bts-classique)
#   2 = le mobile QEMU par l air (--bts --ms, --ms-magnetite)
#   3 = le 2e TRX Calypso, TS1 (--ts 2)
# run.sh lit instances/instance-$INSTANCE.env au demarrage, sous la forme : "${VAR:=valeur}" :
# l environnement (start-native.sh, ligne de commande) garde la priorite, la valeur du fichier vient
# ensuite, puis le defaut de run.sh. Valeur VIDE = defaut.

# ── /opt/GSM/c54x_exe/run.sh ──────────────────────────────

# defaut : dsp   (run.sh:52)
# choix : dsp | grgsm
MODE=""
# defaut : 0   (run.sh:53)
# choix : 0 | 1
PONT=""
# defaut : /tmp/osmocom_l2_$INSTANCE   (run.sh:68)
L2_SOCK=""
# defaut : /tmp/qemu-monitor-pont$INSTANCE.sock   (run.sh:69)
MONITOR=""
# defaut : /tmp/calypso_dsp$INSTANCE.sock   (run.sh:70)
DSP_SOCK=""
# defaut : $((1233 + INSTANCE))   (run.sh:71)
GDB_STUB=""
# defaut : $((44443 + INSTANCE))   (run.sh:72)
GDB_TELNET=""
# defaut : none   (run.sh:73)
PANNEAU_LOGS=""
# defaut : $((6702 + 10 * (INSTANCE - 1) + (INSTANCE >= 3 ? 30 : 0)))   (run.sh:76)
CALYPSO_BSP_PORT=""
# defaut : /tmp/calypso_keys$INSTANCE.sock   (run.sh:77)
CALYPSO_KEYS_SOCK=""
# defaut : 1   (run.sh:78)
# choix : 0 | 1
AIR=""
# defaut : /opt/GSM/osmo-operator/pont/air.py   (run.sh:81)
AIR_PY=""
```

```
# defaut : /opt/GSM/qosmo   (run.sh:82)
QOSMO=""
# defaut : $QOSMO/build/qemu-system-arm   (run.sh:83)
QEMU=""
# defaut : 0   (run.sh:90)
# choix : 0 | 1
OSMO_TRX=""
# defaut : (vide)   (run.sh:91)
FIRMWARE_APP=""
# defaut : 0   (run.sh:106)
# choix : 0 | 1
OSMO_TRX_TS1=""
# defaut : 3   (run.sh:107)
OSMO_TRX_TS1_INSTANCE=""
# defaut : 1   (run.sh:108)
OSMO_TRX_PHONE=""
# defaut : $(awk '/^ *bts 0$/{f=1} f&&/base_station_id_code/{print $2; exit}' /etc/osmocom/osmo-bsc.cfg
 ↪  2>/dev/null)   (run.sh:124)
OSMO_TRX_BSIC=""
# defaut : /opt/GSM/osmocom-bb-transceiver/src/host/layer23/src/transceiver/transceiver   (run.sh:125)
TRANSCEIVER=""
# defaut : $([ "$OSMO_TRX_TS1" = 1 ] && echo 0 || echo 1)   (run.sh:132)
# choix : 0 | 1
OSMO_TRX_TCH=""
# defaut : 0x03   (run.sh:134)
OSMO_TRX_TS_MASK=""
# defaut : 0x03   (run.sh:135)
OSMO_TRX_TX_MASK=""
# defaut : 2   (run.sh:136)
CALYPSO_TRX_TX_SLOTS=""
# defaut : --no-sync -e 3   (run.sh:146)
OSMO_TRX_ARGS=""
# defaut : 1   (run.sh:149)
# choix : 0 | 1
MONTANT_ROM_TX_CHAQUE=""
# defaut : 1   (run.sh:153)
# choix : 0 | 1
CALYPSO_BSP_TRX=""
# defaut : -11   (run.sh:157)
CALYPSO_BSP_TRX_DELTA=""
# defaut : $_FW/layer1.highram.elf   (run.sh:161)
FIRMWARE_ELF=""
# defaut : ${FIRMWARE_ELF%.elf}.bin   (run.sh:162)
FIRMWARE_BIN=""
# defaut : (vide)   (run.sh:171)
FIRMWARE_FLASH=""
# defaut : 0   (run.sh:180)
# choix : 0 | 1
CALYPSO_PACER_RATTRAPAGE=""
# defaut : 1   (run.sh:181)
# choix : 0 | 1
CALYPSO_PONT_ARM_LOCK=""
# defaut : 1   (run.sh:184)
# choix : 0 | 1
CALYPSO_BSP_SB_FIN_TRAME=""
# defaut : 1   (run.sh:185)
# choix : 0 | 1
CALYPSO_BSP_NB_MARGE=""
# defaut : (vide)   (run.sh:188)
PONT_CAN_NB_TOA=""
# defaut : 0.5   (run.sh:193)
CALYPSO_BSP_SB_DEC=""
# defaut : 1   (run.sh:194)
# choix : 0 | 1
CALYPSO_BSP_BRUIT_FOND=""
# defaut : $AIR   (run.sh:199)
```

```
CALYPSO_BSP_RATTRAPE=""
# defaut : 1   (run.sh:200)
# choix : 0 | 1
CALYPSO_RACH_VERS_AIR=""
# defaut : 0   (run.sh:205)
# choix : 0 | 1
PONT_UL_TN_DEFAUT=""
# defaut : 0   (run.sh:209)
# choix : 0 | 1
AIR_B_FN_DELTA=""
# defaut : 0   (run.sh:210)
# choix : 0 | 1
MONTANT_RACH_DIFFERE=""
# defaut : $( . /etc/osmocom/radio-plan.env 2>/dev/null; echo "${PLAN_ARFCN:-}")   (run.sh:211)
CALYPSO_FAUX_AIR=""
# defaut : ti925t   (run.sh:213)
QEMU_CPU=""
# defaut : c139   (run.sh:214)
CALYPSO_LCD=""
# defaut : /opt/GSM/fc-host-tools/rvinterf/lowlevel/rvinterf   (run.sh:215)
RVINTERF_BIN=""
# defaut : /opt/GSM/osmocom-bb/src/host/osmocon/osmocon   (run.sh:217)
OSMOCON=""
# defaut : $(command -v mobile || echo /usr/local/bin/mobile)   (run.sh:218)
MOBILE=""
# defaut : $HERE/mobile_pont.cfg   (run.sh:219)
MOBILE_CFG=""
# defaut : /opt/GSM/osmo-operator/pont/pont.py   (run.sh:224)
PONT_PY=""
# defaut : (vide)   (run.sh:239)
RT_FIFO=""
# defaut : (vide)   (run.sh:240)
RT_CPUS=""
# defaut : (vide)   (run.sh:241)
RT_RENICE_AUTRES=""
# defaut : 1   (run.sh:263)
# choix : 0 | 1
GDB=""
# defaut : $QOSMO/tools/gdb-telnet.py   (run.sh:266)
GDB_TELNET_PY=""
# defaut : 120000   (run.sh:269)
INSNS=""
# defaut : 1   (run.sh:276)
# choix : 0 | 1
LOCKSTEP=""
# defaut : -v   (run.sh:278)
VERB=""
# defaut : none   (run.sh:279)
IQ=""
# defaut : 30000   (run.sh:280)
AMP=""
# defaut : 1   (run.sh:294)
# choix : 0 | 1
PONT_AIRREC=""
# defaut : 4194304   (run.sh:385)
QEMU_LOG_BUFFERED=""
# defaut : 64   (run.sh:422)
CALYPSO_PONT_RETRY_DIV=""
# defaut : 10   (run.sh:722)
JOURNAUX_GARDES=""
# defaut : (vide)   (run.sh:763)
BRUIT_MODE=""
# defaut : (vide)   (run.sh:764)
BRUIT_SNR_DB=""
# defaut : 16702   (run.sh:765)
BRUIT_PORT_DSP=""
```

```
# defaut : 14730   (run.sh:766)
BRUIT_PORT_GRGSM=""
# defaut : $BRUIT_SNR_DB   (run.sh:811)
CALYPSO_BSP_SNR_DB=""
```

3.125 /opt/GSM/osmo-operator/packaging/.gitignore

6 octets, 1 lignes → 1 lignes

```
dist/
```

3.126 /opt/GSM/osmo-operator/services/80-osmo-operator.preset

1972 octets, 53 lignes → 53 lignes

```
# 80-osmo-operator.preset - default policy: no telecom service at boot
#
# A mobile network is started on purpose (icon, start.sh, systemctl start): it
# is not a side effect of the machine booting. This systemd preset (installed in
# /usr/lib/systemd/system-preset/, read before 90-systemd.preset) says
# « disable » for everything that runs a network element: the bench and its
# variants, the Osmocom core, the BTSs, Asterisk, oFono, Open5GS, srsRAN.
# It touches nothing else: the web consoles (osmo-egprs-web, osmo-reglages,
# open5gs-webui), the sound (osmo-pulse), the address plan (osmo-ip-plan), the
# purge, the updates and the desktop keep their state.
#
# Applied by tools/osmo-telecom-boot.sh (ISO: iso_modules/82-services.sh;
# already installed machines: update.sh, once). A service enabled by hand
# afterwards (systemctl enable osmo-banc) stays enabled: this file is only used
# at install time and by `systemctl preset`, never in the background.

# the benches
disable osmo-banc.service
disable osmo-multi.service
disable osmo-lte.service
disable osmo-egprs-dsp.service
disable osmo-dsp-rom-max.service
disable osmo-interstp.service
# the Osmocom core and the base stations
disable osmo-msc.service
disable osmo-bsc.service
disable osmo-hlr.service
disable osmo-mgw.service
disable osmo-stp.service
disable osmo-sgsn.service
disable osmo-ggsn.service
disable osmo-gtphub.service
disable osmo-gbproxy.service
disable osmo-pcu.service
disable osmo-sip-connector.service
disable osmo-cbc.service
disable osmo-smlc.service
disable osmo-hnbgw.service
disable osmo-hnodeb.service
disable osmo-bts-*.service
disable osmo-trx-*.service
disable osmo-nitb.service
# voice and modems
disable asterisk.service
disable ofono.service
disable kamailio.service
disable yate.service
```

```
# 4G / 5G (the open5gs-webui web console is not a network element)
disable open5gs-*d.service
disable srsenb.service
disable srsepc.service
disable srsue.service
disable srsran*.service
```

3.127 /opt/GSM/osmo-operator/services/open5gs-webui.service

2284 octets, 48 lignes → 48 lignes

```
[Unit]
# ============================================================================
#  open5gs-webui.service - the 4G subscriber console (Open5GS WebUI), on :9999
# ============================================================================
#  The Open5GS WebUI lives in the sources (webui/ of the open5gs repository)
#  and is a Next.js application: node server/index.js. It is the console used
#  to provision LTE subscribers (the alternative is MongoDB by hand, or
#  « osmo-epc subscribers », which replays the repository JSON).
#
#  It does not talk to the daemons, it reads MongoDB - the same database as the
#  HSS and the PCRF (DB_URI). Hence After=mongod: without the database the WebUI
#  starts and serves empty pages. It does not depend on Open5GS itself:
#  subscribers are usually provisioned with the core stopped.
#
#  Condition, not dependency. The unit is enabled by default (ISO and native),
#  but the WebUI is not always built: the bench .deb packages
#  (osmo-build-open5gs) only ship the install prefix, not the sources nor
#  node_modules. Without server/index.js, systemd skips the unit instead of
#  marking it failed (a red service on a healthy bench would be a false alarm,
#  and the Conky counts them). « osmo-lte-install --webui » builds it.
Description=Open5GS WebUI - console des abonnes 4G (:9999)
Documentation=https://open5gs.org/open5gs/docs/
After=network.target mongod.service
Wants=mongod.service
ConditionPathExists=/opt/LTE/open5gs/webui/server/index.js

[Service]
Type=simple
WorkingDirectory=/opt/LTE/open5gs/webui
Environment=NODE_ENV=production
# 0.0.0.0: the console is used from the operator's workstation, not only from
# the bench machine.
Environment=HOSTNAME=0.0.0.0
Environment=PORT=9999
# The same database as hss.yaml and pcrf.yaml (configs/open5gs/): a console
# writing elsewhere would provision subscribers the HSS never sees.
Environment=DB_URI=mongodb://localhost/open5gs
# node lives in /usr/local/bin on the bench (image) and in /usr/bin on an apt
# system: env finds it in both cases, ExecStart requires an absolute path.
ExecStart=/usr/bin/env node server/index.js
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal
SyslogIdentifier=open5gs-webui

[Install]
WantedBy=multi-user.target
```

3.128 /opt/GSM/osmo-operator/services/osmo-banc.service

5654 octets, 94 lignes → 94 lignes

```
[Unit]
Description=osmo-operator - banc GSM autonome (start.sh, session tmux « calypso »)
Documentation=file:/opt/GSM/osmo-operator/README.md
# The bench needs the system sound (osmo-pulse), the node's private addresses
# and the dashboard (the bridge pushes the FFT to it). No Requires=: a missing
# service must not prevent the radio from coming up.
# network.target only, not network-online.target: the latter can cost up to
# 2 min at boot (systemd-networkd-wait-online) on a bench whose interfaces
# (apn0, the tun/veth of the packet core) are never "online" in that sense and
# which often boots without a cable. The interfaces are configured; the default
# route may arrive later.
After=network.target osmo-pulse.service osmo-ip-plan.service osmo-egprs-web.service osmo-qemu-link.service
# osmo-egprs-web is pulled in by the bench (Wants=): a stop/start cycle must
# not leave the dashboard down.
Wants=osmo-pulse.service osmo-egprs-web.service
ConditionPathExists=/opt/GSM/osmo-operator/start.sh
# One native bench at a time: the multi-operator bench depends on this one
# (Requires= in osmo-multi.service), it does not replace it.
#
# Installed, not enabled: a fresh machine must not bring up QEMU/Calypso and
# the whole core before the session opens, on a configuration (node, --dsp,
# topology) nobody has chosen.
#   systemctl start osmo-banc           for one session (or the desktop icon)
#   systemctl enable --now osmo-banc    to start it at every boot
# At build time: OSMO_ISO_BANC=1 (or --banc) enables the unit in the image.

[Service]
# ── Oneshot + RemainAfterExit: the bench lives in tmux, not in ExecStart ──
# start.sh hands over to run.sh (qosmo-grgsm), which brings everything up in a
# tmux session and then exits without attaching (no tty). With Type=simple,
# systemd would take that exit for the end of the service and kill the cgroup,
# hence the whole stack. With oneshot + RemainAfterExit the unit stays
# « active (exited) » and the cgroup processes (tmux, daemons, qemu, bridge)
# live until ExecStop.
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/opt/GSM/osmo-operator
# ── The same environment as a root terminal ──────────────────────────────
# HOME: mobile.cfg lives in /root/.osmocom/bb. PATH: the python venv
# (/root/.env) that ~/.bashrc activates; gr-gsm and the bridges depend on it and
# systemd does not read ~/.bashrc. /etc/osmocom/coeur.env (N_MS) is loaded by
# start.sh itself.
Environment=HOME=/root USER=root LOGNAME=root
Environment=VIRTUAL_ENV=/root/.env
Environment=PATH=/root/.env/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
Environment=PULSE_SERVER=unix:/var/run/pulse/native
# No menu, no tmux attach, no colours: nobody is in front of it.
Environment=NO_MENU=1 CALYPSO_NO_ATTACH=1 NO_COLOR=1 OSMO_NO_TERM=1 OSMO_LAUNCH_APPS=0
# Launch options (--grgsm, --op 2, --node 3, --regen...); the DSP is the
# default of start.sh, --grgsm the gr-gsm chain:
#   - persistent: OSMO_BANC_ARGS="--grgsm" in /etc/default/osmo-banc
#   - one-off: launch.sh sets them through systemctl set-environment
EnvironmentFile=-/etc/default/osmo-banc
# --no-attach: the start.sh option (same effect as CALYPSO_NO_ATTACH=1 above).
# It goes here, on the command line - not on ConditionPathExists, which only
# takes a path: « path --no-attach » does not exist, and the unit would be
# skipped without ever starting.
ExecStart=/bin/bash /opt/GSM/osmo-operator/start.sh --no-attach $OSMO_BANC_ARGS
ExecStop=/bin/bash /opt/GSM/osmo-operator/start.sh --stop
# The core follows the bench. osmo-hlr is a separate unit started by run.sh;
# it is stopped here so that it does not keep its subscribers after the bench
# is gone. Leading `-`: an already stopped or missing unit must not fail the
```

```
# stop. osmo-egprs-web is not named here: the dashboard stays online by
# default, bench stopped included (see osmo-egprs-web.service).
ExecStop=-/usr/bin/systemctl stop osmo-hlr.service
# The desktop widget follows the bench. osmo-fft-snap.py reads /psd and the
# mobile.log of the run: a new bench means new paths under /run/user/0 and a
# new log. It is restarted with the bench, never required - leading `-` and
# --no-block: the widget is an ornament, it must neither fail nor delay the
# start. try-restart does nothing if the unit is absent (image without desktop).
ExecStartPost=-/usr/bin/systemctl --no-block try-restart osmo-fft-snap.service
# osmo-wallpaper is not restarted here: the background stays the raw
# illustration (/usr/share/backgrounds/gsm-lab-wallpaper.jpg).
# QEMU + Calypso + mobile attach: several minutes on a VM.
TimeoutStartSec=900
# At machine shutdown systemd waits this long before killing what is left. The
# stack stops in a few seconds; past 30 s it is stuck and SIGKILL is the right
# answer.
TimeoutStopSec=30
# At stop, whatever start.sh --stop did not tear down is terminated by systemd
# (SIGTERM then SIGKILL on the cgroup): nothing left from a previous run.
KillMode=control-group
StandardOutput=journal
StandardError=journal
SyslogIdentifier=osmo-banc

# ── Taking over ──────────────────────────────────────────────────────────
#   tmux attach -t calypso        (root; Ctrl-b d to detach)
#   journalctl -u osmo-banc -f    (the output of start.sh / run.sh)
# The tmux socket is root's default one (/tmp/tmux-0/default): PrivateTmp is
# NOT enabled here, otherwise the session would be invisible outside the service.

[Install]
WantedBy=multi-user.target
```

3.129 /opt/GSM/osmo-operator/services/osmo-bts-trx.service

1110 octets, 34 lignes → 34 lignes

```
[Unit]
Description=Osmocom GSM Base Transceiver Station (BTS)
# Network and loopback interface must be ready.
# network.target only, not network-online.target: the latter can cost up to
# 2 min at boot (systemd-networkd-wait-online) on a bench whose interfaces
# (apn0, the tun/veth of the packet core) are never "online" in that sense and
# which often boots without a cable. The interfaces are configured; the default
# route may arrive later.
After=network.target
# When osmo-bsc runs on the same machine, uncomment the next line
# After=osmo-bsc.service

[Service]
Type=simple
# Adjust the user (e.g. root or the current user 'nirvana')
User=root
Group=root

# Path to the executable (check with 'which osmo-bts-trx')
# -c selects the configuration file
# -r 1 enables real-time priority when the kernel allows it
ExecStart=/usr/bin/osmo-bts-trx -c /etc/osmocom/osmo-bts-trx.cfg -r 1

# Automatic restart on crash (radio stability)
Restart=always
RestartSec=5
```

```
# Resource limits to avoid system "hangs"
LimitNOFILE=65535
CPUSchedulingPolicy=rr
CPUSchedulingPriority=1

[Install]
WantedBy=multi-user.target
```

3.130 /opt/GSM/osmo-operator/services/osmo-dsp-rom-max.service

833 octets, 22 lignes → 22 lignes

```
[Unit]
Description=osmo-operator - workflow « la ROM fait le maximum » : profil ROM max, banc DSP, echelle de tests,
 ↪  bilan ROM/hote
Documentation=file:/opt/GSM/osmo-operator/scripts/dsp-rom-max.sh
After=network.target
ConditionPathExists=/opt/GSM/osmo-operator/scripts/dsp-rom-max.sh

[Service]
# On demand (systemctl start): a test campaign has no business at boot. The
# ROM max profile itself lives in /etc/default/osmo-banc and applies at every
# start of osmo-banc.
Type=oneshot
WorkingDirectory=/opt/GSM/osmo-operator
Environment=HOME=/root NO_COLOR=1
EnvironmentFile=-/etc/default/osmo-dsp-rom-max
ExecStart=/bin/bash /opt/GSM/osmo-operator/scripts/dsp-rom-max.sh run $DSP_ROM_MAX_ARGS
TimeoutStartSec=3600
StandardOutput=journal
StandardError=journal
SyslogIdentifier=osmo-dsp-rom-max

[Install]
WantedBy=multi-user.target
```

3.131 /opt/GSM/osmo-operator/services/osmo-egprs-dsp.service

1323 octets, 31 lignes → 31 lignes

```
[Unit]
Description=osmo-operator - workflow EGPRS : cellule paquet virtuelle + modem GPRS, sur le banc DSP (ROM TI)
Documentation=file:/opt/GSM/osmo-operator/scripts/egprs-dsp.sh
# The bench (core, BTS 0 on the C54x, SGSN/GGSN) must be there: Requires=, not
# only Wants= - without it this workflow has nothing to attach to.
Requires=osmo-banc.service
After=osmo-banc.service network.target
ConditionPathExists=/opt/GSM/osmo-operator/scripts/egprs-dsp.sh

[Service]
# Oneshot + RemainAfterExit: the script starts the cell and the modem as
# transient units (systemd-run: osmo-bts-virtual-gprs, osmo-pcu-gprs,
# osmo-virtphy-gprs, osmo-modem-gprs), then exits. The unit stays « active
# (exited) »; ExecStop tears them down.
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/opt/GSM/osmo-operator
Environment=HOME=/root NO_COLOR=1 EGPRS_BANC=1
# EGPRS_L1=auto|virtphy|dsp, EGPRS_IMSI..., OSMOCOM_CFG: set them here.
EnvironmentFile=-/etc/default/osmo-egprs-dsp
ExecStart=/bin/bash /opt/GSM/osmo-operator/scripts/egprs-dsp.sh start
ExecStop=/bin/bash /opt/GSM/osmo-operator/scripts/egprs-dsp.sh stop
```

```
# The bench is already up (Requires); here: BTS 2, PCU, attach, PDP.
TimeoutStartSec=600
TimeoutStopSec=30
StandardOutput=journal
StandardError=journal
SyslogIdentifier=osmo-egprs-dsp

[Install]
WantedBy=multi-user.target
```

3.132 /opt/GSM/osmo-operator/services/osmo-egprs-web-install.service

2040 octets, 42 lignes → 42 lignes

```
[Unit]
# Runs install-web-service.sh at container boot, not at image build.
#
# Why not a `RUN` in the Dockerfile. Two reasons, each sufficient:
#   1. the script ends with `systemctl daemon-reload` + `systemctl restart`;
#      during `docker build` there is no systemd (« System has not been
#      booted with systemd »), and the script runs under `set -eu`: the build
#      would fail;
#   2. it generates a self-signed TLS certificate. A private key created at
#      build time would be identical for everyone pulling the image — anyone
#      could impersonate the console. It is therefore generated once per
#      container, at first boot.
#
# No `Before=osmo-egprs-web.service` — deliberate. The script ends with
# `systemctl restart osmo-egprs-web`: ordering this unit before the service
# would create a cycle (systemctl would wait on a job waiting on this unit).
# Accepted consequence, visible in the journal: at the very first boot the
# dashboard starts once without TLS (« HTTPS non arme : certificat absent »),
# then this unit installs the certificate and restarts it with TLS. At later
# boots the certificate is already there (the script is idempotent, 30-day
# guard) and there is no second start.
#
# ⚠ The certificate lives in /opt/GSM/osmo-egprs-web/tls, inside the container:
# it does not survive a `docker rm` (containers are started with --rm). This is
# intended — it is regenerated at the next boot, with the current IPs in the SAN.
Description=Installe/rafraichit le dashboard osmo-egprs-web (runtime, unit, certificat TLS)
Documentation=file:/opt/GSM/osmo-egprs-web/install-web-service.sh
After=network.target local-fs.target
ConditionPathIsDirectory=/opt/GSM/osmo-egprs-web

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/bin/bash /opt/GSM/osmo-egprs-web/install-web-service.sh
# The script may run an `npm install`: generous, but bounded.
TimeoutStartSec=600
StandardOutput=journal
StandardError=journal
SyslogIdentifier=osmo-egprs-web-install

[Install]
WantedBy=multi-user.target
```

3.133 /opt/GSM/osmo-operator/services/osmo-egprs-web.service

3718 octets, 67 lignes → 67 lignes

```
[Unit]
Description=osmo-nitb-for-calypso Operations Console (web dashboard)
Documentation=https://github.com/bbaranoff/osmo-egprs-web
After=network.target
# ── The dashboard stays up independently of the bench ──────────────────
# No PartOf=osmo-banc.service: the dashboard stays online by default, bench
# stopped included. To stop it with the bench: CALYPSO_DASHBOARD_SUIT_BANC=1
# (startup_scripts/10-identite-noeud.sh); for systemd, add
# PartOf=osmo-banc.service (osmo-banc already has Wants= and After= towards
# this unit, so do not add After=osmo-banc.service here: it would be a cycle).

[Service]
Type=simple
WorkingDirectory=/opt/GSM/osmo-egprs-web
# TLS takes port 80. Plain HTTP goes to 8080: the two listeners cannot share a
# port, and removing plain HTTP would deprive `curl http://localhost` of any
# diagnostic entry point.
Environment=HTTP_PORT=8080
Environment=OSMO_NATIVE=1
Environment=OSMO_OP_IDS=1
# ⚠ Do not remove: capture interface of the dashboard (GSMTAP/SCTP). Since
# install-web-service.sh reinstalls the unit from this file (the Dockerfile
# copies it to /opt/GSM/osmo-egprs-web/, see Dockerfile.run), this is the single
# source. Without CAP_IFACE the capture falls back to a default and the traffic
# tab stays silent — no error message.
Environment=CAP_IFACE=any
Environment=PULSE_SERVER=unix:/var/run/pulse/native
Environment=AUDIO_SOURCE=gsm_audio.monitor
# --- TLS ---------------------------------------------------------------
# server.js only arms the HTTPS listener if both files below exist; otherwise it
# logs « HTTPS non arme : certificat absent » and stays in plain HTTP.
# They are installed by install-web-service.sh (self-signed, never in the image:
# a private key in an image would be shared by everyone pulling it).
# The same request handler and the same WebSocket instance serve both ports --
# one entry is added, nothing is moved: http:// keeps working.
# ⚠ TLS on port 80, not 443: the URL must be written `https://<ip>:80`
# explicitly (a browser reading `https://<ip>` would go to 443, closed), and
# `http://<ip>` no longer answers in plain HTTP — TLS listens there.
Environment=HTTPS_PORT=80
# ⚠ /etc/osmo-web-tls, not /opt/GSM/osmo-egprs-web/tls. This is where
# install-web-service.sh writes the certificate (TLS_DIR), and it is also the
# default of server.js. Another path would leave the HTTPS listener unarmed and
# the dashboard in plain HTTP; a direct consequence is that the microphone
# button is then refused by the browser (navigator.mediaDevices does not exist
# outside a secure context).
Environment=TLS_CERT=/etc/osmo-web-tls/cert.pem
Environment=TLS_KEY=/etc/osmo-web-tls/key.pem
# ── The node path is not the same everywhere ───────────────────────────
# On the host node comes from the nodesource package (/usr/bin/node; build-iso.sh
# removes the /usr/local/bin/node,npm,npx links, which would dangle). In the
# osmo-operator-* containers it comes from install-web-service.sh, which unpacks
# /opt/node and links it into /usr/local/bin. A hard-coded path fails on one
# side with status=203/EXEC (restart loop).
#
# /usr/bin/env resolves node through the PATH, so at either location. The PATH
# is set explicitly just above: systemd's own PATH does not always contain
# /usr/local/bin.
Environment=PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
ExecStart=/usr/bin/env node /opt/GSM/osmo-egprs-web/server.js --verbose
Restart=on-failure
RestartSec=3
StandardOutput=journal
StandardError=journal
SyslogIdentifier=osmo-egprs-web

[Install]
```

```
WantedBy=multi-user.target
```

3.134 /opt/GSM/osmo-operator/services/osmo-lte.service

2259 octets, 47 lignes → 47 lignes

```
[Unit]
Description=osmo-operator - la 4G du banc (Open5GS + srsENB + srsUE ZeroMQ, via osmo-lte)
Documentation=file:/opt/GSM/osmo-operator/tools/osmo-lte.sh
# The 4G as a unit, like osmo-banc: a state, a journal, a clean stop, and a
# desktop icon (data/desktop/osmo-lte.desktop -> « osmo-lte toggle ») that
# toggles it.
# ExecStart = osmo-lte start: the Open5GS core (osmo-epc, subscribers restored
# when needed), then srsENB, then srsUE in the ue1 netns - the PPP of the pmOS
# phone ends up there (osmo-pmos-setup).
# The core needs MongoDB (subscribers): pulled in, not required.
After=network.target mongod.service
Wants=mongod.service
ConditionPathExists=/opt/GSM/osmo-operator/tools/osmo-lte.sh
# No srsRAN (interstp hub, arm64): the unit is skipped, not failed.
ConditionPathExists=/usr/local/bin/srsenb
# Installed, not enabled (same rule as osmo-banc):
#   systemctl start osmo-lte            for one session (or the icon)
#   systemctl enable --now osmo-lte     at every boot
# At build time: OSMO_ISO_LTE=1 enables the unit in the image.

[Service]
# oneshot + RemainAfterExit: osmo-lte start launches eNB/UE with setsid and the
# Open5GS daemons with -D, then exits. With Type=simple systemd would take that
# exit for the end of the service and kill the cgroup - the whole 4G.
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/opt/GSM/osmo-operator
Environment=HOME=/root USER=root LOGNAME=root
Environment=PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
# Persistent settings (OSMO_LTE_SRATE, OSMO_LTE_NETNS, SRS_WAN...):
EnvironmentFile=-/etc/default/osmo-lte
ExecStart=/bin/bash /opt/GSM/osmo-operator/tools/osmo-lte.sh start
# Stop takes everything down: eNB and UE (osmo-lte stop), then the core
# (osmo-epc stop) - by hand, « osmo-lte stop » leaves the core running.
ExecStop=-/bin/bash /opt/GSM/osmo-operator/tools/osmo-lte.sh stop
ExecStop=-/bin/bash /opt/GSM/osmo-operator/tools/osmo-epc.sh stop
# osmo-lte start waits up to 30 s for the UE attach; the core comes before.
TimeoutStartSec=120
TimeoutStopSec=30
# Whatever the two stops did not tear down is terminated by systemd.
KillMode=control-group
StandardOutput=journal
StandardError=journal
SyslogIdentifier=osmo-lte

[Install]
WantedBy=multi-user.target
```

3.135 /opt/GSM/osmo-operator/services/osmo-multi.service

3650 octets, 67 lignes → 67 lignes

```
[Unit]
Description=osmo-operator - banc multi-operateur (start-multi.sh : natif + conteneurs + inter-STP)
Documentation=file:/opt/GSM/osmo-operator/README.md
# The multi-operator bench connects to the native one, it does not start it:
```

```
# osmo-banc holds it.
# Wants=, not Requires=: start-multi.sh applies « one click = a fresh bench »
# and runs `systemctl stop osmo-banc` at every launch (tout_arreter), then
# restarts it (lancer_natif_si_absent). Requires= would propagate that explicit
# stop to this unit, in the middle of its own ExecStart. Wants= pulls the native
# bench in at boot in the same order (After= below) without propagating the
# stop. The guarantee « no native bench = one ASP out of three » is held by
# start-multi.sh, which restarts osmo-banc when needed.
Wants=osmo-banc.service
# network.target only, not network-online.target: the latter can cost up to
# 2 min at boot (systemd-networkd-wait-online) on a bench whose interfaces
# (apn0, the tun/veth of the packet core) are never "online" in that sense and
# which often boots without a cable. The interfaces are configured; the default
# route may arrive later.
After=osmo-banc.service docker.service network.target
Wants=docker.service
ConditionPathExists=/opt/GSM/osmo-operator/launch/start-multi.sh
# ── And the topology must exist ──────────────────────────────────────
# On a fresh ISO neither docker nor the osmocom-run image are in the squashfs:
# they come with the SS7 supplement (addition.sh), which also writes this file.
# An unmet condition is not a failure for systemd: the unit is skipped silently
# and `systemctl --failed` stays empty (the Conky names it). The condition holds
# at boot as for a manual `systemctl start`: without the topology there is
# nothing to bring up.
ConditionPathExists=/etc/osmocom/osmo-multi.conf
# Neither this unit nor osmo-banc is enabled by default. Both are installed by
# the ISO (iso_modules/82-services.sh) and by addition.sh; the operator starts
# the bench - not the boot, on a configuration not yet chosen.
#   systemctl start osmo-multi          for one session
#   systemctl enable --now osmo-multi   to start it at every boot
# At build time: OSMO_ISO_MULTI=1 (or --multi) enables the unit in the image.

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/opt/GSM/osmo-operator
Environment=HOME=/root USER=root LOGNAME=root
Environment=VIRTUAL_ENV=/root/.env
Environment=PATH=/root/.env/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
# OSMO_NO_TERM: start-multi.sh must neither open a terminal emulator nor hold a
# window. OSMO_BANC_SERVICE: the native bench is a systemd unit, start-multi.sh
# (re)starts it through systemctl and not through the desktop shortcut.
Environment=NO_COLOR=1 OSMO_NO_TERM=1 OSMO_BANC_SERVICE=osmo-banc.service
EnvironmentFile=-/etc/default/osmo-multi
ExecStart=/bin/bash /opt/GSM/osmo-operator/launch/start-multi.sh $OSMO_MULTI_ARGS
# --stop only stops the containers and the hub: the native bench stays with
# osmo-banc.
ExecStop=/bin/bash /opt/GSM/osmo-operator/launch/start-multi.sh --stop
TimeoutStartSec=1200
# At machine shutdown systemd waits this long before killing what is left. The
# stack stops in a few seconds; past 30 s it is stuck and SIGKILL is the right
# answer.
TimeoutStopSec=30
KillMode=control-group
StandardOutput=journal
StandardError=journal
SyslogIdentifier=osmo-multi

# Taking over: tmux attach -t calypso (native); docker logs -f osmo-op2
# (containers); journalctl -u osmo-multi -f.

[Install]
WantedBy=multi-user.target
```

3.136 /opt/GSM/osmo-operator/services/osmo-reglages.service

690 octets, 20 lignes → 20 lignes

```
[Unit]
Description=Page web de reglage de la pile Osmocom (http://127.0.0.1:8090/)
After=network.target
# Online by default, independent of the bench: globals.conf and the config
# files can be edited with the bench stopped; the Daemons/State tabs simply say
# that no daemon answers. 127.0.0.1 only: the VTY console of the page executes
# anything (see tools/osmo-reglages.py).

[Service]
Type=simple
WorkingDirectory=/opt/GSM/osmo-operator
ExecStart=/usr/bin/python3 /opt/GSM/osmo-operator/tools/osmo-reglages.py --port 8090 --bind 127.0.0.1
Restart=on-failure
RestartSec=3
StandardOutput=journal
StandardError=journal
SyslogIdentifier=osmo-reglages

[Install]
WantedBy=multi-user.target
```

3.137 /opt/GSM/osmo-operator/services/osmo-supplements.service

816 octets, 21 lignes → 21 lignes

```
[Unit]
Description=osmo-operator - les supplements choisis dans l installeur (premier demarrage)
Documentation=file:/opt/GSM/osmo-operator/tools/osmo-supplements-firstboot.sh
# Enabled by the Calamares job osmosupplements when a box of the « Supplements »
# page is ticked; disables itself once the supplements are installed.
Wants=network-online.target
After=network-online.target
ConditionPathExists=/etc/osmo-supplements

[Service]
# simple, not oneshot: a oneshot in multi-user.target would hold the boot (and
# the login screen) until the end - one hour for the operator image.
Type=simple
Environment=HOME=/root
ExecStart=/bin/bash /opt/GSM/osmo-operator/tools/osmo-supplements-firstboot.sh
StandardOutput=journal
StandardError=journal
SyslogIdentifier=osmo-supplements

[Install]
WantedBy=multi-user.target
```

3.138 /opt/GSM/osmo-operator/tools/vty-connect.exp

756 octets, 30 lignes → 30 lignes

```
#!/usr/bin/env expect

set container [lindex $argv 0]
set port [lindex $argv 1]
set vty_type [lindex $argv 2]
# On récupère l'IP du groupe, par défaut 127.0.0.1
```

```
set target_ip [lindex $argv 3]
if {$target_ip == ""} { set target_ip "127.0.0.1" }

# On se connecte à l'IP spécifique du groupe Baseband
spawn sudo docker exec -ti $container telnet $target_ip $port

expect {
    "Connection refused" { exit 1 }
    "Connected to" { }
    ">"
}

# La suite du script reste identique pour activer les logs automatiques
send "enable\r"
expect "#"
send "logging enable\r"
expect "#"
send "logging level set-all notice\r"
expect "#"
send "logging filter all 1\r"
expect "#"

puts "\n--- CONSOLE VTY ACTIVE : $vty_type ($target_ip:$port) ---"
interact
```

3.139 /opt/GSM/osmo-operator/tools/softsim/comp128.rb

7307 octets, 138 lignes → 138 lignes

```
# frozen_string_literal: true
# tools/softsim/comp128.rb - COMP128 v1 (A3/A8 de GSM) en Ruby.
#
# Port de libosmocore src/gsm/comp128.c (GPL-2.0+, (C) 2009 Sylvain Munaut),
# lui-meme derive du document GSM 10-1617-01 (1988) et corrige sur trois
# coquilles connues. Valide octet pour octet contre le C (voir tests).
#
#   sres, kc = Comp128.v1(ki_bytes, rand_bytes)   # String binaire, 16 octets chacun
module Comp128
  TABLE_0 = [
    102, 177, 186, 162,   2, 156, 112,  75,  55,  25,   8,  12, 251, 193, 246, 188,
    109, 213, 151,  53,  42,  79, 191, 115, 233, 242, 164, 223, 209, 148, 108, 161,
    252,  37, 244,  47,  64, 211,   6, 237, 185, 160, 139, 113,  76, 138,  59,  70,
     67,  26,  13, 157,  63, 179, 221,  30, 214,  36, 166,  69, 152, 124, 207, 116,
    247, 194,  41,  84,  71,   1,  49,  14,  95,  35, 169,  21,  96,  78, 215, 225,
    182, 243,  28,  92, 201, 118,   4,  74, 248, 128,  17,  11, 146, 132, 245,  48,
    149,  90, 120,  39,  87, 230, 106, 232, 175,  19, 126, 190, 202, 141, 137, 176,
    250,  27, 101,  40, 219, 227,  58,  20,  51, 178,  98, 216, 140,  22,  32, 121,
     61, 103, 203,  72,  29, 110,  85, 212, 180, 204, 150, 183,  15,  66, 172, 196,
     56, 197, 158,   0, 100,  45, 153,   7, 144, 222, 163, 167,  60, 135, 210, 231,
    174, 165,  38, 249, 224,  34, 220, 229, 217, 208, 241,  68, 206, 189, 125, 255,
    239,  54, 168,  89, 123, 122,  73, 145, 117, 234, 143,  99, 129, 200, 192,  82,
    104, 170, 136, 235,  93,  81, 205, 173, 236,  94, 105,  52,  46, 228, 198,   5,
     57, 254,  97, 155, 142, 133, 199, 171, 187,  50,  65, 181, 127, 107, 147, 226,
    184, 218, 131,  33,  77,  86,  31,  44,  88,  62, 238,  18,  24,  43, 154,  23,
     80, 159, 134, 111,   9, 114,   3,  91,  16, 130,  83,  10, 195, 240, 253, 119,
    177, 102, 162, 186, 156,   2,  75, 112,  25,  55,  12,   8, 193, 251, 188, 246,
    213, 109,  53, 151,  79,  42, 115, 191, 242, 233, 223, 164, 148, 209, 161, 108,
     37, 252,  47, 244, 211,  64, 237,   6, 160, 185, 113, 139, 138,  76,  70,  59,
     26,  67, 157,  13, 179,  63,  30, 221,  36, 214,  69, 166, 124, 152, 116, 207,
    194, 247,  84,  41,   1,  71,  14,  49,  35,  95,  21, 169,  78,  96, 225, 215,
    243, 182,  92,  28, 118, 201,  74,   4, 128, 248,  11,  17, 132, 146,  48, 245,
     90, 149,  39, 120, 230,  87, 232, 106,  19, 175, 190, 126, 141, 202, 176, 137,
     27, 250,  40, 101, 227, 219,  20,  58, 178,  51, 216,  98,  22, 140, 121,  32,
    103,  61,  72, 203, 110,  29, 212,  85, 204, 180, 183, 150,  66,  15, 196, 172,
```

```
  197,  56,   0, 158,  45, 100,   7, 153, 222, 144, 167, 163, 135,  60, 231, 210,
  165, 174, 249,  38,  34, 224, 229, 220, 208, 217,  68, 241, 189, 206, 255, 125,
   54, 239,  89, 168, 122, 123, 145,  73, 234, 117,  99, 143, 200, 129,  82, 192,
  170, 104, 235, 136,  81,  93, 173, 205,  94, 236,  52, 105, 228,  46,   5, 198,
  254,  57, 155,  97, 133, 142, 171, 199,  50, 187, 181,  65, 107, 127, 226, 147,
  218, 184,  33, 131,  86,  77,  44,  31,  62,  88,  18, 238,  43,  24,  23, 154,
  159,  80, 111, 134, 114,   9,  91,   3, 130,  16,  10,  83, 240, 195, 119, 253
].freeze
TABLE_1 = [
   19,  11,  80, 114,  43,   1,  69,  94,  39,  18, 127, 117,  97,   3,  85,  43,
   27, 124,  70,  83,  47,  71,  63,  10,  47,  89,  79,   4,  14,  59,  11,   5,
   35, 107, 103,  68,  21,  86,  36,  91,  85, 126,  32,  50, 109,  94, 120,   6,
   53,  79,  28,  45,  99,  95,  41,  34,  88,  68,  93,  55, 110, 125, 105,  20,
   90,  80,  76,  96,  23,  60,  89,  64, 121,  56,  14,  74, 101,   8,  19,  78,
   76,  66, 104,  46, 111,  50,  32,   3,  39,   0,  58,  25,  92,  22,  18,  51,
   57,  65, 119, 116,  22, 109,   7,  86,  59,  93,  62, 110,  78,  99,  77,  67,
   12, 113,  87,  98, 102,   5,  88,  33,  38,  56,  23,   8,  75,  45,  13,  75,
   95,  63,  28,  49, 123, 120,  20, 112,  44,  30,  15,  98, 106,   2, 103,  29,
   82, 107,  42, 124,  24,  30,  41,  16, 108, 100, 117,  40,  73,  40,   7, 114,
   82, 115,  36, 112,  12, 102, 100,  84,  92,  48,  72,  97,   9,  54,  55,  74,
  113, 123,  17,  26,  53,  58,   4,   9,  69, 122,  21, 118,  42,  60,  27,  73,
  118, 125,  34,  15,  65, 115,  84,  64,  62,  81,  70,   1,  24, 111, 121,  83,
  104,  81,  49, 127,  48, 105,  31,  10,   6,  91,  87,  37,  16,  54, 116, 126,
   31,  38,  13,   0,  72, 106,  77,  61,  26,  67,  46,  29,  96,  37,  61,  52,
  101,  17,  44, 108,  71,  52,  66,  57,  33,  51,  25,  90,   2, 119, 122,  35
].freeze
TABLE_2 = [
   52,  50,  44,   6,  21,  49,  41,  59,  39,  51,  25,  32,  51,  47,  52,  43,
   37,   4,  40,  34,  61,  12,  28,   4,  58,  23,   8,  15,  12,  22,   9,  18,
   55,  10,  33,  35,  50,   1,  43,   3,  57,  13,  62,  14,   7,  42,  44,  59,
   62,  57,  27,   6,   8,  31,  26,  54,  41,  22,  45,  20,  39,   3,  16,  56,
   48,   2,  21,  28,  36,  42,  60,  33,  34,  18,   0,  11,  24,  10,  17,  61,
   29,  14,  45,  26,  55,  46,  11,  17,  54,  46,   9,  24,  30,  60,  32,   0,
   20,  38,   2,  30,  58,  35,   1,  16,  56,  40,  23,  48,  13,  19,  19,  27,
   31,  53,  47,  38,  63,  15,  49,   5,  37,  53,  25,  36,  63,  29,   5,   7
].freeze
TABLE_3 = [
    1,   5,  29,   6,  25,   1,  18,  23,  17,  19,   0,   9,  24,  25,   6,  31,
   28,  20,  24,  30,   4,  27,   3,  13,  15,  16,  14,  18,   4,   3,   8,   9,
   20,   0,  12,  26,  21,   8,  28,   2,  29,   2,  15,   7,  11,  22,  14,  10,
   17,  21,  12,  30,  26,  27,  16,  31,  11,   7,  13,  23,  10,   5,  22,  19
].freeze
TABLE_4 = [
   15,  12,  10,   4,   1,  14,  11,   7,   5,   0,  14,   7,   1,   2,  13,   8,
   10,   3,   4,   9,   6,   0,   3,   2,   5,   6,   8,   9,  11,  13,  15,  12
].freeze
TABLES = [TABLE_0, TABLE_1, TABLE_2, TABLE_3, TABLE_4].freeze

module_function

# x: Array de 32 entiers (octets), modifie en place
def compression_round(x, n, tbl)
  m = 4 - n
  mask = (32 << m) - 1
  (1 << n).times do |i|
    (1 << m).times do |j|
      a = j + i * (2 << m)
      b = a + (1 << m)
      y = (x[a] + (x[b] << 1)) & mask
      z = ((x[a] << 1) + x[b]) & mask
      x[a] = tbl[y]
      x[b] = tbl[z]
    end
  end
end
```

```
  def compression(x)
    5.times { |n| compression_round(x, n, TABLES[n]) }
  end

  def bits_from_bytes(x)
    Array.new(128) { |i| (x[i >> 2] >> (3 - (i & 3))) & 1 }
  end

  def permutation(x, bits)
    16.upto(31) { |k| x[k] = 0 }
    128.times do |i|
      x[(i >> 3) + 16] |= bits[(i * 17) & 127] << (7 - (i & 7))
    end
  end

  # Retourne [sres (4 octets), kc (8 octets)], deux String binaires
  def v1(ki, rand)
    raise ArgumentError, 'ki/rand: 16 octets attendus' unless ki.bytesize == 16 && rand.bytesize == 16
    k = ki.bytes
    x = Array.new(32, 0)
    rand.bytes.each_with_index { |byte, i| x[16 + i] = byte }
    7.times do
      x[0, 16] = k
      compression(x)
      bits = bits_from_bytes(x)
      permutation(x, bits)
    end
    x[0, 16] = k
    compression(x)
    sres = Array.new(4) { |i| (x[2 * i] << 4) | x[2 * i + 1] }
    kc = Array.new(8, 0)
    (0...12).step(2) { |i| kc[i >> 1] = ((x[i + 18] << 6) | (x[i + 19] << 2) | (x[i + 20] >> 2)) & 0xFF }
    kc[6] = ((x[30] << 6) | (x[31] << 2)) & 0xFF
    kc[7] = 0
    [sres.pack('C*'), kc.pack('C*')]
  end
end
```

3.140 /opt/GSM/osmo-operator/tools/softsim/ruby_compat.rb

446 octets, 11 lignes → 11 lignes

```
# tools/softsim/ruby_compat.rb - shim pour softSIM (gitea.osmocom.org/sim-card/softsim)
# sous Ruby >= 3.2 : File.exists? et Fixnum/Bignum ont ete supprimes. Charge avec
#   ruby -r/usr/local/share/softsim/ruby_compat.rb demo_server.rb ...
# Ne modifie pas le depot softSIM.
class File
  def self.exists?(p) = exist?(p)
end
Fixnum = Integer unless defined?(Fixnum)
Bignum = Integer unless defined?(Bignum)
$stdout.sync = true
$stderr.sync = true
```

3.141 /opt/GSM/osmo-operator/wiki/Untitled.ipynb

617 octets, 33 lignes → 33 lignes

```
{
 "cells": [
  {
   "cell_type": "code",
```

```
   "execution_count": null,
   "id": "e5e3c376-b60e-4d06-be7d-69cbd9b51330",
   "metadata": {},
   "outputs": [],
   "source": []
  }
 ],
 "metadata": {
  "kernelspec": {
   "display_name": "Python 3 (ipykernel)",
   "language": "python",
   "name": "python3"
  },
  "language_info": {
   "codemirror_mode": {
    "name": "ipython",
    "version": 3
   },
   "file_extension": ".py",
   "mimetype": "text/x-python",
   "name": "python",
   "nbconvert_exporter": "python",
   "pygments_lexer": "ipython3",
   "version": "3.12.3"
  }
 },
 "nbformat": 4,
 "nbformat_minor": 5
}
```

3.142 /opt/GSM/osmo-operator/wiki/installer-osmo-jusquau-dsp.ipynb

72396 octets, 1661 lignes → 1658 lignes (1 groupes compactés)

```
{
 "cells": [
  {
   "cell_type": "markdown",
   "id": "c00",
   "metadata": {},
   "source": [
    "# Native install on Ubuntu 24.04, up to a call on the Calypso DSP\n",
    "\n",
    "This notebook builds the whole bench on a **fresh Ubuntu 24.04** (desktop or server, x86_64), without\n",
    "Docker. It ends with a verified call in DSP mode (`c54x_exe` + TI mask ROM). It replays the build steps of\n",
    "the `Dockerfile` (stages `base`, `osmo-core`, `bb`, `qemu`, `l1`, and the runtime setup at the end of\n",
    "`osmocom-nitb`), with `sudo make install` in place of the `.deb` cache (`osmo-deb pack`): same result in\n",
    "`/usr/local`, without the package.\n",
    "\n",
    "**Why a notebook and not `install.sh`?** `sudo ./install.sh` installs the apt dependencies, osmocom-bb,\n",
    "gapk and the configs, but **not** the Osmocom core (osmo-stp, msc, hlr, bsc, bts…), qosmo, c54x_exe or the\n",
    "firmware: its last step fails with \"demons absents\" on a stock 24.04. This notebook reuses `install.sh`\n",
    "for the parts it does well (the apt list, extracted from the `Dockerfile`, and the service account) and\n",
    "builds the rest.\n",
    "\n",
    "Rules: **one step = one cell**, followed by its **check** cell. Cells are idempotent, so you can re-run\n",
    "them. `%%bash` cells do not share their environment: each one sources `/opt/GSM/.notebook.env`.\n",
    "\n",
    "| | |\n",
    "|---|---|\n",
    "| OS | Ubuntu 24.04 LTS (noble), x86_64 |\n",
    "| Disk | ~12 GB free under `/opt` |\n",
    "| Time | ~1 h on 8 cores (core 20–30 min, QEMU 10–15 min) |\n",
    "| Network | gitea.osmocom.org, github.com, ftp.freecalypso.org, archive.ubuntu.com |\n",
```

```
   "\n",
   "### Before opening the notebook\n",
   "\n",
   "Ubuntu 24.04 refuses `pip install` outside a venv (PEP 668). Install Jupyter in a venv, and **unlock sudo\n",
  "in the same terminal** before starting it: a notebook cannot answer a sudo password prompt, so a cell that\n",
   "asks for one would hang.\n",
   "\n",
   "```bash\n",
   "sudo apt-get install -y python3-venv git\n",
   "python3 -m venv ~/.venv-jupyter\n",
   "~/.venv-jupyter/bin/pip install jupyterlab\n",
   "sudo -v                      # caches your sudo credentials (15 min by default)\n",
   "~/.venv-jupyter/bin/jupyter lab\n",
   "```\n",
   "\n",
   "For a long run, give your user a temporary `NOPASSWD` rule (`sudo visudo -f /etc/sudoers.d/osmo-nb`) or\n",
   "run the kernel as root. The cells work in both cases.\n",
   "\n",
   "### Recommended: a disposable container\n",
   "\n",
   "Every `make install` writes into `/usr/local`, and the Osmocom libraries installed there shadow any other\n",
   "version on the machine. On a machine that is not a **fresh** Ubuntu 24.04 (an earlier install, other\n",
   "Osmocom builds, packages already half there), prefer a throw-away container: nothing leaks onto the host,\n",
   "and a failed run is deleted in one command.\n",
   "\n",
   "```bash\n",
   "docker run -it --name nb-ubuntu --privileged --shm-size=2g -p 8888:8888 ubuntu:24.04 bash\n",
   "```\n",
   "\n",
   "The full procedure (in the container, stop and resume, testing local changes, unattended run) is in\n",
   "[README-notebook](README-notebook.md).\n",
   "\n",
   "`--privileged` gives the stack what it needs at run time: `/dev/net/tun` (the APN interface) and\n",
   "`NET_ADMIN` (the addressing plan). Step 0 warns when the system does not look fresh.\n",
   "\n",
   "> [2026-10-20] The same installation, generated step by step from the Dockerfile (every `RUN` as a bash
↪  block): [doc/install-from-scratch.md](../doc/install-from-scratch.md) (French, with the Dockerfile's
↪  rationale) and [doc/install-from-scratch.en.md](../doc/install-from-scratch.en.md) (English notes per stage).
↪  Regenerate with `tools/dockerfile-to-install-md.sh`.\n",
   "\n",
   "> [2026-10-23] The launcher is now `./start.sh`. `start-native.sh` is kept as a symlink to it, so the old\n",
   "> name and every command written with it still work.\n"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "c01",
  "metadata": {},
  "source": [
   "## 0. Root, environment, sudo\n",
   "\n",
   "`Dockerfile` stage `base`: `ARG ROOT=/opt/GSM`, `PKG_CONFIG_PATH=/usr/local/lib/pkgconfig`,\n",
   "`LD_LIBRARY_PATH=/usr/local/lib`, `GIT_TERMINAL_PROMPT=0`. They go into a file that every later cell\n",
   "sources, together with `SUDO` (empty when the kernel already runs as root)."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 1,
  "id": "c02",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
```

```
    "Ubuntu 24.04.5 LTS\n",
    "WARNING: Osmocom already present in /usr/local (this is not a fresh Ubuntu 24.04):\n",
    "           /usr/local/bin/osmo-addition-anim\n",
    "           /usr/local/bin/osmo-aka-verify\n",
    "           /usr/local/bin/osmo-arfcn\n",
    "           /usr/local/bin/osmo-auc-gen\n",
    "           /usr/local/bin/osmo-banner\n",
    "         it will be overwritten by this notebook; prefer a disposable ubuntu:24.04 container (see above)\n",
    "export ROOT=/opt/GSM\n",
    "export PKG_CONFIG_PATH=/usr/local/lib/pkgconfig\n",
    "export LD_LIBRARY_PATH=/usr/local/lib\n",
    "export GIT_TERMINAL_PROMPT=0\n",
    "export DEBIAN_FRONTEND=noninteractive\n",
    "SUDO=\"\"\n"
   ]
  }
 ],
 "source": [
  "%%bash\n",
  "set -e\n",
  ". /etc/os-release; echo \"$PRETTY_NAME\"\n",
  "[ \"${ID:-}\" = ubuntu ] && [ \"${VERSION_ID:-}\" = \"24.04\" ] \\\n",
  "  || echo \"WARNING: this notebook targets Ubuntu 24.04, found ${ID:-?} ${VERSION_ID:-?}: prefer a disposable
↪ ubuntu:24.04 container (see above)\"\n",
  "# Not a fresh system? make install writes into /usr/local: an earlier Osmocom there gets overwritten or mixed
↪ in\n",
  "prev=\"$(ls -d /usr/local/lib/libosmo* /usr/local/bin/osmo-* 2>/dev/null | head -5)\"\n",
  "[ -z \"$prev\" ] || { echo \"WARNING: Osmocom already present in /usr/local (this is not a fresh Ubuntu
↪ 24.04):\"\n",
  "                     echo \"$prev\" | sed 's/^/           /'\n",
  "                     echo \"         it will be overwritten by this notebook; prefer a disposable ubuntu:24.04
↪ container (see above)\"; }\n",
  "if [ \"$(id -u)\" = 0 ]; then SUDO=\"\"; else SUDO=\"sudo -n\"; fi\n",
  "$SUDO true || { echo \"sudo needs a password: run 'sudo -v' in the terminal that started Jupyter, then
↪ re-run\"; exit 1; }\n",
  "$SUDO mkdir -p /opt/GSM\n",
  "$SUDO chown \"$(id -un):$(id -gn)\" /opt/GSM\n",
  "cat > /opt/GSM/.notebook.env <<EOF\n",
  "export ROOT=/opt/GSM\n",
  "export PKG_CONFIG_PATH=/usr/local/lib/pkgconfig\n",
  "export LD_LIBRARY_PATH=/usr/local/lib\n",
  "export GIT_TERMINAL_PROMPT=0\n",
  "export DEBIAN_FRONTEND=noninteractive\n",
  "SUDO=\"$SUDO\"\n",
  "EOF\n",
  "cat /opt/GSM/.notebook.env"
 ]
},
{
 "cell_type": "code",
 "execution_count": 2,
 "id": "c03",
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "OK: /opt/GSM writable\n",
    "OK: sudo without prompt\n",
    "/dev/nvme0n1p2  466G   98G  345G  23% /\n"
   ]
  }
 ],
 "source": [
  "%%bash\n",
```

```
   ". /opt/GSM/.notebook.env\n",
   "# Check\n",
   "test -w \"$ROOT\" && echo \"OK: $ROOT writable\" && $SUDO true && echo \"OK: sudo without prompt\"\n",
   "df -h /opt | tail -1"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "c04",
  "metadata": {},
  "source": [
   "## 1. osmo-operator, then the apt dependencies\n",
   "\n",
   "The repository is cloned first: it carries the patches, the configs, the bridge (`pont/`) and\n",
   "`install.sh`. `install.sh --only deps` installs the apt list **extracted from the `Dockerfile`**, so the\n",
   "two can never diverge. The `debconf` line answers the wireshark-common question in advance (`Dockerfile`,\n",
   "stage `base`), and `DEBIAN_FRONTEND=noninteractive` keeps apt from waiting on a prompt nobody can see."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 3,
  "id": "c05",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "Lecture des listes de paquets…\n",
     "Construction de l'arbre des dépendances…\n",
     "Lecture des informations d'état…\n",
     "git est déjà la version la plus récente (1:2.43.0-1ubuntu7.4).\n",
     "ca-certificates est déjà la version la plus récente (20260816ubuntu0.24.04.1).\n",
     "netcat-openbsd est déjà la version la plus récente (1.226-1ubuntu2).\n",
     "0 mis à jour, 0 nouvellement installés, 0 à enlever et 0 non mis à jour.\n",
     "[ .. ] Dependances systeme (apt)\n",
     "[SKIP] Dependances systeme (apt) (deja installe)\n",
     "\n",
     "0 ok · 1 ignores · 0 echecs\n",
     "\n",
     "  verifier   ./install.sh --check\n",
     "  lancer     ./start-native.sh --list   puis   sudo ./start-native.sh\n",
     "\n",
     "[ .. ] Utilisateur et groupe osmocom\n",
     "[SKIP] Utilisateur et groupe osmocom (deja installe)\n",
     "\n",
     "0 ok · 1 ignores · 0 echecs\n",
     "\n",
     "  verifier   ./install.sh --check\n",
     "  lancer     ./start-native.sh --list   puis   sudo ./start-native.sh\n",
     "\n"
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "set -e\n",
   "$SUDO apt-get update -qq\n",
   "$SUDO apt-get install -y --no-install-recommends git ca-certificates netcat-openbsd   # nc: VTY checks of
↪  step 13\n",
   "cd \"$ROOT\"; [ -d osmo-operator ] || git clone https://github.com/bbaranoff/osmo-operator\n",
   "echo 'wireshark-common wireshark-common/install-setuid boolean true' | $SUDO debconf-set-selections\n",
   "cd \"$ROOT/osmo-operator\"\n",
   "$SUDO env DEBIAN_FRONTEND=noninteractive GSM_ROOT=\"$ROOT\" ./install.sh --only deps\n",
```

```
   "$SUDO env GSM_ROOT=\"$ROOT\" ./install.sh --only user"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 4,
  "id": "c06",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "ok      build-essential\n",
     "ok      libtalloc-dev\n",
     "ok      libsctp-dev\n",
     "ok      libdbi-dev\n",
     "ok      libdbd-sqlite3\n",
     "ok      libsqlite3-dev\n",
     "ok      libglib2.0-dev\n",
     "ok      libpixman-1-dev\n",
     "ok      ninja-build\n",
     "ok      python3-numpy\n",
     "ok      socat\n",
     "ok      asterisk\n",
     "ok      tmux\n",
     "ok      tshark\n",
     "ok      liburing-dev\n",
     "ok      libgsm1-dev\n",
     "osmocom:x:997:1004::/var/lib/osmocom:/sbin/nologin\n"
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "# Check: witness packages (sqlite/dbi: osmo-hlr's configure needs them), and the service account.\n",
   "# Fails if anything is missing, so the run stops here and not three steps later.\n",
   "miss=0\n",
   "for p in build-essential libtalloc-dev libsctp-dev libdbi-dev libdbd-sqlite3 libsqlite3-dev libglib2.0-dev
↪  \\\n",
   "        libpixman-1-dev ninja-build python3-numpy socat asterisk tmux tshark liburing-dev libgsm1-dev; do\n",
   "  if dpkg-query -W -f='${db:Status-Status}\\n' \"$p\" 2>/dev/null | grep -qx installed; then echo \"ok
↪  $p\"\n",
   "  else echo \"MISSING $p\"; miss=1; fi\n",
   "done\n",
   "getent passwd osmocom || { echo \"MISSING account osmocom\"; miss=1; }\n",
   "exit $miss"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "c07",
  "metadata": {},
  "source": [
   "### 1b. (optional) GNU Radio build dependencies\n",
   "\n",
   "Only for the gr-gsm chain (`--grgsm`), `si_bridge.py` and the FFT views. **Not needed for the DSP chain**:\n",
   "the bridge loads libosmocoding through ctypes and does not import GNU Radio."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 5,
  "id": "c08",
  "metadata": {},
```

```
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "left commented: GNU Radio is not required for the DSP chain\n"
   ]
  }
 ],
 "source": [
  "%%bash\n",
  ". /opt/GSM/.notebook.env\n",
  "# Dockerfile, end of the apt block: apt-fast build-dep -y gnuradio (needs deb-src)\n",
  "# $SUDO sed -i 's/^Types: deb$/Types: deb deb-src/' /etc/apt/sources.list.d/ubuntu.sources\n",
  "# $SUDO apt-get update && $SUDO apt-get build-dep -y gnuradio\n",
  "echo \"left commented: GNU Radio is not required for the DSP chain\""
 ]
},
{
 "cell_type": "markdown",
 "id": "c09",
 "metadata": {},
 "source": [
  "## 2. Non-root capture\n",
  "\n",
  "`Dockerfile` stage `base`: `dumpcap` gets the capture capabilities, so Wireshark works without root."
 ]
},
{
 "cell_type": "code",
 "execution_count": 6,
 "id": "c10",
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "/usr/bin/dumpcap cap_net_admin,cap_net_raw=eip\n"
   ]
  }
 ],
 "source": [
  "%%bash\n",
  ". /opt/GSM/.notebook.env\n",
  "$SUDO setcap cap_net_raw,cap_net_admin+eip \"$(command -v dumpcap)\"\n",
  "getcap \"$(command -v dumpcap)\""
 ]
},
{
 "cell_type": "markdown",
 "id": "c11",
 "metadata": {},
 "source": [
  "## 3. The Osmocom core (stage `osmo-core`)\n",
  "\n",
  "Same repositories, versions and configure options as the `Dockerfile` loop. The three\n",
  "`force-rand-toy` patches (osmo-bts, osmo-hlr, osmo-msc) are applied **before** `configure`, as in the\n",
 "image: RAND becomes deterministic, so authentication is reproducible from one start to the next, which the\n",
  "test suite relies on.\n",
  "\n",
  "`osmo-trx` (IPC patches) is skipped: the DSP and gr-gsm chains use `osmo-bts-trx` with the bridge, not\n",
  "`osmo-trx`. Long: 20–30 min.\n",
  "\n",
  "Each repository's full output (clone, autoreconf, configure, make, install) goes to\n",
  "`$ROOT/logs/<name>.log`; the cell prints one line per repository, or the end of the log on failure."
```

```
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 7,
   "id": "c12",
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "ok     libosmocore 1.12.1\n",
      "ok     libosmo-netif 1.7.0\n",
      "ok     libosmo-abis 2.1.0\n",
      "ok     libosmo-sigtran 2.2.1\n",
      "ok     libsmpp34 1.14.5\n",
      "ok     libgtpnl 1.3.3\n",
      "  patch already applied: osmo-hlr-force-rand-toy.patch\n",
      "ok     osmo-hlr 1.9.2\n",
      "ok     osmo-mgw 1.15.0\n",
      "ok     osmo-ggsn 1.14.0\n",
      "ok     osmo-sgsn 1.13.1\n",
      "  patch already applied: osmo-msc-force-rand-toy.patch\n",
      "ok     osmo-msc 1.15.0\n",
      "ok     osmo-bsc 1.14.0\n",
      "  patch already applied: osmo-bts-force-rand-toy.patch\n",
      "ok     osmo-bts 1.10.0\n",
      "ok     osmo-pcu 1.5.2\n",
      "ok     osmo-sip-connector 1.7.2\n",
      "ok     libosmo-gprs 0.2.1\n"
     ]
    }
   ],
   "source": [
    "%%bash\n",
    ". /opt/GSM/.notebook.env\n",
    "P=\"$ROOT/osmo-operator/patches\"\n",
    "mkdir -p \"$ROOT/logs\"\n",
    "# A patch is applied if it applies in reverse; applicable if it applies forward; otherwise the tree does\n",
    "# not match it, which is an error, never \"already applied\".\n",
    "apply_patch() {   # $1 = patch file\n",
    "  if git apply --reverse --check \"$1\" 2>/dev/null; then echo \"  patch already applied: $(basename
↪  \"$1\")\"\n",
    "  elif git apply --check \"$1\" 2>/dev/null; then git apply \"$1\" && echo \"  patched: $(basename
↪  \"$1\")\"\n",
    "  else echo \"  PATCH DOES NOT APPLY: $(basename \"$1\") (tree $(git describe --tags --always) differs from
↪  what the patch expects)\"; return 1; fi\n",
    "}\n",
    "build() {         # $1 = name (already cloned, checked out and patched)\n",
    "  local name=$1 flags=\"\"\n",
    "  cd \"$ROOT/$name\" || return 1\n",
    "  case \"$name\" in\n",
    "    libosmo-abis) flags=\"--disable-dahdi\" ;;\n",
    "    osmo-msc)     flags=\"--enable-smpp\" ;;\n",
    "    osmo-mgw)     flags=\"--enable-alsa\" ;;\n",
    "    osmo-bts)     flags=\"--enable-virtual --enable-trx\" ;;\n",
    "    osmo-ggsn)    flags=\"--enable-gtp-linux\" ;;\n",
    "  esac\n",
    "  autoreconf -fi && ./configure $flags && make -j\"$(nproc)\" \\\n",
    "    && $SUDO make install && $SUDO ldconfig      # Dockerfile: osmo-deb pack \"$name\" \"$version\" make
↪  install\n",
    "}\n",
    "for repo in \\\n",
    "    libosmocore:1.12.1 libosmo-netif:1.7.0 libosmo-abis:2.1.0 libosmo-sigtran:2.2.1 \\\n",
    "    libsmpp34:1.14.5 libgtpnl:1.3.3 osmo-hlr:1.9.2 osmo-mgw:1.15.0 osmo-ggsn:1.14.0 \\\n",
```

```
   "    osmo-sgsn:1.13.1 osmo-msc:1.15.0 osmo-bsc:1.14.0 osmo-bts:1.10.0 osmo-pcu:1.5.2 \\\n",
   "    osmo-sip-connector:1.7.2 libosmo-gprs:0.2.1; do\n",
   "  name=${repo%%:*}; version=${repo##*:}\n",
   "  case \"$name\" in libosmo*) url=\"https://gitea.osmocom.org/osmocom/$name\" ;;\n",
   "                  *)        url=\"https://gitea.osmocom.org/cellular-infrastructure/$name\" ;; esac\n",
   "  log=\"$ROOT/logs/$name.log\"\n",
   "  # clone first (outside the log redirection) so the patch can be checked against the right version\n",
   "  ( cd \"$ROOT\"; [ -d \"$name\" ] || git clone -q \"$url\"; cd \"$name\" && git checkout -q \"$version\" ) >
↪  \"$log\" 2>&1 \\\n",
   "    || { echo \"FAILED $name (clone/checkout $version)\"; tail -20 \"$log\"; exit 1; }\n",
   "  if [ -f \"$P/$name-force-rand-toy.patch\" ]; then\n",
   "    ( cd \"$ROOT/$name\" && apply_patch \"$P/$name-force-rand-toy.patch\" ) || exit 1\n",
   "  fi\n",
   "  if build \"$name\" >> \"$log\" 2>&1; then\n",
   "    echo \"ok     $name $version\"\n",
   "  else\n",
   "    echo \"FAILED $name $version  (full log: $log)\"; tail -20 \"$log\"; exit 1\n",
   "  fi\n",
   "done"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 8,
  "id": "c13",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "1.12.1\n",
     "1.12.1\n",
     "ok      osmo-stp\n",
     "ok      osmo-hlr\n",
     "ok      osmo-msc\n",
     "ok      osmo-mgw\n",
     "ok      osmo-bsc\n",
     "ok      osmo-bts-trx\n",
     "ok      osmo-pcu\n",
     "ok      osmo-sgsn\n",
     "ok      osmo-ggsn\n",
     "ok      osmo-sip-connector\n"
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "# Check: libraries for c54x_exe (pkg-config libosmocoding) and the daemons install.sh's verify step looks
↪  for.\n",
   "# Fails if anything is missing.\n",
   "miss=0\n",
   "pkg-config --modversion libosmocore libosmocoding || miss=1\n",
   "for b in osmo-stp osmo-hlr osmo-msc osmo-mgw osmo-bsc osmo-bts-trx osmo-pcu osmo-sgsn osmo-ggsn
↪  osmo-sip-connector; do\n",
   "  if command -v \"$b\" >/dev/null; then echo \"ok      $b\"; else echo \"MISSING $b\"; miss=1; fi\n",
   "done\n",
   "exit $miss"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "c14",
  "metadata": {},
  "source": [
```

```
   "## 4. osmo-gapk (before osmocom-bb)\n",
   "\n",
   "The bench's mobile uses `tch-voice io-handler gapk`: osmocom-bb must find libosmogapk at configure time,\n",
   "so gapk comes **first**."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 9,
  "id": "c15",
  "metadata": {},
  "outputs": [
   {
    "name": "stderr",
    "output_type": "stream",
    "text": [
     "configure.ac:7: installing './compile'\n",
     "configure.ac:6: installing './missing'\n",
     "libgsmhr/Makefile.am: installing './depcomp'\n",
     "configure: WARNING: unrecognized options: --enable-alsa\n",
     "configure: WARNING: unrecognized options: --enable-alsa\n"
    ]
   },
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "Making all in include\n",
     "Making all in src\n",
     "  CC       app_osmo_gapk.o\n",
     "  CC       procqueue.lo\n",
     "  CC       pq_format.lo\n",
     "  CC       pq_codec.lo\n",
     "  CC       pq_file.lo\n",
     "  CC       pq_alsa.lo\n",
     "  CC       pq_rtp.lo\n",
     "  CC       pq_ecu.lo\n",
     "  CC       ecu_fr.lo\n",
     "  CC       fmt_ti.lo\n",
     "  CC       fmt_amr.lo\n",
     "  CC       fmt_racal.lo\n",
     "  CC       fmt_rtp_amr.lo\n",
     "  CC       fmt_rtp_efr.lo\n"
    ]
   },
   {
    "name": "stderr",
    "output_type": "stream",
    "text": [
     "ecu_fr.c: In function 'ecu_proc_fr':\n",
     "ecu_fr.c:52:17: warning: 'osmo_ecu_fr_reset' is deprecated: Use generic ECU abstraction layer instead
↪  [-Wdeprecated-declarations]\n",
     "   52 |                 osmo_ecu_fr_reset(state, (uint8_t *) in);\n",
     "      |                 ^~~~~~~~~~~~~~~~~\n",
     "In file included from ecu_fr.c:30:\n",
     "/usr/local/include/osmocom/codec/ecu.h:15:6: note: declared here\n",
     "   15 | void osmo_ecu_fr_reset(struct osmo_ecu_fr_state *state, const uint8_t *frame)\n",
     "      |      ^~~~~~~~~~~~~~~~~\n",
     "ecu_fr.c:73:9: warning: 'osmo_ecu_fr_conceal' is deprecated: Use generic ECU abstraction layer instead
↪  [-Wdeprecated-declarations]\n",
     "   73 |         rc = osmo_ecu_fr_conceal(state, out);\n",
     "      |         ^~\n",
     "/usr/local/include/osmocom/codec/ecu.h:17:5: note: declared here\n",
     "   17 | int osmo_ecu_fr_conceal(struct osmo_ecu_fr_state *state, uint8_t *frame)\n",
     "      |     ^~~~~~~~~~~~~~~~~~\n"
    ]
```

```
   },
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "  CCLD     libosmogapk.la\n",
     "  CCLD     osmo-gapk\n",
     "Making all in tests\n",
     "Making install in include\n",
     " /usr/bin/mkdir -p '/usr/local/include'\n",
     " /usr/bin/mkdir -p '/usr/local/include/osmocom/gapk'\n",
     " /usr/bin/install -c -m 644  osmocom/gapk/get_cycles.h osmocom/gapk/benchmark.h osmocom/gapk/procqueue.h
↪  osmocom/gapk/formats.h osmocom/gapk/codecs.h osmocom/gapk/common.h '/usr/local/include/osmocom/gapk'\n",
     "Making install in src\n",
     " /usr/bin/mkdir -p '/usr/local/lib'\n",
     " /bin/bash ../libtool   --mode=install /usr/bin/install -c   libosmogapk.la '/usr/local/lib'\n",
     "libtool: install: /usr/bin/install -c .libs/libosmogapk.so.0.0.0 /usr/local/lib/libosmogapk.so.0.0.0\n",
     "libtool: install: (cd /usr/local/lib && { ln -s -f libosmogapk.so.0.0.0 libosmogapk.so.0 || { rm -f
↪  libosmogapk.so.0 && ln -s libosmogapk.so.0.0.0 libosmogapk.so.0; }; })\n",
     "libtool: install: (cd /usr/local/lib && { ln -s -f libosmogapk.so.0.0.0 libosmogapk.so || { rm -f
↪  libosmogapk.so && ln -s libosmogapk.so.0.0.0 libosmogapk.so; }; })\n",
     "libtool: install: /usr/bin/install -c .libs/libosmogapk.lai /usr/local/lib/libosmogapk.la\n",
     "libtool: finish:
↪  PATH=\"/root/.local/bin:/root/.env/bin:/usr/local/sbin:/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin:/sbin\"
↪  ldconfig -n /usr/local/lib\n",
     "----------------------------------------------------------------------\n",
     "Libraries have been installed in:\n",
     "   /usr/local/lib\n",
     "\n",
     "If you ever happen to want to link against installed libraries\n",
     "in a given directory, LIBDIR, you must either use libtool, and\n",
     "specify the full pathname of the library, or use the '-LLIBDIR'\n",
     "flag during linking and do at least one of the following:\n",
     "   - add LIBDIR to the 'LD_LIBRARY_PATH' environment variable\n",
     "     during execution\n",
     "   - add LIBDIR to the 'LD_RUN_PATH' environment variable\n",
     "     during linking\n",
     "   - use the '-Wl,-rpath -Wl,LIBDIR' linker flag\n",
     "   - have your system administrator add LIBDIR to '/etc/ld.so.conf'\n",
     "\n",
     "See any operating system documentation about shared libraries for\n",
     "more information, such as the ld(1) and ld.so(8) manual pages.\n",
     "----------------------------------------------------------------------\n",
     " /usr/bin/mkdir -p '/usr/local/bin'\n",
     "  /bin/bash ../libtool   --mode=install /usr/bin/install -c osmo-gapk '/usr/local/bin'\n",
     "libtool: install: /usr/bin/install -c .libs/osmo-gapk /usr/local/bin/osmo-gapk\n",
     "Making install in tests\n",
     " /usr/bin/mkdir -p '/usr/local/lib/pkgconfig'\n",
     " /usr/bin/install -c -m 644 libosmogapk.pc '/usr/local/lib/pkgconfig'\n"
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "set -e\n",
   "cd \"$ROOT\"; [ -d osmo-gapk ] || git clone -q https://gitea.osmocom.org/osmocom/gapk osmo-gapk\n",
   "cd osmo-gapk && autoreconf -fi >/dev/null && ./configure -q --enable-alsa && make -s -j\"$(nproc)\"\n",
   "$SUDO make -s install && $SUDO ldconfig"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 10,
  "id": "c16",
  "metadata": {},
```

```
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "1.1.2.2-beab\n",
    "/usr/local/bin/osmo-gapk\n"
   ]
  }
 ],
 "source": [
  "%%bash\n",
  ". /opt/GSM/.notebook.env\n",
  "# Check\n",
  "pkg-config --modversion libosmogapk && command -v osmo-gapk"
 ]
},
{
 "cell_type": "markdown",
 "id": "c17",
 "metadata": {},
 "source": [
  "## 5. The SMSC: gsup-smsc-proto and sms-coding-utils\n",
  "\n",
  "`proto-smsc-daemon` receives MO SMS from OsmoHLR over GSUP, and `proto-smsc-sendmt` injects MT SMS.\n",
  "`run_modules/22-smsc.sh` starts them: **without them there is no SMS**. Its top-level Makefile forces\n",
  "`DESTDIR=`, so the two sub-directories are installed one by one, as in the `Dockerfile`.\n",
  "`sms-coding-utils` (FreeCalypso) adds the PDU encode/decode tools."
 ]
},
{
 "cell_type": "code",
 "execution_count": 11,
 "id": "c18",
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "C code will be compiled with:\n",
    "  CC=gcc\n",
    "  CFLAGS=-O2\n",
    "Installation directory:\n",
    "  bindir=/usr/local/bin\n",
    "See config.defs output for Osmocom dependencies\n"
   ]
  },
  {
   "name": "stderr",
   "output_type": "stream",
   "text": [
    "make[1]: warning: -j32 forced in submake: resetting jobserver mode.\n",
    "make[1]: warning: -j32 forced in submake: resetting jobserver mode.\n"
   ]
  },
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "C code will be compiled with:\n",
    "  CC=gcc\n",
    "  CFLAGS=-O2\n",
    "Installation directory:\n",
    "  bindir=/usr/local/bin\n"
   ]
```

```
  },
  {
   "name": "stderr",
   "output_type": "stream",
   "text": [
    "make[1]: warning: -j32 forced in submake: resetting jobserver mode.\n",  ×4
    "make[1]: warning: -j32 forced in submake: resetting jobserver mode.\n"
   ]
  }
 ],
 "source": [
  "%%bash\n",
  ". /opt/GSM/.notebook.env\n",
  "set -e\n",
  "cd \"$ROOT\"; [ -d gsup-smsc-proto ] || git clone -q https://gitea.osmocom.org/themwi/gsup-smsc-proto\n",
  "cd gsup-smsc-proto && ./configure --with-osmo=/usr/local && make -s -j\"$(nproc)\"\n",
  "for d in daemon sendmt; do $SUDO make -s -C \"$d\" install; done\n",
  "cd \"$ROOT\"\n",
  "if [ ! -d sms-coding-utils-r1 ]; then\n",
  "  wget -q https://www.freecalypso.org/pub/GSM/FreeCalypso/sms-coding-utils-latest.tar.bz2\n",
  "  tar xf sms-coding-utils-latest.tar.bz2\n",
  "fi\n",
  "cd sms-coding-utils-r1 && ./configure && make -s -j\"$(nproc)\" && $SUDO make -s install\n",
  "$SUDO ldconfig"
 ]
},
{
 "cell_type": "code",
 "execution_count": 12,
 "id": "c19",
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "/usr/local/bin/proto-smsc-daemon\n",
    "/usr/local/bin/proto-smsc-sendmt\n",
    "/usr/local/bin/sms-encode-text\n"
   ]
  }
 ],
 "source": [
  "%%bash\n",
  ". /opt/GSM/.notebook.env\n",
  "# Check\n",
  "command -v proto-smsc-daemon proto-smsc-sendmt sms-encode-text"
 ]
},
{
 "cell_type": "markdown",
 "id": "c20",
 "metadata": {},
 "source": [
  "## 6. osmocom-bb (stage `bb`): osmocon, mobile, trxcon\n",
  "\n",
  "Clone, the **three** patches the image applies (`clck-gen-frame-tolerance` for fake_trx,\n",
  "`plmn-a6-reinsert` and `sim-plmn-from-bcd` for the mobile's PLMN selection), then `make nofirmware`\n",
  "(host tools only: the firmware comes from step 7). `osmocon` stays in the tree, where `run.sh` and\n",
  "`c54x_exe/run.sh` look for it."
 ]
},
{
 "cell_type": "code",
 "execution_count": 13,
 "id": "c21",
```

```
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "patch already applied: clck-gen-frame-tolerance\n",
    "patch already applied: plmn-a6-reinsert\n",
    "patch already applied: sim-plmn-from-bcd\n",
    "Making all in include\n",
    "Making all in osmocom\n",
    "Making all in bb\n",
    "Making all in common\n",
    "Making all in misc\n",
    "Making all in mobile\n",
    "Making all in modem\n",
    "Making all in src\n",
    "Making all in common\n",
    "  CC       apn.o\n",
    "  CC       apn_fsm.o\n",
    "  CC       gps.o\n",
    "  CC       l1ctl.o\n",
    "  CC       l1l2_interface.o\n",
    "  CC       l1ctl_lapdm_glue.o\n",
    "  CC       logging.o\n",
    "  CC       ms.o\n"
   ]
  },
  {
   "name": "stderr",
   "output_type": "stream",
   "text": [
    "ms.c: In function ‘osmocom_ms_alloc’:\n",
    "ms.c:58:9: warning: ‘lapdm_channel_init’ is deprecated: Use lapdm_channel_init3() instead
↪ [-Wdeprecated-declarations]\n",
    "   58 |         lapdm_channel_init(&ms->lapdm_channel, LAPDM_MODE_MS);\n",
    "      |         ^~~~~~~~~~~~~~~~~~\n",
    "In file included from ../../include/osmocom/bb/common/ms.h:8,\n",
    "                 from ms.c:21:\n",
    "/usr/local/include/osmocom/gsm/lapdm.h:99:6: note: declared here\n",
    "   99 | void lapdm_channel_init(struct lapdm_channel *lc, enum lapdm_mode mode)\n",
    "      |      ^~~~~~~~~~~~~~~~~~\n"
   ]
  },
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "  CC       networks.o\n",
    "  CC       sap_fsm.o\n",
    "  CC       sap_proto.o\n",
    "  CC       sap_interface.o\n",
    "  CC       settings.o\n",
    "  CC       sim.o\n",
    "  CC       subscriber.o\n",
    "  CC       support.o\n",
    "  CC       sysinfo.o\n",
    "  CC       utils.o\n",
    "  CC       vty.o\n",
    "  AR       liblayer23.a\n"
   ]
  },
  {
   "name": "stderr",
   "output_type": "stream",
   "text": [
```

```
  "ar: `u' modifier ignored since `D' is the default (see `U')\n"
 ]
},
{
 "name": "stdout",
 "output_type": "stream",
 "text": [
  "Making all in misc\n",
  "  CC       main.o\n",
  "  CC       app_bcch_scan.o\n",
  "  CC       bcch_scan.o\n",
  "  CCLD     bcch_scan\n",
  "  CC       app_ccch_scan.o\n",
  "  CC       rslms.o\n",
  "  CCLD     ccch_scan\n",
  "  CC       app_echo_test.o\n",
  "  CCLD     echo_test\n",
  "  CC       app_cell_log.o\n",
  "  CC       cell_log.o\n",
  "  CCLD     cell_log\n",
  "  CC       app_cbch_sniff.o\n",
  "  CCLD     cbch_sniff\n",
  "  CC       gsmmap.o\n"
 ]
},
{
 "name": "stderr",
 "output_type": "stream",
 "text": [
  "gsmmap.c:18:2: warning: #warning todo bsic [-Wcpp]\n",
  "   18 | #warning todo bsic\n",
  "      |  ^~~~~~\n"
 ]
},
{
 "name": "stdout",
 "output_type": "stream",
 "text": [
  "  CC       log.o\n",
  "  CCLD     gsmmap\n",
  "Making all in mobile\n",
  "  CC       main.o\n",
  "  CC       app_mobile.o\n",
  "  CC       gsm322.o\n",
  "  CC       gsm480_ss.o\n",
  "  CC       gsm411_sms.o\n",
  "  CC       gsm48_cc.o\n",
  "  CC       gsm44068_gcc_bcc.o\n",
  "  CC       gsm48_mm.o\n",
  "  CC       gsm48_rr.o\n",
  "  CC       gsm414.o\n",
  "  CC       mnccms.o\n",
  "  CC       mncc_sock.o\n",
  "  CC       primitives.o\n",
  "  CC       tch.o\n",
  "  CC       tch_data.o\n",
  "  CC       tch_data_sock.o\n",
  "  CC       tch_voice.o\n",
  "  CC       transaction.o\n",
  "  CC       vty_interface.o\n",
  "  CC       script_nolua.o\n",
  "  CC       gapk_io.o\n",
  "  AR       libmobile.a\n"
 ]
},
{
```

```
 "name": "stderr",
 "output_type": "stream",
 "text": [
  "ar: `u' modifier ignored since `D' is the default (see `U')\n"
 ]
},
{
 "name": "stdout",
 "output_type": "stream",
 "text": [
  "  CCLD     mobile\n",
  "Making all in modem\n",
  "  CC       main.o\n",
  "  CC       app_modem.o\n",
  "  CC       gmm.o\n",
  "  CC       grr.o\n",
  "  CC       llc.o\n",
  "  CC       rlcmac.o\n",
  "  CC       sm.o\n",
  "  CC       sndcp.o\n",
  "  CC       vty.o\n",
  "  CCLD     modem\n",
  "Making all in include\n",
  "Making all in osmocom\n",
  "Making all in bb\n",
  "Making all in virtphy\n",
  "Making all in src\n",
  "  CC       virtphy.o\n",
  "  CC       l1gprs.o\n",
  "  CC       logging.o\n",
  "  CC       gsmtapl1_if.o\n",
  "  CC       l1ctl_sock.o\n",
  "  CC       l1ctl_sap.o\n",
  "  CC       virt_prim_pm.o\n",
  "  CC       virt_prim_fbsb.o\n",
  "  CC       virt_prim_rach.o\n",
  "  CC       virt_prim_data.o\n",
  "  CC       virt_prim_pdch.o\n",
  "  CC       virt_prim_traffic.o\n",
  "  CC       virt_l1_sched_simple.o\n",
  "  CC       virt_l1_model.o\n",
  "  CC       shared/virtual_um.o\n",
  "  CC       shared/osmo_mcast_sock.o\n",
  "  CCLD     virtphy\n"
 ]
},
{
 "name": "stderr",
 "output_type": "stream",
 "text": [
  "install: cannot stat '/opt/GSM/osmocom-bb/src/host/trxcon/src/trxcon': No such file or directory\n"
 ]
},
{
 "ename": "CalledProcessError",
 "evalue": "Command 'b'. /opt/GSM/.notebook.env\\nset -e\\nP=\"$ROOT/osmo-operator/patches\"\\n# [2026-10-23]
↪ pinned like the Dockerfile (OSMOCOM_BB_REF): upstream a0a315e2 moved trxcon to libosmo-trx and out of\\n#
↪ `make nofirmware`, so a clone of master no longer builds src/host/trxcon/src/trxcon\\ncd \"$ROOT\"; [ -d
↪ osmocom-bb ] || { git clone -q https://gitea.osmocom.org/phone-side/osmocom-bb \\\\\\n
↪ && git -C osmocom-bb checkout -q \"${OSMOCOM_BB_REF:-eac4eed1}\"; }\\n# Same three cases as step 3: applied
↪ (reverse applies) / applicable / does not match = error\\nfor p in clck-gen-frame-tolerance plmn-a6-reinsert
↪ sim-plmn-from-bcd; do\\n  f=\"$P/osmocom-bb-$p.patch\"\\n  if git -C osmocom-bb apply --reverse --check
↪ \"$f\" 2>/dev/null; then echo \"patch already applied: $p\"\\n  elif git -C osmocom-bb apply --check \"$f\"
↪ 2>/dev/null; then git -C osmocom-bb apply \"$f\" && echo \"patched: $p\"\\n  else echo \"PATCH DOES NOT APPLY:
↪ osmocom-bb-$p.↩
```

```
patch (tree $(git -C osmocom-bb describe --tags --always) differs from what the patch expects)\"; exit 1;
 ↪  fi\\ndone\\ncd osmocom-bb/src && make -s nofirmware\\nfor b in trxcon:host/trxcon/src/trxcon
 ↪  mobile:host/layer23/src/mobile/mobile \\\\\\n         virtphy:host/virt_phy/src/virtphy
 ↪  ccch_scan:host/layer23/src/misc/ccch_scan; do\\n  $SUDO install -m755 \"$ROOT/osmocom-bb/src/${b#*:}\"
 ↪  \"/usr/local/bin/${b%%:*}\"\\ndone\\n'' returned non-zero exit status 1.",
     "output_type": "error",
     "traceback": [
      "\u001b[31m---------------------------------------------------------------------------\u001b[39m",
      "\u001b[31mCalledProcessError\u001b[39m                        Traceback (most recent call last)",
      "\u001b[36mCell\u001b[39m\u001b[36m \u001b[39m\u001b[32mIn[13]\u001b[39m\u001b[32m, line
 ↪  1\u001b[39m\n\u001b[32m----> \u001b[39m\u001b[32m1\u001b[39m
 ↪  get_ipython().run_cell_magic(\u001b[33m'bash'\u001b[39m, \u001b[33m''\u001b[39m, \u001b[33m'.
 ↪  /opt/GSM/.notebook.env\\nset -e\\nP=\"$ROOT/osmo-operator/patches\"\\n# [2026-10-23] pinned like the
 ↪  Dockerfile (OSMOCOM_BB_REF): upstream a0a315e2 moved trxcon to libosmo-trx and out of\\n# `make nofirmware`,
 ↪  so a clone of master no longer builds src/host/trxcon/src/trxcon\\ncd \"$ROOT\"; [ -d osmocom-bb ] || { git
 ↪  clone -q https://gitea.osmocom.org/phone-side/osmocom-bb \\\\\\n                                    && git -C
 ↪  osmocom-bb checkout -q \"${OSMOCOM_BB_REF:-eac4eed1}\"; }\\n# Same three cases as step 3: applied (reverse
 ↪  applies) / applicable / does not match = error\\nfor p in clck-gen-frame-tolerance plmn-a6-reinsert
 ↪  sim-plmn-from-bcd; do\\n  f=\"$P/osmocom-bb-$p.patch\"\\n  if git -C osmocom-bb apply --reverse --check
 ↪  \"$f\" 2>/de↩
v/null; then echo \"patch already applied: $p\"\\n  elif git -C osmocom-bb apply --check \"$f\" 2>/dev/null; then
 ↪  git -C osmocom-bb apply \"$f\" && echo \"patched: $p\"\\n  else echo \"PATCH DOES NOT APPLY:
 ↪  osmocom-bb-$p.patch (tree $(git -C osmocom-bb describe --tags --always) differs from what the patch
 ↪  expects)\"; exit 1; fi\\ndone\\ncd osmocom-bb/src && make -s nofirmware\\nfor b in
 ↪  trxcon:host/trxcon/src/trxcon mobile:host/layer23/src/mobile/mobile \\\\\\n
 ↪  virtphy:host/virt_phy/src/virtphy ccch_scan:host/layer23/src/misc/ccch_scan; do\\n  $SUDO install -m755
 ↪  \"$ROOT/osmocom-bb/src/${b#*:}\" \"/usr/local/bin/${b%%:*}\"\\ndone\\n'\u001b[39m)\n",
       "\u001b[31mCalledProcessError\u001b[39m: Command 'b'. /opt/GSM/.notebook.env\\nset
 ↪  -e\\nP=\"$ROOT/osmo-operator/patches\"\\n# [2026-10-23] pinned like the Dockerfile (OSMOCOM_BB_REF): upstream
 ↪  a0a315e2 moved trxcon to libosmo-trx and out of\\n# `make nofirmware`, so a clone of master no longer builds
 ↪  src/host/trxcon/src/trxcon\\ncd \"$ROOT\"; [ -d osmocom-bb ] || { git clone -q
 ↪  https://gitea.osmocom.org/phone-side/osmocom-bb \\\\\\n                                    && git -C osmocom-bb
 ↪  checkout -q \"${OSMOCOM_BB_REF:-eac4eed1}\"; }\\n# Same three cases as step 3: applied (reverse applies) /
 ↪  applicable / does not match = error\\nfor p in clck-gen-frame-tolerance plmn-a6-reinsert sim-plmn-from-bcd;
 ↪  do\\n  f=\"$P/osmocom-bb-$p.patch\"\\n  if git -C osmocom-bb apply --reverse --check \"$f\" 2>/dev/null; then
 ↪  echo \"patch already applied: $p\"\\n  elif git -C osmocom-bb apply --check \"$f\" 2>/dev/null; then git -C
 ↪  osmocom-bb apply \"$f\" && echo \"patched: $p\"\\n  else echo \"PATCH↩
 DOES NOT APPLY: osmocom-bb-$p.patch (tree $(git -C osmocom-bb describe --tags --always) differs from what the
 ↪  patch expects)\"; exit 1; fi\\ndone\\ncd osmocom-bb/src && make -s nofirmware\\nfor b in
 ↪  trxcon:host/trxcon/src/trxcon mobile:host/layer23/src/mobile/mobile \\\\\\n
 ↪  virtphy:host/virt_phy/src/virtphy ccch_scan:host/layer23/src/misc/ccch_scan; do\\n  $SUDO install -m755
 ↪  \"$ROOT/osmocom-bb/src/${b#*:}\" \"/usr/local/bin/${b%%:*}\"\\ndone\\n'' returned non-zero exit status 1."
     ]
    }
   ],
   "source": [
    "%%bash\n",
    ". /opt/GSM/.notebook.env\n",
    "set -e\n",
    "P=\"$ROOT/osmo-operator/patches\"\n",
    "# [2026-10-23] pinned like the Dockerfile (OSMOCOM_BB_REF): upstream a0a315e2 moved trxcon to libosmo-trx and
 ↪  out of\n",
    "# `make nofirmware`, so a clone of master no longer builds src/host/trxcon/src/trxcon\n",
    "cd \"$ROOT\"; [ -d osmocom-bb ] || { git clone -q https://gitea.osmocom.org/phone-side/osmocom-bb \\\n",
    "                                  && git -C osmocom-bb checkout -q \"${OSMOCOM_BB_REF:-eac4eed1}\"; }\n",
    "# Same three cases as step 3: applied (reverse applies) / applicable / does not match = error\n",
    "for p in clck-gen-frame-tolerance plmn-a6-reinsert sim-plmn-from-bcd; do\n",
    "  f=\"$P/osmocom-bb-$p.patch\"\n",
    "  if git -C osmocom-bb apply --reverse --check \"$f\" 2>/dev/null; then echo \"patch already applied: $p\"\n",
    "  elif git -C osmocom-bb apply --check \"$f\" 2>/dev/null; then git -C osmocom-bb apply \"$f\" && echo
 ↪  \"patched: $p\"\n",
    "  else echo \"PATCH DOES NOT APPLY: osmocom-bb-$p.patch (tree $(git -C osmocom-bb describe --tags --always)
 ↪  differs from what the patch expects)\"; exit 1; fi\n",
    "done\n",
    "cd osmocom-bb/src && make -s nofirmware\n",
    "for b in trxcon:host/trxcon/src/trxcon mobile:host/layer23/src/mobile/mobile \\\n",
```

```
   "         virtphy:host/virt_phy/src/virtphy ccch_scan:host/layer23/src/misc/ccch_scan; do\n",
   "  $SUDO install -m755 \"$ROOT/osmocom-bb/src/${b#*:}\" \"/usr/local/bin/${b%%:*}\"\n",
   "done"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "c22",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "# Check: osmocon in the tree, mobile in the PATH and linked to gapk\n",
   "ls -la \"$ROOT/osmocom-bb/src/host/osmocon/osmocon\" && command -v mobile\n",
   "ldd \"$(command -v mobile)\" | grep -i gapk || echo \"WARNING: mobile is not linked to libosmogapk (gapk
↪  built after osmocom-bb?)\""
  ]
 },
 {
  "cell_type": "markdown",
  "id": "c23",
  "metadata": {},
  "source": [
   "## 7. Prebuilt Calypso firmware\n",
   "\n",
   "`make nofirmware` does not build the firmware: the bench takes the binaries from `bbaranoff/firmware`,\n",
   "the only path QEMU (`-kernel .elf`) and osmocon (`.bin`) read (`qosmo/environnement/bench.env`)."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "c24",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "cd \"$ROOT\"; [ -d firmware ] || git clone -q --depth 1 https://github.com/bbaranoff/firmware\n",
   "ls -la \"$ROOT\"/firmware/board/compal_e88/layer1.highram.{elf,bin}"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "x01",
  "metadata": {},
  "source": [
   "### 7b. The transceiver tree and the `trx` firmware (`--osmo-trx`)\n",
   "\n",
   "[2026-10-20] A second osmocom-bb tree, Sylvain Munaut's `jolly/testing` branch, provides the `trx`\n",
   "firmware application (the Calypso as a BTS transceiver) and the host `transceiver` program that speaks\n",
   "TRXD (ports 5700-5702) to osmo-bts-trx. The bench uses them with `./start.sh --bts` (alias `--osmo-trx`)\n",
   "(`c54x_exe/run.sh` with `OSMO_TRX=1`, alias `banc-trx`). The project patch\n",
   "`patches/osmocom-bb-transceiver-qemu-bench.patch` covers both sides: the host program (`--no-sync`,\n",
  "`--ts-mask`/`--tx-mask`, `--l1ctl`, `--base-port`, SETTSC/SETFORMAT/NOMTXPOWER accepted) and the firmware\n",
   "(build with `arm-none-eabi-gcc`, `prim_bts.c`: RX offset reset and TX slot/timeslot mapping)."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "x02",
  "metadata": {},
  "outputs": [],
```

```
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "set -e\n",
   "P=\"$ROOT/osmo-operator/patches\"; T=\"$ROOT/osmocom-bb-transceiver\"\n",
   "[ -d \"$T\" ] || git clone -q --branch jolly/testing https://gitea.osmocom.org/phone-side/osmocom-bb.git
↪  \"$T\"\n",
   "f=\"$P/osmocom-bb-transceiver-qemu-bench.patch\"\n",
   "if git -C \"$T\" apply --reverse --check \"$f\" 2>/dev/null; then echo \"patch already applied\"\n",
   "elif git -C \"$T\" apply --check \"$f\" 2>/dev/null; then git -C \"$T\" apply \"$f\" && echo \"patched\"\n",
   "else echo \"PATCH DOES NOT APPLY (tree $(git -C \"$T\" describe --always) differs)\"; exit 1; fi\n",
   "# host side: the transceiver program (layer23 configured with --enable-transceiver)\n",
   "cd \"$T/src\" && make -s HOST_layer23_CONFARGS=--enable-transceiver nofirmware\n",
   "$SUDO install -m755 \"$T/src/host/layer23/src/transceiver/transceiver\" /usr/local/bin/transceiver\n",
   "transceiver -h 2>&1 | grep -E 'no-sync|ts-mask|tx-mask'"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "x03",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "set -e\n",
   "T=\"$ROOT/osmocom-bb-transceiver/src/target/firmware\"\n",
   "# firmware side: trx.highram.{elf,bin} with the system arm-none-eabi toolchain (libosmocore for the target
↪  first)\n",
   "command -v arm-none-eabi-gcc >/dev/null || $SUDO apt-get install -y gcc-arm-none-eabi\n",
   "cd \"$ROOT/osmocom-bb-transceiver/src\"\n",
   "[ -f shared/libosmocore/build-target/src/.libs/libosmocore.a ] || { rm -rf shared/libosmocore/build-target &&
↪  mkdir shared/libosmocore/build-target \\\n",
   "  && cd shared/libosmocore/build-target && ../configure --host=arm-none-eabi --enable-embedded
↪  --disable-shared --disable-tests \\\n",
   "     ac_cv_header_sys_select_h=no ac_cv_header_sys_socket_h=no \\\n",
   "    CFLAGS=\"-Os -ffunction-sections -fcommon -I$T/include -nostartfiles -nodefaultlibs\" >/dev/null && make
↪  -s; }\n",
   "cd \"$T\" && make -s BOARDS=compal_e88 APPLICATIONS=trx ENVIRONMENTS=highram CROSS_COMPILE=arm-none-eabi-
↪  2>&1 | grep -v '^make' | tail -3 || true\n",
   "ls -la board/compal_e88/trx.highram.elf board/compal_e88/trx.highram.bin\n",
   "$SUDO install -m755 board/compal_e88/trx.highram.elf board/compal_e88/trx.highram.bin
↪  \"$ROOT/firmware/board/compal_e88/\""
  ]
 },
 {
  "cell_type": "markdown",
  "id": "x04",
  "metadata": {},
  "source": [
   "Optional: the historical `arm-elf` toolchain (gcc 4.5.2, binutils 2.21.1, newlib 1.19) from the\n",
   "[gnu-arm-installer](https://github.com/axilirator/gnu-arm-installer), the one osmocom-bb documents for
↪  its\n",
   "firmware, installed in `/root/gnuarm/install`. On Ubuntu 24.04 its `cc1` wants `libmpfr.so.4`; a symlink
↪  to\n",
   "the system's `libmpfr.so.6` is enough."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "x05",
  "metadata": {},
  "outputs": [],
  "source": [
```

```
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "[ -x /root/gnuarm/install/bin/arm-elf-gcc ] || { echo \"gnuarm not installed (optional): git clone
↪  https://github.com/axilirator/gnu-arm-installer /root/gnuarm && cd /root/gnuarm && ./download.sh &&
↪  ./build.sh\"; exit 0; }\n",
   "[ -e /usr/local/lib/libmpfr.so.4 ] || { $SUDO ln -sf /usr/lib/x86_64-linux-gnu/libmpfr.so.6
↪  /usr/local/lib/libmpfr.so.4 && $SUDO ldconfig; }\n",
   "/root/gnuarm/install/bin/arm-elf-gcc --version | head -1"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "c25",
  "metadata": {},
  "source": [
   "## 8. qosmo: the Calypso QEMU (stage `qemu`)\n",
   "\n",
   "One binary for both chains: built with `--enable-l1-grgsm`, it runs the DSP chain with\n",
   "`CALYPSO_DSP_EXTERN=1` set at launch. `ninja install` also installs the `qosmo` launcher next to\n",
   "`qemu-system-arm`. Long: 10–15 min."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "c26",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "set -e\n",
   "cd \"$ROOT\"; [ -d qosmo ] || git clone -q https://github.com/bbaranoff/qosmO qosmo\n",
   "cd qosmo\n",
   "python3 -m venv ~/.venv-qemu && . ~/.venv-qemu/bin/activate\n",
   "pip install -q --no-cache-dir tomli\n",
   "mkdir -p build && cd build\n",
   "[ -f build.ninja ] || ../configure --target-list=arm-softmmu --enable-l1-grgsm \\\n",
   "    --prefix=\"$ROOT/qemu-install\" --disable-werror --disable-docs\n",
   "make -s -j\"$(nproc)\"\n",
   "make -s install\n",
   "$SUDO cp \"$ROOT/qemu-install/bin/qemu-system-arm\" /usr/local/bin/qemu-system-arm\n",
   "$SUDO cp \"$ROOT/qemu-install/bin/qosmo\" /usr/local/bin/qosmo"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "c27",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "# Check: the binary run.sh expects (QEMU_BIN in qosmo/environnement/bench.env), the launcher, the linked
↪  layer 1\n",
   "ls -la \"$ROOT/qosmo/build/qemu-system-arm\"\n",
   "qosmo -V\n",
   "\"$ROOT/qosmo/build/qemu-system-arm\" -M help | grep -iE \"calypso|couche\""
  ]
 },
 {
  "cell_type": "markdown",
  "id": "c28",
  "metadata": {},
  "source": [
```

```
   "## 9. c54x_exe and the DSP ROM (stage `l1`)\n",
   "\n",
   "`c54x_exe` compiles qosmo's sources and links against libosmocoding. The CFLAGS are the Makefile's\n",
   "**without `-march=native`**. The ROM is not in the repository: `rom/fetch-rom.sh` downloads FreeCalypso's\n",
   "dump 3606, converts it and checks its checksum. The raw dump `calypso_dsp.txt` is downloaded separately,\n",
   "as in the image."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "c29",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "set -e\n",
   "F=ftp://ftp.freecalypso.org/pub/GSM/Calypso\n",
   "cd \"$ROOT\"; [ -d c54x_exe ] || git clone -q https://github.com/bbaranoff/c54x_exe\n",
   "cd c54x_exe\n",
   "make -s QOSMO=\"$ROOT/qosmo\" \\\n",
   "  CFLAGS=\"-O3 -g -Wall -Werror=format -Werror=format-extra-args -Wno-unused-function -Wno-unused-variable
↪  -Wno-unused-but-set-variable -Wno-sign-compare\"\n",
   "FREECALYPSO_URL=\"$F\" bash rom/fetch-rom.sh --dest \"$ROOT\" --dest \"$ROOT/c54x_exe/rom\"\n",
   "[ -s \"$ROOT/calypso_dsp.txt\" ] || curl -fsS --retry 2 -o \"$ROOT/calypso_dsp.txt\"
↪  \"$F/dsp-rom-3606-dump.txt\"\n",
   "[ -e rom/calypso_dsp.txt ] || cp \"$ROOT/calypso_dsp.txt\" rom/calypso_dsp.txt"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "c30",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "# Check: the ROM (fetch-rom.sh re-verifies it) and the DSP alone, no ARM (c54x_exe/README.md)\n",
   "cd \"$ROOT/c54x_exe\"\n",
   "ls \"$ROOT\"/calypso_dsp.{PROM0,PROM1,PROM2,PROM3,DROM,PDROM}.bin\n",
   "./c54x_exe --trames 200 | tail -5"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "m01",
  "metadata": {},
  "source": [
   "### 9b. (optional) FreeCalypso Magnetite, the real TI firmware for the C139\n",
   "\n",
   "[2026-10-21] With `--ms-magnetite` the emulated mobile does not run the osmocom-bb layer 1 but the
↪  original\n",
   "TI TCS211 firmware, rebuilt by FreeCalypso Magnetite (target `c139`, configuration `hybrid-ui-vo`):
↪  Nucleus,\n",
   "TI's L2/L3 stack and the phone UI all run on the emulated ARM, with the DSP of step 9 under it. There is no\n",
   "osmocon and no `mobile`: `rvinterf` (FreeCalypso host tools) decodes the firmware's traces, and the C139's\n",
   "colour screen and keypad are in `tools/osmo-c123-gtk.py --modele c139`.\n",
   "\n",
   "Magnetite does **not** build with gcc: it needs TI's TMS470 compiler, a Windows binary run under Wine\n",
   "(environment `installed-env.tar.xz` from ftp.freecalypso.org). The 4 MiB flash image the bench loads is\n",
   "the FreeCalypso Compal bootloader (`compal-flash-boot-for-fc.bin`, 8 KiB, at 0) plus `fwimage.bin` at\n",
   "0x10000: `/opt/GSM/firmware/magnetite/c139-flash.img` (`MAGNETITE_FLASH` overrides it). Full notes, and\n",
   "the qosmo changes it relies on: [doc/magnetite.md](../doc/magnetite.md). Not needed for the call of step
↪  13.\n"
```

```
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "m02",
   "metadata": {},
   "outputs": [],
   "source": [
    "%%bash\n",
    ". /opt/GSM/.notebook.env\n",
    "[ \"${NB_MAGNETITE:-0}\" = 1 ] || { echo \"Magnetite skipped (optional): NB_MAGNETITE=1 in the environment of
↪  Jupyter to build it\"; exit 0; }\n",
    "set -e\n",
    "cd \"$ROOT\"\n",
    "# Mercurial (the two FreeCalypso repositories) in a venv, Wine with its 32-bit side\n",
    "[ -x ~/.venv-hg/bin/hg ] || { python3 -m venv ~/.venv-hg && ~/.venv-hg/bin/pip install -q mercurial; }\n",
    "[ -d fc-magnetite ]  || ~/.venv-hg/bin/hg clone -q https://hg.themyscira.org/fc-magnetite fc-magnetite\n",
    "[ -d fc-host-tools ] || ~/.venv-hg/bin/hg clone -q https://hg.themyscira.org/freecalypso-tools
↪  fc-host-tools\n",
    "make -s -C fc-host-tools                      # rvinterf, fc-shell, fc-fsio (run.sh:
↪  fc-host-tools/rvinterf/lowlevel/rvinterf)\n",
    "command -v wine >/dev/null || { $SUDO dpkg --add-architecture i386 && $SUDO apt-get update -qq \\\n",
    "                              && $SUDO apt-get install -y wine64 wine32:i386; }\n",
    "# the TI environment (compiler, cygwin, perl, java) and the nowhine wrapper, same FTP directory\n",
    "W=ftp://ftp.freecalypso.org/pub/GSM/TI_src/wine; mkdir -p fc-wine-dl\n",
    "[ -s fc-wine-dl/installed-env.tar.xz ] || curl -fsS -o fc-wine-dl/installed-env.tar.xz
↪  \"$W/installed-env.tar.xz\"\n",
    "[ -s fc-wine-dl/nowhine.c ]            || curl -fsS -o fc-wine-dl/nowhine.c \"$W/nowhine.c\"\n",
    "[ -d ~/.wine/drive_c/Dvpt_tools ] || { mkdir -p ~/.wine/drive_c && tar xJf fc-wine-dl/installed-env.tar.xz -C
↪  ~/.wine/drive_c && wineboot -u; }\n",
    "command -v nowhine >/dev/null || $SUDO gcc -O2 -o /usr/local/bin/nowhine fc-wine-dl/nowhine.c\n",
    "cd fc-magnetite && (cd helpers && make -s)\n",
    "[ -d build-c139-hybrid-ui-vo ] || ./configure.sh c139 hybrid-ui-vo\n",
    "make -s -C build-c139-hybrid-ui-vo -j4        # ~1 min, 4 cl470 under Wine\n",
    "ls -la build-c139-hybrid-ui-vo/fwimage.bin\n"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "m03",
   "metadata": {},
   "outputs": [],
   "source": [
    "%%bash\n",
    ". /opt/GSM/.notebook.env\n",
    "[ \"${NB_MAGNETITE:-0}\" = 1 ] || { echo \"Magnetite skipped (optional, NB_MAGNETITE=1)\"; exit 0; }\n",
    "# The flash image: bootloader at 0, fwimage.bin at 0x10000, 0xFF elsewhere (doc/magnetite.md, section 2).\n",
    "# The bootloader is not built here: put compal-flash-boot-for-fc.bin (FreeCalypso) in $ROOT/fc-dl first.\n",
    "B=\"$ROOT/fc-dl/compal-flash-boot-for-fc.bin\";
↪  F=\"$ROOT/fc-magnetite/build-c139-hybrid-ui-vo/fwimage.bin\"\n",
    "[ -s \"$B\" ] || { echo \"MISSING $B (FreeCalypso Compal bootloader, 8 KiB)\"; exit 1; }\n",
    "mkdir -p \"$ROOT/firmware/magnetite\"\n",
    "python3 - \"$B\" \"$F\" \"$ROOT/firmware/magnetite/c139-flash.img\" <<'EOF'\n",
    "import sys\n",
    "img = bytearray(b\"\\xff\" * (4 << 20))\n",
    "boot = open(sys.argv[1], \"rb\").read(); fw = open(sys.argv[2], \"rb\").read()\n",
    "img[0:len(boot)] = boot; img[0x10000:0x10000 + len(fw)] = fw\n",
    "open(sys.argv[3], \"wb\").write(img)\n",
    "EOF\n",
    "# Check: the image and the tools the bench uses with it\n",
    "ls -la \"$ROOT/firmware/magnetite/c139-flash.img\" \"$ROOT/fc-host-tools/rvinterf/lowlevel/rvinterf\"\n",
    "\"$ROOT/osmo-operator/scripts/magnetite-c139.sh\" status 2>/dev/null || true\n"
   ]
  },
```

```
{
 "cell_type": "markdown",
 "id": "c31",
 "metadata": {},
 "source": [
  "## 10. (optional) grgsm_exe\n",
  "\n",
  "Bench tool (gr-gsm layer 1 outside QEMU, no firmware): it does **not** make the mobile camp. Not needed\n",
  "for a call on the DSP."
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "c32",
 "metadata": {},
 "outputs": [],
 "source": [
  "%%bash\n",
  ". /opt/GSM/.notebook.env\n",
  "cd \"$ROOT\"; [ -d grgsm_exe ] || git clone -q https://github.com/bbaranoff/grgsm_exE grgsm_exe\n",
  "cd grgsm_exe && make -s QOSMO=\"$ROOT/qosmo\" && ./grgsm_exe --trames 5000 | tail -5"
 ]
},
{
 "cell_type": "markdown",
 "id": "c33",
 "metadata": {},
 "source": [
  "## 11. Runtime setup\n",
  "\n",
  "What the end of the `Dockerfile` does for the running stack:\n",
  "\n",
  "- `install.sh --only configs`: the templates of `configs/` into `/etc/osmocom` (never overwrites an\n",
  "  existing file) and `mobile.cfg`;\n",
  "- the run-time scripts of `scripts/` into `/etc/osmocom`: `run_modules/22-smsc.sh` starts\n",
  "  `/etc/osmocom/smsc-start.sh` and `sms-interop-relay.py`, the audio chain uses `gapk-start.sh`;\n",
  "- the run, log and Asterisk directories;\n",
  "- the cell: the DSP mobile looks for **ARFCN 514, BSIC 7** (`stick 514` in `c54x_exe/mobile_pont.cfg`,\n",
  "  `pont/config.py`), which is the bench's default in `globals.conf`.\n",
  "\n",
  "The `__MCC__`, `__ARFCN__`… markers are filled at launch by `./start.sh --regen` from `globals.conf`.\n",
  "\n",
  "[2026-10-23] The bench's shell helpers (`banc-trx`, `banc-mobile`, `banc-rssi`, `banc-stop`, `c123`…)\n",
  "come from `scripts/osmo-banc-aliases.sh`, installed as `/etc/profile.d/98-osmo-banc-aliases.sh` (login\n",
  "shells source it)."
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "c34",
 "metadata": {},
 "outputs": [],
 "source": [
  "%%bash\n",
  ". /opt/GSM/.notebook.env\n",
  "set -e\n",
  "O=\"$ROOT/osmo-operator\"\n",
  "cd \"$O\" && $SUDO env GSM_ROOT=\"$ROOT\" OSMOCOM_HOME=/root/.osmocom ./install.sh --only configs\n",
  "$SUDO mkdir -p /var/log/osmocom /var/run/smsc /var/lib/gapk /root/.osmocom/bb \\\n",
  "               /etc/asterisk /var/lib/asterisk /var/log/asterisk /var/run/asterisk\n",
  "$SUDO install -d -o osmocom -g osmocom /var/lib/osmocom    # osmo-hlr keeps its database there\n",
  "for f in \"$O\"/scripts/*.sh \"$O\"/scripts/*.py; do\n",
  "  [ \"$(basename \"$f\")\" = run.sh ] && continue        # the image moves it to /root/run.sh; start.sh uses
↪  qosmo/run.sh\n",
```

```
   "  $SUDO install -m755 \"$f\" /etc/osmocom/\n",
   "done\n",
   "$SUDO ln -sf /etc/osmocom/gapk-start.sh /usr/local/bin/gapk-start.sh\n",
   "$SUDO install -m644 \"$O/scripts/osmo-banc-aliases.sh\" /etc/profile.d/98-osmo-banc-aliases.sh   #
↪  banc-rssi, banc-stop, c123...\n",
   "$SUDO test -e /root/.osmocom/bb/mobile.cfg || $SUDO cp \"$O/configs/mobile.cfg\"
↪  /root/.osmocom/bb/mobile.cfg"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "c35",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "# Check\n",
   "ls /etc/osmocom/osmo-bsc.cfg /etc/osmocom/smsc-start.sh /etc/osmocom/sms-interop-relay.py
↪  /etc/osmocom/gapk-start.sh\n",
   "ls -ld /var/lib/osmocom\n",
   "# effective radio/PLMN settings (an empty value in globals.conf means the built-in default)\n",
   "cd \"$ROOT/osmo-operator\" && ./generate_configs.sh --show | grep -E \"MCC|MNC|BAND|ARFCN|BSIC|LAC\" | head"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "c36",
  "metadata": {},
  "source": [
   "### 11b. (optional) audio sinks\n",
   "\n",
   "Only for **hearing** the call (the `voix` rung of the tests). The core, the camp, the location update,\n",
   "the SMS and the call set-up do not need audio. Ubuntu 24.04 desktop runs PipeWire with its PulseAudio\n",
   "server, which `pactl` drives. `configs/asound.conf` routes ALSA to Pulse (`type pulse`), which needs\n",
   "`libasound2-plugins`, installed at step 1."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "c37",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   "# run as YOUR user (the desktop session's audio server), not with sudo\n",
   "for s in gsm_audio gsm_mic; do\n",
   "  pactl list short sinks | grep -q \"$s\" || pactl load-module module-null-sink sink_name=$s format=s16le
↪  rate=8000 channels=1\n",
   "done\n",
   "pactl list short sinks | grep gsm_ || echo \"no audio server reachable (server install): skip, the call still
↪  sets up\""
  ]
 },
 {
  "cell_type": "markdown",
  "id": "c38",
  "metadata": {},
  "source": [
   "## 12. Start: the network, the BTS, then the DSP bench\n",
   "\n",
   "`./start.sh` (DSP by default since 2026-10-01; `start-native.sh` is the same script under its old name)\n",
   "plays `qosmo/run.sh`'s plan **without** `qemu,pty,osmocon,l2` (Osmocom core, HLR subscribers,\n",
   "osmo-bts-trx), then hands over to `c54x_exe/run.sh` for the phone chain. First the dependency check and a\n",
```

```
   "dry run, then the real start. `./start.sh --help` lists every option; the ones that pick the set-up:\n",
   "\n",
   "| option | what runs |\n",
   "|---|---|\n",
   "| `--bts-classic` / `--bts-classique` | the **classic BTS**: osmo-bts-trx talks to `pont/pont_dsp.py` (TRXD
↪  5700-5702), the emulated phone (QEMU + c54x_exe) is plugged on the bridge. The default set-up of `--dsp`,
↪  named; exclusive with `--bts` / `--ts` |\n",
   "| `--ms` | the emulated phone is an osmocom-bb **mobile** (the default) |\n",
   "| `--ms-magnetite` | like `--ms`, but the mobile runs the real TI firmware, FreeCalypso Magnetite for the
↪  C139 (step 9b) |\n",
   "| `--bts` (alias `--osmo-trx`) | the Calypso of QEMU **is the TRX** of BTS#0 (`trx.highram.elf` + host
↪  `transceiver`); with `--ms` / `--ms-magnetite` a second Calypso camps on it over the air |\n",
   "| `--ts N` | timeslots of that Calypso TRX (implies `--bts`): `1` TS0 only, `2` one Calypso per timeslot,
↪  `2/1` TS0 + TS1 on one Calypso (the default) |\n",
   "| `--vocoder-rom` / `--gapk` | the speech codec: the DSP mask ROM's FR codec (default), or the host
↪  (osmo-gapk in the mobile; with Magnetite, `pont/voix_hote.py` in the bridge). The emulated DSP keeps real time
↪  only with `--gapk` |\n",
   "| `--no-attach` | do not attach to tmux at the end (what a notebook needs) |\n",
   "\n",
   "[2026-10-20] With `--bts`, the emulated phone boots `trx.highram.elf` and becomes the **transceiver of\n",
   "BTS#0** (host `transceiver` on TRXD 5700-5702, no bridge, no mobile). A second emulated phone
↪  (`INSTANCE=2`,\n",
   "alias `banc-mobile`) then camps on it through `pont/air.py`. See the run notebook, step 12b, and\n",
   "`wiki/Reference.md` section 2.7.\n"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "c39",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "# --check-paths only reads; --dry-run walks the plan with no side effects\n",
   "cd \"$ROOT/osmo-operator\" && $SUDO ./start.sh --check-paths && $SUDO ./start.sh --dry-run"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "c40",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "# --regen fills the templates of /etc/osmocom ; --no-attach: do not take the terminal (no tmux attach)\n",
   "# classic BTS (osmo-bts-trx + pont_dsp.py) and the osmocom-bb mobile on the bridge\n",
   "cd \"$ROOT/osmo-operator\" && $SUDO ./start.sh --regen --bts-classique --ms --no-attach\n"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "m04",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "[ \"${NB_MAGNETITE:-0}\" = 1 ] && [ -s \"$ROOT/firmware/magnetite/c139-flash.img\" ] || { echo \"Magnetite
↪  variant skipped (NB_MAGNETITE=1 and step 9b)\"; exit 0; }\n",
   "# Variant (instead of the previous cell, needs step 9b): the C139 Magnetite on the classic BTS, voice by the
↪  host.\n",
```

```
   "# ENCRYPTION=\"a5 0\": calls on Magnetite need it (with A5/1 the FACCH fails and the call never sets up);\n",
   "# --regen carries it into osmo-bsc.cfg / osmo-msc.cfg (encryption a5 0). Screen: c123, or\n",
   "#   /usr/bin/python3 $ROOT/osmo-operator/tools/osmo-c123-gtk.py --modele c139 --instance 1\n",
   "cd \"$ROOT/osmo-operator\" && $SUDO env ENCRYPTION=\"a5 0\" ./start.sh --regen --bts-classique --ms-magnetite
↪  --gapk --no-attach\n",
   "grep -E '^ *encryption' /etc/osmocom/osmo-bsc.cfg /etc/osmocom/osmo-msc.cfg\n"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "x06",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   ". /opt/GSM/.notebook.env\n",
   "# Variant: the phone as the BTS transceiver (instead of the previous cells). Expected afterwards in\n",
   "# /tmp/c54x-pont/dsp.log: \"[rom-tx] ... SB n, FB n\" with SB = FB = 1/10 of the bursts; no \"STALE BURST\"
↪  in\n",
   "# osmocon.log once osmo-bts-trx has \"osmotrx fn-advance 20\" (configs/osmo-bts-trx.cfg).\n",
   "# Add --ms for a second phone over the air (instance 2), or --ms-magnetite for the C139.\n",
   "cd \"$ROOT/osmo-operator\" && $SUDO env NO_MENU=1 ./start.sh --bts --no-attach\n",
   "grep -a 'rom-tx\\] [0-9]' /tmp/c54x-pont/dsp.log | tail -1\n"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "c41",
  "metadata": {},
  "source": [
   "Manual alternative, process by process: the notebook [lancer-pas-a-pas.ipynb](lancer-pas-a-pas.ipynb)\n",
   "(network daemons one by one, then `c54x_exe --arm`, QEMU, osmocon, mobile, `pont_dsp.py`), or the page\n",
   "[Lancer-a-la-main](Lancer-a-la-main.md)."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "c42",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   "# Check the phone chain (c54x_exe/run.sh --status) and its key log lines\n",
   "# (Magnetite: no osmocon, the third line is then rvinterf's trace log, /tmp/c54x-pont/rvinterf.log)\n",
   "/opt/GSM/c54x_exe/run.sh --status\n",
   "grep -a \"en attente de l'ARM\"          /tmp/c54x-pont/dsp.log     | tail -1\n",
   "grep -a \"pont DSP : API RAM partagee\"  /tmp/c54x-pont/qemu.log    | tail -1\n",
   "grep -a \"your code is running now\"     /tmp/c54x-pont/osmocon.log | tail -1\n",
   "grep -a \"pont TRX : ports\"             /tmp/c54x-pont/pont.log    | tail -1"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "c43",
  "metadata": {},
  "source": [
   "## 13. Camp, location update, then a call\n",
   "\n",
   "The same checks as the test suite ([bbaranoff/tests](https://github.com/bbaranoff/tests),\n",
   "`modules/_lib.sh`, `60-camp.sh`, `70-attache.sh`, `90-appel.sh`). The DSP mobile's VTY is **4247**\n",
   "(`c54x_exe/mobile_pont.cfg`, `bind 127.0.0.1 4247`), the MSC's is 4254. `600` is the echo test.\n",
   "Reference: [`tout-en-un-20261016-164539.qmd`](https://github.com/bbaranoff/tests/blob/main/tout-en-un-
↪  20261016-164539.qmd)\n",
   "reaches camp after 1 s and CC ACTIVE after 3 s."
```

```
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "c44",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   "# Camp: wait for \"C3 camped normally\"\n",
   "vty() { { printf 'enable\\n%s\\n' \"$2\"; sleep 1; } | timeout --foreground 6 nc -N 127.0.0.1 \"$1\"
↪  2>/dev/null | tr -d '\\r'; }\n",
   "for i in $(seq 1 60); do vty 4247 \"show ms\" | grep -q \"C3 camped normally\" && { echo \"camp OK after
↪  $((i*2)) s\"; break; }; sleep 2; done\n",
   "vty 4247 \"show ms\" | grep -E \"camped|MM|IMSI\""
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "c45",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   "# Location update: in the mobile's log, and the subscriber in the MSC's VLR\n",
   "sed 's/\\x1b\\[[0-9;]*m//g' /tmp/c54x-pont/mobile.log | grep -ac 'LOCATION UPDATING ACCEPT'\n",
   "sed 's/\\x1b\\[[0-9;]*m//g' /tmp/c54x-pont/mobile.log | grep -aoE 'lai=[0-9]+-[0-9]+-[0-9]+' | tail -1\n",
   "{ printf 'enable\\nshow subscriber cache\\n'; sleep 1; } | timeout 6 nc -N 127.0.0.1 4254 | grep -E
↪  'IMSI|MSISDN'"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "c46",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   "# Call 600 (echo test), wait for ACTIVE, then hang up\n",
   "vty() { { printf 'enable\\n%s\\n' \"$2\"; sleep 2; } | timeout --foreground 6 nc -N 127.0.0.1 \"$1\"
↪  2>/dev/null | tr -d '\\r'; }\n",
   "vty 4247 \"call 1 600\"\n",
   "for i in $(seq 1 15); do st=\"$(vty 4247 'show ms' | sed -n 's/^ *call control state: //p' | head -1)\"; echo
↪  \"$st\"; [ \"$st\" = ACTIVE ] && break; sleep 2; done\n",
   "grep -a \"\\[montant\\] TCH\" /tmp/c54x-pont/dsp.log | tail -2\n",
   "vty 4247 \"call 1 hangup\""
  ]
 },
 {
  "cell_type": "markdown",
  "id": "m05",
  "metadata": {},
  "source": [
   "With `--ms-magnetite` there is no `mobile` and no VTY 4247: the C139's own UI is the phone. Watch the\n",
   "camp and the location update on its screen (`c123`, or `tools/osmo-c123-gtk.py --modele c139`) and in the\n",
   "MSC (`show subscriber cache` on 4254, as above), and dial 600 on the window's keypad. The firmware's
↪  traces\n",
   "are in `/tmp/c54x-pont/rvinterf.log`. The call needs `encryption a5 0` (step 12 variant); voice on this\n",
   "path is recommended with `--gapk` (state on 2026-10-22: speech both ways, downlink at about 27 frames/s).\n"
  ]
 },
 {
  "cell_type": "markdown",
```

```
 "id": "c47",
 "metadata": {},
 "source": [
  "## 14. Full test ladder (optional)\n",
  "\n",
  "The end-to-end suite climbs from core to voice and says how far it got. See\n",
  "[README-tests](../doc/README-tests.md)."
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "c48",
 "metadata": {},
 "outputs": [],
 "source": [
  "%%bash\n",
  ". /opt/GSM/.notebook.env\n",
  "cd \"$ROOT\"; [ -d tests ] || git clone -q https://github.com/bbaranoff/tests\n",
  "cd tests && $SUDO ./banc-max.sh --dsp --continue | tail -40"
 ]
},
{
 "cell_type": "markdown",
 "id": "c49",
 "metadata": {},
 "source": [
  "## 15. Stop\n",
  "\n",
  "`./start.sh --stop` stops the stack **and** the DSP bench, without repeating `--dsp`.\n",
  "`c54x_exe/run.sh --stop` alone stops only the phone chain and purges `/dev/shm/calypso_*`. The rssi\n",
  "application (instance 4, `banc-rssi`) is left running on purpose: stop it with `banc-rssi --stop`\n",
  "(or `banc-rssi-stop`).\n",
  "\n",
  "[2026-10-23] Two safeguards against losing the network in the middle of a run:\n",
  "\n",
  "- **BTS watchdog.** osmo-bts-trx exits on \"PC clock skew too high\" when the bridge's clock, paced by the\n",
  "  DSP, stalls more than 50 frames. `qosmo/run_modules/60-bts.sh` now runs BTS#0 inside a loop that\n",
  "  restarts it (`BTS_GARDIEN=1`, the default; `BTS_GARDIEN_PAUSE` seconds between restarts, 3 by default;\n",
  "  `BTS_GARDIEN=0` for the old single start). Each restart leaves a `[gardien bts]` line in the BTS log.\n",
  "- **Clock pause cap.** `pont/trx.py` no longer asks the BTS for a pause close to that 50-frame limit:\n",
  "  when the BTS is between `PONT_CLCK_PAUSE_PLAFOND` (38) and `PONT_CLCK_SKEW_ZONE` (60) frames ahead, the\n",
  "  `IND CLOCK` is capped and the rest is absorbed by the next ones (`pause bornee` in `pont.log`;\n",
  "  `PONT_CLCK_PAUSE_PLAFOND=0` restores the old behaviour).\n"
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "c50",
 "metadata": {},
 "outputs": [],
 "source": [
  "%%bash\n",
  ". /opt/GSM/.notebook.env\n",
  "# Before stopping: did the BTS restart, did the bridge cap a clock pause?\n",
  "sed 's/\\x1b\\[[0-9;]*m//g' /run/user/*/osmo-nitb/logs/bts.log 2>/dev/null | grep -aE 'gardien bts|clock
↪  skew' | tail -3\n",
  "grep -a 'pause bornee' /tmp/c54x-pont/pont.log 2>/dev/null | tail -1\n",
  "cd \"$ROOT/osmo-operator\" && $SUDO ./start.sh --stop\n"
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "50ad82ac-e969-47b0-b142-681c559e914a",
```

```
   "metadata": {},
   "outputs": [],
   "source": []
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "ea59bd68-2770-43b2-87f3-7e9b7ce96db6",
   "metadata": {},
   "outputs": [],
   "source": []
  }
 ],
 "metadata": {
  "kernelspec": {
   "display_name": "Python 3 (ipykernel)",
   "language": "python",
   "name": "python3"
  },
  "language_info": {
   "codemirror_mode": {
    "name": "ipython",
    "version": 3
   },
   "file_extension": ".py",
   "mimetype": "text/x-python",
   "name": "python",
   "nbconvert_exporter": "python",
   "pygments_lexer": "ipython3",
   "version": "3.12.3"
  }
 },
 "nbformat": 4,
 "nbformat_minor": 5
}
```

3.143 /opt/GSM/osmo-operator/wiki/lancer-les-modes.ipynb

34563 octets, 744 lignes → 744 lignes

```
{
 "cells": [
  {
   "cell_type": "markdown",
   "id": "m00",
   "metadata": {},
   "source": [
    "# Running the bench: one mode per section\n",
    "\n",
    "Companion of the help page (the **?** icon of the dock, `osmo-info`, http://127.0.0.1:8080/info.html) and
↪  of\n",
    "[lancer-pas-a-pas.ipynb](lancer-pas-a-pas.ipynb) (the same bench, process by process). This notebook
↪  installs\n",
    "nothing: it assumes the bench of [installer-osmo-jusquau-dsp.ipynb](installer-osmo-jusquau-dsp.ipynb) or the
↪  ISO.\n",
    "\n",
    "Each mode has three cells: **start**, **check**, **stop**. Run one mode at a time. The phone screens are
↪  shown\n",
    "here, inline (`screen(n)`), and you can press keys (`keys(n, ...)`); the GTK window is `c123 --instance
↪  N`.\n",
    "\n",
    "| Instance | What | Window |\n",
    "|---|---|---|\n",
```

```
   "| 1 | the phone of `--bts-classique` (osmocom or Magnetite), or the TRX of `--bts` | `c123` (`c123 --instance
↪  1`) |\n",
   "| 2 | the phone of `--bts --ms` (over the air) | `c123-1` (`c123 --instance 2`) |\n",
   "| 3 | the 2nd TRX phone of `--ts 2` (TS1) | `c123-2` (`c123 --instance 3`) |\n",
   "| 4 | the `rssi` app (`banc-rssi`) | `c123-3` (`c123 --instance 4`) |\n",
   "\n",
   "The commands need root (the container runs as root; on a desktop, start Jupyter after `sudo -v`)."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 1,
  "id": "m01",
  "metadata": {},
  "outputs": [],
  "source": [
   "# Helpers: run a command and stream it, show a phone screen, press keys, check the network.\n",
   "import os, socket, struct, subprocess, time\n",
   "from IPython.display import display\n",
   "from PIL import Image\n",
   "\n",
   "OP = \"/opt/GSM/osmo-operator\"\n",
   "ALIASES = f\"{OP}/scripts/osmo-banc-aliases.sh\"\n",
   "\n",
   "def sh(cmd, timeout=900):\n",
   "    \"\"\"bash -c with the bench aliases (banc-rssi, c123...) loaded; output streamed.\"\"\"\n",
   "    p = subprocess.Popen([\"bash\", \"-c\", f\". {ALIASES} 2>/dev/null; {cmd}\"], cwd=OP,\n",
   "                         stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True)\n",
   "    t0 = time.time()\n",
   "    for line in p.stdout:\n",
   "        print(line, end=\"\")\n",
   "        if time.time() - t0 > timeout:\n",
   "            p.kill(); print(\"... timeout\"); break\n",
   "    return p.wait()\n",
   "\n",
   "def _lcd_path(n):\n",
   "    return \"/dev/shm/calypso_lcd\" if n == 1 else f\"/dev/shm/calypso{n}/calypso_lcd\"\n",
   "\n",
   "def screen(n=1, scale=4):\n",
   "    \"\"\"The emulated screen of instance n: C123 (ST7558, 1 bit) or C139 (colour RGB565).\"\"\"\n",
   "    b = open(_lcd_path(n), \"rb\").read()\n",
   "    w, h, pages, flags, _, seq = struct.unpack_from(\"<HHHBBI\", b, 8)\n",
   "    d = b[32:]\n",
   "    if b[:7] == b\"C139LCD\":\n",
   "        im = Image.new(\"RGB\", (w, h))\n",
   "        px = [((p >> 11 & 31) << 3, (p >> 5 & 63) << 2, (p & 31) << 3)\n",
   "              for p in struct.unpack_from(\"<%dH\" % (w * h), d)]\n",
   "        im.putdata(px)\n",
   "    else:\n",
   "        im = Image.new(\"1\", (w, h), 1)\n",
   "        for y in range(h):\n",
   "            for x in range(w):\n",
   "                if d[(y >> 3) * w + x] & (1 << (y & 7)):\n",
   "                    im.putpixel((x, y), 0)\n",
   "    display(im.resize((w * scale, h * scale), Image.NEAREST))\n",
   "\n",
   "def keys(n, *names, wait=0.6):\n",
   "    \"\"\"Press keys on instance n: '0'..'9', 'ok', 'up', 'down', 'left', 'right', 'lsb', 'rsb', 'menu',
↪  'power'.\"\"\"\n",
   "    path = \"/tmp/calypso_keys.sock\" if n == 1 else f\"/tmp/calypso_keys{n}.sock\"\n",
   "    s = socket.socket(socket.AF_UNIX, socket.SOCK_DGRAM)\n",
   "    for k in names:\n",
   "        s.sendto(f\"press {k}\\n\".encode(), path); time.sleep(0.25)\n",
   "        s.sendto(f\"release {k}\\n\".encode(), path); time.sleep(wait)\n",
   "\n",
   "def vty(port, cmd):\n",
```

```
    "    \"\"\"One VTY command (4254 MSC, 4242 BSC, 4241 BTS, 4247 mobile).\"\"\"\n",
    "    with socket.create_connection((\"127.0.0.1\", port), timeout=5) as c:\n",
    "        c.sendall(f\"enable\\n{cmd}\\nexit\\n\\n\".encode()); time.sleep(1.0)\n",
    "        out = b\"\"\n",
    "        c.settimeout(1.0)\n",
    "        try:\n",
    "            while chunk := c.recv(65536): out += chunk\n",
    "        except socket.timeout:\n",
    "            pass\n",
    "    return out.decode(errors=\"replace\")\n",
    "\n",
    "def subscribers():\n",
    "    \"\"\"IMSI and cell of every subscriber attached at the MSC.\"\"\"\n",
    "    lines = [l.strip() for l in vty(4254, \"show subscriber cache\").splitlines()]\n",
    "    for l in lines:\n",
    "        if l.startswith((\"IMSI:\", \"MSISDN:\", \"LAC / cell ID:\")):\n",
    "            print(l)"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "m02",
   "metadata": {},
   "source": [
    "## 1. Classic BTS + osmocom phone (the default)\n",
    "\n",
    "`osmo-bts-trx` talks to `pont/pont_dsp.py`; the emulated Calypso runs the osmocom-bb `layer1` firmware and
↪  `mobile`,\n",
    "on the real DSP mask ROM. This is what the red phone icon starts."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "m03",
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "  hub        deduit du remote-ip de l ASP : 127.0.0.1\n",
      "[ .. ] Chargement de l'environnement\n",
      "[ OK ] Chargement de l'environnement (environment/load.env)\n",
      "[ .. ] Resolution de run.sh\n",
      "[ OK ] Resolution de run.sh (/opt/GSM/qosmo/run.sh)\n",
      "[ .. ] Lanceur QEMU\n",
      "[ OK ] Lanceur QEMU (/usr/local/bin/qosmo (qosmo local (/opt/GSM/qosmo)))\n",
      "[ .. ] Validation des chemins\n",
      "[ OK ] Validation des chemins\n",
      "  plan radio  lu dans /etc/osmocom/radio-plan.env\n",
      "  PLMN MS    MCC 001  MNC 01  operateur 1  ->  IMSI 001010001000001 / 001010001000002\n",
      "  radio      MS#1 sur ARFCN 1 (BTS principale)   MS#2 sur ARFCN 614 (side-car, unit-id 6012)\n",
      "[ .. ] Generation mobile MS#1\n",
      "  MS#1       aussi ecrit dans /root/.osmocom/bb/mobile_group1.cfg (le fichier que run.sh ouvre)\n",
      "[ OK ] Generation mobile MS#1 (/root/.osmocom/bb/mobile.cfg)\n",
      "[ .. ] Generation mobile MS#2 (faketrx)\n",
      "[ OK ] Generation mobile MS#2 (faketrx) (/root/.osmocom/bb/mobile_faketrx_bts1.cfg)\n",
      "\n",
      "╔══════════════════════════════════════════════════════╗\n",
      "║   Calypso GSM - bootstrap → run.sh                   ║\n",
      "╚══════════════════════════════════════════════════════╝\n",
      "\n",
      "  profil     hybrid (faketrx-qemu)\n",
      "  environ.   machine physique\n",
      "  noeud      aucun - identite SS7 inchangee\n",
```

```
    "  run.sh     /opt/GSM/qosmo/run.sh\n",
    "  MS#1       /opt/GSM/c54x_exe/mobile_pont.cfg  IMSI 001010001000001  ARFCN 1  VTY 4247  (banc DSP)\n",
    "             /root/.osmocom/bb/mobile.cfg non lance (module l2 retire du plan)\n",
    "  MS#2       /root/.osmocom/bb/mobile_faketrx_bts1.cfg  IMSI 001010001000002  ARFCN 614  VTY 4248\n",
    "  journaux   /run/user/1001/osmo-nitb/logs\n",
    "  mobile.log /tmp/c54x-pont/mobile.log\n",
    "  chiffrement a5 1\n",
    "\n",
    "  gabarits   regeneration demandee (--regen)\n",
    "[ .. ] Arret de la pile avant regeneration\n",
    "[ OK ] Arret de la pile avant regeneration\n",
    "[ .. ] Regeneration des configurations (noeud 1, operateur 1)\n",
    "  plan natif coeur 127.0.0.2  gsmtap 127.0.0.1  PC 1.1.2  rctx 1150  PLMN 001-01 (OsmoDirect)  2 MS\n",
    "  sauvegarde /run/user/1001/osmo-nitb/logs/osmocom-avant-regen-20261028-140908.tar.gz\n",
    "  ✓ adressage SS7 du noeud 1 reapplique (set-node-id.sh)\n",
    "[ OK ] Regeneration des configurations (/etc/osmocom + /etc/asterisk  ·  PC 1.1.x)\n",
    "  HLR        abonnes a reapprovisionner : MSISDN 100101..100102\n",
    "[ .. ] Identite SS7 du noeud 1\n",
    "[ OK ] Identite SS7 (noeud 1 · PC 1.1.x · mode native)\n",
    "  SS7        point-code 1.1.2   inter-STP 127.0.0.1\n",
    "[ .. ] Arret de la pile avant demarrage\n",
    "[ OK ] Arret de la pile avant demarrage\n",
    "[ .. ] Socket PCU\n",
    "[ OK ] Socket PCU (voir /run/user/1001/osmo-nitb/logs/pcu-socket.log)\n",
    "[ .. ] Sortie Internet APN (NAT)\n",
    "[ OK ] Sortie Internet APN (voir /run/user/1001/osmo-nitb/logs/apn-nat.log)\n",
    "[ .. ] Adresses privees du noeud\n",
    "[ OK ] Adresses privees du noeud (192.168.2.1 192.168.2.10 pose(s) en /32 sur enxa0cec8a98b06 (carte qui
↪ fournit Internet))\n",
    "[ .. ] Tableau de bord (:8080)\n",
    "[ OK ] Tableau de bord (:8080) (osmo-egprs-web.service)\n",
    "[ OK ] Fiche du noeud 1 (/etc/osmocom/mynode1.conf  (pour un autre noeud : --wan mynode1.conf))\n",
    "[ .. ] Raccord mobile (oFono)\n",
    "[ OK ] Raccord mobile (oFono) (modem /osmo ; pty RIL : /run/osmo-ril/at-pty)\n",
    "[ .. ] Audio (PulseAudio allege)\n",
    "[ OK ] Audio (PulseAudio allege) (journal /tmp/osmo-audio-chain.log)\n",
    "[ .. ] Transmission a run.sh (sans qemu,pty,osmocon,l2)\n",
    "\n",
    "Remise à zéro — rien ne sera relancé.\n",
    "\n",
    "  arrêt de la pile…\n",
    "  journaux archivés     /run/user/1001/osmo-nitb/logs/../archives/20261028-140921\n",
    "  journaux remis à zéro /run/user/1001/osmo-nitb/logs\n",
    "  espace disponible     3,1G (1% utilisé)\n",
    "  supprimé              /dev/shm/calypso_bsp_dedie\n",
    "  supprimé              /dev/shm/calypso_dcch_cfg\n",
    "  supprimé              /dev/shm/calypso_kc_l1\n",
    "  supprimé              /dev/shm/calypso_lcd\n",
    "  supprimé              /dev/shm/calypso_pcm_dl\n",
    "  supprimé              /dev/shm/calypso_pcm_ul\n",
    "  supprimé              /dev/shm/calypso_rejeu_tch.bin\n",
    "  supprimé              /dev/shm/calypso_rf_arfcn\n",
    "  supprimé              /dev/shm/calypso_tx_rom\n",
    "  supprimé              /dev/shm/calypso_xcch_ul_rom\n",
    "\n",
    "État propre. Relancez :\n",
    "   ./run.sh\n",
    "\n",
    "[ .. ] Prerequisite checks\n",
    "[ OK ] Prerequisite checks\n",
    "[ .. ] Host tooling\n",
    "[ OK ] Host tooling\n",
    "[SKIP] Run directory (unmet dependency: profil)\n",
    "[ .. ] Shutting down recorded processes\n",
    "[SKIP] Shutting down recorded processes (already running)\n",
    "[ .. ] Resolving paths and ports\n",
```

```
    "[ OK ] Resolving paths and ports\n",
    "[ .. ] Core configuration templates\n",
    "[ OK ] Core configuration templates\n",
    "[ .. ] Logs and working area\n",
    "[ OK ] Logs and working area\n",
    "[ .. ] Cleaning up the previous run\n",
    "[SKIP] Cleaning up the previous run (already running)\n",
    "[ .. ] Interfaces and addressing plan\n",
    "[SKIP] Interfaces and addressing plan (already running)\n",
    "[ .. ] Core — OsmoSTP (SS7/M3UA)\n",
    "[ OK ] Core — OsmoSTP (SS7/M3UA)\n",
    "[ .. ] Core — OsmoHLR (GSUP registry)\n",
    "[ OK ] Core — OsmoHLR (GSUP registry)\n",
    "[ .. ] Core — OsmoMGW (MGCP/RTP media)\n",
    "[ OK ] Core — OsmoMGW (MGCP/RTP media)\n",
    "[ .. ] Core — OsmoMSC (CS switching)\n",
    "[ OK ] Core — OsmoMSC (CS switching)\n",
    "[ .. ] BTS#1 configurations (side-car)\n",
    "[SKIP] BTS#1 configurations (side-car) (already running)\n",
    "[ .. ] Core — OsmoBSC (BTS controller)\n",
    "[ OK ] Core — OsmoBSC (BTS controller)\n",
    "[ .. ] Core — APN TUN interface\n",
    "[ OK ] Core — APN TUN interface\n",
    "[ .. ] Core — OsmoGGSN (GTP data)\n",
    "[ OK ] Core — OsmoGGSN (GTP data)\n",
    "[ .. ] Core — OsmoSGSN (packet attach)\n",
    "[ OK ] Core — OsmoSGSN (packet attach)\n",
    "[ .. ] Core — OsmoPCU (packet scheduler)\n",
    "[ OK ] Core — OsmoPCU (packet scheduler)\n",
    "[ .. ] Core — Asterisk (SIP PBX)\n",
    "[ OK ] Core — Asterisk (SIP PBX)\n",
    "[ .. ] Mobile configuration\n",
    "[SKIP] Mobile configuration (already running)\n",
    "[ .. ] Core — MNCC↔SIP gateway\n",
    "[ OK ] Core — MNCC↔SIP gateway\n",
    "[ .. ] Core — subscribers provisioned in the HLR\n",
    "[SKIP] Core — subscribers provisioned in the HLR (already running)\n",
    "[ .. ] Core — SMS gateway (proto-SMSC)\n",
    "[ OK ] Core — SMS gateway (proto-SMSC)\n",
    "[ .. ] Audio bridge (PulseAudio + GAPK)\n",
    "[ OK ] Audio bridge (PulseAudio + GAPK)\n",
    "[ .. ] tmux observation session\n",
    "[ OK ] tmux observation session\n",
    "[SKIP] Noise injector (downlink) (disabled by [ -n \"${BRUIT_MODE:-}${BRUIT_SNR_DB:-}\" ])\n",
    "[SKIP] Console gdb ARM en telnet (non bloquante) (unmet dependency: qemu)\n",
    "[ .. ] BTS#1 simulated transceiver\n",
    "[ OK ] BTS#1 simulated transceiver\n",
    "[ .. ] Base station BTS#1 (side-car)\n",
    "[ OK ] Base station BTS#1 (side-car)\n",
    "[ .. ] Base station osmo-bts-trx\n",
    "[ OK ] Base station osmo-bts-trx\n",
    "[SKIP] Second OsmoPCU (BTS#1, GPRS du modem) (disabled by [ \"${NO_OSMO_START:-0}\" != 1 ] && [
↪  \"${CORE_GPRS:-1}\" = 1 ] && [ \"${MODEM_GPRS:-1}\" = 1 ])\n",
    "[ .. ] MS#2 layer 1 (trxcon)\n",
    "[ OK ] MS#2 layer 1 (trxcon)\n",
    "[ .. ] Second subscriber (MS#2)\n",
    "[ OK ] Second subscriber (MS#2)\n",
    "[SKIP] Couche 1 du MS GPRS (trxcon, second MS du fake_trx) (disabled by [ \"${MODEM_GPRS:-1}\" = 1 ])\n",
    "[SKIP] MS GPRS (modem osmocom-bb : attache, PDP, tun) (disabled by [ \"${MODEM_GPRS:-1}\" = 1 ])\n",
    "[SKIP] GSMTAP capture (pcap) (unmet dependency: l2)\n",
    "[ .. ] Supervision view (tmux)\n",
    "[ OK ] Supervision view (tmux)\n",
    "[ .. ] Build identity and summary\n",
    "[ OK ] Build identity and summary\n",
    "[SKIP] Attaching to the tmux session (disabled by [ \"${CALYPSO_NO_ATTACH:-0}\" != 1 ] && [ -t 1 ])\n",
    "\n",
```

```
    "26 ok · 14 skipped · 0 failed\n",
    "\n",
    "  attach   tmux attach -t calypso\n",
    "  logs       /run/user/1001/osmo-nitb/logs\n",
    "  status           ./run.sh --status\n",
    "  stop        ./run.sh --stop\n",
    "\n",
    "[ OK ] Transmission a run.sh (profil=hybrid, chaine Calypso laissee au banc DSP)\n",
    "[ .. ] Banc DSP C54x\n",
    "[ OK ] Banc DSP C54x (/opt/GSM/c54x_exe/run.sh  (INSNS=120000 LOCKSTEP=1 IQ=none))\n",
    "\u001b[1m[run dsp]\u001b[0m 1. c54x_exe --arm  pid 643745  (120000 insn/trame, iq=none,
↪  /dev/shm/calypso_api_ram, /tmp/calypso_dsp.sock)\n",
    "  \u001b[1m[run dsp]\u001b[0m 2. qemu-system-arm  pid 643754  pty modem /dev/pts/26  moniteur
↪  /tmp/qemu-monitor-pont.sock  L1=DSP externe\n",
    "  \u001b[1m[run dsp]\u001b[0m    gdb ARM : telnet 0 44444  (stub tcp::1234, journal
↪  /tmp/c54x-pont/gdb.log)\n",
    "  \u001b[1m[run dsp]\u001b[0m 3. osmocon  pid 643777  L1CTL sur /tmp/osmocom_l2\n",
    "  \u001b[1m[run dsp]\u001b[0m 4. vocodeur ROM (BANC_VOCODEUR=rom) : tch-voice en l1phy ->
↪  /tmp/c54x-pont/mobile_pont_rom.cfg\n",
    "  \u001b[1m[run dsp]\u001b[0m 4. mobile  pid 643855  config /tmp/c54x-pont/mobile_pont_rom.cfg  vty telnet
↪  127.0.0.1 4247\n",
    "  \u001b[1m[run dsp]\u001b[0m 5. pont.py  pid 644098  TRXD 5700-5702 <- BTS ; vers le DSP (udp 6702)\n",
    "  \u001b[1m[run dsp]\u001b[0m 5b. pont audio VSP  pid 644159  gsm_mic.monitor -> calypso_pcm_ul,
↪  calypso_pcm_dl -> gsm_audio\n",
    "  \u001b[1m[run dsp]\u001b[0m tout tourne. Journaux : /tmp/c54x-pont/*.log   suivre : ./run.sh --logs
↪  arreter : ./run.sh --stop\n"
   ]
  }
 ],
 "source": [
  "sh(\"./start.sh --regen --no-attach --bts-classique --ms\")"
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "m04",
 "metadata": {},
 "outputs": [],
 "source": [
  "# Check: BTS#0 alive, the phone camped and registered (allow ~1 min after the start)\n",
  "sh(\"pgrep -af 'osmo-bts-trx -c /etc/osmocom/osmo-bts-trx.cfg$' | head -1; tail -2 /dev/shm/pont.log\")\n",
  "subscribers()"
 ]
},
{
 "cell_type": "code",
 "execution_count": 3,
 "id": "m05",
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "  hub        deduit du remote-ip de l ASP : 127.0.0.1\n",
    "[ .. ] Chargement de l'environnement\n",
    "[ OK ] Chargement de l'environnement (environment/load.env)\n",
    "[ .. ] Resolution de run.sh\n",
    "[ OK ] Resolution de run.sh (/opt/GSM/qosmo/run.sh)\n",
    "[ .. ] Lanceur QEMU\n",
    "[ OK ] Lanceur QEMU (/usr/local/bin/qosmo (qosmo local (/opt/GSM/qosmo)))\n",
    "[ .. ] Validation des chemins\n",
    "[ OK ] Validation des chemins\n",
    "  plan radio  lu dans /etc/osmocom/radio-plan.env\n",
    "  PLMN MS    MCC 001  MNC 01  operateur 1  ->  IMSI 001010001000001 / 001010001000002\n",
```

```
    "  radio      MS#1 sur ARFCN 1 (BTS principale)   MS#2 sur ARFCN 614 (side-car, unit-id 6012)\n",
    "[ .. ] Generation mobile MS#1\n",
    "  MS#1       aussi ecrit dans /root/.osmocom/bb/mobile_group1.cfg (le fichier que run.sh ouvre)\n",
    "[ OK ] Generation mobile MS#1 (/root/.osmocom/bb/mobile.cfg)\n",
    "[ .. ] Generation mobile MS#2 (faketrx)\n",
    "[ OK ] Generation mobile MS#2 (faketrx) (/root/.osmocom/bb/mobile_faketrx_bts1.cfg)\n",
    "\n",
    "╔═══════════════════════════════════════════════════════════╗\n",
    "║   Calypso GSM - bootstrap → run.sh                        ║\n",
    "╚═══════════════════════════════════════════════════════════╝\n",
    "\n",
    "  profil     hybrid (faketrx-qemu)\n",
    "  environ.   machine physique\n",
    "  noeud      aucun - identite SS7 inchangee\n",
    "  run.sh     /opt/GSM/qosmo/run.sh\n",
    "  MS#1       /opt/GSM/c54x_exe/mobile_pont.cfg  IMSI 001010001000001  ARFCN 1  VTY 4247  (banc DSP)\n",
    "             /root/.osmocom/bb/mobile.cfg non lance (module l2 retire du plan)\n",
    "  MS#2       /root/.osmocom/bb/mobile_faketrx_bts1.cfg  IMSI 001010001000002  ARFCN 614  VTY 4248\n",
    "  journaux   /run/user/1001/osmo-nitb/logs\n",
    "  mobile.log /tmp/c54x-pont/mobile.log\n",
    "  chiffrement a5 1\n",
    "\n",
    "[ .. ] Arret de la pile via run.sh\n",
    "\u001b[1m[run dsp]\u001b[0m arret gdb (pid 632935)\n",
    "\u001b[1m[run dsp]\u001b[0m journaux de la session precedente ranges dans
↪ /tmp/c54x-pont/archives/20261028-140831\n",
    "[ .. ] Stopping Attaching to the tmux session\n",
    "[ OK ] Stopping Attaching to the tmux session\n",
    "[ .. ] Stopping Build identity and summary\n",
    "[ OK ] Stopping Build identity and summary\n",
    "[ .. ] Stopping Supervision view (tmux)\n",
    "[ OK ] Stopping Supervision view (tmux)\n",
    "[ .. ] Stopping GSMTAP capture (pcap)\n",
    "[ OK ] Stopping GSMTAP capture (pcap)\n",
    "[ .. ] Stopping MS GPRS (modem osmocom-bb : attache, PDP, tun)\n",
    "[ OK ] Stopping MS GPRS (modem osmocom-bb : attache, PDP, tun)\n",
    "[ .. ] Stopping Layer 2 client\n",
    "[ OK ] Stopping Layer 2 client\n",
    "[ .. ] Stopping Couche 1 du MS GPRS (trxcon, second MS du fake_trx)\n",
    "[ OK ] Stopping Couche 1 du MS GPRS (trxcon, second MS du fake_trx)\n",
    "[ .. ] Stopping Second subscriber (MS#2)\n",
    "[ OK ] Stopping Second subscriber (MS#2)\n",
    "[ .. ] Stopping MS#2 layer 1 (trxcon)\n",
    "[ OK ] Stopping MS#2 layer 1 (trxcon)\n",
    "[ .. ] Stopping Second OsmoPCU (BTS#1, GPRS du modem)\n",
    "[ OK ] Stopping Second OsmoPCU (BTS#1, GPRS du modem)\n",
    "[ .. ] Stopping Base station osmo-bts-trx\n",
    "[ OK ] Stopping Base station osmo-bts-trx\n",
    "[ .. ] Stopping Base station BTS#1 (side-car)\n",
    "[ OK ] Stopping Base station BTS#1 (side-car)\n",
    "[ .. ] Stopping BTS#1 simulated transceiver\n",
    "[ OK ] Stopping BTS#1 simulated transceiver\n",
    "[ .. ] Stopping osmocon gateway\n",
    "[ OK ] Stopping osmocon gateway\n",
    "[ .. ] Stopping Console gdb ARM en telnet (non bloquante)\n",
    "[ OK ] Stopping Console gdb ARM en telnet (non bloquante)\n",
    "[ .. ] Stopping Emulator serial PTY\n",
    "[ OK ] Stopping Emulator serial PTY\n",
    "[ .. ] Stopping Calypso emulator (QEMU)\n",
    "[ OK ] Stopping Calypso emulator (QEMU)\n",
    "[ .. ] Stopping Noise injector (downlink)\n",
    "[ OK ] Stopping Noise injector (downlink)\n",
    "[ .. ] Stopping tmux observation session\n",
    "[ OK ] Stopping tmux observation session\n",
    "[ .. ] Stopping Audio bridge (PulseAudio + GAPK)\n",
    "[ OK ] Stopping Audio bridge (PulseAudio + GAPK)\n",
```

```
    "[ .. ] Stopping Core — SMS gateway (proto-SMSC)\n",
    "[ OK ] Stopping Core — SMS gateway (proto-SMSC)\n",
    "[ .. ] Stopping Core — subscribers provisioned in the HLR\n",
    "[ OK ] Stopping Core — subscribers provisioned in the HLR\n",
    "[ .. ] Stopping Core — MNCC↔SIP gateway\n",
    "[ OK ] Stopping Core — MNCC↔SIP gateway\n",
    "[ .. ] Stopping Mobile configuration\n",
    "[ OK ] Stopping Mobile configuration\n",
    "[ .. ] Stopping Core — Asterisk (SIP PBX)\n",
    "[ OK ] Stopping Core — Asterisk (SIP PBX)\n",
    "[ .. ] Stopping Core — OsmoPCU (packet scheduler)\n",
    "[ OK ] Stopping Core — OsmoPCU (packet scheduler)\n",
    "[ .. ] Stopping Core — OsmoSGSN (packet attach)\n",
    "[ OK ] Stopping Core — OsmoSGSN (packet attach)\n",
    "[ .. ] Stopping Core — OsmoGGSN (GTP data)\n",
    "[ OK ] Stopping Core — OsmoGGSN (GTP data)\n",
    "[ .. ] Stopping Core — APN TUN interface\n",
    "[ OK ] Stopping Core — APN TUN interface\n",
    "[ .. ] Stopping Core — OsmoBSC (BTS controller)\n",
    "[ OK ] Stopping Core — OsmoBSC (BTS controller)\n",
    "[ .. ] Stopping BTS#1 configurations (side-car)\n",
    "[ OK ] Stopping BTS#1 configurations (side-car)\n",
    "[ .. ] Stopping Core — OsmoMSC (CS switching)\n",
    "[ OK ] Stopping Core — OsmoMSC (CS switching)\n",
    "[ .. ] Stopping Core — OsmoMGW (MGCP/RTP media)\n",
    "[ OK ] Stopping Core — OsmoMGW (MGCP/RTP media)\n",
    "[ .. ] Stopping Core — OsmoHLR (GSUP registry)\n",
    "[ OK ] Stopping Core — OsmoHLR (GSUP registry)\n",
    "[ .. ] Stopping Core — OsmoSTP (SS7/M3UA)\n",
    "[ OK ] Stopping Core — OsmoSTP (SS7/M3UA)\n",
    "[ .. ] Stopping Interfaces and addressing plan\n",
    "[ OK ] Stopping Interfaces and addressing plan\n",
    "[ .. ] Stopping Cleaning up the previous run\n",
    "[ OK ] Stopping Cleaning up the previous run\n",
    "[ .. ] Stopping Logs and working area\n",
    "[ OK ] Stopping Logs and working area\n",
    "[ .. ] Stopping Core configuration templates\n",
    "[ OK ] Stopping Core configuration templates\n",
    "[ .. ] Stopping Resolving paths and ports\n",
    "[ OK ] Stopping Resolving paths and ports\n",
    "[ .. ] Stopping Shutting down recorded processes\n",
    "[ OK ] Stopping Shutting down recorded processes\n",
    "[ .. ] Stopping Run directory\n",
    "[ OK ] Stopping Run directory\n",
    "[ .. ] Stopping Host tooling\n",
    "[ OK ] Stopping Host tooling\n",
    "[ .. ] Stopping Prerequisite checks\n",
    "[ OK ] Stopping Prerequisite checks\n",
    "\n",
    "0 ok · 0 skipped · 0 failed\n",
    "[ OK ] Arret de la pile via run.sh\n",
    "[ OK ] python3 du banc termines\n"
   ]
  },
  {
   "data": {
    "text/plain": [
     "0"
    ]
   },
   "execution_count": 3,
   "metadata": {},
   "output_type": "execute_result"
  }
 ],
 "source": [
```

```
   "sh(\"./start.sh --stop\")"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "m06",
  "metadata": {},
  "source": [
   "## 2. Classic BTS + C139 Magnetite (the real TI firmware)\n",
   "\n",
   "Same network; the phone is a Motorola C139 running FreeCalypso Magnetite (its own L2/L3 and UI). Calls need
↪  A5/0\n",
   "(with A5/1 the FACCH fails): `ENCRYPTION=\"a5 0\"` with `--regen`. `--gapk` puts the voice on the host.\n",
   "Building the firmware: `sudo ./addition.sh --magnetite` (Wine + TI TMS470)."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 5,
  "id": "m07",
  "metadata": {},
  "outputs": [
   {
    "ename": "NameError",
    "evalue": "name 'sh' is not defined",
    "output_type": "error",
    "traceback": [
     "\u001b[31m---------------------------------------------------------------------------\u001b[39m",
     "\u001b[31mNameError\u001b[39m                                 Traceback (most recent call last)",
     "\u001b[36mCell\u001b[39m\u001b[36m \u001b[39m\u001b[32mIn[5]\u001b[39m\u001b[32m, line
↪  1\u001b[39m\n\u001b[32m----> \u001b[39m\u001b[32m1\u001b[39m sh(\u001b[33m'ENCRYPTION=\"a5 0\" ./start.sh
↪  --regen --no-attach --bts-classique --ms-magnetite --gapk'\u001b[39m)\n",
     "\u001b[31mNameError\u001b[39m: name 'sh' is not defined"
    ]
   }
  ],
  "source": [
   "sh('ENCRYPTION=\"a5 0\" ./start.sh --regen --no-attach --bts-classique --ms-magnetite --gapk')"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 6,
  "id": "m08",
  "metadata": {},
  "outputs": [
   {
    "ename": "NameError",
    "evalue": "name 'screen' is not defined",
    "output_type": "error",
    "traceback": [
     "\u001b[31m---------------------------------------------------------------------------\u001b[39m",
     "\u001b[31mNameError\u001b[39m                                 Traceback (most recent call last)",
     "\u001b[36mCell\u001b[39m\u001b[36m \u001b[39m\u001b[32mIn[6]\u001b[39m\u001b[32m, line
↪  2\u001b[39m\n\u001b[32m      1\u001b[39m \u001b[38;5;66;03m# The C139 screen (colour). Its traces are in
↪  /tmp/c54x-pont/rvinterf.log.\u001b[39;00m\n\u001b[32m----> \u001b[39m\u001b[32m2\u001b[39m
↪  screen(\u001b[32m1\u001b[39m, scale=\u001b[32m5\u001b[39m)\n\u001b[32m      3\u001b[39m sh(\u001b[33m\"grep
↪  -aE 'L1: (NP_I|RA_C|IAS_C|DDS_R)' /tmp/c54x-pont/rvinterf.log | tail -4\"\u001b[39m)\n\u001b[32m
↪  4\u001b[39m subscribers()\n",
     "\u001b[31mNameError\u001b[39m: name 'screen' is not defined"
    ]
   }
  ],
  "source": [
   "# The C139 screen (colour). Its traces are in /tmp/c54x-pont/rvinterf.log.\n",
   "screen(1, scale=5)\n",
```

```
   "sh(\"grep -aE 'L1: (NP_I|RA_C|IAS_C|DDS_R)' /tmp/c54x-pont/rvinterf.log | tail -4\")\n",
   "subscribers()"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 7,
  "id": "m09",
  "metadata": {},
  "outputs": [
   {
    "ename": "NameError",
    "evalue": "name 'keys' is not defined",
    "output_type": "error",
    "traceback": [
     "\u001b[31m---------------------------------------------------------------------------\u001b[39m",
     "\u001b[31mNameError\u001b[39m                                 Traceback (most recent call last)",
     "\u001b[36mCell\u001b[39m\u001b[36m \u001b[39m\u001b[32mIn[7]\u001b[39m\u001b[32m, line
↪  2\u001b[39m\n\u001b[32m      1\u001b[39m \u001b[38;5;66;03m# Call the echo test 600 from the C139 keypad, then
↪  look at the screen\u001b[39;00m\n\u001b[32m----> \u001b[39m\u001b[32m2\u001b[39m keys(\u001b[32m1\u001b[39m,
↪  \u001b[33m\"6\"\u001b[39m, \u001b[33m\"0\"\u001b[39m, \u001b[33m\"0\"\u001b[39m,
↪  \u001b[33m\"ok\"\u001b[39m)\n\u001b[32m      3\u001b[39m time.sleep(\u001b[32m8\u001b[39m);
↪  screen(\u001b[32m1\u001b[39m, scale=\u001b[32m5\u001b[39m)\n",
     "\u001b[31mNameError\u001b[39m: name 'keys' is not defined"
    ]
   }
  ],
  "source": [
   "# Call the echo test 600 from the C139 keypad, then look at the screen\n",
   "keys(1, \"6\", \"0\", \"0\", \"ok\")\n",
   "time.sleep(8); screen(1, scale=5)"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 8,
  "id": "m10",
  "metadata": {},
  "outputs": [
   {
    "ename": "NameError",
    "evalue": "name 'sh' is not defined",
    "output_type": "error",
    "traceback": [
     "\u001b[31m---------------------------------------------------------------------------\u001b[39m",
     "\u001b[31mNameError\u001b[39m                                 Traceback (most recent call last)",
     "\u001b[36mCell\u001b[39m\u001b[36m \u001b[39m\u001b[32mIn[8]\u001b[39m\u001b[32m, line
↪  1\u001b[39m\n\u001b[32m----> \u001b[39m\u001b[32m1\u001b[39m sh(\u001b[33m\"./start.sh
↪  --stop\"\u001b[39m)\n",
     "\u001b[31mNameError\u001b[39m: name 'sh' is not defined"
    ]
   }
  ],
  "source": [
   "sh(\"./start.sh --stop\")"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "m11",
  "metadata": {},
  "source": [
   "## 3. The Calypso as the BTS radio (`--bts`)\n",
   "\n",
   "The Calypso of QEMU runs the `trx` firmware and the host `transceiver`: it **is** the radio of BTS#0. A
↪  second\n",
```

```
   "Calypso (`--ms`, instance 2) is the phone and camps on it over the air (`pont/air.py`).\n",
   "\n",
   "| `--ts` | Timeslots | State |\n",
   "|---|---|---|\n",
   "| `--ts 1` | TS0 only (CCCH + SDCCH/4) | LU, SMS, no call |\n",
   "| `--ts 2/1` | TS0 + TS1 by one Calypso (default of `--bts`) | calls |\n",
   "| `--ts 2` | TS0 and TS1 by one Calypso each (instances 1 and 3) | camps, LU; no call yet (2nd TRX not
↪  frame-aligned) |"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 9,
  "id": "m12",
  "metadata": {},
  "outputs": [
   {
    "ename": "NameError",
    "evalue": "name 'sh' is not defined",
    "output_type": "error",
    "traceback": [
     "\u001b[31m---------------------------------------------------------------------------\u001b[39m",
     "\u001b[31mNameError\u001b[39m                                 Traceback (most recent call last)",
     "\u001b[36mCell\u001b[39m\u001b[36m \u001b[39m\u001b[32mIn[9]\u001b[39m\u001b[32m, line
↪  1\u001b[39m\n\u001b[32m----> \u001b[39m\u001b[32m1\u001b[39m sh(\u001b[33m\"./start.sh --regen --no-attach
↪  --bts --ms --ts 2\"\u001b[39m)\n",
     "\u001b[31mNameError\u001b[39m: name 'sh' is not defined"
    ]
   }
  ],
  "source": [
   "sh(\"./start.sh --regen --no-attach --bts --ms --ts 2\")"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "m13",
  "metadata": {},
  "outputs": [],
  "source": [
   "# The TRX (instance 1), the phone over the air (instance 2), the 2nd TRX (instance 3)\n",
   "for n in (2,):\n",
   "    print(f\"instance {n}\"); screen(n)\n",
   "sh(\"tail -3 /tmp/c54x-pont2/air.log 2>/dev/null; tail -3 /tmp/c54x-pont/transceiver.log 2>/dev/null\")\n",
   "subscribers()"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "m14",
  "metadata": {},
  "outputs": [],
  "source": [
   "sh(\"./start.sh --stop\")"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "m15",
  "metadata": {},
  "source": [
   "## 4. `banc-rssi`: the rssi app on a 4th Calypso\n",
   "\n",
```

```
   "The osmocom-bb `rssi` app (spectrum, sync, per-timeslot levels) runs on instance 4. In `--bts-classique` the
↪  pont\n",
   "copies every downlink (and uplink) burst to it (UDP 6762, `PONT_DSP_COPIES`); with `--bts` it listens to the
↪  air.\n",
   "The realistic air (`CALYPSO_AIR_NIVEAUX=1`) plays the cell only on its ARFCN, at -60 dBm downlink and -75 dBm
↪  uplink,\n",
   "with noise elsewhere. Start a network first (section 1 or 2), then:"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "m16",
  "metadata": {},
  "outputs": [],
  "source": [
   "sh(\"banc-rssi\")"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "m17",
  "metadata": {},
  "outputs": [],
  "source": [
   "# Spectrum view: ARFCN 1 (the cell) should read about -60 dBm, the next ARFCN the noise floor\n",
   "time.sleep(20); screen(4)\n",
   "keys(4, \"right\"); time.sleep(3); screen(4)     # ARFCN 2: noise\n",
   "keys(4, \"left\")"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "m18",
  "metadata": {},
  "outputs": [],
  "source": [
   "# Sync on the cell (OK), then the per-timeslot view (Up), then the uplink (right soft key)\n",
   "keys(4, \"ok\"); time.sleep(40); screen(4)       # BSIC 7/7, MCC/MNC/LAC, cell id\n",
   "keys(4, \"up\"); time.sleep(5); screen(4)        # 8 timeslots, downlink: all at about -60\n",
   "keys(4, \"rsb\"); time.sleep(5); screen(4)       # uplink: the timeslot of the phone (when it transmits)"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "m19",
  "metadata": {},
  "outputs": [],
  "source": [
   "sh(\"banc-rssi --stop\")"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "m20",
  "metadata": {},
  "source": [
   "## 5. Windows, consoles, stop\n",
   "\n",
   "- Phone windows: `c123 --instance N` (`c123`, `c123-1`, `c123-2`, `c123-3`, `c123-tous` for all of them).\n",
   "- Consoles: `sudo tmux attach -t calypso`; VTY: MSC 4254, BSC 4242, BTS 4241, mobile 4247.\n",
   "- DSP bench logs: `/tmp/c54x-pont*/*.log`; `cd /opt/GSM/c54x_exe && ./run.sh --logs`.\n",
   "- BTS#0 runs under a watchdog (`BTS_GARDIEN=1`): a \"PC clock skew too high\" exit restarts it after 3 s.\n",
```

```
    "- Nothing telecom starts at boot (`services/80-osmo-operator.preset`): `systemctl enable osmo-banc` to change
↪  that."
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "m21",
   "metadata": {},
   "outputs": [],
   "source": [
    "sh(\"./start.sh --stop; banc-rssi --stop\")"
   ]
  }
 ],
 "metadata": {
  "kernelspec": {
   "display_name": "Python 3 (ipykernel)",
   "language": "python",
   "name": "python3"
  },
  "language_info": {
   "codemirror_mode": {
    "name": "ipython",
    "version": 3
   },
   "file_extension": ".py",
   "mimetype": "text/x-python",
   "name": "python",
   "nbconvert_exporter": "python",
   "pygments_lexer": "ipython3",
   "version": "3.12.3"
  }
 },
 "nbformat": 4,
 "nbformat_minor": 5
}
```

3.144 /opt/GSM/osmo-operator/wiki/lancer-pas-a-pas.ipynb

58974 octets, 1445 lignes → 1445 lignes

```
{
 "cells": [
  {
   "cell_type": "markdown",
   "id": "c00",
   "metadata": {},
   "source": [
    "# Launching the bench step by step, up to a call on the Calypso DSP\n",
    "\n",
    "This notebook starts the bench **one process at a time** and checks each one before adding the next:\n",
    "first the Osmocom network (STP, HLR, MGW, MSC, BSC, SIP side, BTS), then the emulated phone (the C54x DSP,\n",
    "QEMU with the ARM and the layer 1 firmware, osmocon, mobile, the bridge), and finally camp, location\n",
    "update and a call. It runs the same commands as `start.sh` and `c54x_exe/run.sh`, written out by\n",
    "hand, so you can see what each component does and where a fault comes from.\n",
    "\n",
    "**It installs nothing.** Run it where the bench is already built: the `osmocom-run` image from\n",
    "`build.sh`, the ISO, or a machine after the [install notebook](installer-osmo-jusquau-dsp.ipynb).\n",
    "\n",
    "| | |\n",
    "|---|---|\n",
    "| Needs | everything under `/opt/GSM` (osmo-operator, c54x_exe, qosmo, osmocom-bb, firmware, DSP ROM) and the
↪  Osmocom daemons in the `PATH` |\n",
```

```
  "| Runs as | root (or a kernel started after `sudo -v`) |\n",
  "| Logs | `/tmp/c54x-pont/<name>.log`, one per process, pid in `<name>.pid` |\n",
  "| Time | a few minutes |\n",
  "\n",
  "Rules: one process = one cell, followed by its **check**. A check exits with an error if what it waits\n",
  "for does not show up, and prints the end of the log. Do not move on after a failed check: fix it first,\n",
  "the next steps depend on it. Stop everything with the last cell before starting over.\n",
  "\n",
  "> [2026-10-23] The launcher is now `./start.sh`; `start-native.sh` is a symlink to it and still works.\n",
  "\n",
  "In the `osmocom-run` image:\n",
  "\n",
  "```bash\n",
  "docker run -it --rm --privileged --shm-size=2g -p 8888:8888 --entrypoint bash osmocom-run:latest\n",
  "python3 -m venv /venv && /venv/bin/pip install -q jupyterlab\n",
  "cd /opt/GSM/osmo-operator/wiki && /venv/bin/jupyter lab --ip=0.0.0.0 --allow-root --no-browser \\\n",
  "  --LabApp.default_url=/lab/tree/lancer-pas-a-pas.ipynb\n",
  "```"
 ]
},
{
 "cell_type": "markdown",
 "id": "c01",
 "metadata": {},
 "source": [
  "## 0. Helpers, cleanup, configs\n",
  "\n",
  "The helpers go into `/tmp/c54x-pont/lib.sh`, sourced by every cell (`%%bash` cells do not share their\n",
  "environment):\n",
  "\n",
  "- `start NAME CMD…` starts a process detached from the notebook, log in `NAME.log`, pid in `NAME.pid`;\n",
  "- `wait_log FILE PATTERN [SECONDS]` waits for a line, prints it, or prints the end of the log and fails;\n",
  "- `wait_tcp PORT [SECONDS]` / `wait_udp PORT [SECONDS]` wait for a listening port;\n",
  "- `vty PORT COMMAND` sends one command to an Osmocom VTY.\n",
  "\n",
  "The three `CALYPSO_*` variables are the DSP-mode settings `start.sh` exports before handing over\n",
  "to `c54x_exe/run.sh`; every phone-side process inherits them."
 ]
},
{
 "cell_type": "code",
 "execution_count": 3,
 "id": "c02",
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "helpers written to /tmp/c54x-pont/lib.sh\n"
   ]
  }
 ],
 "source": [
  "%%bash\n",
  "mkdir -p /tmp/c54x-pont\n",
  "cat > /tmp/c54x-pont/lib.sh <<'EOF'\n",
  "L=/tmp/c54x-pont\n",
  "ROOT=/opt/GSM\n",
  "O=$ROOT/osmo-operator\n",
  "export CALYPSO_BSP_STREAM=1        # one TS0 burst per frame, in FN order\n",
  "export CALYPSO_RHEA_DMA_XFER=1     # without it: no SB, no BCCH, no SI\n",
  "export CALYPSO_PONT_LOCKSTEP=1     # QEMU only advances the frame once the DSP is done\n",
  "[ \"$(id -u)\" = 0 ] && SUDO=\"\" || SUDO=\"sudo -n\"\n",
  "\n",
  "start() {   # start NAME CMD...\n",
```

```
   "  local n=$1; shift\n",
   "  setsid nohup \"$@\" > \"$L/$n.log\" 2>&1 < /dev/null &\n",
   "  echo $! > \"$L/$n.pid\"\n",
   "  echo \"started $n (pid $!)\"\n",
   "}\n",
   "alive() { kill -0 \"$(cat \"$L/$1.pid\" 2>/dev/null)\" 2>/dev/null; }\n",
   "wait_log() {   # wait_log FILE PATTERN [SECONDS]\n",
   "  local f=$1 p=$2 t=${3:-10} i\n",
   "  for i in $(seq $((t*2))); do\n",
   "    grep -aq -- \"$p\" \"$f\" 2>/dev/null && { echo \"ok      $(grep -a -- \"$p\" \"$f\" | tail -1)\"; return
↪  0; }\n",
   "    sleep 0.5\n",
   "  done\n",
   "  echo \"TIMEOUT waiting for \\\"$p\\\" in $f\"; tail -15 \"$f\" 2>/dev/null; return 1\n",
   "}\n",
   "wait_tcp() { local i; for i in $(seq $((${2:-10}*2))); do ss -ltnH \"( sport = :$1 )\" | grep -q . && { echo
↪  \"ok      tcp $1 listening\"; return 0; }; sleep 0.5; done; echo \"TIMEOUT tcp $1\"; return 1; }\n",
   "wait_udp() { local i; for i in $(seq $((${2:-10}*2))); do ss -ulnH \"( sport = :$1 )\" | grep -q . && { echo
↪  \"ok      udp $1 bound\"; return 0; }; sleep 0.5; done; echo \"TIMEOUT udp $1\"; return 1; }\n",
   "vty() { { printf 'enable\\n%s\\n' \"$2\"; sleep 1; } | timeout --foreground 6 nc -N 127.0.0.1 \"$1\"
↪  2>/dev/null | tr -d '\\r'; }\n",
   "EOF\n",
   "echo \"helpers written to /tmp/c54x-pont/lib.sh\""
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 4,
  "id": "c03",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "ok      /opt/GSM/c54x_exe/c54x_exe\n",
     "ok      /opt/GSM/c54x_exe/mobile_pont.cfg\n",
     "ok      /opt/GSM/qosmo/build/qemu-system-arm\n",
     "ok      /opt/GSM/osmocom-bb/src/host/osmocon/osmocon\n",
     "ok      /opt/GSM/firmware/board/compal_e88/layer1.highram.elf\n",
     "ok      /opt/GSM/firmware/board/compal_e88/layer1.highram.bin\n",
     "ok      /opt/GSM/osmo-operator/pont/pont_dsp.py\n",
     "ok      osmo-stp\n",
     "ok      osmo-hlr\n",
     "ok      osmo-mgw\n",
     "ok      osmo-msc\n",
     "ok      osmo-bsc\n",
     "ok      osmo-bts-trx\n",
     "ok      osmo-sip-connector\n",
     "ok      asterisk\n",
     "ok      mobile\n",
     "ok      nc\n",
     "ok      socat\n",
     "ALREADY RUNNING (run the Stop cell at the end first):\n",
     "678859 osmo-stp -c /etc/osmocom/osmo-stp.cfg\n",
     "678905 osmo-hlr -c /etc/osmocom/osmo-hlr.cfg\n",
     "679200 osmo-mgw -c /etc/osmocom/osmo-mgw.cfg\n",
     "679216 osmo-msc -c /etc/osmocom/osmo-msc.cfg\n",
     "679277 osmo-bsc -c /etc/osmocom/osmo-bsc.cfg\n",
     "679451 osmo-sip-connector -c /etc/osmocom/osmo-sip-connector.cfg\n"
    ]
   },
   {
    "ename": "CalledProcessError",
```

```
    "evalue": "Command 'b'. /tmp/c54x-pont/lib.sh\\n# Check: the pieces this notebook runs, and nothing from a
 ↪  previous session still running\\nfail=0\\nfor f in \"$ROOT/c54x_exe/c54x_exe\"
 ↪  \"$ROOT/c54x_exe/mobile_pont.cfg\" \"$ROOT/qosmo/build/qemu-system-arm\" \\\\\\n
 ↪  \"$ROOT/osmocom-bb/src/host/osmocon/osmocon\" \"$ROOT/firmware/board/compal_e88/layer1.highram.elf\" \\\\\\n
 ↪  \"$ROOT/firmware/board/compal_e88/layer1.highram.bin\" \"$O/pont/pont_dsp.py\"; do\\n  [ -e \"$f\" ] && echo
 ↪  \"ok      $f\" || { echo \"MISSING $f\"; fail=1; }\\ndone\\nfor b in osmo-stp osmo-hlr osmo-mgw osmo-msc
 ↪  osmo-bsc osmo-bts-trx osmo-sip-connector asterisk mobile nc socat; do\\n  command -v \"$b\" >/dev/null && echo
 ↪  \"ok      $b\" || { echo \"MISSING $b\"; fail=1; }\\ndone\\nrunning=$(pgrep -a -f
 ↪  \\'osmo-(stp|hlr|mgw|msc|bsc|bts-trx|sip-connector)|c54x_exe|qemu-system-arm|osmocon|mobile -c|pont_dsp\\' ||
 ↪  true)\\n[ -n \"$running\" ] && { echo \"ALREADY RUNNING (run the Stop cell at the end ↩
first):\"; echo \"$running\"; fail=1; }\\nexit $fail\\n'' returned non-zero exit status 1.",
    "output_type": "error",
    "traceback": [
     "\u001b[31m---------------------------------------------------------------------------\u001b[39m",
     "\u001b[31mCalledProcessError\u001b[39m                        Traceback (most recent call last)",
     "\u001b[36mCell\u001b[39m\u001b[36m \u001b[39m\u001b[32mIn[4]\u001b[39m\u001b[32m, line
 ↪  1\u001b[39m\n\u001b[32m----> \u001b[39m\u001b[32m1\u001b[39m
 ↪  get_ipython().run_cell_magic(\u001b[33m'bash'\u001b[39m, \u001b[33m''\u001b[39m, \u001b[33m'.
 ↪  /tmp/c54x-pont/lib.sh\\n# Check: the pieces this notebook runs, and nothing from a previous session still
 ↪  running\\nfail=0\\nfor f in \"$ROOT/c54x_exe/c54x_exe\" \"$ROOT/c54x_exe/mobile_pont.cfg\"
 ↪  \"$ROOT/qosmo/build/qemu-system-arm\" \\\\\\n         \"$ROOT/osmocom-bb/src/host/osmocon/osmocon\"
 ↪  \"$ROOT/firmware/board/compal_e88/layer1.highram.elf\" \\\\\\n
 ↪  \"$ROOT/firmware/board/compal_e88/layer1.highram.bin\" \"$O/pont/pont_dsp.py\"; do\\n  [ -e \"$f\" ] && echo
 ↪  \"ok      $f\" || { echo \"MISSING $f\"; fail=1; }\\ndone\\nfor b in osmo-stp osmo-hlr osmo-mgw osmo-msc
 ↪  osmo-bsc osmo-bts-trx osmo-sip-connector asterisk mobile nc socat; do\\n  command -v \"$b\" >/dev/null && echo
 ↪  \"ok      $b\" || { echo \"MISSING $b\"; fail=1; }\\ndone\↩
\nrunning=$(pgrep -a -f
 ↪  \\'osmo-(stp|hlr|mgw|msc|bsc|bts-trx|sip-connector)|c54x_exe|qemu-system-arm|osmocon|mobile -c|pont_dsp\\' ||
 ↪  true)\\n[ -n \"$running\" ] && { echo \"ALREADY RUNNING (run the Stop cell at the end first):\"; echo
 ↪  \"$running\"; fail=1; }\\nexit $fail\\n'\u001b[39m)\n",
     "\u001b[31mCalledProcessError\u001b[39m: Command 'b'. /tmp/c54x-pont/lib.sh\\n# Check: the pieces this
 ↪  notebook runs, and nothing from a previous session still running\\nfail=0\\nfor f in
 ↪  \"$ROOT/c54x_exe/c54x_exe\" \"$ROOT/c54x_exe/mobile_pont.cfg\" \"$ROOT/qosmo/build/qemu-system-arm\" \\\\\\n
 ↪  \"$ROOT/osmocom-bb/src/host/osmocon/osmocon\" \"$ROOT/firmware/board/compal_e88/layer1.highram.elf\" \\\\\\n
 ↪  \"$ROOT/firmware/board/compal_e88/layer1.highram.bin\" \"$O/pont/pont_dsp.py\"; do\\n  [ -e \"$f\" ] && echo
 ↪  \"ok      $f\" || { echo \"MISSING $f\"; fail=1; }\\ndone\\nfor b in osmo-stp osmo-hlr osmo-mgw osmo-msc
 ↪  osmo-bsc osmo-bts-trx osmo-sip-connector asterisk mobile nc socat; do\\n  command -v \"$b\" >/dev/null && echo
 ↪  \"ok      $b\" || { echo \"MISSING $b\"; fail=1; }\\ndone\\nrunning=$(pgrep -a -f
 ↪  \\'osmo-(stp|hlr|mgw|msc|bsc|bts-trx|sip-connector)|c54x_exe|qemu-system-arm|osmocon|mobile -c|pont_dsp\\' ||
 ↪  true)\\n[ -n \"$running\" ] && { echo \"ALREADY RUNNING↩
 (run the Stop cell at the end first):\"; echo \"$running\"; fail=1; }\\nexit $fail\\n'' returned non-zero exit
 ↪  status 1."
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /tmp/c54x-pont/lib.sh\n",
   "# Check: the pieces this notebook runs, and nothing from a previous session still running\n",
   "fail=0\n",
   "for f in \"$ROOT/c54x_exe/c54x_exe\" \"$ROOT/c54x_exe/mobile_pont.cfg\"
 ↪  \"$ROOT/qosmo/build/qemu-system-arm\" \\\n",
   "        \"$ROOT/osmocom-bb/src/host/osmocon/osmocon\" \"$ROOT/firmware/board/compal_e88/layer1.highram.elf\"
 ↪  \\\n",
   "         \"$ROOT/firmware/board/compal_e88/layer1.highram.bin\" \"$O/pont/pont_dsp.py\"; do\n",
   "  [ -e \"$f\" ] && echo \"ok      $f\" || { echo \"MISSING $f\"; fail=1; }\n",
   "done\n",
   "for b in osmo-stp osmo-hlr osmo-mgw osmo-msc osmo-bsc osmo-bts-trx osmo-sip-connector asterisk mobile nc
 ↪  socat; do\n",
   "  command -v \"$b\" >/dev/null && echo \"ok      $b\" || { echo \"MISSING $b\"; fail=1; }\n",
   "done\n",
   "running=$(pgrep -a -f
 ↪  'osmo-(stp|hlr|mgw|msc|bsc|bts-trx|sip-connector)|c54x_exe|qemu-system-arm|osmocon|mobile -c|pont_dsp' ||
 ↪  true)\n",
   "[ -n \"$running\" ] && { echo \"ALREADY RUNNING (run the Stop cell at the end first):\"; echo \"$running\";
 ↪  fail=1; }\n",
```

```
  "exit $fail"
 ]
},
{
 "cell_type": "markdown",
 "id": "c04",
 "metadata": {},
 "source": [
  "The Osmocom configs in `/etc/osmocom` are templates: markers such as `__MCC__`, `__ARFCN__`, `__IMSI__`\n",
  "are filled from `globals.conf` by `./start.sh --regen`. A daemon started on an unfilled config\n",
  "refuses it, so check first."
 ]
},
{
 "cell_type": "code",
 "execution_count": 3,
 "id": "c05",
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "ok      all configs filled\n",
    "MCC=\"001\"\n",
    "MNC=\"01\"\n",
    "ARFCN=\"1\"\n",
    "BAND=\"DCS1800\"\n",
    "BSIC=\"7\"\n",
    "LAC=\"1\"\n"
   ]
  }
 ],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "# Check: no template marker left in the configs this notebook uses\n",
  "left=$(grep -l '__[A-Z0-9_]*__' /etc/osmocom/osmo-{stp,hlr,mgw,msc,bsc,bts-trx,sip-connector}.cfg \\\n",
  "       \"$ROOT/c54x_exe/mobile_pont.cfg\" 2>/dev/null || true)\n",
  "if [ -n \"$left\" ]; then\n",
  "  echo \"unfilled templates:\"; echo \"$left\"\n",
  "  echo \"fill them first: cd $O && ./start.sh --regen, then ./start.sh --stop\"\n",
  "  exit 1\n",
  "fi\n",
  "echo \"ok      all configs filled\"\n",
  "grep -E '^(MCC|MNC|BAND|ARFCN|BSIC|LAC)=' \"$O/globals.conf\""
 ]
},
{
 "cell_type": "markdown",
 "id": "c06",
 "metadata": {},
 "source": [
  "---\n",
  "# Part 1: the network\n",
  "\n",
  "Bench order: STP, HLR, MGW, MSC, BSC, then the SIP side for calls, then the BTS. Each daemon gets its VTY\n",
  "checked before the next one starts. VTY ports are the Osmocom defaults: STP 4239, HLR 4258, MGW 4243,\n",
  "MSC 4254, BSC 4242, SIP connector 4255, BTS 4241.\n",
  "\n",
  "## 1. osmo-stp, the SS7 signalling point\n",
  "\n",
  "Every link of the core (BSC↔MSC over SCCP/M3UA) goes through it, so it starts first."
 ]
},
{
```

```
  "cell_type": "code",
  "execution_count": 4,
  "id": "c07",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "started stp (pid 678859)\n"
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /tmp/c54x-pont/lib.sh\n",
   "start stp osmo-stp -c /etc/osmocom/osmo-stp.cfg"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 5,
  "id": "c08",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "ok      tcp 4239 listening\n",
     "Welcome to the OsmoSTP VTY interface\n",
     "\n",
     "Copyright (C) 2015-2020 by Harald Welte <laforge@gnumonks.org>\n",
     "Contributions by Holger Freyther, Neels Hofmeyr, Pau Espin, Vadim Yanitskiy\n",
     "License GPLv2+: GNU GPL Version 2 or later <http://gnu.org/licenses/gpl-2.0.html>\n",
     "This is free software: you are free to change and redistribute it.\n",
     "There is NO WARRANTY, to the extent permitted by law.\n",
     "\n",
     "Free Software lives by contribution.  If you use this, please contribute!\n",
     "\u0001\u0003\"\u001fOsmoSTP> enable\n",
     "OsmoSTP# show cs7 instance 0 asp\n",
    "ASP Name     AS Name      State         Type  Role  SCTP Role  Local Addresses         Remote Addresses\n",
     "------------  ------------  ------------  ----  ----  ---------  ----------------------
↪  ----------------------\n",
     "asp-to-inter  as-inter      ASP_DOWN       m3ua  asp   client     127.0.0.1:2910          127.0.0.1:2908
↪  \n",
     "OsmoSTP# "
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /tmp/c54x-pont/lib.sh\n",
   "# Check\n",
   "wait_tcp 4239 && vty 4239 \"show cs7 instance 0 asp\""
  ]
 },
 {
  "cell_type": "markdown",
  "id": "c09",
  "metadata": {},
  "source": [
   "## 2. osmo-hlr, the subscriber database\n",
   "\n",
   "The database path is set in `osmo-hlr.cfg` (`/var/lib/osmocom/hlr.db`)."
  ]
```

```
},
{
 "cell_type": "code",
 "execution_count": 6,
 "id": "c10",
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "started hlr (pid 678905)\n"
   ]
  }
 ],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "mkdir -p /var/lib/osmocom\n",
  "start hlr osmo-hlr -c /etc/osmocom/osmo-hlr.cfg"
 ]
},
{
 "cell_type": "code",
 "execution_count": 7,
 "id": "c11",
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "ok      tcp 4258 listening\n",
    "Copyright (C) 2016-2023 by Harald Welte, sysmocom s.f.m.c. GmbH\n",
    "License AGPLv3+: GNU AGPL version 3 or later <http://gnu.org/licenses/agpl-3.0.html>\n",
    "This is free software: you are free to change and redistribute it.\n",
    "There is NO WARRANTY, to the extent permitted by law.\n",
    "\u0001\u0003\"\u001fOsmoHLR> enable\n",
    "OsmoHLR# show subscribers all\n",
    "ID     MSISDN        IMSI              IMEI              NAM\n",
    "-----  ------------  ----------------  ----------------  -----\n",
    "1      100101        001010001000001   490154203237518   CSPS  \n",
    "2      100102        001010001000002   358925003056310   CSPS  \n",
    " Subscribers Shown: 2\n",
    "OsmoHLR# "
   ]
  }
 ],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "# Check\n",
  "wait_tcp 4258 && vty 4258 \"show subscribers all\" | tail -n +3"
 ]
},
{
 "cell_type": "markdown",
 "id": "c12",
 "metadata": {},
 "source": [
  "### 2b. The phone's subscriber\n",
  "\n",
  "The emulated phone uses the test SIM of `mobile_pont.cfg` (`test-sim`: `imsi`, `ki`). Its IMSI must be in\n",
  "the HLR with the same Ki, or the location update is rejected. This cell reads them from the config and\n",
  "creates the subscriber if it is missing. The MSISDN is up to you."
 ]
```

```
},
{
 "cell_type": "code",
 "execution_count": 8,
 "id": "c13",
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "IMSI=001010001000001 ALGO=comp128 KI=00112233445566778899aabbccdd0101\n",
    "subscriber already present\n"
   ]
  }
 ],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "cfg=\"$ROOT/c54x_exe/mobile_pont.cfg\"\n",
  "IMSI=$(awk '/^ *imsi [0-9]+/ {print $2; exit}' \"$cfg\")\n",
  "ALGO=$(awk '/^ *ki / {print $2; exit}' \"$cfg\")\n",
  "KI=$(awk '/^ *ki / {s=\"\"; for (i=3; i<=NF; i++) s=s $i; print s; exit}' \"$cfg\")\n",
  "MSISDN=1001\n",
  "echo \"IMSI=$IMSI ALGO=$ALGO KI=$KI\"\n",
  "[ -n \"$IMSI\" ] && [ -n \"$KI\" ] || { echo \"no test-sim imsi/ki found in $cfg\"; exit 1; }\n",
  "case \"$ALGO\" in comp128) HALGO=comp128v1 ;; xor) HALGO=xor-2g ;; *) HALGO=$ALGO ;; esac\n",
  "if vty 4258 \"show subscriber imsi $IMSI\" | grep -q \"IMSI: $IMSI\"; then\n",
  "  echo \"subscriber already present\"\n",
  "else\n",
  "  { printf 'enable\\n'\n",
  "    printf 'subscriber imsi %s create\\n' \"$IMSI\"\n",
  "    printf 'subscriber imsi %s update msisdn %s\\n' \"$IMSI\" \"$MSISDN\"\n",
  "    printf 'subscriber imsi %s update aud2g %s ki %s\\n' \"$IMSI\" \"$HALGO\" \"$KI\"\n",
  "    sleep 1; } | timeout 6 nc -N 127.0.0.1 4258 | tr -d '\\r' | grep -iE 'error|%' || true\n",
  "fi"
 ]
},
{
 "cell_type": "code",
 "execution_count": 9,
 "id": "c14",
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "Copyright (C) 2016-2023 by Harald Welte, sysmocom s.f.m.c. GmbH\n",
    "License AGPLv3+: GNU AGPL version 3 or later <http://gnu.org/licenses/agpl-3.0.html>\n",
    "This is free software: you are free to change and redistribute it.\n",
    "There is NO WARRANTY, to the extent permitted by law.\n",
    "\u0001\u0003\"\u001fOsmoHLR> enable\n",
    "OsmoHLR# show subscriber imsi 001010001000001\n",
    "    ID: 1\n",
    "    IMSI: 001010001000001\n",
    "    MSISDN: 100101\n",
    "    IMEI: 490154203237518\n",
    "    VLR number: VLR-SDR-OP1\n",
    "    CS purged\n",
    "    last LU seen on CS: 2026-10-26T19:47:42+00:00 (19h53m38s ago)\n",
    "    2G auth: COMP128v1\n",
    "             KI=00112233445566778899aabbccdd0101\n",
    "OsmoHLR# \n"
   ]
  }
```

```
  ],
  "source": [
   "%%bash\n",
   ". /tmp/c54x-pont/lib.sh\n",
   "# Check: the subscriber, with its 2G key\n",
   "IMSI=$(awk '/^ *imsi [0-9]+/ {print $2; exit}' \"$ROOT/c54x_exe/mobile_pont.cfg\")\n",
   "out=$(vty 4258 \"show subscriber imsi $IMSI\"); echo \"$out\" | tail -n +3\n",
   "echo \"$out\" | grep -q \"IMSI: $IMSI\" && echo \"$out\" | grep -qi \"2G\" || { echo \"subscriber or its 2G
↪  key missing\"; exit 1; }"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "c15",
  "metadata": {},
  "source": [
   "## 3. osmo-mgw, the media gateway\n",
   "\n",
   "Carries the voice (RTP) between the BTS and the MSC side."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 10,
  "id": "c16",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "started mgw (pid 679200)\n"
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /tmp/c54x-pont/lib.sh\n",
   "start mgw osmo-mgw -c /etc/osmocom/osmo-mgw.cfg"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 11,
  "id": "c17",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "ok      tcp 4243 listening\n",
     "Welcome to the OsmoMGW VTY interface\n",
     "\n",
     "Copyright (C) 2009-2010 Holger Freyther and On-Waves\n",
     "Copyright (C) 2017-2022 by sysmocom s.f.m.c. GmbH\n",
     "Contributions by Pablo Neira Ayuso, Jacob Erlbeck, Neels Hofmeyr\n",
     "Philipp Maier\n",
     "\n",
     "License AGPLv3+: GNU AGPL version 3 or later <http://gnu.org/licenses/agpl-3.0.html>\n"
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /tmp/c54x-pont/lib.sh\n",
```

```
   "# Check\n",
   "wait_tcp 4243 && vty 4243 \"show mgcp\" | head -8"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "c18",
  "metadata": {},
  "source": [
   "## 4. osmo-msc, the switching centre\n",
   "\n",
   "It connects to the STP (SCCP) and to the HLR (GSUP). Its calls go out through an **external MNCC socket**\n",
   "(`mncc external /tmp/msc_mncc` in `osmo-msc.cfg`): without the SIP connector of step 6, the network\n",
   "works for camp, LU and SMS, but a call cannot be set up."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 12,
  "id": "c19",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "started msc (pid 679216)\n"
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /tmp/c54x-pont/lib.sh\n",
   "start msc osmo-msc -c /etc/osmocom/osmo-msc.cfg"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 13,
  "id": "c20",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "ok      tcp 4254 listening\n",
     "ok      /tmp/msc_mncc\n",
     "Welcome to the OsmoSTP VTY interface\n",
     "\n",
     "Copyright (C) 2015-2020 by Harald Welte <laforge@gnumonks.org>\n",
     "Contributions by Holger Freyther, Neels Hofmeyr, Pau Espin, Vadim Yanitskiy\n",
     "License GPLv2+: GNU GPL Version 2 or later <http://gnu.org/licenses/gpl-2.0.html>\n",
     "This is free software: you are free to change and redistribute it.\n",
     "There is NO WARRANTY, to the extent permitted by law.\n",
     "\n",
     "Free Software lives by contribution.  If you use this, please contribute!\n",
     "\u0001\u0003\"\u001fOsmoSTP> enable\n",
     "OsmoSTP# show cs7 instance 0 asp\n",
    "ASP Name     AS Name      State         Type  Role  SCTP Role  Local Addresses         Remote Addresses\n",
     "------------  ------------  ------------  ----  ----  ---------  ----------------------
↪  ----------------------\n",
     "asp-to-inter  as-inter      ASP_DOWN       m3ua  asp   client     127.0.0.1:2910          127.0.0.1:2908
↪  \n",
     "asp-dyn-0     ?            ASP_INACTIVE   m3ua  sg    server     (127.0.0.1|127.0.0.2):2905  127.0.0.1:2906
↪  \n",
```

```
    "OsmoSTP# "
   ]
  }
 ],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "# Check: VTY, the MNCC socket, and the MSC's SCCP user on the STP\n",
  "wait_tcp 4254 && test -S /tmp/msc_mncc && echo \"ok      /tmp/msc_mncc\"\n",
  "vty 4239 \"show cs7 instance 0 asp\""
 ]
},
{
 "cell_type": "markdown",
 "id": "c21",
 "metadata": {},
 "source": [
  "## 5. osmo-bsc, the base station controller\n",
  "\n",
  "Connects to the MSC through the STP, and waits for the BTS (OML/RSL) on Abis."
 ]
},
{
 "cell_type": "code",
 "execution_count": 14,
 "id": "c22",
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "started bsc (pid 679277)\n"
   ]
  }
 ],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "start bsc osmo-bsc -c /etc/osmocom/osmo-bsc.cfg"
 ]
},
{
 "cell_type": "code",
 "execution_count": 15,
 "id": "c23",
 "metadata": {},
 "outputs": [
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "ok      tcp 4242 listening\n",
    "BTS 0 is of osmo-bts type in band GSM900, has CI 6007 LAC 1, BSIC 7 (NCC=0, BCC=7) and 1 TRX\n",
    "  OML Link: \n",
    "  OML Link state: disconnected\n"
   ]
  }
 ],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "# Check: VTY, and BTS 0 declared (OML not connected yet: the BTS comes at step 7)\n",
  "wait_tcp 4242 && vty 4242 \"show bts 0\" | grep -E \"BTS 0|OML Link\""
 ]
},
```

```
 {
  "cell_type": "markdown",
  "id": "c24",
  "metadata": {},
  "source": [
   "## 6. Asterisk and osmo-sip-connector, for calls\n",
   "\n",
   "`osmo-sip-connector` takes the MSC's MNCC socket and turns the call into SIP towards Asterisk, where\n",
   "`extensions.conf` routes number 600 to the echo test. Skip this step if you only want camp, LU and SMS."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 16,
  "id": "c25",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "/etc/asterisk/extensions.conf\n",
     "/etc/asterisk/pjsip.conf\n",
     "started sip (pid 679451)\n"
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /tmp/c54x-pont/lib.sh\n",
   "ls /etc/asterisk/extensions.conf /etc/asterisk/pjsip.conf || { echo \"Asterisk configs missing\"; exit 1;
↪  }\n",
   "mkdir -p /var/run/asterisk /var/log/asterisk /var/lib/asterisk\n",
   "asterisk -rx \"core show version\" >/dev/null 2>&1 || asterisk\n",
   "start sip osmo-sip-connector -c /etc/osmocom/osmo-sip-connector.cfg"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 17,
  "id": "c26",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "No such command 'core show version' (type 'core show help core' for other possible commands)\n",
     "Command 'dialplan show 600@default' failed.\n",
     "ok      tcp 4255 listening\n",
     "<0001> mncc.c:1179 Scheduling MNCC connect\n",
     "<0001> mncc.c:1031 Reconnected to /tmp/msc_mncc\n",
     "<0001> mncc.c:918 Got hello message version 8\n"
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /tmp/c54x-pont/lib.sh\n",
   "# Check: Asterisk answers, number 600 exists, the connector holds the MNCC socket.\n",
   "# Asterisk forks and needs a few seconds before its control socket answers.\n",
   "for i in $(seq 1 20); do asterisk -rx \"core show version\" 2>/dev/null && break; sleep 0.5; done\n",
   "asterisk -rx \"core show version\" >/dev/null 2>&1 || { echo \"Asterisk not answering after 10 s\"; tail -15
↪  /var/log/asterisk/messages* 2>/dev/null; exit 1; }\n",
   "asterisk -rx \"dialplan show 600@default\" 2>/dev/null | grep -E \"600|Echo\" || asterisk -rx \"dialplan
↪  show\" | grep -m3 600\n",
```

```
   "wait_tcp 4255 && grep -ai \"mncc\" \"$L/sip.log\" | tail -3"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "c27",
  "metadata": {},
  "source": [
   "## 7. osmo-bts-trx, the base station\n",
   "\n",
   "It connects to the BSC and starts sending its TRXC commands to UDP 5701, where **nobody answers yet**:\n",
   "the bridge (step 12) will. That is expected.\n",
   "\n",
   "Started by hand like this, osmo-bts-trx runs **once**: if it exits (\"PC clock skew too high\" when the\n",
   "DSP stalls the bridge's clock), restart this cell. `./start.sh` runs it inside a watchdog loop instead\n",
   "(see the Stop section)."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 18,
  "id": "c28",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "started bts (pid 679600)\n"
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /tmp/c54x-pont/lib.sh\n",
   "start bts osmo-bts-trx -c /etc/osmocom/osmo-bts-trx.cfg"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 19,
  "id": "c29",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "ok      tcp 4241 listening\n",
     "osmo-bts-trx not bound on 5800-5802\n"
    ]
   },
   {
    "ename": "CalledProcessError",
    "evalue": "Command 'b'. /tmp/c54x-pont/lib.sh\\n# Check: VTY, the clock/data ports, and OML seen by the
↪  BSC\\nwait_tcp 4241 || exit 1\\nss -ulnp | grep -E \\':580[0-2] \\' || { echo \"osmo-bts-trx not bound on
↪  5800-5802\"; exit 1; }\\nsleep 2; vty 4242 \"show bts 0\" | grep -E \"OML Link\"\\n'' returned non-zero exit
↪  status 1.",
    "output_type": "error",
    "traceback": [
     "\u001b[31m---------------------------------------------------------------------------\u001b[39m",
     "\u001b[31mCalledProcessError\u001b[39m                        Traceback (most recent call last)",
```

```
    "\u001b[36mCell\u001b[39m\u001b[36m \u001b[39m\u001b[32mIn[19]\u001b[39m\u001b[32m, line
↪ 1\u001b[39m\n\u001b[32m----> \u001b[39m\u001b[32m1\u001b[39m
↪ get_ipython().run_cell_magic(\u001b[33m'bash'\u001b[39m, \u001b[33m''\u001b[39m, \u001b[33m'.
↪ /tmp/c54x-pont/lib.sh\\n# Check: VTY, the clock/data ports, and OML seen by the BSC\\nwait_tcp 4241 || exit
↪ 1\\nss -ulnp | grep -E \\':580[0-2] \\' || { echo \"osmo-bts-trx not bound on 5800-5802\"; exit 1; }\\nsleep
↪ 2; vty 4242 \"show bts 0\" | grep -E \"OML Link\"\\n'\u001b[39m)\n",
    "\u001b[31mCalledProcessError\u001b[39m: Command 'b'. /tmp/c54x-pont/lib.sh\\n# Check: VTY, the clock/data
↪ ports, and OML seen by the BSC\\nwait_tcp 4241 || exit 1\\nss -ulnp | grep -E \\':580[0-2] \\' || { echo
↪ \"osmo-bts-trx not bound on 5800-5802\"; exit 1; }\\nsleep 2; vty 4242 \"show bts 0\" | grep -E \"OML
↪ Link\"\\n'' returned non-zero exit status 1."
   ]
  }
 ],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "# Check: VTY, the clock/data ports, and OML seen by the BSC\n",
  "wait_tcp 4241 || exit 1\n",
  "ss -ulnp | grep -E ':580[0-2] ' || { echo \"osmo-bts-trx not bound on 5800-5802\"; exit 1; }\n",
  "sleep 2; vty 4242 \"show bts 0\" | grep -E \"OML Link\""
 ]
},
{
 "cell_type": "markdown",
 "id": "c30",
 "metadata": {},
 "source": [
  "---\n",
  "# Part 2: the emulated phone\n",
  "\n",
  "Five processes, in this order: the C54x DSP, QEMU (ARM + layer 1 firmware), osmocon (loads the firmware\n",
  "over the serial line), mobile (layers 2/3), and the bridge between the BTS and the DSP.\n",
  "\n",
  "## 8. c54x_exe, the DSP\n",
  "\n",
  "It runs TI's mask ROM, publishes the shared API RAM (`/dev/shm/calypso_api_ram`) and its socket\n",
  "(`/tmp/calypso_dsp.sock`), then waits for the ARM. It also listens on UDP 6702 for bursts (BSP)."
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "c31",
 "metadata": {},
 "outputs": [],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "rm -f /dev/shm/calypso_api_ram /tmp/calypso_dsp.sock /dev/shm/calypso_tch_cfg\n",
  "cd \"$ROOT/c54x_exe\"\n",
  "start dsp env CALYPSO_IQDUMP_FCCH=1 CALYPSO_BSP_ATTENTE_MS=40 \\\n",
  "  ./c54x_exe --arm --insns 120000 --iq none --amp 30000 -v"
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "c32",
 "metadata": {},
 "outputs": [],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "# Check (\"en attente de l'ARM\" = \"waiting for the ARM\")\n",
  "wait_log \"$L/dsp.log\" \"en attente de l'ARM\" 5 && test -S /tmp/calypso_dsp.sock && echo \"ok
↪ /tmp/calypso_dsp.sock\" && wait_udp 6702"
```

```
 ]
},
{
 "cell_type": "markdown",
 "id": "c33",
 "metadata": {},
 "source": [
  "## 9. QEMU (qosmo): the ARM and the layer 1 firmware\n",
  "\n",
  "`-M calypso` emulates the Compal E88 board. `CALYPSO_DSP_EXTERN=1` hands the DSP to `c54x_exe` instead of\n",
  "QEMU's built-in model. The first serial port becomes a pty: that is the modem line osmocon will talk to.\n",
  "The gdb stub listens on 1234."
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "c34",
 "metadata": {},
 "outputs": [],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "rm -f /tmp/qemu-monitor-pont.sock\n",
  "start qemu env CALYPSO_PONT_RETRY_DIV=64 CALYPSO_DSP_EXTERN=1 \\\n",
  "  \"$ROOT/qosmo/build/qemu-system-arm\" -M calypso -cpu arm946 -display none -parallel none \\\n",
  "    -serial pty -serial pty -monitor unix:/tmp/qemu-monitor-pont.sock,server,nowait \\\n",
  "    -gdb tcp:127.0.0.1:1234 \\\n",
  "    -kernel \"$ROOT/firmware/board/compal_e88/layer1.highram.elf\""
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "c35",
 "metadata": {},
 "outputs": [],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "# Check: the modem pty, the DSP bridge on the QEMU side (\"API RAM partagee\" = \"shared API RAM\"), the VM
↪  running\n",
  "wait_log \"$L/qemu.log\" \"label serial0\" 10 || exit 1\n",
  "wait_log \"$L/qemu.log\" \"pont DSP : API RAM partagee\" 10 || exit 1\n",
  "sed -n 's/.*redirected to \\(\\/dev\\/pts\\/[0-9]*\\) (label serial0).*/\\1/p' \"$L/qemu.log\" | head -1 >
↪  \"$L/modem.pty\"\n",
  "echo \"modem pty: $(cat \"$L/modem.pty\")\"\n",
  "printf 'info status\\n' | socat - UNIX-CONNECT:/tmp/qemu-monitor-pont.sock | grep \"VM status\""
 ]
},
{
 "cell_type": "markdown",
 "id": "c36",
 "metadata": {},
 "source": [
  "## 10. osmocon: load the firmware, relay L1CTL\n",
  "\n",
  "`-m romload` speaks the Calypso boot ROM protocol over the modem pty and uploads `layer1.highram.bin`.\n",
  "Then osmocon relays layer 1 control messages (L1CTL) on `/tmp/osmocom_l2`, where `mobile` will connect."
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "c37",
 "metadata": {},
```

```
 "outputs": [],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "rm -f /tmp/osmocom_l2\n",
  "start osmocon stdbuf -oL -eL \"$ROOT/osmocom-bb/src/host/osmocon/osmocon\" -m romload -i 100 \\\n",
  "  -p \"$(cat \"$L/modem.pty\")\" -s /tmp/osmocom_l2 \\\n",
  "  \"$ROOT/firmware/board/compal_e88/layer1.highram.bin\""
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "c38",
 "metadata": {},
 "outputs": [],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "# Check: the upload finished and the L1CTL socket exists; the DSP has booted on the firmware's reset\n",
  "wait_log \"$L/osmocon.log\" \"your code is running now\" 30 || exit 1\n",
  "test -S /tmp/osmocom_l2 && echo \"ok      /tmp/osmocom_l2\"\n",
  "grep -a \"RESET_DSP relache\" \"$L/qemu.log\" | tail -1\n",
  "grep -a \"DSP boote\" \"$L/dsp.log\" | tail -1"
 ]
},
{
 "cell_type": "markdown",
 "id": "c39",
 "metadata": {},
 "source": [
  "## 11. mobile: layers 2 and 3\n",
  "\n",
  "The osmocom-bb `mobile` application with `mobile_pont.cfg`: test SIM, cell lock on the bench ARFCN, VTY.\n",
  "It immediately asks layer 1 for a power scan and a sync on the cell (`L1CTL_FBSB_REQ`). As long as the\n",
  "bridge is not running there is no downlink, so `FBSB RESP: result=255` in a loop is **expected** here."
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "c40",
 "metadata": {},
 "outputs": [],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "MS_VTY=$(awk '/^ *bind 127\\.0\\.0\\.1 [0-9]+/ {print $3; exit}' \"$ROOT/c54x_exe/mobile_pont.cfg\")\n",
  "echo \"mobile VTY port from mobile_pont.cfg: $MS_VTY\"; echo \"$MS_VTY\" > \"$L/ms_vty\"\n",
  "ss -ltnH \"( sport = :$MS_VTY )\" | grep -q . && { echo \"port $MS_VTY already taken\"; exit 1; }\n",
  "rm -f /tmp/ms_data\n",
  "start mobile env L23_SYNC_RETRIES_SELECTION=8 stdbuf -oL mobile -c \"$ROOT/c54x_exe/mobile_pont.cfg\""
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "c41",
 "metadata": {},
 "outputs": [],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "# Check: still alive, VTY up, layer 1 asked to sync on the cell\n",
  "sleep 2; alive mobile && echo \"ok      mobile alive\" || { tail -15 \"$L/mobile.log\"; exit 1; }\n",
  "wait_tcp \"$(cat \"$L/ms_vty\")\" 5\n",
```

```
   "wait_log \"$L/osmocon.log\" \"L1CTL_FBSB_REQ\" 5"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "c42",
  "metadata": {},
  "source": [
   "## 12. pont_dsp.py: the bridge between the BTS and the DSP\n",
   "\n",
   "This is the radio link. On one side it speaks TRX to osmo-bts-trx (ports 5700–5702: clock, control,\n",
   "bursts); on the other it feeds the DSP's burst port (6702). Downlink bursts go to the DSP, uplink bursts\n",
   "(RACH, SDCCH, TCH) come back to the BTS. `ARFCN`/`BSIC` must match the BSC config (514 / 7 on the bench).\n",
   "`--no-record` skips the I/Q recording to `/root/record.cfile`; remove it if you want the capture."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 20,
  "id": "c43",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "started pont (pid 679671)\n"
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /tmp/c54x-pont/lib.sh\n",
   "cd \"$O\"\n",
   "start pont env PONT_BSC_CFG=/etc/osmocom/osmo-bsc.cfg PONT_ARFCN=514 PONT_BSIC=7 \\\n",
   "  python3 pont/pont_dsp.py --dsp-port 6702 --no-record"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 21,
  "id": "c44",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "ok      17:41:27 [pont] pont TRX : ports 5700/5701/5702, ARFCN 514, BSIC 7, avance UL 3 trames\n",
     "ok      17:41:28 [pont] BTS 127.0.0.1 : horloge armee vers le port 5800\n",
     "17:41:28 [pont] CTRL POWEROFF\n",
     "17:41:28 [pont] CTRL RFMUTE 1\n",
     "17:41:28 [pont] CTRL RFMUTE 0\n",
     "17:41:28 [pont] CTRL SETFORMAT 2\n",
     "17:41:28 [pont] CTRL RXTUNE 890200\n"
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /tmp/c54x-pont/lib.sh\n",
   "# Check: ports, the BTS clock armed (\"horloge armee\" = \"clock armed\"), the BTS's TRXC commands, bursts
↪  flowing\n",
   "wait_log \"$L/pont.log\" \"pont TRX : ports\" 10 || exit 1\n",
   "wait_log \"$L/pont.log\" \"horloge armee\" 10\n",
   "grep -a \"CTRL \" \"$L/pont.log\" | head -5\n",
```

```
   "sleep 3; grep -a \"STATS fn=\" \"$L/pont.log\" | tail -2"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "k01",
  "metadata": {},
  "source": [
   "### 12a. What you just built: the classic BTS (`--bts-classique`)\n",
   "\n",
   "[2026-10-21] Steps 7 to 12 are the **classic BTS** set-up: osmo-bts-trx is the base station and\n",
   "`pont_dsp.py` is its radio, feeding the emulated phone's DSP. In one command it is\n",
   "`./start.sh --bts-classique --ms` (`--bts-classic` is the same option; it is also what plain `--dsp`\n",
   "starts). Two things the bridge does in this set-up since 2026-10-23:\n",
   "\n",
   "- it copies every downlink burst (and, marked, every uplink burst) to the UDP ports of\n",
   "  `PONT_DSP_COPIES` (6762 by default, set by `pont_dsp.py`: the BSP of instance 4, the rssi application of\n",
   "  the last section). A port nobody listens on changes nothing for the mobile; `PONT_DSP_COPIES=` turns the\n",
   "  copy off, `PONT_DSP_COPIES_UL=0` keeps the downlink only;\n",
   "- it caps the clock pause it asks of the BTS (`PONT_CLCK_PAUSE_PLAFOND`, 38 frames), so a slow DSP no\n",
   "  longer pushes osmo-bts-trx past its 50-frame limit (\"PC clock skew too high\").\n",
   "\n",
   "The same bridge can carry the **C139 Magnetite** instead of the osmocom-bb mobile (steps 10-11 are then\n",
   "replaced by `rvinterf`, the UI is in the firmware): `ENCRYPTION=\"a5 0\" ./start.sh --bts-classique\n",
   "--ms-magnetite --gapk` (with A5/1 the FACCH fails and calls do not set up; `--gapk` puts the voice codec
↪  in\n",
   "the bridge, `pont/voix_hote.py`). See [doc/magnetite.md](../doc/magnetite.md).\n"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 22,
  "id": "k02",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "defaults: PONT_DSP_COPIES=6762 (set inside pont_dsp.py), PONT_CLCK_PAUSE_PLAFOND=38,
↪  PONT_CLCK_SKEW_ZONE=60\n"
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /tmp/c54x-pont/lib.sh\n",
   "# Check: the bridge's copy port and clock cap as it sees them, and whether it already had to cap a pause\n",
   "tr '\\0' '\\n' < /proc/\"$(cat \"$L/pont.pid\")\"/environ | grep -E
↪  '^PONT_(DSP_COPIES|CLCK_PAUSE_PLAFOND|CLCK_SKEW_ZONE)=' \\\n",
   "  || echo \"defaults: PONT_DSP_COPIES=6762 (set inside pont_dsp.py), PONT_CLCK_PAUSE_PLAFOND=38,
↪  PONT_CLCK_SKEW_ZONE=60\"\n",
   "grep -a 'pause bornee' \"$L/pont.log\" | tail -1 || true\n"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "x01",
  "metadata": {},
  "source": [
   "---\n",
   "## 12b. Variant: the phone as the BTS transceiver (`--bts`, alias `--osmo-trx`), and a second phone over the
↪  air\n",
   "\n",
   "[2026-10-20] Instead of mobile + bridge (steps 11-12), the emulated Calypso can **be the transceiver of\n",
   "BTS#0**: QEMU boots `trx.highram.elf` (osmocom-bb `jolly/testing`, `trx` application) and the host\n",
```

```
   "`transceiver` program replaces both `mobile` and `pont_dsp.py`: it talks L1CTL to the firmware and TRXD\n",
   "(5700-5702) to osmo-bts-trx, `--no-sync` (no cell to listen to, BTS mode right after reset), RX mask\n",
  "`0x01`, TX mask `0x01` (one TX window at TPU slot 5 = downlink TS0). The DSP extension code of the firmware\n",
   "turns the ROM's RACH transmit task into FB / SB / dummy / normal bursts, which `c54x_exe` publishes in\n",
   "`/dev/shm/calypso_tx_rom`.\n",
   "\n",
   "Everything is driven by `c54x_exe/run.sh` with `OSMO_TRX=1` (alias `banc-trx`; in one command, network\n",
   "included: `./start.sh --bts --ms`). Stop steps 8-12 first:\n",
  "the DSP, QEMU and osmocon are restarted with the other firmware. Note that osmo-bts-trx **exits** when its\n",
   "transceiver disappears (\"No more clock from transceiver\"); `run.sh` relaunches it (step 4b) when BTS#0
↪  is\n",
   "absent.\n",
   "\n",
   "Then a **second emulated phone** (`INSTANCE=2`, alias `banc-mobile`: its own `/dev/shm` through a private\n",
  "mount namespace, `/tmp/c54x-pont2`, L1CTL `/tmp/osmocom_l2_2`, BSP UDP 6712, mobile VTY 4347) camps on it:\n",
   "`pont/air.py` relays the TRX's published bursts to the mobile's BSP and the mobile's bursts back.\n",
   "\n",
   "State on 2026-10-20 evening: the second phone camps, sends its RACH, gets its IMMEDIATE ASSIGNMENT on the\n",
   "SDCCH/4 of TS0 and completes the **location update** (`LOCATION UPDATING ACCEPT`, TMSI allocated). Two\n",
   "prostheses in `c54x_exe` make up for what the firmware's DSP extension code gets wrong on the emulated
↪  core:\n",
   "the SCH is re-encoded before publishing (`MONTANT_ROM_TX_SB_RECODE`) and the exact bits of each uplink\n",
  "normal burst are written into the extension's result page (`CALYPSO_BSP_TRX_UL`). The uplink is aligned on\n",
   "the firmware's clock from the SB it emits (`CALYPSO_BSP_TRX`, `BTS_RX_LAG 3` / delta -2). SMS and calls\n",
   "over this path are not exercised yet. See `wiki/Reference.md` 2.5 and 2.7."
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 23,
  "id": "x02",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "stopped pont\n",
     "\u001b[1m[run dsp]\u001b[0m 1. c54x_exe --arm  pid 680062  (120000 insn/trame, iq=none,
↪  /dev/shm/calypso_api_ram, /tmp/calypso_dsp.sock)\n",
     "\u001b[1m[run dsp]\u001b[0m 2. qemu-system-arm  pid 680071  pty modem /dev/pts/3  moniteur
↪  /tmp/qemu-monitor-pont.sock  L1=DSP externe\n",
     "\u001b[1m[run dsp]\u001b[0m    gdb ARM : telnet 0 44444  (stub tcp::1234, journal
↪  /tmp/c54x-pont/gdb.log)\n",
     "\u001b[1m[run dsp]\u001b[0m 3. osmocon  pid 680093  L1CTL sur /tmp/osmocom_l2\n",
     "\u001b[1m[run dsp]\u001b[0m    BTS#0 presente mais muette : on attend qu'elle allume le firmware ou qu'elle
↪  finisse de s'eteindre (90 s max)\n",
     "\u001b[1m[run dsp]\u001b[0m 4b. osmo-bts-trx relancee (BTS#0 absente : elle sort quand le transceiver
↪  disparait)  pid 680209  /etc/osmocom/osmo-bts-trx.cfg  journal /tmp/c54x-pont/bts.log\n",
     "\u001b[1m[run dsp]\u001b[0m 4. transceiver  pid 680146  L1CTL /tmp/osmocom_l2  TRXD 5700-5702 <-
↪  osmo-bts-trx  timeslots RX 0x03 TX 0x03  bsic 7  (--no-sync -e 3 -r 20)\n",
     "\u001b[1m[run dsp]\u001b[0m 5. (OSMO_TRX=1 : pas de pont, transceiver tient 5700-5702)\n",
     "\u001b[1m[run dsp]\u001b[0m tout tourne. Journaux : /tmp/c54x-pont/*.log   suivre : ./run.sh --logs
↪  arreter : ./run.sh --stop\n"
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /tmp/c54x-pont/lib.sh\n",
   "for n in pont mobile osmocon gdb qemu dsp; do p=$(cat \"$L/$n.pid\" 2>/dev/null) && kill \"$p\" 2>/dev/null
↪  && echo \"stopped $n\"; done\n",
   "sleep 1; rm -f /dev/shm/calypso_api_ram /tmp/calypso_dsp.sock /tmp/osmocom_l2 /tmp/qemu-monitor-pont.sock
↪  \"$L\"/{pont,mobile,osmocon,gdb,qemu,dsp}.pid\n",
   "# the TRX chain: DSP, QEMU (trx.highram.elf), osmocon, transceiver ; osmo-bts-trx relaunched if it exited\n",
```

```
   "MODE=dsp OSMO_TRX=1 INSNS=120000 LOCKSTEP=1 IQ=none CALYPSO_BSP_STREAM=1 CALYPSO_RHEA_DMA_XFER=1 bash
↪  \"$ROOT/c54x_exe/run.sh\""
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 24,
  "id": "x03",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "BTS MODE: 7 1 {5,7,1,1,1,1,13,13} TX 0..1 RX 0..1\n",
     "STALE BURST (must stop growing): 0\n"
    ]
   }
  ],
  "source": [
   "%%bash\n",
   ". /tmp/c54x-pont/lib.sh\n",
   "# Check: the firmware in BTS mode, FB and SB one burst in ten each, the BTS clock, no stale bursts\n",
   "sleep 30\n",
   "grep -a 'BTS MODE' \"$L/osmocon.log\" | tail -1                      # BTS MODE: 7 514 {5,...} TX 0..0 RX
↪  0..0\n",
   "grep -a 'rom-tx\\] [0-9]' \"$L/dsp.log\" | tail -1                    # SB and FB ~ 1/10 of the bursts, NB
↪  from the BTS\n",
   "grep -a 'SB recode' \"$L/dsp.log\" | tail -1\n",
   "echo \"STALE BURST (must stop growing): $(grep -ac STALE \"$L/osmocon.log\")\"\n",
   "sed 's/\\x1b\\[[0-9;]*m//g' /var/log/osmocom/osmo-bts-trx.log /run/user/*/osmo-nitb/logs/bts.log 2>/dev/null
↪  | grep -E 'clock (started|skew)' | tail -2"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 25,
  "id": "x04",
  "metadata": {},
  "outputs": [
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "\u001b[1m[run dsp]\u001b[0m 1. c54x_exe --arm  pid 681551  (120000 insn/trame, iq=none,
↪  /dev/shm/calypso_api_ram, /tmp/calypso_dsp2.sock)\n",
     "\u001b[1m[run dsp]\u001b[0m 2. qemu-system-arm  pid 681567  pty modem /dev/pts/5  moniteur
↪  /tmp/qemu-monitor-pont2.sock  L1=DSP externe\n",
     "\u001b[1m[run dsp]\u001b[0m    gdb ARM : telnet 0 44445  (stub tcp::1235, journal
↪  /tmp/c54x-pont2/gdb.log)\n",
     "\u001b[1m[run dsp]\u001b[0m 3. osmocon  pid 681589  L1CTL sur /tmp/osmocom_l2_2\n",
     "\u001b[1m[run dsp]\u001b[0m 4. instance 2 : config mobile derivee ->
↪  /tmp/c54x-pont2/mobile_instance2.cfg\n",
     "\u001b[1m[run dsp]\u001b[0m 4. mobile  pid 681622  config /tmp/c54x-pont2/mobile_instance2.cfg  vty telnet
↪  127.0.0.1 4347\n",
     "\u001b[1m[run dsp]\u001b[0m 5. air  pid 681785  /run/calypso-shm-hote/calypso_tx_rom -> udp 6712 (ici) ;
↪  /dev/shm/calypso_tx_rom (ici) -> udp 6702 (l'autre)\n",
     "\u001b[1m[run dsp]\u001b[0m tout tourne. Journaux : /tmp/c54x-pont2/*.log   suivre : ./run.sh --logs
↪  arreter : ./run.sh --stop\n",
     "air : A->B 17332 (NB=2839 SB=849 DUMMY=12795 FB=849 TS1=8666)  B->A 372 (RACH=2 NB=370)  rates A 0 B 0
↪  SDCCH blocs 0/0\n",
     "\tCharging at 239 LSB (204 mA).\n",
     "\tBattery full at 468 LSB .. full at 585 LSB\n",
     "\tCharging at 239 LSB (204 mA).\n",
     "Tue Oct 27 17:42:18 2026 DRR <0001> gsm48_rr.c:2940 New SYSTEM INFORMATION 5\n",
```

```
    "Tue Oct 27 17:42:18 2026 DRR <0001> gsm48_rr.c:3033 New SYSTEM INFORMATION 6 (lai=001-01-1 SACCH-timeout
↪  32)\n",
    "Tue Oct 27 17:42:42 2026 DRR <0001> gsm48_rr.c:2940 New SYSTEM INFORMATION 5\n",
    "stop: INSTANCE=2 MODE=dsp bash /opt/GSM/c54x_exe/run.sh --stop ; MODE=dsp bash /opt/GSM/c54x_exe/run.sh
↪  --stop\n"
   ]
  }
 ],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "# The second phone (instance 2) and the air between the two Calypsos\n",
  "INSTANCE=2 MODE=dsp AIR=1 INSNS=120000 LOCKSTEP=1 IQ=none CALYPSO_BSP_STREAM=1 CALYPSO_RHEA_DMA_XFER=1 bash
↪  \"$ROOT/c54x_exe/run.sh\"\n",
  "sleep 40\n",
  "tail -1 /tmp/c54x-pont2/air.log                                    # A->B n (NB SB DUMMY FB) : the TRX's
↪  bursts reach the mobile\n",
  "grep -a 'FB1\\|SB' /tmp/c54x-pont2/osmocon.log | grep -av 'FBSB_REQ\\|DSP Error' | tail -3   # FB detected,
↪  then \"=> SB ... BSIC=7\"\n",
  "sed 's/\\x1b\\[[0-9;]*m//g' /tmp/c54x-pont2/mobile.log | grep -iE 'FBSB RESP|SYSTEM INFO|camped' | tail -3\n",
  "echo \"stop: INSTANCE=2 MODE=dsp bash $ROOT/c54x_exe/run.sh --stop ; MODE=dsp bash $ROOT/c54x_exe/run.sh
↪  --stop\""
 ]
},
{
 "cell_type": "markdown",
 "id": "c45",
 "metadata": {},
 "source": [
  "---\n",
  "# Part 3: the service\n",
  "\n",
  "## 13. System information, then camp\n",
  "\n",
  "With the downlink flowing, the phone syncs (FCCH, SCH), reads the system information on the BCCH and\n",
  "camps on the cell."
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "c46",
 "metadata": {},
 "outputs": [
  {
   "name": "stderr",
   "output_type": "stream",
   "text": [
    "cat: /tmp/c54x-pont/ms_vty: No such file or directory\n"
   ]
  }
 ],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "MS_VTY=$(cat \"$L/ms_vty\")\n",
  "wait_log \"$L/mobile.log\" \"New SYSTEM INFORMATION\" 30 || exit 1\n",
  "for i in $(seq 1 60); do vty \"$MS_VTY\" \"show ms\" | grep -q \"C3 camped normally\" && { echo \"ok
↪  camped after $((i*2)) s\"; break; }; sleep 2; done\n",
  "vty \"$MS_VTY\" \"show ms\" | grep -E \"camped|MM|IMSI|cell\""
 ]
},
{
 "cell_type": "markdown",
 "id": "c47",
 "metadata": {},
```

```
 "source": [
  "## 14. Location update\n",
  "\n",
  "The phone sends a RACH, gets a dedicated channel (SDCCH), authenticates against the HLR and registers."
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "c48",
 "metadata": {},
 "outputs": [],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "grep -a \"\\[montant\\] RACH\" \"$L/dsp.log\" | tail -1\n",
  "grep -ao 'UL bursts=[0-9]* tard=[0-9]* rach=[0-9]*' \"$L/pont.log\" | tail -1\n",
  "wait_log \"$L/mobile.log\" \"LOCATION UPDATING ACCEPT\" 60 || exit 1\n",
  "sed 's/\\x1b\\[[0-9;]*m//g' \"$L/mobile.log\" | grep -aoE 'lai=[0-9]+-[0-9]+-[0-9]+' | tail -1\n",
  "vty 4254 \"show subscriber cache\" | grep -E 'IMSI|MSISDN'"
 ]
},
{
 "cell_type": "markdown",
 "id": "c49",
 "metadata": {},
 "source": [
  "## 15. A call to 600 (echo test)\n",
  "\n",
  "Needs step 6. The call goes mobile → BTS → BSC → MSC → MNCC → osmo-sip-connector → Asterisk, and back."
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "c50",
 "metadata": {},
 "outputs": [],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "MS_VTY=$(cat \"$L/ms_vty\")\n",
  "vty \"$MS_VTY\" \"call 1 600\" > /dev/null\n",
  "for i in $(seq 1 15); do\n",
  "  st=\"$(vty \"$MS_VTY\" 'show ms' | sed -n 's/^ *call control state: //p' | head -1)\"\n",
  "  echo \"${st:-(no call yet)}\"; [ \"$st\" = ACTIVE ] && break; sleep 2\n",
  "done\n",
  "grep -a \"\\[montant\\] TCH\" \"$L/dsp.log\" | tail -2"
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "c51",
 "metadata": {},
 "outputs": [],
 "source": [
  "%%bash\n",
  ". /tmp/c54x-pont/lib.sh\n",
  "# Hang up\n",
  "vty \"$(cat \"$L/ms_vty\")\" \"call 1 hangup\" > /dev/null; sleep 2\n",
  "vty \"$(cat \"$L/ms_vty\")\" \"show ms\" | grep -E \"call control state|MM\" || echo \"no call\""
 ]
},
{
 "cell_type": "markdown",
```

```
  "id": "c52",
  "metadata": {},
  "source": [
   "---\n",
   "## Stop and clean up\n",
   "\n",
   "Reverse order, then the shared files: a leftover `/tmp/ms_data` makes the next mobile's bind fail\n",
   "silently, and a leftover `/dev/shm/calypso_horloge` slaves the bridge to a dead session. If you started\n",
   "the rssi application (last section), stop it first with `banc-rssi --stop`.\n",
   "\n",
   "If the network dropped during the run, look at the BTS first: `osmo-bts-trx` exits on \"PC clock skew too\n",
   "high\". Started by hand (step 7) it stays down; `./start.sh` restarts it by itself since 2026-10-23\n",
   "(`BTS_GARDIEN=1` in `qosmo/run_modules/60-bts.sh`, `BTS_GARDIEN_PAUSE` seconds apart, a `[gardien bts]`\n",
   "line in its log), and the bridge caps the pause it asks for (`PONT_CLCK_PAUSE_PLAFOND=38`,\n",
   "`PONT_CLCK_SKEW_ZONE=60` in `pont/trx.py`).\n"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": null,
  "id": "c53",
  "metadata": {},
  "outputs": [],
  "source": [
   "%%bash\n",
   ". /tmp/c54x-pont/lib.sh\n",
   "for n in pont mobile osmocon gdb qemu dsp bts sip bsc msc mgw hlr stp; do\n",
   "  p=$(cat \"$L/$n.pid\" 2>/dev/null) && kill \"$p\" 2>/dev/null && echo \"stopped $n\"\n",
   "done\n",
   "asterisk -rx \"core stop now\" 2>/dev/null || true\n",
   "sleep 1\n",
   "rm -f /dev/shm/calypso_api_ram /tmp/calypso_dsp.sock /tmp/osmocom_l2 /tmp/qemu-monitor-pont.sock \\\n",
   "      \"$L/modem.pty\" /tmp/osmocom_sap /tmp/ms_data /dev/shm/calypso_horloge \\\n",
   "      /dev/shm/calypso_rach /dev/shm/calypso_sdcch_ul /dev/shm/calypso_tch_facch_ul \\\n",
   "      /dev/shm/calypso_tch_sacch_ul /dev/shm/calypso_tch_ul /dev/shm/calypso_tch_cfg \"$L\"/*.pid\n",
   "pgrep -a -f 'osmo-(stp|hlr|mgw|msc|bsc|bts-trx|sip-connector)|c54x_exe|qemu-system-arm|osmocon|mobile
↪  -c|pont_dsp' \\\n",
   "  && echo \"still running (above)\" || echo \"all stopped\""
  ]
 },
 {
  "cell_type": "markdown",
  "id": "r01",
  "metadata": {},
  "source": [
   "---\n",
   "## Optional: the rssi application on a fourth Calypso (`banc-rssi`)\n",
   "\n",
   "[2026-10-21, 2026-10-23] `banc-rssi` starts **instance 4** of `c54x_exe/run.sh` with `FIRMWARE_APP=rssi`:\n",
   "a C54x DSP, QEMU running osmocom-bb's `rssi.highram.elf` and osmocon, no mobile and no bridge of its own.\n",
   "Its C123 screen and keypad are emulated (window: `c123-3`). It is kept apart from the rest of the bench:\n",
   "`./start.sh --stop` and `banc-stop` leave it running; `banc-rssi --stop` (or `banc-rssi-stop`) stops the\n",
   "whole instance.\n",
   "\n",
   "Where its downlink comes from:\n",
   "\n",
   "- **classic BTS** (`pont_dsp.py` running, the case of this notebook): no air, `AIR=0`; the bridge's copy\n",
   "  to UDP 6762 (`PONT_DSP_COPIES`, step 12a) feeds its BSP;\n",
   "- **`--bts`**: `pont/air.py` relays the TRX's bursts, as for the mobile of step 12b.\n",
   "\n",
   "Realistic air levels are on by default (`CALYPSO_AIR_NIVEAUX=1`, `qosmo` `calypso_bsp.c`): the cell is
↪  heard\n",
   "only on its ARFCN (`CALYPSO_FAUX_AIR`, taken from `PLAN_ARFCN` in `/etc/osmocom/radio-plan.env`), the\n",
   "downlink at `CALYPSO_AIR_DL_DBM`, the mobile's uplink at `CALYPSO_AIR_UL_DBM`, noise elsewhere.\n",
   "`CALYPSO_AIR_NIVEAUX=0 banc-rssi` gives the old behaviour (everything at full level, on every frequency).\n",
   "\n",
```

```
    "It needs the network and the bridge of Part 1 / step 12 still running: run it **before** the Stop cell.\n",
    "The helpers are shell functions from `scripts/osmo-banc-aliases.sh` (installed as\n",
    "`/etc/profile.d/98-osmo-banc-aliases.sh`); the cells source it.\n"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "r02",
   "metadata": {},
   "outputs": [],
   "source": [
    "%%bash\n",
    ". /tmp/c54x-pont/lib.sh\n",
    ". \"$O/scripts/osmo-banc-aliases.sh\"\n",
    "banc-rssi\n"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "r03",
   "metadata": {},
   "outputs": [],
   "source": [
    "%%bash\n",
    ". /tmp/c54x-pont/lib.sh\n",
    ". \"$O/scripts/osmo-banc-aliases.sh\"\n",
    "# Check: instance 4 up (logs in /tmp/c54x-pont4), the downlink reaching its BSP; window: c123-3\n",
    "banc-rssi --status\n",
    "ss -ulnH '( sport = :6762 )' | grep -q . && echo \"ok      udp 6762 bound (BSP of instance 4, where the bridge
↪  copies its bursts)\"\n",
    "tail -3 /tmp/c54x-pont4/dsp.log\n",
    "echo \"window: /usr/bin/python3 $O/tools/osmo-c123-gtk.py --instance 4   (alias c123-3)\"\n",
    "echo \"stop:   banc-rssi --stop\"\n"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "74084ff2-b8fb-485c-a513-4874995645a3",
   "metadata": {},
   "outputs": [],
   "source": []
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "660f5909-c900-438d-8ff0-dde1b063ec22",
   "metadata": {},
   "outputs": [],
   "source": []
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "a5bbd17c-47b4-42e1-b980-ee59fd838268",
   "metadata": {},
   "outputs": [],
   "source": []
  }
 ],
 "metadata": {
  "kernelspec": {
   "display_name": "Python 3 (ipykernel)",
   "language": "python",
```

```
   "name": "python3"
  },
  "language_info": {
   "codemirror_mode": {
    "name": "ipython",
    "version": 3
   },
   "file_extension": ".py",
   "mimetype": "text/x-python",
   "name": "python",
   "nbconvert_exporter": "python",
   "pygments_lexer": "ipython3",
   "version": "3.12.3"
  }
 },
 "nbformat": 4,
 "nbformat_minor": 5
}
```

3.145 /opt/GSM/osmo-operator/wiki/.ipynb_checkpoints/lancer-les-modes-checkpoint.ipynb

identique à /opt/GSM/osmo-operator/wiki/lancer-les-modes.ipynb

3.146 /root/osmo-operator-20261028-171615-docs_files/mediabag/badge.svg

2285 octets, 35 lignes → 35 lignes

```
<svg xmlns="http://www.w3.org/2000/svg" width="130" height="20">
  <title>Build ISO - passing</title>
  <defs>
    <linearGradient id="workflow-fill" x1="50%" y1="0%" x2="50%" y2="100%">
      <stop stop-color="#444D56" offset="0%"></stop>
      <stop stop-color="#24292E" offset="100%"></stop>
    </linearGradient>
    <linearGradient id="state-fill" x1="50%" y1="0%" x2="50%" y2="100%">
      <stop stop-color="#34D058" offset="0%"></stop>
      <stop stop-color="#28A745" offset="100%"></stop>
    </linearGradient>
  </defs>
  <g fill="none" fill-rule="evenodd">
    <g font-family="&#39;DejaVu Sans&#39;,Verdana,Geneva,sans-serif" font-size="11">
      <path id="workflow-bg" d="M0,3 C0,1.3431 1.3552,0 3.02702703,0 L80,0 L80,20 L3.02702703,20 C1.3552,20
↪  0,18.6569 0,17 L0,3 Z" fill="url(#workflow-fill)" fill-rule="nonzero"></path>
      <text fill="#010101" fill-opacity=".3">
        <tspan x="22.1981982" y="15" aria-hidden="true">Build ISO</tspan>
      </text>
      <text fill="#FFFFFF">
        <tspan x="22.1981982" y="14">Build ISO</tspan>
      </text>
    </g>
    <g transform="translate(80)" font-family="&#39;DejaVu Sans&#39;,Verdana,Geneva,sans-serif" font-size="11">
      <path d="M0 0h46.939C48.629 0 50 1.343 50 3v14c0 1.657-1.37 3-3.061 3H0V0z" id="state-bg"
↪  fill="url(#state-fill)" fill-rule="nonzero"></path>
      <text fill="#010101" fill-opacity=".3" aria-hidden="true">
        <tspan x="4" y="15">passing</tspan>
      </text>
      <text fill="#FFFFFF">
        <tspan x="4" y="14">passing</tspan>
      </text>
    </g>
```

```
    <path fill="#959DA5" d="M11 3c-3.868 0-7 3.132-7 7a6.996 6.996 0 0 0 4.786 6.641c.35.062.482-.148.482-.332
↪  0-.166-.01-.718-.01-1.304-1.758.324-2.213-.429-2.353-.822-.079-.202-.42-.823-.717-.99-.245-.13-.595-.454-.01-
↪  .463.552-.009.946.508 1.077.718.63 1.058 1.636.76
↪  2.039.577.061-.455.245-.761.446-.936-1.557-.175-3.185-.779-3.185-3.456
↪  0-.762.271-1.392.718-1.882-.07-.175-.315-.892.07-1.855 0 0 .586-.183 1.925.718a6.5 6.5 0 0 1 1.75-.236 6.5
↪  6.5 0 0 1 1.75.236c1.338-.91 1.925-.718 1.925-.718.385.963.14 1.68.07 1.855.446.49.717 1.112.717 1.882 0
↪  2.686-1.636 3.28-3.194 3.456.254.219.473.639.473 1.295 0 .936-.009 1.689-.009 1.925 0
↪  .184.131.402.481.332A7.011 7.011 0 0 0 18 10c0-3.867-3.133-7-7-7z"></path>
  </g>
</svg>
```

3.147 /root/osmo-operator-20261028-171615-docs_files/mediabag/badge.svg-branch=main-event=workflow_dispatch

identique à /root/osmo-operator-20261028-171615-docs_files/mediabag/badge.svg